

O'REILLY®

3rd Edition

High Performance Python

Practical Performant
Programming for Humans



Micha Gorelick & Ian Ozsvald
Foreword by Hilary Mason

"Ian and Micha's new book combines an overview of modern performance tooling with deep insights into general principles of code optimization, making it an essential read for every Python developer."

Mikhail Timonin, quant developer, Engelhart

High Performance Python

Your Python code may run correctly, but what if you need it to run faster? This practical book shows you how to locate performance bottlenecks and significantly speed up your code in high-data-volume programs. By explaining the fundamental theory behind design choices, this expanded edition of *High Performance Python* helps experienced Python programmers gain a deeper understanding of Python's implementation.

How do you take advantage of multicore architectures or compilation? Or build a system that scales up beyond RAM limits or with a GPU? Authors Micha Gorelick and Ian Ozsvald reveal concrete solutions to many issues and include war stories from companies that use high-performance Python for GenAI data extraction, productionized machine learning, and more.

- Get a better grasp of NumPy, Cython, and profilers
- Learn how Python abstracts the underlying computer architecture
- Use profiling to find bottlenecks in CPU time and memory usage
- Write efficient programs by choosing appropriate data structures
- Speed up matrix and vector computations
- Process DataFrames quickly with Pandas, Dask, and Polars
- Speed up your neural networks and GPU computations
- Use tools to compile Python down to machine code
- Manage multiple I/O and computational operations concurrently
- Convert multiprocessing code to run on local or remote clusters

Micha Gorelick is a machine learning researcher and engineer, an educator, and an advocate for socially conscious computing.

Ian Ozsvald is a London-based independent chief data scientist who coaches teams, teaches, and creates data products.

PYTHON / PROGRAMMING

US \$65.99 CAN \$82.99

ISBN: 978-1-098-16596-3



5 6 5 9 9

O'REILLY®

THIRD EDITION

High Performance Python

Practical Performant Programming for Humans

Micha Gorelick and Ian Ozsvald

Foreword by Hilary Mason

O'REILLY®

High Performance Python

by Micha Gorelick and Ian Ozsvald

Copyright © 2025 Micha Gorelick and Ian Ozsvald. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Michelle Smith

Development Editor: Sara Hunter

Production Editor: Clare Laylock

Copyeditor: Dwight Ramsey

Proofreader: Arthur Johnson

Indexer: Potomac Indexing, LLC

Interior Designer: David Futato

Cover Designer: Jose Marzan

Illustrator: Kate Dullea

September 2014: First Edition
May 2020: Second Edition
May 2025: Third Edition

Revision History for the Third Edition

2025-04-29: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098165963> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *High Performance Python*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

High Performance Python is available under the Creative Commons Attribution-NonCommercial-NoDerivs 3.0 License.

978-1-098-16596-3

[LSI]

Table of Contents

Foreword.....	ix
Preface.....	xi
1. Understanding Performant Python.....	1
The Fundamental Computer System	2
Computing Units	2
Memory Units	6
Communications Layers	8
Idealized Computing Versus the Python Virtual Machine	10
Idealized Computing	11
Python's Virtual Machine	12
So Why Use Python?	15
How to Be a Highly Performant Programmer	17
Good Working Practices	18
Optimizing for the Team Rather than the Code Block	21
The Remote Performant Programmer	23
Some Thoughts on Good Notebook Practice	23
Your Work	24
The Future of Python	25
Where Did the GIL Go?	25
Does Python Have a JIT?	25
Wrap-Up	26
2. Profiling to Find Bottlenecks.....	27
Profiling Efficiently	28
Introducing the Julia Set	30
Calculating the Full Julia Set	33

Simple Approaches to Timing—print and a Decorator	36
Simple Timing Using the Unix time Command	39
Using the cProfile Module	41
Visualizing cProfile Output with SnakeViz	46
Using line_profiler for Line-by-Line Measurements	48
Using memory_profiler to Diagnose Memory Usage	53
Combining CPU and Memory Profiling with Scalene	61
Introspecting an Existing Process with PySpy	63
VizTracer for an Interactive Time-Based Call Stack	65
Bytecode: Under the Hood	67
Using the dis Module to Examine CPython Bytecode	67
Digging into Bytecode Specialization with Specialist	69
Different Approaches, Different Complexity	71
Unit Testing During Optimization to Maintain Correctness	73
No-op @profile Decorator	74
Strategies to Profile Your Code Successfully	77
Wrap-Up	78
3. Lists and Tuples.	79
A More Efficient Search	82
Lists Versus Tuples	85
Lists as Dynamic Arrays	86
Tuples as Static Arrays	90
Wrap-Up	93
4. Dictionaries and Sets.	95
How Do Dictionaries and Sets Work?	99
Inserting and Retrieving	99
Deletion	104
Resizing	104
Hash Functions and Entropy	106
Wrap-Up	111
5. Iterators and Generators.	113
Iterators for Infinite Series	118
Lazy Generator Evaluation	120
Wrap-Up	124
6. Matrix and Vector Computation.	125
Introduction to the Problem	126
Aren't Python Lists Good Enough?	131
Problems with Allocating Too Much	133

Memory Fragmentation	136
Understanding perf	139
Making Decisions with perf's Output	142
Enter numpy	143
Applying numpy to the Diffusion Problem	146
Memory Allocations and In-Place Operations	149
Selective Optimizations: Finding What Needs to Be Fixed	153
numexpr: Making In-Place Operations Faster and Easier	156
Graphics Processing Units (GPUs)	158
Dynamic Graphs: PyTorch	159
GPU Speed and Numerical Precision	162
GPU-Specific Operations	165
Basic GPU Profiling	168
Performance Considerations of GPUs	170
When to Use GPUs	172
Deep Learning Performance Considerations	174
A Cautionary Tale: Verify "Optimizations" (scipy)	179
Lessons from Matrix Optimizations	180
Wrap-Up	183
7. Pandas, Dask, and Polars.....	185
Pandas	186
Pandas's Internal Model	187
Arrow and NumPy	189
Applying a Function to Many Rows of Data	189
Numba to Compile NumPy for Pandas	199
Building from Partial Results Rather than Concatenating	200
There's More Than One (and Possibly a Faster) Way to Do a Job	202
Advice for Effective Pandas Development	203
Dask for Distributed Data Structures and DataFrames	205
Diagnostics	206
Parallel Pandas with Dask	207
Parallelized apply with Swifter on Dask	209
Polars for Fast DataFrames	210
Wrap-Up	211
8. Compiling to C.....	213
What Sort of Speed Gains Are Possible?	214
JIT Versus AOT Compilers	216
Why Does Type Information Help the Code Run Faster?	216
Using a C Compiler	217
Reviewing the Julia Set Example	218

Cython	219
Compiling a Pure Python Version Using Cython	219
pyximport	221
Cython Annotations to Analyze a Block of Code	222
Adding Some Type Annotations	224
Cython and numpy	229
Parallelizing the Solution with OpenMP on One Machine	232
Numba	234
PyPy	236
Garbage Collection Differences	238
Running PyPy and Installing Modules	238
A Summary of Speed Improvements	240
When to Use Each Technology	241
Foreign Function Interfaces	242
ctypes	243
cffi	246
f2py	249
CPython Extensions: C	252
CPython Extensions: Rust	256
Wrap-Up	260
9. Asynchronous I/O.....	261
Introduction to Asynchronous Programming	263
How Does async/await Work?	267
Serial Web Crawler	268
Asynchronous Web Crawler	270
Shared CPU–I/O Workload	275
Serial CPU Workload	276
Batched CPU Workload	278
Fully Asynchronous CPU Workload	281
Wrap-Up	286
10. The multiprocessing Module.....	289
An Overview of the multiprocessing Module	292
Estimating Pi Using the Monte Carlo Method	294
Estimating Pi Using Processes and Threads	296
Using Python Objects	296
Replacing multiprocessing with Joblib	304
Random Numbers in Parallel Systems	308
Using numpy	309
Finding Prime Numbers	312
Queues of Work	319

Asynchronously Adding Jobs to the Queue	323
Verifying Primes Using Interprocess Communication	324
Serial Solution	329
Naive Pool Solution	329
A Less Naive Pool Solution	330
Using manager.Value as a Flag	331
Using Redis as a Flag	333
Using RawValue as a Flag	336
Using mmap as a Flag	337
Using mmap as a Flag Redux	338
Sharing numpy Data with multiprocessing	340
Synchronizing File and Variable Access	348
File Locking	348
Locking a Value	352
Wrap-Up	356
11. Clusters and Job Queues.....	357
Benefits of Clustering	358
Drawbacks of Clustering	359
\$462 Million Wall Street Loss Through Poor Cluster Upgrade Strategy	360
Skype's 24-Hour Global Outage	361
Common Cluster Designs	362
How to Start a Clustered Solution	362
Ways to Avoid Pain When Using Clusters	363
Two Clustering Solutions	365
Using IPython Parallel to Support Research	365
Message Brokering for Cluster Efficiency	368
Other Clustering Tools to Look At	372
Docker	373
Docker's Performance	373
Advantages of Docker	377
Wrap-Up	378
12. Using Less RAM.....	379
Objects for Primitives Are Expensive	380
The array Module Stores Many Primitive Objects Cheaply	382
Using Less RAM in NumPy with NumExpr	385
Understanding the RAM Used in a Collection	389
Bytes Versus Unicode	390
Efficiently Storing Lots of Text in RAM	391
Trying These Approaches on 11 Million Tokens	392
Modeling More Text with scikit-learn's FeatureHasher	400

Introducing DictVectorizer and FeatureHasher	401
Comparing DictVectorizer and FeatureHasher on a Real Problem	404
SciPy's Sparse Matrices	405
Tips for Using Less RAM	408
Probabilistic Data Structures	409
Very Approximate Counting with a 1-Byte Morris Counter	410
K-Minimum Values	413
Bloom Filters	417
LogLog Counter	423
Real-World Example	427
Wrap-Up	430
13. Lessons from the Field.....	431
Developing a High Performance Machine Learning Algorithm	432
High Performance Computing in Journalism	435
Lessons from the Field of Cyber Reinsurance	441
Python in Quant Finance	451
Maintain Flexibility to Achieve High Performance	455
Streamlining Feature Engineering Pipelines with Feature-engine (2020)	458
Highly Performant Data Science Teams (2020)	464
Numba (2020)	468
Optimizing Versus Thinking (2020)	474
Making Deep Learning Fly with RadimRehurek.com (2014)	477
Large-Scale Social Media Analysis at Smesh (2014)	483
Index.....	487

Foreword

When you think about high performance computing, you might imagine giant clusters of machines modeling complex weather phenomena or trying to understand signals in data collected about far-off stars. It's easy to assume that only people building specialized systems should worry about the performance characteristics of their code. By picking up this book, you've taken a step toward learning the theory and practices you'll need to write highly performant code. Every programmer can benefit from understanding how to build performant systems.

There is an obvious set of applications that are just on the edge of possible, and you won't be able to approach them without writing optimally performant code. If that's your practice, you're in the right place. But there is a much broader set of applications that can benefit from performant code.

We often think that new technical capabilities are what drives innovation, but I'm equally fond of capabilities that increase the accessibility of technology by orders of magnitude. When something becomes ten times cheaper in time or compute costs, suddenly the set of applications you can address is wider than you imagined.

The first time this principle manifested in my own work was over a decade ago, when I was working at a social media company, and we ran an analysis over multiple terabytes of data to determine whether people clicked on more photos of cats or dogs on social media.

It was dogs, of course. Cats just have better branding.

This was an outstandingly frivolous use of compute time and infrastructure at the time! Gaining the ability to apply techniques that had previously been restricted to sufficiently high-value applications, such as fraud detection, to a seemingly trivial question opened up a new world of now-accessible possibilities. We were able to take what we learned from these experiments and build a whole new set of products in search and content discovery.

For an example that you might encounter today, consider a machine learning system that recognizes unexpected animals or people in security video footage. A sufficiently performant system could allow you to embed that model into the camera itself, improving privacy or, even if running in the cloud, using significantly less compute and power—benefiting the environment and reducing your operating costs. This can free up resources for you to look at adjacent problems, potentially building a more valuable system.

We all desire to create systems that are effective, easy to understand, and performant. Unfortunately, it often feels like we have to pick only two (or one) out of the three! *High Performance Python* is a handbook for people who want to make things that are capable of all three.

This book stands apart from other texts on the subject in three ways. First, it's written for us—humans who write code. You'll find all of the context you need to understand why you might make certain choices. Second, Gorelick and Ozsvald do a wonderful job of curating and explaining the necessary theory to support that context. You'll also learn the specific quirks of the most useful libraries for implementing these approaches today.

This is one of a rare class of programming books that will change the way you think about the practice of programming. I've given this book to many people who could benefit from the additional tools it provides. The ideas that you'll explore in its pages will make you a better programmer, no matter what language or environment you choose to work in.

Enjoy the adventure.

— Hilary Mason,
Cofounder & CEO at Hidden Door

Preface

Python is easy to learn. You're probably here because now that your code runs correctly, you need it to run faster. You like the fact that your code is easy to modify and you can iterate with ideas quickly. The trade-off between *easy to develop* and *runs as quickly as I need* is a well-understood and often-bemoaned phenomenon. There are solutions.

Some people have serial processes that have to run faster. Others have problems that could take advantage of multicore architectures, clusters, or graphics processing units. Some need scalable systems that can process more or less as expediency and funds allow, without losing reliability. Others will realize that their coding techniques, often borrowed from other languages, perhaps aren't as natural as examples they see from others.

In this book we will cover all of these topics, giving practical guidance for understanding bottlenecks and producing faster and more scalable solutions. We also include some war stories from those who went ahead of you, who took the knocks so you don't have to.

Python is well suited for rapid development, production deployments, and scalable systems. The ecosystem is full of people who are working to make it scale on your behalf, leaving you more time to focus on the more challenging tasks around you.

Who This Book Is For

You've used Python for long enough to have an idea about why certain things are slow and to have seen technologies like Cython, numpy, and PyPy being discussed as possible solutions. You might also have programmed with other languages and so know that there's more than one way to solve a performance problem.

While this book is primarily aimed at people with CPU-bound problems, we also look at data transfer and memory-bound solutions. Typically, these problems are faced by scientists, engineers, quants, and academics.

We also look at problems that a web developer might face, including the movement of data and the use of just-in-time (JIT) compilers like PyPy and asynchronous I/O for easy-win performance gains.

It might help if you have a background in C (or C++, or maybe Java), but it isn't a prerequisite. Python's most common interpreter (CPython—the standard you normally get if you type `python` at the command line) is written in C, and so the hooks and libraries all expose the gory inner C machinery. There are lots of other techniques that we cover that don't assume any knowledge of C.

You might also have a lower-level knowledge of the CPU, memory architecture, and data buses, but again, that's not strictly necessary.

Who This Book Is Not For

This book is meant for intermediate to advanced Python programmers. Motivated novice Python programmers may be able to follow along as well, but we recommend having a solid Python foundation.

We don't cover storage-system optimization. If you have a SQL or NoSQL bottleneck, then this book probably won't help you.

What You'll Learn

Your authors have been working with large volumes of data, a requirement for *I want the answers faster!* and a need for scalable architectures, for many years in both industry and academia. We'll try to impart our hard-won experience to save you from making the mistakes that we've made.

At the start of each chapter, we'll list questions that the following text should answer. (If it doesn't, tell us and we'll fix it in the next revision!)

We cover the following topics:

- Background on the machinery of a computer so you know what's happening behind the scenes
- Lists and tuples—the subtle semantic and speed differences in these fundamental data structures
- Dictionaries and sets—memory allocation strategies and access algorithms in these important data structures
- Iterators—how to write in a more Pythonic way and open the door to infinite data streams using iteration
- Pure Python approaches—how to use Python and its modules effectively

- Matrices with `numpy`—how to use the beloved `numpy` library like a beast
- Compilation and just-in-time computing—processing faster by compiling down to machine code, making sure you’re guided by the results of profiling
- Concurrency—ways to move data efficiently
- `multiprocessing`—various ways to use the built-in `multiprocessing` library for parallel computing and to efficiently share `numpy` matrices, and some costs and benefits of interprocess communication (IPC)
- Cluster computing—convert your `multiprocessing` code to run on a local or remote cluster for both research and production systems
- Using less RAM—approaches to solving large problems without buying a humongous computer
- Lessons from the field—lessons encoded in war stories from those who took the blows so you don’t have to

Python 3

Python 3 is the standard version of Python as of 2020, with Python 2.7 deprecated after a 10-year migration process. If you’re still on Python 2.7, you’re doing it wrong—many libraries are no longer supported for your line of Python, and support will become more expensive over time. Please do the community a favor and migrate to Python 3, and make sure that all new projects use Python 3. Unless stated otherwise we’re using Python 3.12 (released 2023).

In this book, we use 64-bit Python. While 32-bit Python is supported, it is far less common for scientific work. We’d expect all the libraries to work as usual, but numeric precision, which depends on the number of bits available for counting, is likely to change. 64-bit is dominant in this field, along with `*nix` environments (often Linux or Mac). 64-bit lets you address larger amounts of RAM. `*nix` lets you build applications that can be deployed and configured in well-understood ways with well-understood behaviors.

If you’re a Windows user, you’ll have to buckle up. Most of what we show will work just fine, but some things are OS-specific, and you’ll have to research a Windows solution. The biggest difficulty a Windows user might face is the installation of modules: research in sites like Stack Overflow should give you the solutions you need. If you’re on Windows, having a virtual machine (e.g., using VirtualBox) with a running Linux installation might help you to experiment more freely.

Windows users should definitely look at a packaged solution like those available through Anaconda, Canopy, Python(x,y), or Sage. These same distributions will make the lives of Linux and Mac users far simpler too.

License

This book is licensed under [Creative Commons Attribution-NonCommercial-NoDerivs 3.0](#).

You're welcome to use this book for noncommercial purposes, including for noncommercial teaching. The license allows only for complete reproductions; for partial reproductions, please contact O'Reilly (see [“How to Contact Us”](#) on page xvi). Please attribute the book as noted in the following section.

We negotiated that the book should have a Creative Commons license so the contents could spread further around the world. We'd be quite happy to receive a beer if this decision has helped you. We suspect that the O'Reilly staff would feel similarly about the beer.

How to Make an Attribution

The Creative Commons license requires that you attribute your use of a part of this book. Attribution just means that you should write something that someone else can follow to find this book. The following would be sensible: “*High Performance Python*, 3rd ed., by Micha Gorelick and Ian Ozsvald (O'Reilly). Copyright 2025 Micha Gorelick and Ian Ozsvald, 978-1-098-16596-3.”

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://oreil.ly/supp-hpp3e>.

If you have a technical question or a problem using the code examples, please email support@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution (see the previous section for attribution).

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

Errata and Feedback

We encourage you to review this book on public sites like Amazon—please help others understand if they would benefit from this book! You can also email us at feedback@highperformancepython.com.

We're particularly keen to hear about errors in the book, successful use cases where the book has helped you, and high performance techniques that we should cover in the next edition. You can access the web page for this book at <https://oreil.ly/supp-hpp3e>.

Complaints are welcomed through the instant-complaint-transmission-service > /dev/null.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/hpp-3e>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

Hilary Mason wrote our foreword—thanks for composing such a wonderful opening narrative for our book. Giles Weaver, Alexander Mitchell, Mani Sarkar, and Pete Bleackley provided invaluable technical feedback on this edition.

Thanks to Patrick Cooper, Kyran Dale, Dan Foreman-Mackey, Calvin Giles, Brian Granger, Jamie Matthews, John Montgomery, Christian Schou Oxvig, Matt “snakes” Reiferson, Balthazar Rouberol, Michael Skirpan, Luke Underwood, Jake Vanderplas, and William Winter for invaluable feedback and contributions.

Ian thanks his wife, Emily, for letting him disappear for another eight months to write this third edition (thankfully, she’s terribly understanding). Ian apologizes to his dog for sitting and writing rather than walking in the woods quite as much as she’d have liked.

Micha thanks Storm, Velvet, Matthias, Oscar, and the rest of her friends and family for being so patient while she learned to write.

O’Reilly editors are rather lovely to work with; do strongly consider talking to them if you want to write your own book.

Our contributors to the “Lessons from the Field” chapter very kindly shared their time and hard-won lessons. We give thanks to Martin Goodson, James Poynter, David Rawlinson, Mikhail Timonin, and Leon Yin for this edition, and to Soledad Galli, Andrew Godwin, Valentin Haenel, Ben Jackson, Alex Kelly, Radim Řehůřek, Marko Tasic, Sebastjan Trepca, Linda Uruchurtu, and Vincent D. Warmerdam for their time and effort during the previous editions.

Understanding Performant Python

Questions You'll Be Able to Answer After This Chapter

- What are the elements of a computer's architecture?
- What are some common alternate computer architectures?
- How does Python abstract the underlying computer architecture?
- What are some of the hurdles to making performant Python code?
- What strategies can help you become a highly performant programmer?

Programming computers can be thought of as moving bits of data and transforming them in special ways to achieve a particular result. However, these actions have a time cost. Consequently, *high performance programming* can be thought of as the act of minimizing these operations either by reducing the overhead (i.e., writing more efficient code) or by changing the way that we do these operations to make each one more meaningful (i.e., finding a more suitable algorithm).

Let's focus on reducing the overhead in code in order to gain more insight into the actual hardware on which we are moving these bits. This may seem like a futile exercise, since Python works quite hard to abstract away direct interactions with the hardware. However, by understanding both the best way that bits can be moved in the real hardware and the ways that Python's abstractions force your bits to move, you can make progress toward writing high performance programs in Python.

The Fundamental Computer System

The underlying components that make up a computer can be simplified into three basic parts: the computing units, the memory units, and the connections between them. In addition, each of these units has different properties that we can use to understand it. The computational unit has the property of how many computations it can do per second, the memory unit has the properties of how much data it can hold and how fast we can read from and write to it, and finally, the connections have the property of how fast they can move data from one place to another.

Using these building blocks, we can talk about a standard workstation at multiple levels of sophistication. For example, the standard workstation can be thought of as having a central processing unit (CPU) as the computational unit, connected to both the random access memory (RAM) and the hard drive as two separate memory units (each having different capacities and read/write speeds), and finally a bus that provides the connections between all of these parts. However, we can also go into more detail and see that the CPU itself has several memory units in it: the L1, the L2, and sometimes even the L3 and L4 caches, which have small capacities but very fast speeds (from several kilobytes to a dozen megabytes). Furthermore, new computer architectures generally come with new configurations (for example, Intel's Skylake CPUs replaced the frontside bus with the Intel Ultra Path Interconnect and restructured many connections and more recently, AMD's Infinity Fabric created dedicated interconnects for faster CPU-CPU or CPU-GPU communication). Finally, in both of these approximations of a workstation, we have neglected the network connection, which is effectively a very slow connection to potentially many other computing and memory units!

To help untangle these various intricacies, let's go over a brief description of these fundamental blocks.

Computing Units

The *computing unit* of a computer is the centerpiece of its usefulness—it provides the ability to transform any bits it receives into other bits or to change the state of the current process. CPUs are the most commonly used computing unit; however, graphics processing units (GPUs), as well as the myriad of other options such as TPUs, IPU's, and FPGAs, are gaining popularity as auxiliary computing units. They were originally used to speed up computer graphics but are becoming more applicable for numerical applications and are useful thanks to their intrinsically parallel nature, which allows many calculations to happen simultaneously. Regardless of its type, a computing unit takes in a series of bits (for example, bits representing numbers) and outputs another set of bits (for example, bits representing the sum of those numbers). In addition to the basic arithmetic operations on integers and real numbers and bitwise operations on binary numbers, some computing units also provide very

specialized operations, such as the “fused multiply add” operation, which takes in three numbers, A, B, and C, and returns the value $A * B + C$.

The main properties of interest in a computing unit are the number of operations it can do in one cycle and the number of cycles it can do in one second. The first value is measured by its instructions per cycle (IPC),¹ while the latter value is measured by its clock speed. These two measures are always competing with each other when new computing units are being made. For example, the Intel Core series has a very high IPC but a lower clock speed, while the Pentium 4 chip has the reverse. GPUs, on the other hand, have a very high IPC and clock speed, but they suffer from other problems, such as the slow communications that we discuss in “Communications Layers” on page 8.

Furthermore, although increasing clock speed almost immediately speeds up all programs running on that computational unit (because they can do more calculations per second), having a higher IPC can also drastically affect computing by changing the level of *vectorization* that is possible. Vectorization occurs when a CPU is provided with multiple pieces of data at a time and is able to operate on all of them at once. This sort of CPU instruction is known as single instruction, multiple data (SIMD).

In general, computing units have advanced quite slowly over the past decade (see Figure 1-1). Clock speeds and IPC have both been stagnant because of the physical limitations of making transistors smaller and smaller. As a result, chip manufacturers have been relying on other methods to gain more speed, including simultaneous multithreading (where multiple threads can run at once), more clever out-of-order execution, and multicore architectures.

Hyperthreading presents a virtual second CPU to the host operating system (OS), and clever hardware logic tries to interleave two threads of instructions into the execution units on a single CPU. When successful, **gains of up to 30% over a single thread** can be achieved. Typically, this works well when the units of work across both threads use different types of execution units—for example, one performs floating-point operations and the other performs integer operations.

Out-of-order execution enables a compiler to spot that some parts of a linear program sequence do not depend on the results of a previous piece of work, and therefore those pieces of work could occur in any order or at the same time. As long as sequential results are presented at the right time, the program continues to execute correctly, even though pieces of work are computed out of their programmed order. This enables some instructions to execute when others might be blocked (e.g., waiting for a memory access), allowing greater overall utilization of the available resources.

¹ Not to be confused with interprocess communication, which shares the same acronym—we’ll look at that topic in Chapter 10.

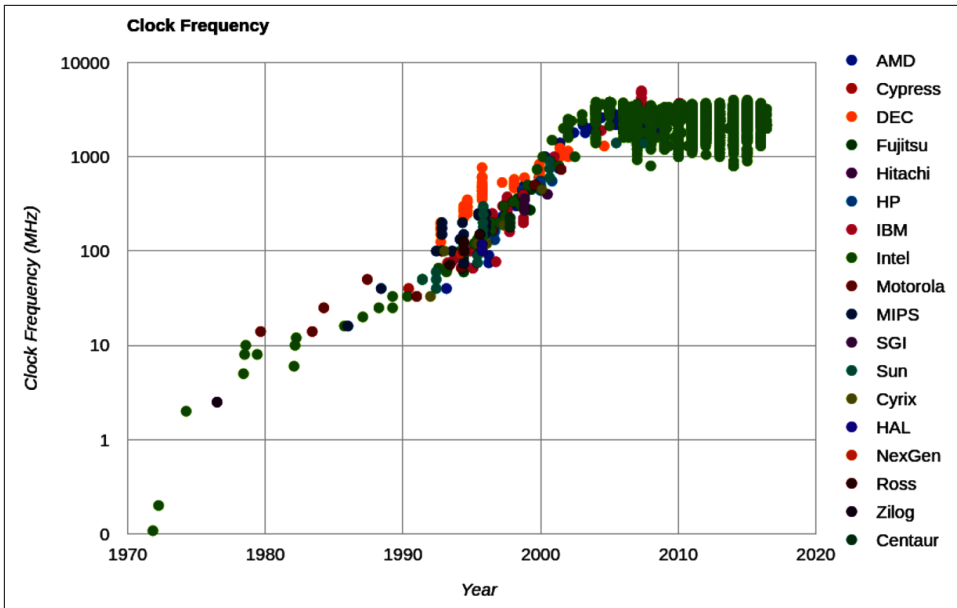


Figure 1-1. Clock speed of CPUs over time (from *CPU DB*)

Finally, and most important for the higher-level programmer, there is the prevalence of multicore architectures. These architectures include multiple CPUs within the same chip, which increases the total capability without running into barriers to making each individual unit faster. This is why it is currently hard to find any machine with fewer than two cores—in this case, the computer has two physical computing units that are connected to each other. While this increases the total number of operations that *can* be done per second, it can make writing code more difficult!

Simply adding more cores to a CPU does not always speed up a program's execution time. This is because of something known as *Amdahl's law*. In our context, Amdahl's law can be interpreted as saying: if a program designed to run on multiple cores has some subroutines that must run on one core, this will be the limitation for the maximum speedup that can be achieved by allocating more cores.

For example, if we had a survey we wanted one hundred people to fill out, and that survey took 1 minute to complete, we could complete this task in 100 minutes if we had one person asking the questions (i.e., this person goes to participant 1, asks the questions, waits for the responses, and then moves to participant 2). This method of having one person asking the questions and waiting for responses is similar to a serial process. In serial processes, we have operations being satisfied one at a time, each one waiting for the previous operation to complete.

However, we could perform the survey in parallel if we had two people asking the questions, which would let us finish the process in only 50 minutes. This can be done because each individual person asking the questions does not need to know anything about the other person asking questions. As a result, the task can easily be split up without having any dependency between the question askers.

Adding more people asking the questions will give us more speedups, until we have one hundred people asking questions. At this point, the process would take 1 minute and would be limited simply by the time it takes a participant to answer questions. Adding more people asking questions will not result in any further speedups, because these extra people will have no tasks to perform—all the participants are already being asked questions! At this point, the only way to reduce the overall time to run the survey is to reduce the amount of time it takes for an individual survey, the serial portion of the problem, to complete. Similarly, with CPUs, we can add more cores that can perform various chunks of the computation as necessary until we reach a point where the bottleneck is the time it takes for a specific core to finish its task. In other words, the bottleneck in any parallel calculation is always the smaller serial tasks that are being spread out. On top of this are physical constraints where, as we pack more and more cores into smaller and smaller spaces, heat becomes an issue, and thermal throttling (i.e., cores being slowed down to keep them from overheating) becomes an important issue!

However, a major hurdle with utilizing multiple cores in Python is Python's use of a *global interpreter lock* (GIL). The GIL makes sure that a Python process can run only one instruction at a time, regardless of the number of cores it is currently using. This means that even though some Python code has access to multiple cores at a time, only one core is running a Python instruction at any given time. Using the previous example of a survey, this would mean that even if we had 100 question askers, only one person could ask a question and listen to a response at a time. This effectively removes any sort of benefit from having multiple question askers! While this may seem like quite a hurdle, especially if the current trend in computing is to have multiple computing units rather than having faster ones, this problem can be avoided by using other standard library tools like `multiprocessing` (Chapter 10); technologies like `numpy` or `numexpr` (Chapter 6), `Cython` or `Numba` (Chapter 8); or distributed models of computing (Chapter 11).



Python 3.2 also saw a major rewrite of the GIL which made the system much more nimble, alleviating many of the concerns around the system for single-thread performance. Furthermore, there are proposals to make the GIL itself optional (see “Where Did the GIL Go?” on page 25). Although it still locks Python into running only one instruction at a time, the GIL now does better at switching between those instructions and doing so with less overhead.

Memory Units

Memory units in computers are used to store bits. These could be bits representing variables in your program or representing the pixels of an image. Thus, the abstraction of a memory unit applies to the registers in your motherboard as well as your RAM and hard drive. The major difference between all of these types of memory units is the speed at which they can read/write data. To make things more complicated, the read/write speed is heavily dependent on the way that data is being read.

For example, most memory units perform much better when they read one large chunk of data as opposed to many small chunks (this is referred to as *sequential read* versus *random data*). If the data in these memory units is thought of as pages in a large book, this means that most memory units have better read/write speeds when going through the book page by page rather than constantly flipping from one random page to another. While this fact is generally true across all memory units, the amount that this affects each type is drastically different.

In addition to the read/write speeds, memory units also have *latency*, which can be characterized as the time it takes the device to find the data that is being used. For a spinning hard drive, this latency can be high because the disk needs to physically spin up to speed and the read head must move to the right position. On the other hand, for RAM this latency can be quite small because everything is solid state. Here is a short description of the various memory units that are commonly found inside a standard workstation, in order of read/write speeds:²

Hard disk drive (HDD)

Long-term storage that persists even when the computer is shut down. Generally has slow read/write speeds because it operates with a physical disk that must be spun up to speed and a read head that must move to the correct position to find your data. Degraded performance with random access patterns but very large capacity (20 terabyte range).

Solid-state drive (SSD)

Similar to a spinning hard drive, with faster read/write speeds but smaller capacity (1 terabyte range).

RAM

Used to store application code and data (such as any variables being used). Has fast read/write characteristics and performs well with random access patterns, but is generally limited in capacity (64 gigabyte range).

² Speeds in this section are from [Colin Scott's interactive latency trends website](#).

L1/L2 cache

Extremely fast read/write speeds. Data going to the CPU *must* go through here. Very small capacity (dozens of megabytes range).

Figure 1-2 gives a graphic representation of the differences between these types of memory units by looking at the characteristics of currently available consumer hardware.

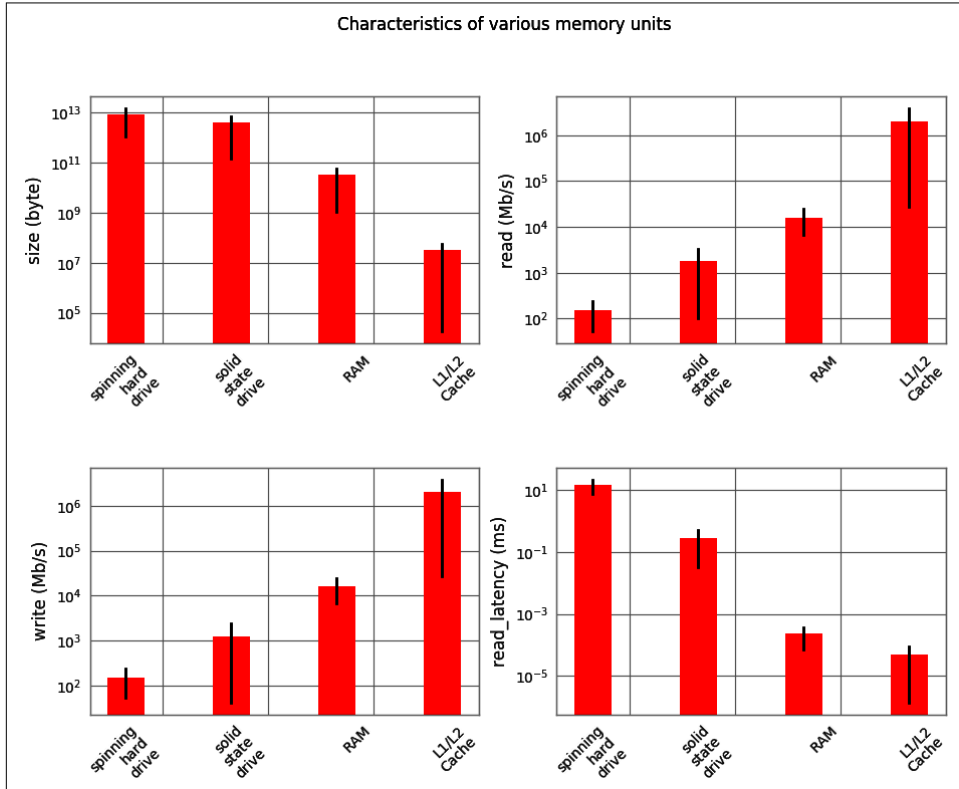


Figure 1-2. Characteristic values for different types of memory units³

A clearly visible trend is that read/write speeds and capacity are inversely proportional—as we try to increase speed, capacity gets reduced. Because of this, many systems implement a tiered approach to memory: data starts in its full state in the hard drive, part of it moves to RAM, and then a much smaller subset moves to the L1/L2 cache. This method of tiering enables programs to keep memory in different places depending on access speed requirements. When trying to optimize

³ The data for this figure is from February 2014; however, the order of magnitude differences are still the same in 2025.

the memory patterns of a program, we are simply optimizing which data is placed where, how it is laid out (in order to increase the number of sequential reads), and how many times it is moved among the various locations. In addition, methods such as asynchronous I/O and preemptive caching provide ways to make sure that data is always where it needs to be without having to waste computing time waiting for the I/O to complete—most of these processes can happen independently, while other calculations are being performed! We will discuss these methods in [Chapter 9](#).

Communications Layers

Finally, let's look at how all of these fundamental blocks communicate with each other. Many modes of communication exist, but all are variants on a thing called a *bus*.

The *frontside bus*, for example, is the connection between the RAM and the L1/L2 cache. It moves data that is ready to be transformed by the processor into the staging ground to get ready for calculation, and it moves finished calculations out. There are other buses, too, such as the external bus that acts as the main route from hardware devices (such as hard drives and networking cards) to the CPU and system memory. This external bus is generally slower than the frontside bus.

In fact, many of the benefits of the L1/L2 cache are attributable to the faster bus. Being able to queue up data necessary for computation in large chunks on a slow bus (from RAM to cache) and then having it available at very fast speeds from the cache lines (from cache to CPU) enables the CPU to do more calculations without waiting such a long time.

Similarly, many of the drawbacks of using a GPU come from the bus it is connected on: since the GPU is generally a peripheral device, it communicates through the PCI bus, which is much slower than the frontside bus. As a result, getting data into and out of the GPU can be quite a taxing operation. The advent of heterogeneous computing, or computing blocks that have both a CPU and a GPU on the frontside bus or connected with the Infinity Fabric, aims at reducing the data transfer cost and making GPU computing more of an available option, even when a lot of data must be transferred. Nowadays it is even common for some chips to include CPU and GPU in the same package. More recently, NPU (or neural processing units) are even included to help speed up specific AI applications.

In addition to the communication blocks within the computer, the network can be thought of as yet another communication block. This block, though, is much more pliable than the ones discussed previously; a network device can be connected to a memory device, such as a network attached storage (NAS) device or another computing block, as in a computing node in a cluster. However, network communications are generally much slower than the other types of communications mentioned previously. While the frontside bus can transfer dozens of gigabits per second, the network is limited to the order of several dozen megabits.

It is clear, then, that the main property of a bus is its speed—how much data it can move in a given amount of time. This property is given by combining two quantities: how much data can be moved in one transfer (bus width) and how many transfers the bus can do per second (bus frequency). It is important to note that the data moved in one transfer is always sequential: a chunk of data is read off of the memory and moved to a different place.

Thus, the speed of a bus is broken into these two quantities because individually they can affect different aspects of computation. A large bus width can help vectorized code (or any code that sequentially reads through memory) by making it possible to move all the relevant data in one transfer. On the other hand, having a small bus width but a very high frequency of transfers can help code that must do many reads from random parts of memory. Interestingly, one of the ways that these properties are changed by computer designers is through the physical layout of the motherboard: when chips are placed close to one another, the length of the physical wires joining them is smaller, which can allow for faster transfer speeds. In addition, the number of printed circuit board traces itself dictates the width of the bus (giving real physical meaning to the term!).

Since interfaces can be tuned to give the right performance for a specific application, it is no surprise that there are hundreds of types. **Figure 1-3** shows the bitrates for a sampling of common interfaces. Note that this doesn't speak at all about the latency of the connections, which dictates how long it takes for a data request to be responded to (although latency is very computer-dependent, some basic limitations are inherent to the interfaces being used).

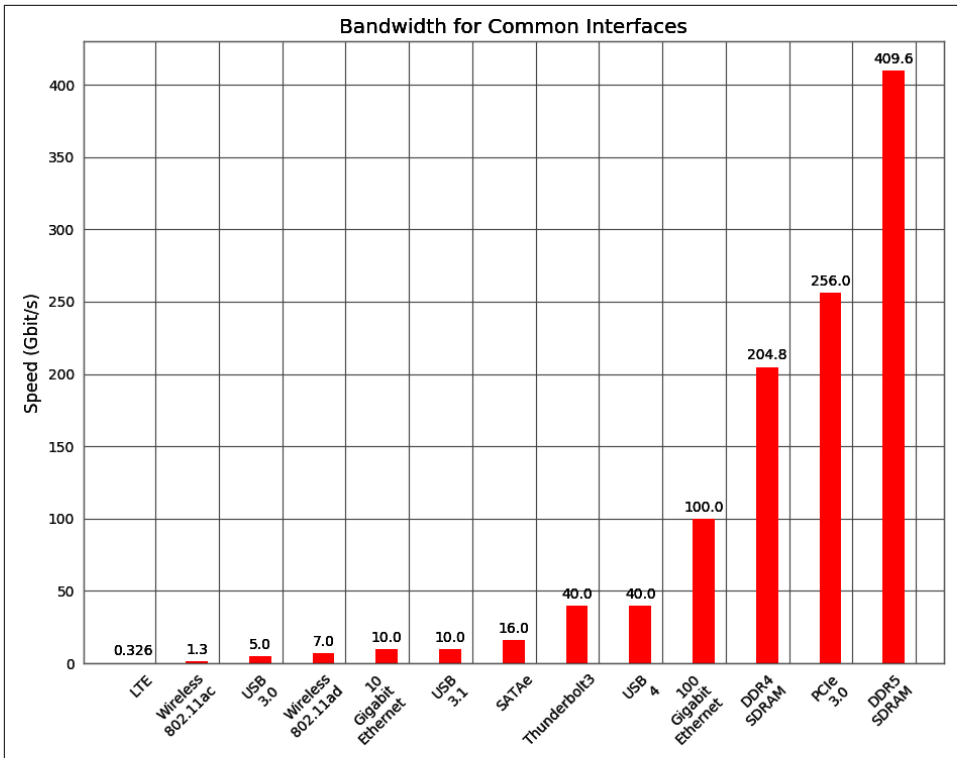


Figure 1-3. Connection speeds of various common interfaces⁴

Idealized Computing Versus the Python Virtual Machine

Understanding the basic components of a computer is not enough to fully understand the problems of high performance programming. The interplay of all of these components and how they work together to solve a problem introduces extra levels of complexity. In this section we will explore some toy problems, illustrating how the ideal solutions would work and how Python approaches them.

To better understand the components of high performance programming, let's look at a simple code sample that checks whether a number is prime:

```
import math

def check_prime(number):
    sqrt_number = math.sqrt(number)
    for i in range(2, int(sqrt_number) + 1):
        if (number / i).is_integer():
```

⁴ Data is from [Wikipedia's list of interface bit rates](#).

```
        return False
    return True
```

```
print(f"check_prime(10,000,000) = {check_prime(10_000_000)}")
# check_prime(10,000,000) = False
print(f"check_prime(10,000,019) = {check_prime(10_000_019)}")
# check_prime(10,000,019) = True
```

Let's analyze this code using our abstract model of computation and then draw comparisons to what happens when Python runs this code. As with any abstraction, we will neglect many of the subtleties in both the idealized computer and the way that Python runs the code. However, this is generally a good exercise to perform before solving a problem: think about the general components of the algorithm and what would be the best way for the computing components to come together to find a solution. By understanding this ideal situation and having knowledge of what is actually happening under the hood in Python, we can iteratively bring our Python code closer to the optimal code.



This section may seem bleak—most of the remarks in this section seem to say that Python is natively incapable of dealing with the problems of performance. This is untrue, for two reasons. First, among all of the “components of performing computing,” we have neglected one very important component: the developer. What native Python may lack in performance, it gets back right away with speed of development. Furthermore, throughout the book we will introduce modules and philosophies that can help mitigate many of the problems described here with relative ease. With both of these aspects combined, we will keep the fast development mindset of Python while removing many of the performance constraints.

Idealized Computing

When the code starts, we have the value of `number` stored in RAM. To calculate `sqrt_number`, we need to send the value of `number` to the CPU. Ideally, we could send the value once; it would get stored inside the CPU's L1/L2 cache, and the CPU would do the calculations and then send the values back to RAM to get stored. This scenario is ideal because we have minimized the number of reads of the value of `number` from RAM, instead opting for reads from the L1/L2 cache, which are much faster. Furthermore, we have minimized the number of data transfers through the frontside bus by using the L1/L2 cache, which is connected directly to the CPU.



This theme of keeping data where it is needed and moving it as little as possible is very important when it comes to optimization. The concept of “heavy data” refers to the time and effort required to move data around, which is something we would like to avoid.

For the loop in the code, rather than sending one value of *i* at a time to the CPU, we would like to send both *number* and *several* values of *i* to the CPU to check at the same time. This is possible because the CPU vectorizes operations with no additional time cost, meaning it can do multiple independent computations at the same time. So we want to send *number* to the CPU cache, in addition to as many values of *i* as the cache can hold. For each of the *number*/*i* pairs, we will divide them and check if the result is a whole number; then we will send a signal back indicating whether any of the values was indeed an integer. If so, the function ends. If not, we repeat. In this way, we need to communicate back only one result for many values of *i*, rather than depending on the slow bus for every value. This takes advantage of a CPU’s ability to *vectorize* a calculation, or run one instruction on multiple data in one clock cycle.

This concept of vectorization is illustrated by the following code:

```
import math

def check_prime(number, V=8):
    sqrt_number = math.sqrt(number)
    numbers = range(2, int(sqrt_number)+1)
    for i in range(0, len(numbers), V):
        # the following line is not valid Python code
        result = (number / numbers[i:(i + V)]).is_integer()
        if any(result):
            return False
    return True
```

Here, we set up the processing such that the division and the checking for integers are done on a set of *V* values of *i* at a time. If properly vectorized, the CPU can do this line in one step as opposed to doing a separate calculation for every *i*. Ideally, the `any(result)` operation would also happen in the CPU without having to transfer the results back to RAM. We will talk more about vectorization, how it works, and when it benefits your code in [Chapter 6](#).

Python’s Virtual Machine

The Python interpreter does a lot of work to try to abstract away the underlying computing elements that are being used. At no point does a programmer need to worry about allocating memory for arrays, how to arrange that memory, or in what sequence it is being sent to the CPU. This is a benefit of Python, since it lets you focus on the algorithms that are being implemented. However, it comes at a huge performance cost.

It is important to realize that at its core, Python is indeed running a set of very optimized instructions. The trick, however, is getting Python to perform them in the correct sequence to achieve better performance. For example, it is quite easy to see that, in the following example, `search_fast` will run faster than `search_slow` simply because it skips the unnecessary computations that result from not terminating the loop early, even though both solutions have runtime $O(n)$. However, things can get complicated when dealing with derived types, special Python methods, or third-party modules. For example, can you immediately tell which function will be faster: `search_unknown1` or `search_unknown2`?

```
def search_fast(haystack, needle):
    for item in haystack:
        if item == needle:
            return True
    return False

def search_slow(haystack, needle):
    return_value = False
    for item in haystack:
        if item == needle:
            return_value = True
    return return_value

def search_unknown1(haystack, needle):
    return any(item == needle for item in haystack)

def search_unknown2(haystack, needle):
    return any([item == needle for item in haystack])
```

Identifying slow regions of code through profiling and finding more efficient ways of doing the same calculations is similar to finding these useless operations and removing them; the end result is the same, but the number of computations and data transfers is reduced drastically.



The `search_unknown1` and `search_unknown2` example is particularly diabolical. Do you know which function would be faster for a small haystack? How about a large but sorted haystack? What if the haystack had no order? What if the needle was near the beginning or near the end? Each of these factors change which one is faster and for what reason. This is why actively profiling your code is so important. We also hope that by the time you finish reading this book, you'll have some intuition about which cases affect the different functions, why, and what the ramifications are.

One of the impacts of this abstraction layer is that vectorization is not immediately achievable. Our initial prime number routine will run one iteration of the loop per value of `i` instead of combining several iterations. However, looking at the abstracted

vectorization example, we see that it is not valid Python code, since we cannot divide a float by a list. External libraries such as `numpy` will help with this situation by adding the ability to do vectorized mathematical operations.

Furthermore, Python's abstraction hurts any optimizations that rely on keeping the L1/L2 cache filled with the relevant data for the next computation. This comes from many factors, the first being that Python objects aren't laid out in the most optimal way in memory. This is a consequence of Python being a garbage-collected language—memory is automatically allocated and freed when needed. This creates memory fragmentation that can hurt the transfers to the CPU caches. In addition, at no point is there an opportunity to change the layout of a data structure directly in memory, which means that one transfer on the bus may not contain all the relevant information for a computation, even though it might have all fit within the bus width.⁵

A second, more fundamental problem comes from Python's dynamic types and the language not being compiled. As many C programmers have learned throughout the years, the compiler is often smarter than you are. When compiling code that is typed and static, the compiler can do many tricks to change the way things are laid out and how the CPU will run certain instructions in order to optimize them. Python, however, is not compiled; to make matters worse, it has dynamic types, which means that inferring any possible opportunities for optimizations algorithmically is drastically harder, since code functionality can be changed during runtime. There are many ways to mitigate this problem, foremost being the use of Cython, which allows Python code to be compiled and allows the user to create “hints” to the compiler as to how dynamic the code actually is. Furthermore, Python is on track to have a just-in-time (JIT) compiler, which will allow the code to be compiled and optimized during runtime (more on this in [“Does Python Have a JIT?” on page 25](#)).

Finally, the previously mentioned GIL can hurt performance if trying to parallelize this code. For example, let's assume we change the code to use multiple CPU cores such that each core gets a chunk of the numbers from 2 to `sqrtN`. Each core can do its calculation for its chunk of numbers, and then, when the calculations are all done, the cores can compare their calculations. Although we lose the early termination of the loop since each core doesn't know if a solution has been found, we can reduce the number of checks each core has to do (if we had `M` cores, each core would have to do `sqrtN / M` checks). However, because of the GIL, only one core can be used at a time. This means that we would effectively be running the same code as the unparallelized version, but we no longer have early termination. We can avoid this problem by using

⁵ In [Chapter 6](#), we'll see how we can regain this control and tune our code all the way down to the memory utilization patterns.

multiple processes (with the `multiprocessing` module) instead of multiple threads, or by using Cython or foreign functions.

So Why Use Python?

Python is highly expressive and easy to learn—new programmers quickly discover that they can do quite a lot in a short amount of time. Many Python libraries wrap tools written in other languages to make it easy to call other systems; for example, the scikit-learn machine learning system wraps LIBLINEAR and LIBSVM (both of which are written in C), and the `numpy` library includes BLAS and other C and Fortran libraries. As a result, Python code that properly utilizes these modules can indeed be as fast as comparable C code.

Python is described as “batteries included,” as many important tools and stable libraries are built in. These include the following:

`io`

All sorts of IO for bytes, strings, and all the encodings you will have to deal with

`array`

Memory-efficient arrays for primitive types

`math`

Basic mathematical operations, including some simple statistics

`sqlite3`

A wrapper around the prevalent SQL file-based storage engine SQLite3

`collections`

A wide variety of objects, including a deque, a counter, and dictionary variants

`asyncio`

Concurrent support for I/O-bound tasks using `async` and `await` syntax

A huge variety of libraries can be found outside the core language, including these:

`numpy`

A numerical Python library (a bedrock library for anything to do with matrices)

`scipy`

A very large collection of trusted scientific libraries, often wrapping highly respected C and Fortran libraries

`pandas`

A library for data analysis, similar to R's dataframes or an Excel spreadsheet, built on `scipy` and `numpy`

`polars`

An alternative to `pandas` with a built-in query planner for faster and parallelized execution of queries

scikit-learn

The default machine learning library, built on `scipy`

PyTorch and TensorFlow

Deep learning frameworks from Facebook and Google with strong Python and GPU support

`NLTK`, `spaCy`, and `Gensim`

Natural language processing libraries with deep Python support

Database bindings

For communicating with virtually all databases, including Redis, Elasticsearch, HDF5, and SQL

Web development frameworks

Performant systems for creating websites, such as `aiohttp`, `django`, `pyramid`, `fastapi`, or `flask`

`OpenCV`

Bindings for computer vision

API bindings

For easy access to popular web APIs such as Google, Twitter, and LinkedIn

A large selection of managed environments and shells is available to fit various deployment scenarios, including the following:

- The standard distribution, available at <http://python.org>
- `pipenv`, `pyenv`, and `virtualenv` for simple, lightweight, and portable Python environments
- Docker for simple-to-start-and-reproduce environments for development or production
- Anaconda Inc.'s Anaconda, a scientifically focused environment
- IPython, an interactive Python shell heavily used by scientists and developers
- Jupyter Notebook and JupyterLab, a browser-based extension to IPython, heavily used for teaching and demonstrations

One of Python's main strengths is that it enables fast prototyping of an idea. Because of the wide variety of supporting libraries, it is easy to test whether an idea is feasible, even if the first implementation might be rather flaky.

If you want to make your mathematical routines faster, look to `numpy`. If you want to experiment with machine learning, try `scikit-learn`. If you are cleaning and manipulating data, then `pandas` is a good choice.

In general, it is sensible to raise the question, “If our system runs faster, will we as a team run slower in the long run?” It is always possible to squeeze more performance out of a system if enough work hours are invested, but this might lead to brittle and poorly understood optimizations that ultimately trip up the team.

One example might be the introduction of Cython (see “[Cython](#)” on page 219), a compiler-based approach to annotating Python code with C-like types so the transformed code can be compiled using a C compiler. While the speed gains can be impressive (often achieving C-like speeds with relatively little effort), the cost of supporting this code will increase. In particular, it might be harder to support this new module, as team members will need a certain maturity in their programming ability to understand some of the trade-offs that have occurred when leaving the Python virtual machine that introduced the performance increase.

How to Be a Highly Performant Programmer

Writing high performance code is only one part of being highly performant with successful projects over the longer term. Overall team velocity is far more important than speedups and complicated solutions. Several factors are key to this—good structure, documentation, debuggability, and shared standards.

Let's say you create a prototype. You didn't test it thoroughly, and it didn't get reviewed by your team. It does seem to be “good enough,” and it gets pushed to production. Since it was never written in a structured way, it lacks tests and is undocumented. All of a sudden there's an inertia-causing piece of code for someone else to support, and often management can't quantify the cost to the team.

As this solution is hard to maintain, it tends to stay unloved—it never gets restructured, it doesn't get the tests that'd help the team refactor it, and nobody else likes to touch it, so it falls to one developer to keep it running. This can cause an awful bottleneck at times of stress and raises a significant risk: what would happen if that developer left the project?

Typically, this development style occurs when the management team doesn't understand the ongoing inertia that's caused by hard-to-maintain code. Demonstrating that in the longer term, tests and documentation can help a team stay highly productive and can help convince managers to allocate time to "cleaning up" this prototype code.

In a research environment, it is common to create many Jupyter Notebooks using poor coding practices while iterating through ideas and different datasets. The intention is always to "write it up properly" at a later stage, but that later stage never occurs. In the end, a working result is obtained, but the infrastructure to reproduce it, test it, and trust the result is missing. Once again the risk factors are high, and the trust in the result will be low.

Here's a general approach that will serve you well:

1. Make it work

First you build a good-enough solution. It is very sensible to build "one to throw away" that acts as a prototype solution, enabling a better structure to be used for the second version. It is always sensible to do some up-front planning before coding; otherwise, you'll come to reflect that "We saved an hour's thinking by coding all afternoon." In some fields this is better known as "Measure twice, cut once."

2. Make it right

Next, you add a strong test suite backed by documentation and clear reproducibility instructions so that another team member can take it on. This is also a good place to talk about the intention of the code, the challenges that were faced while coming up with the solution, and any notes about the process of building the working version. This will help any future team members when this code needs to be refactored, fixed, or rebuilt.

3. Make it fast

Finally, you can focus on profiling and compiling or parallelization and using the existing test suite to confirm that the new, faster solution still works as expected.

Good Working Practices

There are a few "must haves"—documentation, good structure, and testing are key.

Some project-level documentation will help you stick to a clean structure. It'll also help you and your colleagues in the future. Nobody will thank you (yourself included) if you skip this part. Writing this up in a *README* file at the top level is a sensible starting point; it can always be expanded into a *docs/* folder later if required.

Explain the purpose of the project, what's in the folders, where the data comes from, which files are critical, and how to run it all, including how to run the tests.

A *NOTES* file is also a good solution for temporarily storing useful commands, function defaults, or other wisdom, tips, or tricks for using the code. While this should ideally be put in the documentation, having a scratchpad to keep this information in before it (hopefully) gets into the documentation can be invaluable in not forgetting the important little bits.⁶

Micha recommends also using Docker. A top-level Dockerfile will explain to your future self exactly which libraries you need from the operating system to make this project run successfully. It also removes the difficulty of running this code on other machines or deploying it to a cloud environment. Often when inheriting new code, simply getting it up and running to play with can be a major hurdle. A Dockerfile removes this hurdle and lets other developers start interacting with your code immediately.

Add a *tests/* folder and add some unit tests. We prefer *pytest* as a modern test runner, as it builds on Python's built-in *unittest* module. Start with just a couple of tests and then build them up. Progress to using the *coverage* tool, which will report how many lines of your code are actually covered by the tests—it'll help avoid nasty surprises.

If you're inheriting legacy code and it lacks tests, a high-value activity is to add some tests up front. Some "integration tests" that check the overall flow of the project and confirm that with certain input data you get specific output results will help your sanity as you subsequently make modifications.

Every time something in the code bites you, add a test. There's no value to being bitten twice by the same problem.

Docstrings in your code for each function, class, and module will always help you. Aim to provide a useful description of what's *achieved* by the function, and where possible include a short example to demonstrate the expected output. Look at the docstrings inside *numpy* and *scikit-learn* if you'd like inspiration.

Whenever your code becomes too long—such as functions longer than one screen—be comfortable with refactoring the code to make it shorter. Shorter code is easier to test and easier to support.

⁶ Micha generally keeps a notes file open while developing a solution; once things are working, she spends time clearing out the notes file into proper documentation and auxiliary tests and benchmarks.



When you're developing your tests, think about following a test-driven development methodology. When you know exactly what you need to develop and you have testable examples at hand, this method becomes very efficient.

You write your tests, run them, watch them fail, and *then* add the functions and the necessary minimum logic to support the tests that you've written. When your tests all work, you're done. By figuring out the expected input and output of a function ahead of time, you'll find implementing the logic of the function relatively straightforward.

If you can't define your tests ahead of time, it naturally raises the question: do you really understand what your function needs to do? If not, can you write it correctly in an efficient manner? This method doesn't work so well if you're in a creative process and researching data that you don't yet understand well.

Always use source control—you'll only thank yourself when you overwrite something critical at an inconvenient moment. Get used to committing frequently (daily, or even every 10 minutes) and pushing to your repository every day.

Keep to the PEP 8 coding standard. Even better, adopt `black` (the opinionated code formatter) on a pre-commit source control hook so it just rewrites your code to the standard for you. Use `flake8` to lint your code to avoid other mistakes.

Creating environments that are isolated from the operating system will make your life easier. Ian prefers Anaconda, while Micha prefers `pyenv` coupled with `virtualenv` or just using Docker. Both are sensible solutions and are significantly better than using the operating system's global Python environment!

Remember that automation is your friend. Doing less manual work means there's less chance of errors creeping in. Automated build systems, continuous integration with automated test suite runners, and automated deployment systems turn tedious and error-prone tasks into standard processes that anyone can run and support. It is never a waste of time to build out your continuous integration toolkit (like running tests automatically when code is checked into your code repository), as it will speed up and streamline future development.



Remember that “doing things the hard way” is the way we learn—if we outsource our thinking to a GenAI system, then we’ll always get an answer, and sometimes it might even be right. By creatively figuring out your own solutions you’ll continue to build new patterns in your head, rather than reinforcing those that already exist in the wider world. This is to your benefit. As a simple example, GitHub Copilot wrote a simple regular expression for Ian—but it wasn’t how Ian would write a regular expression, so it took a while to figure it out. In hindsight it would have been faster to think for a bit longer and write a solution that “fitted how Ian thought about this problem,” rather than introducing a solution from a random internet typist.

Building libraries is a great way to save on copy-and-paste solutions between early stage projects. It is tempting to copy and paste snippets of code because it is quick, but over time you’ll have a set of slightly different but basically the same solutions, each with few or no tests, thus allowing more bugs and edge cases to impact your work. Sometimes stepping back and identifying opportunities to write a first library can yield a significant win for a team.

Finally, remember that readability is far more important than being clever. Short snippets of complex and hard-to-read code will be hard for you and your colleagues to maintain, so people will be scared of touching this code. Instead, write a longer, easier-to-read function and back it with useful documentation showing what it’ll return, and complement this with tests to confirm that it *does* work as you expect.

Optimizing for the Team Rather than the Code Block

There are many ways to lose time when building a solution. At worst maybe you’re working on the *wrong problem* or with the *wrong approach*, or maybe you’re on the right track but there are taxes in your development process that slow you down, or maybe you haven’t estimated the true costs and uncertainties that might get in your way. Or maybe you misunderstand the needs of the stakeholders and spend time building a feature or solving a problem that doesn’t actually exist.⁷

Making sure you’re solving a *useful problem* is critical. Finding a cool project with cutting-edge technology and lots of neat acronyms can be wonderfully fun—but it is unlikely to deliver the value that other project members will appreciate. If you’re in an organization that is trying to cause a positive change, you’ll want to focus on problems that act as blockers, where a fix leads to a clear positive outcome.

⁷ Micha has, on several occasions, shadowed stakeholders throughout their day to better understand how they worked, how they approached problems, and what their day-to-day was like. This “take a developer to work day” approach helped her better adapt her technical solutions to their needs.

Having found potentially useful problems to solve, it is worth reflecting—can we achieve a *meaningful* change? Just fixing “the tech” behind a problem won’t change the real world. The solution needs to be deployed and maintained and needs to be adopted by human users. If there’s resistance or blockage to the technical solution, then your work will go nowhere.

Having decided that those blockers aren’t a worry, have you estimated the potential impact you can *realistically* have? If you find a part of your problem space where you can have a 100 times impact, great! Does that part of the problem represent a meaningful chunk of work for the day-to-day of your organization? If you make a 100 times impact on a problem that’s seen just a few hours a year, then the work is (probably) without use. If you can make a 1% improvement on something that hurts the team *every single day*, you’ll be a hero.

One way to estimate the value you provide is to think about the cost of the current state and the potential gain of the future state (when you’ve written your solution). How do you quantify the cost and improvement? Tying estimates down to money (as “time is money” and we all burn time) is a great way to figure out what kind of impact you’ll have and how to communicate that to colleagues. This is also a great way of prioritizing potential project options.

Once you’ve found useful and valuable problems to solve, you need to make sure you’re solving them in sensible ways. Taking a hard problem and deciding immediately to use a hard solution *might* be sensible, but starting with a simple solution and learning why it does and doesn’t work can quickly yield valuable insights that inform subsequent iterations of your solution. What’s the quickest and simplest way you can learn something useful?

Ian has worked with clients with near-release complex NLP pipelines but low confidence that they actually work. After a review, it was revealed that a team had built a complex system but missed the upstream poor-data-annotation problem that was confounding the NLP ML process. By switching to a far simpler solution (without deep neural networks, using old-fashioned NLP tooling), the issues were identified and the data consistently relabeled; only then could we build up toward more sophisticated solutions, now that upstream issues had sensibly been removed.

Is your team communicating its results clearly to stakeholders? Are you communicating clearly within your team? A lack of communication is an easy way to add a frustrating cost to your team’s progress.

Review your collaborative practices to check that processes such as frequent code reviews are in place. It is so easy to “save some time” by ignoring a code review, forgetting that you’re letting colleagues (and yourself) get away with unreviewed code that might be solving the wrong problem or that may contain errors that a fresh set of eyes could see before they have a worse and later impact.

The Remote Performant Programmer

Since the COVID-19 pandemic we've witnessed a switch to fully remote and hybrid practices. While some organizations have tried to bring teams back on-site, most have adopted hybrid or fully remote practices now that best practices are reasonably well understood.

Remote practices mean we can live anywhere and the hiring and collaborator pool can be far wider, whether limited to similar time zones or not limited at all. Some organizations have noticed that open source projects such as Python, pandas, scikit-learn, and more are working wonderfully successfully with a globally distributed team whose members rarely ever meet in person.

Increased communication is critical, and often a “documentation first” culture has to be developed. Some teams go so far as to say that “if it isn't documented on our chat tool (like Slack), then it never happened”—this means that every decision ends up being written down so it is communicated and can be searched for.

It is also easy to feel isolated when working fully remotely for a long time. Having regular check-ins with team members, even if you are not working on the same project, and unstructured time where you can talk at a higher level (or just about life!) is important in feeling connected and part of a team.

Some Thoughts on Good Notebook Practice

If you're using Jupyter Notebooks, you may have found that they're great for visual communication but they facilitate laziness. If you find yourself leaving long functions inside your Notebooks, be comfortable extracting them out to a Python module and then adding tests.

Practice prototyping your code in IPython or the QtConsole; turn lines of code into functions in a Notebook and then promote them out of the Notebook and into a module complemented by tests. Finally, consider wrapping the code in a class if encapsulation and data hiding are useful.

Liberally spread `assert` statements throughout a Notebook to check that your functions are behaving as expected. You can't easily test code inside a Notebook, and until you've refactored your functions into separate modules, `assert` checks are a simple way to add some level of validation. You shouldn't trust this code until you've extracted it to a module and written sensible unit tests.

Using `assert` statements to check data in your code should be frowned upon. It is an easy way to assert that certain conditions are being met, but it isn't idiomatic Python. To make your code easier to read by other developers, check your expected data state and then raise an appropriate exception if the check fails. A common exception would be `ValueError` if a function encounters an unexpected value. The **Pandera**

`library` is an example of a testing framework focused on Pandas and Polars to check that your data meets the specified constraints.

You may also want to add some sanity checks at the end of your Notebook—a mixture of logic checks and `raise` and `print` statements that demonstrate that you’ve just generated exactly what you needed. When you return to this code in six months, you’ll thank yourself for making it easy to see that it worked correctly all the way through!

One difficulty with Notebooks is sharing code with source control systems. `nbdime` is one of a growing set of new tools that let you diff your Notebooks. It is a lifesaver and enables collaboration with colleagues.

Your Work

Life can be complicated. In the 10 years since your authors wrote the first edition of this book, we’ve jointly experienced through friends and family a number of life situations, including new children, depression, cancer, home relocations, successful business exits and business failures, and career direction shifts. Inevitably, these external events will have an impact on one’s work and outlook on life.

Remember to keep looking for the joy in new activities. There are always interesting details or requirements once you start poking around. You might ask “Why did they make that decision?” and “How would I do it differently?” and all of a sudden you’re ready to start a conversation about how things might be changed or improved.

Keep a log of things that are worth celebrating. It is so easy to forget about accomplishments and to get caught up in the day-to-day. People get burned out because they’re always running to keep up, and they forget how much progress they’ve made.

We suggest that you build a list of items worth celebrating and note how you celebrate them. Ian keeps such a list—he’s happily surprised when he goes to update the list and sees just how many cool things have happened (and might otherwise have been forgotten!) in the last year. These shouldn’t just be work milestones; include hobbies and sports, and celebrate the milestones you’ve achieved. Micha makes sure to prioritize her personal life and spend days away from the computer to work on nontechnical projects or to prioritize rest, relaxation, and slowness. It is critical to keep developing your skill set, but it is not necessary to burn out!

Programming, particularly when performance focused, thrives on a sense of curiosity and a willingness to always delve deeper into the technical details. Unfortunately, this curiosity is the first thing to go when you burn out; so take your time and make sure you enjoy the journey, and keep the joy and the curiosity.

The Future of Python

At the time of publication, two interesting developments are occurring—the global interpreter lock *might* eventually be removed, and a just-in-time compiler *might* be added. Experimental work is underway, but it is not clear how these changes will land for future production releases of the Python interpreter.

Where Did the GIL Go?

As discussed in “[Computing Units](#)” on [page 2](#), the *global interpreter lock (GIL)* is the standard memory locking mechanism that can unfortunately make multithreaded code run—at worst—at single-thread speeds. The GIL’s job is to make sure that only one thread can modify a Python object at a time, so if multiple threads in one program try to modify the same object, they effectively each get to make their modifications one at a time.

This massively simplified the early design of Python, but as the processor count has increased, it has added a growing tax to writing multicore code. The GIL is a core part of Python’s reference-counting garbage collection machinery.

In 2023 a decision was made to investigate building a GIL-free version of Python that would still support threads in addition to the long-standing GIL build. Since third-party libraries (e.g., NumPy, Pandas, scikit-learn) have compiled C code that *relies* upon the current GIL implementation, some code gymnastics will be required for external libraries to support both builds of Python and to move to a GIL-less build in the longer term. Nobody wants a repeat of the 10-year Python 2 to Python 3 transition!

[Python Enhancement Proposal PEP 703](#) describes the proposal with a focus on scientific and AI applications. The main issue in this domain is that with CPU-intensive code and 10–100 threads, the overhead of the GIL can significantly reduce the parallelization opportunity. By switching to the standard solutions (e.g., multiprocessing) described in this book, a significant developer overhead and communications overhead can be introduced. None of these options enable the best use of the machine’s resources without significant effort.

This PEP notes the issues with non-atomic object modifications that need to be controlled for, along with a new small-object memory allocator that is thread-safe.

We might expect a GIL-less version of Python to be generally available from 2028—if no significant blockers are discovered during this journey.

Does Python Have a JIT?

Starting with Python 3.13, we expect that a just-in-time compiler (JIT) will be built into the main CPython that almost everyone uses.

This JIT follows a 2021 design called “copy and patch” that was first used in the Lua language. As a contrast, in technologies such as PyPy and Numba, an analyzer discovers slow code sections (aka hot spots) and then compiles a machine-code version that matches this code block with whatever specializations are available to the CPU on that machine. You get really fast code, but the compilation process can be expensive on the early passes.

The “copy and patch” process is a little different from the contrasting approach. When the python executable is built (normally by the Python Software Foundation), the LLVM compiler toolchain is used to build a set of predefined “stencils.” These stencils are semi-compiled versions of critical op-codes from the Python virtual machine. They’re called “stencils” because they have “holes” that are filled in later.

At runtime, when a hot spot is identified—typically a loop where the datatypes don’t change—you can take a set of stencils that match the op-codes and fill in the “holes” by pasting in the memory addresses of the relevant variables; then the op-codes will no longer need to be interpreted, as the machine code equivalent is available. This promises to be much faster than compiling each hot spot that’s identified; it may not be as optimal, but it’s hoped to provide significant gains without a slow analysis and compilation pass.

Getting to the point where a JIT is possible has taken a few evolutionary stages in major Python releases:

- 3.11 introduced an adaptive type specializing interpreter that provided 10–25% speedups.
- 3.12 introduced internal cleanups and a domain-specific language for the creation of the interpreter, enabling modification at build time.
- 3.13 introduced a hot spot detector to build on the specialized types with the copy-and-patch JIT.

It is worth noting that while the introduction of a JIT in Python 3.13 is a great step, it is unlikely to impact any of our Pandas, NumPy, and SciPy code, as internally these libraries often use C and Cython to precompile faster solutions. The JIT will have an impact on anyone writing native Python, particularly numeric Python.

Wrap-Up

We’ve looked at the underlying components of Python and its wider ecosystem, reflected on how to work with Python to develop scientific code efficiently, and touched on some of the current changes to the Python core language that may fundamentally improve its efficiency in the years ahead. Now let’s look at profiling to quickly identify where your bottlenecks occur, so you can focus your time in the correct place.

Profiling to Find Bottlenecks

Questions You'll Be Able to Answer After This Chapter

- How can I identify speed and RAM bottlenecks in my code?
- How do I profile CPU and memory usage?
- What depth of profiling should I use?
- How can I profile a long-running application?
- What's happening under the hood with CPython?
- How do I keep my code correct while tuning performance?

Profiling lets us find bottlenecks so we can do the least amount of work to get the biggest practical performance gain. While we'd like to get huge gains in speed and reductions in resource usage with little work, practically you'll aim for your code to run "fast enough" and "lean enough" to fit your needs. Profiling will let you make the most pragmatic decisions for the least overall effort.

Any measurable resource can be profiled (not just the CPU!). In this chapter we look at both CPU time and memory usage. You could apply similar techniques to measure network bandwidth and disk I/O too.

If a program is running too slowly or using too much RAM, you'll want to fix whichever parts of your code are responsible. You could, of course, skip profiling and fix what you *believe* might be the problem—but be wary, as you'll often end up "fixing" the wrong thing. Rather than using your intuition, it is far more sensible to first profile, having defined a hypothesis, before making changes to the structure of your code.

Sometimes it's good to be lazy. By profiling first, you can quickly identify the bottlenecks that need to be solved, and then you can solve just enough of these to achieve the performance you need. If you avoid profiling and jump to optimization, you'll quite likely do more work in the long run. Always be driven by the results of profiling.

Profiling Efficiently

The first aim of profiling is to test a representative system to identify what's slow (or using too much RAM, or causing too much disk I/O or network I/O). Profiling typically adds an overhead (10 to 100 times slowdowns can be typical), and you still want your code to be used in as similar to a real-world situation as possible. Extract a test case and isolate the piece of the system that you need to test. Preferably, it'll have been written to be in its own set of modules already.

The basic techniques that are introduced first in this chapter include the magic `%timeit` in IPython, `time.time()`, and a timing decorator. You can use these techniques to understand the behavior of statements and functions.

Then we will cover `cProfile` (“Using the `cProfile` Module” on page 41), showing you how to use this built-in tool to understand which functions in your code take the longest to run. This will give you a high-level view of the problem so you can direct your attention to the critical functions.

Next, we'll look at `line_profiler` (“Using `line_profiler` for Line-by-Line Measurements” on page 48), which will profile your chosen functions on a line-by-line basis. The result will include a count of the number of times each line is called and the percentage of time spent on each line. This is exactly the information you need to understand what's running slowly and why.

Armed with the results of `line_profiler`, you'll have the information you need to move on to using a compiler (Chapter 8)—if you tried compiling without having first profiled, it is quite possible you'd make a non-bottleneck code block faster, which ultimately wouldn't improve your overall speed.

In Chapter 6, you'll learn how to use `perf stat` to understand the number of instructions that are ultimately executed on a CPU and how efficiently the CPU's caches are utilized. This allows for advanced-level tuning of matrix operations. You should take a look at Example 6-8 when you're done with this chapter. This example is saved for later as it is specific to the use of `numpy` arrays.

After `line_profiler`, if you're working with long-running systems, then you'll be interested in using `py-spy` to peek into already-running Python processes.

To help you understand why your RAM usage is high, we'll show you `memory_profiler` ([“Using memory_profiler to Diagnose Memory Usage” on page 53](#)). It is particularly useful for tracking RAM usage over time on a labeled chart, so you can explain to colleagues why certain functions use more RAM than expected.

If you'd like to combine CPU and RAM profiling, you'll want to read about `Scalene` ([“Combining CPU and Memory Profiling with Scalene” on page 61](#)). It combines the jobs of `line_profiler` and `memory_profiler` with a novel low-impact memory allocator and also contains experimental GPU profiling support.

`VizTracer` ([“VizTracer for an Interactive Time-Based Call Stack” on page 65](#)) will let you see a time-based view on your code's execution; it presents a call stack down the page with time running from left to right. You can click into the call stack and even annotate custom messages and behavior.



Whatever approach you take to profiling your code, you must remember to have adequate unit test coverage in the code. Unit tests help you to avoid silly mistakes and to keep your results reproducible. Avoid them at your peril.

Always profile your code before compiling or rewriting your algorithms. You need evidence to determine the most efficient ways to make your code run faster.

Next, we'll give you an introduction to the Python bytecode inside CPython ([“Using the `dis` Module to Examine CPython Bytecode” on page 67](#)), so you can understand what's happening “under the hood.” In particular, having an understanding of how Python's stack-based virtual machine operates will help you understand why certain coding styles run more slowly than others. `Specialist` ([“Digging into Bytecode Specialization with `Specialist`” on page 69](#)) will then help us see which parts of the bytecode can be identified for performance improvements from Python 3.11 and above.

Before the end of the chapter, we'll review how to integrate unit tests while profiling ([“Unit Testing During Optimization to Maintain Correctness” on page 73](#)) to preserve the correctness of your code while you make it run more efficiently.



When using GenAI solutions such as GitHub Copilot, remember that *they'll always give you an answer*—and sometimes they might be right. When it comes to profiling, there are a million out-of-date examples in the training data that will inform the answers that a GenAI system might give you. Your authors have been confused, distracted, and occasionally amused by the suggestions from Copilot while working on this book. Always be skeptical.

We'll finish with a discussion of profiling strategies (“[Strategies to Profile Your Code Successfully](#)” on [page 77](#)) so you can reliably profile your code and gather the correct data to test your hypotheses. Here you'll learn how dynamic CPU frequency scaling and features like Turbo Boost can skew your profiling results, and you'll learn how they can be disabled.

To walk through all of these steps, we need an easy-to-analyze function. The next section introduces the Julia set. It is a CPU-bound function that's a little hungry for RAM; it also exhibits nonlinear behavior (so we can't easily predict the outcomes), which means we need to profile it at runtime rather than analyzing it offline.

Introducing the Julia Set

The [Julia set](#) is an interesting CPU-bound problem for us to begin with. It is a fractal sequence that generates a complex output image, named after Gaston Julia.

The code that follows is a little longer than a version you might write yourself. It has a CPU-bound component and a very explicit set of inputs. This configuration allows us to profile both the CPU usage and the RAM usage so we can understand which parts of our code are consuming two of our scarce computing resources. This implementation is *deliberately* suboptimal, so we can identify memory-consuming operations and slow statements. Later in this chapter we'll fix a slow logic statement and a memory-consuming statement, and in [Chapter 8](#) we'll significantly speed up the overall execution time of this function.

We will analyze a block of code that produces both a false grayscale plot ([Figure 2-1](#)) and a pure grayscale variant of the Julia set ([Figure 2-3](#)), at the complex point $c = -0.62772 - 0.42193j$ (try this line yourself—it is legitimate Python!). A Julia set is produced by calculating each pixel in isolation; this is an “embarrassingly parallel problem,” as no data is shared between points.

If we chose a different c , we'd get a different image. The location we have chosen has regions that are quick to calculate and others that are slow to calculate; this is useful for our analysis.

The problem is interesting because we calculate each pixel by applying a loop that could be applied an indeterminate number of times. On each iteration we test to see if this coordinate's value escapes toward infinity, or if it seems to be held by an attractor. Coordinates that cause few iterations are colored darkly in [Figure 2-1](#), and those that cause a high number of iterations are colored white. White regions are more complex to calculate and so take longer to generate.

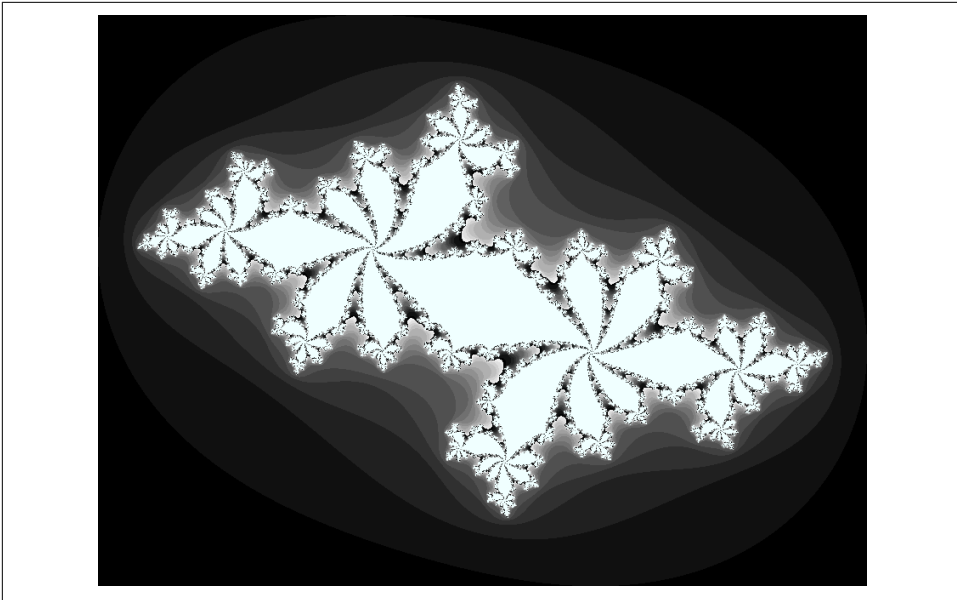


Figure 2-1. Julia set plot with a false grayscale to highlight detail

We define a set of z coordinates that we'll test. The function that we calculate squares the complex number z and adds c :

$$f(z) = z^2 + c$$

We iterate on this function while testing to see if the escape condition holds using `abs`. If the escape function is `False`, we break out of the loop and record the number of iterations we performed at this coordinate. If the escape function is never `False`, we stop after `maxiter` iterations. We will later turn this z 's result into a colored pixel representing this complex location.

In pseudocode, it might look like this:

```
for z in coordinates:
    for iteration in range(maxiter): # limited iterations per point
        if abs(z) < 2.0: # has the escape condition been broken?
            z = z*z + c
        else:
            break
    # store the iteration count for each z and draw later
```

To explain this function, let's try two coordinates.

We'll use the coordinate that we draw in the top-left corner of the plot at $-1.8-1.8j$. We must test `abs(z) < 2` before we can try the update rule:

```
z = -1.8-1.8j
print(abs(z))

2.54558441227
```

We can see that for the top-left coordinate, the `abs(z)` test will be `False` on the zeroth iteration as $2.54 \geq 2.0$, so we do not perform the update rule. The output value for this coordinate is 0.

Now let's jump to the center of the plot at $z = 0+0j$ and try a few iterations:

```
c = -0.62772-0.42193j
z = 0+0j
for n in range(9):
    z = z*z + c
    print(f"{n}: z={z: .5f}, abs(z)={abs(z):0.3f}, c={c: .5f}")

0: z=-0.62772-0.42193j, abs(z)=0.756, c=-0.62772-0.42193j
1: z=-0.41171+0.10778j, abs(z)=0.426, c=-0.62772-0.42193j
2: z=-0.46983-0.51068j, abs(z)=0.694, c=-0.62772-0.42193j
3: z=-0.66777+0.05793j, abs(z)=0.670, c=-0.62772-0.42193j
4: z=-0.18516-0.49930j, abs(z)=0.533, c=-0.62772-0.42193j
5: z=-0.84274-0.23703j, abs(z)=0.875, c=-0.62772-0.42193j
6: z= 0.02630-0.02242j, abs(z)=0.035, c=-0.62772-0.42193j
7: z=-0.62753-0.42311j, abs(z)=0.757, c=-0.62772-0.42193j
8: z=-0.41295+0.10910j, abs(z)=0.427, c=-0.62772-0.42193j
```

We can see that each update to `z` for these first iterations leaves it with a value where `abs(z) < 2` is `True`. For this coordinate we can iterate 300 times and still the test will be `True`. We cannot tell how many iterations we must perform before the condition becomes `False`, and this may be an infinite sequence. The maximum iteration (`maxiter`) break clause will stop us from iterating potentially forever.

In [Figure 2-2](#), we see the first 50 iterations of the preceding sequence. For $0+0j$ (the solid line with circle markers), the sequence appears to repeat every eighth iteration, but each sequence of seven calculations has a minor deviation from the previous sequence—we can't tell if this point will iterate forever within the boundary condition, or for a long time, or maybe for just a few more iterations. The dashed cutoff line shows the boundary at $+2$.

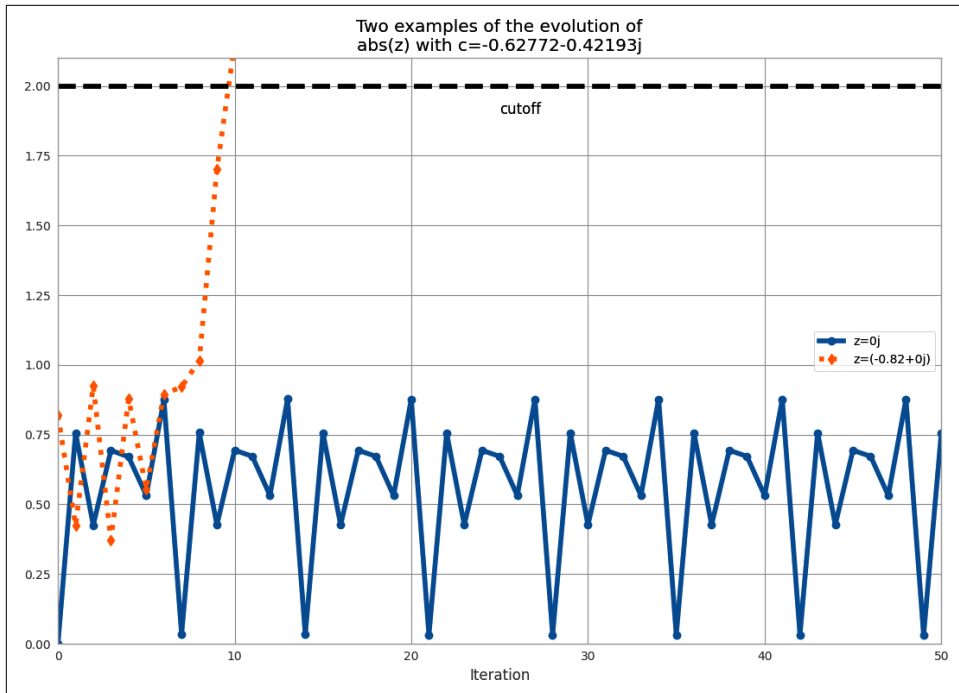


Figure 2-2. Two coordinate examples evolving for the Julia set

For $-0.82+0j$ (the dashed line with diamond markers), we can see that after the ninth update, the absolute result has exceeded the +2 cutoff, so we stop updating this value.

Calculating the Full Julia Set

In this section we break down the code that generates the Julia set. We'll analyze it in various ways throughout this chapter. As shown in [Example 2-1](#), at the start of our module we import the `time` module for our first profiling approach and define some coordinate constants.

Example 2-1. Defining global constants for the coordinate space

```
"""Julia set generator without optional PIL-based image drawing"""
import time

# area of complex space to investigate
x1, x2, y1, y2 = -1.8, 1.8, -1.8, 1.8
c_real, c_imag = -0.62772, -.42193
```

To generate the plot, we create two lists of input data. The first is `zs` (complex z coordinates), and the second is `cs` (a complex initial condition). Neither list varies,

and we could optimize `cs` to a single `c` value as a constant. The rationale for building two input lists is that we want to have some reasonable-looking data to profile when we profile RAM usage later in this chapter.

To build the `zs` and `cs` lists, we need to know the coordinates for each `z`. In [Example 2-2](#), we build up these coordinates using `xcoord` and `ycoord` and a specified `x_step` and `y_step`. The somewhat verbose nature of this setup is useful when porting the code to other tools (such as `numpy`) and to other Python environments, as it helps to have everything *very* clearly defined for debugging.

Example 2-2. Establishing the coordinate lists as inputs to our calculation function

```
def calc_pure_python(desired_width, max_iterations):
    """Create a list of complex coordinates (zs) and complex parameters (cs),
    build Julia set"""
    x_step = (x2 - x1) / desired_width
    y_step = (y1 - y2) / desired_width
    x = []
    y = []
    ycoord = y2
    while ycoord > y1:
        y.append(ycoord)
        ycoord += y_step
    xcoord = x1
    while xcoord < x2:
        x.append(xcoord)
        xcoord += x_step
    # build a list of coordinates and the initial condition for each cell.
    # Note that our initial condition is a constant and could easily be removed,
    # we use it to simulate a real-world scenario with several inputs to our
    # function
    zs = []
    cs = []
    for ycoord in y:
        for xcoord in x:
            zs.append(complex(xcoord, ycoord))
            cs.append(complex(c_real, c_imag))

    print("Length of x:", len(x))
    print("Total elements:", len(zs))
    start_time = time.time()
    output = calculate_z_serial_purepython(max_iterations, zs, cs)
    end_time = time.time()
    secs = end_time - start_time
    print(f"{calculate_z_serial_purepython.__name__} took {secs:0.2f} seconds")

    # This sum is expected for a 1000^2 grid with 300 iterations
    # It ensures that our code evolves exactly as we'd intended
    assert sum(output) == 33219980
```

Having built the `zs` and `cs` lists, we output some information about the size of the lists and calculate the output list via `calculate_z_serial_purepython`. Finally, we sum the contents of `output` and assert that it matches the expected output value. Ian uses it here to confirm that no errors creep into the book.

As the code is deterministic, we can verify that the function works as we expect by summing all the calculated values. This is useful as a sanity check—when we make changes to numerical code, it is *very* sensible to check that we haven't broken the algorithm. Ideally, we would use unit tests and test more than one configuration of the problem.

Next, in [Example 2-3](#), we define the `calculate_z_serial_purepython` function, which expands on the algorithm we discussed earlier. Notably, we also define an output list at the start that has the same length as the input `zs` and `cs` lists.

Example 2-3. Our CPU-bound calculation function

```
def calculate_z_serial_purepython(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while abs(z) < 2 and n < maxiter:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

Now we call the calculation routine in [Example 2-4](#). By wrapping it in a `__main__` check, we can safely import the module without starting the calculations for some of the profiling methods. Here, we're not showing the method used to plot the output.

Example 2-4. `__main__` for our code

```
if __name__ == "__main__":
    # Calculate the Julia set using a pure Python solution with
    # reasonable defaults for a laptop
    calc_pure_python(desired_width=1000, max_iterations=300)
```

Once we run the code, we see some output about the complexity of the problem:

```
# running the above produces:
Length of x: 1,000
Total elements: 1,000,000
calculate_z_serial_purepython took 5.80 seconds
```

In the false-grayscale plot (Figure 2-1), the high-contrast color changes gave us an idea of where the cost of the function was slow changing or fast changing. Here, in Figure 2-3, we have a linear color map: black is quick to calculate, and white is expensive to calculate.

By showing two representations of the same data, we can see that lots of detail is lost in the linear mapping. Sometimes it can be useful to have various representations in mind when investigating the cost of a function.

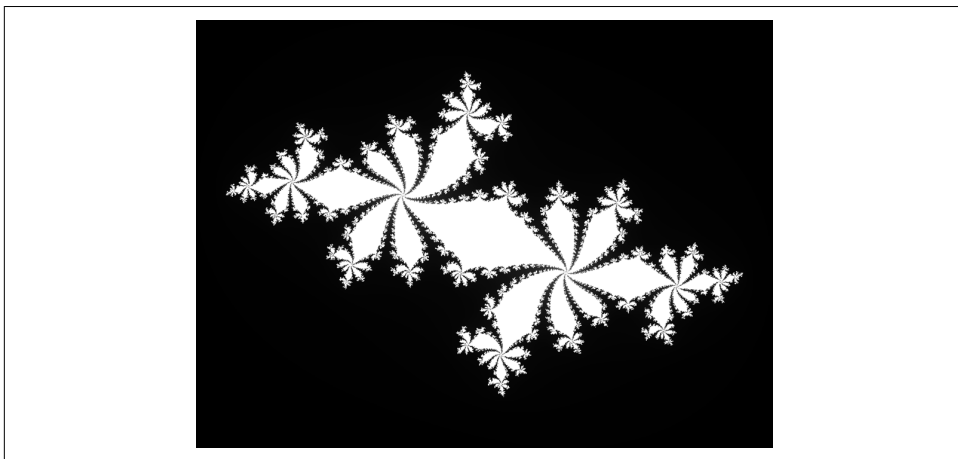


Figure 2-3. Julia plot example using a pure grayscale

Simple Approaches to Timing—print and a Decorator

After Example 2-4, we saw the output generated by several print statements in our code. On Ian's laptop, this code takes approximately 5 seconds to run using CPython 3.12. It is useful to note that execution time always varies. You must observe the normal variation when you're timing your code, or you might incorrectly attribute an improvement in your code to what is simply a random variation in execution time.

Your computer will be performing other tasks while running your code, such as accessing the network, disk, or RAM, and these factors can cause variations in the execution time of your program.

Ian's laptop is a Dell XPS 15 9510 with an Intel Core I7-11800H (2.3 GHz, 24 MB Level 3 cache, eight physical cores with Hyper-Threading) with 64 GB system RAM running Linux Mincx 21.2 (based on Ubuntu 22.04).

In `calc_pure_python` (Example 2-2), we can see several print statements. This is the simplest way to measure the execution time of a piece of code *inside* a function. It is a basic approach, but despite being quick and dirty, it can be very useful when you're first looking at a piece of code.

Using `print` statements is commonplace when debugging and profiling code. It quickly becomes unmanageable but is useful for short investigations. Try to tidy up the `print` statements when you're done with them, or they will clutter your `stdout`.

A slightly cleaner approach is to use a *decorator*—here, we add one line of code above the function that we care about. Our decorator can be very simple and just replicate the effect of the `print` statements. Later, we can make it more advanced.

In [Example 2-5](#), we define a new function, `timefn`, which takes a function as an argument: the inner function, `measure_time`, takes `*args` (a variable number of positional arguments) and `**kwargs` (a variable number of key/value arguments) and passes them through to `fn` for execution.

Around the execution of `fn`, we capture `time.time()` and then print the result along with `fn.__name__`. The overhead of using this decorator is small, but if you're calling `fn` millions of times, the overhead might become noticeable. We use `@wraps(fn)` to expose the function name and docstring to the caller of the decorated function (otherwise, we would see the function name and docstring for the decorator, not the function it decorates).

Example 2-5. Defining a decorator to automate timing measurements

```
from functools import wraps

def timefn(fn):
    @wraps(fn)
    def measure_time(*args, **kwargs):
        t1 = time.time()
        result = fn(*args, **kwargs)
        t2 = time.time()
        print(f"@timefn: {fn.__name__} took {(t2 - t1):0.2f} seconds")
        return result
    return measure_time

@timefn
def calculate_z_serial_purepython(maxiter, zs, cs):
    ...
```

When we run this version (we keep the `print` statements from before), we can see that the execution time in the decorated version is ever so slightly quicker than the call from `calc_pure_python`. This is due to the overhead of calling a function (the difference is very tiny):

```
Length of x: 1,000
Total elements: 1,000,000
@timefn: calculate_z_serial_purepython took 5.78 seconds
calculate_z_serial_purepython took 5.78 seconds
```



The addition of profiling information will inevitably slow down your code—some profiling options are very informative and induce a heavy speed penalty. The trade-off between profiling detail and speed will be something you have to consider.

We can use the `timeit` module as another way to get a coarse measurement of the execution speed of our CPU-bound function. More typically, you would use this when timing different types of simple expressions as you experiment with ways to solve a problem.



The `timeit` module temporarily disables the garbage collector. This might impact the speed you'll see with real-world operations if the garbage collector would normally be invoked by your operations. See the [Python documentation](#) for help on this.

From the command line, you can run `timeit` as follows:

```
python -m timeit -n 5 -r 1 -s "import julia1_nopil" \  
"julia1_nopil.calc_pure_python(desired_width=1000, max_iterations=300)"
```

Note that you have to import the module as a setup step using `-s`, as `calc_pure_python` is inside that module. `timeit` has some sensible defaults for short sections of code, but for longer-running functions it can be sensible to specify the number of loops (`-n 5`) and the number of repetitions (`-r 5`) to repeat the experiments. The best result of all the repetitions is given as the answer. Adding the verbose flag (`-v`) shows the cumulative time of all the loops by each repetition, which can help your variability in the results.

By default, if we run `timeit` on this function without specifying `-n` and `-r`, it runs 10 loops with 5 repetitions, and this takes six minutes to complete. Overriding the defaults can make sense if you want to get your results a little faster.

We're interested only in the best-case results, as other results will probably have been impacted by other processes:

```
Length of x: 1,000  
Total elements: 1,000,000  
calculate_z_serial_purepython took 5.78 seconds  
...  
5 loops, best of 1: 6.1 sec per loop
```

Try running the benchmark several times to check if you get varying results—you may need more repetitions to settle on a stable fastest-result time. There is no “correct” configuration, so if you see a wide variation in your timing results, do more repetitions until your final result is stable.

Our results show that the overall cost of calling `calc_pure_python` is 6.1 seconds (as the best case), while single calls to `calc_pure_python` take approximately 5.8 seconds as measured by the `@timefn` decorator. The difference is mainly the time taken to create the `zs` and `cs` lists before `start_time` is recorded.

Inside IPython, we can use the magic `%timeit` in the same way. If you are developing your code interactively in IPython or in a Jupyter Notebook, you can use this:

```
In [1]: import julia1_nopil
In [2]: %timeit julia1_nopil.calc_pure_python(desired_width=1000,
                                              max_iterations=300)
```



Be aware that “best” is calculated differently by the `timeit.py` approach and the `%timeit` approach in Jupyter and IPython. `timeit.py` uses the minimum value seen. IPython in 2016 switched to using the mean and standard deviation. Both methods have their flaws, but generally they’re both “reasonably good”; you can’t compare between them, though. Use one method or the other; don’t mix them.

It is worth considering the variation in load that you get on a normal computer. Many background tasks are running (e.g., Dropbox, backups) that could impact the CPU and disk resources at random. Scripts in web pages can also cause unpredictable resource usage. [Figure 2-4](#) shows the single CPU being used at 100% for some of the timing steps we just performed; the other cores on this machine are each lightly working on other tasks.

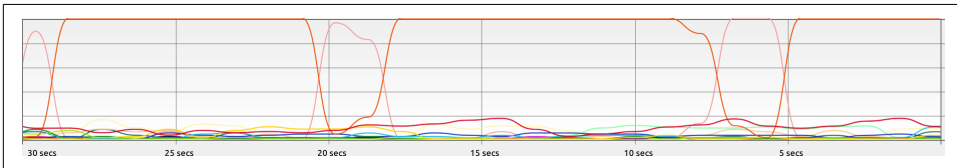


Figure 2-4. System Monitor on Ubuntu showing variation in background CPU usage while we time our function

Occasionally, the System Monitor shows spikes of activity on this machine. It is sensible to watch your System Monitor to check that nothing else is interfering with your critical resources (CPU, disk, network).

Simple Timing Using the Unix `time` Command

We can step outside of Python for a moment to use a standard system utility on Unix-like systems. The following will record various views on the execution time of your program:

```
$ /usr/bin/time -p python julia1_nopil.py
Length of x: 1,000
Total elements: 1,000,000
calculate_z_serial_purepython took 5.71 seconds
real 6.02
user 5.96
sys 0.05
```

Note that we specifically use `/usr/bin/time` rather than `time` so we get the system's `time` and not the simpler (and less useful) version built into our shell. If you try `time --verbose` and you get an error, you're probably looking at the shell's built-in `time` command and not the system command.

Using the `-p` portability flag, we get three results:

- `real` records the wall clock or elapsed time.
- `user` records the amount of time the CPU spent on your task outside of kernel functions.
- `sys` records the time spent in kernel-level functions.

By adding `user` and `sys`, you get a sense of how much time was spent in the CPU. The difference between this and `real` might tell you about the amount of time spent waiting for I/O; it might also suggest that your system is busy running other tasks that are distorting your measurements.

`time` is useful because it isn't specific to Python. It includes the time taken to start the python executable, which might be significant if you start lots of fresh processes (rather than having a long-running single process). If you often have short-running scripts where the startup time is a significant part of the overall runtime, then `time` can be a more useful measure.

We can add the `--verbose` flag to get even more output:

```
Length of x: 1,000
Total elements: 1,000,000
calculate_z_serial_purepython took 5.76 seconds
  Command being timed: "python julia1_nopil.py"
    User time (seconds): 6.01
    System time (seconds): 0.05
  Percent of CPU this job got: 99%
  Elapsed (wall clock) time (h:mm:ss or m:ss): 0:06.07
  Average shared text size (kbytes): 0
  Average unshared data size (kbytes): 0
  Average stack size (kbytes): 0
  Average total size (kbytes): 0
  Maximum resident set size (kbytes): 98432
  Average resident set size (kbytes): 0
  Major (requiring I/O) page faults: 0
```



```
Minor (reclaiming a frame) page faults: 23334
Voluntary context switches: 1
Involuntary context switches: 37
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

One useful indicator is `Maximum resident set size`, which indicates the maximum amount of RAM used during execution. If it nears the physical RAM you have available, you'll be close to either running out of RAM or using disk-swap, which is very slow. This execution cost 98 MB at its worst.

Another useful indicator here is `Major (requiring I/O) page faults`. This indicates whether the operating system is having to load pages of data from the disk because the data no longer resides in RAM, which will cause a speed penalty; here it doesn't, as it records zero page faults.

In our example, the code and data requirements are small, so no page faults occur. If you have a memory-bound process, or several programs that use variable and large amounts of RAM, you might find that this gives you a clue as to which program is being slowed down by disk accesses at the operating system level because parts of it have been swapped out of RAM to disk.

Using the cProfile Module

`cProfile` is a built-in profiling tool in the standard library. It hooks into the virtual machine in CPython to measure the time taken to run every function that it sees. This introduces a greater overhead, but you get correspondingly more information. Sometimes the additional information can lead to surprising insights into your code.

`cProfile` is one of two profilers in the standard library, alongside `profile`. `profile` is the original and slower pure Python profiler. `cProfile` has the same interface as `profile` and is written in C for a lower overhead. If you're curious about the history of these libraries, see [Armin Rigo's 2005 request](#) to include `cProfile` in the standard library.

A good practice when profiling is to generate a *hypothesis* about the speed of parts of your code before you profile it. Ian likes to print out the code snippet in question and annotate it. Forming a hypothesis ahead of time means you can measure how wrong you are (and you will be!) and improve your intuition about certain coding styles.



You should never avoid profiling in favor of a gut instinct (we warn you—you *will* get it wrong!). It is definitely worth forming a hypothesis ahead of profiling to help you learn to spot possible slow choices in your code, and you should always back up your choices with evidence.

Always be driven by results that you have measured, and always start with some quick-and-dirty profiling to make sure you're addressing the right area. There's nothing more humbling than cleverly optimizing a section of code only to realize (hours or days later) that you missed the slowest part of the process and haven't really addressed the underlying problem at all.

Let's hypothesize that `calculate_z_serial_purepython` is the slowest part of the code. In that function, we do a lot of dereferencing and make many calls to basic arithmetic operators and the `abs` function. These will probably show up as consumers of CPU resources.

Here, we'll use the `cProfile` module to run a variant of the code. The output is spartan but helps us figure out where to analyze further.

The `-s cumulative` flag tells `cProfile` to sort by cumulative time spent inside each function; this gives us a view into the slowest parts of a section of code. The `cProfile` output is written to screen directly after our usual `print` results:

```
$ python -m cProfile -s cumulative julia1_nopil.py
Length of x: 1,000
Total elements: 1,000,000
calculate_z_serial_purepython took 13.15 seconds
36221995 function calls in 14.301 seconds

Ordered by: cumulative time
```

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	14.301	14.301	{built-in method builtins.exec}
1	0.035	0.035	14.301	14.301	julia1_nopil.py:1(<module>)
1	0.803	0.803	14.267	14.267	julia1_nopil.py:23 (calc_pure_python)
1	8.420	8.420	13.150	13.150	julia1_nopil.py:9 (calculate_z_serial_purepython)
34219980	4.730	0.000	4.730	0.000	{built-in method builtins.abs}
2002000	0.306	0.000	0.306	0.000	{method 'append' of 'list' objects}
1	0.007	0.007	0.007	0.007	{built-in method builtins.sum}
3	0.000	0.000	0.000	0.000	{built-in method builtins.print}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}
2	0.000	0.000	0.000	0.000	{built-in method time.time}
4	0.000	0.000	0.000	0.000	{built-in method builtins.len}

Sorting by cumulative time gives us an idea about where the majority of execution time is spent. This result shows us that 36,221,995 function calls occurred in just over 13 seconds (this time includes the overhead of using cProfile). Previously, our code took around 5 seconds to execute—we’ve just added an 8-second penalty by measuring how long each function takes to execute.

We can see that the entry point to the code `julia1_nopil.py` on line 1 takes a total of 14 seconds. This is the entry point to the module, which starts by executing the `import` statements. `ncalls` is 1, indicating that this line is executed only once.

Inside `calc_pure_python`, the call to `calculate_z_serial_purepython` consumes 13 seconds. Both functions are called only once. We can derive that approximately 1 second is spent on lines of code inside `calc_pure_python`, separate to calling the CPU-intensive `calculate_z_serial_purepython` function. However, we can’t derive *which* lines take the time inside the function using cProfile.

Inside `calculate_z_serial_purepython`, the time spent on lines of code (without calling other functions) is 8 seconds. This function makes 34,219,980 calls to `abs`, which take a total of 4 seconds, along with other calls that do not cost much time.

What about the `{abs}` call? This line is measuring the individual calls to the `abs` function inside `calculate_z_serial_purepython`. While the per-call cost is negligible (it is recorded as 0.000 seconds), the total time for 34,219,980 calls is 4 seconds. We couldn’t predict in advance exactly how many calls would be made to `abs`, as the Julia function has unpredictable dynamics (that’s why it is so interesting to look at).

At best we could have said that it will be called a minimum of 1 million times, as we’re calculating 1000×1000 pixels. At most it will be called 300 million times, as we calculate 1,000,000 pixels with a maximum of 300 iterations. So 34 million calls is roughly 10% of the worst case.

If we look at the original grayscale image ([Figure 2-3](#)) and, in our mind’s eye, squash the white parts together and into a corner, we can estimate that the expensive white region accounts for roughly 10% of the rest of the image.

The next line in the profiled output, `{method 'append' of 'list' objects}`, details the creation of 2,002,000 list items.

Here’s a question to ask yourself: why 2,002,000 items? Before you read on, think about how many list items are being constructed.

This creation of 2,002,000 items is occurring in `calc_pure_python` during the setup phase.

The `zs` and `cs` lists will be 1000×1000 items each (generating $1,000,000 \times 2$ calls), and these are built from a list of 1,000 x and 1,000 y coordinates. In total, this is 2,002,000 calls to `append`.

It is important to note that this cProfile output is not ordered by parent functions; it is summarizing the expense of all functions in the executed block of code. Figuring out what is happening on a line-by-line basis is very hard with cProfile, as we get profile information only for the function calls themselves, not for each line within the functions.

Inside `calculate_z_serial_purepython`, we can account for `{abs}`, and in total this function costs approximately 4.7 seconds. We know that `calculate_z_serial_purepython` costs 13.1 seconds in total.

Near the end the profiling output refers to `lsprof`; this is the original name of the tool that evolved into cProfile and can be ignored.

To get more control over the results of cProfile, we can write a statistics file and then analyze it in Python:

```
$ python -m cProfile -o profile.stats julia1_nopil.py
```

We can load this into Python as follows, and it will give us the same cumulative time report as before:

```
In [1]: import pstats
In [2]: p = pstats.Stats("profile.stats")
In [3]: p.sort_stats("cumulative")
Out[3]: <pstats.Stats at 0x7fe8747e8470>
```

```
In [4]: p.print_stats()
Thu Feb 22 20:38:55 2024    profile.stats
```

```
36221995 function calls in 14.398 seconds
```

```
Ordered by: cumulative time
```

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	14.398	14.398	{built-in method builtins.exec}
1	0.036	0.036	14.398	14.398	julia1_nopil.py:1(<module>)
1	0.799	0.799	14.363	14.363	julia1_nopil.py:23 (calc_pure_python)
1	8.453	8.453	13.252	13.252	julia1_nopil.py:9 (calculate_z_serial_purepython)
34219980	4.799	0.000	4.799	0.000	{built-in method builtins.abs}
2002000	0.304	0.000	0.304	0.000	{method 'append' of 'list' objects}
1	0.008	0.008	0.008	0.008	{built-in method builtins.sum}
3	0.000	0.000	0.000	0.000	{built-in method builtins.print}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}
2	0.000	0.000	0.000	0.000	{built-in method time.time}
4	0.000	0.000	0.000	0.000	{built-in method builtins.len}

To trace which functions we're profiling, we can print the caller information. In the following two listings we can see that `calculate_z_serial_purepython` is the most expensive function, and it is called from one place. If it were called from many places, these listings might help us narrow down the locations of the most expensive parents:

```
In [5]: p.print_callers()
        Ordered by: cumulative time
```

Function	was called by...	ncalls	totttime	cumtime
{built-in method builtins.exec}	<-			
julia1_nopil.py:1(<module>)	<-	1	0.036	14.398
julia1_nopil.py:23(calc_pure_python)	<-	1	0.799	14.363
julia1_nopil.py:9(...)	<-	1	8.453	13.252
{built-in method builtins.abs}	<-	34219980	4.799	4.799
{method 'append' of 'list' objects}	<-	2002000	0.304	0.304
{built-in method builtins.sum}	<-	1	0.008	0.008
{built-in method builtins.print}	<-	3	0.000	0.000
{built-in method time.time}	<-	2	0.000	0.000
{built-in method builtins.len}	<-	2	0.000	0.000

We can flip this around the other way to show which functions call other functions:

```
In [6]: p.print_callees()
        Ordered by: cumulative time
```

Function	called...	ncalls	totttime	cumtime
{built-in method builtins.exec}	->	1	0.036	14.398
julia1_nopil.py:1(<module>)	->	1	0.799	14.363
julia1_nopil.py:23(calc_pure_python)	->	1	8.453	13.252
julia1_nopil.py:9(...)	->	2	0.000	0.000
{built-in method builtins.len}	->	3	0.000	0.000
{built-in method builtins.print}	->	1	0.008	0.008
{built-in method builtins.sum}	->	2	0.000	0.000
{built-in method time.time}	->	2002000	0.304	0.304

```
julia1_nopil.py:9(...)
{method 'append' of 'list' objects}
-> 34219980 4.799 4.799
{built-in method builtins.abs}
2 0.000 0.000
{built-in method builtins.len}

{built-in method builtins.abs} ->
{method 'append' of 'list' objects} ->
{built-in method builtins.sum} ->
{built-in method builtins.print} ->
{built-in method time.time} ->
{built-in method builtins.len} ->
```

cProfile is rather verbose, and you need a side screen to see it without lots of word wrapping. Since it is built in, though, it is a convenient tool for quickly identifying bottlenecks. Tools like `line_profiler` and `memory_profiler`, which we discuss later in this chapter, will then help you to drill down to the specific lines that you should pay attention to.

Visualizing cProfile Output with SnakeViz

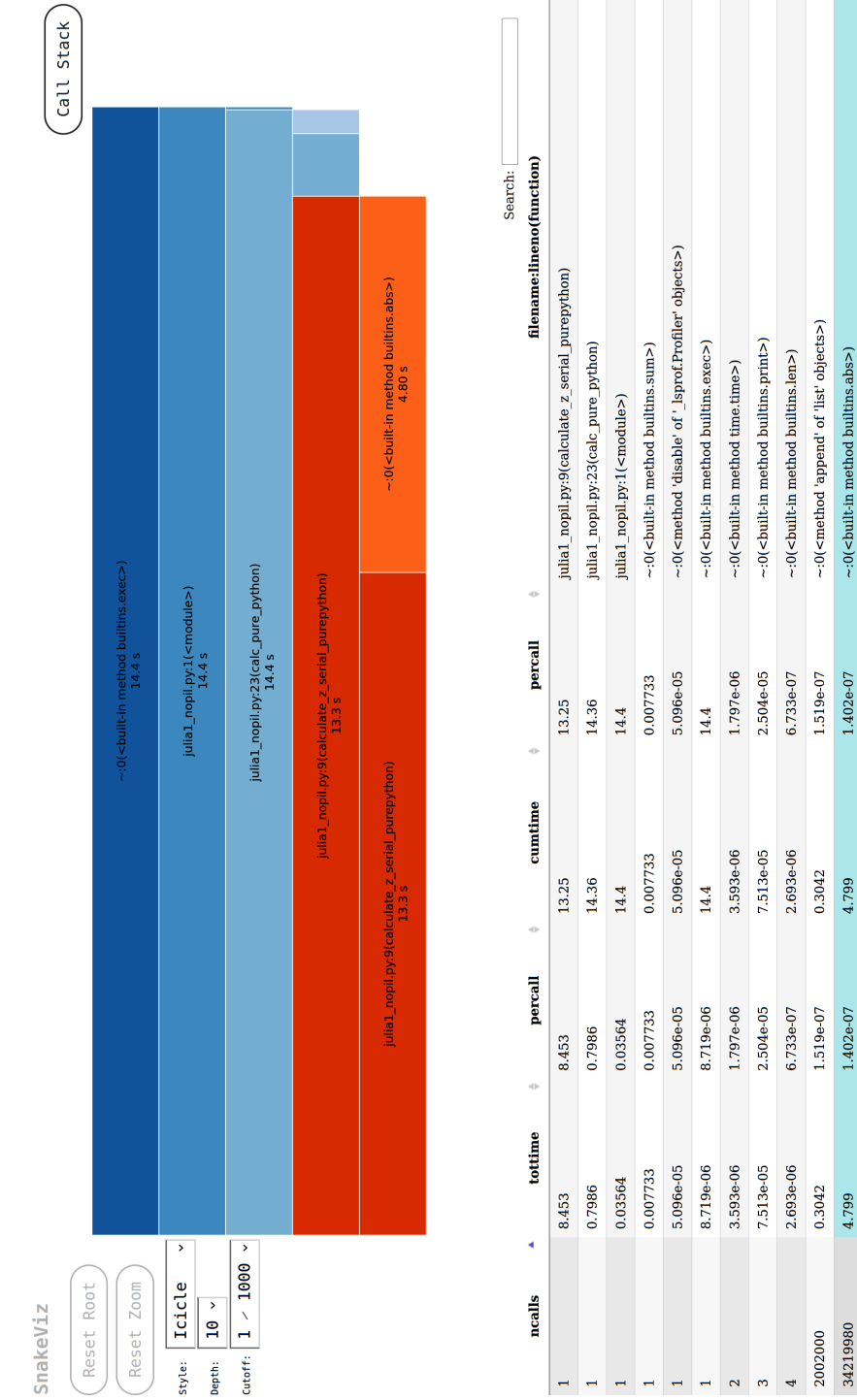
SnakeViz is a visualizer that draws the output of cProfile as a diagram in which larger boxes are areas of code that take longer to run. It replaces the older `runsnake` tool.

Use SnakeViz to get a high-level understanding of a cProfile statistics file, particularly if you're investigating a new project for which you have little intuition. The diagram will help you visualize the CPU-usage behavior of the system, and it may highlight areas that you hadn't expected to be expensive.

To install SnakeViz, use `pip install snakeviz`.

In [Figure 2-5](#) we have the visual output of the `profile.stats` file we've just generated. The entry point for the program is shown at the top of the diagram. Each layer down is a function called from the function above.

The width of the diagram represents the entire time taken by the program's execution. The fourth layer shows that the majority of the time is spent in `calculate_z_serial_purepython`. The fifth layer breaks this down some more—the unannotated block to the right occupying approximately 33% of that layer represents the time spent in the `abs` function. Seeing these larger blocks quickly brings home how the time is spent inside your program.



Below the graphical bars we have a table that is a pretty-printed version of the statistics we’ve just been looking at, which you can sort by `cumtime` (cumulative time), `percall` (cost per call), or `ncalls` (number of calls altogether), among other categories. Starting with `cumtime` will tell you which functions cost the most overall. They’re a pretty good place to start your investigations.

If you’re comfortable looking at tables, the console output for `cProfile` may be adequate for you. To communicate to others, we strongly suggest you use diagrams—such as this output from `SnakeViz`—to help others quickly understand the point you’re making.

Using `line_profiler` for Line-by-Line Measurements

In Ian’s opinion, Robert Kern’s `line_profiler` is the strongest tool for identifying the cause of CPU-bound problems in Python code. It works by profiling individual functions on a line-by-line basis, so you should start with `cProfile` and use the high-level view to guide which functions to profile with `line_profiler`.

It is worthwhile printing and annotating versions of the output from this tool as you modify your code, so you have a record of changes (successful or not) that you can quickly refer to. Don’t rely on your memory when you’re working on line-by-line changes.

To install `line_profiler`, issue the command `pip install line_profiler`.

A decorator (`@profile`) is used to mark the chosen function. The `kernprof` script is used to execute your code, and the CPU time and other statistics for each line of the chosen function are recorded.

The arguments are `-l` for line-by-line (rather than function-level) profiling and `-v` for verbose output. Without `-v`, you receive an `.lprof` output that you can later analyze with the `line_profiler` module. In [Example 2-6](#), we’ll do a full run on our CPU-bound function.

Example 2-6. Running `kernprof` with line-by-line output on a decorated function to record the CPU cost of each line’s execution

```
$ kernprof -l -v julia1_lineprofiler.py
...
Wrote profile results to julia1_lineprofiler.py.lprof
Timer unit: 1e-06 s

Total time: 31.0996 s
File: julia1_lineprofiler.py
Function: calculate_z_serial_purepython at line 16
```


Line #	Hits	Per Hit	% Time	Line Contents
=====				
16				@profile
17				def calculate_z_serial_purepython(maxiter,
				zs, cs):
18				"""Calculate output list using
				Julia update rule"""
19	1	3720.4	0.0	output = [0] * len(zs)
20	1000001	0.3	0.8	for i in range(len(zs)):
21	1000000	0.2	0.6	n = 0
22	1000000	0.2	0.7	z = zs[i]
23	1000000	0.2	0.7	c = cs[i]
24	34219980	0.4	44.7	while abs(z) < 2 and n < maxiter:
25	33219980	0.3	29.2	z = z * z + c
26	33219980	0.2	22.5	n += 1
27	1000000	0.2	0.7	output[i] = n
28	1	3.6	0.0	return output

Introducing kernprof.py adds a substantial amount to the runtime. In this example, calculate_z_serial_purepython takes 31 seconds; this is up from 5 seconds using simple print statements and 13 seconds using cProfile. The gain is that we get a line-by-line breakdown of where the time is spent inside the function.

The % Time column is the most helpful—we can see that 44% of the time is spent on the while testing. We don't know whether the first statement (`abs(z) < 2`) is more expensive than the second (`n < maxiter`), though. Inside the loop, we see that the update to `z` is also fairly expensive. Even `n += 1` is expensive! Python's dynamic lookup machinery is at work for every loop, even though we're using the same types for each variable in each loop—this is where compiling and type specialization ([Chapter 8](#)) give us a massive win. The creation of the output list and the updates on line 19 are relatively cheap compared to the cost of the while loop.

If you haven't thought about the complexity of Python's dynamic machinery before, do think about what happens in that `n += 1` operation. Python has to check that the `n` object has an `__add__` function (and if it didn't, it'd walk up any inherited classes to see if they provided this functionality), and then the other object (`1` in this case) is passed in so that the `__add__` function can decide how to handle the operation. Remember that the second argument might be a float or other object that may or may not be compatible. This all happens dynamically.

The obvious way to further analyze the while statement is to break it up. While there has been some discussion in the Python community around the idea of rewriting the .pyc files with more detailed information for multipart, single-line statements, we are unaware of any production tools that offer a more fine-grained analysis than `line_profiler`.

In [Example 2-7](#), we break the while logic into several statements. This additional complexity will increase the runtime of the function, as we have more lines of code to execute, but it *might* also help us understand the costs incurred in this part of the code.



Before you look at the code, do you think we'll learn about the costs of the fundamental operations this way? Might other factors complicate the analysis?

Example 2-7. Breaking the compound while statement into individual statements to record the cost of each part of the original statement

```
$ kernprof -l -v julia1_lineprofiler2.py
...
Wrote profile results to julia1_lineprofiler2.py.lprof
Timer unit: 1e-06 s

Total time: 63.3558 s
File: julia1_lineprofiler2.py
Function: calculate_z_serial_purepython at line 15
```

Line #	Hits	Per Hit	% Time	Line Contents
15				@profile
16				def calculate_z_serial_purepython(maxiter, zs, cs):
17				"""Calculate output list using Julia update rule"""
18	1	3862.9	0.0	output = [0] * len(zs)
19	1000001	0.3	0.5	for i in range(len(zs)):
20	1000000	0.3	0.4	n = 0
21	1000000	0.3	0.5	z = zs[i]
22	1000000	0.3	0.4	c = cs[i]
23	34219980	0.2	12.3	while True:
24	34219980	0.4	21.4	not_yet_escaped = abs(z) < 2
25	34219980	0.3	14.8	iterations_left = n < maxiter
26	34219980	0.3	18.6	if not_yet_escaped and iterations_left:
27	33219980	0.3	15.7	z = z * z + c
28	33219980	0.3	14.6	n += 1
29				else:
30	1000000	0.2	0.4	break
31	1000000	0.3	0.4	output[i] = n
32	1	3.0	0.0	return output

This version takes 63 seconds to execute, while the previous version took 31 seconds. Other factors *did* complicate the analysis. In this case, having extra statements that have to be executed 34,219,980 times each slows down the code. If we hadn't used

kernprof.py to investigate the line-by-line effect of this change, we might have drawn other conclusions about the reason for the slowdown, as we'd have lacked the necessary evidence.

At this point it makes sense to step back to the earlier `timeit` technique to test the cost of individual expressions:

```
Python 3.12.0 | packaged by Anaconda, Inc. | (main, Oct 2 2023, 17:29:18)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.21.0 -- An enhanced Interactive Python. Type '?' for help.
In [1]: z = 0+0j
In [2]: %timeit abs(z) < 2
66.8 ns ± 2.07 ns per loop (mean ± std. dev. of 7 runs, 10,000,000 loops each)
In [3]: n = 1
In [4]: maxiter = 300
In [5]: %timeit n < maxiter
20.7 ns ± 0.0611 ns per loop (mean ± std. dev. of 7 runs, 10,000,000 loops each)
```

From this simple analysis, it looks as though the logic test on `n` is more than three times faster than the call to `abs`. Since the order of evaluation for Python statements is both left to right and opportunistic, it makes sense to put the cheapest test on the left side of the equation. On 1 in every 301 tests for each coordinate, the `n < maxiter` test will be `False`, so Python wouldn't need to evaluate the other side of the `and` operator.

We never know whether `abs(z) < 2` will be `False` until we evaluate it, and our earlier observations for this region of the complex plane suggest it is `True` around 10% of the time for all 300 iterations. If we wanted to have a strong understanding of the time complexity of this part of the code, it would make sense to continue the numerical analysis, but in this situation, we want an easy check to see if we can get a quick win.

We can form a new hypothesis stating, “By swapping the order of the operators in the `while` statement, we will achieve a reliable speedup.” We *can* test this hypothesis using `kernprof`, but the additional overheads of profiling this way might add too much noise. Instead, we can use an earlier version of the code, running a test comparing `while abs(z) < 2 and n < maxiter:` against `while n < maxiter and abs(z) < 2:`, which we see in [Example 2-8](#).

Running the two variants *outside* of `line_profiler` means they run at similar speeds, on the same problem size that we've been using (with `x==1000`).

The overheads of `line_profiler` also confuse the result; the results on line 23 for both versions are similar, and the timing for this final result is a little slower at 33 seconds. We might reject the hypothesis that in Python 3.12 changing the order of the logic results in a consistent speedup—there's no clear evidence for this.

However, if the problem is made harder by setting `x==5000`, the version with the swapped `if` statement is consistently slightly faster. Ian measures 143 seconds for the original order and 142 seconds for the swapped order of operations.

Using a more suitable approach to solve this problem (e.g., swapping to using Cython or PyPy, as described in [Chapter 8](#)) would yield greater gains.

We can be confident in our result because of the following:

- We stated a hypothesis that was easy to test.
- We changed our code so that only the hypothesis would be tested (never test two things at once!).
- We gathered enough evidence to support our conclusion.

For completeness, we can run a final kernprof on the two main functions including our optimization to confirm that we have a full picture of the overall complexity of our code.

Example 2-8. Swapping the order of the compound while statement makes the function fractionally faster

```
$ kernprof -l -v julia1_lineprofiler3.py
...
Wrote profile results to julia1_lineprofiler3.py.lprof
Timer unit: 1e-06 s

Total time: 33.9135 s
File: julia1_lineprofiler3.py
Function: calculate_z_serial_purepython at line 15

Line #      Hits Per Hit % Time  Line Contents
=====
15          @profile
16      def calculate_z_serial_purepython(maxiter,
17                                     zs, cs):
18          """Calculate output list using Julia
19             update rule"""
18          1 4015.8   0.0      output = [0] * len(zs)
19 1000001   0.3   1.0      for i in range(len(zs)):
20 1000000   0.2   0.7          n = 0
21 1000000   0.3   0.8          z = zs[i]
22 1000000   0.2   0.7          c = cs[i]
23 34219980  0.4  44.1          while n < maxiter and abs(z) < 2:
24 33219980  0.3  29.0              z = z * z + c
25 33219980  0.2  22.9              n += 1
26 1000000   0.3   0.8              output[i] = n
27          1      2.1   0.0      return output
```

As expected, we can see from the output in [Example 2-9](#) that `calculate_z_serial_purepython` takes most (98%) of the time of its parent function. The list-creation steps are minor in comparison.

Example 2-9. Testing the line-by-line costs of the setup routine

Total time: 64.0333 s
File: julia1_lineprofiler3.py
Function: calc_pure_python at line 30

Line #	Hits	Per Hit	% Time	Line Contents
=====				
30				@profile
31				def calc_pure_python(draw_output,
				desired_width,
				max_iterations):
...				
52	1001	0.3	0.0	for ycoord in y:
53	1001000	0.3	0.5	for xcoord in x:
54	1000000	0.4	0.6	zs.append(complex(xcoord, ycoord))
55	1000000	0.4	0.6	cs.append(complex(c_real, c_imag))
56				
57	1	56.2	0.0	print(f"Length of x: {len(x):,}")
58	1	7.4	0.0	print(f"Total elements: {len(zs):,}")
59	1	5.1	0.0	start_time = time.time()
60	1	6e+07	98.3	output = calculate_z_serial_purepython(max_iterations, zs, cs)
61	1	6.0	0.0	end_time = time.time()
62	1	1.3	0.0	secs = end_time - start_time
63	1	48.6	0.0	print(f"{calculate_z_serial_purepython.__name__} took {secs:0.2f} seconds")
64				
65	1	7129.7	0.0	assert sum(output) == 33219980 # this sum is expected for 1000^2 grid with 300 iterations

line_profiler gives us a great insight into the cost of lines inside loops and expensive functions; even though profiling adds a speed penalty, it is a great boon to scientific developers. Remember to use representative data to make sure you're focusing on the lines of code that'll give you the biggest win.

Using memory_profiler to Diagnose Memory Usage

Just as Robert Kern's line_profiler package measures CPU usage, the memory_profiler module by Fabian Pedregosa and Philippe Gervais measures memory usage on a line-by-line basis. Understanding the memory usage characteristics of your code allows you to ask yourself two questions:

- Could we use *less* RAM by rewriting this function to work more efficiently?
- Could we use *more* RAM and save CPU cycles by caching?

`memory_profiler` operates in a very similar way to `line_profiler` but runs far more slowly. If you install the `psutil` package (optional but recommended), `memory_profiler` will run faster. Memory profiling may easily make your code run 10 to 100 times slower. In practice, you will probably use `memory_profiler` occasionally and `line_profiler` (for CPU profiling) more frequently.

Install `memory_profiler` with the command `pip install memory_profiler` (and optionally with `pip install psutil`).

As mentioned, the implementation of `memory_profiler` is not as performant as the implementation of `line_profiler`. It may therefore make sense to run your tests on a smaller problem that completes in a useful amount of time. Overnight runs might be sensible for validation, but you need quick and reasonable iterations to diagnose problems and hypothesize solutions. The code in [Example 2-10](#) uses the full 1,000 × 1,000 grid, and the statistics took about two hours to collect on Ian’s laptop.



The requirement to modify the source code is a minor annoyance. As with `line_profiler`, a decorator (`@profile`) is used to mark the chosen function. This will break your unit tests unless you make a dummy decorator—see “[No-op @profile Decorator](#)” on page 74.

When dealing with memory allocation, you must be aware that the situation is not as clear-cut as it is with CPU usage. Generally, it is more efficient to overallocate memory in a process that can be used at leisure, as memory allocation operations are relatively expensive. Furthermore, garbage collection is not instantaneous, so objects may be unavailable but still in the garbage collection pool for some time.

The outcome of this is that it is hard to really understand what is happening with memory usage and release inside a Python program, as a line of code may not allocate a deterministic amount of memory *as observed from outside the process*. Observing the gross trend over a set of lines is likely to lead to better insight than would be gained by observing the behavior of just one line.

Let’s take a look at the output from `memory_profiler` in [Example 2-10](#). Inside `calculate_z_serial_purepython` on line 18, we see that the allocation of 1,000,000 items causes approximately 7 MB of RAM to be added to this process.¹ This does not mean that the output list is definitely 7 MB in size, just that the process grew by approximately 7 MB during the internal allocation of the list.

¹ `memory_profiler` measures memory usage according to the International Electrotechnical Commission’s MiB (mebibyte) of 2^{20} bytes. This is slightly different from the more common but also more ambiguous MB (megabyte has two commonly accepted definitions!). 1 MiB is equal to 1.048576 (or approximately 1.05) MB. For our discussion, unless we’re dealing with very specific amounts, we’ll consider the two equivalent.

In the parent function on line 52, we see that the allocation of the `zs` and `cs` lists changes the `Mem usage` column from 47 MB to 123 MB (a change of +76 MB). Again, it is worth noting that this is not necessarily the true size of the arrays, just the size that the process grew by after these lists had been created.

At the time of writing, the `memory_usage` module exhibits a bug—the `Increment` column does not always match the change in the `Mem usage` column. During the first edition of this book, these columns were correctly tracked; you might want to check the status of this bug on [GitHub](#). We recommend you use the `Mem usage` column, as this correctly tracks the change in process size per line of code.

Example 2-10. `memory_profiler`'s result on both of our main functions, showing an unexpected memory use in `calculate_z_serial_purepython`

```
$ python -m memory_profiler julia1_memoryprofiler.py
...
```

Line #	Mem usage	Increment	Line Contents
15	123.086 MiB	123.086 MiB	@profile
16			def calculate_z_serial_purepython(maxiter, zs, cs):
17			"""Calculate output list...
18	130.711 MiB	7.625 MiB	output = [0] * len(zs)
19	133.461 MiB	0.000 MiB	for i in range(len(zs)):
20	133.461 MiB	0.000 MiB	n = 0
21	133.461 MiB	0.000 MiB	z = zs[i]
22	133.461 MiB	0.000 MiB	c = cs[i]
23	133.461 MiB	2.750 MiB	while n < maxiter and abs(z) < 2:
24	133.461 MiB	0.000 MiB	z = z * z + c
25	133.461 MiB	0.000 MiB	n += 1
26	133.461 MiB	0.000 MiB	output[i] = n
27	133.461 MiB	0.000 MiB	return output

...

Line #	Mem usage	Increment	Line Contents
30	47.137 MiB	47.137 MiB	@profile
31			def calc_pure_python(draw_output, desired_width, max_iterations):
32			"""Create a list of...
33	47.137 MiB	0.000 MiB	x_step = (x2 - x1) / desired_width
34	47.137 MiB	0.000 MiB	y_step = (y1 - y2) / desired_width
35	47.137 MiB	0.000 MiB	x = []
36	47.137 MiB	0.000 MiB	y = []
37	47.137 MiB	0.000 MiB	ycoord = y2
38	47.137 MiB	0.000 MiB	while ycoord > y1:
39	47.137 MiB	0.000 MiB	y.append(ycoord)

```

40  47.137 MiB    0.000 MiB          ycoord += y_step
41  47.137 MiB    0.000 MiB          xcoord = x1
42  47.137 MiB    0.000 MiB          while xcoord < x2:
43  47.137 MiB    0.000 MiB              x.append(xcoord)
44  47.137 MiB    0.000 MiB              xcoord += x_step
50  47.137 MiB    0.000 MiB          zs = []
51  47.137 MiB    0.000 MiB          cs = []
52  123.086 MiB   -1.055 MiB          for ycoord in y:
53  123.086 MiB -893.922 MiB              for xcoord in x:
54  123.086 MiB -900.219 MiB                  zs.append(complex(xcoord, ycoord))
55  123.086 MiB -853.844 MiB                  cs.append(complex(c_real, c_imag))
56
57  123.086 MiB    0.000 MiB          print("Length of x:", len(x))
58  123.086 MiB    0.000 MiB          print("Total elements:", len(zs))
59  123.086 MiB    0.000 MiB          start_time = time.time()
60  133.461 MiB  133.461 MiB          output = calculate_z_serial...
61  133.461 MiB    0.000 MiB          end_time = time.time()
62  133.461 MiB    0.000 MiB          secs = end_time - start_time
63  133.461 MiB    0.000 MiB          print(calculate_z_serial_purepython.__name__
                                     + " took", secs, "seconds")

64
66  133.461 MiB    0.000 MiB          assert sum(output) == 33219980

```

Another way to visualize the change in memory use is to sample over time and plot the result. `memory_profiler` has a utility called `mprof`, used once to sample the memory usage and a second time to visualize the samples. It samples by time and not by line, so it barely impacts the runtime of the code.

Figure 2-6 is created using `mprof run julia1_memoryprofiler.py`. This writes a statistics file that is then visualized using `mprof plot`. Our two functions are bracketed: this shows where in time they are entered, and we can see the growth in RAM as they run. Inside `calculate_z_serial_purepython`, we can see the steady increase in RAM usage throughout the execution of the function; this is caused by all the small objects (int and float types) that are created.

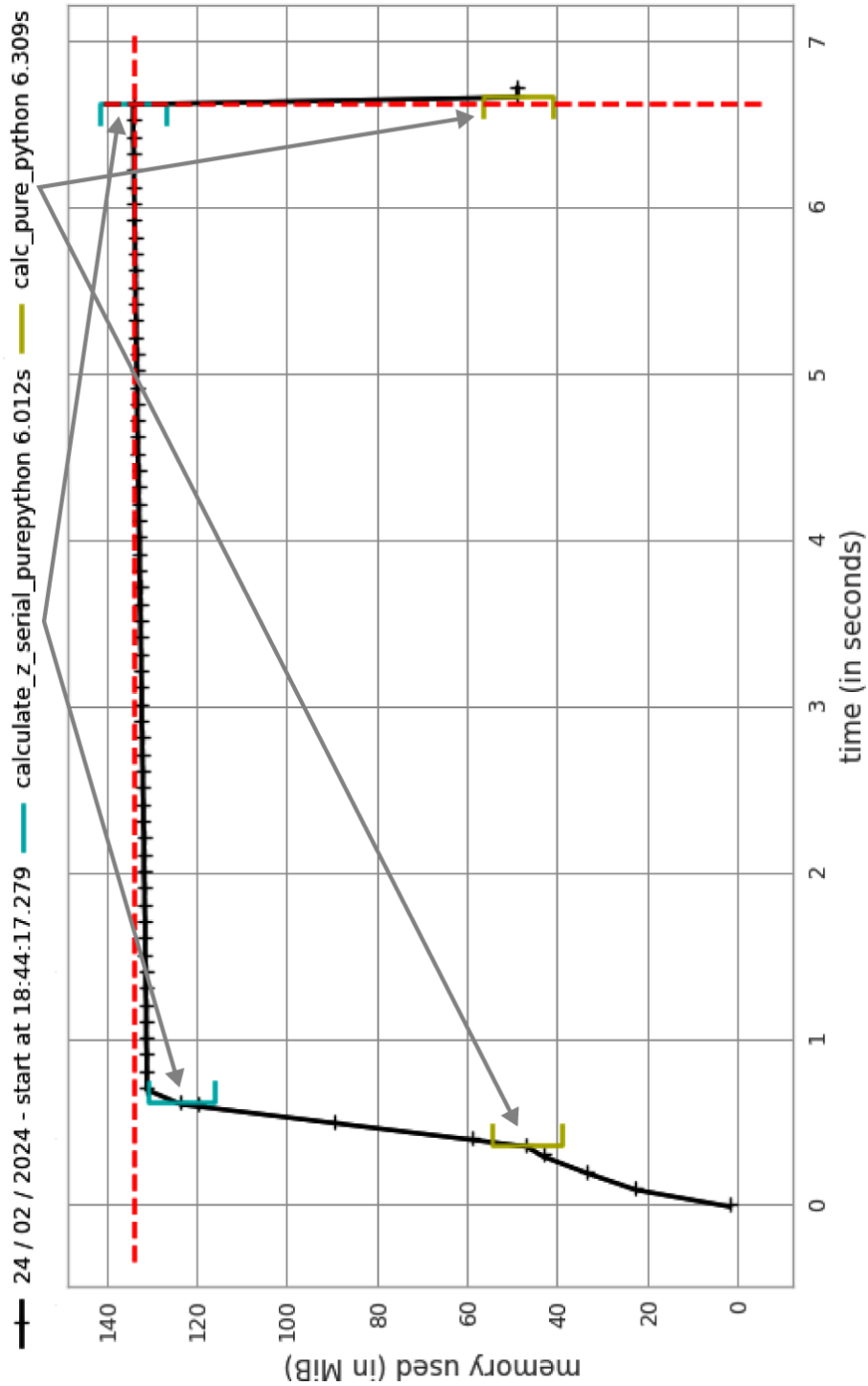


Figure 2-6. memory_profiler report using mprof

In addition to observing the behavior at the function level, we can add labels using a context manager. The snippet in [Example 2-11](#) is used to generate the graph in [Figure 2-7](#). We can see the `create_output_list` label: it appears momentarily at around 1.5 seconds after `calculate_z_serial_purepython` and results in the process being allocated more RAM. We then pause for a second; `time.sleep(1)` is an artificial addition to make the graph easier to understand.

Example 2-11. Using a context manager to add labels to the `mprof` graph

```
@profile
def calculate_z_serial_purepython(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    with profile.timestamp("create_output_list"):
        output = [0] * len(zs)
        time.sleep(1)
    with profile.timestamp("calculate_output"):
        for i in range(len(zs)):
            n = 0
            z = zs[i]
            c = cs[i]
            while n < maxiter and abs(z) < 2:
                z = z * z + c
                n += 1
            output[i] = n
    return output
```

In the `calculate_output` block that runs for most of the graph, we see a very slow, linear increase in RAM usage. This will be from all of the temporary numbers used in the inner loops. Using the labels really helps us to understand at a fine-grained level where memory is being consumed. Interestingly, we see the “peak RAM usage” line—a dashed vertical line just before the 7-second mark—occurring before the termination of the program. Potentially this is due to the garbage collector recovering some RAM from the temporary objects used during `calculate_output`.

What happens if we simplify our code and remove the creation of the `zs` and `cs` lists? We then have to calculate these coordinates inside `calculate_z_serial_purepython` (so the same work is performed), but we’ll save RAM by not storing them in lists. You can see the code in [Example 2-12](#).

In [Figure 2-8](#), we see a major change in behavior—the overall envelope of RAM usage drops from 140 MB to 60 MB, reducing our RAM usage by half!

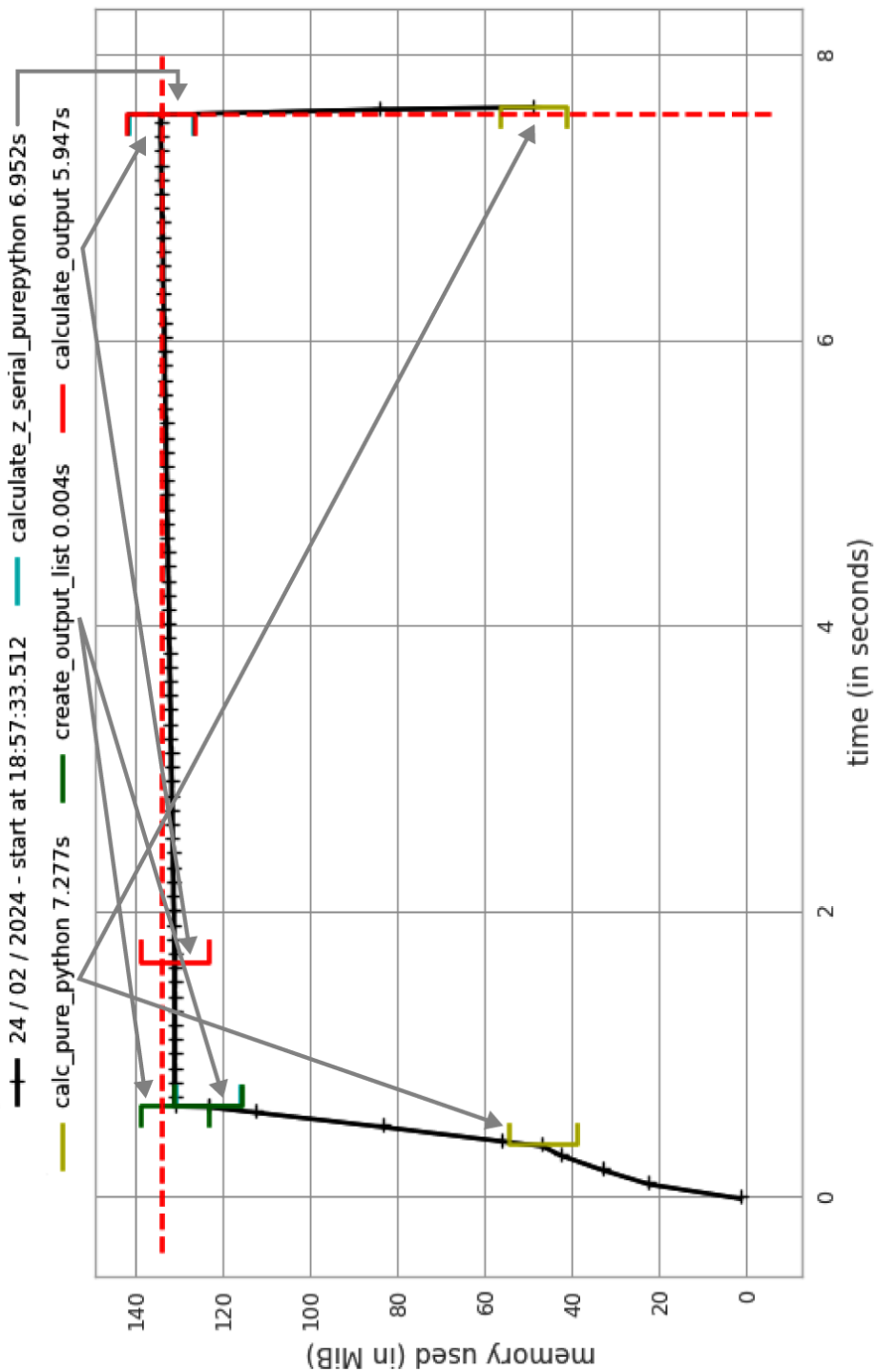


Figure 2-7. memory_profiler report using mprof with labels

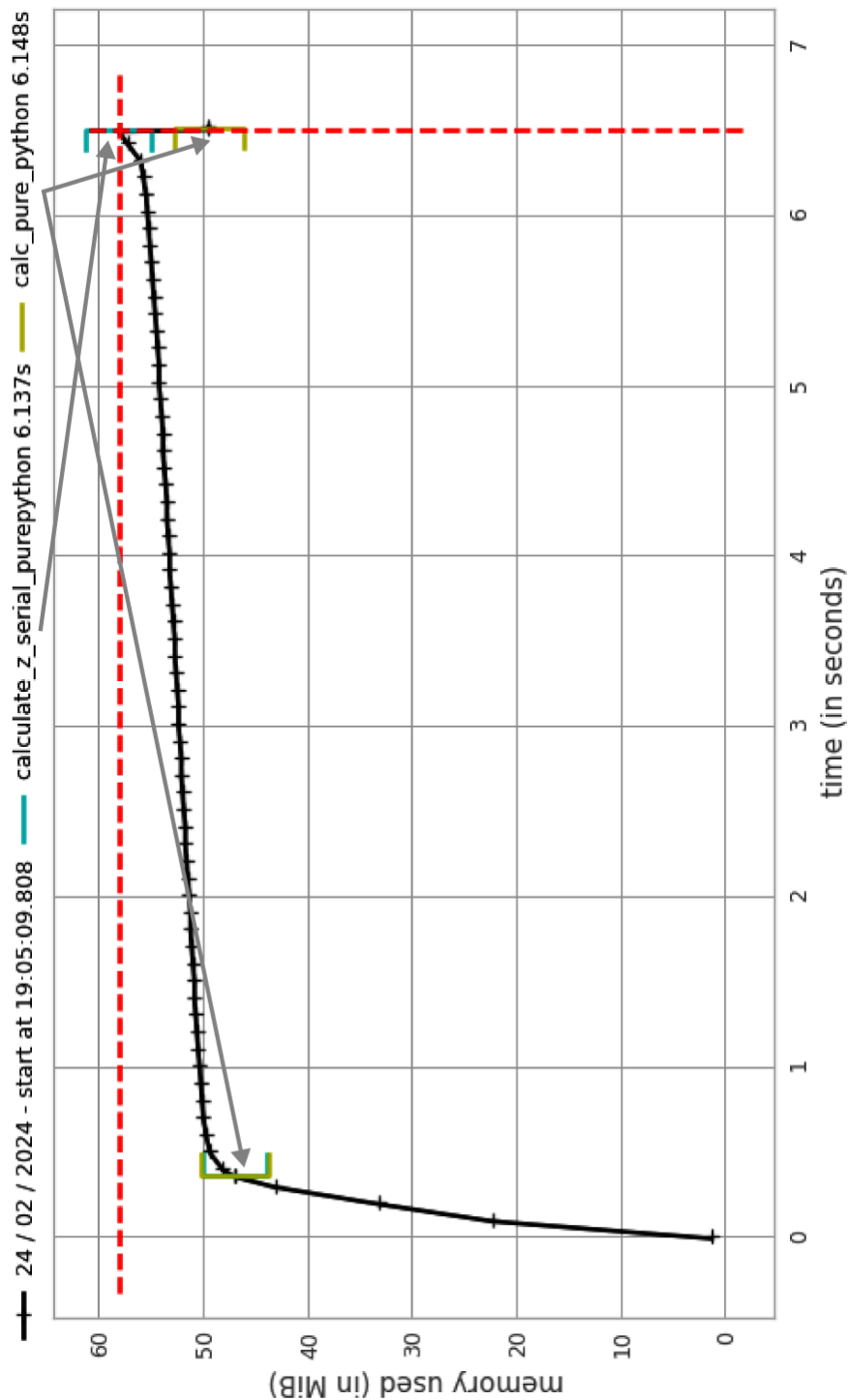


Figure 2-8. *memory_profiler* after removing two large lists

Example 2-12. Creating complex coordinates on the fly to save RAM

```
@profile
def calculate_z_serial_purepython(maxiter, x, y):
    """Calculate output list using Julia update rule"""
    output = []
    for ycoord in y:
        for xcoord in x:
            z = complex(xcoord, ycoord)
            c = complex(c_real, c_imag)
            n = 0
            while n < maxiter and abs(z) < 2:
                z = z * z + c
                n += 1
            output.append(n)
    return output
```

If we want to measure the RAM used by several statements, we can use the IPython magic `%memit`, which works just like `%timeit`. In [Chapter 12](#), we will look at using `%memit` to measure the memory cost of lists and discuss various ways of using RAM more efficiently.

`memory_profiler` offers an interesting aid to debugging a large process via the `--pdb-mem=XXX` flag. The `pdb` debugger will be activated after the process exceeds `XXX` MB. This will drop you in directly at the point in your code where too many allocations are occurring, if you're in a space-constrained environment.

Combining CPU and Memory Profiling with Scalene

Scalene combines CPU, memory profiling, and GPU profiling into one easy-to-run package. It runs on all platforms and is installed with `pip install scalene`.

Scalene uses its own lightweight profiler library, so it has very little impact on execution speed (unlike both `memory_profiler` and `line_profiler`). We can invoke it with `scalene julia1_memoryprofiler.py`, and it will profile the whole program without needing any modification.

It has options to restrict profiling with an `@profile` decorator (similar to `line_profiler` and `memory_profiler`) and to filter in or out certain filenames. A magic extension is provided for use in Jupyter Notebooks with `%%scalene` to profile a whole cell.

Looking at the left-most “TIME” column in the screenshot in [Figure 2-9](#), we see that the while loop occupies 85% of the execution time, shown using a large horizontal bar. This bar has three components: the longest (and darkest) shows native Python time, while two smaller chunks on the right edge (both with lighter shades) represent native and system execution time.

Native Python time refers to the time spent executing Python instructions, which you can control. Native time refers to libraries (i.e., libraries written in C++ and compiled), which the developer is unlikely to optimize. System time refers to operating system calls such as I/O, which again probably can't be optimized by the developer but may highlight I/O bottlenecks (which can be approached in a different way).

The second “MEMORY” column shows the peak memory allocation per line. We can see that `zs.append(complex(xcoord, ycoord))` uses 40 MB, while `cs.append(complex(c_real, c_imag))` uses 31 MB. Different runs on this produce different peak-allocation numbers, and sometimes `cs` takes more than `zs`. So be cautious about forming a strong opinion without making multiple runs.

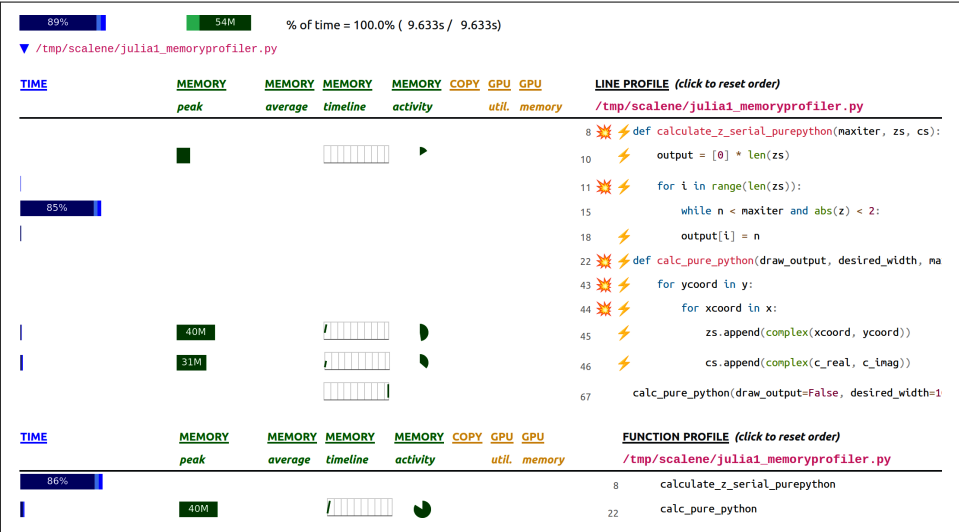


Figure 2-9. Combined CPU and memory profiling output with Scalene

Many of the lines have either an “explosion icon” or a “lightning icon.” Scalene creates calls to ChatGPT if you provide an API key so that optimizations can be automatically proposed.



Be cautious when taking tips from a GenAI system—it’ll write something that’s convincing, but it could easily be working from outdated suggestions that have been well-voted and well-cited due to age. Always be skeptical and run your own benchmarks.

Scalene can generate both a console-only (text) output and an interactive HTML report. The snippet in [Example 2-13](#) shows the console report with two extra lines added making NumPy arrays from the pure Python list containers.

Both cause a duplication in the underlying data into `complex128` (8 byte float $\times$ 2) 16 byte dtype. With 1,000,000 elements in each, these lines cause a 16 MB copy of each array to be constructed. A user might not realize this as they code (or during a code review may not appreciate what's happening), while a profiler clearly brings out the memory duplication that's occurring.

Example 2-13. Console output from Scalene with two NumPy arrays added

```
Line ... Python |peak |timeline/%
...
49 |...      20% | 19M |_ 15% | zs_np = np.array(zs)
50 |...      | 15M |_ 12% | cs_np = np.array(cs)
```

You should expect to see different results to any you record with `line_profiler` or `memory_profiler`. Each profiling tool has a different impact and typically looks at CPU and memory usage in slightly different ways and with different resolutions—the big-scale picture will be similar, but the details will be different.

Scalene's lightweight profiling library should add an overhead of under 20% to your execution speed, and it should also reasonably measure the actual time and memory cost—the authors claim that it is potentially far more accurate than other profilers.

Introspecting an Existing Process with PySpy

PySpy is an intriguing new sampling profiler—rather than requiring any code changes, it introspects an already-running Python process and reports in the console with a top-like display. Being a sampling profiler, it has almost no runtime impact on your code. It is written in Rust and requires elevated privileges to introspect another process. It can also profile subprocesses and native compiled extensions.

This tool could be very useful in a production environment with long-running processes or complicated installation requirements. It supports Windows, Mac, and Linux. Install it using `pip install py-spy` (note the dash in the name—there's a separate `pyspy` project that isn't related).

If your process is already running, you'll want to use `ps` to get its process identifier (the PID); then this can be passed into `py-spy`, as shown in [Example 2-14](#). `py-spy` needs `sudo` privileges, which would run it in the superuser's environment, so we pass in our login's `PATH` using `sudo env "PATH=$PATH"` when calling `py-spy` along with the process identifier to monitor.

Example 2-14. Running PySpy at the command line

```
$ ps -A -o pid,rss,cmd | ack python
...
95671 94984 python julia1_nopil.py
...
$ sudo env "PATH=$PATH" py-spy top --pid 95671
```

In [Figure 2-10](#), you'll see a static picture of a `top`-like display in the console; this updates every second to show which functions are currently taking most of the time.

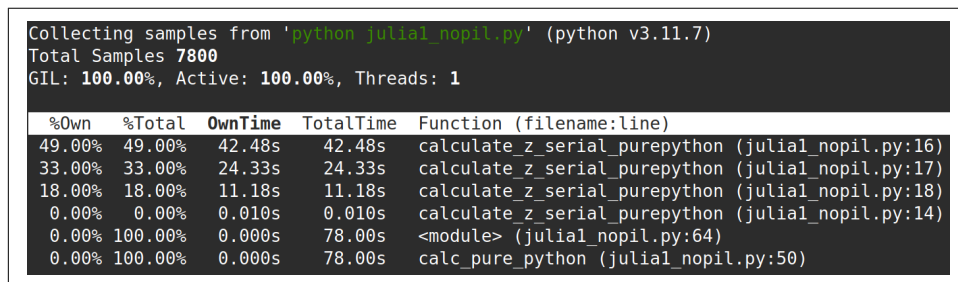


Figure 2-10. Introspecting a Python process using PySpy

PySpy can also export a flame chart. Here, we'll run that option while asking PySpy to run our code directly without requiring a PID using `$ py-spy record -o profile.svg --python julia1_nopil.py`. The `--` tells `py-spy` to take the command that follows (`python julia1_nopil.py`), which might have its own command-line arguments, and ignore any command-line switches—it'll execute that command from inside `py-spy`.

You'll see in [Figure 2-11](#) that the width of the display represents the entire program's runtime, and each layer moving down the image represents functions called from above.

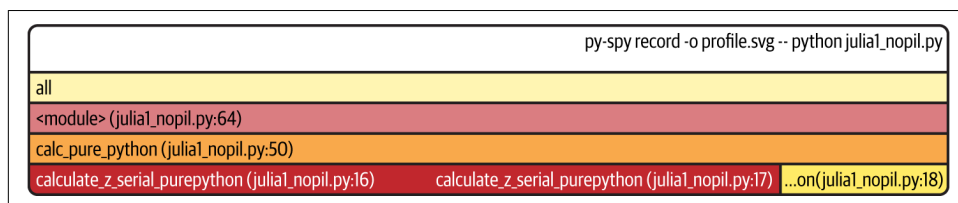


Figure 2-11. Part of a flame chart for PySpy

PySpy also has the useful ability to hook into an already-running Python process; so if you have a web server or a long-running extract-transform-load process, you can connect to that process and introspect where in the program the time seems

to be being spent. This allows for in-situ profiling when something unexpected has occurred, without having to set up a new profiling run.

VizTracer for an Interactive Time-Based Call Stack

VizTracer is a two-part tool to profile and then view a time-based view of Python code execution. This tool is very powerful but can be a little tricky to learn. The fact that it displays time left-to-right and enables you to zoom in on a flame-like chart gives you a bird's-eye view of the entire process, so you can get a feel for how long parts of the execution path take.

It can track the execution, arguments, and return values and time of every function call. It allows custom logging information; it supports multiprocesses and threads; it doesn't require source code changes; it has an API; and it provides an interactive graphical tool.

We install it with `pip install viztracer`, and then we can run the same code we've used previously with `viztracer julia1_memoryprofiler.py`.

Having profiled code using `viztracer`, a `results.json` file is output. We next visualize this with `vizviewer results.json`; for our Julia code, we get a result like that shown in [Figure 2-12](#). The screen has the following sections:

- The very top is an interactive timeline with grab bars to slide and zoom the view to a specific point in time.
- “MainProcess” shows a horizontal timeline of the code's execution and the function calls hanging down the screen in bars.
- “Current Selection” gives details of anything that's clicked in “MainProcess.”

VizTracer can track the execution of multiple threads and multiple processes; these would be displayed alongside “MainProcess.” VizTracer can also log other time-based events such as `print` statements and calls to the garbage collector, enabling you to annotate specific events that you'd like to observe.

The two long bars in the center of the screenshot under “MainProcess” detail the time-based view on `calc_pure_python` and its call into `calculate_z_serial_pure_python`; below this are many tiny spikes—these are some of the individual calls to `abs`.

If you click `calculate_z_serial_purepython` in the UI, you'll get the “Details” section, as shown in the bottom left of the image. This shows the function name and a start time and duration, plus thread-based details. To the right of this is a view of the source code that was executed.



Figure 2-12. Overview of the Julia execution laid out through time

Due to the high level of detail VizTracer can track, it can potentially generate unfeasibly large information dumps that can't be viewed. This can produce a “Circular buffer is full” error.

VizTracer provides various tools to filter the information that's collected. For the Julia example we had to:

- reduce the `calc_pure_python` argument from 1000 to a `desired_width=200`
- filter shorter calls when profiling with `viztracer --min_duration 0.1 julia1_viztracer.py`

Other filtering options can include `--ignore_c_function`, which includes all built-in functions (which would ignore every `abs` call), or limiting the stack depth with `--max_stack_depth 10`. In this case we aren't witnessing a deep stack but instead a large number of calls to the fast `abs` function.

In order to get enough detail without overloading the circular buffer, here we limited the profiling to functions taking longer than 0.1 milliseconds—interestingly, the calls to `abs` could sometimes take longer (but frequently were much faster), so we caught some in this trace. Without this filter we'd have millions to track, and that was the cause of this particular “Circular buffer is full” error.

Author Ian has found this tool very useful for digging into the execution path in libraries like Pandas, so we can see how long certain parts of the functions take and how many other functions are called behind the scenes.

Bytecode: Under the Hood

So far we've reviewed various ways to measure the cost of Python code (for both CPU and RAM usage). We haven't yet looked at the underlying bytecode used by the virtual machine, though. Understanding what's going on “under the hood” helps to build a mental model of what's happening in slow functions, and it'll help when you come to compile your code. So let's introduce some bytecode.

Using the `dis` Module to Examine CPython Bytecode

The `dis` module lets us inspect the underlying bytecode that we run inside the stack-based CPython virtual machine. Having an understanding of what's happening in the virtual machine that runs your higher-level Python code will help you to understand why some styles of coding are faster than others. It will also help when you come to use a tool like Cython, which steps outside of Python and generates C code.

The `dis` module is built in. You can pass it code or a module, and it will print out a disassembly. In [Example 2-15](#), we disassemble the outer loop of our CPU-bound function.



You should try to disassemble one of your own functions and to follow *exactly* how the disassembled code matches to the disassembled output. Can you match the following `dis` output to the original function?

Example 2-15. Using the built-in `dis` to understand the underlying stack-based virtual machine that runs our Python code

```
In [1]: import dis
In [2]: import julia1_nopil
In [3]: dis.dis(julia1_nopil.calculate_z_serial_purepython)
9          0 RESUME          0

11          2 LOAD_CONST          1 (0)
          4 BUILD_LIST          1
          6 LOAD_GLOBAL          1 (NULL + len)
         16 LOAD_FAST          1 (zs)
         18 CALL          1
         26 BINARY_OP          5 (*)
         30 STORE_FAST          3 (output)

12          32 LOAD_GLOBAL          3 (NULL + range)
         42 LOAD_GLOBAL          1 (NULL + len)
         52 LOAD_FAST          1 (zs)
         54 CALL          1
         62 CALL          1
         70 GET_ITER
    >>      72 FOR_ITER          71 (to 218)
         76 STORE_FAST          4 (i)

13          78 LOAD_CONST          1 (0)
         80 STORE_FAST          5 (n)

...

16          166 LOAD_GLOBAL          5 (NULL + abs)
         176 LOAD_FAST          6 (z)
         178 CALL          1
         186 LOAD_CONST          2 (2)
         188 COMPARE_OP          2 (<)
         192 POP_JUMP_IF_FALSE          6 (to 206)
         194 LOAD_FAST          5 (n)
         196 LOAD_FAST          0 (maxiter)
         198 COMPARE_OP          2 (<)
         202 POP_JUMP_IF_FALSE          1 (to 206)
```

```

204 JUMP_BACKWARD          33 (to 140)

19  >> 206 LOAD_FAST        5 (n)
      208 LOAD_FAST        3 (output)
      210 LOAD_FAST        4 (i)
      212 STORE_SUBSCR
      216 JUMP_BACKWARD     73 (to 72)

12  >> 218 END_FOR

20      220 LOAD_FAST        3 (output)
      222 RETURN_VALUE

```

The output is fairly straightforward, if terse. The first column contains line numbers that relate to our original file. The second column contains several `>>` symbols; these are the destinations for jump points elsewhere in the code. The third column is the operation address; the fourth has the operation name. The fifth column contains the parameters for the operation. The sixth column contains annotations to help line up the bytecode with the original Python parameters.

Refer back to [Example 2-3](#) to match the bytecode to the corresponding Python code. The bytecode starts on Python line 11 by putting the constant value 0 onto the stack, and then it builds a single-element list. Next, it searches the namespaces to find the `len` function, puts it on the stack, searches the namespaces again to find `zs`, and then puts that onto the stack. Now a multiply can be performed, consuming the stack items, which are then stored in `output`. That’s the first line of our Python function now dealt with. Follow the next block of bytecode to understand the behavior of the second line of Python code (the outer `for` loop).



The jump points (`>>`) match to instructions like `JUMP_BACKWARD` and `POP_JUMP_IF_FALSE`. Go through your own disassembled function and match the jump points to the jump instructions.

Digging into Bytecode Specialization with Specialist

When Python 3.11 runs your code, it attempts to identify “hot” code that is run frequently enough to warrant optimization. When possible, it will use “specialized” bytecode instructions to replace the more general bytecode that has been used up to Python 3.10. The specialized bytecode can run faster by doing less work (perhaps by avoiding redundant checks or via other optimizations). It can be installed with `pip install specialist`.



The text is color-coded when output from the tool.

Specializations hold true as long as the same types are encountered—if an addition is performed on two integers in a loop, that operation can be specialized (and would be colored green). If, however, the addition sees different types (maybe integers or floating-point types, depending on the iteration), then it'll stay in an “adaptive” state, which is colored red.

Specialist color-codes the “hot” regions of code that Python 3.11+ identifies during execution, allowing us to review whether our code is running as fast as might be possible. Using the Julia example in [Figure 2-13](#), we can see:

- Unshaded lines—no specialization has been attempted
- Lines in green (surrounded by a solid rectangle)—successfully specialized, they’re running fast
- Sections in orange (surrounded by a dashed rectangle)—partly specialized, which sometimes fail and revert to being adaptive (notably `< 2` and `z * z + c`)
- In our example, none are a red color (indicating a lack of specialization)

The `abs(z) < 2` can be made “more green” (i.e., more specialized) by changing it to `abs(z) < 2.0`, as Python 3.12 currently prefers identical numerical types for certain primitive operations. The complex-type operation `z * z + c` refused to be improved.

```
def calculate_z_serial_purepython(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while abs(z) < 2 and n < maxiter:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

Figure 2-13. Viewing colored output from Specialist

In this case, making the 2.0 change did improve performance a little. In the current version of Python, the Specialist tool is more for education about what's happening behind the scenes, but experiments may reveal opportunities to improve execution speed.

Having introduced bytecode, we can now ask: what's the bytecode and time cost of writing a function out explicitly versus using built-ins to perform the same task?

Different Approaches, Different Complexity

There should be one—and preferably only one—obvious way to do it. Although that way may not be obvious at first unless you're Dutch.²

—Tim Peters, The Zen of Python

There will be various ways to express your ideas using Python. Generally, the most sensible option should be clear, but if your experience is primarily with an older version of Python or another programming language, you may have other methods in mind. Some of these ways of expressing an idea may be slower than others.

You probably care more about readability than speed for most of your code, so your team can code efficiently without being puzzled by performant but opaque code. Sometimes you will want performance, though (without sacrificing readability). Some speed testing might be what you need.

Take a look at the two code snippets in [Example 2-16](#). Both do the same job, but the first generates a lot of additional Python bytecode, which will cause more overhead.

Example 2-16. A naive and a more efficient way to solve the same summation problem

```
def fn_expressive(upper=1_000_000):
    total = 0
    for n in range(upper):
        total += n
    return total

def fn_terse(upper=1_000_000):
    return sum(range(upper))

assert fn_expressive() == fn_terse(), "Expect identical results from both functions"
```

² The language creator Guido van Rossum is Dutch, and not everyone has agreed with his “obvious” choices, but on the whole we like the choices that Guido makes!

Both functions calculate the sum of a range of integers. A simple rule of thumb (but one you *must* back up using profiling!) is that more lines of bytecode will execute more slowly than fewer equivalent lines of bytecode that use built-in functions. In [Example 2-17](#), we use IPython’s `%timeit` magic function to measure the best execution time from a set of runs. `fn_terse` runs over twice as fast as `fn_expressive`!

Example 2-17. Using `%timeit` to test our hypothesis that using built-in functions should be faster than writing our own functions

```
In [2]: %timeit fn_expressive()
45.6 ms ± 963 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [3]: %timeit fn_terse()
16.4 ms ± 226 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

If we use the `dis` module to investigate the code for each function, as shown in [Example 2-18](#), we can see that the virtual machine has 17 lines to execute with the more expressive function and only 7 to execute with the very readable but terser second function.

Example 2-18. Using `dis` to view the number of bytecode instructions involved in our two functions

```
In [4]: import dis

In [5]: dis.dis(fn_expressive)
1          0 RESUME                      0

2          2 LOAD_CONST                  1 (0)
          4 STORE_FAST                   1 (total)

3          6 LOAD_GLOBAL                 1 (NULL + range)
          16 LOAD_FAST                   0 (upper)
          18 CALL                        1
          26 GET_ITER
>>       28 FOR_ITER                     7 (to 46)
          32 STORE_FAST                 2 (n)

4          34 LOAD_FAST                  1 (total)
          36 LOAD_FAST                  2 (n)
          38 BINARY_OP                  13 (+=)
          42 STORE_FAST                 1 (total)
          44 JUMP_BACKWARD              9 (to 28)

3  >>   46 END_FOR

5          48 LOAD_FAST                  1 (total)
          50 RETURN_VALUE
```



```

In [6]: dis.dis(fn_terse)
7          0 RESUME                               0

8          2 LOAD_GLOBAL                         1 (NULL + sum)
12         2 LOAD_GLOBAL                         3 (NULL + range)
22         2 LOAD_FAST                           0 (upper)
24         1 CALL
32         1 CALL
40        1 RETURN_VALUE

```

The difference between the two code blocks is striking. Inside `fn_expressive()`, we maintain two local variables and iterate over a list using a `for` statement. The `for` loop will be checking to see if a `StopIteration` exception has been raised on each loop. Each iteration applies the `total.__add__` function, which will check the type of the second variable (`n`) on each iteration. These checks all add a little expense.

Inside `fn_terse()`, we call out to an optimized C list comprehension function that knows how to generate the final result without creating intermediate Python objects. This is much faster, although each iteration must still check for the types of the objects that are being added together (in [Chapter 4](#), we look at ways of fixing the type so we don't need to check it on each iteration).

As noted previously, you *must* profile your code—if you just rely on this heuristic, you will inevitably write slower code at some point. It is definitely worth learning whether a shorter and still readable way to solve your problem is built into Python. If so, it is more likely to be easily readable by another programmer, and it will *probably* run faster.

Unit Testing During Optimization to Maintain Correctness

If you aren't already unit testing your code, you are probably hurting your longer-term productivity. Ian (blushing) is embarrassed to note that he once spent a day optimizing his code, having disabled unit tests because they were inconvenient, only to discover that his significant speedup result was due to breaking a part of the algorithm he was improving. You do not need to make this mistake even once.



Add unit tests to your code for a saner life. You'll be giving your current self and your colleagues faith that your code works, and you'll be giving a present to your future self, who has to maintain this code later. You really will save a lot of time in the long term by adding tests to your code.

In addition to unit testing, you should also strongly consider using `coverage.py`. It checks to see which lines of code are exercised by your tests and identifies the

sections that have no coverage. This quickly lets you figure out whether you're testing the code that you're about to optimize, such that any mistakes that might creep in during the optimization process are quickly caught.

No-op @profile Decorator

Your unit tests will fail with a `NameError` exception if your code uses `@profile` from `line_profiler` or `memory_profiler`. The reason is that the unit test framework will not be injecting the `@profile` decorator into the local namespace. The no-op decorator shown in this section solves this problem. It is easiest to add it to the block of code that you're testing and remove it when you're done.

With the no-op decorator, you can run your tests without modifying the code that you're testing. This means you can run your tests after every profile-led optimization you make so you'll never be caught out by a bad optimization step.

Let's say we have the trivial `ex.py` module shown in [Example 2-19](#). It has a test (for `pytest`) and a function that we've been profiling with either `line_profiler` or `memory_profiler`.

Example 2-19. Simple function and test case where we wish to use `@profile`

```
import time

def test_some_fn():
    """Check basic behaviors for our function"""
    assert some_fn(2) == 4
    assert some_fn(1) == 1
    assert some_fn(-1) == 1

@profile
def some_fn(useful_input):
    """An expensive function that we wish to both test and profile"""
    # artificial "we're doing something clever and expensive" delay
    time.sleep(1)
    return useful_input ** 2

if __name__ == "__main__":
    print(f"Example call `some_fn(2)` == {some_fn(2)}")
```

If we run `pytest` on our code, we'll get a `NameError`, as shown in [Example 2-20](#).

Example 2-20. A missing decorator during testing breaks out tests in an unhelpful way

```
$ pytest test_utility.py
== test session starts ==
```

```
platform linux -- Python 3.12.0, pytest-8.0.1, pluggy-1.4.0
rootdir: ../ch02/noop_profile_decorator
collected 0 items / 1 error
```

```
== ERRORS ==
__ ERROR collecting test_utility.py __
test_utility.py:1: in <module>
    from utility import some_fn
utility.py:20: in <module>
    @profile
E   NameError: name 'profile' is not defined. Did you forget to import 'profile'
== short test summary info ==
ERROR test_utility.py - NameError: name 'profile' is not defined.
Did you forget to import 'profile'
!! Interrupted: 1 error during collection !!
== 1 error in 0.12s ==
```

The solution is to add a no-op decorator at the start of our module (you can remove it after you're done with profiling). If the `@profile` decorator is not found in one of the namespaces (because `line_profiler` or `memory_profiler` is not being used), the no-op version we've written is added. If `line_profiler` or `memory_profiler` has injected the new function into the namespace, our no-op version is ignored.

For both `line_profiler` and `memory_profiler`, we can add the code in [Example 2-21](#).

Example 2-21. Adding a no-op `@profile` decorator to the namespace while unit testing

```
# check for line_profiler or memory_profiler in the local scope, both
# are injected by their respective tools or they're absent
# if these tools aren't being used (in which case we need to substitute
# a dummy @profile decorator)
if 'line_profiler' not in dir() and 'profile' not in dir():
    def profile(func):
        def inner(*args, **kwargs):
            return func(*args, **kwargs)
        return inner
```

Having added the no-op decorator, we can now run our pytest successfully, as shown in [Example 2-22](#), along with our profilers—with no additional code changes.

Example 2-22. With the no-op decorator, we have working tests, and both of our profilers work correctly

```
$ pytest utility.py
== test session starts ==
platform linux -- Python 3.12.0, pytest-8.0.1, pluggy-1.4.0
rootdir: ...
collected 1 item
```

```
utility.py . [100%]
```

```
== 1 passed in 3.01s ==
```

```
$ kernprof -l -v utility.py
Example call `some_fn(2)` == 4
Wrote profile results to utility.py.lprof
Timer unit: 1e-06 s
```

```
Total time: 1.00018 s
File: utility.py
Function: some_fn at line 20
```

```
Line # Hits Per Hit % Time Line Contents
=====
20 @profile
21 def some_fn(useful_input):
22     """An expensive function that we wish
        to both test and profile"""
23     # artificial 'we're doing something
        clever and expensive' delay
24     1 1e+06 100.0 time.sleep(1)
25     1 6.5 0.0 return useful_input ** 2
```

```
$ python -m memory_profiler utility.py
Example call `some_fn(2)` == 4
Filename: utility.py
```

```
Line # Mem usage Increment Line Contents
=====
20 46.898 MiB 46.898 MiB @profile
21 def some_fn(useful_input):
22     """An expensive function that we wish
        to both test and profile"""
23     # artificial 'we're doing something
        clever and expensive' delay
24 46.898 MiB 0.000 MiB time.sleep(1)
25 46.898 MiB 0.000 MiB return useful_input ** 2
```

You can save yourself a few minutes by avoiding the use of these decorators, but once you've lost hours making a false optimization that breaks your code, you'll want to integrate this into your workflow.

Strategies to Profile Your Code Successfully

Profiling requires some time and concentration. You will stand a better chance of understanding your code if you separate the section you want to test from the main body of your code. You can then unit test the code to preserve correctness, and you can pass in realistic fabricated data to exercise the inefficiencies you want to address.

Do remember to disable any BIOS-based accelerators, as they will only confuse your results. On Ian's laptop, the Intel Turbo Boost feature can temporarily accelerate a CPU above its normal maximum speed if it is cool enough. Running the Julia demo code from this chapter on a cold CPU with Turbo Boost enabled took 3.3 seconds. The CPU would temporarily have become hotter if the code had been run repeatedly, and as the CPU heated up, the execution time would have slowed. With Turbo Boost disabled, that same piece of code took 5.6 seconds. In this example with Turbo Boost enabled, a single run took 60% of the time compared to disabling it, and that gap would have closed as the CPU heated. Your operating system may also control the clock speed—a laptop on battery power is likely to more aggressively control CPU speed than a laptop on AC power.

You can imagine that this could confuse your benchmarking as you repeatedly run complex code! AMD has Turbo Core, a similar technology. This applies to laptops; server machines may simply run at a maximum fixed speed (trading off complexity for increased power consumption), so this might not apply to servers.

To create a more stable benchmarking configuration, we do the following:

- Disable Turbo Boost in the BIOS.
- Disable the operating system's ability to override the SpeedStep (you will find this in your BIOS if you're allowed to control it).
- Use only AC power (never battery power).
- Disable background tools like backups and Dropbox while running experiments.
- Run the experiments many times to obtain a stable measurement.
- Possibly drop to run level 1 (Unix) so that no other tasks are running.
- Reboot and rerun the experiments to double-confirm the results.

Try to hypothesize the expected behavior of your code and then validate (or disprove!) the hypothesis with the result of a profiling step. Your choices will not change (you should only drive your decisions by using the profiled results), but your intuitive understanding of the code will improve, and this will pay off in future projects, as you will be more likely to make performant decisions. Of course, you will verify these performant decisions by profiling as you go.

Do not skimp on the preparation. If you try to performance test code deep inside a larger project without separating it from the larger project, you are likely to witness side effects that will sidetrack your efforts. It is likely to be harder to unit test a larger project when you're making fine-grained changes, and this may further hamper your efforts. Side effects could include other threads and processes impacting CPU and memory usage and network and disk activity, which will skew your results.

Naturally, you're already using source code control (e.g., Git or Mercurial), so you'll be able to run multiple experiments in different branches without ever losing "the versions that work well." If you're *not* using source code control, do yourself a huge favor and start to do so!

For web servers, investigate `dowser` and `dozer`; you can use these to visualize in real time the behavior of objects in the namespace. Definitely consider separating the code you want to test out of the main web application if possible, as this will make profiling significantly easier.

Make sure your unit tests exercise all the code paths in the code that you're analyzing. Anything you don't test that is used in your benchmarking may cause subtle errors that will slow down your progress. Use `coverage.py` to confirm that your tests are covering all the code paths.

Unit testing a complicated section of code that generates a large numerical output may be difficult. Do not be afraid to output a text file of results to run through `diff` or to use a pickled object. For numeric optimization problems, Ian likes to create long text files of floating-point numbers and use `diff`—minor rounding errors show up immediately, even if they're rare in the output.

If your code might be subject to numerical rounding issues due to subtle changes, you are better off with a large output that can be used for a before-and-after comparison. One cause of rounding errors is the difference in floating-point precision between CPU registers and main memory. Running your code through a different code path can cause subtle rounding errors that might later confound you—it is better to be aware of these as soon as they occur.

Obviously, it makes sense to use a source code control tool while you are profiling and optimizing. Branching is cheap, and it will preserve your sanity.

Wrap-Up

Having looked at profiling techniques, you should have all the tools you need to identify bottlenecks around CPU and RAM usage in your code. Next, we'll look at how Python implements the most common containers, so you can make sensible decisions about representing larger collections of data.

Lists and Tuples

Questions You'll Be Able to Answer After This Chapter

- What are lists and tuples good for?
- What is the complexity of a lookup in a list/tuple?
- How is that complexity achieved?
- What are the differences between lists and tuples?
- How does appending to a list work?
- When should I use lists and tuples?

One of the most important things in writing efficient programs is understanding the guarantees of the data structures you use. In fact, a large part of performant programming is knowing what questions you are trying to ask of your data and picking a data structure that can answer these questions quickly. In this chapter we will talk about the kinds of questions that lists and tuples can answer quickly, and how they do it.

Lists and tuples are a class of data structures called *arrays*. An array is a flat list of data with some intrinsic ordering. Usually in these sorts of data structures, the relative ordering of the elements is as important as the elements themselves! In addition, this *a priori* knowledge of the ordering is incredibly valuable: by knowing that data in our array is at a specific position, we can retrieve it in $O(1)$ ¹ There are also many ways to

¹ $O(1)$ uses *Big O notation* to denote how efficient an algorithm is. A good introduction to the topic can be found in [this dev.to post by Sarah Chima](#) or in the introductory chapters of *Introduction to Algorithms* by Thomas H. Cormen et al. (MIT Press).

implement arrays, and each solution has its own useful features and guarantees. This is why in Python we have two types of arrays: lists and tuples. *Lists* are dynamic arrays that let us modify and resize the data we are storing, while *tuples* are static arrays whose contents are fixed and immutable.

Let's unpack these previous statements a bit. System memory on a computer can be thought of as a series of numbered buckets, each capable of holding a number. Python stores data in these buckets *by reference*, which means the number itself simply points to, or refers to, the data we actually care about. As a result, these buckets can store any type of data we want (as opposed to numpy arrays, which have a static type and can store only that type of data).²

When we want to create an array (and thus a list or tuple), we first have to allocate a block of system memory (where every section of this block will be used as an integer-sized pointer to actual data). This involves going to the system kernel and requesting the use of *N consecutive* buckets. **Figure 3-1** shows an example of the system memory layout for an array (in this case, a list) of size 6.



In Python, lists also store how large they are, so of the six allocated blocks, only five are usable—the zeroth element is the length.

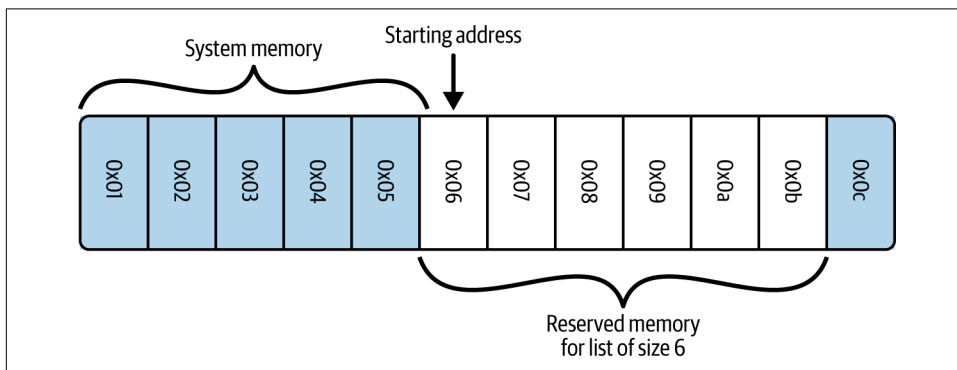


Figure 3-1. Example of system memory layout for an array of size 6

In order to look up any specific element in our list, we simply need to know which element we want and remember which bucket our data started in. Since all of the data will occupy the same amount of space (one “bucket,” or, more specifically, one

² In 64-bit computers, having 12 KB of memory gives you 725 buckets, and having 52 GB of memory gives you 3,250,000,000 buckets!

integer-sized pointer to the actual data), we don't need to know anything about the type of data that is being stored to do this calculation.



If you knew where in memory your list of N elements started, how would you find an arbitrary element in the list?

If, for example, we needed to retrieve the zeroth element in our array, we would simply go to the first bucket in our sequence, M , and read out the value inside it. If, on the other hand, we needed the fifth element in our array, we would go to the bucket at position $M + 5$ and read its content. In general, if we want to retrieve element i from our array, we go to bucket $M + i$. So, by having our data stored in consecutive buckets, and having knowledge of the ordering of our data, we can locate our data by knowing which bucket to look at in one step (or $O(1)$), regardless of how big our array is (Example 3-1).

Example 3-1. Timings for lookups in lists of different sizes

```
>>> %%timeit l = list(range(10))
...: l[5]
...:
14 ns ± 0.182 ns per loop (mean ± std. dev. of 7 runs, 100,000,000 loops each)

>>> %%timeit l = list(range(10_000_000))
...: l[100_000]
...:
13.9 ns ± 0.123 ns per loop (mean ± std. dev. of 7 runs, 100,000,000 loops each)
```

What if we were given an array with an unknown order and wanted to retrieve a particular element? If the ordering were known, we could simply look up that particular value. However, in this case, we must do a search operation. The most basic approach to this problem is called a *linear search*, where we iterate over every element in the array and check if it is the value we want, as seen in Example 3-2.

Example 3-2. A linear search through a list

```
def linear_search(needle, array):
    for i, item in enumerate(array):
        if item == needle:
            return i
    return -1
```

This algorithm has a worst-case performance of $O(n)$. This case occurs when we search for something that isn't in the array. In order to know that the element we

are searching for isn't in the array, we must first check it against every other element. Eventually, we will reach the final `return -1` statement. In fact, this algorithm is exactly the algorithm that `list.index()` uses.

The only way to increase the speed is by having some other understanding of how the data is placed in memory, or of the arrangement of the buckets of data we are holding. For example, hash tables ([“How Do Dictionaries and Sets Work?” on page 99](#)), which are a fundamental data structure powering dictionaries and sets, solve this problem in $O(1)$ by adding extra overhead to insertions/retrievals and enforcing a strict and peculiar sorting of the item. Alternatively, if your data is sorted so that every item is larger (or smaller) than its neighbor to the left (or right), then specialized search algorithms can be used that can bring your lookup time down to $O(\log n)$. This may seem like a huge performance penalty compared to the constant time *lookups* of lists and tuples; however, sometimes using these search algorithms is the best solution (especially since search algorithms are flexible and allow you to define searches in creative ways).

Exercise

Given the following data, write an algorithm to find the index of the value 61:

[9, 18, 18, 19, 29, 42, 56, 61, 88, 95]

Since you know the data is ordered, how can you do this faster?

Hint: If you split the array in half, you know all the values on the left are smaller than the smallest element in the right set. You can use this!

A More Efficient Search

As alluded to previously, we can achieve better search performance if we first sort our data so that all elements to the left of a particular item are smaller (or larger) than that item. The comparison is done through the `__eq__` and `__lt__` magic functions of the object and can be user-defined if using custom objects.



Without the `__eq__` and `__lt__` methods, a custom object will compare only to objects of the same type, and the comparison will be done using the instance's placement in memory. With those two magic functions defined, you can use the `functools.total_ordering` decorator from the standard library to automatically define all the other ordering functions, albeit at a small performance penalty.

The two ingredients necessary are the sorting algorithm and the searching algorithm. Python lists have a built-in sorting algorithm that uses Tim sort. Tim sort can sort through a list in $O(n)$ in the best case (and in $O(n \log n)$ in the worst case). It achieves this performance by utilizing multiple types of sorting algorithms and using heuristics to guess which algorithm will perform the best, given the data (more specifically, it hybridizes insertion and merge sort algorithms).

Once a list has been sorted, we can find our desired element using a binary search (Example 3-3), which has an average case complexity of $O(\log n)$. It achieves this by first looking at the middle of the list and comparing this value with the desired value. If this midpoint's value is less than our desired value, we consider the right half of the list, and we continue halving the list like this until the value is found, or until the value is known not to occur in the sorted list. As a result, we do not need to read all values in the list, as was necessary for the [linear search](#); instead, we read only a small subset of them.

Example 3-3. Efficient searching through a sorted list—binary search

```
def binary_search(needle, haystack):
    imin, imax = 0, len(haystack)
    while True:
        if imin > imax:
            return -1
        midpoint = (imin + imax) // 2
        if haystack[midpoint] > needle:
            imax = midpoint
        elif haystack[midpoint] < needle:
            imin = midpoint+1
        else:
            return midpoint
```

This method allows us to find elements in a list without resorting to the potentially heavyweight solution of a dictionary. This is especially true when the list of data that is being operated on is intrinsically sorted. It is more efficient to do a binary search on the list to find an object rather than first converting your data to a dictionary and then doing a single lookup on it. Although a dictionary lookup takes only $O(1)$, converting the data to a dictionary takes $O(n)$ (and a dictionary's restriction of no repeating keys may be undesirable). On the other hand, the binary search will take $O(\log n)$.

In addition, the `bisect` module from Python's standard library simplifies much of this process by giving easy methods to add elements into a list while maintaining its sorting, in addition to finding elements using a heavily optimized binary search. It does this by providing alternative functions that add the element into the correct sorted placement. With the list always being sorted, we can easily find the elements we are looking for (examples of this can be found in the [documentation for the](#)

`bisect` module). In addition, we can use `bisect` to find the closest element to what we are looking for very quickly (Example 3-4). This can be extremely useful for comparing two datasets that are similar but not identical.

Example 3-4. Finding close values in a list with the `bisect` module

```
import bisect
import random

def find_closest(haystack, needle):
    # bisect.bisect_left will return the first value in the haystack
    # that is greater than the needle
    i = bisect.bisect_left(haystack, needle)
    if i == len(haystack):
        return i - 1
    elif haystack[i] == needle:
        return i
    elif i > 0:
        j = i - 1
        # since we know the value is larger than the needle (and vice versa for
        # the value at j), we don't need to use absolute values here
        if haystack[i] - needle > needle - haystack[j]:
            return j
    return i

important_numbers = []
for i in range(10):
    new_number = random.randint(0, 1000)
    bisect.insort(important_numbers, new_number)

# important_numbers will already be in order because we inserted new elements
# with bisect.insort
print(important_numbers)
# > [14, 265, 496, 661, 683, 734, 881, 892, 973, 992]

closest_index = find_closest(important_numbers, -250)
print(f"Closest value to -250: {important_numbers[closest_index]}")
# > Closest value to -250: 14

closest_index = find_closest(important_numbers, 500)
print(f"Closest value to 500: {important_numbers[closest_index]}")
# > Closest value to 500: 496

closest_index = find_closest(important_numbers, 1100)
print(f"Closest value to 1100: {important_numbers[closest_index]}")
# > Closest value to 1100: 992
```

In general, this touches on a fundamental rule of writing efficient code: pick the right data structure and stick with it! Although there may be more efficient data structures

for particular operations, the cost of converting to those data structures may negate any efficiency boost.

Lists Versus Tuples

If lists and tuples both use the same underlying data structure, what are the differences between the two? Summarized, the main differences are as follows:

- Lists are *dynamic* arrays; they are mutable and allow for resizing (changing the number of elements that are held).
- Tuples are *static* arrays; they are immutable, and the data they reference cannot change once the tuple has been created.³
- Tuples are cached by the Python runtime, which means that we don't need to talk to the kernel to reserve memory every time we want to use one.

These differences outline the philosophical difference between the two: tuples are for describing multiple properties of one unchanging thing, and lists can be used to store collections of data about completely disparate objects. For example, the parts of a telephone number are perfect for a tuple: they won't change, and if they do, they represent a new object or a different phone number. Similarly, the coefficients of a polynomial fit a tuple, since different coefficients represent a different polynomial. On the other hand, the names of the people currently reading this book are better suited for a list: the data is constantly changing both in content and in size but is still always representing the same idea.

It is important to note that both lists and tuples can take mixed types. This can, as you will see, introduce quite a bit of overhead and reduce some potential optimizations. This overhead can be removed if we force all our data to be of the same type. In [Chapter 6](#), we will talk about reducing both the memory used and the computational overhead by using `numpy`. In addition, tools like the standard library module `array` can reduce these overheads for other, nonnumerical situations. This alludes to a major point in performant programming that we will touch on in later chapters: generic code will be much slower than code specifically designed to solve a particular problem.

In addition, the immutability of a tuple as opposed to a list, which can be resized and changed, makes it a lightweight data structure. This means that there isn't much overhead in memory when storing tuples, and operations with them are quite straightforward. With lists, as you will learn, their mutability comes at the price of extra memory needed to store them and extra computations needed when using them.

³ Note that it is the *reference* that cannot change; however, we can still do in-place operations on the object that change its content without changing the reference.

Exercise

For the following example datasets, would you use a tuple or a list? Why?

1. First 20 prime numbers
2. Names of programming languages
3. A person's age, weight, and height
4. A person's birthday and birthplace
5. The result of a particular game of pool
6. The results of a continuing series of pool games

Solution:

1. Tuple, since the data is static and will not change.
2. List, since this dataset is constantly growing.
3. List, since the values will need to be updated.
4. Tuple, since that information is static and will not change.
5. Tuple, since the data is static.
6. List, since more games will be played. (In fact, we could use a list of tuples since each individual game's results will not change, but we will need to add more results as more games are played.)

Lists as Dynamic Arrays

Once we create a list, we are free to change its contents as needed:

```
>>> numbers = [5, 8, 1, 3, 2, 6]
>>> numbers[2] = 2 * numbers[0] ❶
>>> numbers
[5, 8, 10, 3, 2, 6]
```

- ❶ As described previously, this operation is $O(1)$ because we can find the data stored within the zeroth and second elements immediately.

In addition, we can append new data to a list and grow its size:

```
>>> len(numbers)
6
>>> numbers.append(42)
>>> numbers
[5, 8, 10, 3, 2, 6, 42]
>>> len(numbers)
7
```

This is possible because dynamic arrays support a `resize` operation that increases the capacity of the array. When a list of size N is first appended to, Python must create a new list that is big enough to hold the original N items in addition to the extra one that is being appended. However, instead of allocating exactly $N + 1$ items, M items are actually allocated, where $M > N$, in order to provide extra headroom for future appends. Then the data from the old list is copied to the new list, and the old list is destroyed.

The philosophy is that one append is probably the beginning of many appends, and by requesting extra space, we can reduce the number of total times this allocation must happen and thus the total number of memory copies that are necessary. This is important since memory copies can be quite expensive, especially when list sizes start growing. [Figure 3-2](#) shows what this overallocation looks like in Python 3.12. The formula dictating this growth is given in [Example 3-5](#).⁴

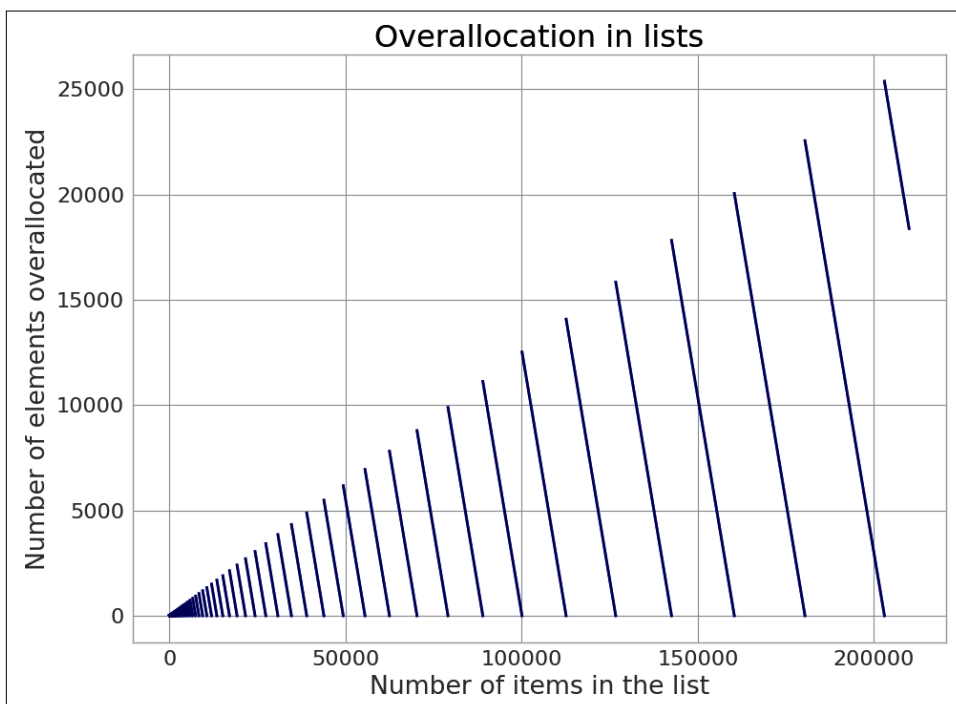


Figure 3-2. Graph showing how many extra elements are being allocated to a list of a particular size. For example, if you create a list with 1,000,000 elements using appends, Python will allocate space for 1,056,084 elements, overallocating 56,084 elements!

⁴ The code responsible for this overallocation can be seen in the Python source code in [Objects/listobject.c:list_resize](#).

Example 3-5. List allocation equation in Python 3.12.2

```
M = (N + (N >> 3) + 6) & ~3
```

As we append data, we utilize the extra space and increase the effective size of the list, N . As a result, N grows as we append new data, until $N == M$. At this point, there is no extra space to insert new data into, and we must create a *new* list with more extra space. This new list has extra headroom as given by the equation in [Example 3-5](#), and we copy the old data into the new space. In using this method, Python generally overallocates about 12.5% of the list space in order to reduce the time of list appends. However, for small lists this can be much higher—for a list with 1 item, 4 are allocated, and for 9 items, 16 are allocated!

This mechanism of copying, overallocating, and appending is shown visually in [Figure 3-3](#). The figure follows the various operations being performed on list `l` in [Example 3-6](#).

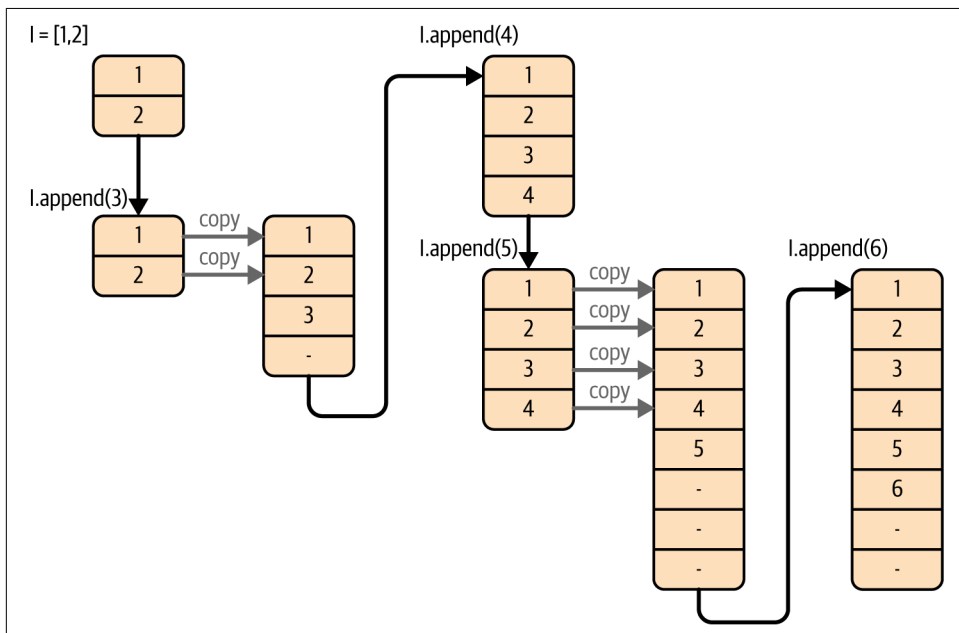


Figure 3-3. Example of how the list in [Example 3-6](#) is being mutated on multiple appends

Example 3-6. Resizing a list

```
l = [1, 2]
for i in range(3, 7):
    l.append(i)
```




In order to keep potentially static lists small, the overallocation only happens on the first append. When a list is directly created, as in the preceding example, only the number of elements needed is allocated.

While the amount of extra headroom allocated is generally quite small, it can add up. In [Example 3-7](#), we can see that for cases where we have many small lists, each with an overhead, this can quickly balloon into a considerable usage of resources.

Example 3-7. Memory consequences of overallocation

```
import sys
import random

def total_size(obj):
    """
    Recursively calculates the total size of a Python object in memory,
    including its contents.

    Returns:
        int: The total size of the object in bytes.
    """
    children = 0
    try:
        children = sum(total_size(item) for item in obj)
    except TypeError:
        pass
    return sys.getsizeof(obj) + children

def sample_comp(a, b, N):
    return [random.randint(a, b) for _ in range(N)] ❶

def sample_list(a, b, N):
    return list([random.randint(a, b) for _ in range(N)]) ❷

N_samples = 1_000_000
sample_size = 9
data_comp = [sample_comp(0, 100, sample_size) for _ in range(N_samples)]
data_list = [sample_list(0, 100, sample_size) for _ in range(N_samples)]

size_comp = max_size = total_size(data_comp) / 1e6
size_list = total_size(data_list) / 1e6

print(f"Creating {N_samples:,d} samples of {sample_size} items each")
print(f"Data Comprehension size: {size_comp:0.2f} Mb")
print(f"Data List size: {size_list:0.2f} Mb ({max_size/size_list:0.2f}x smaller)")
```

```
# Outputs:
#   Creating 1,000,000 samples of 9 items each
#   Data Comprehension size: 444.45 Mb
#   Data List size: 396.45 Mb (1.12x smaller)
```

- ❶ We create N random integers from a to b. This “sample” of data can be anything in the real world—the last five links a user clicked on or the names of files in a directory. Since it is created via list comprehension, however, it has been overallocated.
- ❷ Here we do the same as before; however, once the data has been created, we pass it to `list` in order to re-create the list but without any overallocation.

We can see in the previous example how quickly this overallocation overhead can add up. In this fairly common example of having many small lists, we incurred a 1.12 times increase in memory, which corresponds to 48 Mb of memory wasted. When doing data processing, memory can be your most valuable resource, and this simple change can greatly help your overall memory use. We’ll soon see how we can bring this number down even more with tuples (and in [Chapter 12](#) we’ll go even further into this topic).

Tuples as Static Arrays

Tuples are fixed and immutable. This means that once a tuple is created, unlike a list, it cannot be modified or resized:

```
>>> t = (1, 2, 3, 4)
>>> t[0] = 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

However, although they don’t support resizing, we can concatenate two tuples together and form a new tuple. The operation is similar to the `resize` operation on lists, but we do not overallocate space for the resulting tuple:

```
>>> t1 = (1, 2, 3, 4)
>>> t2 = (5, 6, 7, 8)
>>> t1 + t2
(1, 2, 3, 4, 5, 6, 7, 8)
```

If we consider this comparable to the `append` operation on lists, we see that it performs in $O(n)$ as opposed to the $O(1)$ speed of lists. This is because we must allocate and copy the entire tuple every time something is added to it, as opposed to only when our extra headroom ran out for lists. As a result, there’s no in-place

append-like operation; adding two tuples always returns a new tuple that's in a new location in memory.

Not storing the extra headroom for resizing has the advantage of using fewer resources. A list of size 100,000,000 created with any append operation actually uses 104,391,068 elements' worth of memory, while a tuple holding the same data will only ever use exactly 100,000,000 elements' worth of memory (a 4.4% saving in memory). This makes tuples lightweight and preferable when data becomes static.

Furthermore, even if we create a list *without* append (and thus we don't have the extra headroom introduced by an append operation), it will *still* be larger in memory than a tuple with the same data. This is because lists have to keep track of more information about their current state in order to efficiently resize. While this extra information is quite small (the equivalent of one extra element), it can add up if several million lists are in use.

We can see this by adding tuples to [Example 3-7](#). In [Example 3-8](#), we add the corresponding code and see that our memory use is 1.19 times smaller than the original list comprehension route. This is a total savings of 72 Mb, achieved simply by casting our data to a tuple at the cost of the data being immutable.

Example 3-8. Memory consequences of overallocation

```
def sample_tuple(a, b, N):
    return tuple([random.randint(a, b) for _ in range(N)])

data_tuple = [sample_tuple(0, 100, sample_size) for _ in range(N_samples)]

print(f"Data Tuple size: {size_tuple:0.2f} Mb "
      f"({max_size/size_tuple:0.2f}x smaller)")

# Outputs:
#   Creating 1,000,000 samples of 9 items each
#   Data Comprehension size: 444.45 Mb
#   Data List size: 396.45 Mb (1.12x smaller)
#   Data Tuple size: 372.45 Mb (1.19x smaller)
```

Another benefit of the static nature of tuples is something Python does in the background: resource caching. Python is garbage collected, which means that when a variable isn't used anymore, Python frees the memory used by that variable, giving it back to the operating system for use in other applications (or for other variables). For tuples of sizes 0–20, however, when they are no longer in use, the space isn't immediately given back to the system: up to 2,000 of each size of tuple are saved for future use. This means that when a new tuple of that size is needed in the future, we don't need to communicate with the operating system to find a region in memory to

put the data into, since we have a reserve of free memory already. However, this also means that the Python process will have some extra memory overhead.



Other objects also are cached by Python (a mechanism called a “freelist”); however, the use case is different. When an object is destroyed in your Python code and set to be garbage collected, Python keeps it around in the freelist so that new objects can be instantiated and allocated faster. However, for non-tuple objects these freelists are considerably smaller than the tuple freelist, and the performance benefits are mainly geared toward the inner workings of the Python interpreter. For CPython 3.12.2, the size of the freelist for various objects is:

- Generators: 80
- Contexts: 255
- Dictionaries: 80
- Floats: 100
- Lists: 80
- Tuples: 40,000 (2,000 of each size of tuple from 1–20)

While this may seem like a small benefit, it is one of the fantastic things about tuples: they can be created easily and quickly since they can avoid communications with the operating system, which could cost your program quite a bit of time. **Example 3-9** shows that instantiating a list can be 7.6 times slower than instantiating a tuple—which can add up quickly if this is done in a fast loop!

Example 3-9. Instantiation timings for lists versus tuples

```
>>> %timeit l = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
77.6 ns ± 0.53 ns per loop (mean ± std. dev. of 7 runs, 10,000,000 loops each)

>>> %timeit t = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
9.47 ns ± 0.0681 ns per loop (mean ± std. dev. of 7 runs, 100,000,000 loops each)
```

Wrap-Up

Lists and tuples are fast and low-overhead objects to use when your data already has an intrinsic ordering to it. This intrinsic ordering allows you to sidestep the search problem in these structures: if the ordering is known beforehand, lookups are $O(1)$, and searches can be done in $O(\log n)$. If no order is known beforehand, we must do an expensive $O(n)$ linear search. While lists can be resized, you must take care to properly understand how much overallocation is happening to ensure that the dataset can still fit in memory. On the other hand, tuples can be created quickly and without the added overhead of lists, at the cost of not being modifiable. In “[Aren’t Python Lists Good Enough?](#)” on page 131, we discuss how to preallocate lists to alleviate some of the burden regarding frequent appends to Python lists, and we look at other optimizations that can help manage these problems.

In the next chapter, we go over the computational properties of dictionaries, which solve the search/lookup problems with unordered data at the cost of overhead.

Dictionaries and Sets

Questions You'll Be Able to Answer After This Chapter

- What are dictionaries and sets good for?
- How are dictionaries and sets the same?
- What is the overhead when using a dictionary?
- How can I optimize the performance of a dictionary?
- How does Python use dictionaries to keep track of namespaces?

Sets and dictionaries are ideal data structures to be used when your data has no intrinsic order (except for insertion order) but does have a unique object that can be used to reference it (the reference object is normally a string, but it can be any hashable type). This reference object is called the *key*, while the data is the *value*. Dictionaries and sets are almost identical, except that sets do not actually contain values: a set is simply a collection of unique keys. As the name implies, sets are very useful for doing set operations (such as union, intersection, and difference).



A *hashable* type is one that implements both the `__hash__` magic function and either `__eq__` or `__cmp__`. All native types in Python already implement these, and any user classes have default values. See “[Hash Functions and Entropy](#)” on page 106 for more details.

We saw in the previous chapter that for lists with no intrinsic order we are limited to $O(n)$ lookup time. Dictionaries and sets give us $O(1)$ lookups based on the arbitrary index. In addition, like lists/tuples, dictionaries and sets have $O(1)$ insertion time.¹ As we will see in “How Do Dictionaries and Sets Work?” on page 99, this speed is accomplished through the use of an open address hash table as the underlying data structure.

However, there is a cost to using dictionaries and sets. First, they generally take up a larger footprint in memory. Also, although the complexity for insertions/lookups is $O(1)$, the actual speed depends greatly on the hashing function that is in use. If the hash function is slow to evaluate, any operations on dictionaries or sets will be similarly slow.

Let’s look at an example. Say we want to store contact information for everyone in the phone book. We would like to store this in a form that will make it simple to answer the question “What is Ada Lovelace’s phone number?” in the future. With lists, we would store the phone numbers and names sequentially and scan through the entire list to find the phone number we required, as shown in Example 4-1.

Example 4-1. Phone book lookup with a list

```
def find_phonenumber(phonebook, name):
    for n, p in phonebook:
        if n == name:
            return p
    return None

phonebook = [
    ("Ada Lovelace", "555-555-5555"),
    ("Sophie Wilson", "212-555-5555"),
]

ada = find_phonenumber(phonebook, 'Ada Lovelace')
print(f"Ada Lovelace's phone number is {ada}")
```



We could also do this by sorting the list (at a $O(n \log n)$ penalty) and using the `bisect` module (from Example 3-4) in order to get $O(\log n)$ performance on subsequent lookups.

¹ As we will discuss in “Hash Functions and Entropy” on page 106, dictionaries and sets are very dependent on their hash functions. If the hash function for a particular datatype is not $O(1)$, any dictionary or set containing that type will no longer have its $O(1)$ guarantee.

With a dictionary, however, we can simply have the “index” be the names and the “values” be the phone numbers, as shown in [Example 4-2](#). This allows us to simply look up the value we need and get a direct reference to it, instead of having to read every value in our dataset.

Example 4-2. Phone book lookup with a dictionary

```
phonebook = {  
    "Ada Lovelace": "555-555-5555",  
    "Sophie Wilson" : "212-555-5555",  
}  
print(f"Ada Lovelace's phone number is {phonebook['Ada Lovelace']}")
```

For large phone books, the difference between the $O(1)$ lookup of the dictionary and the $O(n)$ time for linear search over the list (or, at best, the $O(\log n)$ complexity with the `bisect` module) is quite substantial.



Create a script that times the performance of the list-bisect method versus a dictionary for finding a number in a phone book. How does the timing scale as the size of the phone book grows?

If, on the other hand, we wanted to answer the question “How many unique first names are there in my phone book?” we could use the power of sets. Recall that a set is simply a collection of *unique* keys—this is the exact property we would like to enforce in our data. This is in stark contrast to a list-based approach, where that property needs to be enforced separately from the data structure by comparing all names with all other names. [Example 4-3](#) illustrates.

Example 4-3. Finding unique names with lists and sets

```
def list_unique_first_names(phonebook):  
    unique_first_names = []  
    for name, phonenumber in phonebook: ❶  
        first_name, last_name = name.split(" ", 1)  
        for unique in unique_first_names: ❷  
            if unique == first_name:  
                break  
        else:  
            unique_first_names.append(first_name)  
    return len(unique_first_names)  
  
def set_unique_first_names(phonebook):  
    unique_first_names = set()  
    for name, phonenumber in phonebook: ❸  
        first_name, last_name = name.split(" ", 1)
```

```

        unique_first_names.add(first_name) ❹
    return len(unique_first_names)

phonebook = [
    ("Ada Lovelace", "555-555-5555"),
    ("Sophie Wilson", "212-555-5555"),
    ("Grace Hopper", "647-555-5555"),
    ("Emmy Noether", "202-555-5555"),
    ("Guido van Rossum", "301-555-5555"),
]

print("Number of unique names from set method:", set_unique_first_names(phonebook))
print("Number of unique names from list method:", list_unique_first_names(phonebook))

```

- ❶, ❸ We must go over all the items in our phone book, and thus this loop costs $O(n)$.
- ❷ Here, we must check the current name against all the unique names we have already seen. If it is a new unique name, we add it to our list of unique names. We then continue through the list, performing this step for every item in the phone book.
- ❹ For the set method, instead of iterating over all unique names we have already seen, we can simply add the current name to our set of unique names. Because sets guarantee the uniqueness of the keys they contain, if you try to add an item that is already in the set, that item simply won't be added. Furthermore, this operation costs $O(1)$.

The list algorithm's inner loop iterates over `unique_first_names`, which starts out as empty and then grows, in the worst case—that is, when all names are unique—to be the size of `phonebook`. This can be seen as performing a **linear search** for each name in the phone book over a list that is constantly growing. Thus, the complete algorithm performs as $O(n \log n)$.

On the other hand, the set algorithm has no inner loop; the `set.add` operation is an $O(1)$ process that completes in a fixed number of operations regardless of how large the phone book is (there are some minor caveats to this, which we will cover while discussing the implementation of dictionaries and sets). Thus, the only nonconstant contribution to the complexity of this algorithm is the loop over the phone book, making this algorithm perform in $O(n)$.

When timing these two algorithms using a phonebook with 10,000 entries and all unique first names, we see how drastic the difference between $O(n)$ and $O(n \log n)$ can be:

```
>>> %timeit list_unique_first_names(large_phonebook)
1.3 s ± 8.83 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

>>> %timeit set_unique_first_names(large_phonebook)
2.32 ms ± 55.6 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

In other words, the set algorithm gave us a 560 times speedup! In addition, as the size of the phonebook grows, the speed gains increase (we get a 4,300 times speedup with a phonebook with 100,000 entries).

How Do Dictionaries and Sets Work?

Dictionaries and sets use *hash tables* to achieve their $O(1)$ lookups and insertions. This efficiency is the result of a very clever usage of a **hash function** to turn an arbitrary key (i.e., a string or object) into an index for a list. The hash function and list can later be used to determine where any particular piece of data is right away, without a search. By turning the data's key into something that can be used like a list index, we can get the same performance as with a list. In addition, instead of having to refer to data by a numerical index, which itself implies some ordering to the data, we can refer to it by this arbitrary key.



In this chapter we focus on the implementation details of dictionaries; however, sets are very similar. Algorithmically they work in the same way, but several small details are different (for example, the exact growth pattern, the existence of an ordered list of keys, etc.). We focus on dictionaries and point out differences to set objects when necessary.

Inserting and Retrieving

To create a hash table from scratch, we start with some allocated memory, similar to what we started with for arrays. For an array, if we want to insert data, we simply find the first unused bucket and insert our data there (and resize if necessary). For hash tables, we must first figure out the placement of the data in this contiguous chunk of memory.

The placement of the new data is contingent on two properties of the data we are inserting: the hashed value of the key and how the value compares to other objects. This is because when we insert data, the key is first hashed and masked so that it turns into an effective index in an array.² The mask makes sure that the hash value,

² A *mask* is a binary number that truncates the value of a number. So $0b11111101 \ \& \ 0b111 = 0b101 = 5$ represents the operation of $0b111$ masking the number $0b11111101$. This operation can also be thought of as taking a certain number of the least-significant digits of a number.

which can take the value of any integer, fits within the allocated number of buckets. So if we have allocated 8 blocks of memory and our hash value is 28975, we consider the bucket at index $28975 \ \& \ 0b111 = 7$. If, however, our dictionary has grown to require 512 blocks of memory, the mask becomes $0b11111111$ (and in this case, we would consider the bucket at index $28975 \ \& \ 0b11111111 = 47$).

Now we must check if this bucket is already in use. If it is empty, we can insert the key and the value into this block of memory. We store the key so that we can make sure we are retrieving the correct value on lookups. If it is in use and the value of the bucket is equal to the value we wish to insert (a comparison done with the `cmp` built-in), then the key/value pair is already in the hash table and we can return. However, if the values don't match, we must find a new place to put the data.



As an extra optimization for dictionaries, Python appends the key/value data into a standard array and then stores only the *index* into this array in the hash table. This allows us to reduce the amount of memory used by 30–95%.³ In addition, this gives us the interesting property that we keep a record of the order in which new items were added into the dictionary (which, since Python 3.7, is a guarantee that all dictionaries give).

To find the new index, we compute it using a simple linear function, a method called *probing*. Python's probing mechanism adds a contribution from the higher-order bits of the original hash (recall that for a table of length 8 we considered only the last three bits of the hash for the initial index, through the use of a mask value of `mask = 0b111 = bin(8 - 1)`). Using these higher-order bits gives each hash a different sequence of next possible hashes, which helps to avoid future collisions.

There is a lot of freedom when picking the algorithm to generate a new index; however, it is quite important that the scheme visits every possible index in order to evenly distribute the data in the table. How well distributed the data is throughout the hash table is called the *load factor* and is related to the **entropy** of the hash function. The pseudocode in **Example 4-4** illustrates the calculation of hash indices used in CPython 3.7. This also points toward an interesting fact about hash tables: most of the storage space they have is empty!

³ The discussion that led to this improvement can be found at <https://oreil.ly/Pq7Lm>.

Example 4-4. Dictionary lookup sequence

```
def index_sequence(key, mask=0b111, PERTURB_SHIFT=5):
    perturb = hash(key) ❶
    i = perturb & mask
    yield i
    while True:
        perturb >>= PERTURB_SHIFT
        i = (i * 5 + perturb + 1) & mask
        yield i
```

- ❶ hash returns an integer, while the actual C code in CPython uses an unsigned integer. Because of that, this pseudocode doesn't replicate exactly the behavior in CPython; however, it is a good approximation.

This probing is a modification of the naive method of *linear probing*. In linear probing, we simply yield the values $i = (i * 5 + \text{perturb} + 1) \& \text{mask}$, where i is initialized to the hash value of the key.⁴ An important thing to note is that linear probing deals only with the last several bits of the hash and disregards the rest (i.e., for a dictionary with eight elements, we look only at the last three bits since at that point the mask is 0×111). This means that if hashing two items gives the same last three binary digits, then not only will we have a collision, but also the sequence of probed indices will be the same. The perturbed scheme that Python uses will start taking into consideration more bits from the items' hashes to resolve this problem.

A similar procedure is done when we are performing lookups on a specific key: the given key is transformed into an index, and that index is examined. If the key in that index matches (recall that we also store the original key when doing insert operations), then we can return that value. If it doesn't, we keep creating new indices using the same scheme, until we either find the data or hit an empty bucket. If we hit an empty bucket, we can conclude that the data does not exist in the table.

Figure 4-1 illustrates the process of adding data into a hash table. Here, we chose to create a hash function that simply uses the first letter of the input. We accomplish this by using Python's `ord` function on the first letter of the input to get the integer representation of that letter (recall that hash functions must return integers). As we'll see in "Hash Functions and Entropy" on page 106, Python provides hashing functions for most of its types, so typically you won't have to provide one yourself except in extreme situations.

⁴ The value of 5 comes from the properties of a linear congruential generator (LCG), which is used in generating random numbers.

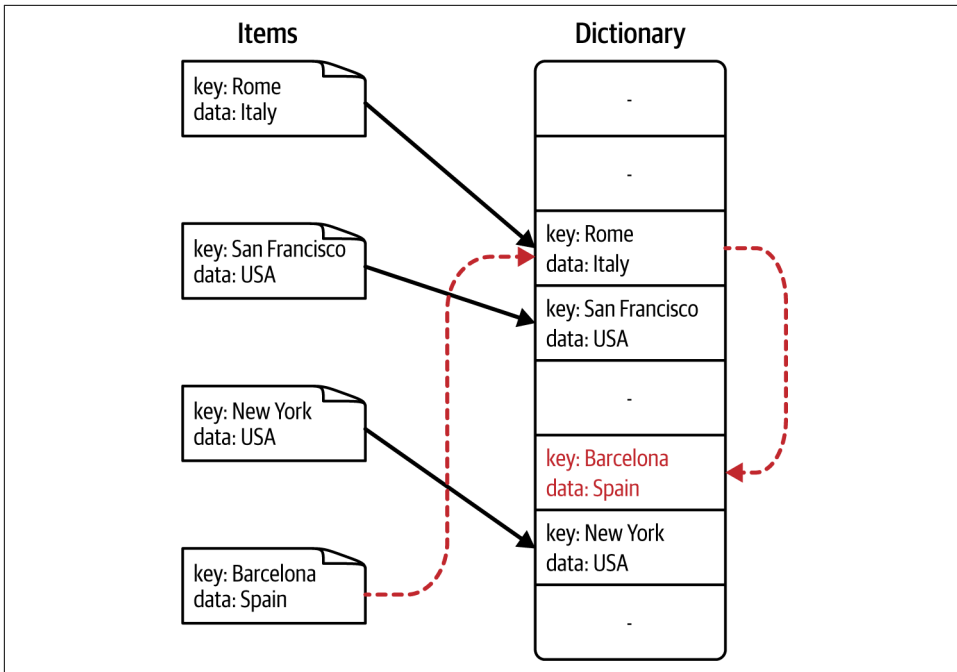


Figure 4-1. The resulting hash table from inserting with collisions

Insertion of the key Barcelona causes a collision, and a new index is calculated using the scheme in [Example 4-4](#). This dictionary can also be created in Python using the code in [Example 4-5](#).

Example 4-5. Custom hashing function

```
class City(str):
    def __hash__(self):
        return ord(self[0])

# We create a dictionary where we assign arbitrary values to cities
data = {
    City("Rome"): 'Italy',
    City("San Francisco"): 'USA',
    City("New York"): 'USA',
    City("Barcelona"): 'Spain',
}
```

In this case, Barcelona and Rome cause the hash collision (Figure 4-1 shows the outcome of this insertion). We see this because, for a dictionary with four elements, we have a mask value of 0b111. As a result, Barcelona and Rome will try to use the same index:

```
hash("Barcelona") = ord("B") & 0b111
                   = 66 & 0b111
                   = 0b1000010 & 0b111
                   = 0b010 = 2
```

```
hash("Rome") = ord("R") & 0b111
              = 82 & 0b111
              = 0b1010010 & 0b111
              = 0b010 = 2
```

Exercise

Work through the following problems; a discussion of hash collisions follows:

1. Finding an element

Using the dictionary created in Example 4-5, what would a lookup on the key Johannesburg look like? What indices would be checked?

2. Deleting an element

Using the dictionary created in Example 4-5, how would you handle the deletion of the key Rome? How would subsequent lookups for the keys Rome and Barcelona be handled?

3. Hash collisions

Considering the dictionary created in Example 4-5, how many hash collisions could you expect if 500 cities, with names all starting with an uppercase letter, were added into a hash table? How about 1,000 cities? Can you think of a way of lowering the number of collisions?

For 500 cities, there would be approximately 474 dictionary elements that collided with a previous value ($500 - 26$), with each hash having $500 / 26 = 19.2$ cities associated with it. For 1,000 cities, 974 elements would collide, and each hash would have $1,000 / 26 = 38.4$ cities associated with it. This is because the hash is based simply on the numerical value of the first letter, which can take only a value from A to Z, allowing for only 26 independent hash values. This means that a lookup in this table could require as many as 38 subsequent lookups to find the correct value. To fix this, we must increase the number of possible hash values by considering other aspects of the city in the hash. The default hash function on a string considers every character in order to maximize the number of possible values. See “Hash Functions and Entropy” on page 106 for more explanation.

Deletion

When a value is deleted from a hash table, we cannot simply write a NULL to that bucket of memory. This is because we have used NULLs as a sentinel value while probing for hash collisions. As a result, we must write a special value that signifies that the bucket is empty, but there still may be values after it to consider when resolving a hash collision. So if “Rome” was deleted from the dictionary, subsequent lookups for “Barcelona” will first see this sentinel value where “Rome” used to be and, instead of stopping, continue to check the next indices given by the `index_sequence`. These empty slots can be written to in the future and are removed when the hash table is resized.

Resizing

As more items are inserted into the hash table, the table itself must be resized to accommodate them. It can be shown that a table that is no more than two-thirds full will have optimal space savings while still having a good bound on the number of collisions to expect.⁵ Thus, when a table reaches this critical point, it is grown. To do this, a larger table is allocated (i.e., more buckets in memory are reserved), the mask is adjusted to fit the new table, and all elements of the old table are reinserted into the new one. This requires recomputing indices, since the changed mask will change the resulting index. As a result, resizing large hash tables can be quite expensive! However, since we do this resizing operation only when the table is too small, as opposed to doing it on every insert, the amortized cost of an insert is still $O(1)$.⁶

The general sizing rules for dictionaries (and sets) are as follows:

1. The dictionary is always less than two-thirds full (three-fifths full for sets).
2. The size of the dictionary is always a power of two.

By default, the smallest size of a dictionary or set is eight (that is, if you are storing only three values, Python will still allocate eight buckets), and it will resize by three times if the dictionary is more than two-thirds full.⁷ So when trying to insert a sixth element, the dictionary will first realize that this will cause it to be overallocated (more than two-thirds full). To calculate its new size, it finds the smallest power of two that will accommodate three times its current size (see [Figure 4-2](#) to see how this scales as the dictionary gets bigger). So in order to hold 15 items, we should resize

⁵ W. D. Maurer and T. G. Lewis, “Hash Table Methods,” *ACM Computing Surveys* 7, no. 1 (March 1975): 5–19.

⁶ Amortized analysis looks at the average complexity of an algorithm. This means that some inserts will be much more expensive, but on average, inserts will be $O(1)$.

⁷ For sets, the limit is three-fifths full, and the resize factor is reduced to two times once the set has grown to at least 50,000 elements.

the dictionary to hold 16 items. At this point, the old data can be copied into the new dictionary, and the new element can be added as well.

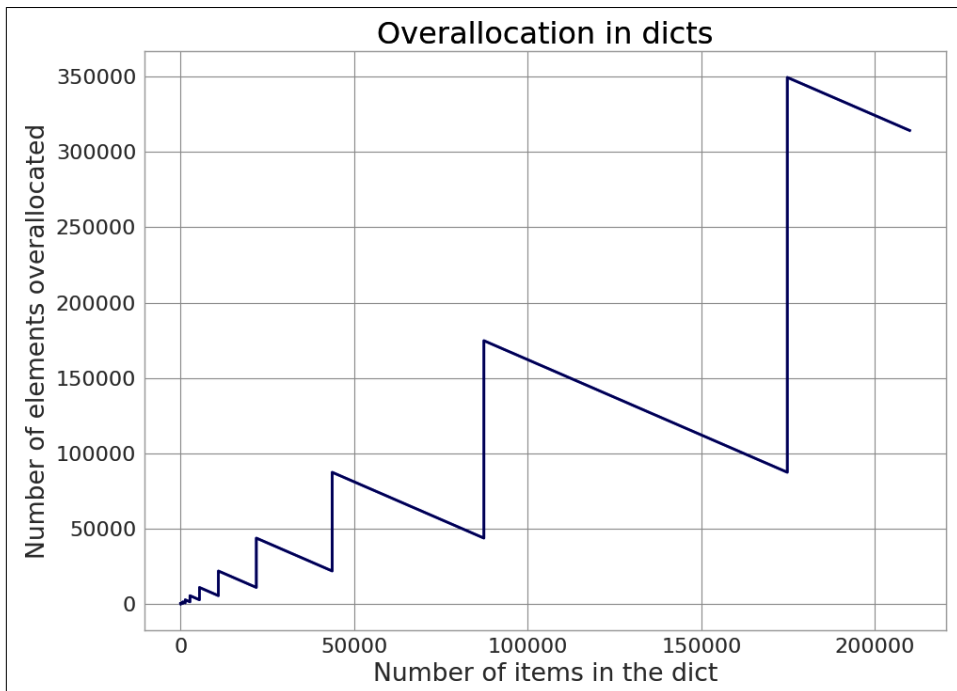


Figure 4-2. Graph showing how many extra elements are being allocated to accommodate a dictionary of a given size. Dictionaries will always be at least one-third empty but will also have even more space allocated to make sure the total size is a power of two. The graph looks similar for sets; however, the rate of increase changes at the 50,000-element mark.

On the 11th insertion, this process happens again. We try resizing to 32 elements (the smallest power of two that holds 3×10 elements). This gives the following possible sizes (which are the powers of two!):

8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, ...

It is important to note that resizing can happen to make a hash table larger *or* smaller. That is, if sufficiently many elements of a hash table are deleted, the table can be scaled down in size. However, *resizing happens only during an insert*. So, counterintuitively, if you have a dictionary that you `dict.pop()` many items out of, you can trigger a resize (and thus save memory!) by adding a new element to the dictionary.

Hash Functions and Entropy

Objects in Python are generally hashable, since they already have built-in `__hash__` and `__cmp__` functions associated with them. For numerical types (`int` and `float`), the hash is based simply on the bit value of the number they represent. Tuples and strings have a hash value that is based on their contents. Lists, on the other hand, do not support hashing because their values can change. Since a list's values can change, so could the hash that represents the list, which would change the relative placement of that key in the hash table.⁸

User-defined classes also have default hash and comparison functions. The default `__hash__` function simply returns the object's placement in memory as given by the built-in `id` function. Similarly, the `__cmp__` operator compares the numerical value of the object's placement in memory.

This is generally acceptable, since two instances of a class are generally different and should not collide in a hash table. However, in some cases we would like to use `set` or `dict` objects to disambiguate between items. Take the following class definition:

```
class Point(object):
    def __init__(self, x, y):
        self.x, self.y = x, y
```

If we were to instantiate multiple `Point` objects with the same values for `x` and `y`, they would all be independent objects in memory and thus have different placements in memory, which would give them all different hash values. This means that putting them all into a `set` would result in all of them having individual entries:

```
>>> p1 = Point(1,1)
>>> p2 = Point(1,1)
>>> set([p1, p2])
set([<__main__.Point at 0x1099bfc90>, <__main__.Point at 0x1099bfd0>])
>>> Point(1,1) in set([p1, p2])
False
```

We can remedy this by forming a custom hash function that is based on the actual contents of the object as opposed to the object's placement in memory. The hash function can be any function as long as it consistently gives the same result for the same object. (There are also considerations regarding the entropy of the hashing function, which we will discuss later.) [Example 4-6](#) redefines the `Point` class, giving it a content-based hashing function, yielding the results we expect (as seen in [Example 4-7](#)).

⁸ More information about this can be found at <https://oreil.ly/g4I5->.

Example 4-6. Definition of a Point object with a custom hash function

```
class Point(object):
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __hash__(self):
        return hash((self.x, self.y)) ❶

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
```

- ❶ Recall that the `__hash__` function gives us the hash for any hashable Python object.

This allows us to create entries in a set or dictionary indexed by the properties of the `Point` object rather than the memory address of the instantiated object, as shown in [Example 4-7](#).

Example 4-7. Use of Point object with a custom hash function

```
>>> p1 = Point(1,1)
>>> p2 = Point(1,1)
>>> set([p1, p2]) ❶
set([<__main__.Point at 0x109b95910>])
>>> Point(1, 1) in set([p1, p2])
True
```

- ❶ Note that specifically only the first element, `p1`, gets inserted into the set object. So if you are using this set object to keep track of existing `Point` objects, `p2` will get lost.

As alluded to when we discussed hash collisions, a custom-selected hash function should be careful to evenly distribute hash values in order to avoid collisions. Having many collisions will degrade the performance of a hash table: if most keys have collisions, we need to constantly “probe” the other values, effectively walking a potentially large portion of the dictionary to find the key in question. In the worst case, when all keys in a dictionary collide, the performance of lookups in the dictionary is $O(n)$ and thus the same as if we were searching through a list.

If we know that we are storing 5,000 values with a custom hash function in a dictionary and we need to create a hashing function for the object we wish to use as a key, we must be aware that the dictionary will be stored in a hash table of size 16,384⁹

⁹ Five thousand values need a dictionary that has at least 8,333 buckets. The first available size that can fit this many elements is 16,384.

and thus only the last 14 bits of our hash are being used to create an index (for a hash table of this size, the mask is `bin(16_384 - 1) = 0b11111111111111`).

This idea of “how well distributed my hash function is” is called the *entropy* of the hash function. Entropy is defined as

$$S = - \sum_i p(i) \cdot \log(p(i))$$

where $p(i)$ is the probability that the hash function gives hash i . It is maximized when every hash value has equal probability of being chosen. A hash function that maximizes entropy is called an *ideal* hash function since it guarantees the minimal number of collisions.



For the most part, creating your own custom hash function is unnecessary. Generally, you can use the Python hash function while indicating specifically which parts of the object need to be hashed (as we did in [Example 4-7](#)). If more complex hashing is needed, there are many prebuilt algorithms that provide different guarantees depending on their inputs. The hash function specifically uses the “[SipHash 1-3 algorithm](#)” for hashing things other than integers.

For an infinitely large dictionary, the hash function used for integers is ideal. This is because the hash value for an integer is simply the integer itself! For an infinitely large dictionary, the mask value is infinite, and thus we consider all bits in the hash value. Therefore, given any two numbers, we can guarantee that their hash values will not be the same.

However, if we made this dictionary finite, we could no longer have this guarantee. For example, for a dictionary with four elements, the mask we use is `0b111`. Thus the hash value for the number 5 is `5 & 0b111 = 5`, and the hash value for 501 is `501 & 0b111 = 5`, and so their entries will collide.



To find the mask for a dictionary with an arbitrary number of elements, N , we first find the minimum number of buckets that the dictionary must have to still be two-thirds full ($N * (2 / 3 + 1)$). Then we find the smallest dictionary size that will hold this number of elements (8; 32; 128; 512; 2,048; etc.) and find the number of bits necessary to hold this number. For example, if $N=1039$, then we must have at least 1,731 buckets, which means we need a dictionary with 2,048 buckets. Thus the mask is `bin(2048 - 1) = 0b111111111111`.

There is no single best hash function to use when using a finite dictionary. However, knowing up front what range of values will be used and how large the dictionary will be helps in making a good selection. For example, if we are storing all 676 combinations of two lowercase letters as keys in a dictionary (*aa, ab, ac*, etc.), a good hashing function for this specific case would be the one shown in [Example 4-8](#).

Example 4-8. Optimal two-letter hashing function

```
def twoletter_hash(key):
    offset = ord('a')
    k1, k2 = key
    return (ord(k2) - offset) + 26 * (ord(k1) - offset)
```

This gives no hash collisions for any combination of two lowercase letters, considering a mask of `0b111111111` (a dictionary of 676 values will be held in a hash table of length 2,048, which has a mask of $\text{bin}(2048 - 1) = 0b11111111111$).

[Example 4-9](#) very explicitly shows the ramifications of having a bad hashing function for a user-defined class—here, the cost of a bad hash function (in fact, it is the worst possible hash function!) is a 54 times slowdown of lookups. This case of a bad hash function even makes the dictionary slower than using a list, which is 13% faster in this case.

Example 4-9. Timing differences between good and bad hashing functions

```
import string
import timeit

class BadHash(str):
    def __hash__(self):
        return 42

class GoodHash(str):
    def __hash__(self):
        """
        This is a slightly optimized version of twoletter_hash
        """
        return ord(self[1]) + 26 * ord(self[0]) - 2619

bad_dict = set()
good_dict = set()
list_control = list()
for i in string.ascii_lowercase:
    for j in string.ascii_lowercase:
        key = i + j
        bad_dict.add(BadHash(key))
        good_dict.add(GoodHash(key))
        list_control.append(key)
```

```

bad_time = timeit.repeat(
    "key in bad_dict",
    setup = "from __main__ import bad_dict, BadHash; key = BadHash('zz')",
    repeat = 3,
    number = 1_000_000,
)
good_time = timeit.repeat(
    "key in good_dict",
    setup = "from __main__ import good_dict, GoodHash; key = GoodHash('zz')",
    repeat = 3,
    number = 1_000_000,
)
list_time = timeit.repeat(
    "key in list_control",
    setup = "from __main__ import list_control; key = 'zz'",
    repeat = 3,
    number = 1_000_000,
)

print(f"Min lookup time for bad_dict: {min(bad_time)}")
print(f"Min lookup time for good_dict: {min(good_time)}")
print(f"Min lookup time for list_control: {min(list_time)}")

# Results:
#   Min lookup time for bad_dict: 10.160735580000619
#   Min lookup time for good_dict: 0.18675472999893827
#   Min lookup time for list_control: 8.958332514994254

```

Exercise

1. Show that, for an infinite dictionary (and thus an infinite mask), using an integer's value as its hash gives no collisions.
2. Show that the hashing function given in [Example 4-8](#) is ideal for a hash table of size 1,024. Why is it not ideal for smaller hash tables?



Readers of previous editions of this book may have realized that the section on scoping in namespaces is suspiciously missing from this version of the book. Recent changes to how CPython does namespace lookups brings the benchmarks so close to one another that the conclusions simply no longer hold. However, we still feel it is important to point out that this has changed as yet another reminder to always and continually profile your code. If you can, add profiling metrics to your unit tests to encode performance assumptions. So as projects live past CPython changes, or library changes, or hardware changes, the performance characteristics you depend on are still valid.

Wrap-Up

Dictionaries and sets provide a fantastic way to store data that can be indexed by a key. The way this key is used, through the hashing function, can greatly affect the resulting performance of the data structure. Furthermore, understanding how dictionaries work gives you a better understanding not only of how to organize your data but also of how to organize your code, since dictionaries are an intrinsic part of Python's internal functionality.

In the next chapter we will explore generators, which allow us to provide data to code with more control over ordering and without having to store full datasets in memory beforehand. This lets us sidestep many of the possible hurdles that we might encounter when using any of Python's intrinsic data structures.

Iterators and Generators

Questions You'll Be Able to Answer After This Chapter

- How do generators save memory?
- When is the best time to use a generator?
- How can I use `itertools` to create complex generator workflows?
- When is lazy evaluation beneficial, and when is it not?

When many people with experience in another language start learning Python, they are taken aback by the difference in `for` loop notation. That is to say, instead of writing

```
# Other languages
for (i=0; i<N; i++) {
    do_work(i);
}
```

they are introduced to a new function called `range`:

```
# Python
for i in range(N):
    do_work(i)
```

It seems that in the Python code sample we are calling a function, `range`, that creates all of the data we need for the `for` loop to continue. Intuitively, this can be quite a time-consuming process—if we are trying to loop over the numbers 1 through 100,000,000, then we need to spend a lot of time creating that array! However, this is where *generators* come into play: they essentially allow us to lazily evaluate these sorts of functions so we can have the code-readability of these special-purpose functions without the performance impacts.



Technically, `range` is a special `range` type and not a generator. However, it still is useful to get at the “lazy evaluation” nature of generators, especially since most uses of this range-type parallel generators. To make them exactly the same, you can run the `range` through the `iter` function to be sure that the behavior you are seeing is generator-specific and not some special attribute of range types.

To understand this concept, let’s implement a function that calculates several Fibonacci numbers both by filling a list and by using a generator:

```
def fibonacci_list(num_items):
    numbers = []
    a, b = 0, 1
    while len(numbers) < num_items:
        numbers.append(a)
        a, b = b, a+b
    return numbers

def fibonacci_gen(num_items):
    a, b = 0, 1
    while num_items:
        yield a ❶
        a, b = b, a+b
        num_items -= 1
```

- ❶ This function will yield many values instead of returning one value. This turns this regular-looking function into a generator that can be polled repeatedly for the next available value.

The first thing to note is that the `fibonacci_list` implementation must create and store the list of all the relevant Fibonacci numbers. So if we want to have 10,000 numbers of the sequence, the function will do 10,000 appends to the `numbers` list (which, as we discussed in [Chapter 3](#), has overhead associated with it) and then return it.

On the other hand, the generator is able to “return” many values. Every time the code gets to the `yield`, the function emits its value, and when another value is requested, the function resumes running (maintaining its previous state) and emits the new value. When the function reaches its end, a `StopIteration` exception is thrown, indicating that the given generator has no more values. As a result, even though both functions must, in the end, do the same number of calculations, the `fibonacci_list` version of the preceding loop uses 10,000 times more memory (or `num_items` times more memory).

With this code in mind, we can decompose the for loops that use our implementations of `fibonacci_list` and `fibonacci_gen`. In Python, for loops require that the object we are looping over supports iteration. This means that we must be able to create an iterator out of the object we want to loop over. To create an iterator from almost any object, we can use Python's built-in `iter` function. This function, for lists, tuples, dictionaries, and sets, returns an iterator over the items or keys in the object. For more complex objects, `iter` returns the result of the `__iter__` property of the object. Since `fibonacci_gen` already returns an iterator, calling `iter` on it is a trivial operation, and it returns the original object (so `type(fibonacci_gen(10)) == type(iter(fibonacci_gen(10)))`). However, since `fibonacci_list` returns a list, we must create a new object, a list iterator, that will iterate over all values in the list. In general, once an iterator is created, we call the `next()` function with it, retrieving new values until a `StopIteration` exception is thrown. This gives us a good deconstructed view of for loops, as illustrated in [Example 5-1](#).

Example 5-1. Python for loop deconstructed

```
# The Python loop
for i in object:
    do_work(i)

# Is equivalent to
object_iterator = iter(object)
while True:
    try:
        i = next(object_iterator)
    except StopIteration:
        break
    else:
        do_work(i)
```

The for loop code shows that we are doing extra work calling `iter` when using `fibonacci_list` instead of `fibonacci_gen`. When using `fibonacci_gen`, we create a generator that is trivially transformed into an iterator (since it is already an iterator!); however, for `fibonacci_list` we need to allocate a new list and precompute its values, and then we still must create an iterator.

More importantly, precomputing the `fibonacci_list` list requires allocating enough space for the full dataset and setting each element to the correct value, even though we always require only one value at a time. This also makes the list allocation useless. In fact, it may even make the loop unrunnable, because it may be trying to allocate more memory than is available (`fibonacci_list(100_000_000)` would create a list 3.1 GB large!). By timing the results, we can see this very explicitly.

On the other hand, we can see in [Example 5-2](#) that the generator version is 2.7 times faster and requires no measurable memory as compared to the `fibonacci_list`'s 418.48 MB. It may seem at this point that you should use generators everywhere in place of creating lists, but that would create many complications.

Example 5-2. Fibonacci sequence using lists versus generators

```
def test_fibonacci_list():
    """
    >>> %timeit test_fibonacci_list()
    227 ms ± 4.18 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

    >>> %memit test_fibonacci_list() ❶
    peak memory: 485.30 MiB, increment: 418.48 MiB
    """
    for i in fibonacci_list(100_000):
        pass

def test_fibonacci_gen():
    """
    >>> %timeit test_fibonacci_gen()
    81.9 ms ± 25.1 μs per loop (mean ± std. dev. of 7 runs, 10 loops each)

    >>> %memit test_fibonacci_gen()
    peak memory: 72.34 MiB, increment: 0.00 MiB
    """
    for i in fibonacci_gen(100_000):
        pass
```

- ❶ `memit` provides us with the peak total memory use of our Python interpreter (in this case 485.3 MiB) and the amount that this memory use increased after running the provided line of code (i.e., running `test_fibonacci_list()` increased our memory use by 418.48 MiB!).

What if, for example, you needed to reference the list of Fibonacci numbers multiple times? In this case, `fibonacci_list` would provide a precomputed list of these digits, while `fibonacci_gen` would have to recompute them over and over again. In general, changing to using generators instead of precomputed arrays requires algorithmic changes that are sometimes not so easy to understand.¹

¹ In general, algorithms that are *online* or *single pass* are a great fit for generators. However, when making the switch you have to ensure your algorithm can still function without being able to reference the data more than once.



An important choice that must be made when architecting your code is whether you are going to optimize CPU speed or memory efficiency. In some cases, using extra memory so that you have values precalculated and ready for future reference will save in overall speed. At other times, memory may be so constrained that the only solution is to recalculate values as opposed to saving them in memory. Every problem has its own considerations for this CPU/memory trade-off.

One simple example of this that is often seen in source code is using a generator to create a sequence of numbers, only to use list comprehension to calculate the length of the result:

```
divisible_by_three = len([n for n in fibonacci_gen(100_000) if n % 3 == 0])
```

While we are still using `fibonacci_gen` to generate the Fibonacci sequence as a generator, we are then saving all values divisible by 3 into an array, only to take the length of that array and then throw away the data. In the process, we're consuming 98 MB of data for no reason.² In fact, if we were doing this for a long enough Fibonacci sequence, the preceding code wouldn't be able to run because of memory issues, even though the calculation itself is quite simple!

Recall that we can create a list comprehension using a statement of the form [*<value>* for *<item>* in *<sequence>* if *<condition>*]. This will create a list of all the *<value>* items. Alternatively, we can use similar syntax to create a generator of the *<value>* items instead of a list with (*<value>* for *<item>* in *<sequence>* if *<condition>*).

Using this subtle difference between list comprehension and generator comprehension, we can optimize the preceding code for `divisible_by_three`. However, generators do not have a `length` property. As a result, we will have to be a bit clever:

```
divisible_by_three = sum(1 for n in fibonacci_gen(100_000) if n % 3 == 0)
```

Here, we have a generator that emits a value of 1 whenever it encounters a number divisible by 3, and nothing otherwise. By summing all elements in this generator, we are essentially doing the same as the list comprehension version and consuming no significant memory.

² Calculated with `%memit len([n for n in fibonacci_gen(100_000) if n % 3 == 0])`.



Many of Python's built-in functions that operate on sequences are generators themselves (albeit sometimes a special type of generator). For example, `range` returns a generator of values as opposed to the actual list of numbers within the specified range. Similarly, `map`, `zip`, `filter`, `reversed`, and `enumerate` all perform the calculation as needed and don't store the full result. This means that the operation `zip(range(100_000), range(100_000))` will always have only two numbers in memory in order to return its corresponding values, instead of precalculating the result for the entire range beforehand.

The performance of the two versions of this code is almost equivalent for these smaller sequence lengths, but the memory impact of the generator version is far less than that of the list comprehension. Furthermore, we transform the list version into a generator, because all that matters for each element of the list is its current value—either the number is divisible by 3 or it is not; it doesn't matter where its placement is in the list of numbers or what the previous/next values are. More complex functions can also be transformed into generators, but depending on their reliance on state, this can become a difficult thing to do.

Iterators for Infinite Series

Instead of calculating a known number of Fibonacci numbers, what if we instead attempted to calculate all of them?

```
def fibonacci():
    i, j = 0, 1
    while True:
        yield i
        i, j = j, i + j
```

In this code we are doing something that wouldn't be possible with the previous `fibonacci_list` code: we are encapsulating an infinite series of numbers into a function. This allows us to take as many values as we'd like from this stream and terminate when our code thinks it has had enough.

One reason generators aren't used as much as they could be is that sometimes it is easier to be more explicit about where various logical decisions are being made (for example, the readability of `fibonacci_transform` and `fibonacci_succinct` below). Generators are really a way of organizing your code and having smarter loops. For example, we could answer the question "How many Fibonacci numbers below 5,000 are odd?" in multiple ways:

```
def fibonacci_naive():
    i, j = 0, 1
    count = 0
    while i <= 5000:
```

```

        if i % 2:
            count += 1
        i, j = j, i + j
    return count

def fibonacci_transform():
    count = 0
    for f in fibonacci():
        if f >= 5000:
            break
        if f % 2:
            count += 1
    return count

from itertools import takewhile
def fibonacci_succinct():
    first_5000 = takewhile(lambda x: x < 5000,
                           fibonacci())
    return sum(1 for x in first_5000
               if x % 2)

```

All of these methods have similar runtime properties (as measured by their memory footprint and runtime performance), but the `fibonacci_transform` function benefits from several things. First, it is much more verbose than `fibonacci_succinct`, which means it will be easy for another developer to debug and understand. The latter mainly stands as a warning for the next section, where we cover some common workflows using `itertools`—while the module greatly simplifies many simple actions with iterators, it can also quickly make Python code very un-Pythonic. Conversely, `fibonacci_naive` is doing multiple things at a time, which hides the actual calculation it is doing! While it is obvious in the generator function that we are iterating over the Fibonacci numbers, we are not overencumbered by the actual calculation. Last, `fibonacci_transform` is more generalizable. This function could be renamed `num_odd_under_5000` and take in the generator by argument, and thus work over any series.

One additional benefit of the `fibonacci_transform` and `fibonacci_succinct` functions is that they support the notion that in computation there are two phases: generating data and transforming data. These functions are very clearly performing a transformation on data, while the `fibonacci` function generates it. This demarcation adds extra clarity and functionality: we can move a transformative function to work on a new set of data, or perform multiple transformations on existing data. This paradigm has always been important when creating complex programs; however, generators facilitate this clearly by making generators responsible for creating the data and normal functions responsible for acting on the generated data.

Lazy Generator Evaluation

As touched on previously, the way we get the memory benefits with a generator is by dealing only with the current values of interest. At any point in our calculation with a generator, we have only the current value and cannot reference any other items in the sequence (algorithms that perform this way are generally called *single pass* or *online*). This can sometimes make generators more difficult to use, but many modules and functions can help.

The main library of interest is `itertools`, in the standard library. It supplies many other useful functions, including these:

`islice`

Allows slicing a potentially infinite generator

`chain`

Chains together multiple generators

`takewhile`

Adds a condition that will end a generator

`cycle`

Makes a finite generator infinite by constantly repeating it

The documentation also provides some fantastic recipes for other use cases, which are always useful to have in the back of your head.

Let's build up an example of using generators to analyze a large dataset. Let's say we've had an analysis routine going over temporal data, one piece of data per second, for the last 20 years—that's 631,152,000 data points! The data is stored in a file, one second per line, and we cannot load the entire dataset into memory. As a result, if we wanted to do some simple anomaly detection, we'd have to use generators to save memory!

The problem will be: Given a datafile of the form “timestamp, value,” find days whose values differ from the normal distribution. We start by writing the code that will read the file, line by line, and output each line's value as a Python object. We will also create a `read_fake_data` generator to generate fake data that we can test our algorithms with. For this function we still take the argument `filename`, so as to have the same function signature as `read_data`; however, we will simply disregard it. These two functions, shown in [Example 5-3](#), are indeed lazily evaluated—we read the next line in the file, or generate new fake data, only when the `next()` function is called.

Example 5-3. Lazily reading data

```
from random import normalvariate, randint
from dataclasses import dataclass
```



```

from itertools import count
from datetime import datetime

@dataclass
class Datum:
    date: datetime
    value: float

def read_data(filename):
    with open(filename) as fd:
        for line in fd:
            data = line.strip().split(',')
            timestamp, value = map(int, data)
            yield Datum(datetime.fromtimestamp(timestamp), value)

def read_fake_data(filename):
    for timestamp in count():
        # We insert an anomalous data point approximately once a week
        if randint(0, 7 * 60 * 60 * 24 - 1) == 1:
            value = 100
        else:
            value = normalvariate(0, 1)
        yield Datum(datetime.fromtimestamp(timestamp), value)

```

Now we'd like to create a function that outputs groups of data that occur in the same day. For this, we can use the `groupby` function in `itertools` (Example 5-4). This function works by taking in a sequence of items and a key used to group these items. The output is a generator that produces tuples whose items are the key for the group and a generator for the items in the group. As our key function, we will output the calendar day that the data was recorded. This “key” function could be anything—we could group our data by hour, by year, or by some property in the actual value. The only limitation is that groups will be formed only for data that is sequential. So if we had the input A A A A B B A A and had `groupby` group by the letter, we would get three groups: (A, [A, A, A, A]), (B, [B, B]), and (A, [A, A]).

Example 5-4. Grouping our data

```

from itertools import groupby

def groupby_day(iterable): ❶
    key = lambda row: row.date.day
    for day, data_group in groupby(iterable, key):
        yield list(data_group)

```

❶ `iterable` is a sequence of `Datum` objects.

Now to do the actual anomaly detection. We do this in [Example 5-5](#) by creating a function that, given one group of data, returns whether it follows the normal distribution (using `scipy.stats.normaltest`). We can use this check with `itertools.filterfalse` to filter down the full dataset only to inputs that *don't* pass the test. These inputs are what we consider to be anomalous.



In [Example 5-4](#), we cast `data_group` into a list, even though it is provided to us as an iterator. This is because the `normaltest` function requires an array-like object. We could, however, write our own `normaltest` function that is “one-pass” and could operate on a single view of the data. This could be done without too much trouble by using [Welford's online averaging algorithm](#) to calculate the skew and kurtosis of the numbers. This would save us even more memory by always storing only a single value of the dataset in memory at once instead of storing a full day at a time. However, performance time regressions and development time should be taken into consideration: is storing one day of data in memory at a time sufficient for this problem, or does it need to be further optimized?

Example 5-5. Generator-based anomaly detection

```
from scipy.stats import normaltest
from itertools import filterfalse
from operator import attrgetter

def is_normal(data, threshold=1e-3):
    values = map(attrgetter("value"), data)
    k2, p_value = normaltest(tuple(values))
    return p_value >= threshold

def filter_anomalous_groups(data):
    yield from filterfalse(is_normal, data)
```

Finally, we can put together the chain of generators to get the days that had anomalous data ([Example 5-6](#)).

Example 5-6. Chaining together our generators

```
from itertools import islice

def filter_anomalous_data(data):
    data_group = groupby_day(data)
    yield from filter_anomalous_groups(data_group)

data = read_fake_data("fake_filename")
```

```

anomaly_generator = filter_anomalous_data(data)
first_five_anomalies = islice(anomaly_generator, 5)

for data_anomaly in first_five_anomalies:
    start_date = data_anomaly[0].date
    end_date = data_anomaly[-1].date
    print(f"Anomaly from {start_date} - {end_date}")

# Output of above code using "read_fake_data"
Anomaly from 1970-01-10 00:00:00 - 1970-01-10 23:59:59
Anomaly from 1970-01-17 00:00:00 - 1970-01-17 23:59:59
Anomaly from 1970-01-18 00:00:00 - 1970-01-18 23:59:59
Anomaly from 1970-01-23 00:00:00 - 1970-01-23 23:59:59
Anomaly from 1970-01-29 00:00:00 - 1970-01-29 23:59:59

```

This method allows us to get the list of days that are anomalous without having to load the entire dataset. Only enough data is read to generate the first five anomalies. Additionally, the `anomaly_generator` object can be read further to continue retrieving anomalous data. This is called *lazy evaluation*—only the calculations that are explicitly requested are performed, which can drastically reduce overall runtime if there is an early termination condition.

Another nicety about organizing analysis this way is it allows us to do more expansive calculations easily, without having to rework large parts of the code. For example, if we want to have a moving window of one day instead of chunking up by days, we can replace the `groupby_day` in [Example 5-4](#) with something like [Example 5-7](#).

Example 5-7. Implementation of windowed group-by function using generators

```

from datetime import datetime

def groupby_window(data, window_size=3600):
    window = tuple(islice(data, window_size))
    for item in data: ❶
        yield window
        window = window[1:] + (item,) ❷

```

- ❶ The for loop will start at the `window_size+1`th item of the data sequence because the above `tuple(islice(...))` already consumed that many items.
- ❷ Here we remove the first item out of window and append the next item to the end of it.

In this version, we also see very explicitly the memory guarantee of this and the previous method—they will store only the window's worth of data as state (in both cases, one day, or 3,600 data points). Note that the first item retrieved by the for loop

is the `window_size`-th value. This is because `data` is an iterator, and in the previous line we consumed the first `window_size` values.

A final note: In the `groupby_window` function, we are constantly creating new tuples, filling them with data, and yielding them to the caller. We can greatly optimize this by using the `deque` object in the `collections` module. This object gives us $O(1)$ appends to and removals from the beginning or end of a list (while normal lists are $O(1)$ for appends to/removals from the end of the list and $O(n)$ for the same operations at the beginning of the list). Using the `deque` object, we can append the new data to the right (or end) of the list and use `deque.popleft()` to delete data from the left (or beginning) of the list without having to allocate more space or perform long $O(n)$ operations. However, we would have to work on the `deque` object in-place and destroy previous views to the rolling window (see “[Memory Allocations and In-Place Operations](#)” on page 149 for more about in-place operations). The only way around this would be to copy the data into a tuple before yielding it back to the caller, which gets rid of any benefit of the change!

Wrap-Up

By formulating our anomaly-finding algorithm with iterators, we can process much more data than could fit into memory. What’s more, we can do it faster than if we had used lists, since we avoid all the costly append operations.

Since iterators are a primitive type in Python, this should always be a go-to method for trying to reduce the memory footprint of an application. The benefits are that results are lazily evaluated, so you process only the data you need, and memory is saved since you don’t store previous results unless explicitly required to. In [Chapter 12](#), we will talk about other methods that can be used for more specific problems and introduce some new ways of looking at problems when RAM is an issue. However, as we’ll see in [Chapter 6](#), if your data does fit into memory, there are many things you can do to optimize memory layout for the most efficient use of your CPU or GPU.

Another benefit of solving problems using iterators is that it prepares your code to be used on multiple CPUs or multiple computers, as we will see in [Chapters 9 and 10](#). As we discussed in “[Iterators for Infinite Series](#)” on page 118, when working with iterators, you must always think about the various states that are necessary for your algorithm to work. Once you figure out how to package the state necessary for the algorithm to run, it doesn’t matter where it runs. We can see this sort of paradigm, for example, with the `multiprocessing` and `ipython` modules, both of which use a `map`-like function to launch parallel tasks.

Matrix and Vector Computation

Questions You'll Be Able to Answer After This Chapter

- What are the bottlenecks in vector calculations?
- What tools can I use to see how efficiently the CPU is doing my calculations?
- Why is `numpy` better at numerical calculations than pure Python?
- What are cache-misses and page-faults?
- How can I track the memory allocations in my code?
- How can I use a GPU to speed up my computation?
- What are the subtleties when using GPUs for numerical code or deep learning?

Regardless of what problem you are trying to solve on a computer, you will encounter vector computation at some point. Vector calculations are integral to how a computer works and how it tries to speed up runtimes of programs down at the silicon level—the only thing the computer knows how to do is operate on numbers, and knowing how to do several of those calculations at once will speed up your program.

In this chapter we try to unwrap some of the complexities of this problem by focusing on a relatively simple mathematical problem, solving the diffusion equation, and understanding what is happening at the CPU level. By understanding how different Python code affects the CPU and how to effectively probe these things, we can learn how to understand other problems as well. We will also talk about GPUs and the nuances of using them for compute.

We will start by introducing the problem and coming up with a quick solution using pure Python. After identifying some memory issues and trying to fix them using

pure Python, we will introduce `numpy` and identify how and why it speeds up our code. Then we will start doing some algorithmic changes and specialize our code to solve the problem at hand. By removing some of the generality of the libraries we are using, we will yet again be able to gain more speed. Next, we'll introduce some extra modules that will help facilitate this sort of process out in the field. Then we will introduce PyTorch to leverage GPUs in order to get even greater speedups. Finally, we'll explore a cautionary tale about optimizing before profiling.

Introduction to the Problem

To explore matrix and vector computation in this chapter, we will repeatedly use the example of diffusion in fluids. Diffusion is one of the mechanisms that moves fluids and tries to make them uniformly mixed.



This section is meant to give a deeper understanding of the equations we will be solving throughout the chapter. It is not strictly necessary that you understand this section in order to approach the rest of the chapter. If you wish to skip this section, make sure to at least look at the algorithm in Examples 6-1 and 6-2 to understand the code we will be optimizing.

On the other hand, if you read this section and want even more explanation, read Chapter 17 of *Numerical Recipes* by William Press et al. (Cambridge University Press).

In this section we will explore the mathematics behind the diffusion equation. This may seem complicated, but don't worry! We will quickly simplify this to make it more understandable. Also, it is important to note that while having a basic understanding of the final equation we are solving will be useful while reading this chapter, it is not completely necessary; the subsequent sections will focus mainly on various formulations of the code, not the equation. However, having an understanding of the equations will help you gain intuition about ways of optimizing your code. This is true in general—understanding the motivation behind your code and the intricacies of the algorithm will give you deeper insight about possible methods of optimization.

One simple example of diffusion is dye in water: if you put several drops of dye into water at room temperature, the dye will slowly move out until it fully mixes with the water. Since we are not stirring the water, nor is it warm enough to create convection currents, diffusion will be the main process mixing the two liquids. When solving these equations numerically, we pick what we want the initial condition to look like and are able to evolve the initial condition forward in time to see what it will look like at a later time (see Figure 6-2).

All this being said, the most important thing to know about diffusion for our purposes is its formulation. Stated as a partial differential equation in one dimension (1D), the diffusion equation is written as follows:

$$\frac{\partial}{\partial t}u(x,t) = D \cdot \frac{\partial^2}{\partial x^2}u(x,t)$$

In this formulation, u is the vector representing the quantities we are diffusing. For example, we could have a vector with values of 0 where there is only water, and of 1 where there is only dye (and values in between where there is mixing). In general, this will be a 2D or 3D matrix representing an actual area or volume of fluid. In this way, we could have u be a 3D matrix representing the fluid in a glass, and instead of simply doing the second derivative along the x direction, we'd have to take it over all axes. In addition, D is a physical value that represents properties of the fluid we are simulating. A large value of D represents a fluid that can diffuse very easily. For simplicity, we will set $D = 1$ for our code but still include it in the calculations.



The diffusion equation is also called the *heat equation*. In this case, u represents the temperature of a region, and D describes how well the material conducts heat. Solving the equation tells us how the heat is being transferred. So instead of solving for how a couple of drops of dye diffuse through water, we might be solving for how the heat generated by a CPU diffuses into a heat sink.

What we will do is take the diffusion equation, which is continuous in space and time, and approximate it using discrete volumes and discrete times. We will do so using Euler's method. Euler's method simply takes the derivative and writes it as a difference, such that

$$\frac{\partial}{\partial t}u(x,t) \approx \frac{u(x,t+dt) - u(x,t)}{dt}$$

where dt is now a fixed number. This fixed number represents the time step, or the resolution in time for which we wish to solve this equation. It can be thought of as the frame rate of the movie we are trying to make. As the frame rate goes up (or dt goes down), we get a clearer picture of what happens. In fact, as dt approaches zero, Euler's approximation becomes exact (note, however, that this exactness can be achieved only theoretically, since there is only finite precision on a computer and numerical errors will quickly dominate any results). We can thus rewrite this equation to figure out what $u(x,t+dt)$ is, given $u(x,t)$. What this means for us is that we can start with some initial state ($u(x,0)$, representing the glass of water just as we add a drop of dye into it) and churn through the mechanisms we've outlined to "evolve" that initial state

and see what it will look like at future times ($u(x, t+dt)$). This type of problem is called an *initial value problem* or a *Cauchy problem*.

Doing a similar trick for the derivative in x using the finite differences approximation, we arrive at the final equation:

$$u(x, t+dt) = u(x, t) + dt \times D \times \frac{u(x+dx, t) + u(x-dx, t) - 2 \cdot u(x, t)}{dx^2}$$

Here, similar to how dt represents the frame rate, dx represents the resolution of the images—the smaller dx is, the smaller a region every cell in our matrix represents. For simplicity, we will set $D = 1$ and $dx = 1$. These two values become very important when doing proper physical simulations; however, since we are solving the diffusion equation for illustrative purposes, they are not important to us.

Using this equation, we can solve almost any diffusion problem. However, there are some considerations regarding this equation. First, we said before that the spatial index in u (i.e., the x parameter) will be represented as the indices into a matrix. What happens when we try to find the value at $x - dx$ when x is at the beginning of the matrix? This problem is called the *boundary condition*. You can have fixed boundary conditions that say “any value out of the bounds of my matrix will be set to 0” (or any other value). Alternatively, you can have periodic boundary conditions that say that the values will wrap around. (That is, if one of the dimensions of your matrix has length N , the value in that dimension at index -1 is the same as at $N - 1$, and the value at N is the same as at index 0. In other words, if you are trying to access the value at index i , you will get the value at index $(i\%N)$.)

Another consideration is how we are going to store the multiple time components of u . We could have one matrix for every time value we do our calculation for. At minimum, it seems that we will need two matrices: one for the current state of the fluid and one for the next state of the fluid. As we’ll see, there are very drastic performance considerations for how we deal with this.

So what does it look like to solve this problem in practice? [Example 6-1](#) contains some pseudocode to illustrate the way we can use our equation to solve the problem.

Example 6-1. Pseudocode for 1D diffusion

```
# Create the initial conditions
u = vector of length N
for i in range(N):
    u = 0 if there is water, 1 if there is dye

# Evolve the initial conditions
D = 1
t = 0
dt = 0.0001
while True:
    print(f"Current time is: {t}")
    unew = vector of size N

    # Update step for every cell
    for i in range(N):
        unew[i] = u[i] + D * dt * (u[(i+1)%N] + u[(i-1)%N] - 2 * u[i]) ❶
    # Move the updated solution into u
    u = unew

    visualize(u)
```

❶ This is the previous equation with dx set to 1.

This code will take an initial condition of the dye in water and tell us what the system looks like at every 0.0001-second interval in the future. In addition, we can handle our circular boundary conditions simply by using the modulo operation on the array index, which wraps the $N+1$ index to 0. The results can be seen in [Figure 6-1](#), where we evolve our very concentrated drop of dye (represented by the top-hat function) into the future. We can see how, far into the future, the dye becomes well mixed, to the point where everywhere has a similar concentration of the dye.

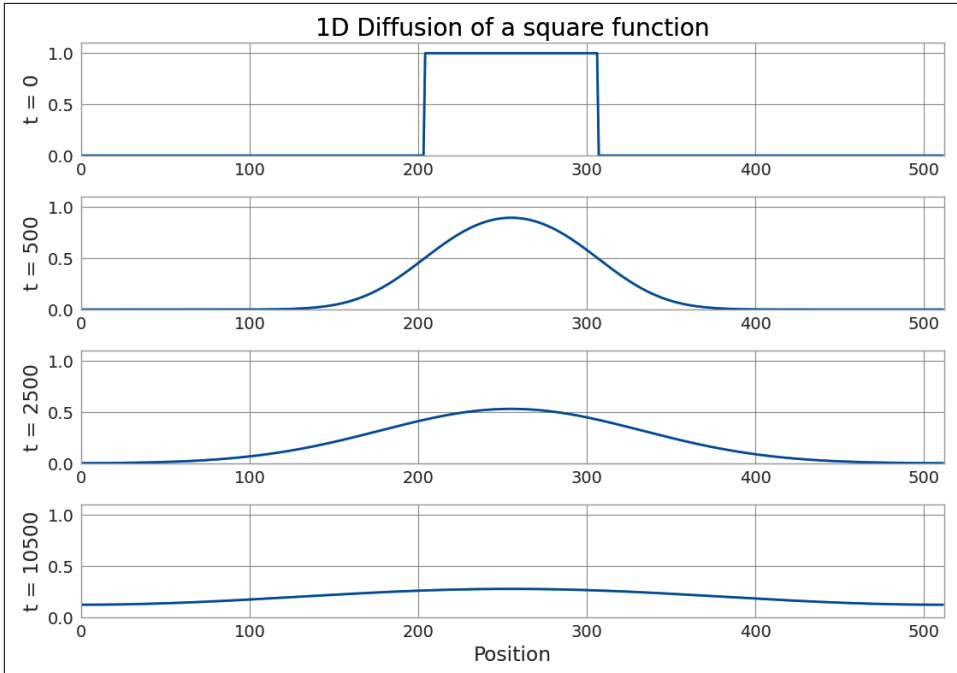


Figure 6-1. Example of 1D diffusion

For the purposes of this chapter, we will be solving the 2D version of the preceding equation. All this means is that instead of operating over a vector (or in other words, a matrix with one index), we will be operating over a 2D matrix. The only change to the equation (and thus to the subsequent code) is that we must now also take the second derivative in the y direction. This simply means that the original equation we were working with becomes the following:

$$\frac{\partial}{\partial t} u(x, y, t) = D \cdot \left(\frac{\partial^2}{\partial x^2} u(x, y, t) + \frac{\partial^2}{\partial y^2} u(x, y, t) \right)$$

This numerical diffusion equation in 2D translates to the pseudocode in [Example 6-2](#), using the same methods we used before.

Example 6-2. Algorithm for calculating 2D diffusion

```
for i in range(N):
    for j in range(M):
        unew[i][j] = u[i][j] + dt * (
            (u[(i + 1) % N][j] + u[(i - 1) % N][j] - 2 * u[i][j]) + # d^2 u / dx^2
            (u[i][(j + 1) % M] + u[i][(j - 1) % M] - 2 * u[i][j]) # d^2 u / dy^2
        )
```

We can now put all of this together and write the full Python 2D diffusion code that we will use as the basis for our benchmarks for the rest of this chapter. While the code looks more complicated, the results are similar to that of the 1D diffusion (as can be seen in [Figure 6-2](#)).

If you'd like to do some additional reading on the topics in this section, check out the [Wikipedia page on the diffusion equation](#) and [Chapter 7 of *Numerical Methods for Complex Systems*](#) by S. V. Gurevich.

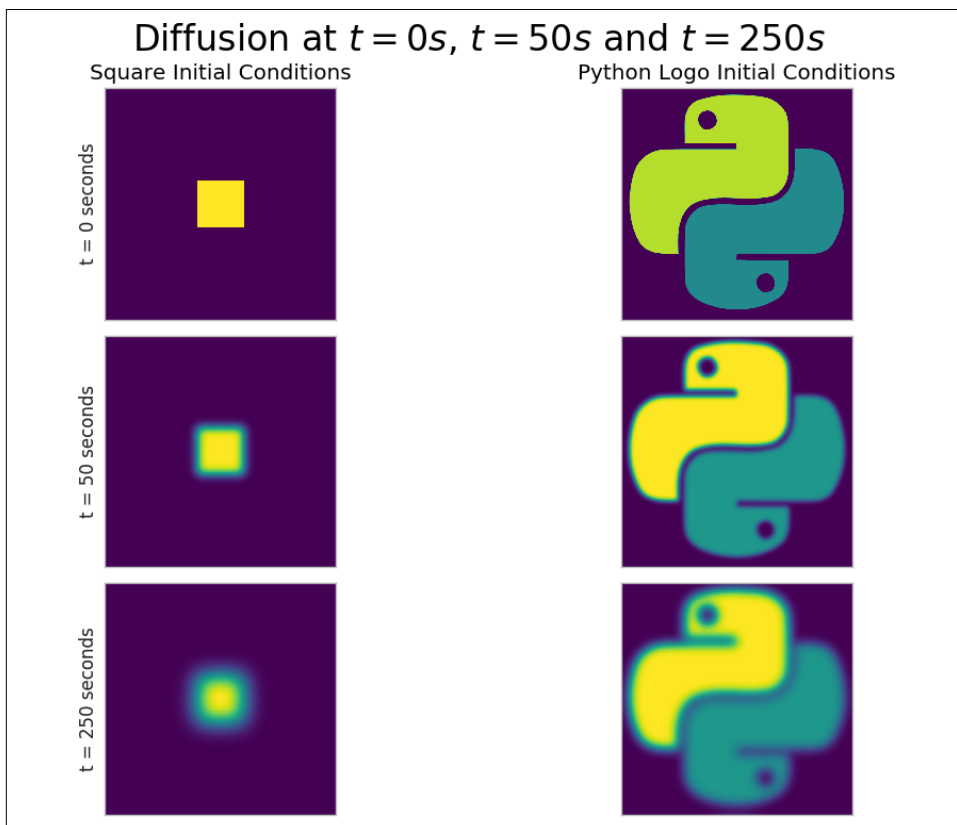


Figure 6-2. Example of 2D diffusion for two sets of initial conditions

Aren't Python Lists Good Enough?

Let's take our pseudocode from [Example 6-1](#) and formalize it so we can better analyze its runtime performance. The first step is to write out the evolution function that takes in the matrix and returns its evolved state. This is shown in [Example 6-3](#).

Example 6-3. Pure Python 2D diffusion

```
grid_shape = (512, 512)
```

```
def evolve(grid, dt, D=1.0):
    xmax, ymax = grid_shape
    new_grid = [[0.0] * ymax for _ in range(xmax)] ❶
    for i in range(xmax):
        for j in range(ymax):
            grid_xx = (
                grid[(i + 1) % xmax][j] + grid[(i - 1) % xmax][j] - 2.0 * grid[i][j]
            )
            grid_yy = (
                grid[i][(j + 1) % ymax] + grid[i][(j - 1) % ymax] - 2.0 * grid[i][j]
            )
            new_grid[i][j] = grid[i][j] + D * (grid_xx + grid_yy) * dt
    return new_grid
```

- ❶ Note that for the purposes of profiling with `line_profiler` later, the list comprehension is moved into another function to turn the line into `new_grid = make_grid(xmax, ymax)`. This is because `kernprof` shows performance for each iteration of the list comprehension, but for performance reasons we are interested in the full grid creation.

The global variable `grid_shape` designates how big a region we will simulate; and, as explained in “[Introduction to the Problem](#)” on page 126, we are using periodic boundary conditions (which is why we use modulo for the indices). To actually use this code, we must initialize a grid and call `evolve` on it. The code in [Example 6-4](#) is a very generic initialization procedure that will be reused throughout the chapter (its performance characteristics will not be analyzed since it must run only once, as opposed to the `evolve` function, which is called repeatedly).

Example 6-4. Pure Python 2D diffusion initialization

```
def run_experiment(num_iterations):
    # Setting up initial conditions ❶
    xmax, ymax = grid_shape
    grid = [[0.0] * ymax for _ in range(xmax)]

    # These initial conditions are simulating a drop of dye in the middle of our
    # simulated region
    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    for i in range(block_low, block_high):
        for j in range(block_low, block_high):
            grid[i][j] = 0.005
```

```
# Evolve the initial conditions
start = time.time()
for i in range(num_iterations):
    grid = evolve(grid, 0.1)
return time.time() - start
```

- ❶ The initial conditions used here are the same as in the square example in [Figure 6-2](#).

The values for `dt` and grid elements have been chosen to be sufficiently small that the algorithm is stable. See *Numerical Recipes* for a more in-depth treatment of this algorithm’s convergence characteristics.

Problems with Allocating Too Much

By using `line_profiler` on the pure Python evolution function, we can start to unravel what is contributing to a possibly slow runtime. Looking at the profiler output in [Example 6-5](#), we see that most of the time in the function is spent doing the derivative calculation and updating the grid.¹ This is what we want, since this is a purely CPU-bound problem—any time not spent on solving the CPU-bound problem is an obvious place for optimization.

Example 6-5. Pure Python 2D diffusion profiling

```
$ kernprof -lv diffusion_python.py
Wrote profile results to diffusion_python.py.lprof
Timer unit: 1e-06 s

Total time: 631.48 s
File: diffusion_python.py
Function: evolve at line 17

Line#      Hits          Time Per Hit   % Time  Line Contents
=====
17          1          1537.4      1.5      0.0      @profile
18          1          58458197.6  58458.2     9.3      def evolve(grid, dt, D=1.0):
19          1000          189066.0      0.4      0.0      xmax, ymax = grid_shape
20          1000          190032271.5    0.7     30.1      new_grid = make_grid(xmax, ymax)
21          513000          102481356.0    0.4     16.2      for i in range(xmax): ❶
22          262144000          176874148.8    0.7     28.0      for j in range(ymax):
23          262144000          176874148.8    0.7     28.0      grid_xx = ...
24          262144000          176874148.8    0.7     28.0      grid_yy = ...
```

¹ This is the code from [Example 6-3](#), truncated to fit within the page margins. Recall that `kernprof` requires functions to be decorated with `@profile` in order to be profiled (see “[Using line_profiler for Line-by-Line Measurements](#)” on page 48).

```

25 262144000 103441328.9 0.4 16.4 new_grid[i][j] = ...
26 1000 2515.0 2.5 0.0 return new_grid ②

```

- ① This line has 513,000 hits associated with it, because the grid we operated over had `xmax = 512` and then we ran the function 1,000 times. The calculation is $(512 + 1) * 1000$, where the extra one evaluation is from the termination of the loop.
- ② This line has 1,000 hits associated with it, which informs us that the function was profiled over 1,000 runs.

It's interesting to see the big difference in the `Per Hit` and `% Time` fields for line 20, where we allocate the new grid. This difference occurs because, while the line itself is quite slow (the `Per Hit` field shows 58 milliseconds per call!), it isn't called as often as other lines inside the loop. If we were to reduce the size of the grid and do more iterations (i.e., reduce the number of iterations of the loop but increase the number of times we call the function), we would see the `% Time` of this line increase and quickly dominate the runtime.

This is a waste, because the properties of `new_grid` do not change—no matter what values we send to `evolve`, the `new_grid` list will always be the same shape and size and contain the same values. A simple optimization would be to allocate this list once and simply reuse it. This way, we need to run this code only once, no matter the size of the grid or the number of iterations. This sort of optimization is similar to moving repetitive code outside a fast loop:

```

from math import sin

def loop_slow(num_iterations):
    """
    >>> %timeit loop_slow(int(1e4))
    947 µs ± 7.1 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
    """
    result = 0
    for i in range(num_iterations):
        result += i * sin(num_iterations) ①
    return result

def loop_fast(num_iterations):
    """
    >>> %timeit loop_fast(int(1e4))
    401 µs ± 4.87 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
    """
    result = 0
    factor = sin(num_iterations)
    for i in range(num_iterations):
        result += i
    return result * factor

```

- ❶ The value of `sin(num_iterations)` doesn't change throughout the loop, so there is no use recalculating it every time.

We can do a similar transformation to our diffusion code, as illustrated in [Example 6-6](#). In this case, we would want to instantiate `new_grid` in [Example 6-4](#) and send it into our `evolve` function. That function will do the same as it did before: read the grid list and write to the `new_grid` list. Then we can simply swap `new_grid` with `grid` and continue again.

Example 6-6. Pure Python 2D diffusion after reducing memory allocations

```
def evolve(grid, dt, out, D=1.0):
    xmax, ymax = grid_shape
    for i in range(xmax):
        for j in range(ymax):
            grid_xx = (
                grid[(i + 1) % xmax][j] + grid[(i - 1) % xmax][j] - 2.0 * grid[i][j]
            )
            grid_yy = (
                grid[i][(j + 1) % ymax] + grid[i][(j - 1) % ymax] - 2.0 * grid[i][j]
            )
            out[i][j] = grid[i][j] + D * (grid_xx + grid_yy) * dt

def run_experiment(num_iterations):
    # Setting up initial conditions
    xmax, ymax = grid_shape
    next_grid = [[0.0] * ymax for x in range(xmax)]
    grid = [[0.0] * ymax for x in range(xmax)]

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    for i in range(block_low, block_high):
        for j in range(block_low, block_high):
            grid[i][j] = 0.005

    start = time.time()
    for i in range(num_iterations):
        # evolve modifies grid and next_grid in-place
        evolve(grid, 0.1, next_grid)
        grid, next_grid = next_grid, grid ❶
    return time.time() - start
```

- ❶ This is where we swap the old grid with the new one, reusing the allocated space.

We can see from the line profile of the modified version of the code in [Example 6-7](#) that this small change has given us a 16% speedup.² This leads us to a conclusion similar to the one made during our discussion of append operations on lists (see “[Lists as Dynamic Arrays](#)” on page 86): memory allocations are not cheap. Most times we request memory to store a variable or a list, Python must take its time to talk to the operating system in order to allocate the new space, and then we must iterate over the newly allocated space to initialize it to some value.³

Whenever possible, reusing space that has already been allocated will give performance speedups. However, be careful when implementing these changes. While the speedups can be substantial, as always you should profile to make sure you are achieving the results you want and are not simply polluting your codebase.

Example 6-7. Line profiling Python diffusion after reducing allocations

```
$ kernprof -lv diffusion_python_memory.py
Wrote profile results to diffusion_python_memory.py.lprof
Timer unit: 1e-06 s

Total time: 543.771 s
File: diffusion_python_memory.py
Function: evolve at line 13
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
13					@profile
14					def evolve(grid, dt, out, D=1.0):
15	1000	948.5	0.9	0.0	xmax, ymax = grid_shape
16	513000	184785.4	0.4	0.0	for i in range(xmax):
17	262656000	92105287.4	0.4	16.9	for j in range(ymax):
18	262144000	176443683.5	0.7	32.4	grid_xx = ...
19	262144000	171283269.6	0.7	31.5	grid_yy = ...
20	262144000	103753019.3	0.4	19.1	out[i][j] = ...

Memory Fragmentation

The Python code we wrote in [Example 6-6](#) still has a problem that is at the heart of using Python for these sorts of vectorized operations: Python doesn’t natively support vectorization. There are two reasons for this: Python lists store pointers to the actual data, and Python bytecode is not optimized for vectorization, so for loops cannot predict when using vectorization would be beneficial.

² The code profiled in [Example 6-7](#) is the code from [Example 6-6](#); it has been truncated to fit within the page margins.

³ This happens “most times” as opposed to “every time” because of the freelist caching mechanism that CPython employs, discussed in “[Tuples as Static Arrays](#)” on page 90.

The fact that Python lists store *pointers* means that, instead of actually holding the data we care about, lists store locations where that data can be found. For most uses, this is good because it allows us to store whatever type of data we like inside a list. However, when it comes to vector and matrix operations, this is a source of a lot of performance degradation.

The degradation occurs because every time we want to fetch an element from the grid matrix, we must do multiple lookups. For example, doing `grid[5][2]` requires us to first do a list lookup for index 5 on the list `grid`. This will return a pointer to where the data at that location is stored. Then we need to do another list lookup on this returned object, for the element at index 2. Once we have this reference, we have the location where the actual data is stored.



Rather than creating a grid as a list-of-lists (`grid[x][y]`), how would you create a grid indexed by a tuple (`grid[(x, y)]`)? How would this affect the performance of the code?

The overhead for one such lookup is not big and can be, in most cases, disregarded. However, if the data we wanted was located in one contiguous block in memory, we could move *all* of the data in one operation instead of needing two operations for each element. This is one of the major points with data fragmentation: when your data is fragmented, you must move each piece over individually instead of moving the entire block over. This means you are invoking more memory transfer overhead, and you are forcing the CPU to wait while data is being transferred. We will see with `perf` just how important this is when looking at the `cache-misses`.

This problem of getting the right data to the CPU when it is needed is related to the *von Neumann bottleneck*. This refers to the limited bandwidth that exists between the memory and the CPU as a result of the tiered memory architecture that modern computers use. If we could move data infinitely fast, we would not need any cache, since the CPU could retrieve any data it needed instantly. This would be a state in which the bottleneck is nonexistent.

Since we can't move data infinitely fast, we must prefetch data from RAM and store it in smaller but faster CPU caches so that, hopefully, when the CPU needs a piece of data, it will be in a location that can be read from quickly. While this is a severely idealized way of looking at the architecture, we can still see some of the problems with it—how do we know what data will be needed in the future? The CPU does a good job with mechanisms called *branch prediction* and *pipelining*, which try to predict the next instruction and load the relevant portions of memory into the cache while still working on the current instruction. However, the best way to minimize the

effects of the bottleneck is to be smart about how we allocate our memory and how we do our calculations over our data.

Probing how well memory is being moved to the CPU can be quite hard; however, in Linux the `perf` tool can be used to get amazing amounts of insight into how the CPU is dealing with the program being run.⁴ For example, we can run `perf` on the pure Python code from [Example 6-6](#) and see just how efficiently the CPU is running our code.

The results are shown in [Example 6-8](#). Note that the output in this example and the following `perf` examples has been truncated to fit within the margins of the page. The data removed for formatting includes indications of variances for each measurement, showing how much the values changed throughout multiple benchmarks. This is useful for seeing how much a measured value is dependent on the actual performance characteristics of the program versus other system properties, such as other running programs using system resources.

Example 6-8. Performance counters for pure Python 2D diffusion with reduced memory allocations (grid size: 512×512 , 1,000 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_python_memory.py

Performance counter stats for 'python diffusion_python_memory.py':

   526,056,413,046      cycles                    #    3.190 GHz
  1,108,102,913,558      instructions             #    2.11  insn per cycle
    1,631,523,018        cache-references          #    9.893 M/sec
      64,843,956         cache-misses             #    3.974 % of all cache refs
   213,479,087,583      branches                  # 1294.434 M/sec
    2,555,147,651        branch-misses            #    1.20% of all branches
    164,920.79 msec      task-clock                #    1.000 CPUs utilized
         12,593          faults                   #    0.076 K/sec
         12,593          page-faults              #    0.076 K/sec
         12,593          minor-faults             #    0.076 K/sec
           452           cs                       #    0.003 K/sec
           19           migrations                #    0.000 K/sec

   164.928479100 seconds time elapsed
```

⁴ On macOS you can get similar metrics by using Google's [gperftools](#) and the provided Instruments app. For Windows, we are told Visual Studio Profiler works well; however, we don't have experience with it.

Understanding perf

Let's take a second to understand the various performance metrics that `perf` is giving us and their connection to our code. The `task-clock` metric tells us how many clock cycles our task took. This is different from the total runtime, because if our program took one second to run but used two CPUs, then `task-clock` would be 2000 (task-clock is generally in milliseconds). Conveniently, `perf` does the calculation for us and tells us, next to this metric, how many CPUs were utilized (where it says "1.000 CPUs utilized"). This number wouldn't be exactly 2 even when two CPUs are being used, though, because the process sometimes relied on other subsystems to do instructions for it (for example, when memory was allocated).

On the other hand, `instructions` tells us how many CPU instructions our code issued, and `cycles` tells us how many CPU cycles it took to run all of these instructions. The difference between these two numbers gives us an indication of how well our code is vectorizing and pipelining. With pipelining, the CPU is able to run the current operation while fetching and preparing the next one.

`cs` (representing "context switches") and `migrations` tell us about how the program is halted in order to wait for a kernel operation to finish (such as I/O), to let other applications run, or to move execution to another CPU core. When a context-switch happens, the program's execution is halted and another program is allowed to run instead. This is a *very* time-intensive task and is something we would like to minimize as much as possible, but we don't have too much control over when this happens. The kernel delegates when programs are allowed to be switched out; however, we can do things to disincentivize the kernel from moving *our* program. In general, the kernel suspends a program when it is doing I/O (such as reading from memory, disk, or the network).

As you'll see in later chapters, we can use asynchronous routines to make sure that our program still uses the CPU even when waiting for I/O, which will let us keep running without being context-switched. In addition, we could set the `nice` value of our program to give our program priority and stop the kernel from context-switching it.⁵ Similarly, `migrations` happen when the program is halted and resumed on a different CPU than the one it was on before, in order to have all CPUs have the same level of utilization. This can be seen as an especially bad context switch, as not only is our program being temporarily halted, but we also lose whatever data we had in the L1 cache (recall that each CPU has its own L1 cache).

⁵ This can be done by running the Python process through the `nice` utility (`nice -n -20 python program.py`). This, however, requires root permission and may not be representative of your real running environment. A `nice` value of `-20` will make sure it yields execution as little as possible.



cs and migrations can be very difficult to control, especially when you do not have exclusive access to the hardware you are running your code on or if your computer is doing multiple tasks. In fact, Micha had issues running the benchmarks for this chapter on her workstation, constantly getting tens to hundreds of thousands of context switches and migrations. After quite a lot of investigation, it turns out that a normally idle process launched periodic processes with a high priority, effectively monopolizing access to the CPU for short but frequent periods. It was only after controlling this process that she was able to reliably get the results presented here. This is why when high numerical performance is a must-have, working on bare metal (i.e., no virtualization) and running one application per machine becomes necessary in order to have control over CPU contention.

page-faults (or just fault) are part of the modern Unix memory allocation scheme. When memory is allocated, the kernel doesn't do much except give the program a reference to memory. Later, however, when the memory is first used, the operating system throws a minor page fault interrupt, which pauses the program that is being run and properly allocates the memory. This is called a *lazy allocation system*. While this method is an optimization over previous memory allocation systems, minor page faults are quite an expensive operation since most of the operations are done outside the scope of the program you are running. There is also a major page fault, which happens when the program requests data from a device (disk, network, etc.) that hasn't been read yet. These are even more expensive operations: not only do they interrupt your program, but they also involve reading from whichever device the data lives on. This sort of page fault does not generally affect CPU-bound work; however, it will be a source of pain for any program that does disk or network reads/writes.⁶

Once we have data in memory and we reference it, the data makes its way through the various tiers of memory (L1/L2/L3 memory—see “[Communications Layers](#)” on [page 8](#) for a discussion of this). Whenever we reference data that is in our cache, the cache-references metric increases. If we do not already have this data in the cache and need to fetch it from RAM, this counts as a cache-miss. We won't get a cache miss if we are reading data we have read recently (that data will still be in the cache) or data that is located *near* data we have recently read (data is sent from RAM into the cache in chunks). Cache misses can be a source of slowdowns when it comes to CPU-bound work, since we need to wait to fetch the data from RAM *and* we interrupt the flow of our execution pipeline (more on this in a second). As a result, reading through an array in order will give many cache-references but not many

⁶ A good survey of the various faults can be found in the article “[Understanding Page Faults and Memory Swap-in/outs: When Should You Worry?](#)”.

cache-misses since if we read element i , element $i + 1$ will already be in cache. If, however, we read randomly through an array or otherwise don't lay out our data in memory well, every read will require access to data that couldn't possibly already be in cache. Later in this chapter we will discuss how to reduce this effect by optimizing the layout of data in memory.

A branch is a time in the code where the execution flow changes. Think of an `if...then` statement—depending on the result of the conditional, we will be executing either one section of code or another. This is essentially a branch in the execution of the code—the next instruction in the program could be one of two things. To optimize this, especially with regard to the pipeline, the CPU tries to guess which direction the branch will take and preload the relevant instructions. When this prediction is incorrect, we will get a branch-miss. Branch misses can be quite confusing and can result in many strange effects (for example, some loops will run substantially faster on sorted lists than on unsorted lists simply because there will be fewer branch misses).⁷

There are a lot more metrics that `perf` can keep track of, many of which are very specific to the CPU you are running the code on. You can run `perf list` to get the list of currently supported metrics on your system. For example, in the previous edition of this book, we ran on a machine that also supported `stalled-cycles-frontend` and `stalled-cycles-backend`, which tell us how many cycles our program was waiting for the frontend or backend of the pipeline to be filled. This can happen because of a cache miss, a mispredicted branch, or a resource conflict. The frontend of the pipeline is responsible for fetching the next instruction from memory and decoding it into a valid operation, while the backend is responsible for actually running the operation. These sorts of metrics can help tune the performance of code to the optimizations and architecture choices of a particular CPU; however, unless you are always running on the same chip-set, it may be excessive to worry too much about them.



If you would like a more thorough explanation of what is going on at the CPU level with the various performance metrics, check out Gurbur M. Prabhu's fantastic "[Computer Architecture Tutorial](#)". It deals with the problems at a very low level, which will give you a good understanding of what is going on under the hood when you run your code.

⁷ This effect is beautifully explained in this [Stack Overflow response](#).

Making Decisions with perf's Output

With all this in mind, the performance metrics in [Example 6-8](#) are telling us that while running our code, the CPU had to reference the L1/L2 cache 1,631,523,018 times. Of those references, 64,843,956 (or 3.974%) were requests for data that wasn't in memory at the time and had to be retrieved. In addition, we can see that in each CPU cycle we are able to perform an average of 2.11 instructions, which tells us the total speed boost from pipelining, out-of-order execution, and hyperthreading (or any other CPU feature that lets you run more than one instruction per clock cycle).

Fragmentation increases the number of memory transfers to the CPU. Additionally, since you don't have multiple pieces of data ready in the CPU cache when a calculation is requested, you cannot vectorize the calculations. As explained in [“Communications Layers” on page 8](#), vectorization of computations (or having the CPU do multiple computations at a time) can occur only if we can fill the CPU cache with all the relevant data. Since the bus can only move contiguous chunks of memory, this is possible only if the grid data is stored sequentially in RAM. Since a list stores pointers to data instead of the actual data, the actual values in the grid are scattered throughout memory and cannot be copied all at once.

We can alleviate this problem by using the `array` module instead of lists. These objects store data sequentially in memory, so that a slice of the array actually represents a continuous range in memory. However, this doesn't completely fix the problem—now we have data that is stored sequentially in memory, but Python still does not know how to vectorize our loops. What we would like is for any loop that does arithmetic on our array one element at a time to work on chunks of data, but as mentioned previously, there is no such bytecode optimization in Python (partly because of the extremely dynamic nature of the language).



Why doesn't having the data we want stored sequentially in memory automatically give us vectorization? If we look at the raw machine code that the CPU is running, vectorized operations (such as multiplying two arrays) use a different part of the CPU and different instructions than nonvectorized operations. For Python to use these special instructions, we must have a module that was created to use them. We will soon see how `numpy` gives us access to these specialized instructions.

Furthermore, because of implementation details, using the `array` type when creating lists of data that must be iterated on is actually *slower* than simply creating a `list`. This is because the `array` object stores a very low-level representation of the numbers it stores, and this must be converted into a Python-compatible version before being returned to the user. This extra overhead happens every time you index an `array` type. That implementation decision has made the `array` object less suitable for math and more suitable for storing fixed-type data more efficiently in memory.

Enter numpy

To deal with the fragmentation we found using `perf`, we must find a package that can efficiently vectorize operations. Luckily, `numpy` has all of the features we need—it stores data in contiguous chunks of memory and supports vectorized operations on its data.

As a result, any arithmetic we do on `numpy` arrays happens in chunks without us having to explicitly loop over each element.⁸ Not only is it much easier to do matrix arithmetic this way, but it is also faster.

Let's look at an example of calculating vector norm-squared values:

```
from array import array
import numpy

def norm_square_list(vector):
    """
    >>> vector = list(range(1_000_000))
    >>> %timeit norm_square_list(vector)
    48.3 ms ± 678 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
    """
    norm = 0
    for v in vector:
        norm += v * v
    return norm

def norm_square_list_comprehension(vector):
    """
    >>> vector = list(range(1_000_000))
    >>> %timeit norm_square_list_comprehension(vector)
    44.8 ms ± 490 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
    """
    return sum([v * v for v in vector])
```

⁸ For an in-depth look at `numpy` over a variety of problems, check out *From Python to Numpy* by Nicolas P. Rougier.

```

def norm_square_array(vector):
    """
    >>> vector_array = array('l', range(1_000_000))
    >>> %timeit norm_square_array(vector_array)
    66.4 ms ± 1.2 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
    """
    norm = 0
    for v in vector:
        norm += v * v
    return norm

def norm_square_numpy(vector):
    """
    >>> vector_np = numpy.arange(1_000_000)
    >>> %timeit norm_square_numpy(vector_np)
    999 µs ± 5.16 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
    """
    return numpy.sum(vector * vector) ❶

def norm_square_numpy_dot(vector):
    """
    >>> vector_np = numpy.arange(1_000_000)
    >>> %timeit norm_square_numpy_dot(vector_np)
    476 µs ± 12.9 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
    """
    return numpy.dot(vector, vector) ❷

```

- ❶ This creates two implied loops over `vector`, one to do the multiplication and one to do the sum. These loops are similar to the loops from `norm_square_list_comprehension`, but they are executed using numpy's optimized numerical code.
- ❷ This is the preferred way of doing vector norms in numpy by using the vectorized `numpy.dot` operation. The less efficient `norm_square_numpy` code is provided for illustration.

The simpler `numpy` code runs 50 times faster than `norm_square_list` and 46.7 times faster than the “optimized” Python list comprehension. The difference in speed between the pure Python looping method and the list comprehension method shows the benefit of doing more calculation behind the scenes rather than explicitly in your Python code. By performing calculations using Python’s already-built machinery, we get the speed of the native C code that Python is built on. This is also partly the same reasoning behind why we have such a drastic speedup in the `numpy` code: instead of using the generalized list structure, we have a finely tuned and specially built object for dealing with arrays of numbers.

In addition to more lightweight and specialized machinery, the `numpy` object gives us memory locality and vectorized operations, which are incredibly important when dealing with numerical computations. The CPU is exceedingly fast, and most of the time simply getting it the data it needs faster is the best way to optimize code quickly. Running each function using the `perf` tool we looked at earlier shows that the `array` and pure Python functions take about double the number of instructions than the `numpy` version (10G⁹ versus 4G). In addition, the `array` and pure Python versions have on the order of 2G branches, while the `numpy` version has half at 0.9G.

In our `norm_square_numpy` code, when doing `vector * vector`, there is an *implied* loop that `numpy` will take care of. The implied loop is the same loop we have explicitly written out in the other examples: loop over all items in `vector`, multiplying each item by itself. However, since we tell `numpy` to do this instead of explicitly writing it out in Python code, `numpy` can take advantage of all the optimizations it wants. In the background, `numpy` has very optimized C code that has been made specifically to take advantage of any vectorization the CPU has enabled. In addition, `numpy` arrays are represented sequentially in memory as low-level numerical types, which gives them the same space requirements as `array` objects (from the `array` module).

As an added bonus, we can reformulate the problem as a dot product, which `numpy` supports. This gives us a single operation to calculate the value we want, as opposed to first taking the product of the two vectors and then summing them. As you can see in [Figure 6-3](#), this operation, `norm_numpy_dot`, outperforms all the others by quite a substantial margin—this is thanks to the specialization of the function, and because we don’t need to store the intermediate value of `vector * vector` as we did in `norm_numpy`.

9 We’re using the giga (G) suffix to simplify the numbers in the 10⁹ range.

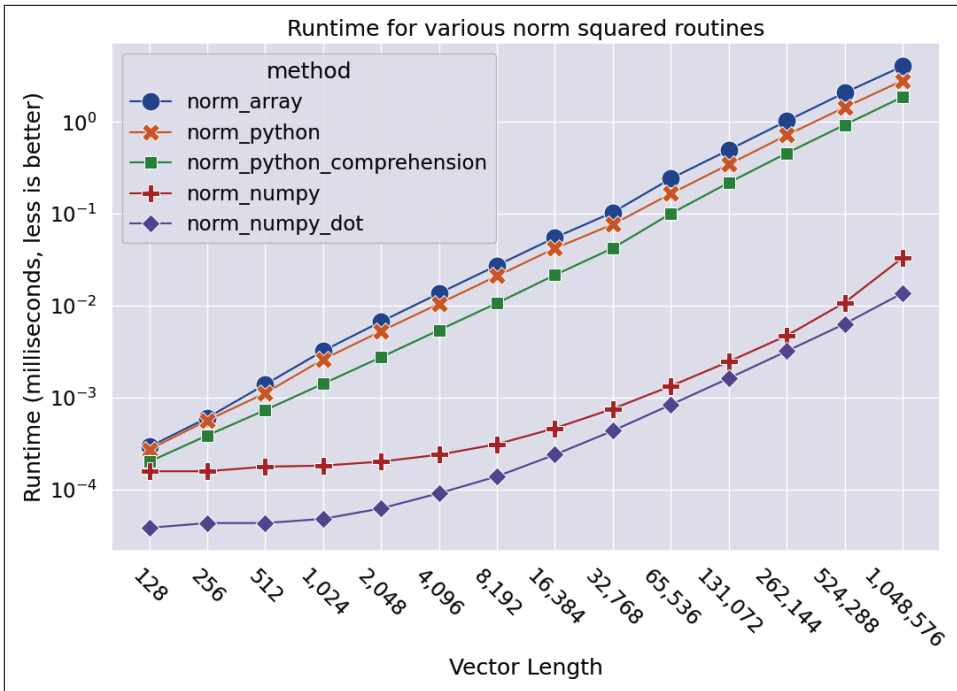


Figure 6-3. Runtimes for the various norm-squared routines with vectors of different lengths

Applying numpy to the Diffusion Problem

Using what we’ve learned about numpy, we can easily adapt our pure Python code to be vectorized. The only new functionality we must introduce is numpy’s `roll` function. This function does the same thing as our modulo-index trick, but it does so for an entire numpy array. In essence, it vectorizes this reindexing:

```
>>> import numpy as np
>>> np.roll([1,2,3,4], 1)
array([4, 1, 2, 3])

>>> np.roll([[1,2,3],[4,5,6]], 1, axis=1)
array([[3, 1, 2],
       [6, 4, 5]])
```

The `roll` function creates a new numpy array, which can be thought of as both good and bad. The downside is that we are taking time to allocate new space, which then needs to be filled with the appropriate data. On the other hand, once we have created this new rolled array, we will be able to vectorize operations on it quite quickly without suffering from cache misses from the CPU cache. This can substantially affect the speed of the actual calculation we must do on the grid. Later in this chapter

we will rewrite this so that we get the same benefit without having to constantly allocate more memory.

With this additional function we can rewrite the Python diffusion code from [Example 6-6](#) using simpler, and vectorized, numpy arrays. [Example 6-9](#) shows our initial numpy diffusion code.

Example 6-9. Initial numpy diffusion

```
from numpy import (zeros, roll)

grid_shape = (512, 512)

def laplacian(grid):
    return (
        roll(grid, +1, 0) +
        roll(grid, -1, 0) +
        roll(grid, +1, 1) +
        roll(grid, -1, 1) -
        4 * grid
    )

def evolve(grid, dt, D=1):
    return grid + dt * D * laplacian(grid)

def run_experiment(num_iterations):
    grid = zeros(grid_shape)

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    grid[block_low:block_high, block_low:block_high] = 0.005

    start = time.time()
    for i in range(num_iterations):
        grid = evolve(grid, 0.1)
    return time.time() - start
```

Immediately we see that this code is much shorter. This is sometimes a good indication of performance: we are doing a lot of the heavy lifting outside the Python interpreter, and hopefully inside a module specially built for performance and for solving a particular problem (however, this should always be tested, as we'll see in [“A Cautionary Tale: Verify “Optimizations” \(scipy\)” on page 179](#)). One of the assumptions here is that numpy is using better memory management to give the CPU the data it needs more quickly. However, since whether this happens or not relies

on the actual implementation of numpy, let's profile our code to see whether our hypothesis is correct. [Example 6-10](#) shows the results.

Example 6-10. Performance counters for numpy 2D diffusion (grid size: 512×512 , 1,000 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy.py

Performance counter stats for 'python diffusion_numpy.py':

11,285,119,496      cycles                    #    3.186 GHz
12,434,411,770      instructions                 #    1.10  insn per cycle
 1,539,129,790      cache-references             # 434.563 M/sec
 142,870,817        cache-misses                #    9.283 % of all cache refs
1,799,214,646       branches                    # 507.997 M/sec
 36,642,230         branch-misses                #    2.04% of all branches
 3,541.78 msec      task-clock                #    1.613 CPUs utilized
 508,036            faults                    #    0.143 M/sec
 508,036            page-faults               #    0.143 M/sec
 508,036            minor-faults              #    0.143 M/sec
      108           cs                      #    0.030 K/sec
      14            migrations                #    0.004 K/sec

2.195482613 seconds time elapsed
```

This shows that the simple change to numpy has given us a 75 times speedup over the pure Python implementation with reduced memory allocations ([Example 6-8](#)). How was this achieved?

Strangely, many of the performance statistics (particularly if we look at the relative percentages) have gone up. More of our cache references were missed (9.28% versus 3.87%). More of the branches were mispredicted (2.0% versus 1.2%). There were many more faults and a decrease in the number of instructions per cycle (1.1 versus 2.1). However, two metrics stand out: instructions and task-clock (in particular the “CPUs utilized” metric).

Using numpy, and thus using specialized code created to manipulate and operate on numerical matrices, we are able to avoid using the entire machinery of Python and thus remove a bunch of general-purpose code. NumPy arrays don't easily support appends, they need to be homogeneous in the types of elements they contain,¹⁰ and they have a whole host of other restrictions when compared to Python lists.

¹⁰ When you try and make an inhomogeneous numpy array—for example, with numbers and strings—numpy instead makes a *homogeneous* array of object types, which can lose many performance benefits of the numpy array.

However, when we do calculations on them, we can use specialized algorithms that are tuned for performance and that don't need to have extra checks or code paths to accommodate the use cases that aren't necessary for us. This theme of trimming out unnecessary machinery will continue in “[Cython](#)” on page 219.

As a result, we reduced the number of instructions by 47.8 times. In addition, the instructions that were left were able to use multiple CPUs. Numerical operations can benefit from a whole array of parallelization options, and most numerical libraries will transparently employ these on our behalf. It seems that `numpy` was able to do this for us in a way that effectively let us use 1.6 cores throughout the runtime of our process.

If we put these two numbers together (the reduced number of operations and the increased number of CPUs to run them on), we can see exactly where our 75 times speedup came from. This reinforces the point that the fastest code is code you don't need to run!

Memory Allocations and In-Place Operations

To optimize the memory-dominated effects, let's try using the same method we used in [Example 6-6](#) to reduce the number of allocations we make in our `numpy` code. Allocations are quite a bit worse than the cache misses we discussed previously. Instead of simply having to find the right data in RAM when it is not found in the cache, an allocation also must make a request to the operating system for an available chunk of data and then reserve it. The request to the operating system generates quite a lot more overhead than simply filling a cache—while filling a cache miss is a hardware routine that is optimized on the motherboard, allocating memory requires talking to another process, the kernel, in order to complete.

To remove the allocations in [Example 6-9](#), we will preallocate some scratch space at the beginning of the code and then use only in-place operations. In-place operations (such as `+=`, `*=`, etc.) reuse one of the inputs as their output. This means that we don't need to allocate space to store the result of the calculation.

To show this explicitly, we'll look at how the `id` of a `numpy` array changes as we perform operations on it ([Example 6-11](#)). The `id` is a good way of tracking this for `numpy` arrays, since the `id` indicates which section of memory is being referenced. If two `numpy` arrays have the same `id`, they are referencing the same section of memory.¹¹

¹¹ This is not strictly true, since any two `numpy` arrays can reference the same section of memory but use different striding information to represent the same data in different ways. These two `numpy` arrays would have different `ids`. There are many subtleties to the `id` structure of `numpy` arrays that are outside the scope of this discussion.

Example 6-11. In-place operations reducing memory allocations

```
>>> import numpy as np
>>> array1 = np.random.random((10,10))
>>> array2 = np.random.random((10,10))
>>> id(array1)
140199765947424 ❶
>>> array1 += array2
>>> id(array1)
140199765947424 ❷
>>> array1 = array1 + array2
>>> id(array1)
140199765969792 ❸
```

- ❶, ❷ These two ids are the same, since we are doing an in-place operation. This means that the memory address of array1 does not change; we are simply changing the data contained within it.
- ❸ Here, the memory address has changed. When doing `array1 + array2`, a new memory address is allocated and filled with the result of the computation. This does have benefits, however, for when the original data needs to be preserved (i.e., `array3 = array1 + array2` allows you to keep using array1 and array2, while in-place operations overwrite the original data).

Furthermore, we can see an expected slowdown from the out-of-place operation. In [Example 6-12](#), we see that using in-place operations gives us a 27% speedup for an array of 100×100 elements. This margin will become larger as the arrays grow, since the memory allocations become more strenuous. However, it is important to note that this effect happens only when the array sizes are bigger than the CPU cache! When the arrays are smaller and the two inputs and the output can all fit into cache, the out-of-place operation is faster because it can benefit from vectorization.

Example 6-12. Runtime differences between in-place and out-of-place operations

```
>>> import numpy as np

>>> %%timeit array1, array2 = np.random.random((2, 100, 100)) ❶ ❸
... array1 = array1 + array2
6.45 µs ± 53.3 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

>>> %%timeit array1, array2 = np.random.random((2, 100, 100)) ❶
... array1 += array2
5.06 µs ± 78.7 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

>>> %%timeit array1, array2 = np.random.random((2, 5, 5)) ❷
... array1 = array1 + array2
518 ns ± 4.88 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

```
>>> %%timeit array1, array2 = np.random.random((2, 5, 5)) ❷
... array1 += array2
1.18 µs ± 6 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

- ❶ These arrays are too big to fit into the CPU cache, and the in-place operation is faster because of fewer allocations and cache misses.
- ❷ These arrays easily fit into cache, and we see the out-of-place operations are faster.
- ❸ Note that we use %%timeit instead of %timeit, which allows us to specify code to set up the experiment that doesn't get timed.

The downside is that while rewriting our code from [Example 6-9](#) to use in-place operations is not very complicated, it does make the resulting code a bit harder to read. In [Example 6-13](#), we can see the results of this refactoring. We instantiate `grid` and `next_grid` vectors, and we constantly swap them with each other. `grid` is the current information we know about the system, and after running `evolve`, `next_grid` contains the updated information.

Example 6-13. Making most numpy operations in-place

```
def laplacian(grid, out):
    np.copyto(out, grid)
    out *= -4 ❶
    out += np.roll(grid, +1, 0)
    out += np.roll(grid, -1, 0)
    out += np.roll(grid, +1, 1)
    out += np.roll(grid, -1, 1)

def evolve(grid, dt, out, D=1):
    laplacian(grid, out)
    out *= D * dt
    out += grid

def run_experiment(num_iterations):
    next_grid = np.zeros(grid_shape)
    grid = np.zeros(grid_shape)

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    grid[block_low:block_high, block_low:block_high] = 0.005

    start = time.time()
    for i in range(num_iterations):
        evolve(grid, 0.1, next_grid)
```

```

    grid, next_grid = next_grid, grid ②
    return time.time() - start

```

- ① Because of order of operations, we start with `grid == out` and then do an in-place multiplication so that `out` becomes `4*grid` and then we can add in the various rolled `grid` values. This gets us at the equation from [Example 6-2](#) by combining the two `2 * grid[i][j]` statements.
- ② Since the output of `evolve` gets stored in the output vector `next_grid`, we must swap these two variables so that, for the next iteration of the loop, `grid` has the most up-to-date information. This swap operation is quite cheap because only the references to the data are changed, not the data itself.



It is important to remember that since we want each operation to be in-place, whenever we do a vector operation, we must put it on its own line. This can make something as simple as `A = A * B + C` become quite convoluted. Since Python has a heavy emphasis on readability, we should make sure that the changes we have made give us sufficient speedups to be justified.

Comparing the performance metrics from [Examples 6-14](#) and [6-10](#), we see that removing the spurious allocations sped up our code by 34.3%. This speedup comes partly from a reduction in the number of cache misses but mostly from a reduction in minor faults. This comes from reducing the number of memory allocations necessary in the code by reusing already allocated space.

Minor faults are caused when a program accesses newly allocated space in memory. Since memory addresses are lazily allocated by the kernel, when you first access newly allocated data, the kernel pauses your execution while it makes sure that the required space exists and creates references to it for the program to use. This added machinery is quite expensive to run and can slow a program down substantially. On top of those additional operations that need to be run, we also lose any state that we had in cache and any possibility of doing instruction pipelining. In essence, we have to drop everything we're doing, including all associated optimizations, in order to go out and allocate some memory.

Example 6-14. Performance metrics for numpy with in-place memory operations (grid size: 512×512 , 1,000 iterations)

```

$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy_memory.py

```

```

Performance counter stats for 'python diffusion_numpy_memory.py':

```


9,497,410,996	cycles	#	3.187 GHz
10,786,935,772	instructions	#	1.14 insn per cycle
1,491,711,066	cache-references	#	500.512 M/sec
82,570,550	cache-misses	#	5.535 % of all cache refs
1,472,992,275	branches	#	494.232 M/sec
33,817,285	branch-misses	#	2.30% of all branches
2,980.37 msec	task-clock	#	1.822 CPUs utilized
13,465	faults	#	0.005 M/sec
13,465	page-faults	#	0.005 M/sec
13,465	minor-faults	#	0.005 M/sec
120	cs	#	0.040 K/sec
25	migrations	#	0.008 K/sec

1.635358825 seconds time elapsed

Selective Optimizations: Finding What Needs to Be Fixed

Looking at the code from [Example 6-13](#), it seems like we have addressed most of the issues at hand: we have reduced the CPU burden by using `numpy`, and we have reduced the number of allocations necessary to solve the problem. However, there is always more investigation to be done. If we do a line profile on that code ([Example 6-15](#)), we see that the majority of our work is done within the `laplacian` function. In fact, 86.9% of the time that `evolve` takes to run is spent in `laplacian`.

Example 6-15. Line profiling shows that `laplacian` is taking too much time

Wrote profile results to `diffusion_numpy_memory.py.lprof`
 Timer unit: 1e-06 s

Total time: 1.49717 s
 File: `diffusion_numpy_memory.py`
 Function: `evolve` at line 28

Line #	Hits	Time	Per Hit	% Time	Line Contents
28					@profile
29					def evolve(grid, dt, out, D=1):
30	1000	1300684.4	1300.7	86.9	laplacian(grid, out=out)
31	1000	84306.0	84.3	5.6	out *= D * dt
32	1000	112176.4	112.2	7.5	out += grid

There could be many reasons `laplacian` is so slow. However, there are two main high-level issues to consider. First, it looks like the calls to `np.roll` are allocating new vectors (we can verify this by looking at the documentation for the function). This means that even though we removed seven memory allocations in our previous refactoring, four outstanding allocations remain. Furthermore, `np.roll` is a very generalized function that has a lot of code to deal with special cases. Since we know

exactly what we want to do (move just the first column of data to be the last in every dimension), we can rewrite this function to eliminate most of the spurious code. We could even merge the `np.roll` logic with the add operation that happens with the rolled data to make a very specialized `roll_add` function that does exactly what we want with the fewest number of allocations and the least extra logic.

Example 6-16 shows what this refactoring would look like. All we need to do is create our new `roll_add` function and have `laplacian` use it. Since numpy supports fancy indexing, implementing such a function is just a matter of not jumbling up the indices. However, as stated earlier, while this code may be more performant, it is much less readable.



Notice the extra work that has gone into having an informative docstring for the function, in addition to full tests. When you are taking a route similar to this one, it is important to maintain the readability of the code, and these steps go a long way toward making sure that your code is always doing what it was intended to do and that future programmers can modify your code and know what things do and when things are not working.

Example 6-16. Creating our own customized roll function

```
import numpy as np
```

```
def roll_add(rollee, shift, axis, out):
```

```
    """
```

```
    Given a matrix, a rollee, and an output matrix, out, this function will
    perform the calculation:
```

```
    >>> out += np.roll(rollee, shift, axis=axis)
```

```
    This is done with the following assumptions:
```

```
    * rollee is 2D
```

```
    * shift will only ever be +1 or -1
```

```
    * axis will only ever be 0 or 1 (also implied by the first assumption)
```

```
    Using these assumptions, we are able to speed up this function by avoiding
    extra machinery that numpy uses to generalize the roll function and also
    by making this operation intrinsically in-place.
```

```
    """
```

```
    if shift == 1 and axis == 0:
```

```
        out[1:, :] += rollee[:-1, :]
```

```
        out[0, :] += rollee[-1, :]
```

```
    elif shift == -1 and axis == 0:
```

```
        out[:-1, :] += rollee[1:, :]
```

```
        out[-1, :] += rollee[0, :]
```

```
    elif shift == 1 and axis == 1:
```

```

        out[:, 1:] += rollee[:, :-1]
        out[:, 0] += rollee[:, -1]
    elif shift == -1 and axis == 1:
        out[:, :-1] += rollee[:, 1:]
        out[:, -1] += rollee[:, 0]

def test_roll_add():
    rollee = np.asarray([[1, 2], [3, 4]])
    for shift in (-1, +1):
        for axis in (0, 1):
            out = np.asarray([[6, 3], [9, 2]])
            expected_result = np.roll(rollee, shift, axis=axis) + out
            roll_add(rollee, shift, axis, out)
            assert np.all(expected_result == out)

def laplacian(grid, out):
    np.copyto(out, grid)
    out *= -4
    roll_add(grid, +1, 0, out)
    roll_add(grid, -1, 0, out)
    roll_add(grid, +1, 1, out)
    roll_add(grid, -1, 1, out)

```

If we look at the performance counters in [Example 6-17](#) for this rewrite, we see that it is 7% faster than [Example 6-13](#). The major difference here is a slight decrease in cache-misses, faults, and branch-misses that are leading to more effective pipelining (through a higher “insn per cycle”) and better parallelization (through a higher “CPUs utilized”). We also see in our results summary in [“Lessons from Matrix Optimizations” on page 180](#) that this speed benefit increases as we increase the grid size.

Example 6-17. Performance metrics for numpy with in-place memory operations and custom laplacian function (grid size: 512×512 , 1,000 iterations)

```

$ perf stat -e cycles,instructions,\
  cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
  minor-faults,cs,migrations python diffusion_numpy_memory2.py

```

Performance counter stats for 'python diffusion_numpy_memory2.py':

9,132,186,073	cycles	#	3.188 GHz
11,037,085,864	instructions	#	1.21 insn per cycle
1,645,897,561	cache-references	#	574.540 M/sec
80,338,970	cache-misses	#	4.881 % of all cache refs
1,487,004,323	branches	#	519.075 M/sec
32,082,251	branch-misses	#	2.16% of all branches
2,864.72 msec	task-clock	#	1.875 CPUs utilized
12,494	faults	#	0.004 M/sec

12,494	page-faults	#	0.004 M/sec
12,494	minor-faults	#	0.004 M/sec
109	cs	#	0.038 K/sec
16	migrations	#	0.006 K/sec

1.527731359 seconds time elapsed

numexpr: Making In-Place Operations Faster and Easier



numexpr comes with many optimizations specifically for Intel CPUs. For the tests, we used only AMD chipsets, and so there are more potential benefits to be seen from numexpr when running on compatible hardware.

One downfall of numpy’s optimization of vector operations is that it occurs on only one operation at a time. That is to say, when we are doing the operation $A * B + C$ with numpy vectors, first the entire $A * B$ operation completes, and the data is stored in a temporary vector; then this new vector is added with C . The in-place version of the diffusion code in [Example 6-13](#) shows this quite explicitly.

However, many modules can help with this. numexpr is a module that can take an entire vector expression and compile it into very efficient code that is optimized to minimize cache misses and temporary space used. It can also take advantage of specialized instructions that are supported by your machine—for example, leveraging the “fused multiply-add” instruction most CPUs implement that is perfect for our specific application. In addition, the expressions can utilize multiple CPU cores (see [Chapter 10](#) for more information) and take advantage of the specialized instructions your CPU may support in order to get even greater speedups. numexpr even supports OpenMP, which parallelizes operations across multiple cores on your machine.

It is very easy to change code to use numexpr: all that’s required is to rewrite the expressions as strings with references to local variables. The expressions are compiled behind the scenes (and cached so that calls to the same expression don’t incur the same cost of compilation) and run using optimized code. [Example 6-18](#) shows the simplicity of changing the evolve function to use numexpr. In this case, we chose to use the out parameter of the evaluate function so that numexpr doesn’t allocate a new vector to which to return the result of the calculation.

Example 6-18. Using numexpr to further optimize large matrix operations

```
from numexpr import evaluate

def evolve(grid, dt, next_grid, D=1):
    laplacian(grid, next_grid)
    evaluate("next_grid * D * dt + grid", out=next_grid)
```

An important feature of numexpr is its consideration of CPU caches. It specifically moves data around so that the various CPU caches have the correct data in order to minimize cache misses. When we run `perf` on the updated code (Example 6-19), we see a slowdown. However, if we look at the performance on larger grids, like 2048×2048 , we see an increase in speed (see Table 6-2). Why is this?

Example 6-19. Performance metrics for numpy with in-place memory operations, custom laplacian function, and numexpr (grid size: 512×512 , 1,000 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy_memory2_numexpr.py
```

Performance counter stats for 'python diffusion_numpy_memory2_numexpr.py':

12,791,619,611	cycles	#	3.147 GHz
19,238,862,629	instructions	#	1.50 insn per cycle
1,756,718,536	cache-references	#	432.205 M/sec
105,088,630	cache-misses	#	5.982 % of all cache refs
2,662,001,883	branches	#	654.931 M/sec
38,141,263	branch-misses	#	1.43% of all branches
4,064.55 msec	task-clock	#	2.440 CPUs utilized
12,972	faults	#	0.003 M/sec
12,972	page-faults	#	0.003 M/sec
12,972	minor-faults	#	0.003 M/sec
18,069	cs	#	0.004 M/sec
21	migrations	#	0.005 K/sec

1.665902807 seconds time elapsed

Much of the extra machinery we are bringing into our program with numexpr deals with cache considerations. When our grid size is small and all the data we need for our calculations fits in the cache, this extra machinery simply adds more instructions that don't help performance. In addition, compiling the vector operation that we encoded as a string adds a large overhead. When the total runtime of the program is small, this overhead can be quite noticeable.

However, as we increase the grid size, we should expect to see `numexpr` utilize our cache better than native `numpy` does. In addition, `numexpr` utilizes multiple cores to do its calculation and tries to saturate each of the cores' caches. When the size of the grid is small, the extra overhead of managing the multiple cores overwhelms any possible increase in speed.

The particular computer we are running the code on has a 16 MB L3, 512 KB L2, and 96 KB L1 cache (AMD Ryzen 7 1700). Since we are operating on two arrays, one for input and one for output, we can easily do the calculation for the size of the grid that will fill up our cache. The number of grid elements we can store at the top-level cache in total is $16 \text{ MB} / 64 \text{ bit} = 2,000,000$. Since we have two grids, this number is split between two objects (so each one can be at most $2,000,000 / 2 = 1,000,000$ elements). Finally, taking the square root of this number gives us the size of the grid that uses that many grid elements. All in all, this means that approximately two 2D arrays of size 1000×1000 would fill up the cache. In practice, however, we do not get to fill up the cache ourselves (other programs will fill up parts of the cache). Looking at Tables 6-1 and 6-2, we see that when the grid size jumps from 512×512 to $1,024 \times 1,024$, the `numexpr` code starts to outperform pure `numpy`.

Graphics Processing Units (GPUs)

Graphics processing units (GPUs) are becoming incredibly popular as a method to speed up arithmetic-heavy computational workloads. Originally designed to help handle the heavy linear algebra requirements of 3D graphics, GPUs are particularly well suited for solving easily parallelizable problems.

Interestingly, GPUs themselves are slower than most CPUs if we just look at clock speeds. This may seem counterintuitive, but as we discussed in “Computing Units” on page 2, clock speed is just one measurement of hardware's ability to compute. GPUs excel at massively parallelized tasks because of the staggering number of compute cores they have. CPUs generally have on the order of 12 cores, while modern-day GPUs have thousands. For example, the machine used to run benchmarks for this section (the AMD Ryzen 7 1700 CPU) has 8 cores, each at 3.2 GHz, while the NVIDIA RTX 2080 TI GPU has 4,352 cores, each at 1.35 GHz.¹²

This incredible amount of parallelism can speed up many numerical tasks by a staggering amount. However, programming on these devices can be quite tough. Because of the amount of parallelism, data locality needs to be considered and can be make-or-break in terms of getting speedups. There are many tools out there to write native GPU code (also called `kernels`) in Python, such as `CuPy`. However, the needs

¹² The RTX 2080 TI also includes 544 tensor cores specifically created to help with mathematical operations that are particularly useful for deep learning.

of modern deep learning algorithms have been pushing new interfaces into GPUs that are easy and intuitive to use.

The two front-runners in terms of easy-to-use GPU mathematics libraries are TensorFlow and PyTorch. We will focus on PyTorch for its ease of use and great speed. For comparisons of the performance of TensorFlow and PyTorch, see “[PyTorch vs TensorFlow in 2025: A Comparative Guide of AI Frameworks](#)”.

Dynamic Graphs: PyTorch

PyTorch is a static computational graph tensor library that is particularly user-friendly and has a very intuitive API for anyone familiar with `numpy`. In addition, since it is a tensor library, it has all the same functionality as `numpy`, with the added advantages of being able to create functions through its static computational graph and calculate derivatives of those functions by using a mechanism called `autograd`.



The `autograd` functionality of PyTorch is left out since it isn't relevant to our discussion. However, this module is quite amazing and can take the derivative of any arbitrary function made up of PyTorch operations. It can do it on the fly and at any value. For a long time, taking derivatives of complex functions could have been the makings of a PhD thesis; however, now we can do it incredibly simply and efficiently. While this may also be off-topic for your work, we recommend learning about `autograd` and automatic differentiation in general, as it truly is an incredible advancement in numerical computation.

When we use the term “dynamic computational graph,” we mean that performing operations on PyTorch objects creates a dynamic definition of a program that gets compiled to GPU code in the background when it is executed (exactly like a JIT from “[JIT Versus AOT Compilers](#)” on page 216). Since it is dynamic, changes to the Python code automatically get reflected in changes in the GPU code without an explicit compilation step needed. This hugely aids debugging and interactivity, as compared to static graph libraries like TensorFlow.

In a static graph, like TensorFlow, we first set up our computation and then compile it. From then on, our compute is set in stone, and we can change it only by recompiling the entire thing. With the dynamic graph of PyTorch, we can conditionally change our compute graph or build it up iteratively. This allows us to have conditional debugging in our code or lets us play with the GPU in an interactive session in IPython. The ability to flexibly control the GPU is a complete game changer when dealing with complex GPU-based workloads.

As an example of the library's ease of use as well as its speed, in [Example 6-20](#) we port the numpy code from [Example 6-13](#) to use the GPU using PyTorch.

Example 6-20. PyTorch 2D diffusion

```
from torch import (roll, zeros) ❶

def laplacian(grid, out):
    out.copy_(grid) ❷
    out *= -4
    out += roll(grid, +1, 0)
    out += roll(grid, -1, 0)
    out += roll(grid, +1, 1)
    out += roll(grid, -1, 1)

def evolve(grid, dt, out, D=1):
    laplacian(grid, out=out)
    out *= D * dt
    out += grid

def run_experiment(num_iterations):
    scratch = torch.zeros(grid_shape)
    grid = torch.zeros(grid_shape)

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    grid[block_low:block_high, block_low:block_high] = 0.005

    scratch = scratch.to(device='cuda') ❸
    grid = grid.to(device='cuda')

    start = time.time()
    for i in range(num_iterations):
        evolve(grid, 0.1, scratch)
        grid, scratch = scratch, grid
    return time.time() - start
```

- ❶ This is the main change needed. Since PyTorch's API mimics NumPy's in many ways, often PyTorch can be a drop-in replacement.
- ❷ PyTorch has a convention that methods that mutate a tensor in-place have an underscore suffix. This is one major API difference with NumPy.
- ❸ This is the magical line that moves our tensors to the GPU and GPU-enabled compute for all future operations involving this data.

Most of the work is done in the modified import, where we changed `numpy` to `torch`. In fact, if we just wanted to run our optimized code on the CPU, we could stop there.¹³ To use the GPU, we simply need to move our data to the GPU, and then `torch` will automatically compile all computations we do on that data into GPU code.

As we can see in [Figure 6-4](#), this small code change has given us an incredible speedup compared to the `numpy` and `numexpr` example from [Example 6-13](#).¹⁴ For a 512×512 grid, we have a 10.5 times speedup, and for a $12,288 \times 12,288$ grid we have a 63.6 times speedup! It is interesting that the GPU code doesn't seem to be as affected by increases to grid size as the `numpy` code is.

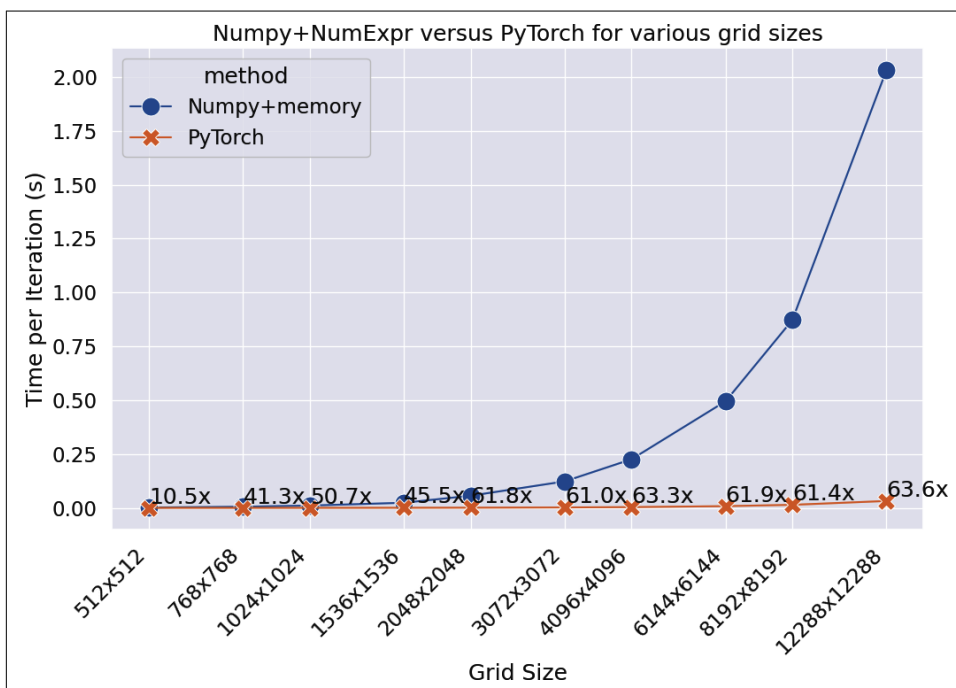


Figure 6-4. Comparison of NumPy and PyTorch on CPU and GPU (using an NVIDIA RTX 2080 TI)

¹³ PyTorch CPU performance isn't terribly great unless you install from source. When installing from source, optimized linear algebra libraries are used to give speeds comparable to NumPy.

¹⁴ As with any JIT, the first time a function is called, it will have the overhead of having to compile the code. In [Example 6-20](#), we profile the functions multiple times and ignore the first time in order to measure only the runtime speeds.

This speedup is a result of how parallelizable the diffusion problem is. As we said before, the GPU we are using has 4,362 independent computation cores. It seems that once the diffusion problem is parallelized, none of these GPU cores are being fully utilized.



When timing the performance of GPU code, it is important to set the environmental flag `CUDA_LAUNCH_BLOCKING=1`. By default, GPU operations are run asynchronously to allow more operations to be pipelined together and thus minimize the total utilization of the GPU and increase parallelization. When asynchronous behavior is enabled, we can guarantee that the computations are done only when either the data is copied to another device or a `torch.cuda.synchronize()` command is issued. By enabling the preceding environment variable, we can make sure that computations are completed when they are issued and that we are indeed measuring compute time.

GPU Speed and Numerical Precision

An interesting thing about GPUs is that their origins are in making graphics for games and not for numerical computation. When doing the calculations necessary for graphics, approximate results have always been acceptable and reducing numerical precision for speed has always been a worthwhile compromise.¹⁵

Nowadays, GPUs are an incredibly powerful hardware addition for neural networks and deep learning. This field also has a lot of robustness regarding approximate or low accuracy calculations. In fact, there is so much randomness involved in training a neural network that even the concept of “high accuracy and precision” seems off-base. As a result, GPUs enable significant speed improvements when operating on lower-precision numbers (i.e., 32- and 16-bit floats, as opposed to the 64 bits that are normal when working in the CPU). In fact, “tensor cores” or “AI Acceleration” in hardware normally takes this to a bigger extreme by having specific hardware to calculate 8- or even 4-bit floats with even higher throughputs (in addition to being specifically tuned to the “multiply-add” operation at the core of deep learning).

As an example, the NVIDIA GeForce RTX 2080 TI has a throughput of 0.4202 TFLOPS for 64-bit floats (that is 420,200,000 floating-point operations per second). However, for 32-bit floats that jumps to 13.45 TFLOPS, and it jumps 26.9 TFLOPS for 16-bit floats! Much of this can be attributed to being able to pack more data on the same bus. For example, if for every clock cycle we can transfer 64 bits, then

¹⁵ For an extreme and beautiful example of this, see the [fast inverse square root algorithm](#) used to make Quake III Arena possible on the limited hardware of its era.

that means we can only transfer one 64-bit float, two 32-bit floats, or four 16-bit floats. This, in addition to specific GPU instructions that can vectorize multiple lower-precision operations at a time, means that more data can be computed at once.

It's important to note that this increase in performance also happens on the CPU, though at quite a different scale. CPUs have specific 32-bit operations and can take advantage of this higher data throughput. However, lower-precision operations (16-bit, 8-bit) are often not natively supported, and you won't get any performance benefits on the CPU for using those types (other than the decreased storage). You may even get a performance penalty because your system must convert the types to a supported precision level or use more generalized, non-float-optimized operations to perform the calculation. This is not the case for GPUs, as low-precision compute is a specific and important use case. We can see in [Example 6-21](#) the differences in speeds we can get on the CPU versus the GPU when lowering precision and the performance penalty using `float16` on the CPU.

Example 6-21. Low-precision CPU performance

```
>>> %%timeit a = np.random.rand(4096, 4096).astype(np.float64)
... a * a + a
...
40.3 ms ± 378 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

>>> %%timeit a = np.random.rand(4096, 4096).astype(np.float32)
... a * a + a
...
20.2 ms ± 63 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

>>> %%timeit a = np.random.rand(4096, 4096).astype(np.float16)
... a * a + a
...
256 ms ± 806 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)

>>> %%timeit a = torch.rand(4096, 4096, dtype=torch.float64, device='cuda')
... a * a + a
...
1.24 ms ± 13.1 ns per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

>>> %%timeit a = torch.rand(4096, 4096, dtype=torch.float32, device='cuda')
... a * a + a
...
623 µs ± 52.1 ns per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

>>> %%timeit a = torch.rand(4096, 4096, dtype=torch.float16, device='cuda')
... a * a + a
...
314 µs ± 4.15 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

In [Example 6-21](#) we can see that in reducing from 64-bit to 32-bit and 16-bit, we get a 2× and a 4× speedup using PyTorch. This makes sense given that we can transfer 2× and 4× more data around with these lower precisions, and it seems that the actual computation can take advantage of this. With numpy we get a 2× speedup when going from 64-bit to 32-bit; however, we get a 6× slowdown when going from 64-bit to 16-bit. This is because the CPU we are using (AMD Ryzen 7 1700) doesn't have the corresponding 16-bit instructions.¹⁶

We can see this consistent 2× and 4× speedup for PyTorch lower precision persists even in our diffusion example. Simply by adding `torch.set_default_dtype(precision)` with precision set to either `torch.float32` or `float16`, we can benefit from this additional compute speed (as seen in [Figure 6-5](#)).



For a great deep dive into float representations, specifically in Python, check out the great talk “[Floats are Friends: Making the Most of IEEE754.000000000000000002](#)” by David Wolever from PyCon 2019.

In addition to `float32` and `float16`, there are also `bfloat16`, `float8_e4m3fn`, and `float8_e5m2`. These last three are nonstandard floating-point numbers that don't follow the standard IEEE floating-point definition. These three allow you to select a floating-point representation at different bit levels that allocate more or less space to the mantissa or exponent of the number. That is to say, they sacrifice precision in favor of range. This is particularly useful in deep learning applications where precision is less valuable than range and performance.

We can understand this trade-off by using the `torch.finfo` function to get dtype information as in [Example 6-22](#). This shows us, with `float16` and `bfloat16`, two numerical representations that use the same amount of space in memory but do so with different compromises.

Example 6-22. Comparison between float16 and bfloat16

```
>>> torch.finfo(torch.float16)
finfo(resolution=0.001, min=-65504, max=65504, eps=0.000976562,
smallest_normal=6.10352e-05, tiny=6.10352e-05, dtype=float16)

>>> torch.finfo(torch.bfloat16)
finfo(resolution=0.01, min=-3.38953e+38, max=3.38953e+38, eps=0.0078125,
smallest_normal=1.17549e-38, tiny=1.17549e-38, dtype=bfloat16)
```

¹⁶ These instructions, called *compressed instructions*, are exceedingly rare for CPUs and are generally only found in special-purpose hardware or embedded devices.

So for float16, the numbers are at minimum 0.001 apart, but for bfloat16 they are 0.01 apart. However, for float16 we have a maximum value of 65,504, while for bfloat16 it is 3.3e38!

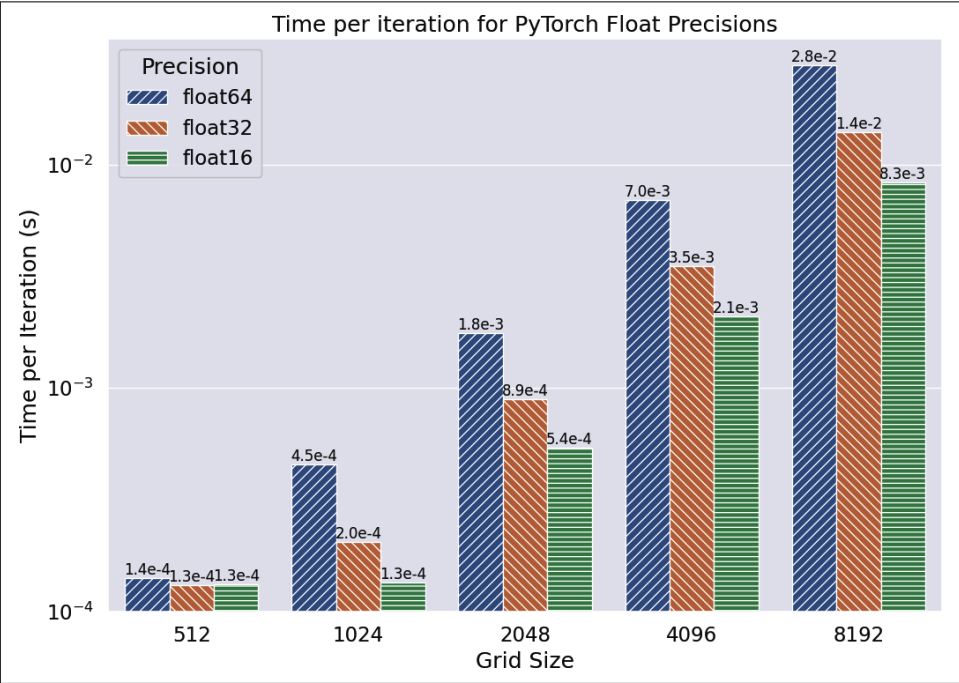


Figure 6-5. Reducing the numerical precision for the PyTorch diffusion example gives us a consistent speedup in performance. Note that the y-axis is log scaled.

If your algorithm is numerically sensitive or you actually care about getting very precise answers, this is not a viable solution for you. However, as we discuss in [Chapter 12](#), many applications don't need exact answers. Knowing this opens up many algorithmic tricks to use this tolerance for lower precision in order to get speed boosts or decreased memory footprints. We will also see in [“Automatic Mixed-Precision” on page 175](#) that PyTorch provides tools to help automatically manage this compromise at a per-operation level to minimally sacrifice precision while still getting speed gains.

GPU-Specific Operations

We can, however, go even faster with the GPU. The Laplacian operation we have been doing can be constructed in terms of a convolution instead of by doing the calculation explicitly as we have been. This is particularly useful for GPUs, since their compute cores have been specifically optimized for this sort of computation.

This hits at the core of GPU programming in Python, and the reason why a dynamic or a static computational graph is important: in the end we are running GPU-specific “kernels” in order to do the calculations we care about. When we tell PyTorch to multiply two matrices, it uses the “matrix multiply” kernel. When we need to take the square root, it uses that kernel. When building up our full compute graph in our Python code, PyTorch has the opportunity to do some optimizations (structure things in memory more efficiently; realize you are always doing a multiply then a divide, so instead of using two kernels it can use the specialized “multiply then divide” kernel). Oftentimes, it’s best to trust that your optimizer is doing the right thing. However, in this case we can shift the problem and specifically use already-optimized code by using a convolution instead of explicit multiplications. These sorts of algorithmic changes will rarely, if ever, be done automatically.

In a convolution, you have a filter (which is just a small matrix) that is overlaid over the matrix you are taking the convolution of, and the filter is multiplied by the subregion of the matrix that it intersects with. The filter is then moved all around so that all parts of the original matrix get this treatment.

We rewrite the Laplacian operation with a convolution in [Example 6-23](#) using a filter with the values from [Figure 6-6](#). This doesn’t automatically take care of our boundary conditions; however, this is something that most convolution operators can do automatically in how they are configured to “pad” the inputs. To implement our cyclical boundary conditions, we set the padding mode to `circular` and the padding size to 1. You can learn more about thinking of Laplacians as a convolution in its [Wikipedia article](#).

0	-1	0
-1	4	-1
0	-1	0

Figure 6-6. 2D laplacian filter

Example 6-23. Diffusion using PyTorch and convolutions

```
def evolve(conv, grid, dt, out, D=1):
    out[:] = conv(grid)
    out *= D * dt
    out += grid

def run_experiment(num_iterations):
    scratch = torch.zeros([1, 1, *grid_shape])
    grid = torch.zeros([1, 1, *grid_shape])
```

```

block_low = int(grid_shape[0] * 0.4)
block_high = int(grid_shape[0] * 0.5)
grid[0, 0, block_low:block_high, block_low:block_high] = 0.005

scratch = scratch.to(device='cuda')
grid = grid.to(device='cuda')

// The following several lines create a reusable GPU convolution object
// with all of the boundary conditions and kernel parameters we need.
kernel = torch.as_tensor(
    [[ 0., -1., 0.],
     [-1., 4., -1.],
     [ 0., -1., 0.]]
).broadcast_to([1, 1, 3, 3]).to(device='cuda')
conv = torch.nn.Conv2d(
    in_channels=1,
    out_channels=1,
    kernel_size=3,
    bias=False,
    padding_mode='circular',
    padding=1
).to(device='cuda')
conv.weight = torch.nn.Parameter(kernel)

start = time.time()
for i in range(num_iterations):
    evolve(conv, grid, 0.1, scratch)
    grid, scratch = scratch, grid
return time.time() - start

```

Here, we are utilizing PyTorch’s neural networks functionality and repurposing the convolutional layer for our purposes. We do this because PyTorch’s direct convolution function doesn’t natively support circular boundary conditions. Because of this, we need to broadcast our grid and kernel to larger sizes; however, the new dimensions are mainly just for bookkeeping. If, however, we wanted to do multiple convolutions at once, this would be easy to do, and in fact GPUs excel at doing multiple batches of work at the same time with no major performance penalty.

For small grid sizes, this change to convolutions is actually slower than before. For a grid size of 512×512 , the time per iteration is 0.00025 versus 0.00014! However, as we start going to larger grids, the convolutions quickly win. Interestingly, we can see in [Figure 6-7](#) that once the grid size surpasses $1,024 \times 1,024$, we start getting a more-or-less consistent 4× speedup using convolutions. This is presumably because of an internal heuristic change in the convolution algorithm that is used depending on the data size.

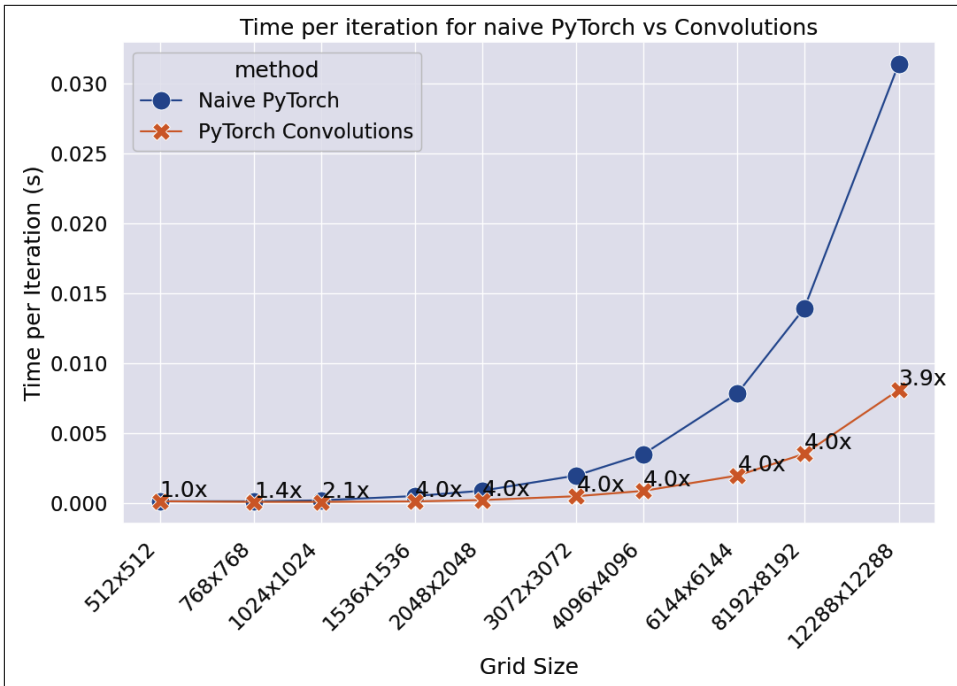


Figure 6-7. Convolutions shows a clear improvement in speed, especially as the grid size increases. Note that the axes are log-log.

This constant factor increase in speed shows that we aren't fundamentally changing the algorithm of what is happening in the background. Instead, we are using a more streamlined implementation of what we were already doing. That is to say, our previous code of rolling the grid and multiplying is exactly the same as what we are doing with the convolutions, but we are now using code optimized to do exactly this on the GPU. As a result, we benefit from all of the internal optimizations to streamline and vectorize the computations.

Basic GPU Profiling

One way to verify exactly how much of the GPU we are utilizing is by using the `nvidia-smi` command to inspect the resource utilization of the GPU. The two values we are most interested in are the power usage and the GPU utilization:

```
$ nvidia-smi
+-----+
| NVIDIA-SMI 535.216.03      Driver Version: 535.216.03      CUDA Version: 12.2      |
+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+
| 0.0  35C    28      12W / 160W          | 1024MiB / 16384MiB | 100%      0.0%      |
+-----+-----+
```


=====							
0	NVIDIA	GeForce	2080TI	Off	00000000:06:00. Off		N/A
41%	67C	P2	198W /	260W	369MiB / 11264MiB		100%
							Default
							N/A
+-----+							
+-----+							
Processes:							
GPU	GI	CI	PID	Type	Process name		GPU Memory
	ID	ID					Usage
=====							
0	N/A	N/A	3914114	C	.../versions/3.12.3/bin/python		366MiB
+-----+							

GPU utilization, here at 100%, is a slightly mislabeled field. It tells us what percentage of the last second has been spent running at least one kernel. So it isn't telling us what percentage of the GPU's total computational power we're using but rather how much time was spent *not* being idle. This is a very useful measurement to look at when debugging memory transfer issues and making sure that the CPU is providing the GPU with enough work.

Power usage, on the other hand, is a good proxy for judging how much of the GPU's compute power is being used. As a rule of thumb, the more power the GPU is drawing, the more compute it is currently doing. If the GPU is waiting for data from the CPU or using only half of the available cores, power use will be reduced from the maximum.

Another useful tool is `gpustat`. This project provides a nice view into many of NVIDIA's stats while using a much friendlier interface than `nvidia-smi`.

To help you understand what specifically is causing slowdowns in your PyTorch code, the project provides a special profiling tool. Running your code with `python -m torch.utils.bottleneck` will show both CPU and GPU runtime stats to help you identify potential optimizations to either portion of your code.

For example, running the code from [Example 6-23](#) through the bottleneck utility gives us the (extremely truncated) profiling results shown in [Example 6-24](#) (the code was run on a 4096 x 4096 grid for 5,000 iterations). From a performance standpoint, we can immediately see our biggest limitation: the padding that we add in our matrix in order to achieve our circular boundary conditions. Following this insight, if we change the `padding_method` to "zeros" instead of circular, we can avoid the slowness of the `_pad_circular` method and get a speedup. Indeed, when we tested this we were able to get a 2x speedup, though at the cost of changing the parameters of the problem we are solving!

Example 6-24. Truncated PyTorch bottleneck profile

```
$ python -m torch.utils.bottleneck diffusion_pytorch_conv.py
```

```
...
```

```
-----  
autograd profiler output (CUDA mode)  
-----
```

```
top 15 events sorted by cpu_time_total
```

```
Because the autograd profiler uses the CUDA event API,  
the CUDA time column reports approximately max(cuda_time, cpu_time).  
Please ignore this output if your code does not use CUDA.
```

```
-----
```

Name	Self CUDA	Self CUDA %	CUDA total	CUDA time avg
aten::pad	12.000us	0.37%	1.915ms	1.915ms
aten::_pad_circular	1.560ms	47.63%	1.903ms	1.903ms
aten::pad	11.000us	0.34%	1.691ms	1.691ms
aten::_pad_circular	140.000us	4.27%	1.680ms	1.680ms
aten::pad	10.000us	0.31%	1.669ms	1.669ms
aten::pad	19.000us	0.58%	1.673ms	1.673ms
aten::pad	11.000us	0.34%	1.664ms	1.664ms
aten::pad	11.000us	0.34%	1.664ms	1.664ms
aten::pad	11.000us	0.34%	1.662ms	1.662ms
aten::pad	10.000us	0.31%	1.661ms	1.661ms
aten::pad	14.000us	0.43%	1.662ms	1.662ms
aten::pad	13.000us	0.40%	1.661ms	1.661ms
aten::_pad_circular	136.000us	4.15%	1.659ms	1.659ms
aten::pad	12.000us	0.37%	1.660ms	1.660ms
aten::_pad_circular	1.305ms	39.85%	1.654ms	1.654ms

```
-----  
Self CPU time total: 4.384ms  
Self CUDA time total: 3.275ms
```

The bottleneck utility is a great starting-off point. However, if you want more granularity and want to get actual profiling logs that you can inspect with other performance tools, we recommend you look into `torch.profile`, or even the `nvprof` utility from NVIDIA. They both give you similar statistics and metrics that you would expect from other profilers like `cProfile`, `gprof`, or `perf`, though they are tuned to GPU applications.

Performance Considerations of GPUs

Since a GPU is a completely auxiliary piece of hardware on the computer, with its own architecture as compared with the CPU, there are many GPU-specific performance considerations to keep in mind.

The biggest speed consideration for GPUs is the transfer time of data from the system memory to the GPU memory. When we use `tensor.to(DEVICE)`, we are triggering a transfer of data that may take some time depending on the speed of the GPU's bus and the amount of data being transferred.

Other operations may trigger a transfer as well. In particular, `tensor.items()` and `tensor.tolist()` often cause problems when introduced for debugging purposes. In fact, running `tensor.numpy()` to convert a PyTorch tensor into a numpy array specifically requires an explicit copy out of the GPU, which ensures you understand the potential penalty.

As an example, let's add a `grid.cpu()` call inside the solver loop of our diffusion code:

```
grid = grid.to(device)
for i in range(num_iterations):
    grid = evolve(grid, 0.1)
    grid.cpu()
```

To ensure that we have a fair comparison, we will also add `torch.cuda.synchronize()` to the control code so that we are simply testing the time to copy the data from the CPU. In addition to this slowing down your code by triggering a data transfer from the GPU to the system memory, your code will slow down because the GPU will pause code execution that would have continued in the background until the transfer is complete.

This change to the code for a $2,048 \times 2,048$ grid slows down our code by 2.54 times! Even though our GPU has an advertised bandwidth of 616.0 GB/s, this overhead can quickly add up. In addition, other overhead costs are associated with a memory copy. First, we are creating a hard stop to any potential pipelining of our code execution. Then, because we are no longer pipelining, our data on the GPU must all be synchronized out of the memory of the individual CUDA cores. Finally, space on system memory needs to be prepared to receive the new data from the GPU.

While this seems like a ridiculous addition to make to our code, this sort of thing happens all the time. In fact, one of the biggest things slowing down PyTorch code when it comes to deep learning is copying training data from the host into the GPU. Often the training data is simply too big to fit on the GPU, and doing these constant data transfers is an unavoidable penalty.

There are ways to alleviate the overhead from this inevitable data transfer when the problem is going from CPU to GPU. First, the memory region can be marked as pinned. This can be done by calling the `Tensor.pin_memory()` method, which returns a copy of the CPU tensor that is copied to a “page locked” region of memory. This page-locked region can be copied to the GPU much more quickly, and it can be copied asynchronously so as to not disturb any computations being done by the

GPU. While training a deep learning model, data loading is generally done with the `DataLoader` class, which conveniently has a `pin_memory` parameter that can automatically do this for all your training data.¹⁷

The most important step is to profile your code by using the tools outlined in “[Basic GPU Profiling](#)” on page 168. When your code is spending most of its time doing data transfers, you will see a low power draw, a smaller GPU utilization (as reported by `nvidia-smi`), and most of your time being spent in the `to` function (as reported by `bottleneck`). Ideally, you will be using the maximum amount of power the GPU can support and have 100% utilization. This is possible even when large amounts of data transfer are required—even when training deep learning models with a large number of images!



GPUs are not particularly good at running multiple tasks at the same time. When starting up a task that requires heavy use of the GPU, ensure that no other tasks are utilizing it by running `nvidia-smi`. However, if you are running a graphical environment, you may have no choice but to have your desktop and GPU code use the GPU at the same time.

When to Use GPUs

We’ve seen that GPUs can be incredibly quick; however, memory considerations and ill-fitting tasks can be quite devastating to this speedup. This seems to indicate that if your task requires mainly linear algebra and matrix manipulations (like multiplication, addition, and Fourier transforms), then GPUs are a fantastic tool. This is particularly true if the calculation can happen on the GPU for a period of time without interruption before being copied back into system memory.

As an example of a task that requires a lot of branching, we can imagine code where every step of the computation requires the previous result. If we compare running [Example 6-25](#) using PyTorch versus using NumPy, we see that NumPy is consistently faster (98% faster for the included example!). This makes sense given the architecture of the GPU. While the GPU can run many more tasks at once than the CPU can, each of those tasks runs more slowly on the GPU than on the CPU. This example task can run only one computation at a time, so having many compute cores doesn’t help at all; it’s better to simply have one very fast core.

¹⁷ The `DataLoader` object also supports running with multiple workers. Having several workers is recommended if data is being loaded from disk in order to minimize I/O time.

Example 6-25. Highly branching task

```
import torch

def task(A, target):
    """
    Given an int array of length N with values from (0, N) and a target value,
    iterates through the array, using the current value to find the next array
    item to look at, until we have seen a total value of at least `target`.
    Returns how many iterations until the value was reached.
    """
    result = 0
    i = 0
    N = 0
    while result < target:
        result += A[i]
        i = A[i]
        N += 1
    return N

if __name__ == "__main__":
    N = 1000

    A_py = (torch.rand(N) * N).type(torch.int).to('cuda:0')
    A_np = A_py.cpu().numpy()

    task(A_py, 500)
    task(A_np, 500)
```

In addition, because of the limited memory of the GPU, it is not a good tool for tasks that require exceedingly large amounts of data, many conditional manipulations of the data, or changing data. Most GPUs made for computational tasks have around 12 to 24 GB of memory, which puts a significant limitation on “large amounts of data.” However, as technology improves, the size of GPU memory increases, so hopefully this limitation becomes less drastic in the future. While there exist GPUs with much larger memory footprints (such as the 80 GB NVIDIA A100) and multi-GPU ways of doing computation, there are major financial hurdles in getting access to these devices.

The general recipe for evaluating whether to use the GPU consists of the following steps:

1. Ensure that the memory use of the problem will fit within the GPU (in [“Using memory_profiler to Diagnose Memory Usage” on page 53](#), we explore profiling memory use).
2. Evaluate whether the algorithm requires a lot of branching conditions versus vectorized operations. As a rule of thumb, numpy functions generally vectorize very well, so if your algorithm can be written in terms of numpy calls, your code probably will vectorize well! You can also check the branches result when running perf (as explained in [“Understanding perf” on page 139](#)).
3. Evaluate how much data needs to be moved between the GPU and the CPU. Some questions to ask here are “How much computation can I do before I need to plot/save results?” and “Are there times my code will have to copy the data to run in a library I know isn’t GPU-compatible?”
4. Make sure PyTorch supports the operations you’d like to do! PyTorch implements a large portion of the numpy API, so this shouldn’t be an issue. For the most part, the API is even the same, so you don’t need to change your code at all. However, in some cases either PyTorch doesn’t support an operation (such as dealing with complex numbers) or the API is slightly different (for example, with generating random numbers).

Considering these four points will help give you confidence that a GPU approach would be worthwhile. There are no hard rules for when the GPU will work better than the CPU, but these questions will help you gain some intuition. However, PyTorch also makes converting code to use the GPU painless, so the barrier to entry is quite low, even if you are just evaluating the GPU’s performance.

Deep Learning Performance Considerations

So far, we have outlined general performance considerations when using GPUs and in particular PyTorch for tensor computation. However, PyTorch is a deep learning library, and it provides many tools to specifically address the performance concerns of that domain.

The considerations are generally the same as what we have already discussed or will discuss in this book. However, PyTorch provides special modules to facilitate the use of the methods for GPUs and for deep learning specifically. Because of this, we won’t dive too deeply into them but instead will give an overview of what to look out for when doing deep learning.

Automatic Mixed-Precision

Automatic Mixed-Precision, or AMP, simplifies much of the work of selecting which floating-point precision to choose for your compute. As discussed in “GPU Speed and Numerical Precision” on page 162, your selection of the level of precision your problem can tolerate can affect the resulting performance of your code quite a lot. AMP provides a way of maximizing the benefits without sacrificing (too much) accuracy. It does this by selecting the lowest precision that still gives high accuracy results for each operation and automatically casting the data to this type for the operation (as seen in Example 6-26). As a result, you never need to explicitly select a floating-point precision and can stick with the floating-point default of 32-bit. However, when you do eligible operations (for example, `multi_dot`, `conv1d`, `conv2d`, or `matmul` on the GPU, to name a few), the data gets autocast to a more performant precision. You can look at the [AMP autocast reference](#) to get a sense of which operations benefit from this. This works also on the CPU, and the results can be quite staggering.

Example 6-26. Automatic Mixed-Precision example

```
>>> a_float32 = torch.rand((2048, 2048), device="cuda") ❶

>>> b_float32 = torch.rand((2048, 2048), device="cuda")

>>> %timeit e_float32 = torch.mm(a_float32, b_float32)
1.32 ms ± 11.3 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)

>>> with torch.amp.autocast(device_type="cuda"):
...     %timeit e_float16 = torch.mm(a_float32, b_float32)
400 µs ± 1.12 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

>>> %%timeit a_float16, b_float16 = a_float32.half(), b_float32.half() ❷
... e_float16 = torch.mm(a_float16, b_float16)
312 µs ± 586 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

- ❶ For brevity, variable names indicate the *type* of that particular tensor. You can verify this by redoing the calculation and outputting the `.dtype` attribute of the tensor.
- ❷ Running `.half()` on a tensor object returns that tensor cast as a `float16` (which is also sometimes called half precision, with `float32` being full precision and `float64` being double precision).

Here we can see some subtleties of AMP’s autocast module. First, we can see that within the autocast context manager, calling `torch.mm` on two `float32`s gives us a `float16`! When doing individual compute using torch, this can become quite the “gotcha,” although when creating models this is generally pretty seamless. We can also see the slight overhead from using autocast versus manually casting the tensors. This

is primarily because the tensors are recast to `float16` before each operation, which requires some overhead. However, we still get a considerable speedup from doing the calculation without any casting!

Interestingly, `autocast` will also upcast low-precision floats to higher precision when you are performing an operation where the final numerical result warrants this. For example, running `torch.sum` on a `float16` in an `autocast` region will result in a `float32`!

As a result, `autocast` is a fantastic way to let `torch` handle the trade-offs between precision and performance. This is especially nice because it can tune the trade-offs depending on what architecture you are running on and what your platform supports.

Model quantization

Model quantization takes lower-precision computing to an extreme. Instead of lowering the precision of specific operations, we lower the precision of an already trained model so that it fundamentally runs at a lower precision. An example of this is a model trained on `float32` or `float16` that is quantized to `int8`. That is to say, quantization is the process of taking data that was computed at a higher precision and moving it to a lower precision.

The benefits of taking a `float32` model and transforming it to an `int8` one are clear—we can hold $4\times$ more data in memory (whether that is a bigger model or more data), and compute should be roughly 4 times faster if our GPU supports `int8` operations. However, going to such extremes in terms of bit representation can come with a big hit to model accuracy if not done correctly. Recall that a `float32` can hold 6.8×10^{44} values while an `int8` can only hold 256. Because of this huge limitation in what the model can represent, model quantization is done after training because of the severe penalty to numerical stability that `int8` would have, and thus it benefits inference.

The two main questions that you'll be asking yourself when quantizing are:

- Will I train the model with quantization in mind (quantization-aware training) or just train a normal floating-point model (post-training quantization)?
- Will I quantize weights and activations (static quantization) or just weights and dynamically quantize activations (dynamic quantization)?

There are many nuances to your choices, but some high-level thought is that quantization-aware training does slow training down and requires extra memory. Also, static quantization does give a smaller and faster model but requires a calibration step in which you need to provide it with a representative dataset to tune the

quantized activation functions (and the resulting model can be quite sensitive to this calibration).

The API and the best practice recommendations for this are evolving quite quickly, so we recommend you check out the [torch.ao documentation](#) or the sister project `torchao`, which provides more cutting-edge quantization support.

Model or data distribution

Just like in any high-compute task, there comes a point where one computer just doesn't cut it anymore. In this case, however, this limitation can come much faster since GPU resources are quite limited. And just like with parallelizing CPU tasks, there are many approaches we can take.

And the parallelism on a GPU follows much of the same paradigms you may already be familiar with from a CPU context:

Data distributed parallelism

We can do the compute on one device just fine; we just have so much data that we want more GPUs processing it at once (good for training).

Model distributed parallelism

The model itself is too big to do the compute on a single GPU (either because of the time it takes or because the model simply doesn't fit in memory), so we need to split the model up over multiple GPUs (good for training or inference).

Data distributed parallelism can be done quite easily—one PyTorch instance can use multiple GPUs on the same machine, and with `torchrun` you can easily connect multiple machines together. Generally, each instance runs through several minibatches of data before all the instances sync their gradients, so that they all effectively learn from the data the other instances have seen. How often this syncing happens is the big trade-off in this scheme—more often and training will be slow, but not often enough and your model may need more epochs to converge (or it won't converge at all!).

Model distributed parallelism requires you to somehow split the model up and place different portions of it on different GPUs. The main subtleties here are, what portions go where, and how/when do we sync these various parts? PyTorch offers three solutions: Fully Sharded Data Parallel (FSDP), Tensor Parallel (TP), and Pipeline Parallel (PP). The TP and PP APIs are still experimental; however, the PP API looks quite promising as an out-of-the-box method that should get you 90% of the way. TP requires a lot of manual work but allows a well-tuned solution. FSDP is a nice general-purpose solution (and not an experimental API!) that loses some of its potential performance gains for its general applicability.

You can learn more about the considerations around these sorts of parallelism for the CPU context in Chapters 10 and 11.

JIT

As we discussed in “[Dynamic Graphs: PyTorch](#)” on page 159, PyTorch uses a dynamic graph to represent the GPU computation that will be done. This is nice because it offers flexibility in what sorts of compute you are doing; however, it makes it harder to optimize this graph to get performance gains. Luckily, the PyTorch project provides a just-in-time compiler (or JIT) that allows you to compile your compute graph and get the benefits of compile-time optimizations when you want them. In addition, these compiled compute graphs are much more portable and can be moved from system to system easily (even loaded directly into C!).

The main workflow when considering whether to use PyTorch’s JIT is to first develop your model without it. Get to a working state on a subset of data where you can use the dynamic nature of PyTorch to help with the debug process. Once things are working, there are two paths you can take:

Scripting mode

Swap out your `torch.nn.Module` subclassing to `torch.jit.ScriptModule` (and decorate any compute methods with `@torch.jit.script_method`).

Trace mode

Run the module through `torch.jit.trace` with some sample input, and any compute that was done on that input will be recorded into a static graph. This can then be frozen and optimized with `torch.jit.freeze`.

Both of these methods will give you substantial speedups for both training and inference. However, they come at a huge cost in terms of flexibility. For the scripting mode method, your code must be deterministic and side-effect free. This is easy for simple models but can quickly become difficult when trying to build more complex systems. For trace mode, you lose any flow control. The final computed graph is only the operations done on the sample data. If there were any `if` statements (or if your code does different things in training mode versus eval mode), you are locked into what happened during the trace.

These are some pretty strict restrictions (more reasons to appreciate the dynamic graph!), but the resulting speed boosts can be worth it.¹⁸

¹⁸ [This tutorial on the PyTorch JIT](#) shows an easy two times speedup on a simple conv-net.

A Cautionary Tale: Verify “Optimizations” (scipy)

An important thing to take away from this chapter is the approach we took to every optimization: profile the code to get a sense of what is going on, come up with a possible solution to fix slow parts, and then profile to make sure the fix actually worked. Although this sounds straightforward, things can get complicated quickly, as we saw in how the performance of `numexpr` depended greatly on the size of the grid we were considering.

Of course, our proposed solutions don’t always work as expected. While writing the code for this chapter, we saw that the `laplacian` function was the slowest routine and hypothesized that the `scipy` routine would be considerably faster. This thinking came from the fact that Laplacians are a common operation in image analysis and probably have a very optimized library to speed up the calls. `scipy` has an image submodule, so we must be in luck!

The implementation was quite simple (Example 6-27) and required little thought about the intricacies of implementing the periodic boundary conditions (or “wrap” condition, as `scipy` calls it).

Example 6-27. Using `scipy`’s `laplace` filter

```
from scipy.ndimage.filters import laplace

def laplacian(grid, out):
    laplace(grid, out, mode="wrap")
```

Ease of implementation is quite important and definitely won this method some points before we considered performance. However, once we benchmarked the `scipy` code (Example 6-28), we had a revelation: this method offers no substantial speedup compared to the code it is based on (Example 6-13). In fact, as we increase the grid size, this method starts performing worse (see Table 6-1 at the end of the chapter).

Example 6-28. Performance metrics for diffusion with `scipy`’s `laplace` function (grid size: 512×512 , 1,000 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_scipy.py
```

Performance counter stats for 'python diffusion_scipy.py':

27,101,948,884	cycles	#	3.188 GHz
25,977,940,518	instructions	#	0.96 insn per cycle
4,973,079,782	cache-references	#	584.901 M/sec
608,156,089	cache-misses	#	12.229 % of all cache refs

4,162,234,707	branches	#	489.534 M/sec
37,407,589	branch-misses	#	0.90% of all branches
8,502.44 msec	task-clock	#	1.447 CPUs utilized
16,398	faults	#	0.002 M/sec
16,398	page-faults	#	0.002 M/sec
16,398	minor-faults	#	0.002 M/sec
58,811	cs	#	0.007 M/sec
25	migrations	#	0.003 K/sec

5.876790262 seconds time elapsed

Comparing the performance metrics of the `scipy` version of the code with those of our custom `laplacian` function (Example 6-17), we can start to get some indication as to why we aren't getting the speedup we were expecting from this rewrite.

The metric that stands out the most is instructions. This shows us that the `scipy` code is requesting that the CPU do more than double the amount of work than our custom `laplacian` code requires. This could be in part because the `scipy` code is written very generally so that it can process all sorts of inputs with different boundary conditions (which requires extra code and thus more instructions). We can see this, in fact, by the high number of branches that the `scipy` code requires. When code has many branches, it means that we run commands based on a condition (like having code inside an `if` statement). The problem is that we don't know whether we can evaluate an expression until we check the conditional, so vectorizing or pipelining isn't possible. The machinery of branch prediction helps with this, but it isn't perfect. This speaks more to the point of the speed of specialized code: if you don't need to constantly check what you need to do and instead know the specific problem at hand, you can solve it much more efficiently.

Lessons from Matrix Optimizations

Looking back on our optimizations, we seem to have taken two main routes: reducing the time taken to get data to the CPU and reducing the amount of work that the CPU had to do. Tables 6-1 and 6-2 compare the results achieved by our various optimization efforts, for various dataset sizes, in relation to the original pure Python implementation.

Figure 6-8 shows graphically how all these methods compared to one another. We can see three bands of performance that correspond to the various improvements. The pure Python bands are at the bottom, the `numpy`/CPU bands are in the middle, and the `PyTorch`/GPU bands are on the top (and the lone `scipy` band is floating in the middle).

Table 6-1. Total runtime of all schemes for various grid sizes and 1,000 iterations of the *evolve* function

	64x64	256x256	4096x4096	512x512	2048x2048	8192x8192
python	2.39s	45.11s	236.08s	4580.22s	18513.58s	74881.26s
python+memory	2.12s	38.87s	208.66s	3950.21s	16582.46s	69609.05s
numpy	0.11s	0.39s	1.94s	53.01s	205.73s	823.73s
numpy+memory	0.11s	0.38s	1.38s	52.86s	212.77s	863.77s
numpy+memory+laplace	0.05s	0.34s	1.29s	36.41s	148.60s	600.51s
numpy+memory+laplace+numexpr	0.18s	0.55s	1.47s	33.95s	135.16s	540.80s
numpy+memory+scipy	0.06s	0.92s	5.55s	208.89s	1413.08s	3620.16s
pytorch	0.12s	0.13s	0.13s	0.91s	3.57s	14.21s
pytorch+convolution	0.10s	0.10s	0.10s	0.24s	0.94s	3.80s

Table 6-2. Speedup compared to naive Python (Example 6-3) for all schemes and various grid sizes over 1,000 iterations of the *evolve* function

	64x64	256x256	4096x4096	512x512	2048x2048	8192x8192
python	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x
python+memory	1.13x	1.16x	1.13x	1.16x	1.12x	1.08x
numpy	21.87x	116.00x	121.45x	86.41x	89.99x	90.90x
numpy+memory	21.86x	120.14x	170.70x	86.64x	87.01x	86.69x
numpy+memory+laplace	44.57x	131.29x	182.90x	125.79x	124.59x	124.70x
numpy+memory+laplace+numexpr	13.22x	82.64x	160.73x	134.92x	136.98x	138.46x
numpy+memory+scipy	39.66x	48.93x	42.56x	21.93x	13.10x	20.68x
pytorch	19.26x	345.43x	1774.02x	5035.74x	5186.95x	5268.71x
pytorch+convolution	24.45x	465.96x	2426.60x	19034.61x	19593.01x	19704.83x

One important lesson to take away from this is that you should always take care of any administrative things the code must do during initialization. This may include allocating memory, or reading configuration from a file, or even precomputing values that will be needed throughout the lifetime of the program. This is important for two reasons. First, you are reducing the total number of times these tasks must be done by doing them once up front, and you know that you will be able to use those resources without too much penalty in the future. Second, you are not disrupting the flow of the program; this allows it to pipeline more efficiently and keep the caches filled with more pertinent data.

You also learned more about the importance of data locality and how important simply getting data to the CPU is. CPU caches can be quite complicated, and often it is best to allow the various provided mechanisms designed to optimize them (such as specialized libraries, compilers, or hardware) to take care of the cache for you. However, understanding what is happening and doing all that is possible to optimize the way memory is handled can make all the difference. For example, by understanding how caches work, we are able to understand that the decrease in performance that leads to a saturated speedup, no matter the grid size, in [Figure 6-8](#) can probably be attributed to the L3 cache being filled up by our grid. When this happens, we stop benefiting from the tiered memory approach to solving the von Neumann bottleneck.

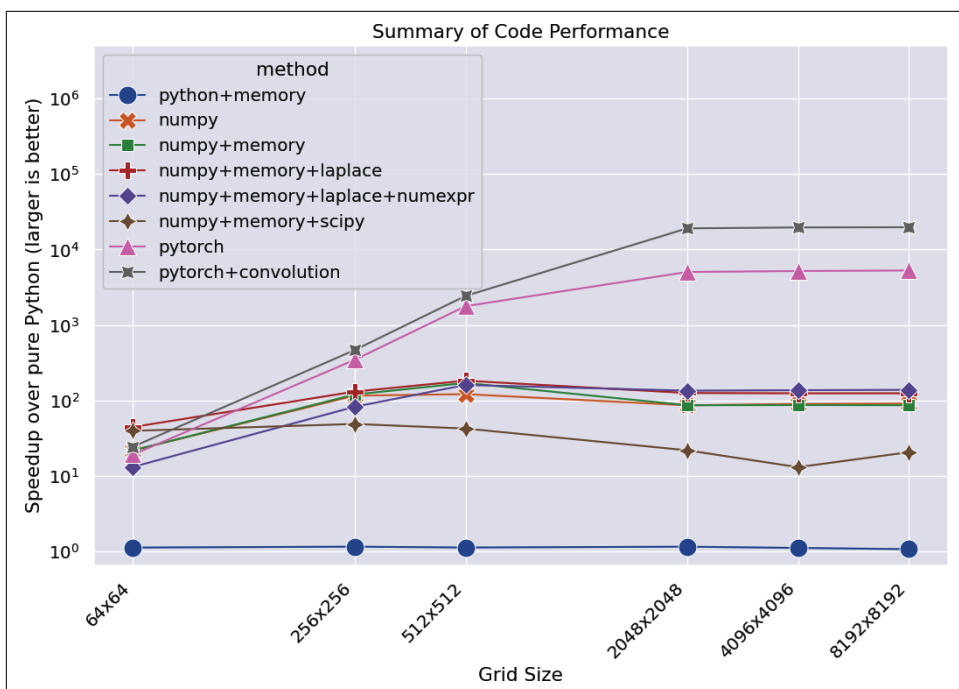


Figure 6-8. Summary of speedups from the methods attempted in this chapter

Another important lesson concerns the use of external libraries. Python is fantastic for its ease of use and its readability, which allow you to write and debug code fast. However, tuning performance down to the external libraries is essential. These external libraries can be extremely fast, because they can be written in lower-level languages—but since they interface with Python, you can also still write code that uses them quickly.

We learned the importance of benchmarking everything and forming hypotheses about performance before running the experiment. By forming a hypothesis before running the benchmark, we are able to form conditions to tell us whether our

optimization actually worked. Was this change able to speed up runtime? Did it reduce the number of allocations? Is the number of cache misses lower? Optimization can be an art at times because of the vast complexity of computer systems, and having a quantitative probe into what is actually happening can help enormously.

One last point about optimization is that a lot of care must be taken to make sure that the optimizations you make generalize to different computers (the assumptions you make and the results of benchmarks you do may be dependent on the architecture of the computer you are running on, how the modules you are using were compiled, etc.). In addition, when making these optimizations, it is incredibly important to consider other developers and how the changes will affect the readability of your code. For example, we realized that the solution we implemented in [Example 6-16](#) was potentially vague, so we took care to make sure that the code was fully documented and tested to help not only us but also other people on the team.

Finally, we dipped our toes into GPUs and saw how many of the same lessons we learned for CPUs are still applicable, though with slight differences (like a cache miss on the CPU is similar, but not as intense as a data transfer done to the GPU). We also learned about some GPU-specific concerns, like the effects of lower-precision and GPU-optimized workloads. And finally, we found ways of benchmarking our GPU code.

Wrap-Up

In the next chapter we'll look at three dataframe libraries, one of which was initially built on NumPy. This builds on our capabilities working on homogenous arrays to allow us to easily manipulate a collection of heterogenous array types.

Typically we build a sequence of operations that must be run on these dataframes; without any optimization across the sequence of operations, we're typically losing performance, as some of the operations may perform redundant or repeated work. Two of the new libraries we'll review (Dask and Polars) have a built-in query optimizer that can make a series of operations far more efficient than with Pandas—the most commonly used library for dataframe operations.

Pandas, Dask, and Polars

Questions You'll Be Able to Answer After This Chapter

- What's the fastest way to apply a function to a Pandas DataFrame?
- What's the quickest way to build a DataFrame from partial results?
- Can we use Numba to compile for more performance inside Pandas?
- Can Dask be used for distributed CPU computation?
- How can Polars execute similar queries faster than Pandas?
- How can I parallelize Pandas using Dask and Swifter?

Many scientific and data science projects use tabular-shaped data that fits into a *dataframe*. A dataframe typically collects a heterogeneous (i.e., *mixed*) collection of datatypes where the datatypes are assigned to columns. Each entry in the dataframe is a row—generally they look like a spreadsheet that you might see in Excel.

Pandas was released in 2008 and quickly became the main dataframe library in the Python ecosystem. As it evolved, a lot of shortcomings were discussed, as later documented in Wes McKinney's infamous 2017 blog post "[Apache Arrow and the '10 Things I Hate About pandas'](#)". At the time of this writing, eight years after that blog post, Pandas is still the most popular Python dataframe library. In the following sections we discuss some of the common ways of writing slow—or fast—Pandas solutions.

Dask, first introduced in 2014, is more than a dataframe library. It is a distributed computing framework for scientific applications—the most famous component is the *distributed dataframe*, which wraps the Pandas API and enables multi-CPU and multimachine execution for high compute and big data applications.

It also works brilliantly on a single machine, and Ian has used it to process very large datasets just on a laptop, using code that's 99% “normal Pandas,” but where the dataset could never easily fit into Pandas. As such, for a Pandas user the Dask library offers a relatively easy path to scaling to larger problems.

Polars is a strong competitor dataframe library released in 2020. Due to its rapid evolution, we'll discuss some results and architectural decisions, but we'll leave out code samples. Anything we write is likely to be out of date shortly after publication.

The project is very interesting, but the API is still evolving (perhaps making it better suited, for now, to research roles or smaller pipelines). The execution speed is typically much faster than if Pandas is used, and some say it is *easier to learn* than Pandas, as the API has more consistency than Pandas's API.

Pandas

First we'll take a look at the Pandas library, which builds upon `numpy` and `arrow` by taking columns of homogeneous data and storing them in a table of heterogeneous types. While Pandas is incredibly popular with scientific developers and data scientists, there's a lot of misinformation about ways to make it run quickly; we address some of these issues and give you tips for writing performant and supportable analysis code.



For this edition we're using Pandas v2.2.0 (2024). When Pandas 3.0 releases, we'll expect to see some performance changes. You'll want to run your own benchmarks.

Sometimes, however, your numerical algorithms also require quite a lot of data wrangling and manipulation that aren't just clear-cut mathematical operations. In these cases, Pandas is a very popular solution, and it has its own performance characteristics. We'll now do a deep dive into Pandas and understand how to better use it to write performant numerical code.



GenAI tools that have consumed old advice for Pandas are likely to mix poor and good optimization suggestions—you'll need to be diligent and run your benchmarks to avoid propagating popular-but-old approaches that might negatively impact performance.

Pandas is the de facto data manipulation tool in the scientific Python ecosystem for tabular data. It enables easy manipulation with Excel-like tables of heterogeneous datatypes, known as DataFrames, and has strong support for time-series operations. Both the public interface and the internal machinery have evolved a lot since 2008, and there's a lot of conflicting information in public forums on “fast ways to solve

problems.” In this section, we’ll clear up some misconceptions about common use cases of Pandas.

We’ll review the internal model for Pandas, find out how to apply a function efficiently across a DataFrame, see why concatenating to a DataFrame repeatedly is a poor way to build up a result, and look at faster ways of handling strings.

Pandas’s Internal Model

Pandas uses an in-memory, 2D, table-like data structure—if you have in mind an Excel sheet, you have a good initial mental model. Originally, Pandas focused on NumPy’s dtype objects, such as signed and unsigned numbers for each column. As the library evolved, it expanded beyond NumPy types and can now handle both Python strings and extension types (including nullable Int64 objects—note the capital “I”—and IP addresses).

Operations on a DataFrame apply to all cells in a column (or all cells in a row if the `axis=1` parameter is used); all operations are executed eagerly; and there’s no support for query planning. Operations on columns often generate temporary intermediate arrays, which consume RAM. The general advice is to expect a temporary memory usage envelope of up to three to five times your current usage when you’re manipulating your DataFrames. Typically, Pandas works well for datasets under 10 GB in size, assuming you have sufficient RAM for temporary results.



Pandas 3.0 will introduce a Copy on Write mode that is expected to be enabled by default. This mode, which can be enabled in Pandas 2.1+, reduces or eliminates many of the background copies, which can significantly reduce memory pressure. Some API changes occur as a consequence, so your code may require some modification. The benefits of the reduced memory pressure include improved execution speed—the changes are well worth your time.

Operations can be single-threaded and may be limited by Python’s GIL. Increasingly, improved internal implementations are allowing the GIL to be disabled automatically, enabling parallelized operations. We’ll explore an approach to parallelization with Dask in [“Dask for Distributed Data Structures and DataFrames” on page 205](#).

Behind the scenes, columns of the same dtype are grouped together by a Block Manager. This piece of hidden machinery works to make row-wise operations on columns of the same datatype faster. It’s one of the many hidden technical details that make the Pandas codebase complex but the high-level user-facing operations faster.¹

¹ See the Dataquest blog post [“Tutorial: Using Pandas with Large Data Sets in Python”](#) for more details.

Performing operations on a subset of data from a single common block typically generates a view, while taking a slice of rows that cross blocks of different dtypes can cause a copy, which may be slower. One consequence is that while numeric columns directly reference their NumPy data, string columns reference a list of Python strings, and these individual strings are scattered in memory—this can lead to unexpected speed differences for numeric and string operations.

Behind the scenes, Pandas uses a mix of NumPy datatypes and its own extension datatypes. Examples from NumPy include `int8` (1 byte), `int64` (8 bytes—and note the lowercase “i”), `float16` (2 bytes), `float64` (8 bytes), and `bool` (1 byte). Additional types provided by Pandas include `categorical` and `datetimez`. Externally, they appear to work similarly, but behind the scenes in the Pandas codebase they cause a lot of type-specific Pandas code and duplication.



While Pandas originally used only numpy datatypes, it has evolved its own set of additional Pandas datatypes that understand missing data (NaN) behavior with three-valued logic. You must distinguish the numpy `int64`, which is not NaN-aware, from the Pandas `Int64`, which uses two columns of data behind the scenes for the integers and for the NaN bit mask. Note that the numpy `float64` is naturally NaN-aware.

One side effect of using NumPy’s datatypes has been that, while a `float` has a NaN (missing value) state, the same is not true for `int` and `bool` objects. If you introduce a NaN value into an `int` or `bool` Series in Pandas, your Series will be promoted to a `float`. Promoting `int` types to a `float` may reduce the numeric accuracy that can be represented in the same bits, and the smallest `float` is `float16`, which takes twice as many bytes as a `bool`.

The nullable `Int64` (note the capitalized “I”) was introduced in version 0.24 as an extension type in Pandas. Internally, it uses a NumPy `int64` and a second Boolean array as a NaN-mask. Equivalents exist for `Int32` and `Int8`. As of Pandas version 1.0, there is also an equivalent nullable Boolean (with dtype `boolean` as opposed to the numpy `bool`, which isn’t NaN-aware). A `StringDType` has been introduced that may in the future offer higher performance and less memory usage than the standard Python `str`, which is stored in a column of object dtype.

Arrow and NumPy

Pandas 2.x introduced PyArrow as a new storage backend to complement the default NumPy array storage. Both backends have different strengths and weaknesses; their behaviors also change with each release of Pandas.

By default, when loading data NumPy was always used in RAM. For floating-point and integer numbers, this was reasonably sensible. For strings this was often awful—the string representation used in NumPy is very expensive on RAM (each string is repeated regardless of whether it is unique) and very slow to operate on.

When the Arrow representation was introduced, we had a new way to store floating point, integer, string, boolean, and date objects. The biggest wins are either with strings (which use a compressed representation and save a huge amount of RAM) or with integers where you want to have missing data without converting the integers to floating point (as happens with NumPy).

In benchmarks that Ian has generated for conference talks, Arrow is *almost always* a better choice for string data than Python strings. For numeric data the RAM usage may not vary much compared to NumPy storage, and execution times may be faster *or slower* depending on the operation.

It would be quite reasonable to use the common NumPy arrays for numeric data and to only use PyArrow for string data. If your data is large and you're loading from a Parquet source (as opposed to `pickle` or `csv` files), loading from Parquet and *converting* to NumPy would be slow—you'd probably be better off overall by staying with PyArrow, as behind the scenes Arrow is used as the data format inside Parquet files.

One performance consideration to remember is that compilation using Numba (see “Numba” on page 234) does not support Arrow data, so only NumPy columns will have faster Numba-backed computation if you're using the `engine` argument.

Applying a Function to Many Rows of Data

It is very common to apply functions to rows of data in Pandas. There's a selection of approaches, and the idiomatic Python approaches using loops are generally the slowest. We'll work through an example based on a real-world challenge, showing different ways of solving this problem and ending with a reflection on the trade-off between speed and maintainability.

Ordinary Least Squares (OLS) is a bread-and-butter method in data science for fitting a line to data. It solves for the slope and intercept in the $m * x + c$ equation, given some data. This can be incredibly useful when trying to understand the trend of the data: is it generally increasing, or is it decreasing?

One example of its use from our personal work was a research project for a telecommunications company where we wanted to analyze a set of potential user-behavior signals (e.g., marketing campaigns, demographics, and geographic behavior). The company has the number of hours a person spends on their cell phone every day, and its question is: is this person increasing or decreasing their usage, and how does this change over time?

One way to approach this problem is to take the company's large dataset of millions of users over years of data and break it into smaller windows of data (each window, for example, representing 14 days out of the years of data). For each window, we model the users' use through OLS and record whether they are increasing or decreasing their usage.

In the end, we have a sequence for each user showing whether, for a given 14-day period, their use was generally increasing or decreasing. However, to get there, we have to run OLS a massive number of times!

For one million users and two years of data, we might have 730 windows,² and thus 730,000,000 calls to OLS! To solve this problem practically, our OLS implementation should be fairly well tuned.

In order to understand the performance of various OLS implementations, we will generate some smaller but representative synthetic data to give us good indications of what to expect on the larger dataset. We'll generate data for 100,000 rows, each representing a synthetic user, and each containing 14 columns representing "hours used per day" for 14 days, as a continuous variable.

We'll draw from a Poisson distribution (with `lambda==60` as minutes) and divide by 60 to give us simulated hours of usage as continuous values. The true nature of the random data doesn't matter for this experiment; it is convenient to use a distribution that has a minimum value of 0, as this represents the real-world minimum. You can see a sample in [Example 7-1](#).

² If we're going to use a sliding window, it might be possible to apply rolling window optimized functions such as `RollingOLS` from `statsmodels`.

Example 7-1. A snippet of our data

```
      0      1      2      ...      12      13
0  1.016667  0.883333  1.033333  ...  1.016667  0.833333
1  1.033333  1.016667  0.833333  ...  1.133333  0.883333
2  0.966667  1.083333  1.183333  ...  1.000000  0.950000
```

In [Figure 7-1](#), we see three rows of 14 days of synthetic data.

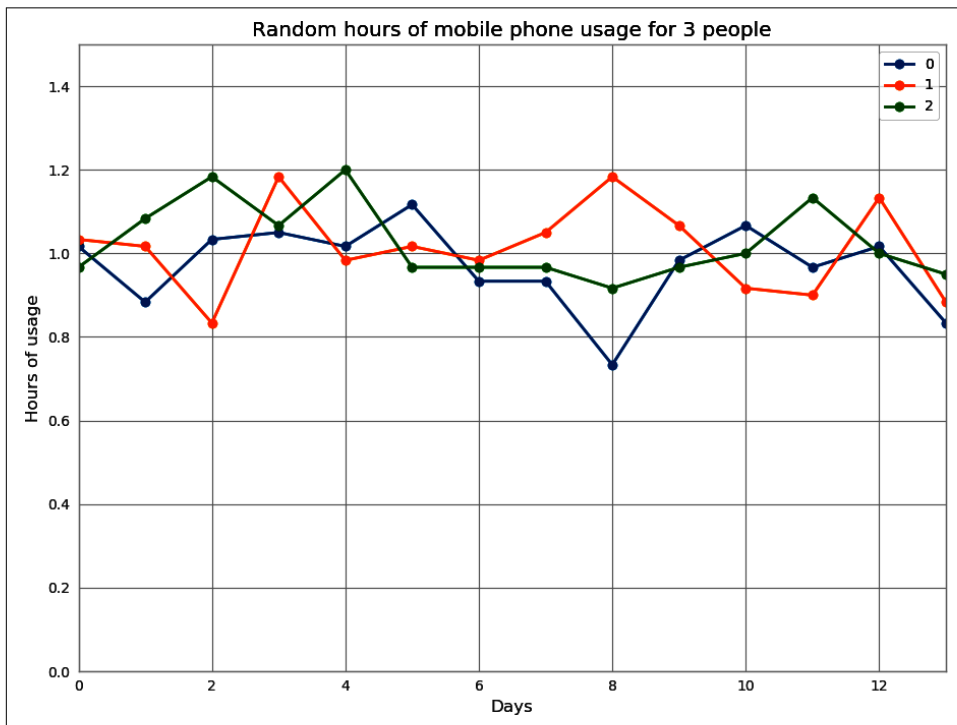


Figure 7-1. Synthetic data for the first three simulated users showing 14 days of cell phone usage

A bonus of generating 100,000 rows of data is that some rows will, by random variation alone, exhibit “increasing counts,” and some will exhibit “decreasing counts.” Note that there is no signal behind this in our synthetic data, since the points are drawn independently; simply because we generate many rows of data, we’re going to see a variance in the ultimate slopes of the lines we calculate.

This is convenient, as we can identify the “most growing” and “most declining” lines and draw them as a validation that we’re identifying the sort of signal we hope to export on the real-world problem. [Figure 7-2](#) shows two of our random traces with maximal and minimal slopes (m).

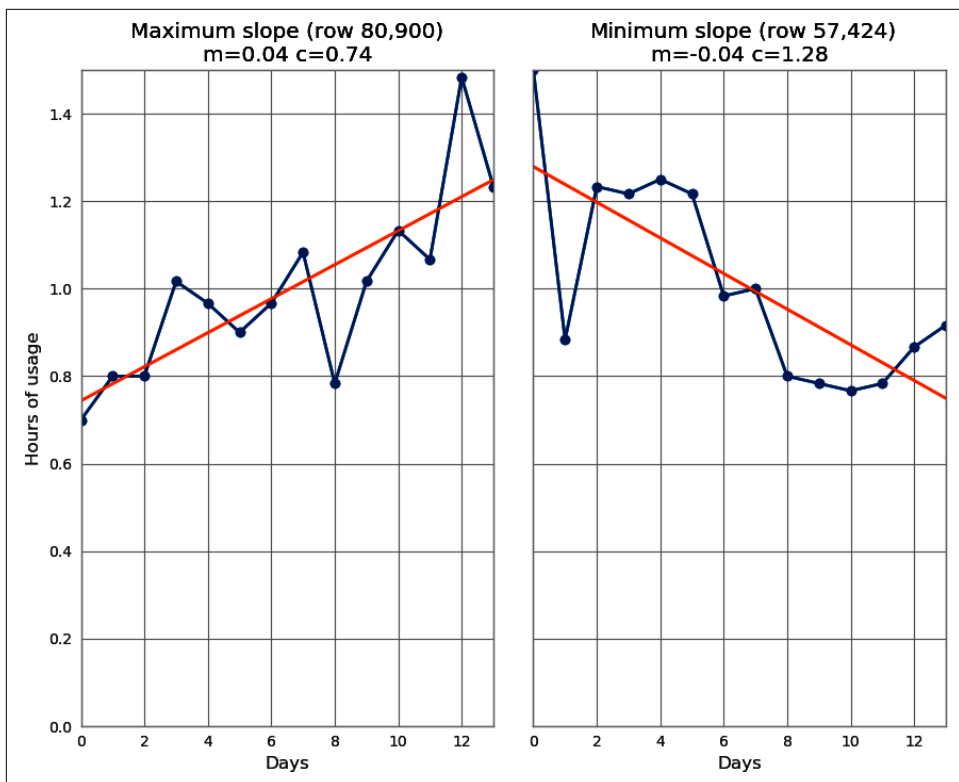


Figure 7-2. The “most increasing” and “most decreasing” usage in our randomly generated dataset

We’ll start with scikit-learn’s `LinearRegression` estimator to calculate each m . While this method is correct, we’ll see in the following section that it incurs a surprising overhead against another approach.

Which OLS implementation should we use?

Example 7-2 shows three implementations that we’d like to try. We’ll evaluate the scikit-learn implementation against the direct linear algebra implementation using NumPy. Both methods ultimately perform the same job and calculate the slopes (m) and intercept (c) of the target data from each Pandas row given an increasing x range (with values $[0, 1, \dots, 13]$).

scikit-learn will be a default choice for many machine learning practitioners, while a linear algebra solution may be preferred by those coming from other disciplines.

Example 7-2. Solving Ordinary Least Squares with NumPy and scikit-learn

```
def ols_sklearn(row):
    """Solve OLS using scikit-learn's LinearRegression"""
    est = LinearRegression()
    X = np.arange(row.shape[0]).reshape(-1, 1) # shape (14, 1)
    # note that the intercept is built inside LinearRegression
    est.fit(X, row.values)
    m = est.coef_[0] # note c is in est.intercept_
    return m

def ols_lstsq(row):
    """Solve OLS using numpy.linalg.lstsq"""
    # build X values for [0, 13]
    X = np.arange(row.shape[0]) # shape (14,)
    ones = np.ones(row.shape[0]) # constant used to build intercept
    A = np.vstack((X, ones)).T # shape(14, 2)
    # lstsq returns the coefficient and intercept as the first result
    # followed by the residuals and other items
    m, c = np.linalg.lstsq(A, row.values, rcond=-1)[0]
    return m

def ols_lstsq_raw(row):
    """Variant of `ols_lstsq` where row is a numpy array (not a Series)"""
    X = np.arange(row.shape[0])
    ones = np.ones(row.shape[0])
    A = np.vstack((X, ones)).T
    m, c = np.linalg.lstsq(A, row, rcond=-1)[0]
    return m
```

Surprisingly, if we call `ols_sklearn` 10,000 times with the `timeit` module, we find that it takes at least 0.515 microseconds to execute, while `ols_lstsq` on the same data takes 0.072 microseconds. The popular scikit-learn solution takes more than seven times as long as the terse NumPy variant!

Building on the profiling from “Using [line_profiler](#) for Line-by-Line Measurements” on [page 48](#), we can use the object interface (rather than the command line or Jupyter magic interfaces) to learn *why* the scikit-learn implementation is slower.



Be aware that `sklearn` makes extensive use of decorators, and `LineProfiler` will profile the decorator, not the underlying `__wrapped__` function; thus it is necessary to follow the advice in this section to look inside the wrapped function to profile the desired (and somewhat hidden) function!

In [Example 7-3](#), we tell `LineProfiler` to profile `est.fit` (that’s the scikit-learn `fit` method on our `LinearRegression` estimator) and then call `run` with arguments based on the `DataFrame` we used before.

Example 7-3. Digging into scikit-learn's `LinearRegression.fit` call and finding the wrapper

```
...
lp = LineProfiler(est.fit)
print("Run on a single row")
lp.run("est.fit(X, row.values)")
lp.print_stats()

Line # ... % Time Contents
=====
1456 ...      @functools.wraps(fit_method)
1457 ...      def wrapper(estimator, *args, **kwargs):
1458 ...          0.2          global_skip_validation = \
                        get_config()["skip_parameter_validation"]
1459 ...
1460 ...          # we don't want to validate again for each
                        call to partial_fit
1461 ...          0.0          partial_fit_and_fitted = (
1462 ...          0.0          fit_method.__name__ == "partial_fit" and \
                        _is_fitted(estimator)
1463 ...          )
1464 ...
1465 ...          0.0          if not global_skip_validation and not \
partial_fit_and_fitted:
1466 ...          13.1          estimator._validate_params()
1467 ...
1468 ...          2.8          with config_context(
1469 ...          skip_parameter_validation=(
1470 ...          0.0          prefer_skip_nested_validation or \
                        global_skip_validation
1471 ...          )
1472 ...          ):
1473 ...          83.9          return fit_method(estimator, *args, **kwargs)
```

The obvious surprise is that we don't profile `.fit` but instead profile a wrapper function! As scikit-learn has evolved, the code's greater complexity makes profiling a bit harder. Reading up on decorators reveals that they contain a `__wrapped__` attribute. We can see that over 80% of the execution is within the `fit_method`—if instead we profile `est.fit.__wrapped__`, then we get what we need in [Example 7-4](#).

Example 7-4. Digging into scikit-learn's `LinearRegression.fit` call behind the wrapper

```
...
lp = LineProfiler(est.fit.__wrapped__) # hook behind the decorator
print("Run on a single row")
lp.run("est.fit(X, row.values)")
lp.print_stats()

Line # ... % Time Line Contents
```

```

===== ... =====
581 ...         @_fit_context(prefer_skip_nested_validation=True)
582 ...         def fit(self, X, y, sample_weight=None):
583 ...             """
584 ...             Fit linear model.
...
604 ...             """
...
609 ...         40.2         X, y = self._validate_data(
610 ...         0.0         X,
611 ...         0.0         y,
612 ...         0.0         accept_sparse=accept_sparse,
613 ...         0.0         y_numeric=True,
614 ...         0.0         multi_output=True,
615 ...         0.0         force_writeable=True,
616 ...         )
...
629 ...         43.8         X, y, X_offset, y_offset, X_scale =
        _preprocess_data(
630 ...         0.0         X,
631 ...         0.0         y,
632 ...         0.0         fit_intercept=self.fit_intercept,
633 ...         0.0         copy=copy_X_in_preprocess_data,
634 ...         0.0         sample_weight=sample_weight)
...
687 ...         13.1         self.coef_, _, self.rank_, self.singular_ =
        linalg.lstsq(X, y)

```

We see a couple of surprises. The very last line of `fit` calls the same `linalg.lstsq` that we've called in `ols_lstsq`—so what else is going on to cause our slowdown? Line Profiler reveals that scikit-learn is calling two other expensive methods, namely `_validate_data` and `_preprocess_data`.

In total the two checker lines take over 85% of the execution time! 13% of our execution is on the `linalg.lstsq` call, which is roughly 1/7th of the time in the function; this explains the seven-time difference to a straight `linalg.lstsq` call noted earlier.

Both of these functions are designed to help us avoid making mistakes—indeed, your author Ian has been saved repeatedly from passing in inappropriate data, such as a wrongly shaped array or one containing NaNs, to scikit-learn estimators. A consequence of this checking is that it takes more time—more safety makes things run slower! We're trading developer time (and sanity) against execution time.

Inside [Example 7-5](#) we add three more functions for profiling (one in the class, one in the module, one in another module). These allow us to see exactly which checks are taking time on this dataset. This required some iteration, digging into the source code of each function, and deciding which additional functions needed to be monitored.

Example 7-5. Digging into scikit-learn's `LinearRegression.fit` call and the underlying functions

```
lp = LineProfiler(est.fit.__wrapped__)
lp.add_function(est._validate_data) # _validate_data is a part of the class
from sklearn.linear_model import _base
lp.add_function(_base._preprocess_data) # _preprocess_data is a module function
from sklearn.utils import check_X_y
lp.add_function(check_X_y)
print("Run on a single row with __wrapped__")
lp.run("est.fit(X, row.values)")
lp.print_stats()
```

Behind the scenes, these two methods are performing various checks, including these:

- Checking for appropriate sparse NumPy arrays (even though we're using dense arrays in this example)
- Offsetting the input array to a mean of 0 to improve numerical stability on wider data ranges than we're using
- Checking that we're providing a 2D X array
- Checking that we're not providing NaN or Inf values
- Checking that we're providing non-empty arrays of data

Generally, we prefer to have all of these checks enabled—they're here to help us avoid painful debugging sessions, which kill developer productivity. If we know that our data is of the correct form for our chosen algorithm, these checks will add a penalty. It is up to you to decide when the safety of these methods is hurting your overall productivity.

scikit-learn has a global parameter system, and using `sklearn.set_config(skip_parameter_validation=True)`, we can disable some of the checks *at the wrapper level*, but not the more expensive checks inside `fit`.

As a general rule, stay with the safer implementations (scikit-learn, in this case) *unless* you're confident that your data is in the right form and you're optimizing for performance. We're after increased performance, so we'll continue with the `ols_lstsq` approach.

Applying `lstsq` to our rows of data

We'll start with an approach that many Python developers who come from other programming languages may try. This is *not* idiomatic Python, nor is it common or even efficient for Pandas. It does have the advantage of being very easy to understand. In [Example 7-6](#), we'll iterate over the index of the DataFrame from row 0 to row

99,999; on each iteration we'll use `iloc` to retrieve a row, and then we'll calculate OLS on that row.

The calculation is common to each of the subsequent methods—what's different is how we iterate over the rows. This method takes 7.3 seconds.

Behind the scenes, each dereference is expensive—`iloc` does a lot of work to get to the row using a fresh `row_idx`, which is then converted into a new Series object, which is returned and assigned to `row`.

Example 7-6. Our worst implementation—counting and fetching rows one at a time with `iloc`

```
ms = []
for row_idx in range(df.shape[0]):
    row = df.iloc[row_idx]
    m = ols_lstsq(row)
    ms.append(m)
results = pd.Series(ms)
```

Next, we'll take a more idiomatic Python approach: in [Example 7-7](#), we iterate over the rows using `iterrows`, which looks similar to how we might iterate over a Python iterable (e.g., a list or set) with a for loop. This method looks sensible and is a little slower—it takes 7.9 seconds. `row` is still created as a fresh Series on each iteration of the loop.

Example 7-7. `iterrows` for more efficient and “Python-like” row operations

```
ms = []
for row_idx, row in df.iterrows():
    m = ols_lstsq(row)
    ms.append(m)
results = pd.Series(ms)
```

[Example 7-8](#) skips a lot of the Pandas machinery. `apply` passes the function `ols_lstsq` a new row of data directly (again, a fresh Series is constructed behind the scenes for each row) without creating Python intermediate references. This costs 3.9 seconds—this is a significant improvement, and the code is more compact and readable! This approach avoids a set of intermediate steps, which explains the speedup.

Example 7-8. `apply` for idiomatic Pandas function application

```
ms = df.apply(ols_lstsq, axis=1)
results = pd.Series(ms)
```

Our final variant in [Example 7-9](#) uses the same `apply` call with an additional `raw=True` argument. Using `raw=True` stops the creation of an intermediate Series object. As we don't have a Series object, we have to use our third OLS function, `ols_lstsq_raw`; this variant has direct access to the underlying NumPy array.

By avoiding the creation and dereferencing of an intermediate Series object, we shave our execution time a little more, down to 3.1 seconds.

Example 7-9. Avoiding intermediate Series creation using `raw=True`

```
ms = df.apply(ols_lstsq_raw, axis=1, raw=True)
results = pd.Series(ms)
```

The use of `raw=True` gives us the option to compile with Numba (see [“Numba to Compile NumPy for Pandas” on page 199](#)) or with Cython, as it removes the complication of compiling Pandas layers that currently aren't supported.

We'll summarize the execution times in [Table 7-1](#) for 100,000 rows of data on a single window of 14 columns of simulated data. New Pandas users often use `iloc` and `iterrows` (or the similar `itertuples`) when `apply` would be preferred.

By performing our analysis and considering our potential need to perform OLS on 1,000,000 rows by up to 730 windows of data, we can see that a first naive approach combining `iloc` with `ols_sklearn` might cost $10 \text{ (our larger dataset factor)} \times 730 \times 7 \text{ seconds} \times 7 \text{ (our slowdown factor against } \text{ols_lstsq}) = 99 \text{ hours}$.

If we used `ols_lstsq_raw` and our fastest approach, the same calculations might take $10 \times 730 \times 3 \text{ seconds} = 6 \text{ hours}$. This is a significant saving for a task that represents what might be a suite of similar operations. We'll see even faster solutions if we compile and run on multiple cores.

Table 7-1. Cost for using `lstsq` with various Pandas row-wise approaches

Method	Time in seconds
<code>iloc</code>	7.3
<code>iterrows</code>	7.9
<code>apply</code>	3.9
<code>apply raw=True</code>	3.1

Earlier we discovered that the scikit-learn approach adds significant overhead to our execution time by covering our data with a safety net of checks. We can remove this safety net but with a potential cost in developer debugging time. Your authors strongly urge you to consider adding unit tests to your code that would verify that a well-known and well-debugged method is used to test any optimized method you settle on.

If you added a unit test to compare the scikit-learn `LinearRegression` approach against `ols_lstsq`, you'd be giving yourself and other colleagues a future hint about why you developed a less obvious solution to what appeared to be a standard problem.

Having experimented, you may also conclude that the heavily tested scikit-learn approach is more than fast enough for your application and that you're more comfortable using a library that is well known by other developers. This could be a very sane conclusion.

In “[Dask for Distributed Data Structures and DataFrames](#)” on page 205, we'll look at running Pandas operations across multiple cores by dividing data into groups of rows using Dask and Swifter.

Next we look at compiling the `raw=True` variant of `apply` to achieve an order of magnitude speedup. Compilation and parallelization can be combined for a really significant final speedup, dropping our expected runtime from around 10 hours to just 30 minutes.

Numba to Compile NumPy for Pandas

In “[Pandas](#)” on page 186, we looked at solving the slope calculation task for 100,000 rows of data in a Pandas DataFrame using Ordinary Least Squares. We can make that approach an order of magnitude faster by using Numba.



Be aware that Numba cannot compile PyArrow arrays, only NumPy arrays. So the compiled speedups work (at the time of writing) only if you're using NumPy for your storage.

We can take the `ols_lstsq_raw` function that we used before and, decorated with `numba.jit` as shown in [Example 7-10](#), generate a compiled variant. In this edition, Numba can compile only NumPy datatypes, not Pandas types like Series.

Example 7-10. Solving Ordinary Least Squares with numpy on a Pandas DataFrame

```
def ols_lstsq_raw(row):
    """Variant of `ols_lstsq` where row is a numpy array (not a Series)"""
    X = np.arange(row.shape[0])
    ones = np.ones(row.shape[0])
    A = np.vstack((X, ones)).T
    m, c = np.linalg.lstsq(A, row, rcond=-1)[0]
    return m
```

```
# generate a Numba compiled variant
```

```
ols_lstsq_raw_values_numba = jit(ols_lstsq_raw, nopython=True)

results = df.apply(ols_lstsq_raw_values_numba, axis=1, raw=True)
```

The first time we call this function in, we get the expected short delay while the function is compiled; processing 100,000 rows takes 4.9 seconds, including compilation time. Subsequent calls to process 100,000 rows are very fast—the noncompiled `ols_lstsq_raw` takes 3.1 seconds per 100,000 rows, whereas after using Numba it takes 0.66 seconds. That’s nearly a fivefold speedup!

More recently, a fast-path for Numba compilation has been added; if instead we supply the function to be compiled (rather than precompiling it) and ask `apply` to use `engine="numba"`, we’ll get a faster outcome. The call will be `results = df.apply(ols_lstsq_raw, axis=1, raw=True, engine="numba")`, and execution drops from 0.66 seconds to 0.40 seconds.



Many Pandas functions have the optional `engine` argument, which lets you enable either the Cython compiler (for small gains) or the Numba compiler (typically for much better gains)—after you’ve profiled and found your bottlenecks, you’ll want to check the documentation to see if this gives you an easy win.

Going one final step, we can ask Numba to use the automated parallelization system to split the task across multiple cores. `results = df.apply(ols_lstsq_raw, axis=1, raw=True, engine="numba", engine_kwargs={'parallel':True})` takes longer to compile (about 10 seconds) on the first run but then drops to 0.13 seconds for each subsequent execution.

By moving from `raw=True` with no compilation to Numba with parallelized execution, we’ve reduced our calculation time from 3.1 seconds to 0.13—a clear 23 times speedup!

Building from Partial Results Rather than Concatenating

You may have wondered why, in [Example 7-6](#), we built up a list of partial results that we then turned into a Series, rather than incrementally building up the Series as we went. Our earlier approach required building up a list (which has a memory overhead) and then building a *second* structure for the Series, giving us two objects in memory. This brings us to another common mistake when using Pandas and NumPy.

As a general rule, you should avoid repeated calls to `concat` in Pandas (and to the equivalent `concatenate` in NumPy). In [Example 7-11](#), we see a similar solution to the preceding one but without the intermediate `ms` list. This solution takes 14 seconds, as opposed to the solution using a list at 7 seconds!

Example 7-11. Concatenating each result incurs a significant overhead—avoid this!

```
results = None
for row_idx in range(df.shape[0]):
    row = df.iloc[row_idx]
    m = ols_lstsq(row)
    if results is None:
        results = pd.Series([m])
    else:
        results = pd.concat((results, pd.Series([m])))
```

Each concatenation creates an entirely *new* Series object in a new section of memory that is one row longer than the previous item. In addition, we have to make a temporary Series object for each new *m* on each iteration.

In [Figure 7-3](#) we capture the time taken to do each 10% of the concatenations. We can see that each subsequent concat operation is slower than the previous one. The overall cost of concatenating 100,000 rows ends up being almost twice as slow by concat as when it started.

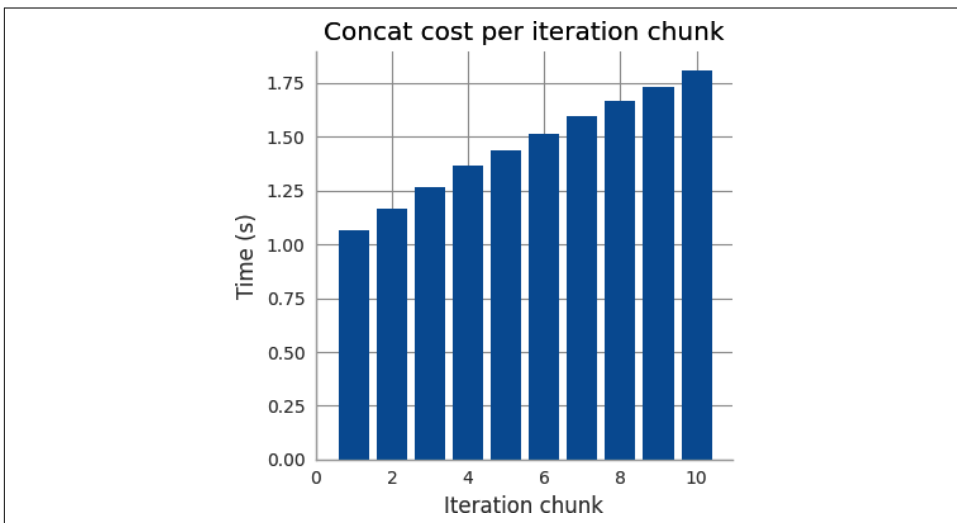


Figure 7-3. Iterative concatenation to a Series becomes increasingly slow

We strongly recommend building up lists of intermediate results and then constructing a Series or DataFrame from this list, rather than concatenating to an existing object.

This same advice applies to building up results in NumPy.

There's More Than One (and Possibly a Faster) Way to Do a Job

Because of the evolution of Pandas, there are typically a couple of approaches to solving the same task, some of which incur more overhead than others. Let's take the OLS DataFrame and convert one column into a string; we'll then time some string operations. With string-based columns containing names, product identifiers, or codes, it is common to have to preprocess the data to turn it into something we can analyze.

Let's say that we need to find the location, if it exists, of the number 9 in the digits from one of the columns. While this operation serves no real purpose, it is very similar to checking for the presence of a code-bearing symbol in an identifier's sequence or checking for an honorific in a name. Typically for these operations, we'd use `strip` to remove extraneous whitespace, `lower` and `replace` to normalize the string, and `find` to locate something of interest.

In [Example 7-12](#), we first build a new Series named `0_as_str`, which is the zeroth Series of random numbers converted into a printable string form. We'll then run two variants of string manipulation code—both will remove the leading digit and decimal point and then use Python's `find` to locate the first 9 if it exists, returning `-1` otherwise.

Example 7-12. str Series operations versus apply for string processing

```
In [10]: df['0_as_str'] = df[0].apply(lambda v: str(v))
Out[10]:
```

	0	0_as_str
0	1.016667	1.0166666666666666
1	1.033333	1.0333333333333334
2	0.966667	0.9666666666666667
...		

```
def find_9(s):
    """Return -1 if '9' not found else its location at position >= 0"""
    return s.split('.')[1].find('9')
```

```
In [11]: df['0_as_str'].str.split('.', expand=True)[1].str.find('9')
Out[11]:
```

0	-1
1	-1
2	0

```
In [12]: %timeit df['0_as_str'].str.split('.', expand=True)[1].str.find('9')
146 ms ± 8.34 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
In [13]: %timeit df['0_as_str'].apply(find_9)
51.7 ms ± 222 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

The one-line approach uses Pandas's `str` operations to access Python's string methods for a Series. For `split`, we expand the returned result into two columns (the first column contains the leading digit, and the second contains everything after the decimal place), and we select column 1. We then apply `find` to locate the digit 9. The second approach uses `apply` and the function `find_9`, which reads like a regular Python string-processing function.

We can use `%timeit` to check the runtime—this shows us that there's a three times speed difference between the two methods, even though they both produce the same result! In the former one-line case, Pandas has to make several new intermediate Series objects, which adds overhead; in the `find_9` case, all of the string-processing work occurs one line at a time without creating new intermediate Pandas objects.

Further benefits of the `apply` approach are that we could parallelize this operation (see [“Dask for Distributed Data Structures and DataFrames” on page 205](#) for an example with Dask and Swifter), and we can write a unit test that succinctly confirms the operations performed by `find_9`, which will aid in readability and maintenance.

Advice for Effective Pandas Development

There's a set of tips we can share that will make your development process with Pandas both easier and more consistent.

Install the optional dependencies `numexpr` and `bottleneck` for additional performance improvements. These don't get installed by default, and you won't be told if they're missing. `numexpr` will give significant speedups in some situations when you use `exec`. You can test for the presence of both in your environment with `import bottleneck` and `import numexpr`.

Don't write your code too tersely; remember to make your code easy to read and debug to help your future self. While the “method chaining” style is supported, your authors would caution against chaining too many rows of Pandas operations in sequence. It typically becomes difficult to figure out which line has problems when debugging, and then you have to split up the lines—you're better off chaining only a couple of operations together, at most, to simplify your maintenance.

Avoid doing more work than necessary: it is preferable to filter your data before calculating on the remaining rows rather than filtering after calculating, if possible. For high performance in general, we want to ask the machine to do as little computation as possible; if you can filter out or mask away portions of your data, you're probably winning.

If you're consuming from a SQL source and later joining or filtering in Pandas, you might want to try to filter first at the SQL level to avoid pulling more data than necessary into Pandas. You may *not* want to do this at first if you're investigating data quality, as having a simplified view on the variety of datatypes you have might be more beneficial.

Check the schema of your DataFrames as they evolve; with a tool like `pandera`, you guarantee at runtime that your schema is being met, and you can visually confirm when you're reviewing code that your expectations are being met.

Keep renaming your columns as you generate new results so that your DataFrame's contents make sense to you; sometimes `groupby` and other operations give you silly default names, which can later be confusing. Drop columns that you no longer need with `.drop()` to reduce bloat and memory usage.

For large Series containing strings with low cardinality ("yes" and "no," for example, or "type_a," "type_b," and "type_c"), try converting the Series to a Category dtype with `df['series_of_strings'].astype('category')`; you may find that operations like `value_counts` and `groupby` run faster, and the Series is likely to consume less RAM. Similarly, you may want to convert 8-byte `float64` and `int64` columns to smaller datatypes—perhaps the 2-byte `float16` or 1-byte `int8` if you need a smaller range to further save RAM.

As you evolve DataFrames and generate new copies, remember that you can use the `del` keyword to delete earlier references and clear them from memory, if they're large and wasting space. You can also use the Pandas drop method to delete unused columns.

If you're manipulating large DataFrames while you prepare your data for processing, it may make sense to do these operations once in a function or in a separate script and then persist the prepared version to disk by using `to_pickle`. You can subsequently work on the prepared DataFrame without having to process it each time.

Avoid the `inplace=True` operator—in-place operations are scheduled to be removed from the library over time.

Finally, always add unit tests to any processing code, as it will quickly become more complex and harder to debug. Developing your tests up front guarantees that your code meets your expectations and helps you to avoid silly mistakes creeping in later that cost developer time to debug.

Existing tools for making Pandas go faster include **Modin** and the GPU-focused **cuDF**. Modin and cuDF take different approaches to parallelizing common data operations on a Pandas DataFrame-like object.

Dask for Distributed Data Structures and DataFrames

Dask aims to provide a suite of parallelization solutions that scales from a single core on a laptop to multicore machines to thousands of cores in a cluster. Think of it as “Apache Spark lite.” If you don’t need all of Apache Spark’s functionality (which includes replicated writes and multimachine failover) and you don’t want to support a second computation and storage environment, then Dask may provide the parallelized and bigger-than-RAM solution you’re after.

Dask is a tried and tested library, commonly used in financial services and scientific organizations where large (often Parquet-based) datasets and CPU-intense computations occur in productionized pipelines.

A task graph is constructed for the lazy evaluation of a number of computation scenarios, including pure Python, scientific Python, and machine learning with small, medium, and big datasets. The tasks in these graphs represent very small units of work such as loading, manipulating, or creating data—they’re combined into discrete steps to solve higher-level problems from Dask’s *collections*, with each serving different scientific needs:

bag

`bag` enables parallelized computation on unstructured and semistructured data, including text files and JSON or user-defined objects. `map`, `filter`, and `groupby` are supported on generic Python objects, including lists and sets.

array

`array` enables distributed and larger-than-RAM NumPy operations. Many common operations are supported, including some linear algebra functions. Operations that are inefficient across cores (sorting, for example, as well as many linear algebra operations) are not supported. Threads are used, as NumPy has good thread support, so data doesn’t have to be copied during parallelized operations.

Distributed DataFrame

`dataframe` enables distributed and larger-than-RAM Pandas operations; behind the scenes, Pandas is used to represent partial DataFrames that have been partitioned using their index. Operations are lazily computed using `.compute()` and otherwise look very similar to their Pandas counterparts. Supported functions include `groupby-aggregate`, `groupby-apply`, `value_counts`, `drop_duplicates`, and `merge`. By default, threads are used, but as Pandas is more GIL-bound than NumPy, you may want to look at the `Process` or `Distributed` scheduler options.

delayed

`delayed` extends the idea we will introduce with Joblib in “[Replacing multiprocessing with Joblib](#)” on page 304 to parallelize *chains* of arbitrary Python functions in a lazy fashion. A `visualize()` function will draw the task graph to assist in diagnosing issues.

future

The `Client` interface enables immediate execution and evolution of tasks, unlike `delayed`, which is lazy and doesn’t allow operations like adding or destroying tasks. The `Future` interface includes `Queue` and `Lock` to support task collaboration.

Dask-ML

A scikit-learn-like interface is provided for scalable machine learning. Dask-ML provides cluster support to some scikit-learn algorithms, and it reimplements some algorithms (e.g., the `linear_model` set) using Dask to enable learning on big data. It closes some of the gap to the Apache Spark distributed machine learning toolkit. It also provides support for XGBoost and TensorFlow to be used in a Dask cluster.

Diagnostics

One of the difficulties with any distributed system is diagnosing errors, failures, and performance issues. Over multiple conversations with end users, Ian has learned that people who learn Dask are pleasantly surprised by the richness of the Dask diagnostic environment compared to a tool like PySpark, which they’ve otherwise been using.

One of the upsides of Dask for a Python team is that all the tracebacks are in Python (unlike perhaps Java for PySpark users).

The main Dask diagnostic dashboard lists:

- Bytes stored per worker (to help identify if workers are running out of RAM)
- Task stream (a live view of color-coded types of work running on each worker)
- Task occupancy (broken down by CPU, Data Transfer, and more)
- Overall progress (a Gantt chart-like display)

These diagnostics alone are very useful for determining whether a cluster is underutilized or either CPU-bound or RAM-bound. Up-to-date diagrams and tutorials can be found on the [Dashboard web page](#).

If a cluster is CPU-bound, then all of the workers sit at 100% CPU usage; you’ll soon wonder what happens if you make more CPUs available.

If a cluster is RAM-bound, then you'd typically see workers being repeatedly killed due to hitting their RAM limit, and that'll make you wonder about either adding RAM, increasing the RAM limit per worker if RAM is available, or reducing the number of workers to make more RAM available to those that remain. Another route is to think about the underlying task graph and to think about whether it can be simplified—perhaps by breaking larger operations into smaller steps, which introduce less RAM pressure.

Each of the distributed DataFrames is a Pandas DataFrame—one of the easiest ways to reduce RAM pressure is to split these DataFrames into smaller units with fewer rows in each.

The task stream can be useful for at-a-glance diagnostics. If you see lots of white-space between tasks, then for some reason your CPUs are idle—often because the right data hasn't yet been provided. This gives you a clue to think about your task graph and how you might experiment with changing your code.

Each task graph can be visualized using `ddf.visualize(filename="graph.png");` this uses `dot` to write out the task graph. While these graphs can be complex to follow, they'll typically comprise repeated blocks of work that you can reasonably easily tie back to each line of code.

In the Dashboard the task graph can be interactively visualized—Ian has found this useful when looking at a live system to think about *where* the graph seems to be stalling. This has led to hypotheses that could be tested until a solution is found.

Parallel Pandas with Dask

For Pandas users, Dask can help in two use cases: larger-than-RAM datasets and a desire for multicore parallelization.

If your dataset is larger than Pandas can fit into RAM, Dask can split the dataset by rows into a set of partitioned DataFrames called a *distributed DataFrame*. These DataFrames are split by their index; a subset of operations can be performed across each partition. As an example, if you have a set of multi-GB CSV files and want to calculate `value_counts` across all the files, Dask will perform partial `value_counts` on each DataFrame (one per file) and then combine the results into a single set of counts.

A large subset of Pandas's common functionality is available with little or no code change, once you're using a distributed DataFrame. Some operations have limits—an `apply` only works with `axis=1` (across-the-row), while the column-wise variant `axis=0` won't work through Dask—due to the data being split into row chunks.

A second use case is to take advantage of the multiple cores on your laptop (and just as easily across a cluster); we'll examine this use case here. Recall that in [Example 7-2](#)

we calculated the slope of the line across rows of values in a `DataFrame` with various approaches. Let's use the two fastest approaches and parallelize them with Dask.



You can use Dask (and Swifter, discussed in the next section) to parallelize any side-effect-free function that you'd usually use in an `apply` call. Ian has done this for numeric calculations and for calculating text metrics on multiple columns of text in a large `DataFrame`.

Dask Expressions

The `dask-expr` library was added to the Dask library in 2024—it introduces a query optimization framework that's mostly invisible to the developer. This library can compress operations to remove duplication or redundancy and so reduce the need to manually tune a set of Dask operations.

An example of inefficiency prior to this would be to read *all* rows from a Parquet dataframe, then filter away many rows, and then choose several columns for a set of operations. We could *manually* request that only a subset of rows that match a predicate filter would be loaded and that only a *subset* of columns (using a projection filter) would be loaded—this significantly reduces the amount of bytes being pulled from storage, but requires intervention by the developer.

One of the goals of `dask-expr` is to identify the subsets of data you need and to pull in only this data—transparently.

As of 2024 this library is a required dependency, and it works transparently in the background on distributed `DataFrames`. If you previously had tried Dask and found that optimizing the query for good performance consumed a lot of your time, you may want to go back and revisit your benchmarks.

Parallelized computations

With Dask, we have to specify the number of *partitions* to make from our `DataFrame`; a good rule of thumb is to use at least as many partitions as cores, so that each core can be used. In [Example 7-13](#), we ask for eight partitions. We use `dd.from_pandas` to convert our regular Pandas `DataFrame` into a Dask distributed `DataFrame` split into eight equal-sized sections.

We call our familiar `ddf.apply` on the distributed `DataFrame`, specifying our function `ols_lstsq` and the optional expected return type via the `meta` argument. Dask requires us to specify when we should apply the computation with the `compute()` call; here, we specify the use of processes rather than the default threads to spread our work over multiple cores, avoiding Python's GIL.

Example 7-13. Calculating line slopes with multiple cores using Dask

```
import dask.dataframe as dd

df = pd.concat([df] * 40) # fake a 40x larger dataframe

N_PARTITIONS = 8
ddf = dd.from_pandas(df, npartitions=N_PARTITIONS, sort=False)
SCHEDULER = "processes"

results = ddf.apply(ols_lstsq, axis=1, meta=(None, 'float64'),). \
    compute(scheduler=SCHEDULER)
```

Running `ols_lstsq_raw` in [Example 7-14](#) with the same eight partitions (on eight physical cores) in Ian’s laptop, we go from the previous single-threaded `apply` on 40 times the original data in 80 seconds to 65 seconds using Dask.

The results are not huge, but they demonstrate that with a CPU-bound function and enough distribution, you could expect to see larger gains, especially on large and slow embarrassingly parallel problems.

Example 7-14. Calculating line slopes with multiple cores using Dask

```
results = ddf.apply(ols_lstsq_raw, axis=1, meta=(None, 'float64'), raw=True). \
    compute(scheduler=SCHEDULER)
```

Using [Example 7-14](#) with `raw=True` further reduces the execution time to 52 seconds. If we also use the compiled Numba function from “[Numba to Compile NumPy for Pandas](#)” on page 199 with `raw=True`, our runtime could drop even further.

Parallelized apply with Swifter on Dask

Swifter builds on Dask to provide three parallelized options with very simple calls—`apply`, `resample`, and `rolling`. Behind the scenes, it takes a subsample of your DataFrame and attempts to vectorize your function call. If that works, Swifter will apply it; if it works but it is slow, Swifter will run it on multiple cores using Dask.

Since Swifter uses heuristics to determine how to run your code, it could run slower than if you didn’t use it at all—but the “cost” of trying it is one line of effort. It is well worth evaluating.

Swifter makes its own decisions about how many cores to use with Dask and how many rows to sample for its evaluation; as a result, in [Example 7-15](#) we see the call to `df.swifter...apply()` looks just like a regular call to `df.apply`. In this case, we’ve disabled the progress bar; the progress bar works fine in a Jupyter Notebook using the excellent `tqdm` library.

Example 7-15. Calculating line slopes with multiple cores using Swifter

```
import swifter

results = df.swifter.progress_bar(False).apply(ols_lstsq_raw, axis=1, raw=True)
```

Swifter with `ols_lstsq_raw` and no partitioning choices takes our previous single-threaded result of 80 seconds to 65 seconds (similar to the Dask result). For this particular function and dataset it doesn't offer much gain—the speedup comes for only one line of code. For different functions and datasets, you'll see different results; it is definitely worth an experiment to see whether you can achieve a very easy win.

Polars for Fast DataFrames

Polars is a young and rapidly evolving competitor to Pandas. Early in its life the API changed rapidly, making it hard to depend on for production systems. Ian has spoken to hedge fund teams who now routinely use Polars both for research and for some production processes, suggesting that it is demonstrating its stability.

In comparison to Pandas, Polars has some clear advantages:

- It has a cleaner API, which is easier to learn.
- It uses Arrow, rather than both Arrow and NumPy (simplifying the codebase).
- It has a query optimizer baked in.
- It supports both greedy and delayed computation.

Ian has found it easier to *think* about a problem when using Polars, as typically there's a more obvious way to express a solution to a problem. In Pandas there are many ways to solve each problem, leading to complex and nonobvious solutions.



Since Polars is young, there are few public examples for GenAI solutions to learn from—be aware that you could well receive suggestions that are hallucinations from related tools like Pandas, which might cause you confusion!

Polars can accept data that's stored in NumPy (e.g., when constructing a DataFrame), but internally it only uses Arrow. By ignoring support for NumPy on the inside, the codebase does not have to double up logic around how some operations work, as happens in Pandas.

The built-in query optimizer is very interesting—a lot of work has gone into the query optimizer both to reorder and simplify steps and to then make them execute efficiently. This includes automated parallelization where possible, without requiring intervention from the developer.

A consequence of these design decisions is that in Ian’s benchmarking for conference talks, Polars is typically faster than Pandas—sometimes two to ten times faster—for an equivalent set of lines of code when the data is held in RAM.

If the data is larger than RAM, then an experimental *streaming* mode can allow a Dask-like processing mode wherein only some of the data is loaded into RAM as needed in a way that’s transparent to the developer. In this case, hand-optimized Dask code and nontuned Polars code can have similar performance. Also remember that “all benchmarks are wrong,” as they always tell someone else’s story on *their* data. You need to run your own benchmarks on your own data to be confident you’re making the right choices for your data.

Wrap-Up

In the next chapter, we will talk about how to create your own external modules that can be fine-tuned to solve specific problems with much greater efficiencies. This allows us to follow the rapid prototyping method of making our programs—first solve the problem with slow code, then identify the elements that are slow, and finally, find ways to make those elements faster. By profiling often and trying to optimize only the sections of code we *know* are slow, we can save ourselves time while still making our programs run as fast as possible.

Compiling to C

Questions You'll Be Able to Answer After This Chapter

- How can I have my Python code run as lower-level code?
- What is the difference between a JIT compiler and an AOT compiler?
- What tasks can compiled Python code perform faster than native Python?
- Why do type annotations speed up compiled Python code?
- How can I write modules for Python using C or Fortran?
- How can I use libraries from C or Fortran in Python?

The easiest way to get your code to run faster is to make it do less work. Assuming you've already chosen good algorithms and you've reduced the amount of data you're processing, the easiest way to execute fewer instructions is to compile your code down to machine code.

Python offers a number of options for this, including pure C-based compiling approaches like Cython; LLVM-based compiling via Numba; and the replacement virtual machine PyPy, which includes a built-in just-in-time (JIT) compiler. You need to balance the requirements of code adaptability and team velocity when deciding which route to take.

Each of these tools adds a new dependency to your toolchain, and Cython requires you to write in a new language type (a hybrid of Python and C), which means you need a new skill. Cython's new language may hurt your team's velocity, as team members without knowledge of C may have trouble supporting this code; in practice, though, this is probably a minor concern, as you'll use Cython only in well-chosen, small regions of your code.

It is worth noting that performing CPU and memory profiling on your code will probably start you thinking about higher-level algorithmic optimizations that you might apply. These algorithmic changes (such as additional logic to avoid computations or caching to avoid recalculation) could help you avoid doing unnecessary work in your code, and Python’s expressivity helps you to spot these algorithmic opportunities. Radim Řehůřek discusses how a Python implementation can beat a pure C implementation in [“Making Deep Learning Fly with RadimRehurek.com \(2014\)” on page 477](#).

In this chapter we’ll review the following:

- Cython, the most commonly used tool for compiling to C, covering both `numpy` and normal Python code (requires some knowledge of C)
- Numba, a compiler specialized for `numpy` code
- PyPy, a stable just-in-time compiler generally for non-`numpy` code that is a replacement for the normal Python executable

Later in the chapter we’ll look at foreign function interfaces, which allow C code to be compiled into extension modules for Python. Python’s native API is used with `ctypes` or with `cffi` (from the authors of PyPy), along with the `f2py` Fortran-to-Python converter.

What Sort of Speed Gains Are Possible?

Gains of an order of magnitude or more are quite possible if your problem yields to a compiled approach. Here, we’ll look at various ways to achieve speedups of one to two orders of magnitude on a single core, along with using multiple cores through OpenMP, which can automatically spread work and synchronize results with little effort by the developer.

Python code that tends to run faster after compiling is mathematical, and it has lots of loops that repeat the same operations many times. Inside these loops, you’re probably making lots of temporary objects.

Code that calls out to external libraries (such as regular expressions, string operations, and calls to database libraries) is unlikely to show any speedup after compiling. Programs that are I/O-bound are also unlikely to show significant speedups.

Similarly, if your Python code focuses on calling vectorized `numpy` routines, it may not run any faster after compilation—it’ll run faster only if the code being compiled is mainly Python (and probably if it is mainly looping). We looked at `numpy` operations in [Chapter 6](#); compiling doesn’t really help because there aren’t many intermediate objects.

Overall, it is very unlikely that your compiled code will run any faster than a hand-crafted C routine, but it is also unlikely to run much slower. It is quite possible that the generated C code from your Python will run as fast as a handwritten C routine, unless the C coder has particularly good knowledge of ways to tune the C code to the target machine's architecture.

For math-focused code, it is possible that a handcoded Fortran routine will beat an equivalent C routine, but again, this probably requires expert-level knowledge. Overall, a compiled result (probably using Cython) will be as close to a handcoded-in-C result as most programmers will need.

Keep the diagram in [Figure 8-1](#) in mind when you profile and work on your algorithm. A small amount of work understanding your code through profiling should enable you to make smarter choices at an algorithmic level. After this, some focused work with a compiler should buy you an additional speedup. It will probably be possible to keep tweaking your algorithm, but don't be surprised to see increasingly small improvements coming from increasingly large amounts of work on your part. Know when additional effort isn't useful.

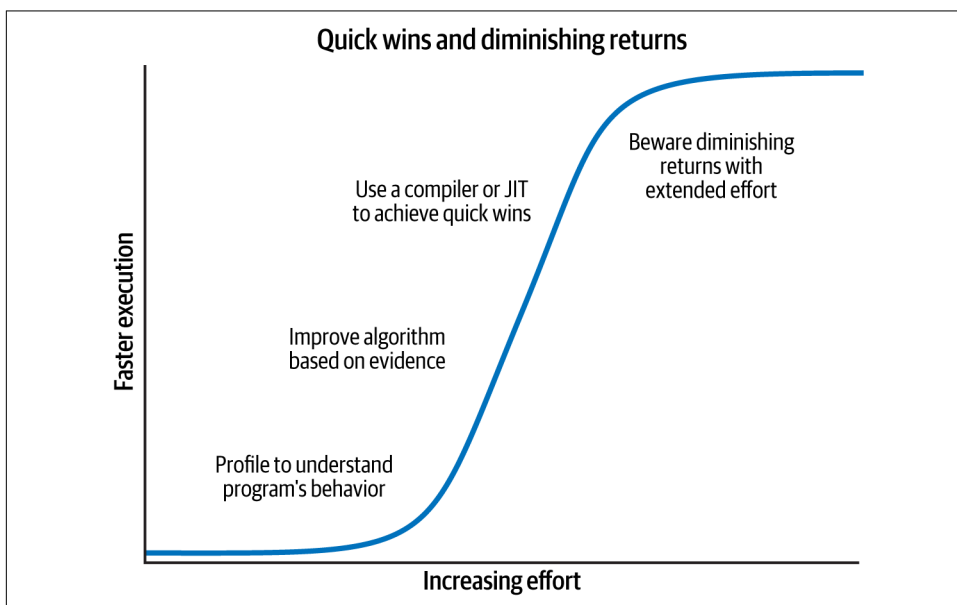


Figure 8-1. Some effort profiling and compiling brings a lot of reward, but continued effort tends to pay increasingly less

If you're dealing with Python code and batteries-included libraries without numpy, Cython and PyPy are your main choices. If you're working with numpy, Cython and Numba are the right choices. Cython and Numba support Python 3.11+; PyPy is written to Python 3.10 at the time of publication.

Some of the following examples require a little understanding of C compilers and C code. If you lack this knowledge, you should learn a little C and compile a working C program before diving in too deeply.

JIT Versus AOT Compilers

The tools we'll look at split roughly into two sets: tools for compiling ahead of time, or AOT (Cython), and tools for compiling “just in time,” or JIT (Numba, PyPy).

By compiling AOT, you create a static library that's specialized to your machine. If you download `numpy`, `scipy`, or `scikit-learn`, it will compile parts of the library using Cython on your machine (or you'll use a prebuilt compiled library, if you're using a distribution like Anaconda). By compiling ahead of use, you'll have a library that can instantly be used to work on solving your problem.

By compiling JIT, you don't have to do much (if any) work up front; you let the compiler step in to compile just the right parts of the code at the time of use. This means you have a “cold start” problem—if most of your program could be compiled and currently none of it is, when you start running your code, it'll run very slowly while it compiles. If this happens every time you run a script and you run the script many times, this cost can become significant. PyPy suffers from this problem, so you may not want to use it for short but frequently running scripts.

The current state of affairs shows us that compiling ahead of time buys us the best speedups, but often this requires the most manual effort. Just-in-time compiling offers some impressive speedups with very little manual intervention, but it can also run into the problem just described. You'll have to consider these trade-offs when choosing the right technology for your problem.

Why Does Type Information Help the Code Run Faster?

Python is dynamically typed—a variable can refer to an object of any type, and any line of code can change the type of the object that is referred to. This makes it difficult for the virtual machine to optimize how the code is executed at the machine code level, as it doesn't know which fundamental datatype will be used for future operations. Keeping the code generic makes it run more slowly.

In the following example, `v` is either a floating-point number or a pair of floating-point numbers that represent a complex number. Both conditions could occur in the same loop at different points in time, or in related serial sections of code:

```
v = -1.0
print(type(v), abs(v))

<class 'float'> 1.0
```



```
v = 1-1j
print(type(v), abs(v))

<class 'complex'> 1.4142135623730951
```

The `abs` function works differently depending on the underlying datatype. Using `abs` for an integer or a floating-point number turns a negative value into a positive value. Using `abs` for a complex number involves taking the square root of the sum of the squared components:

$$\text{abs}(c) = \sqrt{c.\text{real}^2 + c.\text{imag}^2}$$

The machine code for the `complex` example involves more instructions and will take longer to run. Before calling `abs` on a variable, Python first has to look up the type of the variable and then decide which version of a function to call—this overhead adds up when you make a lot of repeated calls.

Inside Python, every fundamental object, such as an integer, will be wrapped up in a higher-level Python object (e.g., an `int` for an integer). The higher-level object has extra functions like `__hash__` to assist with storage and `__str__` for printing.

Inside a section of code that is CPU-bound, it is often the case that the types of variables do not change. This gives us an opportunity for static compilation and faster code execution.

If all we want are a lot of intermediate mathematical operations, we don't need the higher-level functions, and we may not need the machinery for reference counting either. We can drop down to the machine code level and do our calculations quickly using machine code and bytes, rather than manipulating the higher-level Python objects, which involves greater overhead. To do this, we determine the types of our objects ahead of time so we can generate the correct C code.

Using a C Compiler

In the following examples, we'll use `gcc` and `g++` from the GNU C Compiler toolset. You could use an alternative compiler (e.g., Intel's `icc` or Microsoft's `cl`) if you configure your environment correctly. Cython uses `gcc`.

`gcc` is a very good choice for most platforms; it is well supported and quite advanced. It is often possible to squeeze out more performance by using a tuned compiler (e.g., Intel's `icc` may produce faster code than `gcc` on Intel devices), but the cost is that you have to gain more domain knowledge and learn how to tune the flags on the alternative compiler.

C and C++ are often used for static compilation rather than other languages like Fortran because of their ubiquity and the wide range of supporting libraries. The compiler and the converter, such as Cython, can study the annotated code to determine whether static optimization steps (like inlining functions and unrolling loops) can be applied.

Aggressive analysis of the intermediate abstract syntax tree (performed by Numba and PyPy) provides opportunities to combine knowledge of Python's way of expressing things to inform the underlying compiler how best to take advantage of the patterns that have been seen.

Reviewing the Julia Set Example

Back in [Chapter 2](#) we profiled the Julia set generator. This code uses integers and complex numbers to produce an output image. The calculation of the image is CPU-bound.

The main cost in the code was the CPU-bound nature of the inner loop that calculates the output list. This list can be drawn as a square pixel array, where each value represents the cost to generate that pixel.

The code for the inner function is shown in [Example 8-1](#).

Example 8-1. Reviewing the Julia function's CPU-bound code

```
def calculate_z_serial_purepython(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and abs(z) < 2:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

On Ian's laptop, the original Julia set calculation on a 1,000 × 1,000 grid with `maxiter=300` takes approximately 6 seconds using a pure Python implementation running on CPython 3.12.

Cython

Cython is a compiler that converts type-annotated Python into a compiled extension module. The type annotations are C-like. This extension can be imported as a regular Python module using `import`. Getting started is simple, but a learning curve must be climbed with each additional level of complexity and optimization. For Ian, this and Numba are the tools of choice for turning calculation-bound functions into faster code. Cython stands out because of its maturity and its more flexible OpenMP support.

With the OpenMP standard, it is possible to convert parallel problems into multiprocessing-aware modules that run on multiple CPUs on one machine. The threads are hidden from your Python code; they operate via the generated C code.

Cython (announced in 2007) is a fork of Pyrex (announced in 2002) that expands the capabilities beyond the original aims of Pyrex. Libraries that use Cython include SciPy, scikit-learn, lxml, and ZeroMQ.

Cython can be used via a *setup.py* script to compile a module. It can also be used interactively in IPython via a “magic” command and inside Jupyter Notebooks.

Typically, the types are annotated by the developer, although some automated annotation is possible. Normally it takes more work to get fast Cython code compared to using Numba.

With the recent Cython 3.0 release, it is possible to annotate both a pure-Python *.py* file and the more usual *.pyx* file. For pure Python code either can be used; for NumPy compilation it is expected that *.pyx* files are used, so that’s the style we focus on here.

Compiling a Pure Python Version Using Cython

The easy way to begin writing a compiled extension module involves three files. Using our Julia set as an example, they are as follows:

- The calling Python code (the bulk of our Julia code from earlier)
- The function to be compiled in a new *.pyx* file
- A *setup.py* that contains the instructions for calling Cython to make the extension module

Using this approach, the *setup.py* script is called to use Cython to compile the *.pyx* file into a compiled module. On Unix-like systems, the compiled module will probably be a *.so* file; on Windows it should be a *.pyd* (DLL-like Python library).

For the Julia example, we'll use the following:

- *julia1.py* to build the input lists and call the calculation function
- *cythonfn.pyx*, which contains the CPU-bound function that we can annotate
- *setup.py*, which contains the build instructions

The result of running *setup.py* is a module that can be imported. In our *julia1.py* script in [Example 8-2](#), we need only to make some tiny changes to import the new module and call our function.

Example 8-2. Importing the newly compiled module into our main code

```
...
import cythonfn # as defined in setup.py
...
def calc_pure_python(desired_width, max_iterations):
    # ...
    start_time = time.time()
    output = cythonfn.calculate_z(max_iterations, zs, cs)
    end_time = time.time()
    secs = end_time - start_time
    print(f"Took {secs:0.2f} seconds")
...
```

In [Example 8-3](#), we will start with a pure Python version without type annotations.

Example 8-3. Unmodified pure Python code in cythonfn.pyx (renamed from .py) for Cython's setup.py

```
# cythonfn.pyx
def calculate_z(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and abs(z) < 2:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

The *setup.py* script shown in [Example 8-4](#) is short; it defines how to convert *cythonfn.pyx* into *calculate.so*.

Example 8-4. setup.py, which converts cythonfn.pyx into C code for compilation by Cython

```
from setuptools import setup

from Cython.Build import cythonize
setup(ext_modules=cythonize("cythonfn.pyx"))
```

When we run the *setup.py* script in [Example 8-5](#) with the argument *build_ext*, Cython will look for *cythonfn.pyx* and build *cythonfn[...].so*.



Remember that this is a manual step—if you update your *.pyx* or *setup.py* and forget to rerun the build command, you won't have an updated *.so* module to import. If you're unsure whether you compiled the code, check the timestamp for the *.so* file. If in doubt, delete the generated C files and the *.so* file and build them again.

Example 8-5. Running setup.py to build a new compiled module

```
$ python setup.py build_ext --inplace
Compiling cythonfn.pyx because it changed.
[1/1] Cythonizing cythonfn.pyx
running build_ext
building 'cythonfn' extension
gcc -pthread -B ...
```

The *--inplace* argument tells Cython to build the compiled module into the current directory rather than into a separate *build* directory. After the build has completed, we'll have the intermediate *cythonfn.c*, which is rather hard to read, along with *cythonfn[...].so*.

Now when the *julia1.py* code is run, the compiled module is imported, and the Julia set is calculated on Ian's laptop in 4.2 seconds, rather than the more usual 6 seconds. This is a useful improvement for very little effort.

pyximport

A simplified build system has been introduced via *pyximport*. If your code has a simple setup and doesn't require third-party modules, you may be able to do away with *setup.py* completely.

By importing *pyximport* as seen in [Example 8-6](#) and calling *install*, any subsequently imported *.pyx* file will be automatically compiled. This *.pyx* file can include annotations, or in this case, it can be the unannotated code. The result runs in 4.2 seconds, as before; the only difference is that we didn't have to write a *setup.py* file.

Example 8-6. Using `pyximport` to replace `setup.py`

```
import pyximport
pyximport.install()
import cythonfn
# followed by the usual code
...
    output = cythonfn.calculate_z(max_iterations, zs, cs)
...
```

This approach has limitations but can be an easy way to start writing Cython.

Cython Annotations to Analyze a Block of Code

The preceding example shows that we can quickly build a compiled module. For tight loops and mathematical operations, this alone often leads to a speedup. Obviously, though, we should not optimize blindly—we need to know which lines of code take a lot of time so we can decide where to focus our efforts.

Cython has an annotation option that will output an HTML file we can view in a browser. We use the command `cython -a cythonfn.pyx`, and the output file `cythonfn.html` is generated. Viewed in a browser, it looks something like Figure 8-2. A similar image is available in the [Cython documentation](#).

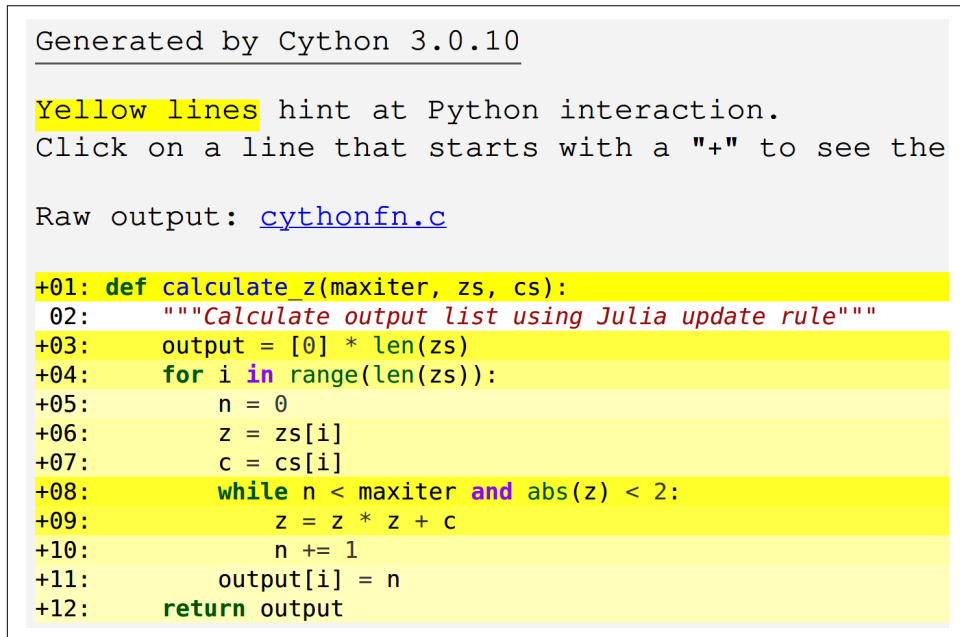


Figure 8-2. Colored Cython output of unannotated function

Each line can be expanded with a double-click to show the generated C code. More shading (or darker yellow for digital viewers) means “more calls into the Python virtual machine,” while more white means “more non-Python C code.” The goal is to remove as many of the shaded lines as possible and end up with as much white as possible.

Although “more shaded lines” means more calls into the virtual machine, this won’t necessarily cause your code to run slower. Each call into the virtual machine has a cost, but the cost of those calls will be significant only if the calls occur inside large loops. Calls outside large loops (for example, the line used to create output at the start of the function) are not expensive relative to the cost of the inner calculation loop. Don’t waste your time on the lines that don’t cause a slowdown.

In our example, the lines with the most calls back into the Python virtual machine (the “most shaded”) are lines 8 and 9. From our previous profiling work, we know that line 8 is likely to be called over 30 million times, so that’s a great candidate to focus on.

Line 9 is inside the tight inner loop and is in part responsible for the bulk of the execution time of this function, so we need to focus on that line as well. Refer back to [“Using line_profiler for Line-by-Line Measurements” on page 48](#) if you need to remind yourself of how much time is spent in this section.

Lines 6 and 7 are less shaded, and since they’re called only 1 million times, they’ll have a much smaller effect on the final speed, so we can focus on them later. In fact, since they are `list` objects, there’s actually nothing we can do to speed up their access except (as you’ll see in [“Cython and numpy” on page 229](#)) to replace the `list` objects with `numpy` arrays, which will buy a small speed advantage.

To better understand the shaded regions, you can expand each line. In [Figure 8-3](#), we can see that to create the output list, we iterate over the length of `zs`, building new Python objects that are reference-counted by the Python virtual machine. Even though these calls are expensive, they won’t really affect the execution time of this function.

To improve the execution time of our function, we need to start declaring the types of objects that are involved in the expensive inner loops. These loops can then make fewer of the relatively expensive calls back into the Python virtual machine, saving us time.

In general, the lines that probably cost the most CPU time are those:

- Inside tight inner loops
- Dereferencing `list`, `array`, or `np.array` items
- Performing mathematical operations

Generated by Cython 3.0.10

Yellow lines hint at Python interaction.

Click on a line that starts with a "+" to see the C code

Raw output: [cythonfn.c](#)

```
+01: def calculate_z(maxiter, zs, cs):
+02:     """Calculate output list using Julia update rule"""
+03:     output = [0] * len(zs)
    __pyx_t_1 = PyObject_Length(__pyx_v_zs); if (unlikely(__pyx_t_1 ==
    __pyx_t_2 = PyList_New(1 * ((__pyx_t_1 < 0) ? 0: __pyx_t_1)); if (unli
    Pyx_GOTREF(__pyx_t_2);
    { Py_ssize_t __pyx_temp;
      for (__pyx_temp=0; __pyx_temp < __pyx_t_1; __pyx_temp++) {
        __Pyx_INCREF(__pyx_int_0);
        __Pyx_GIVEREF(__pyx_int_0);
        if (__Pyx_PyList_SET_ITEM(__pyx_t_2, __pyx_temp, __pyx_int_0))
      }
    }
    __pyx_v_output = ((PyObject*)__pyx_t_2);
    __pyx_t_2 = 0;
+04:     for i in range(len(zs)):
+05:         n = 0
+06:         z = zs[i]
+07:         c = cs[i]
+08:         while n < maxiter and abs(z) < 2:
+09:             z = z * z + c
+10:             n += 1
+11:         output[i] = n
+12:     return output
```

Figure 8-3. C code behind a line of Python code



If you don't know which lines are most frequently executed, using a profiling tool—`line_profiler`, discussed in “Using `line_profiler` for Line-by-Line Measurements” on page 48—would be the most appropriate. You'll learn which lines are executed most frequently and which lines cost the most inside the Python virtual machine, so you'll have clear evidence of which lines you need to focus on to get the best speed gain.

Adding Some Type Annotations

Figure 8-2 shows that almost every line of our function is calling back into the Python virtual machine. All of our numeric work is also calling back into Python, as we are using the higher-level Python objects. We need to convert these into local C objects,

and then, after doing our numerical coding, we need to convert the result back to a Python object.

In **Example 8-7**, we see how to add some primitive types by using the `cdef` syntax.



It is important to note that these types will be understood only by Cython and *not* by Python. Cython uses these types to convert the Python code to C objects, which do not have to call back into the Python stack; this means the operations run at a faster speed, but they lose flexibility and development speed.

The types we add are as follows:

- `int` for a signed integer
- `unsigned int` for an integer that can only be positive
- `double complex` for double-precision complex numbers

The `cdef` keyword lets us declare variables inside the function body. These must be declared at the top of the function, as that's a requirement from the C language specification.

Example 8-7. Adding primitive C types to start making our compiled function run faster by doing more work in C and less via the Python virtual machine

```
def calculate_z(int maxiter, zs, cs):  
    """Calculate output list using Julia update rule"""  
    cdef unsigned int i, n  
    cdef double complex z, c  
    output = [0] * len(zs)  
    for i in range(len(zs)):  
        n = 0  
        z = zs[i]  
        c = cs[i]  
        while n < maxiter and abs(z) < 2:  
            z = z * z + c  
            n += 1  
        output[i] = n  
    return output
```



When adding Cython annotations, you're adding non-Python code to the `.pyx` file. This means you lose the interactive nature of developing Python in the interpreter. For those of you familiar with coding in C, we go back to the code-compile-run-debug cycle.

You might wonder if we could add a type annotation to the lists that we pass in. We can use the `list` keyword, but this has no practical effect for this example. The `list` objects still have to be interrogated at the Python level to pull out their contents, and this is very slow.

The act of giving types to some of the primitive objects is reflected in the annotated output in [Figure 8-4](#). Critically, lines 11 and 12—two of our most frequently called lines—have now turned from shaded to white, indicating that they no longer call back to the Python virtual machine. We can anticipate a great speedup compared to the previous version.

```
Generated by Cython 3.0.10

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see

Raw output: cythonfn.c

+01: def calculate_z(int maxiter, zs, cs):
+02:     """Calculate output list using Julia update rule"""
+03:     cdef unsigned int i, n
+04:     cdef double complex z, c
+05:     output = [0] * len(zs)
+06:     for i in range(len(zs)):
+07:         n = 0
+08:         z = zs[i]
+09:         c = cs[i]
+10:         while n < maxiter and abs(z) < 2:
+11:             z = z * z + c
+12:             n += 1
+13:         output[i] = n
+14:     return output
```

Figure 8-4. Our first type annotations

After compiling, this version takes 0.43 seconds to complete. With only a few changes to the function, we are running at 13 times the speed of the original Python version.

It is important to note that the reason we are gaining speed is that more of the frequently performed operations are being pushed down to the C level—in this case, the updates to `z` and `n`. This means that the C compiler can optimize the way the lower-level functions are operating on the bytes that represent these variables, without calling into the relatively slow Python virtual machine.

As noted earlier in this chapter, `abs` for a complex number involves taking the square root of the sum of the squares of the real and imaginary components. In our test, we want to see if the square root of the result is less than 2. Rather than taking the square root, we can instead square the other side of the comparison, so we turn `< 2` into `< 4`. This avoids having to calculate the square root as the final part of the `abs` function.

In essence, we started with

$$\sqrt{c.real^2 + c.imag^2} < \sqrt{4}$$

and have simplified the operation to

$$c.real^2 + c.imag^2 < 4$$

If we retained the `sqrt` operation in the following code, we would still see an improvement in execution speed. One of the secrets to optimizing code is to make it do as little work as possible. Removing a relatively expensive operation by considering the ultimate aim of a function means that the C compiler can focus on what it is good at, rather than trying to intuit the programmer's ultimate needs.

Writing equivalent but more specialized code to solve the same problem is known as *strength reduction*. You trade worse flexibility (and possibly worse readability) for faster execution.

This mathematical unwinding leads to [Example 8-8](#), in which we have replaced the relatively expensive `abs` function with a simplified line of expanded mathematics.

Example 8-8. Expanding the `abs` function by using Cython

```
def calculate_z(int maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    cdef unsigned int i, n
    cdef double complex z, c
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

By annotating the code, we see that the `while` on line 10 (Figure 8-5) has become a little more shaded—it looks as though it might be doing more work rather than less. It isn't immediately obvious how much of a speed gain we'll get, but we know that this line is called over 30 million times, so we anticipate a good improvement.

This change has a dramatic effect—by reducing the number of Python calls in the innermost loop, we greatly reduce the calculation time of the function. This new version completes in just 0.23 seconds, an amazing 26 times speedup over the original version. As ever, be guided by what you see, but *measure* to test all of your changes!

```
Generated by Cython 3.0.10

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see the C code

Raw output: cythonfn.c

+01: def calculate_z(int maxiter, zs, cs):
02:     """Calculate output list using Julia update rule"""
03:     cdef unsigned int i, n
04:     cdef double complex z, c
+05:     output = [0] * len(zs)
+06:     for i in range(len(zs)):
+07:         n = 0
+08:         z = zs[i]
+09:         c = cs[i]
+10:         while n < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
+11:             z = z * z + c
+12:             n += 1
+13:         output[i] = n
+14:     return output
```

Figure 8-5. Expanded math to get a final win



Cython supports several methods of compiling to C, some easier than the full-type-annotation method described here. You should familiarize yourself with the **pure Python mode** if you'd like an easier start to using Cython, and look at `pyximport` to ease the introduction of Cython to colleagues.

For a final possible improvement on this piece of code, we can disable bounds checking for each dereference in the list. The goal of the bounds checking is to ensure that the program does not access data outside the allocated array—in C it is easy to accidentally access memory outside the bounds of an array, and this will give unexpected results (and probably a segmentation fault!).

By default, Cython protects the developer from accidentally addressing outside the list's limits. This protection costs a little bit of CPU time, but it occurs in the outer

loop of our function, so in total it won't account for much time. Disabling bounds checking is usually safe unless you are performing your own calculations for array addressing, in which case you will have to be careful to stay within the bounds of the list.

Cython has a set of flags that can be expressed in various ways. The easiest is to add them as single-line comments at the start of the `.pyx` file. It is also possible to use a decorator or compile-time flag to change these settings. To disable bounds checking, we add a directive for Cython inside a comment at the start of the `.pyx` file:

```
#cython: boundscheck=False
def calculate_z(int maxiter, zs, cs):
```

As noted, disabling the bounds checking will save only a little bit of time, as it occurs in the outer loop, not in the inner loop, which is more expensive. For this example, it doesn't save us any more time.



Try disabling bounds checking and wraparound checking if your CPU-bound code is in a loop that is dereferencing items frequently.

Cython and numpy

`list` objects (for background, see [Chapter 3](#)) have an overhead for each dereference, as the objects they reference can occur anywhere in memory. In contrast, `array` objects store primitive types in contiguous blocks of RAM, which enables faster addressing.

Python has the `array` module, which offers 1D storage for basic primitives (including integers, floating-point numbers, characters, and Unicode strings). NumPy's `numpy.array` module allows multidimensional storage and a wider range of primitive types, including complex numbers.

When iterating over an `array` object in a predictable fashion, the compiler can be instructed to avoid asking Python to calculate the appropriate address and instead to move to the next primitive item in the sequence by going directly to its memory address. Since the data is laid out in a contiguous block, it is trivial to calculate the address of the next item in C by using an offset, rather than asking CPython to calculate the same result, which would involve a slow call back into the virtual machine.

You should note that if you run the following `numpy` version *without* any Cython annotations (that is, if you just run it as a plain Python script), it'll take about 8 seconds to run—slower than the plain Python `list` version, which takes around

6 seconds. The slowdown is because of the overhead of dereferencing individual elements in the numpy lists—it was never designed to be used this way, even though to a beginner this might feel like the intuitive way of handling operations. By compiling the code, we remove this overhead.

Cython has two special syntax forms for this. Older versions of Cython had a special access type for numpy arrays, but more recently the generalized buffer interface protocol has been introduced through the `memoryview`—this allows the same low-level access to any object that implements the buffer interface, including numpy arrays and Python arrays.

An added bonus of the buffer interface is that blocks of memory can easily be shared with other C libraries, without any need to convert them from Python objects into another form.

The code block in [Example 8-9](#) looks a little like the original implementation, except that we have added `memoryview` annotations. The function's second argument is `double complex[:] zs`, which means we have a double-precision complex object using the buffer protocol as specified using `[]`, which contains a one-dimensional data block specified by the single colon `:`. The `cimport` brings in the C-equivalent code for the numpy library.

Example 8-9. Annotated numpy version of the Julia calculation function

```
# cythonfn.pyx
import numpy as np
cimport numpy as np

def calculate_z(int maxiter, double complex[:] zs, double complex[:] cs):
    """Calculate output list using Julia update rule"""
    cdef unsigned int i, n
    cdef double complex z, c
    cdef int[:] output = np.empty(len(zs), dtype=np.int32)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

In addition to specifying the input arguments by using the buffer annotation syntax, we also annotate the output variable, assigning a 1D numpy array to it via `empty`. The call to `empty` will allocate a block of memory but will not initialize the memory with sane values, so it could contain anything. We will overwrite the contents of this array

in the inner loop so we don't need to reassign it with a default value. This is slightly faster than allocating and setting the contents of the array with a default value.

We also expanded the call to `abs` by using the faster, more explicit math version. This version runs in 0.20 seconds—a slightly faster result than the original Cythonized version of the pure Python Julia example in [Example 8-8](#). In [Figure 8-6](#) we see the `cython -a` output for our NumPy variant.

```
Generated by Cython 3.0.10

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see the C code th

Raw output: cythonfn.c

+01: import numpy as np
02: cimport numpy as np
03:
+04: def calculate_z(int maxiter, double complex[:] zs, double complex[:] cs):
05:     """Calculate output list using Julia update rule"""
06:     cdef unsigned int i, n
07:     cdef double complex z, c
+08:     cdef int[:] output = np.empty(len(zs), dtype=np.int32)
+09:     for i in range(len(zs)):
+10:         n = 0
+11:         z = zs[i]
+12:         c = cs[i]
+13:         while n < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
+14:             z = z * z + c
+15:             n += 1
+16:         output[i] = n
+17:     return output
```

Figure 8-6. Expanded math to get a final win

The pure Python version has an overhead every time it dereferences a Python complex object, but these dereferences occur in the outer loop and so don't account for much of the execution time. After the outer loop, we make native versions of these variables, and they operate at “C speed.” The inner loops for both this numpy example and the former pure Python example are doing the same work on the same data, so the time difference is accounted for by the outer loop dereferences and the creation of the output arrays.

For reference, if we use the preceding code but don't expand the `abs` math, then the Cythonized result takes 0.41 seconds. This result makes it identical to the earlier equivalent pure Python version's runtime.

Parallelizing the Solution with OpenMP on One Machine

As a final step in the evolution of this version of the code, let's look at the use of the OpenMP C++ extensions to parallelize our embarrassingly parallel problem. If your problem fits this pattern, you can quickly take advantage of multiple cores in your computer.

Open Multi-Processing (OpenMP) is a well-defined cross-platform API that supports parallel execution and memory sharing for C, C++, and Fortran. It is built into most modern C compilers, and if the C code is written appropriately, the parallelization occurs at the compiler level, so it comes with relatively little effort to the developer through Cython.

With Cython, OpenMP can be added by using the `prange` (parallel range) operator and adding the `-fopenmp` compiler directive to `setup.py`. Work in a `prange` loop can be performed in parallel because we disable the GIL. The GIL protects access to Python objects, preventing multiple threads or processes from accessing the same memory simultaneously, which might lead to corruption. By manually disabling the GIL, we're asserting that we won't corrupt our own memory. Be careful when you do this, and keep your code as simple as possible to avoid subtle bugs.

Example 8-10 shows a modified version of the code from Example 8-9 where we add `prange` parallel support. The `with nogil:` statement specifies the block where the GIL is disabled. Inside this block, we use `prange` to enable an OpenMP parallel for loop to independently calculate each `i` across multiple CPUs.

Example 8-10. Adding `prange` to enable parallelization using OpenMP

```
# cythonfn.pyx
from cython.parallel import prange
import numpy as np
cimport numpy as np

def calculate_z(int maxiter, double complex[:] zs, double complex[:] cs):
    """Calculate output list using Julia update rule"""
    cdef unsigned int i, length
    cdef double complex z, c
    cdef int[:] output = np.empty(len(zs), dtype=np.int32)
    length = len(zs)
    with nogil:
        for i in prange(length, schedule="guided"):
            z = zs[i]
            c = cs[i]
            output[i] = 0
            while output[i] < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
                z = z * z + c
                output[i] += 1
    return output
```




When disabling the GIL, you must *not* operate on regular Python objects (such as lists); you must operate only on primitive objects and objects that support the `memoryview` interface. If you operated on normal Python objects in parallel, you'd have to solve the associated memory-management problems that the GIL deliberately avoids. Cython doesn't prevent us from manipulating Python objects, and only pain and confusion can result if you do this!

To compile `cythonfn.pyx`, we have to modify the `setup.py` script as shown in [Example 8-11](#). We tell it to inform the C compiler to use `-fopenmp` as an argument during compilation to enable OpenMP and to link with the OpenMP libraries.

Example 8-11. Adding the OpenMP compiler and linker flags to `setup.py` for Cython

```
#setup.py
from setuptools import setup
from distutils.extension import Extension
import numpy as np

ext_modules = [
    Extension(
        "cythonfn",
        ["cythonfn.pyx"],
        extra_compile_args=["-fopenmp"],
        extra_link_args=["-fopenmp"],
    )
]

from Cython.Build import cythonize

setup(ext_modules=cythonize(ext_modules), include_dirs=[np.get_include()])
```

With Cython's `prange`, we can choose different scheduling approaches. With `static`, the workload is distributed evenly across the available CPUs. Some of our calculation regions are expensive in time, and some are cheap. If we ask Cython to schedule the work chunks equally using `static` across the CPUs, the results for some regions will complete faster than others, and those threads will then sit idle.

Both the `dynamic` and `guided` schedule options attempt to mitigate this problem by allocating work in smaller chunks dynamically at runtime, so that the CPUs are more evenly distributed when the workload's calculation time is variable. The correct choice will vary depending on the nature of your workload.

By introducing OpenMP and using `schedule="guided"`, we drop our execution time to approximately 0.03 seconds—the `guided` schedule will dynamically assign work, so fewer threads will wait for new work.

We also could have disabled the bounds checking for this example by using `#cython: boundscheck=False`, but it wouldn't improve our runtime.

Numba

Numba from Anaconda Inc. is a just-in-time compiler that specializes in numpy code, which it compiles via the LLVM compiler (*not* via g++ or gcc, as used by our earlier examples) at runtime. It doesn't require a precompilation pass, so when you run it against new code, it compiles each annotated function for your hardware. The beauty is that you provide a decorator telling it which functions to focus on and then you let Numba take over. It aims to run on all standard numpy code.

Numba has been rapidly evolving since the first edition of this book. It is now fairly stable, so if you use numpy arrays and have nonvectorized code that iterates over many items, Numba should give you a quick and very painless win. Numba does not bind to external C libraries (which Cython can do), but it can automatically generate code for GPUs (which Cython cannot).

One drawback when using Numba is the toolchain—it uses LLVM, and this has many dependencies. We recommend that you use Anaconda Inc. distribution, as everything is provided; otherwise, getting Numba installed in a fresh environment can be a very time-consuming task.

Example 8-12 shows the addition of the `@jit` decorator to our core Julia function. This is all that's required; the fact that numba has been imported means that the LLVM machinery kicks in at execution time to compile this function behind the scenes.

Example 8-12. Applying the @jit decorator to a function

```
from numba import jit
...
@jit
def calculate_z_serial_purepython(maxiter, zs, cs, output):
```

If the `@jit` decorator is removed, this is just the numpy version of the Julia demo running with Python 3.12, and it takes 8 seconds. Adding the `@jit` decorator drops the execution time to 0.89 seconds. This is close to the result we achieved with Cython, but without all of the annotation effort.

If we run the same function a second time in the same Python session, it runs even faster at 0.39 seconds—there's no need to compile the target function on the second pass if the argument types are the same, so the overall execution speed is faster. On the second run, the Numba result is equivalent to the Cython with numpy result we obtained before (so it came out as fast as Cython for very little work!). PyPy has the same warm-up requirement.

If we use the expanded-math variant we drop down to 0.19 seconds for the second and subsequent runs, equal to the equivalent Cython code for hardly any developer effort.

If you'd like to read another view on what Numba offers, see “Numba (2020)” on page 468, where core developer Valentin Haenel talks about the `@jit` decorator, viewing the original Python source, and going further with parallel options and the typed `List` and typed `Dict` for pure Python compiled interoperability.

Just as with Cython, we can add OpenMP parallelization support with `prange`. **Example 8-13** expands the decorator to require `parallel`. Adding `parallel` enables support for `prange`. This version drops the general runtime from 0.39 seconds to 0.05 seconds. Currently Numba lacks support for OpenMP scheduling options (and with Cython, the guided scheduler runs slightly faster for this problem), but we expect support will be added in a future version.

Example 8-13. Using `prange` to add parallelization

```
@jit(parallel=True)
def calculate_z(maxiter, zs, cs, output):
    """Calculate output list using Julia update rule"""
    for i in prange(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and (z.real*z.real + z.imag*z.imag) < 4:
            z = z * z + c
            n += 1
        output[i] = n
```

When debugging with Numba, it is useful to note that you can ask Numba to show both the intermediate representation and the types for the function call. In **Example 8-14**, we can see that `calculate_z` takes an `int64` and three array types.

Example 8-14. Debugging inferred types

```
print(calculate_z.inspect_types())
# calculate_z (int64,
#             Array(complex128, 1, 'C', False, aligned=True),
#             Array(complex128, 1, 'C', False, aligned=True),
#             Array(int32, 1, 'C', False, aligned=True))
```

Example 8-15 shows the continued output from the call to `inspect_types()`, where each line of compiled code is shown augmented by type information. This gives you a view on the low-level code decisions that have been made.

Example 8-15. Viewing the intermediate representation from Numba

```
...
def calculate_z(maxiter, zs, cs, output):

    # --- LINE 14 ---

    """Calculate output list using Julia update rule"""

    # --- LINE 15 ---
    # $load_global.0 = global(range: <class 'range'>)
    #     :: Function(<class 'range'>)
    # $14load_global.2 = global(len: <built-in function len>)
    #     :: Function(<built-in function len>)
    # $26call.5 = call $14load_global.2(zs, func=$14load_global.2,
    #     args=[Var(zs, julia1_numba_expandedmath_inspection.py:12)],
    #     kws=(), vararg=None, varkwarg=None, target=None)
    #     :: (Array{complex128, 1, 'C', False, aligned=True},) -> int64
...
```

Numba is a powerful JIT compiler that is now maturing. While you may have to introspect the generated code to figure out how to make your code compile, generally shorter pure-NumPy functions work fine, and some of SciPy is supported too. Your best approach will be to break your current code into small (<10 line) and discrete functions and to tackle these one at a time. Do not try to throw a large function into Numba; you can debug the process far more quickly if you have only small, discrete chunks to review individually.

PyPy

PyPy is an alternative implementation of the Python language that includes a tracing just-in-time compiler; it is compatible with Python 3.5+. Typically, it lags behind the most recent version of Python; at the time of writing this third edition Python 3.12 is standard, and PyPy supports up to Python 3.10.

PyPy is a drop-in replacement for CPython and offers all the built-in modules. The project comprises the RPython translation toolchain, which is used to build PyPy (and could be used to build other interpreters). The JIT compiler in PyPy is very effective, and good speedups can be seen with little or no work on your part.

PyPy runs our pure Python Julia demo without any modifications. With CPython it takes 5.8 seconds, and with PyPy it takes 0.9 seconds. This means that PyPy achieves a result that's very close to the Cython example in **Example 8-8**, without *any effort*

at all—that’s pretty impressive! As we observed in our discussion of Numba, if the calculations are run again *in the same session*, then the second and subsequent runs are faster than the first one, as they are already compiled.

By expanding the math and removing the call to `abs`, the PyPy runtime drops to 0.2 seconds. This is equivalent to the Cython versions using pure Python and `numpy` without any work! Note that this result is true only if you’re not using `numpy` with PyPy.

The fact that PyPy supports all the built-in modules is interesting—this means that `multiprocessing` works as it does in CPython. If you have a problem that runs with the batteries-included modules and can run in parallel with `multiprocessing`, you can expect that all the speed gains you might hope to get will be available.

PyPy’s speed has evolved over time. The older chart in [Figure 8-7](#) (from speed.pypy.org) will give you an idea about PyPy’s maturity. These speed tests reflect a wide range of use cases, not just mathematical operations. It is clear that PyPy offers a faster experience than CPython.

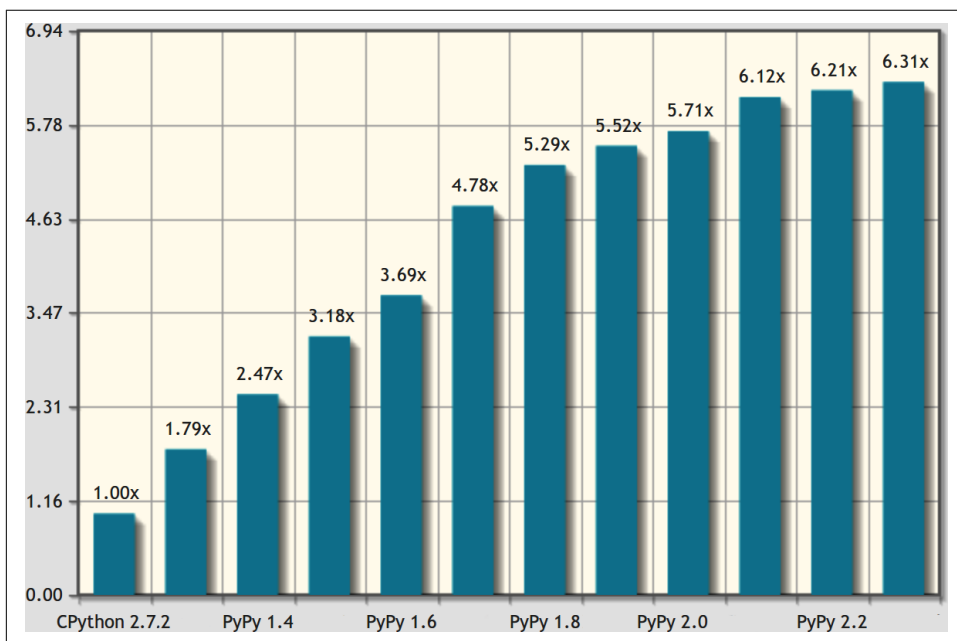


Figure 8-7. Each new version of PyPy offers speed improvements

Garbage Collection Differences

PyPy uses a different type of garbage collector than CPython, and this can cause some nonobvious behavior changes to your code. Whereas CPython uses reference counting, PyPy uses a modified mark-and-sweep approach that may clean up an unused object much later. Both are correct implementations of the Python specification; you just have to be aware that code modifications might be required when swapping.

Some coding approaches seen in CPython depend on the behavior of the reference counter—particularly the flushing of files, if you open and write to them without an explicit file close. With PyPy the same code will run, but the updates to the file might get flushed to disk later, when the garbage collector next runs. An alternative form that works in both PyPy and Python is to use a context manager using `with` to open and automatically close files. The “[Differences Between PyPy and CPython](#)” page on the PyPy website lists the details.

Running PyPy and Installing Modules

If you’ve never run an alternative Python interpreter, you might benefit from a short example. Assuming you’ve downloaded and extracted PyPy, you’ll now have a folder structure containing a *bin* directory. Run it as shown in [Example 8-16](#) to start PyPy.

Example 8-16. Running PyPy to see that it implements Python 3.10

```
...
$ pypy3
Python 3.10.14 (75b3de9d9035, Apr 21 2024, 10:54:48)
[PyPy 7.3.16 with GCC 10.2.1 20210130 (Red Hat 10.2.1-11)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
...
```

Note that PyPy 7.3 runs as Python 3.10. Now we need to set up `pip`, and we’ll want to install IPython. The steps shown in [Example 8-17](#) are the same as you might have performed with CPython if you’ve installed `pip` without the help of an existing distribution or package manager. Note that when running IPython, we get the same build number as we see when running `pypy` in the preceding example.

You can see that IPython runs with PyPy just the same as with CPython, and using the `%run` syntax, we execute the Julia script inside IPython to achieve 0.2-second runtimes.

Example 8-17. Installing pip for PyPy to install third-party modules like IPython

```
...
$ pypy3 -m ensurepip
Looking in links: /tmp/tmpz5okh3uw
Processing /tmp/tmpz5okh3uw/setuptools-65.5.0-py3-none-any.whl
Processing /tmp/tmpz5okh3uw/pip-23.0.1-py3-none-any.whl
Installing collected packages: setuptools, pip
Successfully installed pip-23.0.1 setuptools-65.5.0

$ pip3 install ipython
Collecting ipython
...

$ ipython
Python 3.10.14 (75b3de9d9035, Apr 21 2024, 10:54:48)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.26.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %run julia1_nopil_expanded_math.py
Length of x: 1,000
Total elements: 1,000,000
calculate_z_serial_purepython took 0.26 seconds
...
```

Note that PyPy supported projects like `numpy` that required C bindings through the CPython extension compatibility layer `cpyext` (now replaced by `HPy`), but it had an overhead of four to six times, which generally made `numpy` too slow. If your code is mostly pure Python with only a few calls to `numpy`, you may still see significant overall gains. If your code, like the Julia example, makes many calls to `numpy`, then it'll run significantly slower. The Julia benchmark here with `numpy` arrays runs six times slower than when it is run with CPython.

`HPy` (formerly `PyHandle`) aims to remove the overhead of calling into the C interfaces by providing a higher-level object handle—one not tied to CPython's implementation—which can be shared with other projects like `Cython`. Essentially it is PyPy's `python.h` header file. Typically this overhead costs four to six times, the same overhead as calling from CPython, and this often negates the advantages of using an external tool like `numpy`.

If you want to understand PyPy's performance characteristics, look at the `vmprof` lightweight sampling profiler. It is thread-safe and supports a web-based user interface.

Another downside of PyPy is that it can use a lot of RAM. Each release is better in this respect, but in practice it may use more RAM than CPython. RAM is fairly cheap, though, so it makes sense to try to trade it for enhanced performance. Some users have also reported *lower* RAM usage when using PyPy. As ever, perform an experiment using representative data if this is important to you.

A Summary of Speed Improvements

To summarize the previous results, in [Table 8-1](#) we see that PyPy on a pure Python math-based code sample is approximately six times faster than CPython with no code changes, and it's even faster if the `abs` line is simplified. Cython runs faster than PyPy in both instances but requires annotated code, which increases development and support effort.

Table 8-1. Julia (no numpy) results

	Speed
CPython	5.80s
Cython	0.43s
Cython on expanded math	0.23s
PyPy	0.90s
PyPy on expanded math	0.20s

The Julia solver with `numpy` enables the investigation of OpenMP. In [Table 8-2](#), we see that both Cython and Numba run faster than the non-`numpy` versions with expanded math. When we add OpenMP, both Cython and Numba provide further speedups for very little additional coding.

Table 8-2. Julia (with numpy and expanded math) results

	Speed
CPython	8.00s
Cython	0.20s
Cython and OpenMP “guided”	0.03s
Numba (2nd & subsequent runs)	0.19s
Numba and OpenMP	0.05s

For pure Python code, PyPy is an obvious first choice. For `numpy` code, Numba is a great first choice.

When to Use Each Technology

If you're working on a numeric project, then each of these technologies could be useful to you. [Table 8-3](#) summarizes the main options.

Table 8-3. Compiler options

	Cython	Numba	PyPy
Mature	Y	Y	Y
Widespread	Y	—	—
numpy support	Y	Y	Y
Nonbreaking code changes	—	Y	Y
Needs C knowledge	Y	—	—
Supports OpenMP	Y	Y	—

Numba may offer quick wins for little effort, but it too has limitations that might stop it working well on your code. It is also a relatively young project.

Cython probably offers the best results for the widest set of problems, but it does require more effort and has an additional “support tax” due to mixing Python with C annotations.

PyPy is a strong option if you're not using `numpy` or other hard-to-port C extensions.

If you're working with light numeric requirements, note that Cython's buffer interface accepts `array.array` matrices—this is an easy way to pass a block of data to Cython for fast numeric processing without having to add `numpy` as a project dependency.

Overall, Numba is maturing and is a promising project, whereas Cython is mature. PyPy is regarded as being fairly mature now and should definitely be evaluated for long-running processes.

In a class run by Ian, a capable student implemented a C version of the Julia algorithm and was disappointed to see it execute more slowly than his Cython version. It transpired that he was using 32-bit floats on a 64-bit machine—these run more slowly than 64-bit doubles on a 64-bit machine. The student, despite being a good C programmer, didn't know that this could involve a speed cost. He changed his code, and the C version, despite being significantly shorter than the autogenerated Cython version, ran at roughly the same speed. The act of writing the raw C version, comparing its speed, and figuring out how to fix it took longer than using Cython in the first place.

This is just an anecdote; we're not suggesting that Cython will generate the best code, and competent C programmers can probably figure out how to make *their* code run faster than the version generated by Cython. It is worth noting, though,

that the assumption that handwritten C will be faster than converted Python is not a safe assumption. You must always benchmark and make decisions using evidence. C compilers are pretty good at converting code into fairly efficient machine code, and Python is pretty good at letting you express your problem in an easy-to-understand language—combine these two powers sensibly.

Python boasts a strong set of compiler options. The main options are:

- **Pythran** is an AOT compiler aimed at scientists who are using `numpy`. Using few annotations, it will compile Python numeric code to a faster binary—it produces speedups that are very similar to Cython but for much less work. Among other features, it always releases the GIL and can use both SIMD instructions and OpenMP. Like Numba, it doesn't support classes. If you have tight, locally bound loops in NumPy, Pythran is certainly worth evaluating. The associated FluidPython project aims to make Pythran even easier to write and provides JIT capability.
- **Transonic** attempts to unify Cython, Pythran, and Numba, and potentially other compilers, behind one interface to enable quick evaluation of multiple compilers without having to rewrite code.
- **Shed Skin** is an AOT compiler aimed at nonscientific, pure Python code. It has no `numpy` support, but if your code is pure Python, Shed Skin produces speedups similar to those seen by PyPy (without using `numpy`).
- **PyCUDA** and **PyOpenCL** offer CUDA and OpenCL bindings into Python for direct access to GPUs.
- **Nuitka** is a Python compiler that aims to be an alternative to the usual CPython interpreter, with the option of creating compiled executables.

Our community is rather blessed with a wide array of compilation options. While they all have trade-offs, they also offer a lot of power so that complex projects can take advantage of the full power of CPUs and multicore architectures.

Foreign Function Interfaces

Sometimes the automatic solutions just don't cut it, and you need to write custom C or Fortran code yourself. This could be because the compilation methods don't find some potential optimizations, or because you want to take advantage of libraries or language features that aren't available in Python. In all of these cases, you'll need to use foreign function interfaces, which give you access to code written and compiled in another language.

For the rest of this chapter, we will attempt to use an external library to solve the 2D diffusion equation in the same way we did in [Chapter 6](#).¹ The code for this library, shown in [Example 8-18](#), could be representative of a library you’ve installed or code that you have written. The methods we’ll look at serve as great ways to take small parts of your code and move them to another language in order to do very targeted language-based optimizations.

Example 8-18. Sample C code for solving the 2D diffusion problem

```
void evolve(double in[][512], double out[][512], double D, double dt) {
    int i, j;
    double laplacian;
    for (i=1; i<511; i++) {
        for (j=1; j<511; j++) {
            laplacian = in[i+1][j] + in[i-1][j] + in[i][j+1] + in[i][j-1] \
                - 4 * in[i][j];
            out[i][j] = in[i][j] + D * dt * laplacian;
        }
    }
}
```



We fix the grid size to be 512×512 in order to simplify the example code. To accept an arbitrarily sized grid, you must pass in the `in` and `out` parameters as double pointers and include function arguments for the actual size of the grid.

To use this code, we must compile it into a shared module that creates a `.so` file. We can do this using `gcc` (or any other C compiler) by following these steps:

```
$ gcc -O3 -std=gnu11 -c diffusion.c
$ gcc -shared -o diffusion.so diffusion.o
```

We can place this final shared library file, `diffusion.so`, anywhere that is accessible to our Python code, but standard *nix organization stores shared libraries in `/usr/lib` and `/usr/local/lib`.

ctypes

The most basic foreign function interface in CPython is through the `ctypes` module.² The bare-bones nature of this module can be quite inhibitive at times—you are in charge of doing everything, and it can take quite a while to make sure that you have

¹ For simplicity, we will not implement the boundary conditions.

² This is CPython-dependent. Other versions of Python may have their own versions of `ctypes`, which may work differently.

everything in order. This extra level of complexity is evident in our ctypes diffusion code, shown in [Example 8-19](#).

Example 8-19. ctypes 2D diffusion code

```
import ctypes

grid_shape = (512, 512)
_diffusion = ctypes.CDLL("diffusion.so") ❶

# Create references to the C types that we will need to simplify future code
TYPE_INT = ctypes.c_int
TYPE_DOUBLE = ctypes.c_double
TYPE_DOUBLE_SS = ctypes.POINTER(ctypes.POINTER(ctypes.c_double))

# Initialize the signature of the evolve function to:
# void evolve(int, int, double**, double**, double, double)
_diffusion.evolve.argtypes = [TYPE_DOUBLE_SS, TYPE_DOUBLE_SS, TYPE_DOUBLE,
                              TYPE_DOUBLE]
_diffusion.evolve.restype = None

def evolve(grid, out, dt, D=1.0):
    # First we convert the Python types into the relevant C types
    assert grid.shape == (512, 512)
    cdt = TYPE_DOUBLE(dt)
    cD = TYPE_DOUBLE(D)
    pointer_grid = grid.ctypes.data_as(TYPE_DOUBLE_SS) ❷
    pointer_out = out.ctypes.data_as(TYPE_DOUBLE_SS)

    # Now we can call the function
    _diffusion.evolve(pointer_grid, pointer_out, cD, cdt) ❸
```

- ❶ This is similar to importing the `diffusion.so` library. Either this file is in one of the standard system paths for library files or we can enter an absolute path.
- ❷ `grid` and `out` are both numpy arrays.
- ❸ We finally have all the setup necessary and can call the C function directly.

The first thing we do is “import” our shared library. This is done with the `ctypes.CDLL` call. In this line, we can specify any shared library that Python can access (e.g., the `ctypes-opencv` module loads the `libcv.so` library). From this, we get a `_diffusion` object that contains all the members that the shared library contains. In this example, `diffusion.so` contains only one function, `evolve`, which is now made available to us as a property of the `_diffusion` object. If `diffusion.so` had many functions and properties, we could access them all through the `_diffusion` object.

However, even though the `_diffusion` object has the `evolve` function available within it, Python doesn't know how to use it. C is statically typed, and the function has a very specific signature. To properly work with the `evolve` function, we must explicitly set the input argument types and the return type. This can become quite tedious when developing libraries in tandem with the Python interface, or when dealing with a quickly changing library. Furthermore, since `ctypes` can't check if you have given it the correct types, your code may silently fail or segfault if you make a mistake!

In addition to setting the arguments and return type of the function object, we also need to convert any data we care to use with it (this is called *casting*). Every argument we send to the function must be carefully casted into a native C type. Sometimes this can get quite tricky, since Python is very relaxed about its variable types. For example, if we had `num1 = 1e5`, we would have to know that this is a Python `float`, and thus we should use a `ctype.c_float`. On the other hand, for `num2 = 1e300`, we would have to use `ctype.c_double`, because it would overflow a standard C `float`.

That being said, `numpy` provides a `.ctypes` property to its arrays that makes it easily compatible with `ctypes`. If `numpy` didn't provide this functionality, we would have had to initialize a `ctypes` array of the correct type and then find the location of our original data and have our new `ctypes` object point there.



Unless the object you are turning into a `ctype` object implements a buffer (as do the `array` module, `numpy` arrays, `io.StringIO`, etc.), your data will be copied into the new object. In the case of casting an `int` to a `float`, this doesn't mean much for the performance of your code. However, if you are casting a very long Python list, this can incur quite a penalty! In these cases, using the `array` module or a `numpy` array, or even building up your own buffered object using the `struct` module, would help. This does, however, hurt the readability of your code, since these objects are generally less flexible than their native Python counterparts.

This memory management can get even more complicated if you have to send the library a complicated data structure. For example, if your library expects a C `struct` representing a point in space with the properties `x` and `y`, you would have to define the following:

```
from ctypes import Structure

class cPoint(Structure):
    _fields_ = ("x", c_int), ("y", c_int)
```

At this point you could start creating C-compatible objects by initializing a `cPoint` object (i.e., `point = cPoint(10, 5)`). This isn't a terrible amount of work, but it can become tedious and results in some fragile code. What happens if a new version of the library is released that slightly changes the structure? This will make your code very hard to maintain and generally results in stagnant code, where the developers simply decide never to upgrade the underlying libraries that are being used.

For these reasons, using the `ctypes` module is great if you already have a good understanding of C and want to be able to tune every aspect of the interface. It has great portability since it is part of the standard library, and if your task is simple, it provides simple solutions. Just be careful, because the complexity of `ctypes` solutions (and of similar low-level solutions) quickly becomes unmanageable.

cffi

Realizing that `ctypes` can be quite cumbersome to use at times, `cffi` attempts to simplify many of the standard operations that programmers use. It does this by having an internal C parser that can understand function and structure definitions.

As a result, we can simply write the C code that defines the structure of the library we wish to use, and then `cffi` will do all the heavy work for us: it imports the module and makes sure we specify the correct types to the resulting functions. In fact, this work can be almost trivial if the source for the library is available, since the header files (the files ending in `.h`) will include all the relevant definitions we need.³

Example 8-20 shows the `cffi` version of the 2D diffusion code.

Example 8-20. cffi 2D diffusion code

```
from cffi import FFI, verifier

grid_shape = (512, 512)

ffi = FFI()
ffi.cdef(
    "void evolve(double **in, double **out, double D, double dt);" ❶
)
lib = ffi.dlopen("../diffusion.so")

def evolve(grid, dt, out, D=1.0):
    pointer_grid = ffi.cast("double**", grid.ctypes.data) ❷
    pointer_out = ffi.cast("double**", out.ctypes.data)
    lib.evolve(pointer_grid, pointer_out, D, dt)
```

³ In Unix systems, header files for system libraries can be found in `/usr/include`.

- ❶ The contents of this definition can normally be acquired from the manual of the library that you are using or by looking at the library's header files.
- ❷ While we still need to cast nonnative Python objects for use with our C module, the syntax is very familiar to those with experience in C.

In the preceding code, we can think of the `cffi` initialization as being two-stepped. First, we create an FFI object and give it all the global C declarations we need. This can include datatypes in addition to function signatures. These signatures don't necessarily contain any code; they simply need to define what the code will look like. Then we can import a shared library containing the actual implementation of the functions by using `dlopen`. This means we could have told FFI about the function signature for the `evolve` function and then loaded up two different implementations and stored them in different objects (which is fantastic for debugging and profiling!).

In addition to easily importing a shared C library, `cffi` allows you to write C code and have it be dynamically compiled using the `verify` function. This has many immediate benefits—for example, you can easily rewrite small portions of your code to be in C without invoking the large machinery of a separate C library. Alternatively, if there is a library you wish to use, but some glue code in C is required to have the interface work perfectly, you can inline it into your `cffi` code, as shown in [Example 8-21](#), to have everything be in a centralized location. In addition, since the code is being dynamically compiled, you can specify compile instructions to every chunk of code you need to compile. Note, however, that this compilation has a one-time penalty every time the `verify` function is run to actually perform the compilation.

Example 8-21. `cffi` with inline 2D diffusion code

```
ffi = FFI()
ffi.cdef(
    "void evolve(double **in, double **out, double D, double dt);"
)
lib = ffi.verify(
    r"""
void evolve(double in[][512], double out[][512], double D, double dt) {
    int i, j;
    double laplacian;
    for (i=1; i<511; i++) {
        for (j=1; j<511; j++) {
            laplacian = in[i+1][j] + in[i-1][j] + in[i][j+1] + in[i][j-1] \
                - 4 * in[i][j];
            out[i][j] = in[i][j] + D * dt * laplacian;
        }
    }
}
```

```

}
"""
    extra_compile_args=["-O3"], ❶
)

```

- ❶ Since we are just-in-time compiling this code, we can also provide relevant compilation flags.

Another benefit of the `verify` functionality is that it plays nicely with complicated `cdef` statements. For example, if we were using a library with a complicated structure but wanted to use only a part of it, we could use the partial struct definition. To do this, we add a `...` in the struct definition in `ffi.cdef` and `#include` the relevant header file in a later `verify`.

For example, suppose we were working with a library with header `complicated.h` that included a structure that looked like this:

```

struct Point {
    double x;
    double y;
    bool isActive;
    char *id;
    int num_times_visited;
}

```

If we cared only about the `x` and `y` properties, we could write some simple `cffi` code that cares only about those values:

```

from cffi import FFI

ffi = FFI()
ffi.cdef(r"""
    struct Point {
        double x;
        double y;
        ...;
    };
    struct Point do_calculation();
""")
lib = ffi.verify(r"""
    #include <complicated.h>
""")

```

We could then run the `do_calculation` function from the `complicated.h` library and have returned to us a `Point` object with its `x` and `y` properties accessible. This is amazing for portability, since this code will run just fine on systems with a different implementation of `Point` or when new versions of `complicated.h` come out, as long as they all have the `x` and `y` properties.

All of these niceties really make `cffi` an amazing tool to have when you're working with C code in Python. It is much simpler than `ctypes`, while still giving you the same amount of fine-grained control you may want when working directly with a foreign function interface.

f2py

For many scientific applications, Fortran is still the gold standard. While its days of being a general-purpose language are over, it still has many niceties that make vector operations easy to write and quite quick. In addition, many performance math libraries are written in Fortran (**LAPACK**, **BLAS**, etc.), all of which are fundamental in libraries such as SciPy, and being able to use them in your performance Python code may be critical.

For such situations, **f2py**, which is a part of your standard `numpy` installation, provides a dead-simple way of importing Fortran code into Python. This module is able to be so simple because of the explicitness of types in Fortran. Since the types can be easily parsed and understood, `f2py` can easily make a CPython module that uses the native foreign function support within C to use the Fortran code. This means that when you are using `f2py`, you are actually autogenerating a C module that knows how to use Fortran code! As a result, a lot of the confusion inherent in the `ctypes` and `cffi` solutions simply doesn't exist.

In **Example 8-22**, we can see some simple `f2py`-compatible code for solving the diffusion equation. In fact, all native Fortran code is `f2py`-compatible; however, the annotations to the function arguments (the statements prefaced by `!f2py`) simplify the resulting Python module and make for an easier-to-use interface. The annotations implicitly tell `f2py` whether we intend for an argument to be only an output or only an input, or to be something we want to modify in place or hidden completely. The hidden type is particularly useful for the sizes of vectors: while Fortran may need those numbers explicitly, our Python code already has this information on hand. When we set the type as “hidden,” `f2py` can automatically fill those values for us, essentially keeping them hidden from us in the final Python interface.

Example 8-22. Fortran 2D diffusion code with `f2py` annotations

```
SUBROUTINE evolve(grid, next_grid, D, dt, N, M)
    !f2py threadsafe
    !f2py intent(in) grid
    !f2py intent(inplace) next_grid
    !f2py intent(in) D
    !f2py intent(in) dt
    !f2py intent(hide) N
    !f2py intent(hide) M
    INTEGER :: N, M
```

```

DOUBLE PRECISION, DIMENSION(N,M) :: grid, next_grid
DOUBLE PRECISION, DIMENSION(N-2, M-2) :: laplacian
DOUBLE PRECISION :: D, dt

laplacian = grid(3:N, 2:M-1) + grid(1:N-2, 2:M-1) + &
            grid(2:N-1, 3:M) + grid(2:N-1, 1:M-2) - 4 * grid(2:N-1, 2:M-1)
next_grid(2:N-1, 2:M-1) = grid(2:N-1, 2:M-1) + D * dt * laplacian
END SUBROUTINE evolve

```

Once the numpy and meson libraries are installed, we can build the code into a Python module by running the following command:

```
$ f2py -c -m diffusion --fcompiler=gfortran --opt='-O3' diffusion.f90
```



We specifically use gfortran in the preceding call to f2py. Make sure it is installed on your system or that you change the corresponding argument to use the Fortran compiler you have installed.

This will create a library file pinned to your Python version and operating system (diffusion.cpython-312-x86_64-linux-gnu.so, in our case) that can be imported directly into Python.

If we play around with the resulting module interactively, we can see the niceties that f2py has given us, thanks to our annotations and its ability to parse the Fortran code:

```

>>> import diffusion

>>> diffusion?
Type:          module
String form:   <module 'diffusion' from '[...]cpython-312m-x86_64-linux-gnu.so'>
File:         [...]cut..]/diffusion.cpython-312m-x86_64-linux-gnu.so
Docstring:
This module 'diffusion' is auto-generated with f2py (version:2).
Functions:
    evolve(grid,scratch,d,dt)
.

>>> diffusion.evolve?
Call signature: diffusion.evolve(*args, **kwargs)
Type:          fortran
String form:   <fortran object>
Docstring:
evolve(grid,scratch,d,dt)

Wrapper for ``evolve``.

Parameters
grid : input rank-2 array('d') with bounds (n,m)
scratch : rank-2 array('d') with bounds (n,m)

```

```
d : input float
dt : input float
```

This code shows that the result from the f2py generation is automatically documented, and the interface is quite simplified. For example, instead of us having to extract the sizes of the vectors, f2py has figured out how to automatically find this information and simply hides it in the resulting interface. In fact, the resulting `evolve` function looks exactly the same in its signature as the pure Python version we wrote in [Example 6-13](#).

The only thing we must be careful of is the ordering of the `numpy` arrays in memory. Since most of what we do with `numpy` and Python focuses on code derived from C, we always use the C convention for ordering data in memory (called *row-major ordering*). Fortran uses a different convention (*column-major ordering*) that we must make sure our vectors abide by. These orderings simply state whether, for a 2D array, columns or rows are contiguous in memory.⁴ Luckily, this simply means we specify the `order='F'` parameter to `numpy` when declaring our vectors.



The difference in ordering basically changes which is the outer loop when iterating over a multidimensional array. In Python and C, if you define an array as `array[X][Y]`, your outer loop will be over `X` and your inner loop will be over `Y`. In Fortran, your outer loop will be over `Y` and your inner loop will be over `X`. If you use the wrong loop ordering, you will at best suffer a major performance penalty because of an increase in cache-misses (see “[Memory Fragmentation](#)” on page 136) and at worst access the wrong data!

This results in the following code to use our Fortran subroutine. This code looks exactly the same as what we used in [Example 6-13](#), except for the import from the f2py-derived library and the explicit Fortran ordering of our data:

```
from diffusion import evolve

def run_experiment(num_iterations):
    scratch = np.zeros(grid_shape, dtype=np.double, order="F") ❶
    grid = np.zeros(grid_shape, dtype=np.double, order="F")

    initialize_grid(grid)

    for i in range(num_iterations):
        evolve(grid, scratch, 1.0, 0.1)
        grid, scratch = scratch, grid
```

⁴ For more information, see [the Wikipedia page](#) on row-major and column-major ordering.

- ❶ Fortran orders numbers differently in memory, so we must remember to set our numpy arrays to use that standard.

CPython Extensions: C

We can always go right down to the CPython API level and write a CPython module. This requires us to write code in the same way that CPython is developed and take care of all of the interactions between our code and the implementation of CPython.

This has the advantage that it is incredibly portable, depending on the Python version. We don't require any external modules or libraries, just a C compiler and Python! However, this doesn't necessarily scale well to new versions of Python. For example, CPython modules written for Python 2.7 won't work with Python 3, and vice versa.



In fact, much of the slowdown in the Python 3 rollout was rooted in the difficulty in making this change. When creating a CPython module, you are coupled very closely to the actual Python implementation, and large changes in the language (such as the change from 2.7 to 3) require large modifications to your module.

That portability comes at a big cost—you are responsible for every aspect of the interface between your Python code and the module. This can make even the simplest tasks take dozens of lines of code. For example, to interface with the diffusion library from [Example 8-18](#), we must write 28 lines of code simply to read the arguments to a function and parse them ([Example 8-23](#)). Of course, this does mean that you have incredibly fine-grained control over what is happening. This goes all the way down to being able to manually change the reference counts for Python's garbage collection (which can be the cause of a lot of pain when creating CPython modules that deal with native Python types). Because of this, the resulting code tends to be minutely faster than other interface methods.



All in all, this method should be left as a last resort. While it is quite informative to write a CPython module, the resulting code is not as reusable or maintainable as other potential methods. Making subtle changes in the module can often require completely reworking it. In fact, we include the module code and the required *setup.py* to compile it ([Example 8-24](#)) as a cautionary tale.

Example 8-23. CPython module to interface with the 2D diffusion library

```
// python_interface.c
// - cpython module interface for diffusion.c

#define NPY_NO_DEPRECATED_API  NPY_1_7_API_VERSION

#include <Python.h>
#include <numpy/arrayobject.h>
#include "diffusion.h"

/* Docstrings */
static char module_docstring[] =
    "Provides optimized method to solve the diffusion equation";
static char cdiffusion_evolve_docstring[] =
    "Evolve a 2D grid using the diffusion equation";

PyArrayObject *py_evolve(PyObject *, PyObject *);

/* Module specification */
static PyMethodDef module_methods[] =
{
    /* { method name , C function , argument types , docstring } */
    { "evolve", (PyCFunction)py_evolve, METH_VARARGS, cdiffusion_evolve_docstring },
    { NULL, NULL, 0, NULL }
};

static struct PyModuleDef cdiffusionmodule =
{
    PyModuleDef_HEAD_INIT,
    "cdiffusion", /* name of module */
    module_docstring, /* module documentation, may be NULL */
    -1, /* size of per-interpreter state of the module,
        * or -1 if the module keeps state in global variables. */
    module_methods
};

PyArrayObject *py_evolve(PyObject *self, PyObject *args)
{
    PyArrayObject *data;
    PyArrayObject *next_grid;
    double dt, D = 1.0;

    /* The "evolve" function will have the signature:
     *   evolve(data, next_grid, dt, D=1)
     */
    if (!PyArg_ParseTuple(args, "OOd|d", &data, &next_grid, &dt, &D))
    {
        PyErr_SetString(PyExc_RuntimeError, "Invalid arguments");
        return(NULL);
    }
}
```

```

/* Make sure that the numpy arrays are contiguous in memory */
if (!PyArray_Check(data) || !PyArray_ISCONTIGUOUS(data))
{
    PyErr_SetString(PyExc_RuntimeError, "data is not a contiguous array.");
    return(NULL);
}
if (!PyArray_Check(next_grid) || !PyArray_ISCONTIGUOUS(next_grid))
{
    PyErr_SetString(PyExc_RuntimeError, "next_grid is not a contiguous array.");
    return(NULL);
}

/* Make sure that grid and next_grid are of the same type and have the same
 * dimensions
 */
if (PyArray_TYPE(data) != PyArray_TYPE(next_grid))
{
    PyErr_SetString(PyExc_RuntimeError,
        "next_grid and data should have same type.");
    return(NULL);
}
if (PyArray_NDIM(data) != 2)
{
    PyErr_SetString(PyExc_RuntimeError, "data should be two dimensional");
    return(NULL);
}
if (PyArray_NDIM(next_grid) != 2)
{
    PyErr_SetString(PyExc_RuntimeError, "next_grid should be two dimensional");
    return(NULL);
}
if ((PyArray_DIM(data, 0) != PyArray_DIM(next_grid, 0)) ||
    (PyArray_DIM(data, 1) != PyArray_DIM(next_grid, 1)))
{
    PyErr_SetString(PyExc_RuntimeError,
        "data and next_grid must have the same dimensions");
    return(NULL);
}

evolve(
    PyArray_DATA(data),
    PyArray_DATA(next_grid),
    D,
    dt
);

Py_XINCREF(next_grid);
return(next_grid);
}

/* Initialize the module */
PyMODINIT_FUNC

```

```

PyInit_cdifffusion(void)
{
    PyObject *m;

    m = PyModule_Create(&cdifffusionmodule);
    if (m == NULL)
    {
        return(NULL);
    }

    /* Load `numpy` functionality. */
    import_array();

    return(m);
}

```

To build this code, we need to create a *setup.py* script that uses the `distutils` module to figure out how to build the code such that it is Python-compatible (Example 8-24). In addition to the standard `distutils` module, `numpy` provides its own module to help with adding `numpy` integration in your CPython modules.

Example 8-24. Setup file for the CPython module diffusion interface

```

"""
setup.py for cpython diffusion module. The extension can be built by running

    $ python setup.py build_ext --inplace

which will create the __cdifffusion.so__ file, which can be directly imported into
Python.
"""

from distutils.core import setup, Extension
import numpy as np

__version__ = "0.1"

cdifffusion = Extension(
    'cdifffusion',
    sources = ['cdifffusion/cdifffusion.c', 'cdifffusion/python_interface.c'],
    extra_compile_args = ["-O3", "-std=c11", "-Wall", "-p", "-pg", ],
    extra_link_args = ["-lc"],
)

setup (
    name = 'diffusion',
    version = __version__,
    ext_modules = [cdifffusion,],
    packages = ["diffusion", ],
    include_dirs = np.get_include(),
)

```

The result from this is a `cdiffusion.cpython-312-x86_64-linux-gnu.so` file that can be imported directly from Python and used quite easily. Since we had complete control over the signature of the resulting function and exactly how our C code interacted with the library, we were able (with some hard work) to create a module that is easy to use:

```
from cdiffusion import evolve

def run_experiment(num_iterations):
    next_grid = np.zeros(grid_shape, dtype=np.double)
    grid = np.zeros(grid_shape, dtype=np.double)

    # ... standard initialization ...

    for i in range(num_iterations):
        evolve(grid, next_grid, 1.0, 0.1)
        grid, next_grid = next_grid, grid
```

CPython Extensions: Rust

The following section is a contribution from [Itamar Turner-Trauring](#). In it, he shows the ease of making a Rust extension to CPython. This is an amazing counterpoint to the complex Python extension written in C from [“CPython Extensions: C” on page 252](#).

Rust is a modern compiled language that aims at the same niche as C++, combining a high level of abstraction with compilation to very fast code. However, it has a number of fundamental benefits over the C++ language:

A built-in, modern packaging and build toolchain

Like Python’s PyPI/pip and Conda, Rust comes with an online package repository and corresponding command-line tool to manage your project’s dependencies, also known as *crates*. It also includes a single standardized, built-in build system called Cargo.

Memory safety by default

No more worries about buffer overflows, use-after-free, and the rest of the problems with C and C++ that can result in corrupted data or crashes.

Thread safety by default, aka “fearless concurrency”

The same language features that provide memory safety also ensure that the memory safety problems caused by threading in C++ (and in most other languages, really) aren’t a problem in Rust. That means scaling to multiple CPUs can often be done very simply and safely.

To write Python extensions with Rust, you can use a crate (Rust library) called **PyO3**. At the time of writing, PyO3 has not hit 1.0 yet, which is an example of one of the downsides of using Rust: while it's used extensively in industry and increasingly in science, it's still a newer language with fewer libraries.

Creating a Python extension with Rust

If you have an existing Python project using **setuptools**, you can add support for building Rust extensions in your project using the **setuptools-rust package**.

However, since we're creating a stand-alone extension, we'll use **maturin**, which is an all-in-one packaging tool specifically for Python/Rust extensions. We can install Maturin using **pip** in a new virtualenv:

```
$ python3 -m venv ./venv
$ . venv/bin/activate
$ pip install maturin
```

Now that we've installed Maturin, we can use its command-line tool to create all the boilerplate files we need to create a Rust-based Python extension:

```
$ maturin new diffusion -b pyo3
$ cd diffusion/
```

We now have all the files we need to build a Rust extension for Python, which means we can **pip install** our new package:

```
$ pip install .
...
Successfully installed diffusion-0.1.0
$ python -c "import diffusion; print(diffusion)"
<module 'diffusion' from
'venv/lib/python3.10/site-packages/diffusion/__init__.py'>
```

Here's what the tree of files generated by Maturin looks like (not shown, but also included, are a **.gitignore** file and a minimal GitHub Actions configuration):

```
├─ Cargo.toml      # Configuration for Rust's build system
├─ pyproject.toml  # Enables pip installing the package
└─ src
    └─ lib.rs       # A Rust library, pre-configured to use PyO3
```

Rust packages are called *crates*, and **cargo**, the command-line Rust build tool, can be used to add crates as dependencies for our project. In this case, we're going to want to use the **numpy** crate so we can access NumPy arrays from Rust. We can add it as a dependency like so:

```
$ cargo add numpy
```

It will then get added to `Cargo.toml` as a dependency; `PyO3` was added automatically by `Maturin` when it first created the `Cargo.toml`:

```
[dependencies]
numpy = "0.21.0"
pyo3 = "0.21.0"
```

Writing the extension

When we ran `maturin new`, that created a minimal Rust module for us in a file called `src/lib.rs`, with just enough boilerplate to compile a full-fledged Python extension. Next we want to update this file to run our own code.

We'll start by implementing the same diffusion algorithm we've seen in other languages, only this time in Rust. The `numpy` Rust crate exposes the NumPy arrays to Rust using the functionality of a less Python-specific crate called `ndarray`. Here's how we'd write a function implementing the diffusion algorithm using `ndarray` arrays, with no reference to Python:

```
// Import the structs we'll use in our API:
use numpy::ndarray::{ArrayView2, ArrayViewMut2};

// Calculate the evolution of a grid, operating on ndarray arrays:
fn evolve(
    grid: ArrayView2<f64>,
    mut out_write: ArrayViewMut2<f64>,
    D: f64,
    dt: f64
) {
    let shape = grid.shape();
    assert_eq!(shape, out_write.shape());
    // 1..n is equivalent to range(1, n) in Python:
    for i in 1..(shape[0] - 1) {
        for j in 1..(shape[1] - 1) {
            let laplacian =
                grid[(i + 1, j)] + grid[(i - 1, j)]
                + grid[(i, j + 1)] + grid[(i, j - 1)]
                - 4.0 * grid[(i, j)];
            out_write[(i, j)] = grid[(i, j)] + D * dt * laplacian;
        }
    }
}
```

This is not much different than any other compiled language, though some underlying differences are less visible:

- Rust enforces a strict rule that if you have a mutable (i.e., writable) reference to some data, you cannot have any other references to it, whether writable or read-only. That's how it enforces memory safety. The `ndarray` crate exposes this in its type system by having two kinds of views for arrays, read-only and mutable.

- Rust will do bounds checking on all array lookups, so if you have an off-by-one bug and read entry N+1 from an array that has only N entries, you will get an explicit error, rather than memory corruption or mysterious crashes.

Now that we have a generic implementation of the algorithm, we can expose it as Python function:

```
use numpy::{
    PyArray2, PyArrayMethods, PyReadonlyArray2, PyUntypedArrayMethods,
};
use pyo3::prelude::*;

// Wrap the evolve() function so it can be used from Python:
#[pyfunction(name = "evolve")]
#[pyo3(signature = (grid, dt, D=1.0))]
fn evolve_py<'py>(
    py: Python<'py>,
    grid: PyReadonlyArray2<'py, f64>,
    dt: f64,
    D: f64,
) -> PyResult<Bound<'py, PyArray2<f64>>> {
    let shape = grid.shape();
    // Create a new 2D float64 array filled with zeroes with the same shape as
    // the grid:
    let out_arr = PyArray2::::zeros_bound(py, [shape[0], shape[1]], false);

    evolve(
        // Pass in a read-only view of the grid:
        grid.as_array(),
        // Pass in a writable view of the output array:
        out_arr.readwrite().as_array_mut(),
        D,
        dt
    );
    Ok(out_arr)
}
```

Finally, we register the function with the module our extension will expose to Python:

```
// Expose a Python module called "diffusion", with our new function:
#[pymodule]
fn diffusion(_py: Python, m: &Bound<'_, PyModule>) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(evolve_py, m)??);
    Ok(())
}
```

Installing and using the extension

We can now install the extension:

```
$ pip install .
```

And now we can use it from Python:

```
import numpy as np
from diffusion import evolve

arr = np.random.random((512, 512))
result = evolve(arr, 0.1, D=0.5)
print(result)
```

Rust is a general-purpose compiled programming language with modern tooling, the ability to write both fast code and high-level abstractions, and memory safety and thread safety by default. It also has excellent Python integration. As such, Rust excels at building larger-scale complex libraries; for example, the Polars dataframe library is mostly written in Rust. But because of its excellent tooling, it also scales down to smaller extensions, as we’ve seen in this section.

The main downsides to Rust are complexity—some of its concepts are quite different from other compiled languages—and the limited number of specialized computational libraries. The latter will be solved with time. The former may or may not be a problem depending on your goals; if you expect to write large amounts of compiled code, Rust is likely to be the ideal choice.

Wrap-Up

The various strategies introduced in this chapter allow you to specialize your code to different degrees in order to reduce the number of instructions the CPU must execute and to increase the efficiency of your programs. Sometimes this can be done algorithmically, although often it must be done manually (see “[JIT Versus AOT Compilers](#)” on page 216). Furthermore, sometimes these methods must be employed simply to use libraries that have already been written in other languages. Regardless of the motivation, Python allows us to benefit from the speedups that other languages can offer on some problems, while still maintaining verbosity and flexibility when needed.

It is important to note, though, that these optimizations are done to optimize the efficiency of compute instructions only. If you have I/O-bound processes coupled to a compute-bound problem, simply compiling your code may not provide any reasonable speedups. For these problems, we must rethink our solutions and potentially use parallelism to run different tasks at the same time.

Asynchronous I/O

Questions You'll Be Able to Answer After This Chapter

- What is concurrency, and how is it helpful?
- What is the difference between concurrency and parallelism?
- Which tasks can be done concurrently, and which can't?
- What are the various paradigms for concurrency?
- When is the right time to take advantage of concurrency?
- How can concurrency speed up my programs?

So far we have focused on speeding up code by increasing the number of compute cycles that a program can complete in a given time. However, in the days of big data, getting the relevant data to your code can be the bottleneck, as opposed to the actual code itself. When this is the case, your program is called *I/O bound*; in other words, the speed is bounded by the efficiency of the input/output.

I/O can be quite burdensome to the flow of a program. Every time your code reads from a file or writes to a network socket, it must pause to contact the kernel, request that the actual read happens, and then wait for it to complete. This is because it is not your program but the kernel that does the actual read operation, since the kernel is responsible for managing any interaction with hardware. The additional layer may not seem like the end of the world, especially once you realize that a similar operation happens every time memory is allocated; however, if we look back at [Figure 1-3](#), we see that most of the I/O operations we perform are on devices that are orders of magnitude slower than the CPU. So even if the communication with the kernel is fast,

we will be waiting quite some time for the kernel to get the result from the device and return it to us.

For example, in the time it takes to write to a network socket, an operation that typically takes about 1 millisecond, we could have completed 2,400,000 instructions on a 2.4 GHz computer. Worst of all, our program is halted for much of this 1 millisecond of time—our execution is paused, and then we wait for a signal that the write operation has completed. This time spent in a paused state is called *I/O wait*.

Asynchronous I/O helps us utilize this wasted time by allowing us to perform other operations while we are in the I/O wait state. For example, in [Figure 9-1](#) we see a depiction of a program that must run three tasks, all of which have periods of I/O wait within them. If we run them serially, we suffer the I/O wait penalty three times. However, if we run these tasks concurrently, we can essentially hide the wait time by running other tasks while waiting for the I/O result. It is important to note that this is all still happening on a single thread on a single CPU and with a shared memory space!

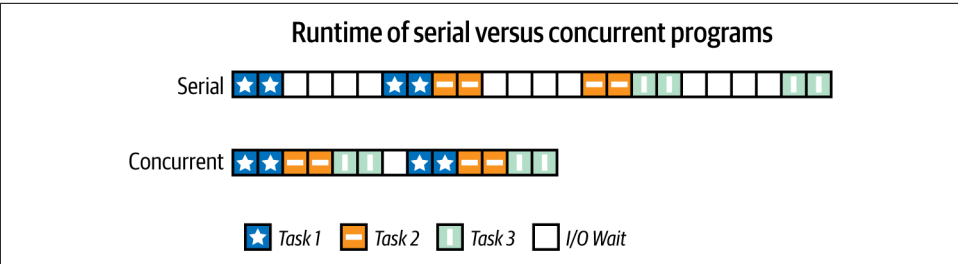


Figure 9-1. Comparison of serial and concurrent programs

Since the multiple tasks are running in the same thread, concurrency is all about how to interleave dead time in each task and to allow multiple tasks to share the same resources. This is in contrast to parallel programming (which we will talk about in [Chapter 10](#)), where each task gets its own individual resources.

This is possible because while our program is in I/O wait, our program is waiting for the kernel to respond with data from whichever device we've requested to read from (hard drive, network adapter, GPU, etc.). Instead of waiting, we can do other tasks and check back on the status of the I/O operation later in order to finish that portion of the program (this mechanism is called the event loop). This is in stark contrast to the multiprocessing/multithreading ([Chapter 10](#)) paradigm, where a new process is launched that does experience I/O wait but uses the multitasking nature of modern CPUs to allow the main process to continue. However, the two mechanisms are often used in tandem, where we launch multiple processes, each of which is efficient at asynchronous I/O, in order to fully take advantage of our compute resources.



Since concurrent programs run on a single thread, they are generally easier to write and manage than standard multithreaded programs. All concurrent functions share the same memory space, so sharing data between them works in the normal ways you would expect. However, you still need to be careful about race conditions since you can't be sure which lines of code get run when. We'll see in [“Verifying Primes Using Interprocess Communication” on page 324](#) just how complicated things can get when the memory space between different processes is not shared.

By modeling a program in this way, where portions of code can be paused and then resumed when the resources they need are available, we are able to take advantage of I/O wait to perform more operations on a single thread than would otherwise be possible.

Introduction to Asynchronous Programming

Typically, when a program enters I/O wait, the execution is paused so that the kernel can perform the low-level operations associated with the I/O request (this is called a *context switch*), and it is not resumed until the I/O operation is completed. Context switching is quite a heavy operation. It requires us to save the state of our program (losing any sort of caching we had at the CPU level) and give up the use of the CPU. Later, when we are allowed to run again, we must spend time filling the various caches and registers, reinitializing our program on the motherboard, and getting ready to resume running (of course, all this happens behind the scenes).

With concurrency, on the other hand, we typically have an *event loop* running that manages what gets to run in our program, and when. In essence, an event loop is simply a list of functions that need to be run. The function at the top of the list gets run, then the next, and so on. [Example 9-1](#) shows a simple example of an event loop.

Example 9-1. Toy event loop

```
from queue import Queue
from functools import partial

eventloop = None

class EventLoop(Queue):
    def start(self):
        while True:
            function = self.get() ❶
            function()

def do_hello():
    global eventloop
```

```

print("Hello")
eventloop.put(do_world) ❷

def do_world():
    global eventloop
    print("world")
    eventloop.put(do_hello) ❷

if __name__ == "__main__":
    eventloop = EventLoop()
    eventloop.put(do_hello)
    eventloop.start()

```

- ❶ get retrieves the first item in the Queue object (which EventLoop subclasses). This item is a runnable function that takes no arguments.
- ❷ Wherever the eventloop object is available, we can put new runnable functions in the queue for it to run at a later time.

This may not seem like a big change; however, we can couple event loops with asynchronous (async) I/O operations for massive gains when performing I/O tasks. In this example, the call `eventloop.put(do_world)` approximates an asynchronous call to the `do_world` function. This operation is called *nonblocking*, meaning it will return immediately but guarantee that `do_world` is called at some point later. Similarly, if this were a network write with an async function, it would return right away even though the write has not happened yet. When the write has completed, an event fires so our program knows about it and can respond accordingly.

Putting these two concepts together, we can have a program that, when an I/O operation is requested, runs other functions while waiting for the original I/O operation to complete. This essentially allows us to still do meaningful calculations when we otherwise would have been in I/O wait.



Switching from function to function does have a cost. The kernel must take the time to set up the function to be called in memory, and the state of our caches won't be as predictable. It is because of this that concurrency gives the best results when your program has a lot of I/O wait—the cost associated with switching can be much less than what is gained by making use of I/O wait time.

Generally, programming using event loops can take two forms: callbacks or futures. In the callback paradigm, functions are called with an argument that is generally called the *callback*. Instead of the function returning its value, it calls the callback function with the value instead. This sets up long chains of functions that are called, with each function getting the result of the previous function in the chain (these

chains are sometimes referred to as “callback hell” because it can become incredibly difficult to trace the actual runtime flow of a program). **Example 9-2** is a simple example of the callback paradigm, but because of the awkward flow control from this method, it can still be difficult to grok.

Example 9-2. Example with callbacks

```
from functools import partial
from some_database_library import save_results_to_db

def save_value(value, callback):
    result = f"Hello {value}"
    print(f"Saving {value} to database")
    save_result_to_db(result, callback) ❶

def print_response(db_response):
    print("Response from database: {db_response}")

if __name__ == "__main__":
    eventloop = EventLoop()
    task = partial(save_value, "World", print_response) ❷
    eventloop.put(task)
    eventloop.start()
```

- ❶ `save_result_to_db` is an asynchronous function; it will return immediately, and the function will end and allow other code to run. At some point later in the execution, once the data is ready, `print_response` will be called with the result of the function call.
- ❷ Partial functions are often used in asynchronous programming. In this example, it sets the value to "World" and the callback to `print_response`, creating a new function, `task`, which takes no arguments. Generally, the `functools.partial` function allows to copy a function but with fixed arguments. This is useful for facilitating calling a function in the future by simplifying its call signature. Most tasks and callbacks are assumed to take no parameters, so partial functions are a way of accomplishing this.

Before Python 3.4, the callback paradigm was quite popular. However, the `asyncio` standard library module and PEP 492 made the futures mechanism native to Python. This was done by creating a standard API for dealing with asynchronous I/O and the new `await` and `async` keywords, which define an asynchronous function and a way to wait for a result.

In this paradigm, an asynchronous function returns a `Future` object, which is a promise of a future result. Because of this, if we want the result, at some point we must wait for the future that is returned by this sort of asynchronous function to complete and be filled with the value we desire (either by doing an `await` on it or by running a function that explicitly waits for a value to be ready). However, this also means that the result can be available in the callers context, while in the callback paradigm the result is available only in the callback function. While waiting for the `Future` object to be filled with the data we requested, we can do other calculations. If we couple this with the concept of generators—functions that can be paused and whose execution can later be resumed—we can write asynchronous code that looks very close to serial code. We can see in [Example 9-3](#) just how much more natural and easy to read this code is.

Example 9-3. Example with `async/await`

```
from some_async_database_library import save_results_to_db

async def save_value(value):
    result = f"Hello {value}"
    print(f"Saving {result} to database")
    db_response = await save_result_to_db(result) ❶
    print("Response from database: {db_response}")

if __name__ == "__main__":
    eventloop = EventLoop()
    eventloop.put(
        partial(save_value, "World")
    )
    eventloop.start()
```

- ❶ In this case, `save_result_to_db` returns a `Future` type. By awaiting it, we ensure that `save_value` gets paused until the value is ready and then resumes and completes its operations.

It's important to realize that the `Future` object returned by `save_result_to_db` holds the *promise* of a `Future` result and doesn't hold the result itself or even call any of the `save_result_to_db` code. If we simply did `db_response_future = save_result_to_db(result)`, the statement would complete immediately and we could do other things with the `Future` object. In fact, none of the `save_value` function would actually run until the event loop schedules time for it. This is often done when we want to collect a list of futures and wait for all of them at the same time.

How Does `async/await` Work?

An `async` function (defined with `async def`) is called a *coroutine*. In Python, coroutines are implemented with the same philosophies as generators. This is convenient because generators already have the machinery to pause their execution and resume later. Using this paradigm, an `await` statement is similar in function to a `yield` statement; the execution of the current function gets paused while other code is run. Once the `await` or `yield` resolves with data, the function is resumed with potentially some extra data. So in the preceding example, our `save_result_to_db` will return a `Future` object, and the `await` statement pauses the function until that `Future` contains a result. The event loop is responsible for scheduling the resumption of `save_value` after the `Future` is ready to return a result.



The `asyncio` library was first introduced as an unstable standard library in Python 3.4 and then in Python 3.8 it graduated to a stable API. In that period of evolution, many large things have changed and even more small things and library nuances have taken shape. If you had some experience with the `asyncio` library before Python 3.8, it would be a good idea to spend some time with the documentation to learn about the new features and potentially some new nuances of the library.

It is critical to realize our reliance on an event loop when running concurrent code. In general, this leads to most fully concurrent code's main code entry point consisting largely of setting up and starting the event loop. However, this assumes that your entire program is concurrent. In other cases, a set of futures is created within the program, and then a temporary event loop is started simply to manage the existing futures, before the event loop exits and the code can resume normally. This is generally done with `asyncio.run(coro)`.

In this chapter we will analyze a web crawler that fetches data from an HTTP server that has latency built into it. This represents the general response-time latency that will occur whenever dealing with I/O. We will first create a serial crawler that looks at the naive Python solution to this problem. Then we will build up to a full `aiohttp` solution to see how we can benefit from asynchronous I/O. Finally, we will look at combining `async` I/O tasks with CPU tasks in order to effectively hide any time spent doing I/O.



The web server we implemented can support multiple connections at a time. This will be true for most services that you will be performing I/O with—most databases can support multiple requests at a time, and most web servers support 10,000+ simultaneous connections. However, when interacting with a service that cannot handle multiple connections at a time, we will always have the same performance as the serial case.

Serial Web Crawler

As a baseline to understand asynchronous I/O, we will write a serial web scraper that takes a list of URLs, fetches them, and sums the total length of the content from the pages. We will use a custom HTTP server that takes two parameters, `name` and `delay`. The `delay` field will tell the server how long, in milliseconds, to pause before responding. The `name` field is for logging purposes.

By controlling the `delay` parameter, we can simulate the time it takes a server to respond to our query. In the real world, this could correspond to a slow web server, a strenuous database call, or any I/O call that takes a long time to perform. For the serial case, this leads to more time that our program is stuck in I/O wait, but in the concurrent example it will provide an opportunity to spend the I/O wait time doing other tasks such as launching more I/O requests.

In [Example 9-4](#), we chose to use the `requests` module to perform the HTTP call. We made this choice because of the ubiquity of the module.

We use HTTP for this section because it is a simple example of I/O that can be performed easily and is used frequently in data science for some of the most I/O intensive tasks. In general, any call to an HTTP library can be replaced with any I/O.

Example 9-4. Serial HTTP scraper

```
import random
import string

import requests

def generate_urls(base_url, num_urls):
    """
    We add random characters to the end of the URL to break any caching
    mechanisms in the requests library or the server
    """
    for i in range(num_urls):
        yield base_url + "".join(random.sample(string.ascii_lowercase, 10))
```

```

def process(session, url)
    response = session.get(url)
    return len(response.text)

def run_experiment(base_url, num_iter=1000):
    response_size = 0
    with requests.Session() as session:
        for url in generate_urls(base_url, num_iter):
            response_size += process(session, url)
    return response_size

if __name__ == "__main__":
    import time

    delay = 100
    num_iter = 1000
    base_url = f"http://127.0.0.1:8080/add?name=serial&delay={delay}&"

    start = time.time()
    result = run_experiment(base_url, num_iter)
    end = time.time()
    print(f"Result: {result}, Time: {end - start}")

```

When running this code, an interesting metric to look at is the start and stop time of each request as seen by the HTTP server. This tells us how efficient our code was during I/O wait—since our task is to launch HTTP requests and then sum the number of characters that were returned, we should be able to launch more HTTP requests, and process any responses, while waiting for other requests to complete.

We can see in [Figure 9-2](#) that, as expected, there is no interleaving of our requests. We do one request at a time and wait for that request to happen before we move to the next request. In fact, the total runtime of the serial process makes perfect sense knowing this. Since each request takes 0.1 seconds (because of our `delay` parameter) and we are doing 1,000 requests, we expect this runtime to be about 100 seconds. Looking at [Figure 9-2](#) we see that because of various overheads, the actual runtime was 101.89 seconds.

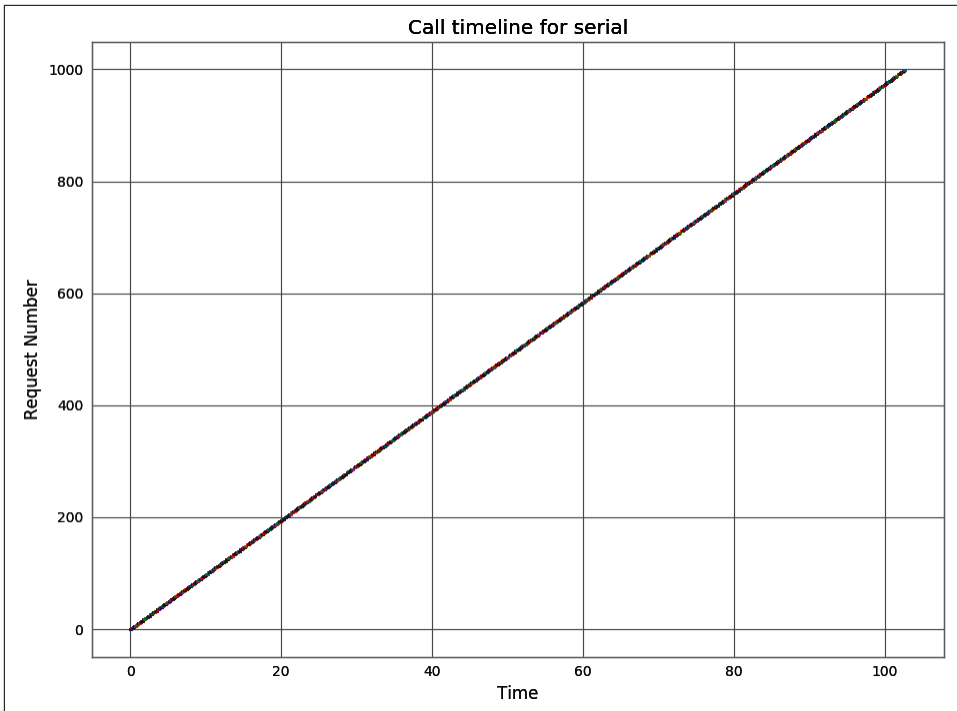


Figure 9-2. Request time for *Example 9-4*

Asynchronous Web Crawler

There are entire ecosystems in Python around asynchronous I/O. There are many asynchronous versions of common libraries (such as `asyncpg` for an async PostgreSQL database client, or `anyio` for generalized async networking). For this chapter we chose to focus on `aiohttp`, which is part of the `aio-lib`s project. This is meant to expose the `asyncio` module in a more user-friendly way.

Especially since we are focusing on HTTP requests, there are many different libraries to choose from, but they generally follow the same paradigm, and learning one will help you understand the others. A good choice for all-around HTTP applications (clients and servers) is `aiohttp` or `httpx` if either HTTP/2 support is critical or you want to use the same module for both synchronous and asynchronous portions of your code.

The big difficulty with asynchronous code is that either you refactor your entire codebase to be asynchronous or you need to explicitly switch between the two behaviors. Switching between the two can be done easily enough using the `asyncio.run` function; however, any code written in an asynchronous function (or coroutine) can only be used while the code is running asynchronously.

Example 9-5 seems quite different, but it also shares many similarities with the serial code. The biggest difference is that instead of simply calling the process function, we are converting it into tasks that are managed by our TaskGroup and then processing all the results at once. A key thing in understanding the asynchronous nature of this code, though, is realizing at what point the first request is launched. Take a second and try to see what your intuition says.

Example 9-5. asyncio HTTP scraper

```
import asyncio
import random
import string

import aiohttp

def generate_urls(base_url, num_urls):
    """
    We add random characters to the end of the URL to break any caching
    mechanisms in the requests library or the server
    """
    for _ in range(num_urls):
        yield base_url + ".".join(random.sample(string.ascii_lowercase, 10))

async def process(session, url):
    async with session.get(url) as response:
        return len(await response.text())

async def run_experiment(base_url, num_iter=1000):
    tasks = []
    async with aiohttp.ClientSession() as session:
        async with asyncio.TaskGroup() as tg: ❶
            for url in generate_urls(base_url, num_iter):
                coro = process(session, url)
                task = tg.create_task(coro)
                tasks.append(task)
    response_size = sum(task.result() for task in tasks) ❷
    return response_size

if __name__ == "__main__":
    import time

    delay = 100
    num_iter = 1000
    base_url = f"http://127.0.0.1:8080/add?name=aiohttp&delay={delay}&"
    experiment = run_experiment(base_url, num_iter)
```

```
start = time.time()
result = asyncio.run(experiment)
end = time.time()
print(f"Result: {result}, Time: {end - start}")
```

- ❶ Task groups were added in Python 3.11 and are an amazing way to simplify the management of many tasks running at once.
- ❷ Since all of our processing tasks are running concurrently, we can either process the results from each one as they come in or we can process them all at once when everything is ready. If the postprocessing is intensive, doing it as the results come in is a better use of compute resources, but in this case we don't lose anything doing it all at once.

It is important to realize that no part of the process function gets called when we create the `coro` object. Instead, we create the promise of a future result. By adding it to our list of tasks we are able to retrieve this result. But when is the result actually calculated?

Coroutines get exclusive use of the event loop until they `await`. This `await` gives back execution control to the event loop so that it can deal with other pending tasks. This is something we can control by having asynchronous “no-op” statements (like, as we'll see later in this chapter, the famous `await asyncio.sleep(0)` statement). As a result, when we don't have an `await` statement within the loop creating all of our tasks, we never give control back to the event loop. This results in the `TaskGroup` never having the time to actually run any of the tasks that are being created until the `TaskGroup` context manager enters its “exit” state that an `await` happens, allowing other tasks to run. Furthermore, the main purpose of the `TaskGroup` object is to ensure that not only do the tasks have time to run but, when the context manager exits, all of the tasks are finished (or canceled in the case of error).



In the default execution, tasks are not immediately run on creation (unlike generators, whose code gets run when the generator is called until the first `yield` is called). Instead a task only starts running once the event loop gives that task time in the schedule. This behavior can be changed using `asyncio.eager_task_factory`, which can immensely help optimize resource use when a task has the chance of not having any asynchronous behavior (as is the case for caching). In the previous case, the first `print` would happen immediately and the `save_result_to_db` would wait until the event loop gave us time on the schedule.

So, very nonintuitively, the first request gets processed after all tasks have been created and in between the `response_size = ...` line and the code before it. We can verify this by having a `print` statement at the end of our `for` loop but before the end of the `TaskGroup` context, and another one within our `process` function.

We can go further and understand better how the asynchronous requests are processed by looking at the chronology of the HTTP requests made while running this program in [Figure 9-3](#).

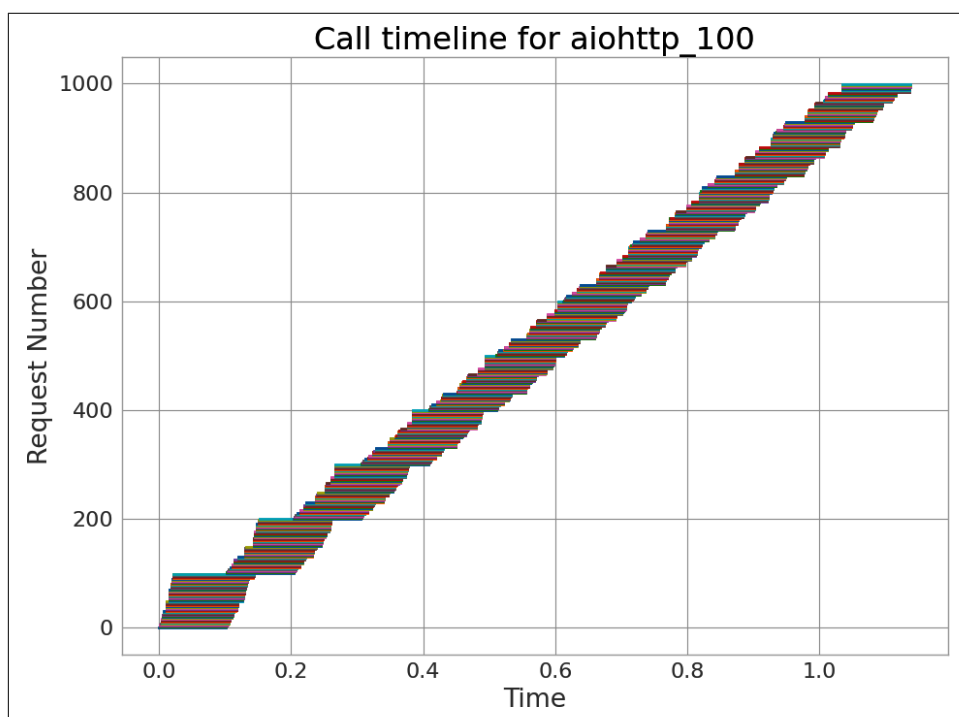


Figure 9-3. Chronology of HTTP requests for [Example 9-5](#)

The first thing to notice is the change in runtime. While the serial version of the crawler in [Example 9-4](#) took 101.89 seconds to run, this async version took only 1.33 seconds, a 76.6 times speedup. Looking at the call graph we can understand that this is happening because we are processing multiple requests at the same time. So instead of waiting the full 100 milliseconds for a request to complete before doing another one, we are stacking them in order to reduce the overall effect.

Looking deeper at the call graph, we can see that the system seems to have issued the first 100 requests pretty steadily before taking a small pause to issue new requests. In fact, we can see that the next set of requests only started once the first 100 had completed. This comes from a configuration within the `aiohttp.ClientSession` object where only 100 simultaneous requests will be active at once. When more come in, they will be queued until one of the 100 slots opens up. It may seem that the effect of this washes away later when more requests are issued, but as we'll see in [Figure 9-4](#), the overall effects persist.

With this better understanding, we can predict the behavior of this async approach. If 100 requests can be processed at once, this means each block of 100 requests feels the 100-millisecond delay all at once. As such, we would expect 1,000 requests to take ~1 second, which is what we see!

What if we tune this 100 request limit? Logic says we should be able to get our time down to 0.1 seconds if we run all requests at the same time! However, this is when we start hitting up against the limitations of our event loop, the network device, the upstream server, or one of the hundred other complex things involved in high-level I/O.

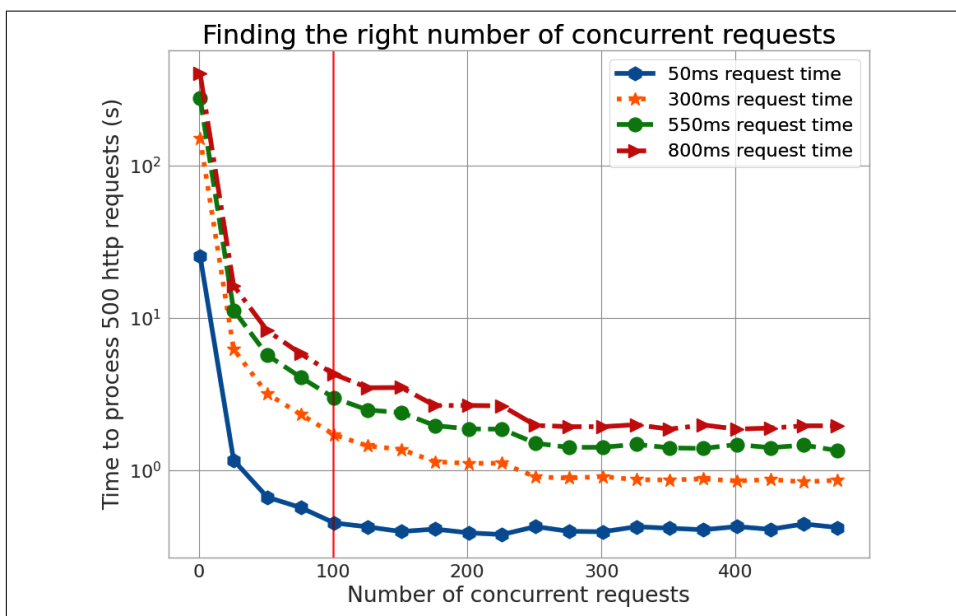


Figure 9-4. Experimenting with different numbers of concurrent requests for various request times

We can experiment with this and look at the total runtime for our task as we change the number of concurrent requests and also the response time from the server. We can see several interesting things going on. First, increasing the number of concurrent requests gives us diminishing returns and seems to taper off in benefit after about 250 concurrent requests. We also see that this behavior seems to be different dependent on the request timing, with short requests seeing their highest benefit from concurrency much sooner.

Both of these things come from the fact that with these higher rates of concurrent requests, we always have a completed request available for processing when we launch a new request. So once the first request completes, we always have more data to process and we become completely CPU-bound. However, we aren't simply doing an optimized CPU operation where we are looping through well-structured and well-laid-out data. Instead, we are context switching between each coroutine and invoking all of the overhead associated with this.

As a result, it's best either to stick with a reasonable default of the number of concurrent tasks you perform (with modern computers, 100–200 is generally a good bet) or to perform tests on your application to see what nuances there are. Remember, there are even more potential factors associated with the performance characteristics of the downstream service you are making requests to!

In many domains of high performance computing, especially those discussed in this book, the goal is to simplify your application or the code or the algorithm. You are trying to reduce the complexity in order to reduce the amount of time spent doing calculations. On the other hand, with asynchronous code we are introducing *more* complexity with the goal of using it to carefully manage and orchestrate the strenuous needs of heavy I/O. This especially comes from the fact that most of the overhead from I/O resides outside the control of your code! Because of this, experimentation for your particular application is key in order to understand how the tools provided to you by the `asyncio` module can best be deployed for your particular situation.

Shared CPU–I/O Workload

To make the preceding examples more concrete, we will create another toy problem in which we have a CPU-bound problem that needs to communicate frequently with a database to save results. The CPU workload can be anything; in this case, we are taking the `bcrypt` hash of a random string with larger and larger workload factors to increase the amount of CPU-bound work (see [Table 9-1](#) to understand how the “difficulty” parameter affects runtime).

This problem is representative of any sort of problem in which your program has heavy calculations to do, and the results of those calculations must be stored into a database, potentially incurring a heavy I/O penalty. The only restrictions we are putting on our database are as follows:

- It has an HTTP API, so we can use code like that in the earlier examples.¹
- Response times are of the order of 100 milliseconds.
- The database can satisfy many requests at a time.²

The response time of this “database” was chosen to be higher than usual in order to exaggerate the turning point in the problem, where the time to do one of the CPU tasks is longer than one of the I/O tasks. For a database that is being used only to store simple values, a response time greater than 10 milliseconds should be considered slow!

Table 9-1. Time to calculate a single hash

Difficulty parameter	8	10	12	14
Milliseconds per iteration	17	67.9	271	1,090

Serial CPU Workload

We start in [Example 9-6](#) with some simple code that calculates the bcrypt hash of a string and makes a request to the database’s HTTP API every time a result is calculated.

Example 9-6. Serial CPU-bound workload

```
import random
import string

import bcrypt
import requests

def do_task(difficulty):
    """
    Hash a random 10 character string using bcrypt with a specified difficulty
    rating.
    """
    passwd = ("".join(random.sample(string.ascii_lowercase, 10))) ❶
    .encode("utf8")
```

¹ This is not necessary; it just serves to simplify our code.

² This is true for all distributed databases and for other popular databases such as Postgres, MongoDB, and so on.

```

salt = bcrypt.gensalt(difficulty) ❷
result = bcrypt.hashpw(passwd, salt)
return result.decode("utf8")

def save_result_serial(result):
    url = f"http://127.0.0.1:8080/add"
    response = requests.post(url, data=result)
    return response.json()

def calculate_task_serial(num_iter, task_difficulty):
    for i in range(num_iter):
        result = do_task(task_difficulty)
        save_number_serial(result)

```

- ❶ We generate a random 10-character byte array.
- ❷ The `difficulty` parameter sets how hard it is to generate the password by increasing the CPU and memory requirements of the hashing algorithm.

Just as in our serial example ([Example 9-4](#)), the request times for each database save (100 milliseconds) do not stack, and we must pay this penalty for each result. As a result, iterating six hundred times with a task difficulty of 8 takes 70.6 seconds. We know, however, that because of the way our serial requests work, we are spending 60 seconds at minimum doing I/O! 85% of our program's runtime is being spent doing I/O and, moreover, just sitting around in “I/O wait,” when it could be doing something else!

Of course, as the CPU problem takes more and more time, the relative slowdown of doing this serial I/O decreases. This is simply because the cost of having a 100-millisecond pause after each task pales in comparison to the long amount of time needed to do this computation (as we can see in [Figure 9-5](#)). This fact highlights how important it is to understand your workload before considering which optimizations to make. If you have a CPU task that takes hours and an I/O task that takes only seconds, work done to speed up the I/O task will not bring the huge speedups you may be looking for!

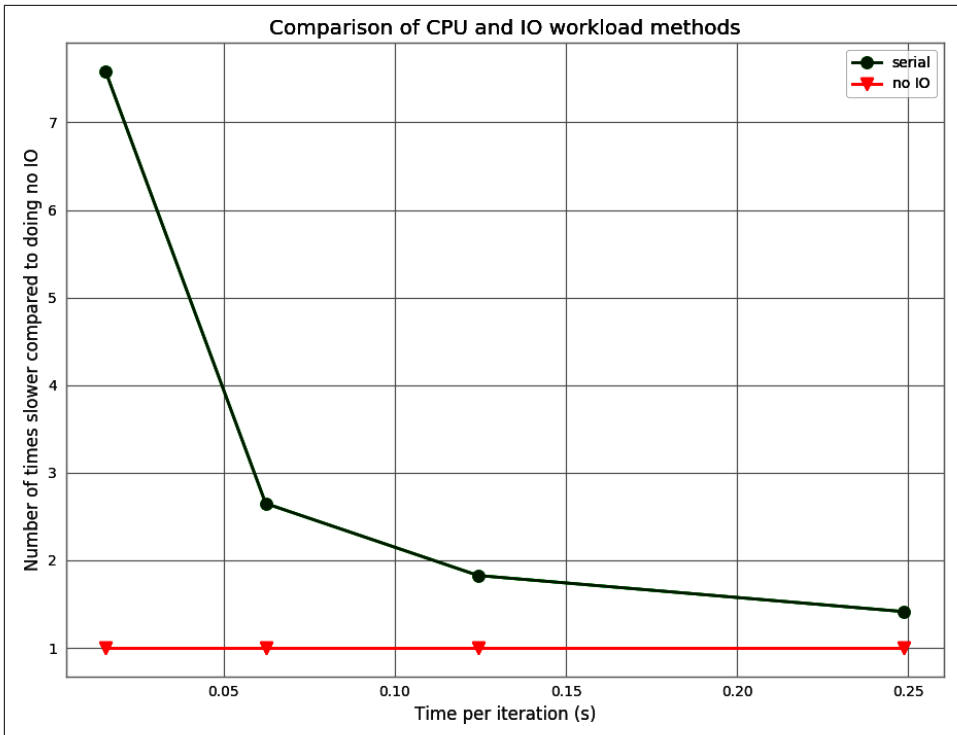


Figure 9-5. Comparison of the serial code to the CPU task with no I/O

Batched CPU Workload

Instead of immediately going to a full asynchronous solution, let's try an intermediate solution. If we don't need to know the results in our database right away, we can batch up the results and send them to the database asynchronously. To do this, in [Example 9-7](#) we create an object, `AsyncBatcher`, that will take care of queuing results to be sent to the database in small asynchronous bursts. This will still pause the program and put it into I/O wait with no CPU tasks; however, during this time we can issue many concurrent requests instead of issuing them one at a time.

Example 9-7. Batched async CPU-bound workload

```
import asyncio
import aiohttp

class AsyncBatcher(object):
    def __init__(self, batch_size):
        self.batch_size = batch_size
        self.batch = []
        self.client_session = None
```

```

self.url = f"http://127.0.0.1:8080/add"

def __enter__(self):
    return self

def __exit__(self, *args, **kwargs):
    self.flush()

def save(self, result):
    self.batch.append(result)
    if len(self.batch) == self.batch_size:
        self.flush()

def flush(self):
    """
    Synchronous flush function which starts an eventloop for the purposes of
    running our async flushing function
    """
    asyncio.run(self.__aflush()) ❶

async def __aflush(self): ❷
    async with aiohttp.ClientSession() as session:
        tasks = [self.fetch(result, session) for result in self.batch]
        asyncio.gather(*tasks) ❸
    self.batch.clear()

async def fetch(self, result, session):
    async with session.post(self.url, data=result) as response:
        return await response.json()

```

- ❶ We are able to start up an event loop just to run a single asynchronous function. The event loop will run until the asynchronous function is complete, and then the code will resume as normal.
- ❷ This function is nearly identical to that of [Example 9-5](#).
- ❸ Here we use `asyncio.gather` rather than `asyncio.TaskGroup` to provide an alternative to running a set of tasks. Both have their use case, with `asyncio.TaskGroup` being more useful for fine-grained exception handling. In addition, `asyncio.gather` doesn't cancel pending tasks when one of them raises an exception, which is useful in cases when each task is independent and one failure shouldn't stop other tasks from completing.

Now we can proceed almost in the same way as we did before. The main difference is that we add our results to our `AsyncBatcher` and let it take care of when to send the requests. Note that we chose to make this object into a context manager so that once we are done batching, the final `flush()` gets called. If we didn't do this, there would be a chance that we still have some results queued that didn't trigger a flush:

```
def calculate_task_batch(num_iter, task_difficulty):
    with AsyncBatcher(100) as batcher: ❶
        for i in range(num_iter):
            result = do_task(i, task_difficulty)
            batcher.save(result)
```

- ❶ We choose to batch at 100 requests, for reasons similar to those illustrated in [Figure 9-4](#).

With this change, we are able to bring our runtime for a difficulty of 8 down to 10.21 seconds. This represents a 6.95 times speedup without our having to do much work or changing the general structure of the larger project this code may be a part of. In a constrained environment such as a real-time data pipeline, this extra speed could mean the difference between a system being able to keep up with demand and that system falling behind (in which case a queue will be required; you'll learn about queues in [Chapter 11](#)).

To understand what is happening in this timing, let's consider the variables that could affect the timings of this batched method. If our database had infinite throughput (i.e., if we could send an infinite number of requests at the same time without penalty), we could take advantage of the fact that we get only the 100-millisecond penalty when our `AsyncBatcher` is full and does a flush. In this case, we'd get the best performance by just saving all of our requests to the database and doing them all at once when the calculation was finished.

However, in the real world, our databases have a maximum throughput that limits the number of concurrent requests they can process. In this case, our server is limited at 100 requests a second, which means we must flush our batcher every one hundred results and take the 100-millisecond penalty then. This is because the batcher still pauses the execution of the program, just as the serial code did; however, in that paused time it performs many requests instead of just one.

If we tried to save all our results to the end and then issued them all at once, the server would *process* only one hundred at a time, and we'd have an extra penalty in terms of the overhead to making all those requests at the same time, in addition to overloading our database, which can cause all sorts of unpredictable slowdowns.

On the other hand, if our server had terrible throughput and could deal with only one request at a time, we may as well run our code in serial! Even if we kept our batching at one hundred results per batch, when we actually go to make the requests, only one would get responded to at a time, effectively invalidating any batching we made.

This mechanism of batching results, also known as *pipelining*, can help tremendously when trying to lower the burden of an I/O task (as seen in [Figure 9-6](#)). It offers a good compromise between the speeds of asynchronous I/O and the ease of writing

serial programs. However, a determination of how much to pipeline at a time is very case-dependent and requires some profiling and tuning to get the best performance.

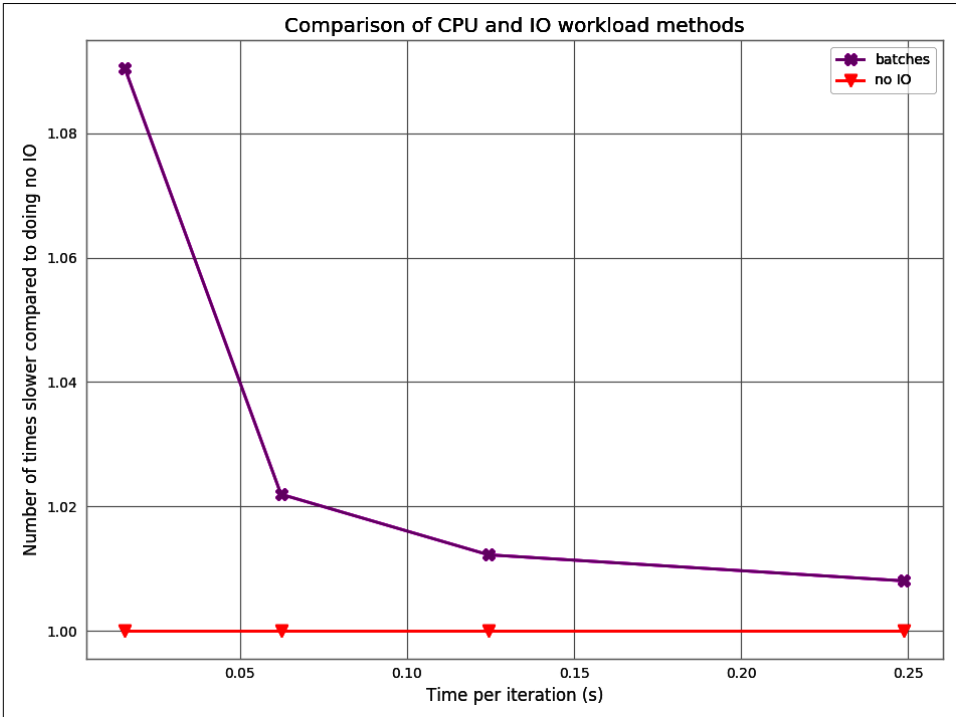


Figure 9-6. Comparison of batching requests versus not doing any I/O

Fully Asynchronous CPU Workload

In some cases, we may need to implement a full asynchronous solution. This may happen if the CPU task is part of a larger I/O-bound program, such as an HTTP server. Imagine that you have an API service that, in response to some of its endpoints, has to perform heavy computational tasks. We still want the API to be able to handle concurrent requests and be performant in its tasks, but we also want the CPU task to run quickly.

The implementation of this solution in [Example 9-8](#) uses code very similar to that of [Example 9-5](#).

Example 9-8. Async CPU workload

```
def save_result_aiohttp(session, url):
    async with session.post(url, data=result) as response:
        return await response.json()
```

```

async def calculate_task_aiohttp(num_iter, task_difficulty):
    async with asyncio.TaskGroup() as tg:
        async with aiohttp.ClientSession() as client_session:
            for i in range(num_iter):
                result = do_task(i, task_difficulty)
                task = save_result_aiohttp(session, result) ❶
                tg.create_task(task)
                await asyncio.sleep(0) ❷

```

- ❶ Instead of awaiting our database save immediately, we create the Future and add it to a task group for it to run when there is time. The `asyncio.TaskGroup` is a convenient way of creating many tasks and ensuring that they will all be awaited when the context manager exits.
- ❷ This is arguably the most important line in the function. Here, we pause the main function to allow the event loop to take care of any pending tasks. Without this, none of our queued tasks would run until the end of the function when the `TaskGroup` exits.

Before we go into the performance characteristics of this code, we should first talk about the importance of the `asyncio.sleep(0)` statement. It may seem strange to be sleeping for zero seconds, but this statement is a way to force the function to defer execution to the event loop and allow other tasks to run. In general in asynchronous code, this deferring happens every time an `await` statement is run. Since we generally don't `await` in CPU-bound code, it's important to force this deferment, or else no other task will run until the CPU-bound code is complete. In this case, if we didn't have the sleep statement, all the HTTP requests would be paused until the `asyncio.wait` statement, and then all the requests would be issued at once, which is definitely not what we want!



We didn't need the `asyncio.sleep(0)` statement in [Example 9-5](#) because the process of creating all of our tasks was very quick. In this case, however, creating each task requires running through our complicated bcrypt calculation, so giving the event loop a chance to give priority to other tasks becomes quite important.

One nice thing about having this control is that we can choose the best times to defer back to the event loop. There are many considerations when doing this. Since the run state of the program changes when we defer, we don't want to do it in the middle of a calculation and potentially change our CPU cache. In addition, deferring to the event loop has an overhead cost, so we don't want to do it too frequently. However, while we are bound up doing the CPU task, we cannot do any I/O tasks. So if our full application is an API, no requests can be handled during the CPU time!

Our general rule of thumb is to try to issue an `asyncio.sleep(0)` at any loop that we expect to iterate every 50 to 100 milliseconds or so. Some applications use `time.perf_counter` and allow the CPU task to have a specific amount of runtime before forcing a sleep. For a situation such as this, though, since we have control of the number of CPU and I/O tasks, we just need to make sure that the time between sleeps coincides with the time needed for pending I/O tasks to complete.

One major performance benefit to the full asynchronous solution is that we can perform all of our I/O *while* we are doing our CPU work, effectively hiding it from our total runtime (as we can see from the overlapping lines in [Figure 9-7](#)). While it will never be completely hidden because of the overhead costs of the event loop, we can get very close. In fact, for a difficulty of 8 with 600 iterations, our code runs 7.3× faster than the serial code and performs its total I/O workload 2× faster than the batched code (and this benefit over the batched code would only get better as we do more iterations, since the batched code loses time versus the asynchronous code every time it has to pause the CPU task to flush a batch).

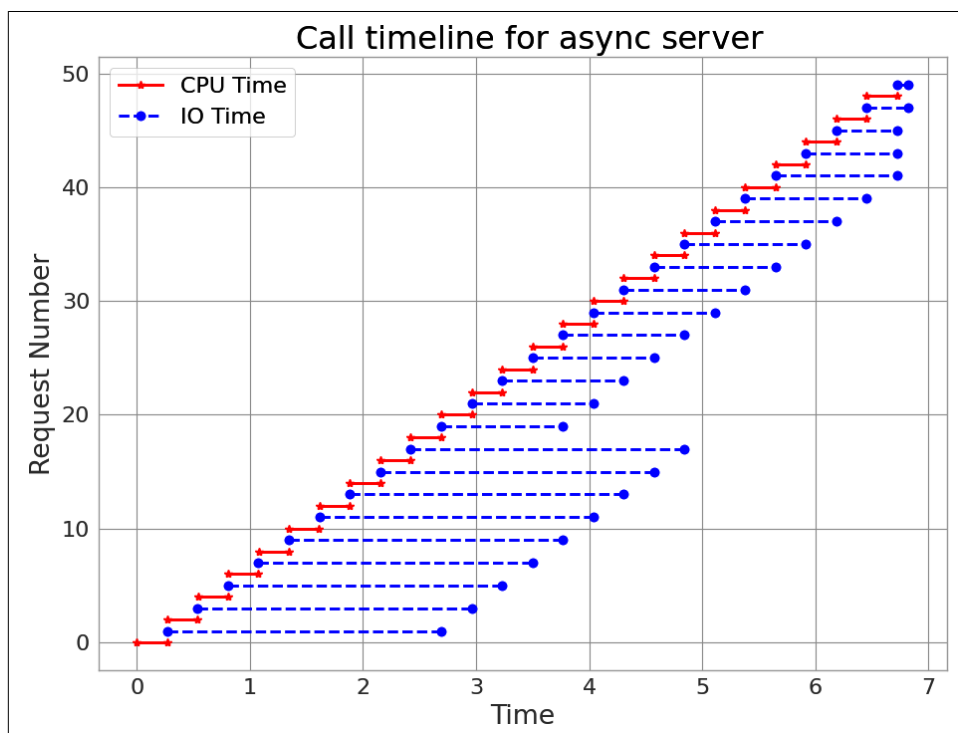


Figure 9-7. Call graph for 25 difficulty-8 CPU tasks using the `aiohttp` solution—the solid lines represent time working on a CPU task, while dashed lines represent time sending a result to the server

In the call timeline, we can really see what is going on. What we've done is mark the beginning and end of each CPU and I/O task for a short run of 25 CPU tasks with a difficulty of 8. The first several I/O tasks are the slowest, taking a while to make the initial connection to our server. Because of our use of `aiohttp`'s `ClientSession`, these connections are cached, and all subsequent connections to the same server are much faster.

After this, if we just focus on the dashed lines, they seem to happen very regularly without much of a pause between CPU tasks. Indeed, we don't see the 100-millisecond delay from the HTTP request between tasks. Instead, we see the HTTP request being issued quickly at the end of each CPU task and later being marked as completed at the end of another CPU task.

We do see, though, that each individual I/O task takes longer than the 100-millisecond response time from the server. This longer wait time is given by the frequency of our `asyncio.sleep(0)` statements (since each CPU task has one `await`, while each I/O task has three) and the way the event loop decides which tasks come next. For the I/O task, this extra wait time is OK because it doesn't interrupt the CPU task at hand. In fact, at the end of the run we can see the I/O runtimes shorten until the final I/O task is run. This final dashed line is triggered by the `asyncio.wait` statement and runs incredibly quickly since it is the only remaining task and never needs to switch to other tasks.

In Figures 9-8 and 9-9, we can see a summary of how these changes affect the runtime of our code for different workloads. The speedup in the async code over the serial code is significant, although we are still a ways away from the speeds achieved in the raw CPU problem. For this to be completely remedied, we would need to use modules like `multiprocessing` to have a completely separate process that can deal with the I/O burden of our program without slowing down the CPU portion of the problem.

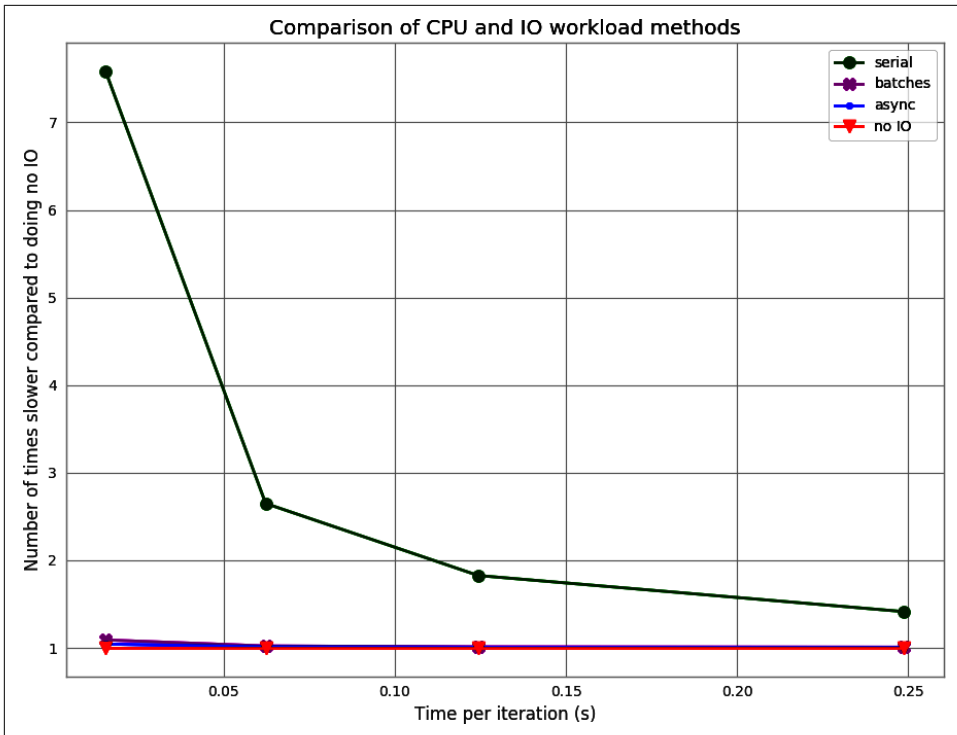


Figure 9-8. Processing time differences between serial I/O, batched async I/O, full async I/O, and a control case in which I/O is completely disabled

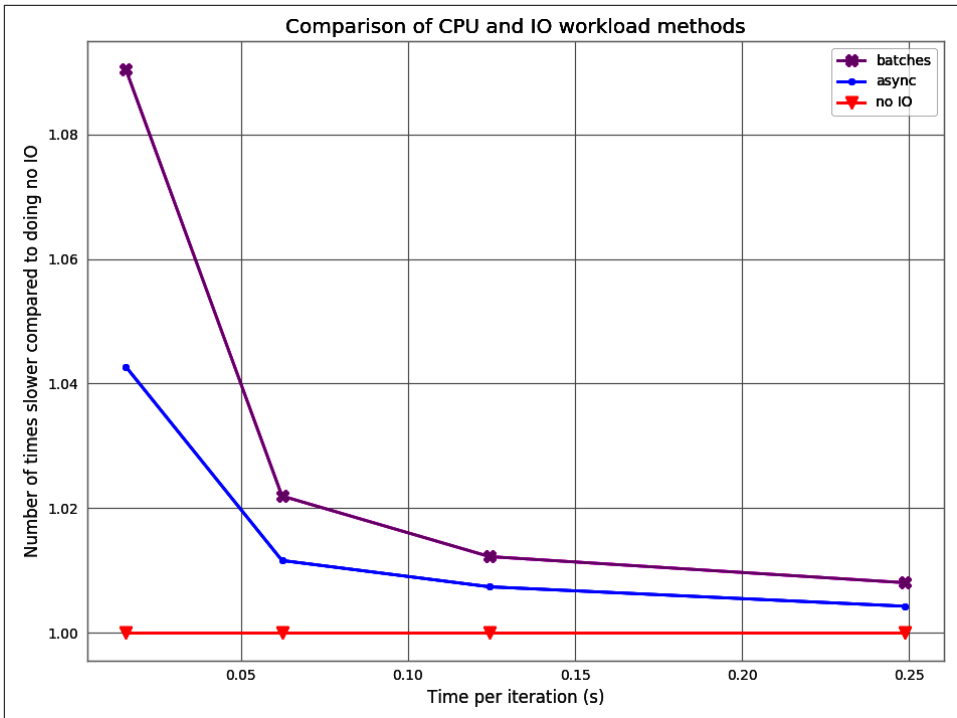


Figure 9-9. Processing time differences between batched async, full async I/O, and I/O disabled

Wrap-Up

When solving problems in real-world and production systems, it is often necessary to communicate with an outside source. This outside source could be a database running on another server, another worker computer, or a data service that is providing the raw data that must be processed. Whenever this is the case, your problem can quickly become I/O-bound, meaning that most of the runtime is dominated by dealing with input/output.

Concurrency helps with I/O-bound problems by allowing you to interleave computation with potentially multiple I/O operations. This allows you to exploit the fundamental difference between I/O and CPU operations in order to speed up overall runtime.

In this chapter we investigated the runtime dynamics of asynchronous programs and looked at how to better understand performance characteristics as we push these processes to their limits.

We also saw how to mesh CPU and I/O tasks together and how to consider the various performance characteristics of each to come up with a good solution to the problem. While it may be appealing to go to a full asynchronous code immediately, sometimes intermediate solutions work almost as well without having quite the engineering burden.

In the next chapter, we will take this concept of interleaving computation from I/O-bound problems and apply it to CPU-bound problems. With this new ability, we will be able to perform not only multiple I/O operations at once but also many computational operations. This capability will allow us to start to make fully scalable programs where we can achieve more speed by simply adding more computer resources that can each handle a chunk of the problem.

The multiprocessing Module

Questions You'll Be Able to Answer After This Chapter

- What does the `multiprocessing` module offer?
- What's the difference between processes and threads?
- How do I choose the right size for a process pool?
- How do I use nonpersistent queues for work processing?
- What are the costs and benefits of interprocess communication?
- How can I process `numpy` data with many CPUs?
- How would I use `Joblib` to simplify parallelized and cached scientific work?
- Why do I need locking to avoid data loss?

CPython doesn't use multiple CPUs by default. This is partly because Python was designed back in a single-core era, and partly because parallelizing can actually be quite difficult to do efficiently. Python gives us the tools to do it but leaves us to make our own choices. It is painful to see your multicore machine using just one CPU on a long-running process, though, so in this chapter we'll review ways of using all the machine's cores at once.



We just mentioned *CPython*—the common implementation that we all use. Nothing in the Python language stops it from using multicore systems. CPython's implementation cannot efficiently use multiple cores, but future implementations may not be bound by this restriction.

We live in a multicore world—six cores are common in laptops, and ninety-six core desktop configurations are available. If your job can be split to run on multiple CPUs *without* too much engineering effort, this is a wise direction to consider.

When Python is used to parallelize a problem over a set of CPUs, you can expect *up to* an n -times (n times) speedup with n cores. If you have a quad-core machine and you can use all four cores for your task, it might run in a quarter of the original runtime. You are unlikely to see a greater than four times speedup; in practice, you'll probably see gains of three to four times.

Each additional process will increase the communication overhead and decrease the available RAM, so you rarely get a full n -times speedup. Depending on which problem you are solving, the communication overhead can even get so large that you can see very significant slowdowns. These sorts of problems are often where the complexity lies for any sort of parallel programming and normally require a change in algorithm. This is why parallel programming is often considered an art.

If you're not familiar with [Amdahl's law](#), it is worth doing some background reading. The law shows that if only a small part of your code can be parallelized, it doesn't matter how many CPUs you throw at it; it still won't run much faster overall. Even if a large fraction of your runtime could be parallelized, there's a finite number of CPUs that can be used efficiently to make the overall process run faster before you get to a point of diminishing returns.

The `multiprocessing` module lets you use process- and thread-based parallel processing, share work over queues, and share data among processes. It is mostly focused on single-machine multicore parallelism (there are better options for multimachine parallelism). A very common use is to parallelize a task over a set of processes for a CPU-bound problem. You might also use OpenMP to parallelize an I/O-bound problem, but as we saw in [Chapter 9](#), there are better tools for this (e.g., the new `asyncio` module in Python 3).



OpenMP is a low-level interface to multiple cores—you might wonder whether to focus on it rather than `multiprocessing`. We introduced it with Cython back in [Chapter 8](#), but we don't cover it in this chapter. `multiprocessing` works at a higher level, sharing Python data structures, while OpenMP works with C primitive objects (e.g., integers and floats) once you've compiled to C. Using OpenMP makes sense only if you're compiling your code; if you're not compiling (e.g., if you're using efficient `numpy` code and you want to run on many cores), then sticking with `multiprocessing` is probably the right approach.

To parallelize your task, you have to think a little differently from the normal way of writing a serial process. You must also accept that debugging a parallelized task is

harder—often, it can be very frustrating. We’d recommend keeping the parallelism as simple as possible (even if you’re not squeezing every last drop of power from your machine) so that your development velocity is kept high.

One particularly difficult topic is the sharing of state in a parallel system—it feels like it should be easy, but it incurs lots of overhead and can be hard to get right. There are many use cases, each with different trade-offs, so there’s definitely no one solution for everyone. In “[Verifying Primes Using Interprocess Communication](#)” on page 324, we’ll go through state sharing with an eye on the synchronization costs. Avoiding shared state will make your life far easier.

We can make a guess about how well an algorithm will perform if we add parallelization based on how much state is being shared.

For example, if we can have multiple Python processes all solving the same problem without communicating with one another (a situation known as *embarrassingly parallel*), not much of a penalty will be incurred as we add more and more Python processes.

On the other hand, if each process needs to communicate with every other Python process, the communication overhead will slowly overwhelm the processing and slow things down. This means that as we add more and more Python processes, we can actually slow down our overall performance.

As a result, sometimes some counterintuitive algorithmic changes must be made to efficiently solve a problem in parallel. For example, when solving the diffusion equation ([Chapter 6](#)) in parallel, each process actually does some redundant work that another process also does. This redundancy reduces the amount of communication required and speeds up the overall calculation!

Here are some typical jobs for the `multiprocessing` module:

- Parallelize a CPU-bound task with `Process` or `Pool` objects
- Parallelize an I/O-bound task in a `Pool` with threads using the (oddly named) `dummy` module
- Share pickled work via a `Queue`
- Share state between parallelized workers, including bytes, primitive datatypes, dictionaries, and lists

If you come from a language where threads are used for CPU-bound tasks (e.g., C++ or Java), you should know that while threads in Python are OS-native (they’re not simulated—they are actual operating system threads), they are bound by the GIL, so only one thread may interact with Python objects at a time.

By using processes, we run a number of Python interpreters in parallel, each with a private memory space with its own GIL, and each runs in series (so there's no competition for each GIL). This is the easiest way to speed up a CPU-bound task in Python. If we need to share state, we need to add some communication overhead; we'll explore that in [“Verifying Primes Using Interprocess Communication” on page 324](#).

If you work with numpy arrays, you might wonder if you can create a larger array (e.g., a large 2D matrix) and ask processes to work on segments of the array in parallel. You can, but it is hard to discover how by trial and error, so in [“Sharing numpy Data with multiprocessing” on page 340](#) we'll work through sharing a 30.5 GB numpy array across eight CPUs. Rather than sending partial copies of the data (which would at least double the working size required in RAM and create a massive communication overhead), we share the underlying bytes of the array among the processes. This is an ideal approach to sharing a large array among local workers on one machine.

In this chapter we also introduce the [Joblib](#) library—this builds on the multiprocessing library and offers improved cross-platform compatibility, a simple API for parallelization, and convenient persistence of cached results. Joblib is designed for scientific use, and we urge you to check it out.



Here, we discuss multiprocessing on *nix-based machines (this chapter is written using Ubuntu; the code should run unchanged on a Mac). Since Python 3.4, the quirks that appeared on Windows have been dealt with. Joblib has stronger cross-platform support than multiprocessing, and we recommend you review it ahead of multiprocessing.

In this chapter we'll hardcode the number of processes (`NUM_PROCESSES=8`) to match the eight physical cores on Ian's laptop. By default, multiprocessing will use as many cores as it can see (the machine presents sixteen—eight CPUs and eight hyper-threads). Normally you would avoid hardcoding the number of processes to create unless you were specifically managing your resources.

An Overview of the multiprocessing Module

The multiprocessing module provides a low-level interface to process- and thread-based parallelism. Its main components are as follows:

Process

A forked copy of the current process; this creates a new process identifier, and the task runs as an independent child process in the operating system. You can start and query the state of the Process and provide it with a target method to run.

Pool

Wraps the `Process` or `threading.Thread` API into a convenient pool of workers that share a chunk of work and return an aggregated result.

Queue

A FIFO queue allowing multiple producers and consumers.

Pipe

A uni- or bidirectional communication channel between two processes.

Manager

A high-level managed interface to share Python objects between processes.

ctypes

Allows sharing of primitive datatypes (e.g., integers, floats, and bytes) between processes after they have forked.

Synchronization primitives

Locks and semaphores to synchronize control flow between processes.



In Python 3.2, the `concurrent.futures` module was introduced (via [PEP 3148](#)); this provides the core behavior of multiprocessing, with a simpler interface based on Java's `java.util.concurrent`. It is available as a [backport to earlier versions of Python](#). We expect multiprocessing to continue to be preferred for CPU-intensive work and won't be surprised if `concurrent.futures` becomes more popular for I/O-bound tasks.

In the rest of the chapter, we'll introduce a set of examples to demonstrate common ways of using the `multiprocessing` module.

We'll estimate pi using a Monte Carlo approach with a `Pool` of processes or threads, using normal Python and `numpy`. This is a simple problem with well-understood complexity, so it parallelizes easily; we can also see an unexpected result from using threads with `numpy`. Next, we'll search for primes using the same `Pool` approach; we'll investigate the nonpredictable complexity of searching for primes and look at how we can efficiently (and inefficiently!) split the workload to best use our computing resources. We'll finish the primes search by switching to queues, where we introduce `Process` objects in place of a `Pool` and use a list of work and poison pills to control the lifetime of workers.

Next, we'll tackle interprocess communication (IPC) to validate a small set of possible primes. By splitting each number's workload across multiple CPUs, we use IPC to end the search early if a factor is found so that we can significantly beat the speed of a

single-CPU search process. We'll cover shared Python objects, OS primitives, and a Redis server to investigate the complexity and capability trade-offs of each approach.

We can share a 25 GB numpy array across four CPUs to split a large workload *without* copying data. If you have large arrays with parallelizable operations, this technique should buy you a great speedup, since you have to allocate less space in RAM and copy less data. Finally, we'll look at synchronizing access to a file and a variable (as a `Value`) between processes without corrupting data to illustrate how to correctly lock shared state.



PyPy (discussed in [Chapter 8](#)) has full support for the multiprocessing library, and the following CPython examples (though not the numpy examples, at the time of this writing) all run far quicker using PyPy. If you're using only CPython code (no C extensions or more complex libraries) for parallel processing, PyPy might be a quick win for you.

Estimating Pi Using the Monte Carlo Method

We can estimate pi by throwing thousands of imaginary darts at a “dartboard” represented by a unit circle with a Monte Carlo method. The relationship between the number of darts falling inside the circle's edge and the number falling outside it will allow us to approximate pi.

This is an ideal first problem, as we can split the total workload evenly across a number of processes, each one running on a separate CPU. Each process will end at the same time since the workload for each is equal, so we can investigate the speedups available as we add new CPUs and hyperthreads to the problem.

With the Monte Carlo method, we use the [Pythagorean theorem](#) to test if a dart has landed inside our circle; on each iteration we generate a random x and y coordinate and test:

$$estimate = \sum_1^N (x^2 + y^2 \leq 1^2)$$

We then multiply by 4 as if we'd thrown darts into a full unit circle and divide by the number of iterations we've run. Note the 1^2 will always equal 1, so in code we can trivially simplify this to remove the square operation.

In [Figure 10-1](#), we throw $N=10,000$ darts into the unit square, and a percentage of them fall into the quarter of the unit circle that's drawn. This estimate is rather bad—10,000 dart throws does not reliably give us a three-decimal-place result. If you ran your own code, you'd see this estimate vary between 3.0 and 3.2 on each run.

To be confident of the first three decimal places, we need to generate 10,000,000 random dart throws.¹ This is massively inefficient (and better methods for pi's estimation exist), but it is rather convenient to demonstrate the benefits of parallelization using multiprocessing.

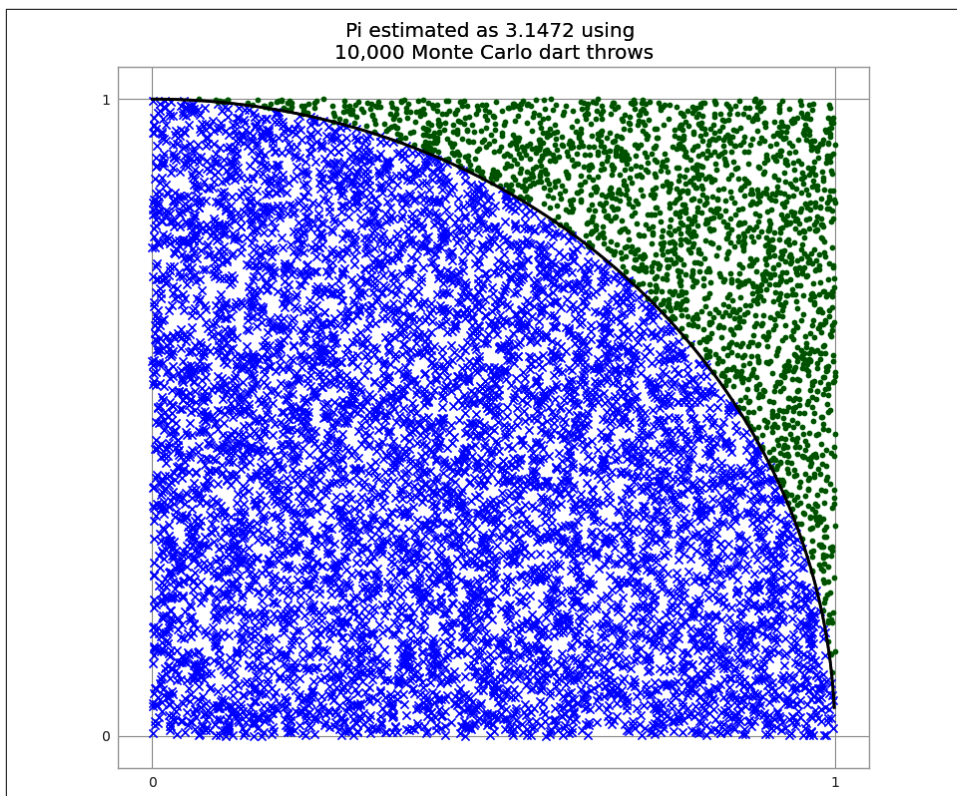


Figure 10-1. Estimating pi using the Monte Carlo method

We'll look at a loop version of this in [Example 10-1](#). We'll implement both a normal Python version and, later, a numpy version, and we'll use both threads and processes to parallelize the problem.

¹ See [Brett Foster's PowerPoint presentation](#) on using the Monte Carlo method to estimate pi.

Estimating Pi Using Processes and Threads

It is easier to understand a normal Python implementation, so we'll start with that in this section, using float objects in a loop. We'll parallelize this using processes to use all of our available CPUs, and we'll visualize the state of the machine as we use more CPUs.

Using Python Objects

The Python implementation is easy to follow, but it carries an overhead, as each Python float object has to be managed, referenced, and synchronized in turn. This overhead slows down our runtime, but it has bought us thinking time, as the implementation was quick to put together. By parallelizing this version, we get additional speedups for very little extra work.

Figure 10-2 shows three implementations of the Python example:

- No use of multiprocessing (named “Serial”)—one for loop in the main process
- Using threads
- Using processes

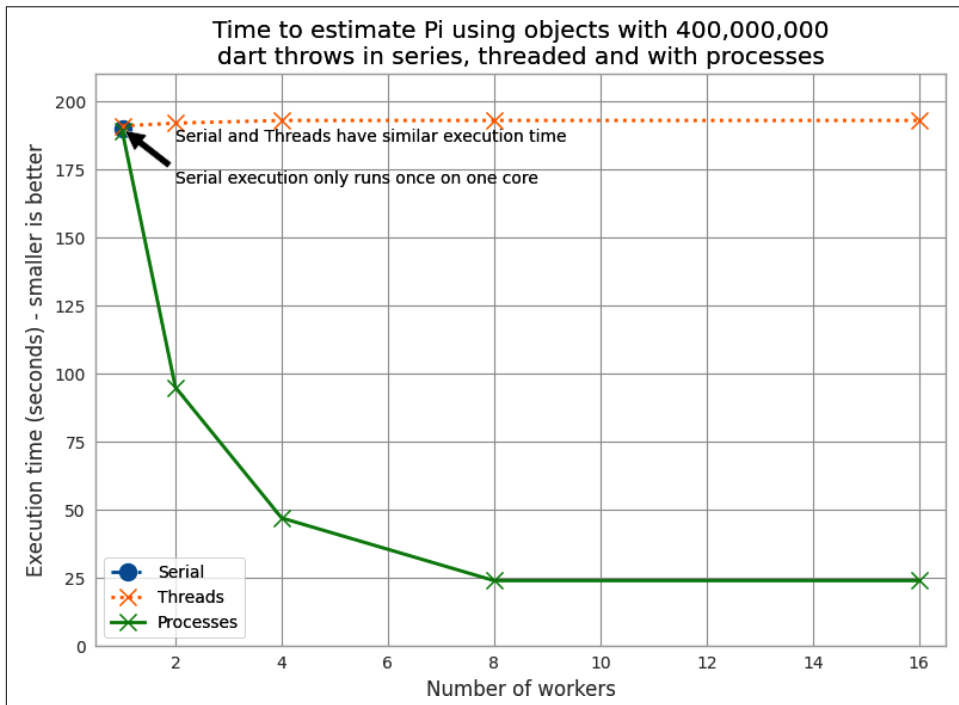


Figure 10-2. Working in series, with threads, and with processes

When we use more than one thread or process, we're asking Python to calculate the same total number of dart throws and to divide the work evenly between workers. If we want 400,000,000 dart throws in total using our Python implementation and we use two workers, we'll be asking both threads or both processes to generate 200,000,000 dart throws per worker.

Using one thread takes approximately 182 seconds, with no speedup when using more threads. Notably, with two or more threads there's a consistent, albeit tiny, slow-down relative to using no threads.

By using two or more processes, we make the runtime *shorter*. The cost of using no processes or threads (the series implementation) is the same as running with one process.

By using processes, we get a linear speedup when using two to eight cores on Ian's laptop. For the sixteen-worker case, we're using Intel's Hyper-Threading Technology—the laptop has eight physical cores, so we get barely any change in speedup by running sixteen processes.

Example 10-1 shows the Python version of our pi estimator. If we're using threads, each instruction is bound by the GIL, so although each thread could run on a separate CPU, it will execute only when no other threads are running. The process version is not bound by this restriction, as each forked process has a private Python interpreter running as a single thread—there's no GIL contention, as no objects are shared. We use Python's built-in random number generator, but see [“Random Numbers in Parallel Systems” on page 308](#) for some notes about the dangers of parallelized random number sequences.

Example 10-1. Estimating pi using a loop in Python

```
def estimate_nbr_points_in_quarter_circle(nbr_estimates):
    """Monte Carlo estimate of the number of points in a
       quarter circle using pure Python"""
    print(f"Executing estimate_nbr_points_in_quarter_circle \
          with {nbr_estimates:,} on pid {os.getpid()}")
    nbr_trials_in_quarter_unit_circle = 0
    for step in range(int(nbr_estimates)):
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        is_in_unit_circle = x * x + y * y <= 1.0
        nbr_trials_in_quarter_unit_circle += is_in_unit_circle

    return nbr_trials_in_quarter_unit_circle
```

Example 10-2 shows the `__main__` block. Note that we build the `Pool` before we start the timer. Spawning threads is relatively instant; spawning processes involves a

fork, and this takes a measurable fraction of a second. We ignore this overhead in [Figure 10-2](#), as this cost will be a tiny fraction of the overall execution time.

Example 10-2. `__main__` for estimating pi using a loop

```
from multiprocessing import Pool
...

if __name__ == "__main__":
    nbr_samples_in_total = 4e8
    nbr_parallel_blocks = 4
    pool = Pool(processes=nbr_parallel_blocks)
    nbr_samples_per_worker = nbr_samples_in_total / nbr_parallel_blocks
    print("Making {:,} samples per {} worker".format(nbr_samples_per_worker,
                                                    nbr_parallel_blocks))
    nbr_trials_per_process = [nbr_samples_per_worker] * nbr_parallel_blocks
    t1 = time.time()
    nbr_in_quarter_unit_circles = pool.map(estimate_nbr_points_in_quarter_circle,
                                           nbr_trials_per_process)
    pi_estimate = sum(nbr_in_quarter_unit_circles) * 4 / float(nbr_samples_in_total)
    print("Estimated pi", pi_estimate)
    print("Delta:", time.time() - t1)
```

We create a list containing `nbr_samples_in_total` divided by the number of workers. This new argument will be sent to each worker. After execution, we'll receive the same number of results back; we'll sum these to estimate the number of darts in the unit circle.

We import the process-based `Pool` from `multiprocessing`. We also could have used `from multiprocessing.dummy import Pool` to get a threaded version. The “dummy” name is rather misleading (we confess to not understanding why it is named this way); it is simply a light wrapper around the `threading` module to present the same interface as the process-based `Pool`.



Each process we create consumes some RAM from the system. You can expect a forked process using the standard libraries to take on the order of 10–20 MB of RAM; if you're using many libraries and lots of data, you might expect each forked copy to take hundreds of megabytes. On a system with a RAM constraint, this might be a significant issue—if you run out of RAM and the system reverts to using the disk's swap space, any parallelization advantage will be massively lost to the slow paging of RAM back and forth to disk!

The following figures plot the average CPU utilization of Ian's laptop's eight physical cores and their eight associated hyperthreads (each hyperthread runs on unutilized silicon in a physical core). The data gathered for these figures *includes* the startup

time of the first Python process and the cost of starting subprocesses. The CPU sampler records the entire state of the laptop, not just the CPU time used by this task.

Note that the following diagrams are created using a different timing method with a slower sampling rate than [Figure 10-2](#), so the overall runtime is a little longer.

The execution behavior in [Figure 10-3](#) with one process in the Pool (along with the parent process) shows some overhead in the first seconds as the Pool is created, and then a consistent close-to-100% CPU utilization throughout the run. With one process, we're efficiently using one core.

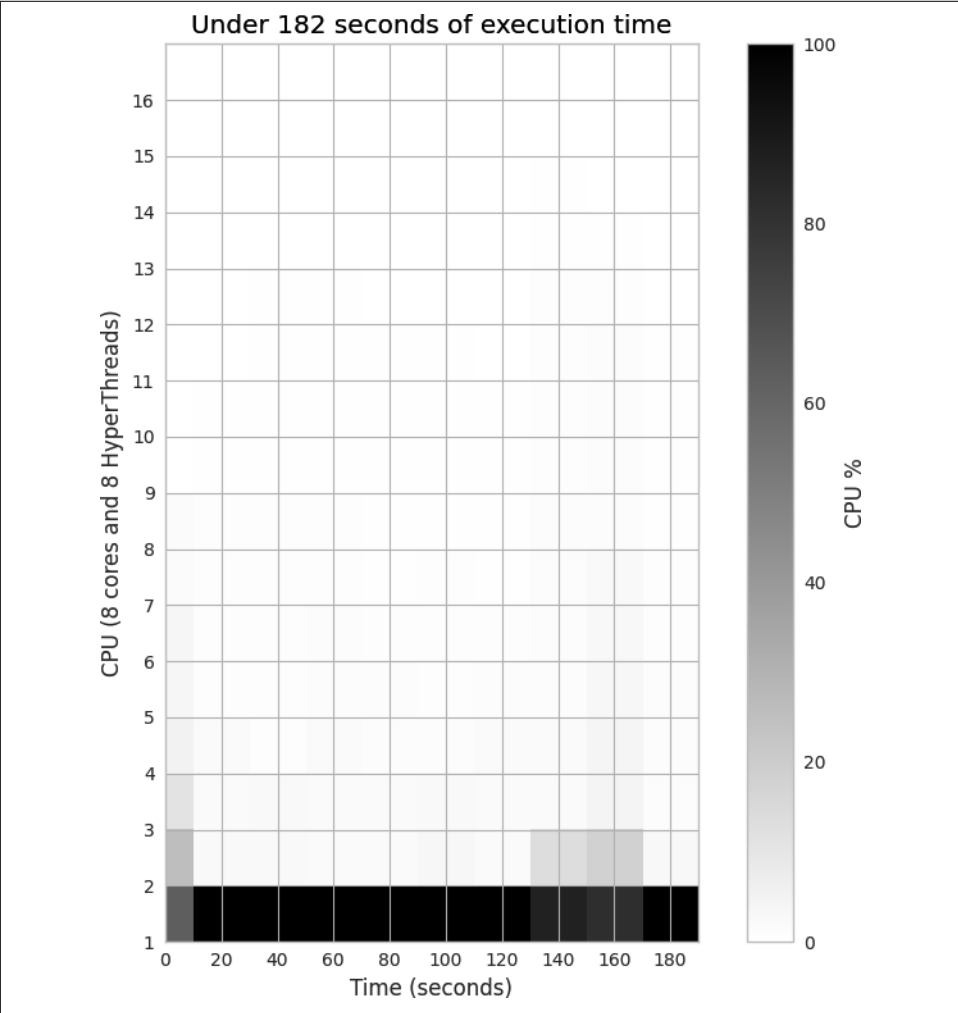


Figure 10-3. Estimating pi using Python objects and one process

Next we'll add a second process, effectively saying `Pool(processes=2)`. As you can see in [Figure 10-4](#), adding a second process roughly halves the execution time to 92 seconds, and two CPUs are fully occupied. This is the best result we can expect—we've efficiently used all the new computing resources, and we're not losing any speed to other overheads like communication, paging to disk, or contention with competing processes that want to use the same CPUs.

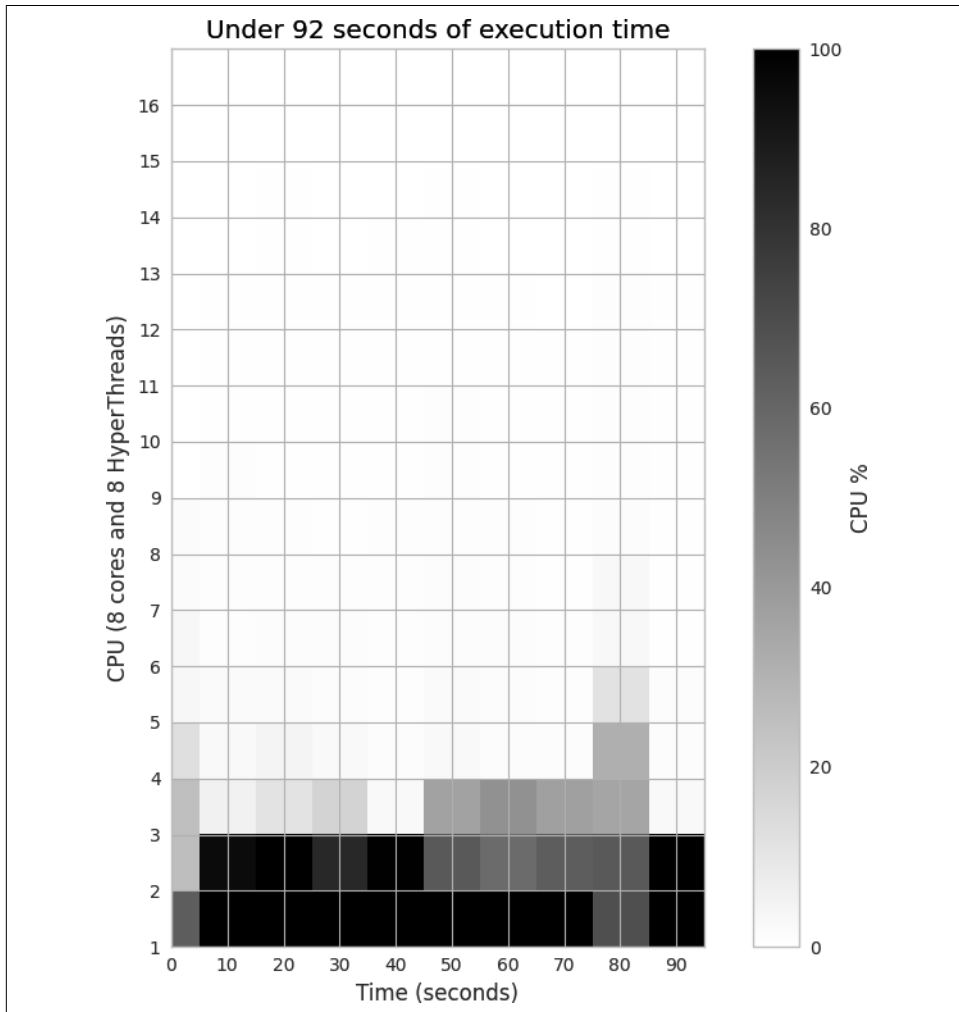


Figure 10-4. Estimating pi using Python objects and two processes

Figure 10-5 shows the results when using four physical CPUs—now we are using half of the raw power of this laptop. Execution time is 46 seconds, or roughly a quarter that of the single-process version.

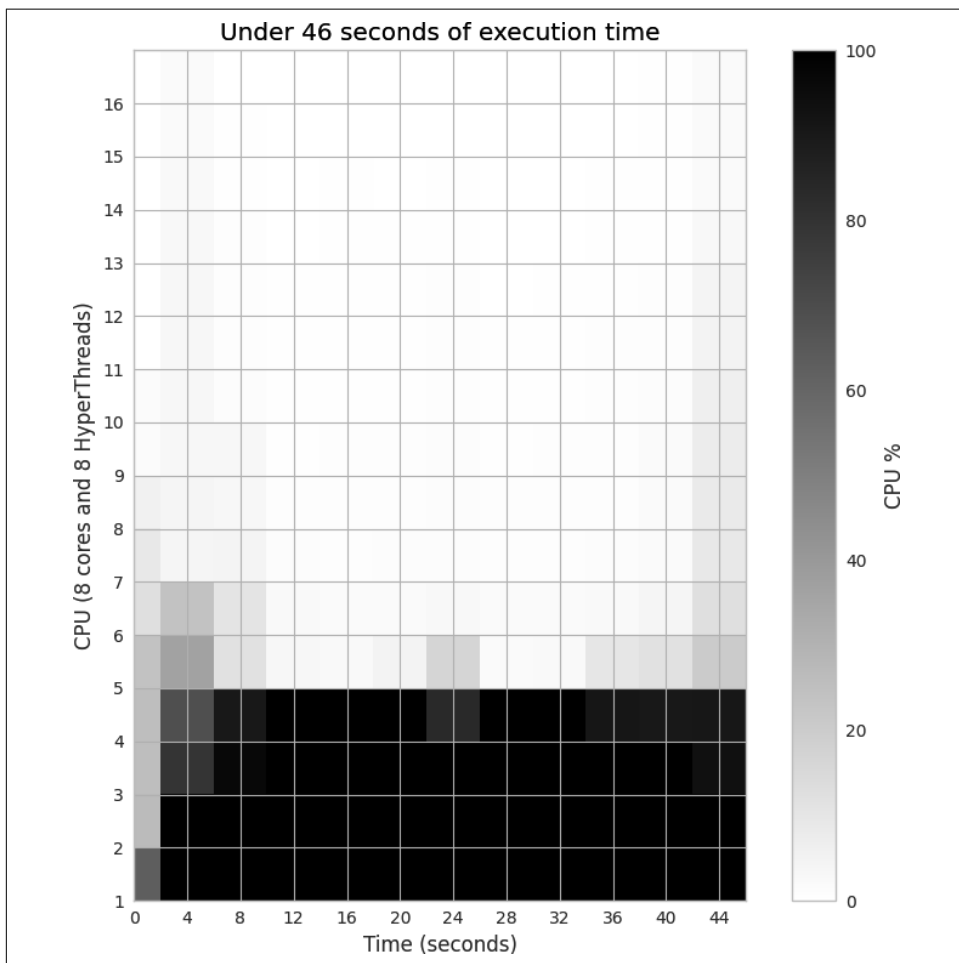


Figure 10-5. Estimating π using Python objects and four processes

By switching to eight processes, as seen in Figure 10-6, we roughly halve the processing time again.

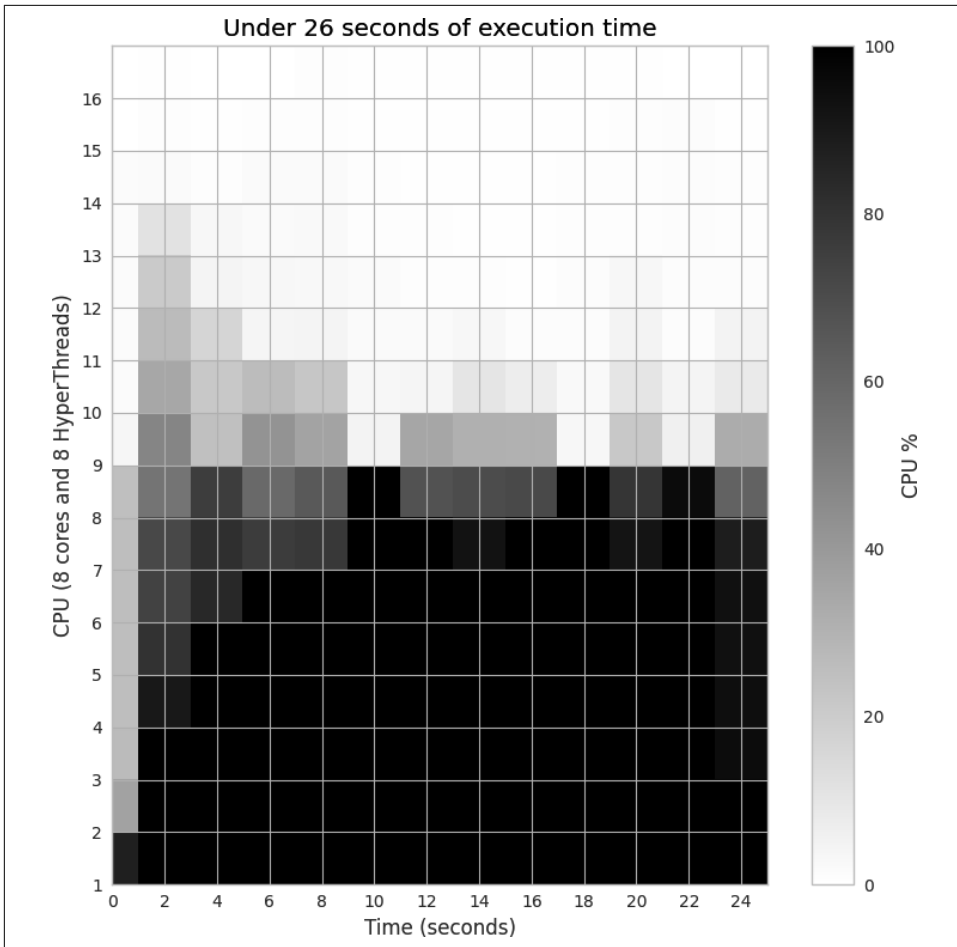


Figure 10-6. Estimating pi using Python objects and eight processes

By switching to sixteen processes, as seen in [Figure 10-7](#), we execute in approximately the same time as the eight-process version. That is because the eight hyperthreads are able to squeeze only a little extra processing power out of the spare silicon on the CPUs, and the eight CPUs are already maximally utilized.

These diagrams show that we're efficiently using more of the available CPU resources at each step, and that the hyperthread resources are a poor addition. The biggest problem when using hyperthreads is that CPython is using a lot of RAM—hyperthreading is not cache friendly, so the spare resources on each chip are very poorly utilized. As we'll see in the next section, numpy makes better use of these resources.

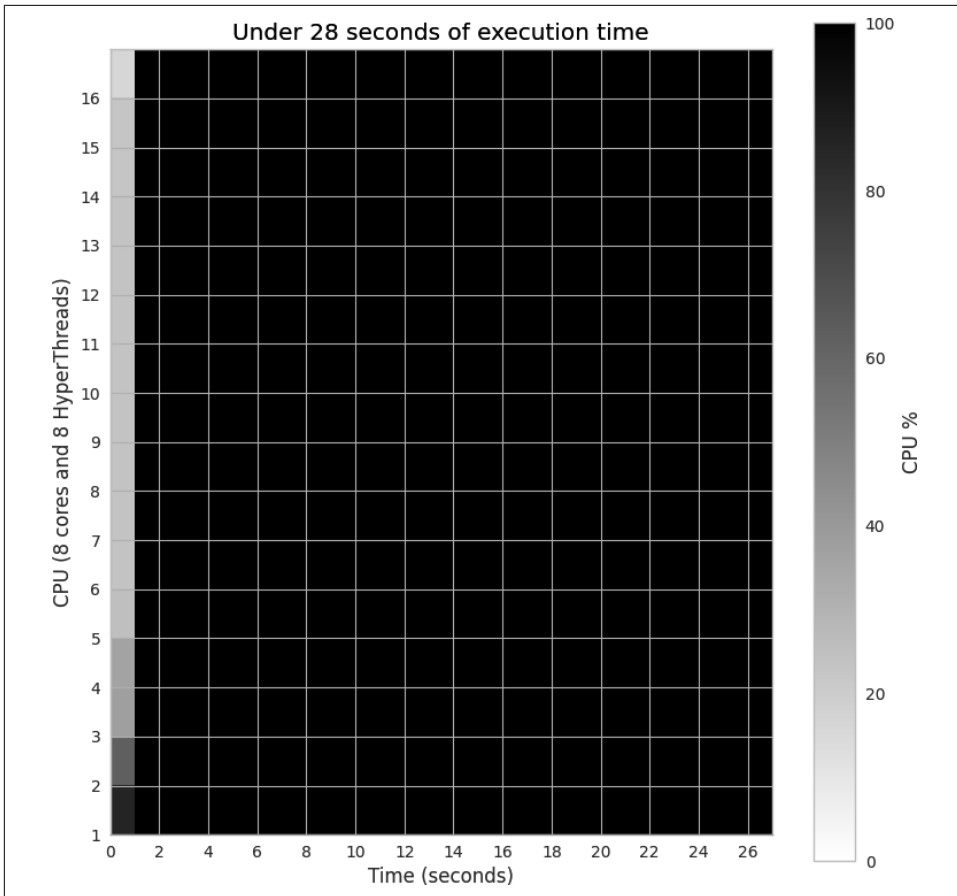


Figure 10-7. Estimating π using Python objects and sixteen processes, with little additional gain



In our experience, hyperthreading can give up to a 30% performance gain *if* there are enough spare computing resources. This works if, for example, you have a mix of floating-point and integer arithmetic rather than just the floating-point operations we have here. By mixing the resource requirements, the hyperthreads can schedule more of the CPU's silicon to be working concurrently. Generally, we see hyperthreads as an added bonus and not a resource to be optimized against, as adding more CPUs is probably more economical than tuning your code (which adds a support overhead).

Now we'll switch to using threads in one process, rather than multiple processes.

Figure 10-8 shows the results of running the same code that we used in Figure 10-5, but with threads in place of processes. Although eight CPUs are being used, they each share the workload very lightly. If each thread was running without the GIL, then we'd see 100% CPU utilization on the eight CPUs. Instead, each CPU is partially utilized (because of the GIL). The overall execution time is comparable to when we used one process.

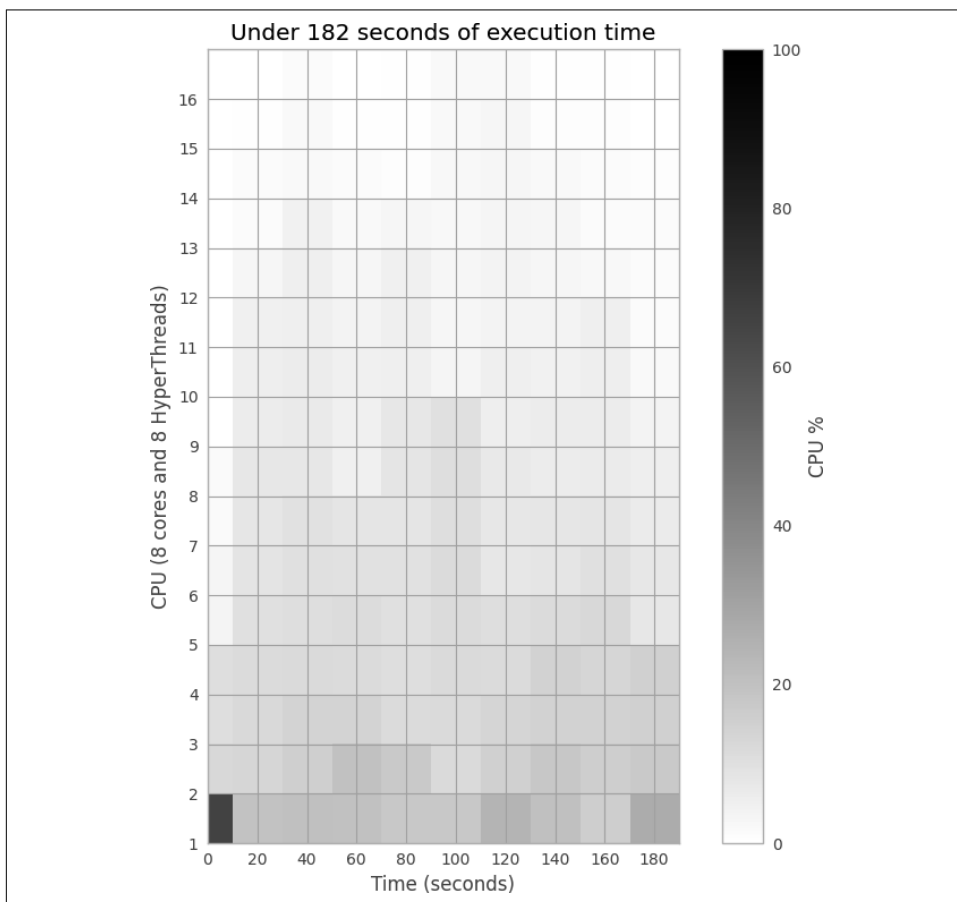


Figure 10-8. Estimating pi using Python objects and eight threads

Replacing multiprocessing with Joblib

Joblib is an improvement on multiprocessing that enables lightweight pipelining with a focus on easy parallel computing and transparent disk-based caching of results. It focuses on NumPy arrays for scientific computing. It may offer a quick win for you if you're:

- Using pure Python, with or without NumPy, to process a loop that could be embarrassingly parallel
- Calling expensive functions that have no side effects, where the output could be cached to disk between sessions
- Able to share NumPy data between processes but don't know how (and you haven't yet read [“Sharing numpy Data with multiprocessing” on page 340](#))

Joblib builds on the Loky library (itself an improvement over Python's `concurrent.futures`) and uses `cloudpickle` to enable the pickling of functions defined in the interactive scope. This solves a couple of common issues that are encountered with the built-in multiprocessing library.

For parallel computing, we need the `Parallel` class and the `delayed` decorator. The `Parallel` class sets up a process pool, similar to the multiprocessing pool we used in the previous section. The `delayed` decorator wraps our target function so it can be applied to the instantiated `Parallel` object via an iterator.

The syntax is a little confusing to read—take a look at [Example 10-3](#). The call is written on one line; this includes our target function `estimate_nbr_points_in_quarter_circle` and the iterator `(delayed(...)(nbr_samples_per_worker) for sample_idx in range(nbr_parallel_blocks))`. Let's break this down.

Example 10-3. Using Joblib to parallelize pi estimation

```
...
from joblib import Parallel, delayed

if __name__ == "__main__":
    ...
    nbr_in_quarter_unit_circles = Parallel(n_jobs=nbr_parallel_blocks, verbose=1) \
        (delayed(estimate_nbr_points_in_quarter_circle)(nbr_samples_per_worker) \
         for sample_idx in range(nbr_parallel_blocks))
    ...
```

`Parallel` is a class; we can set parameters such as `n_jobs` to dictate how many processes will run, along with optional arguments like `verbose` for debugging information. Other arguments can set timeouts, change between threads or processes, change the backends (which can help speed up certain edge cases), and configure memory mapping.

`Parallel` has a `__call__` callable method that takes an iterable. We supply the iterable in the following round brackets (... for `sample_idx` in `range(...)`). The callable iterates over each `delayed(estimate_nbr_points_in_quarter_circle)` function, batching the execution of these functions to their arguments (in this case,

`nbr_samples_per_worker`). Ian has found it helpful to build up a parallelized call one step at a time, starting from a function with no arguments and building up arguments as needed. This makes diagnosing missteps much easier.

`nbr_in_quarter_unit_circles` will be a list containing the count of positive cases for each call as before. **Example 10-4** shows the console output for eight parallel blocks; each process ID (PID) is freshly created, and a summary is printed in a progress bar at the end of the output. In total this takes 24 seconds, the same amount of time as when we created our own Pool in the previous section.



Avoid passing large structures; passing large pickled objects to each process may be expensive. Ian had a case with a prebuilt cache of Pandas DataFrames in a dictionary object; the cost of serializing these via the `pickle` module negated the gains from parallelization, and the serial version actually worked faster overall. The solution in this case was to build the DataFrame cache using Python's built-in **shelve** module, storing the dictionary to a file. A single DataFrame was loaded with `shelve` on each call to the target function; hardly anything had to be passed to the functions, and then the parallelized benefit of Joblib was clear.

Example 10-4. Output of Joblib calls

```
$ python pi_lists_parallel_joblib.py
Making 50,000,000 samples per 8 workers
[Parallel(n_jobs=8)]: Using backend LokyBackend with 8 concurrent workers.
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78227
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78228
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78231
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78229
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78230
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78233
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78232
Executing estimate_nbr_points_in_quarter_circle w. 50,000,000 on pid 78234
[Parallel(n_jobs=8)]: Done 2 out of 8 | elapsed: 24.0s remaining: 1.2min
[Parallel(n_jobs=8)]: Done 8 out of 8 | elapsed: 24.9s finished
Estimated pi 3.14156077
Delta: 24.930650234222412
```



To simplify debugging, we can set `n_jobs=1`, and the parallelized code is dropped. You don't have to modify your code any further, and you can drop a call to `breakpoint()` in your function to ease your debugging.

Intelligent caching of function call results

A useful feature in Joblib is the Memory cache; this is a decorator that caches function results based on the input arguments to a disk cache. This cache persists between Python sessions, so if you turn off your machine and then run the same code the next day, the cached results will be used.

For our pi estimation, this presents a small problem. We don't pass in unique arguments to `estimate_nbr_points_in_quarter_circle`; for each call we pass in `nbr_estimates`, so the call signature is the same, but we're after different results.

In this situation, once the first call has completed (taking around 24 seconds), any subsequent call with the same argument will get the cached result. This means that if we rerun our code a second time, it completes instantly, but it uses only one of the eight sample results as the result for each call—this obviously breaks our Monte Carlo sampling! If the last process to complete resulted in 39271178 points in the quarter circle, the cache for the function call would always answer this. Repeating the call eight times would generate [39271178, 39271178, 39271178, 39271178, 39271178, 39271178, 39271178, 39271178] rather than eight unique estimates.

The solution in [Example 10-5](#) is to pass in a second argument, `idx`, which takes on a value between 0 and `nbr_parallel_blocks-1`. This unique combination of arguments will let the cache store each positive count, so that on the second run we get the same result as on the first run, but without the wait.

This is configured using Memory, which takes a folder for persisting the function results. This persistence is kept between Python sessions; it is refreshed if you change the function that is being called, or if you empty the files in the cache folder.

Note that this refresh applies only to a change to the function that's been decorated (in this case, `estimate_nbr_points_in_quarter_circle_with_idx`), not to any sub-functions that are called from inside that function.

Example 10-5. Caching results with Joblib

```
...
from joblib import Memory

memory = Memory("/tmp/joblib_cache", verbose=0)

@memory.cache
def estimate_nbr_points_in_quarter_circle_with_idx(nbr_estimates, idx):
    print(f"Executing estimate_nbr_points_in_quarter_circle with \
          {nbr_estimates} on sample {idx} on pid {os.getpid()}")
    ...

if __name__ == "__main__":
    ...
```

```

nbr_in_quarter_unit_circles = Parallel(n_jobs=nbr_parallel_blocks) \
    (delayed(
        estimate_nbr_points_in_quarter_circle_with_idx) \
        (nbr_samples_per_worker, idx) for idx in range(nbr_parallel_blocks))
...

```

In [Example 10-6](#), we can see that while the first call costs 24 seconds, the second call takes only a fraction of a second and has the same estimated pi. In this run, the estimates were [39269171, 39271290, 39267758, 39269557, 39273421, 39272052, 39269960, 39271178].

Example 10-6. The zero-cost second call to the code thanks to cached results

```

$ python pi_lists_parallel_joblib_cache.py
Making 50,000,000 samples per 8 workers
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 0 on pid 80549
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 1 on pid 80550
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 2 on pid 80554
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 3 on pid 80553
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 4 on pid 80551
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 5 on pid 80555
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 6 on pid 80552
Executing estimate_nbr_points_in_quarter_circle w. 50000000 on sample 7 on pid 80556
Results collected:
[39269171, 39271290, 39267758, 39269557, 39273421, 39272052, 39269960, 39271178]
Estimated pi 3.14164387
Delta: 24.31753587227783

$ python pi_lists_parallel_joblib_cache.py
Making 50,000,000 samples per 8 workers
Results collected:
[39269171, 39271290, 39267758, 39269557, 39273421, 39272052, 39269960, 39271178]
Estimated pi 3.14164387
Delta: 0.5606045722961426

```

Joblib wraps up a lot of multiprocessing functionality with a simple (if slightly hard to read) interface. Ian has moved to using Joblib in favor of multiprocessing; he recommends you try it too.

Random Numbers in Parallel Systems

Generating good random number sequences is a hard problem, and it is easy to get it wrong if you try to do it yourself. Getting a good sequence quickly in parallel is even harder—suddenly you have to worry about whether you’ll get repeating or correlated sequences in the parallel processes.

We used Python’s built-in random number generator in [Example 10-1](#), and we’ll use the `numpy` random number generator in [Example 10-7](#) in the next section. In both cases, the random number generators are seeded in their forked process. For the Python `random` example, the seeding is handled internally by `multiprocessing`—if during a fork it sees that `random` is in the namespace, it will force a call to seed the generators in each of the new processes.



Set the `numpy` seed when parallelizing your function calls. In the forthcoming `numpy` example, we have to explicitly set the random number seed. If you forget to seed the random number sequence with `numpy`, each of your forked processes will generate an identical sequence of random numbers—it’ll appear to be working as you wanted it to, but behind the scenes each parallel process will evolve with identical results!

If you care about the quality of the random numbers used in the parallel processes, we urge you to research this topic. *Probably* the `numpy` and Python random number generators are good enough, but if significant outcomes depend on the quality of the random sequences (e.g., for medical or financial systems), then you must read up on this area.

In Python 3, the [Mersenne Twister algorithm](#) is used—it has a long period, so the sequence won’t repeat for a long time. It is heavily tested, as it is used in other languages, and it is thread-safe. It is probably not suitable for cryptographic purposes.

Using `numpy`

In this section, we switch to using `numpy`. Our dart-throwing problem is ideal for `numpy` vectorized operations—we generate the same estimate 16 times faster than in the previous Python examples.

The main reason that `numpy` is faster than pure Python when solving the same problem is that `numpy` is creating and manipulating the same object types at a very low level in contiguous blocks of RAM, rather than creating many higher-level Python objects that each require individual management and addressing.

As `numpy` is far more cache friendly, we’ll also get a small speed boost when using the eight hyperthreads. We didn’t get this in the pure Python version, as caches aren’t used efficiently by larger Python objects.

In [Figure 10-9](#), we see three scenarios:

- No use of multiprocessing (named “Serial”)
- Using threads
- Using processes

The serial and single-worker versions execute at the same speed—there’s no overhead to using threads with numpy (and with only one worker, there’s also no gain).

When using multiple processes, we see a classic 100% utilization of each additional CPU. The result mirrors the plots shown in [Figures 10-3](#) through [10-6](#), but the code is much faster using numpy.

Interestingly, the threaded version runs *faster* with more threads. As discussed on the [SciPy wiki](#), by working outside the GIL, numpy can achieve some level of additional speedup around threads.

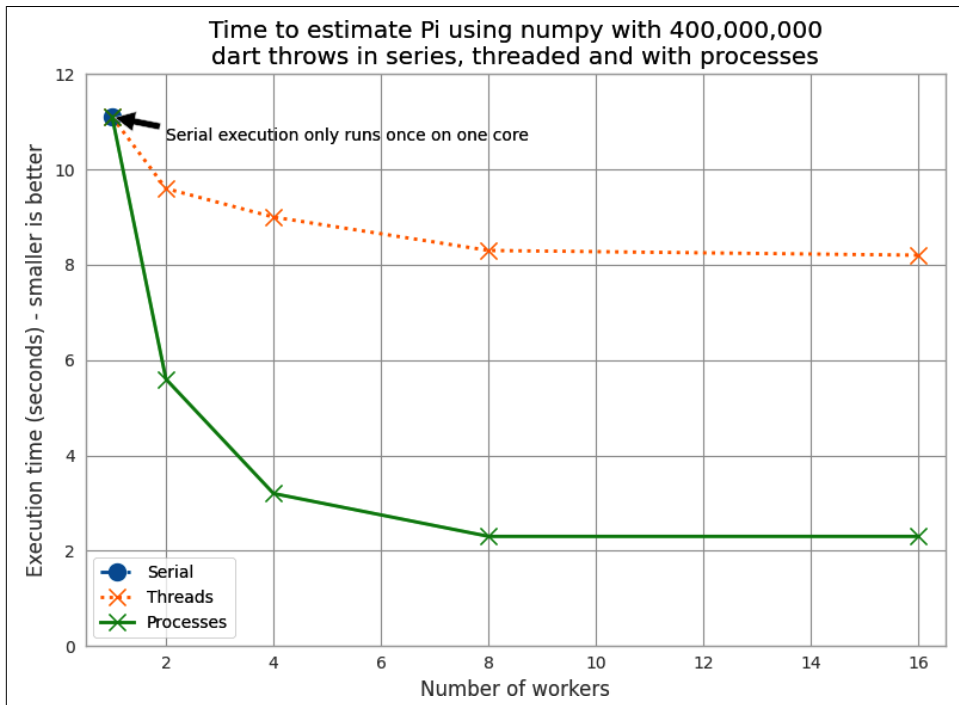


Figure 10-9. Working in series, with threads, and with processes using numpy

Using processes gives us a predictable speedup, just as it did in the pure Python example. A second CPU nearly doubles the speed, and when using four CPUs the speed is nearly quadrupled. We rarely achieve a pure doubling or quadrupling of

execution speeds due to other overheads. Using 8 physical cores gives a further improvement, though with diminishing returns, and trying to squeeze extra calculations using hyperthreads, when no spare floating-point silicon is available, results in no improvement with 16 processes.

Example 10-7 shows the vectorized form of our code. Note that the random number generator is seeded when this function is called. For the threaded version, this isn't necessary, as each thread shares the same random number generator, and the threads access it in series. For the process version, as each new process is a fork, all the forked versions will share the *same state*. This means the random number calls in each will return the same sequence!



Remember to call `seed()` per process with `numpy` to ensure that each of the forked processes generates a unique sequence of random numbers, as a random source is used to set the seed for each call. Look back at “Random Numbers in Parallel Systems” on [page 308](#) for some notes about the dangers of parallelized random number sequences.

Example 10-7. Estimating pi using numpy

```
def estimate_nbr_points_in_quarter_circle(nbr_samples):
    """Estimate pi using vectorized numpy arrays"""
    np.random.seed() # remember to set the seed per process
    xs = np.random.uniform(0, 1, nbr_samples)
    ys = np.random.uniform(0, 1, nbr_samples)
    estimate_inside_quarter_unit_circle = (xs * xs + ys * ys) <= 1
    nbr_trials_in_quarter_unit_circle = np.sum(estimate_inside_quarter_unit_circle)
    return nbr_trials_in_quarter_unit_circle
```

A short code analysis shows that the calls to `random` run a little slower on this machine when executed with multiple threads, and the call to `(xs * xs + ys * ys) <= 1` parallelizes well. Calls to the random number generator are GIL-bound, as the internal state variable is a Python object.

The process to understand this was basic but reliable:

1. Comment out all of the `numpy` lines, and run with *no* threads using the serial version. Run several times and record the execution times using `time.time()` in `__main__`.
2. Add a line back (we added `xs = np.random.uniform(...)` first) and run several times, again recording completion times.
3. Add the next line back (now adding `ys = ...`), run again, and record completion time.

4. Repeat, including the `nbr_trials_in_quarter_unit_circle = np.sum(...)` line.
5. Repeat this process again, but this time with four threads. Repeat line by line.
6. Compare the difference in runtime at each step for no threads and four threads.

Because we're running code in parallel, it becomes harder to use tools like `line_profiler` or `cProfile`. Recording the raw runtimes and observing the differences in behavior with different configurations takes patience but gives solid evidence from which to draw conclusions.



If you want to understand the serial behavior of the `uniform` call, take a look at the `mtrand` code in the `numpy` source and follow the call to `def uniform` in `mtrand.pyx`. This is a useful exercise if you haven't looked at the `numpy` source code before.

The libraries used when building `numpy` are important for some of the parallelization opportunities. Depending on the underlying libraries used when building `numpy` (e.g., whether Intel's Math Kernel Library or OpenBLAS were included or not), you'll see different speedup behavior.

You can check your `numpy` configuration using `numpy.show_config()`. Stack Over-flow has some [example timings](#) if you're curious about the possibilities. Only some `numpy` calls will benefit from parallelization by external libraries.

Finding Prime Numbers

Next, we'll look at testing for prime numbers over a large number range. This is a different problem from estimating π , as the workload varies depending on your location in the number range, and each individual number's check has an unpredictable complexity. We can create a serial routine that checks for primality and then pass sets of possible factors to each process for checking. This problem is embarrassingly parallel, which means there is no state that needs to be shared.

The `multiprocessing` module makes it easy to control the workload, so we shall investigate how we can tune the work queue to use (and misuse!) our computing resources, and we will explore an easy way to use our resources slightly more efficiently. This means we'll be looking at *load balancing* to try to efficiently distribute our varying-complexity tasks to our fixed set of resources.

We'll use an algorithm that is slightly removed from the one earlier in the book (see “Idealized Computing Versus the Python Virtual Machine” on page 10); it exits early if we have an even number—see Example 10-8.

Example 10-8. Finding prime numbers using Python

```
def check_prime(n):  
    if n % 2 == 0:  
        return False  
    for i in range(3, int(math.sqrt(n)) + 1, 2):  
        if n % i == 0:  
            return False  
    return True
```

How much variety in the workload do we see when testing for a prime with this approach? Figure 10-10 shows the increasing time cost to check for primality as the possibly prime n increases from 10,000 to 1,000,000.

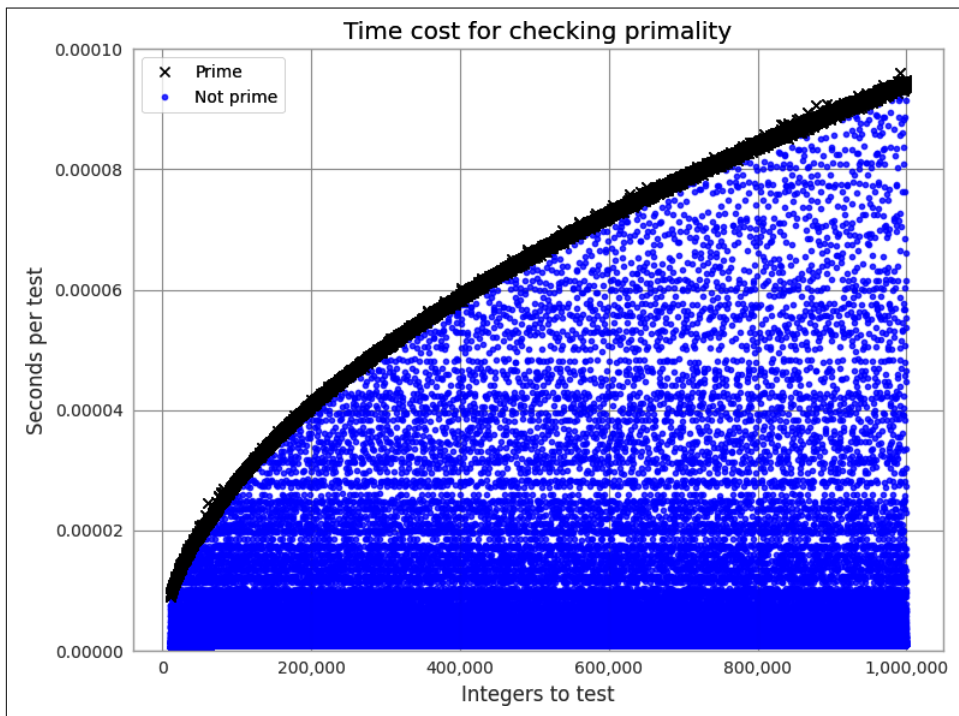


Figure 10-10. Time required to check primality as n increases

Most numbers are nonprime; they're drawn with a dot. Some can be cheap to check for, while others require the checking of many factors. Primes are drawn with an x and form the thick darker band; they're the most expensive to check for. The time cost of checking a number increases as n increases, as the range of possible factors to check increases with the square root of n . The sequence of primes is not predictable, so we can't determine the expected cost of a range of numbers (we could estimate it, but we can't be sure of its complexity).

For the figure, we test each n two hundred times and take the fastest result to remove jitter from the results. If we took only one result, we'd see wide variance in timing that would be caused by system load from other processes; by taking many readings and keeping the fastest, we get to see the expected best-case timing.

When we distribute work to a Pool of processes, we can specify how much work is passed to each worker. We could divide all of the work evenly and aim for one pass, or we could make many chunks of work and pass them out whenever a CPU is free. This is controlled using the `chunksz` parameter. Larger chunks of work mean less communication overhead, while smaller chunks of work mean more control over how resources are allocated.

For our prime finder, a single piece of work is a number n that is checked by `check_prime`. A `chunksz` of 10 would mean that each process handles a list of 10 integers, one list at a time.

In **Figure 10-11**, we can see the effect of varying the `chunksz` from 1 (every job is a single piece of work) to 64 (every job is a list of 64 numbers). Although having many tiny jobs gives us the greatest flexibility, it also imposes the greatest communication overhead. All eight CPUs will be utilized efficiently, but the communication pipe will become a bottleneck as each job and result is passed through this single channel.

If we double the `chunksz` to 2, our task gets solved twice as quickly, as we have less contention on the communication pipe. We might naively assume that by increasing the `chunksz`, we will continue to improve the execution time. However, as you can see in the figure, we will again come to a point of diminishing returns.

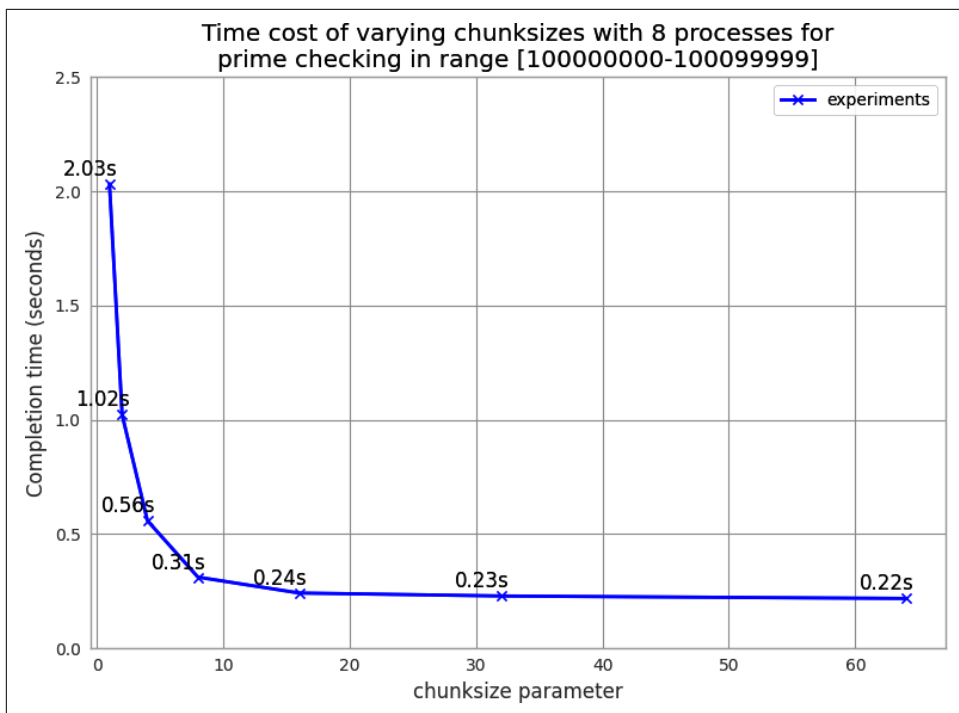


Figure 10-11. Choosing a sensible chunksize value

We can continue to increase the chunksize until we start to see a worsening of behavior. In [Figure 10-12](#), we expand the range of chunksizes, making them not just tiny but also huge. At the larger end of the scale, the worst result shown is 0.79 seconds, where we've asked for chunksize to be 50000—this means our 100,000 items are divided into two work chunks, leaving most CPUs idle for that entire pass. With a chunksize of 10000 items, we are creating ten chunks of work; this means that eight chunks of work will run in parallel, followed by the two remaining chunks. This leaves most CPUs idle in the last round of work, which is an inefficient usage of resources.

An optimal solution in this case is to divide the total number of jobs by the number of CPUs. This is the default behavior in multiprocessing, shown as the “default” dot in the figure.

As a general rule, the default behavior is sensible; tune it only if you expect to see a real gain, and definitely confirm your hypothesis against the default behavior.

Unlike the Monte Carlo pi problem, our prime testing calculation has varying complexity—sometimes a job exits quickly (an even number is detected the fastest), and sometimes the number is large and a prime (this takes a much longer time to check).

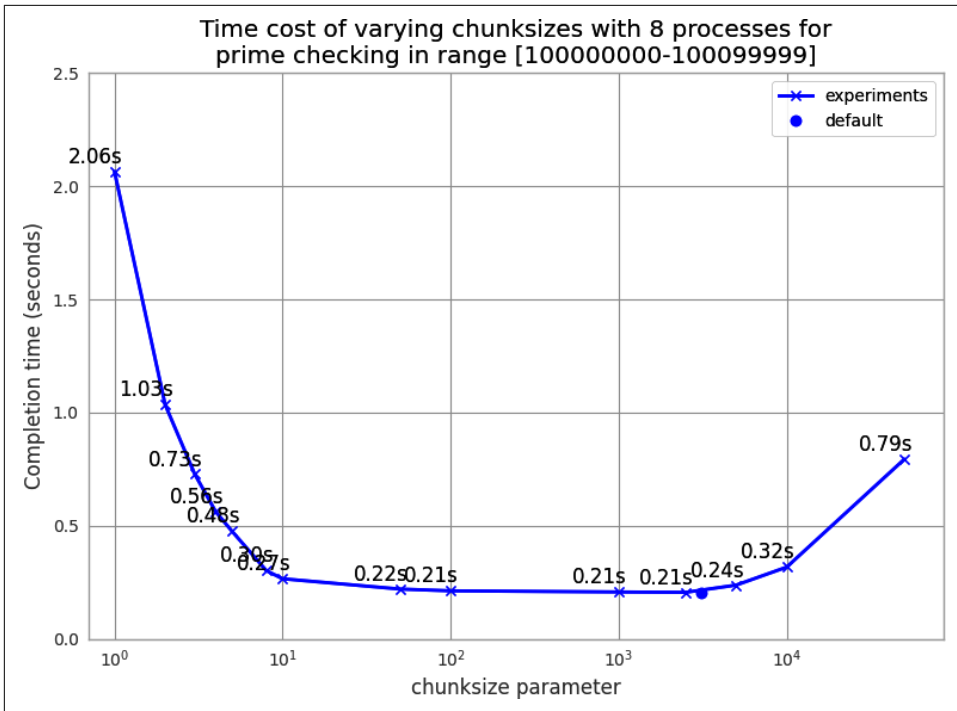


Figure 10-12. Choosing a sensible chunksize value (continued)

What happens if we randomize our job sequence? In earlier editions of this book, with slower CPUs, we could reliably make a tiny gain—in this edition, with i7 chips, there’s no gain, as you can see in [Figure 10-13](#). By randomizing, we reduce the likelihood of the final job in the sequence taking longer than the others, leaving all but one CPU active, but since this occurs rarely it barely offers us an improvement.

As our earlier example using a chunksize of 10000 demonstrated, misaligning the workload with the number of available resources leads to inefficiency. With 10000 items of work per chunk and 100,000 items to process, and with 8 cores, 8 chunks run in parallel and then the last 2 chunks are run. 50000 items takes 0.81 seconds; here only two CPUs are running.

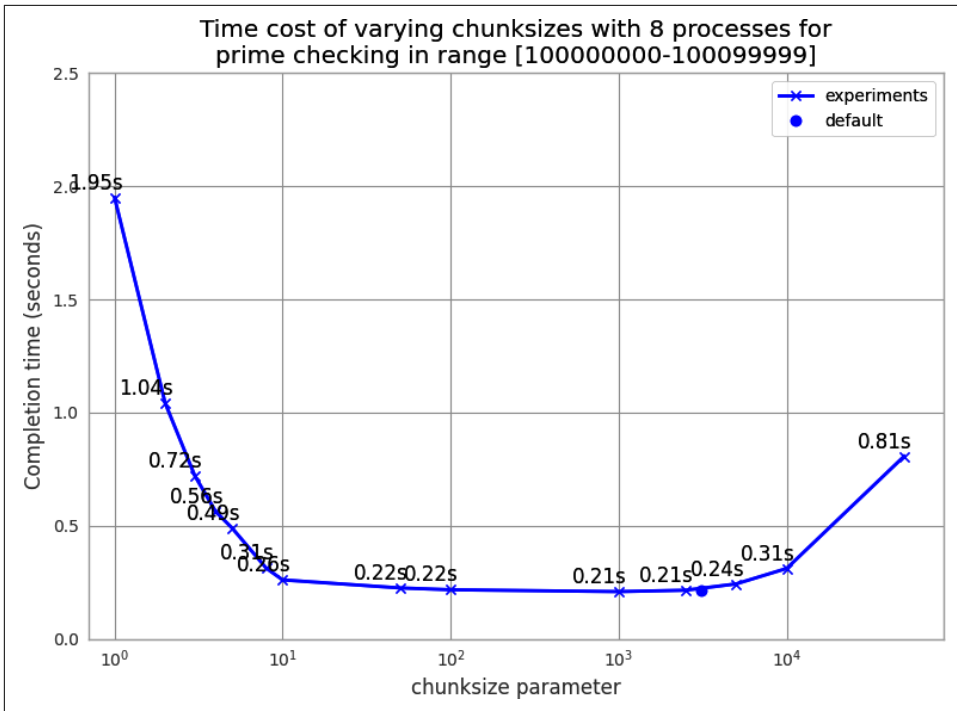


Figure 10-13. Randomizing the job sequence

Figure 10-14 shows the odd effect that occurs when we misalign the number of chunks of work against the number of processors. Mismatches will underutilize the available resources. The slowest overall runtime occurs when only one chunk of work is created: this leaves seven CPUs unutilized. Two work chunks leave six CPUs unutilized, and so on; only when we have eight work chunks or multiples of eight are we using all of our resources. But if we add a ninth work chunk, we're underutilizing our resources again—eight CPUs will work on their chunks, and then one CPU will run to calculate the ninth chunk.

As we increase the number of chunks of work, we see that the inefficiencies decrease: the difference in runtime between 8 and 9 chunks is 0.13 seconds, while between 24 and 25 work chunks it is approximately 0.04 seconds. The general rule is to make lots of small jobs for efficient resource utilization if your jobs have varying runtimes.

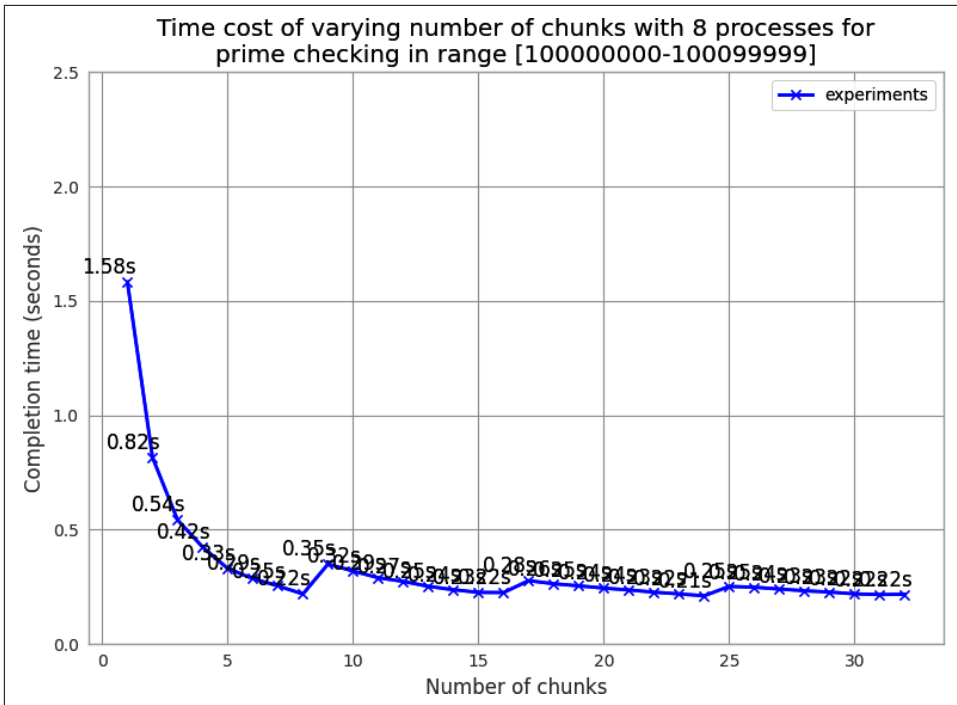


Figure 10-14. The danger of choosing an inappropriate number of chunks

Here are some strategies for efficiently using multiprocessing for embarrassingly parallel problems:

- Split your jobs into independent units of work.
- If your workers take varying amounts of time, consider randomizing the sequence of work (another example would be for processing variable-sized files).
- Sorting your work queue so that the slowest jobs go first may be an equally useful strategy.
- Use the default chunksize unless you have verified reasons for adjusting it.
- Align the number of jobs with the number of physical CPUs. (Again, the default chunksize takes care of this for you, although it will use any hyperthreads by default, which may not offer any additional gain.)

Note that by default, multiprocessing will see hyperthreads as additional CPUs. This means that on Ian's laptop, it will allocate sixteen processes, when only eight will really be running at 100% speed. The additional eight processes could be taking up valuable RAM while barely offering any additional speed gain.

With a Pool, we can split up a chunk of predefined work among the available CPUs up front. This is less helpful if we have dynamic workloads, though, and particularly if we have workloads that arrive over time. For this sort of workload, we might want to use a Queue, introduced in the next section.

Queues of Work

`multiprocessing.Queue` objects give us nonpersistent queues that can send any pickleable Python objects between processes. They carry an overhead, as each object must be pickled to be sent and then unpickled in the consumer (along with some locking operations). In the following example, we'll see that this cost is not negligible. However, if your workers are processing larger jobs, the communication overhead is probably acceptable.

Working with the queues is fairly easy. In this example, we'll check for primes by consuming a list of candidate numbers and posting confirmed primes back to a `definite_primes_queue`. We'll run this with one, two, four, and eight processes and confirm that the latter three approaches all take longer than just running a single process that checks the same range.

A Queue gives us the ability to perform lots of interprocess communication using native Python objects. This can be useful if you're passing around objects with lots of state. Since the Queue lacks persistence, though, you probably don't want to use queues for jobs that might require robustness in the face of failure (e.g., if you lose power or a hard drive gets corrupted).

Example 10-9 shows the `check_prime` function. We're already familiar with the basic primality test. We run in an infinite loop, blocking (waiting until work is available) on `possible_primes_queue.get()` to consume an item from the queue. Only one process at a time can get an item, as the Queue object takes care of synchronizing the accesses. If there's no work in the queue, the `.get()` blocks until a task is available. When primes are found, they are put back on the `definite_primes_queue` for consumption by the parent process.

Example 10-9. Using two queues for interprocess communication (IPC)

```
FLAG_ALL_DONE = b"WORK_FINISHED"
FLAG_WORKER_FINISHED_PROCESSING = b"WORKER_FINISHED_PROCESSING"

def check_prime(possible_primes_queue, definite_primes_queue):
    while True:
        n = possible_primes_queue.get()
        if n == FLAG_ALL_DONE:
            # flag that our results have all been pushed to the results queue
            definite_primes_queue.put(FLAG_WORKER_FINISHED_PROCESSING)
```

```

        break
    else:
        if n % 2 == 0:
            continue
        for i in range(3, int(math.sqrt(n)) + 1, 2):
            if n % i == 0:
                break
        else:
            definite_primes_queue.put(n)

```

We define two flags: one is fed by the parent process as a poison pill to indicate that no more work is available, while the second is fed by the worker to confirm that it has seen the poison pill and has closed itself down. The first poison pill is also known as a *sentinel*, as it guarantees the termination of the processing loop.

When dealing with queues of work and remote workers, it can be helpful to use flags like these to record that the poison pills were sent and to check that responses were sent from the children in a sensible time window, indicating that they are shutting down. We don't handle that process here, but adding some timekeeping is a fairly simple addition to the code. The receipt of these flags can be logged or printed during debugging.

The Queue objects are created out of a Manager in [Example 10-10](#). We'll use the familiar process of building a list of Process objects that each contain a forked process. The two queues are sent as arguments, and multiprocessing handles their synchronization. Having started the new processes, we hand a list of jobs to the possible_primes_queue and end with one poison pill per process. The jobs will be consumed in FIFO order, leaving the poison pills for last. In check_prime we use a blocking .get(), as the new processes will have to wait for work to appear in the queue. Since we use flags, we could add some work, deal with the results, and then iterate by adding more work, and signal the end of life of the workers by adding the poison pills later.

Example 10-10. Building two queues for IPC

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Project description")
    parser.add_argument(
        "nbr_workers", type=int, help="Number of workers e.g. 1, 2, 4, 8"
    )
    args = parser.parse_args()
    primes = []

    manager = multiprocessing.Manager()
    possible_primes_queue = manager.Queue()
    definite_primes_queue = manager.Queue()

```



```

pool = Pool(processes=args.nbr_workers)
processes = []
for _ in range(args.nbr_workers):
    p = multiprocessing.Process(
        target=check_prime, args=(possible_primes_queue,
                                   definite_primes_queue)
    )
    processes.append(p)
    p.start()

t1 = time.time()
number_range = range(100_000_000, 101_000_000)

# add jobs to the inbound work queue
for possible_prime in number_range:
    possible_primes_queue.put(possible_prime)

# add poison pills to stop the remote workers
for n in range(args.nbr_workers):
    possible_primes_queue.put(FLAGS_ALL_DONE)

```

To consume the results, we start another infinite loop in [Example 10-11](#), using a blocking `.get()` on the `definite_primes_queue`. If the `FLAG_WORKER_FINISHED_PROCESSING` flag is found, we take a count of the number of processes that have signaled their exit. If not, we have a new prime, and we add this to the primes list. We exit the infinite loop when all of our processes have signaled their exit.

Example 10-11. Using two queues for IPC

```

processors Indicating they have finished = 0
while True:
    new_result = definite_primes_queue.get() # block while waiting for results
    if new_result == FLAG_WORKER_FINISHED_PROCESSING:
        processors Indicating they have finished += 1
        if processors Indicating they have finished == args.nbr_workers:
            break
    else:
        primes.append(new_result)
assert processors Indicating they have finished == args.nbr_workers

print("Took:", time.time() - t1)
print(len(primes), primes[:10], primes[-10:])

```

There is quite an overhead to using a Queue, due to the pickling and synchronization. As you can see in [Figure 10-15](#), using a Queue-less single-process solution is significantly faster than using two or more processes. The reason in this case is because our workload is very light—the communication cost dominates the overall time for this task. With Queues, two processes complete this example a little more slowly than one process, while four and eight processes are both slower still.

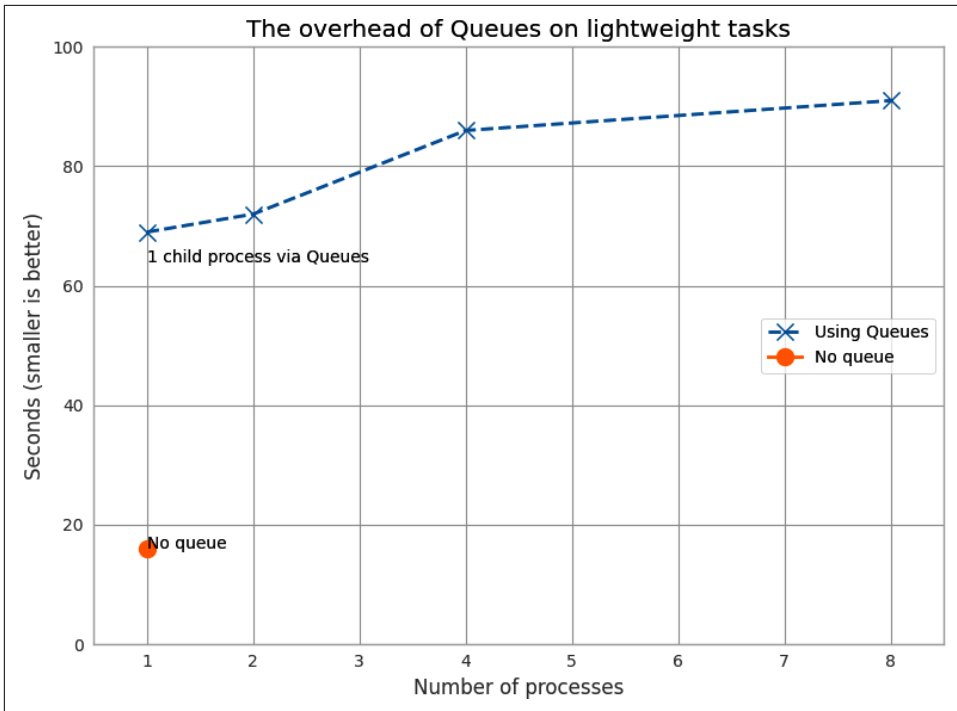


Figure 10-15. Cost of using Queue objects

If your task has a long completion time (at least a sizable fraction of a second) with a small amount of communication, a Queue approach might be the right answer. You will have to verify whether the communication cost makes this approach useful enough.

You might wonder what happens if we remove the redundant half of the job queue (all the even numbers—these are rejected very quickly in `check_prime`). Halving the size of the input queue halves our execution time in each case, but it still doesn't beat the single-process non-Queue example! This helps to illustrate that the communication cost is the dominating factor in this problem.

Asynchronously Adding Jobs to the Queue

By adding a Thread into the main process, we can feed jobs asynchronously into the possible_primes_queue. In [Example 10-12](#), we define a feed_new_jobs function: it performs the same job as the job setup routine that we had in `__main__` before, but it does it in a separate thread.

Example 10-12. Asynchronous job-feeding function

```
def feed_new_jobs(number_range, possible_primes_queue, nbr_poison_pills):
    for possible_prime in number_range:
        possible_primes_queue.put(possible_prime)
    # add poison pills to stop the remote workers
    for n in range(nbr_poison_pills):
        possible_primes_queue.put(FLAG_ALL_DONE)
```

Now, in [Example 10-13](#), our `__main__` will set up the Thread using the possible_primes_queue and then move on to the result-collection phase *before* any work has been issued. The asynchronous job feeder could consume work from external sources (e.g., from a database or I/O-bound communication) while the `__main__` thread handles each processed result. This means that the input sequence and output sequence do not need to be created in advance; they can both be handled on the fly.

Example 10-13. Using a Thread to set up an asynchronous job feeder

```
if __name__ == "__main__":
    primes = []
    manager = multiprocessing.Manager()
    possible_primes_queue = manager.Queue()

    ...

    import threading
    thrd = threading.Thread(target=feed_new_jobs,
                           args=(number_range,
                                possible_primes_queue,
                                NBR_PROCESSES))

    thrd.start()

    # deal with the results
```

If you want robust asynchronous systems, you should almost certainly look to using `asyncio` or an external library such as `aiohttp`. For a full discussion of these approaches, check out [Chapter 9](#). The examples we've looked at here will get you started, but pragmatically they are more useful for very simple systems and education than for production systems.

Be *very aware* that asynchronous systems require a special level of patience—you will end up tearing out your hair while you are debugging. We suggest the following:

- Applying the “Keep It Simple, Stupid” principle
- Avoiding asynchronous self-contained systems (like our example) if possible, as they will grow in complexity and quickly become hard to maintain
- Using mature libraries like `asyncio` that give you tried-and-tested approaches to dealing with certain problem sets

Furthermore, we strongly suggest using an external queue system that gives you external visibility on the state of the queues (e.g., Kafka, ZeroMQ, or Celery, discussed in “[Message Brokering for Cluster Efficiency](#)” on page 368). This requires more thought but is likely to save you time because of increased debug efficiency and better system visibility for production systems.



Consider using a task graph for resilience. Data science tasks requiring long-running queues are frequently served well by specifying pipelines of work in acyclic graphs. Two strong libraries are [Airflow](#) and [Luigi](#). These are very frequently used in industrial settings and enable arbitrary task chaining, online monitoring, and flexible scaling.

Verifying Primes Using Interprocess Communication

Prime numbers are numbers that have no factor other than themselves and 1. It stands to reason that the most common factor is 2 (no even number can be a prime). After that, the low prime numbers (e.g., 3, 5, 7) become common factors of larger nonprimes (e.g., 9, 15, and 21, respectively).

Let’s say that we are given a large number and are asked to verify if it is prime. We will probably have a large space of factors to search. [Figure 10-16](#) shows the frequency of each factor for nonprimes up to 10,000,000. Low factors are far more likely to occur than high factors, but there’s no predictable pattern.

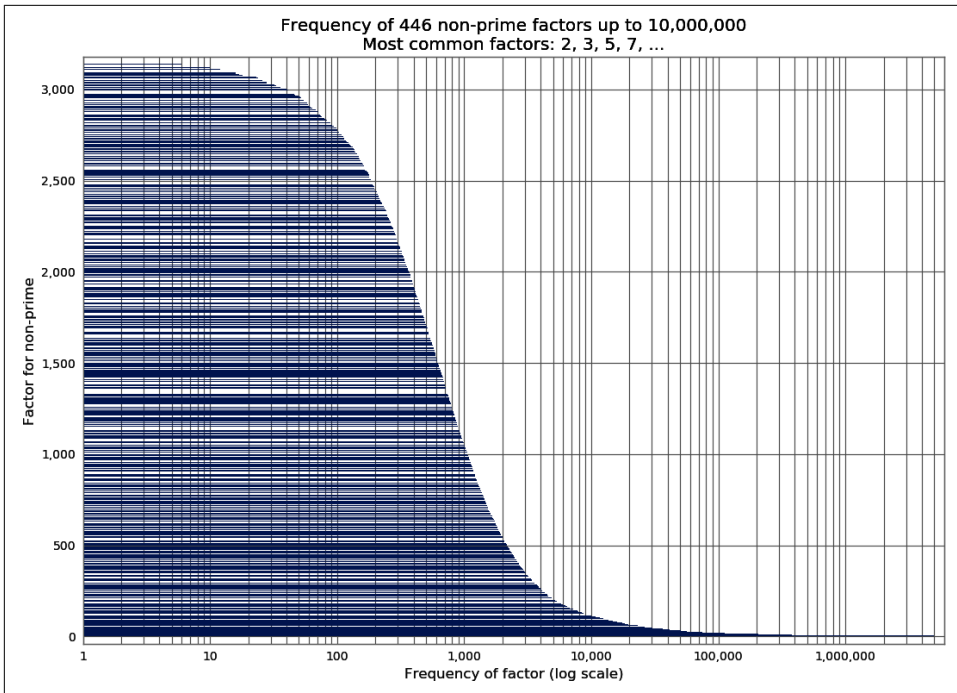


Figure 10-16. The frequency of factors of nonprimes

Let's define a new problem—suppose we have a *small* set of numbers, and our task is to efficiently use our CPU resources to figure out if each number is a prime, one number at a time. Possibly we'll have just one large number to test. It no longer makes sense to use one CPU to do the check; we want to coordinate the work across many CPUs.

For this section we'll look at some larger numbers, one with 15 digits and four with 18 digits:

- Small nonprime: 112,272,535,095,295
- Large nonprime 1: 100,109,100,129,100,369
- Large nonprime 2: 100,109,100,129,101,027
- Prime 1: 100,109,100,129,100,151
- Prime 2: 100,109,100,129,162,907

By using a smaller nonprime and some larger nonprimes, we get to verify that our chosen process not only is faster at checking for primes but also is not getting slower at checking nonprimes. We'll assume that we don't know the size or type of numbers that we're being given, so we want the fastest possible result for all our use cases.

Cooperation comes at a cost—the cost of synchronizing data and checking the shared data can be quite high. We'll work through several approaches here that can be used in different ways for task coordination.

Note that we're *not* covering the somewhat specialized message passing interface (MPI) here; we're looking at batteries-included modules and Redis (which is very common). If you want to use MPI, we assume you already know what you're doing. The [mpi4py project](#) would be a good place to start. It is an ideal technology if you want to control latency when lots of processes are collaborating, whether you have one or many machines.

For the following runs, each test is performed 20 times, and the minimum time is taken to show the fastest speed that is possible for that method. In these examples we're using various techniques to share a flag (often as 1 byte). We could use a basic object like a Lock, but then we'd be able to share only 1 bit of state. We're choosing to show you how to share a primitive type so that more expressive state sharing is possible (even though we don't need a more expressive state for this example).

We must emphasize that sharing state tends to make things *complicated*—you can easily end up in another hair-pulling state. Be careful and try to keep things as simple as they can be. It might be the case that less efficient resource usage is trumped by developer time spent on other challenges.

First we'll discuss the results and then we'll work through the code.

Figure 10-17 shows the first approaches to trying to use interprocess communication to test for primality faster. The benchmark is the serial version, which does not use any interprocess communication; each attempt to speed up our code must at least be faster than this.

The Less Naive Pool version has a predictable (and good) speed. It is good enough to be rather hard to beat. Don't overlook the obvious in your search for high-speed solutions—sometimes a dumb and good-enough solution is all you need.

The approach for the Less Naive Pool solution is to take our number under test, divide its possible-factor range evenly among the available CPUs, and then push the work out to each CPU. If any CPU finds a factor, it will exit early, but it won't communicate this fact; the other CPUs will continue to work through their part of the range. This means for an 18-digit number (our four larger examples), the search time is the same whether it is prime or nonprime.

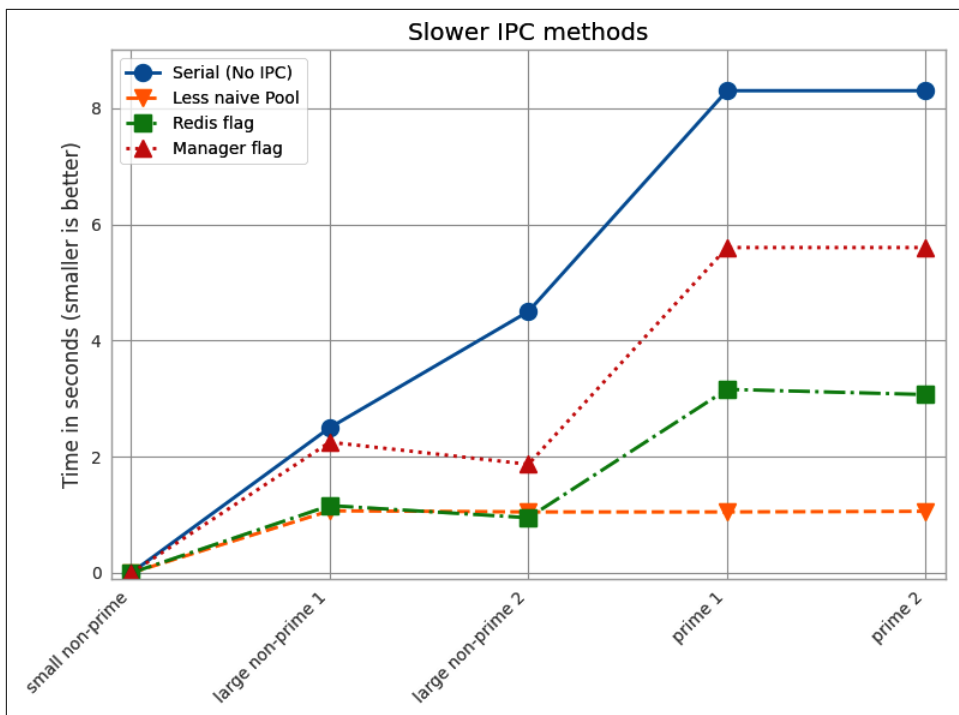


Figure 10-17. The slower ways to use IPC to validate primality

The Redis and Manager solutions are slower when it comes to testing a larger number of factors for primality because of the communication overhead. They use a shared flag to indicate that a factor has been found and the search should be called off.

Redis lets you share state not just with other Python processes but also with other tools and other machines, and even to expose that state over a web-browser interface (which might be useful for remote monitoring). The Manager is a part of `multiprocessing`; it provides a high-level synchronized set of Python objects (including primitives, the list, and the dict).

For the larger nonprime cases, although there is a cost to checking the shared flag, this is dwarfed by the savings in search time gained by signaling early that a factor has been found.

For the prime cases, though, there is no way to exit early, as no factor will be found, so the cost of checking the shared flag will become the dominating cost.



A little bit of thought is often enough. Here we explore various IPC-based solutions to making the prime-validation task faster. In terms of “minutes of typing” versus “gains made,” the first step—introducing naive parallel processing—gave us the largest win for the smallest effort. Subsequent gains took a lot of extra experimentation. Always think about the ultimate runtime, especially for ad hoc tasks. Sometimes it is just easier to let a loop run all weekend for a one-off task than to optimize the code so it runs quicker.

Figure 10-18 shows that we can get a considerably faster result with a bit of effort. The Less Naive Pool result is still our benchmark, but the RawValue and MMap (memory map) results are much faster than the previous Redis and Manager results. The real magic comes from taking the fastest solution and performing some less-obvious code manipulations to make a near-optimal MMap solution—this final version is faster than the Less Naive Pool solution for nonprimes and almost as fast for primes.

In the following sections, we’ll work through various ways of using IPC in Python to solve our cooperative search problem. We hope you’ll see that IPC is fairly easy but generally comes with a cost.

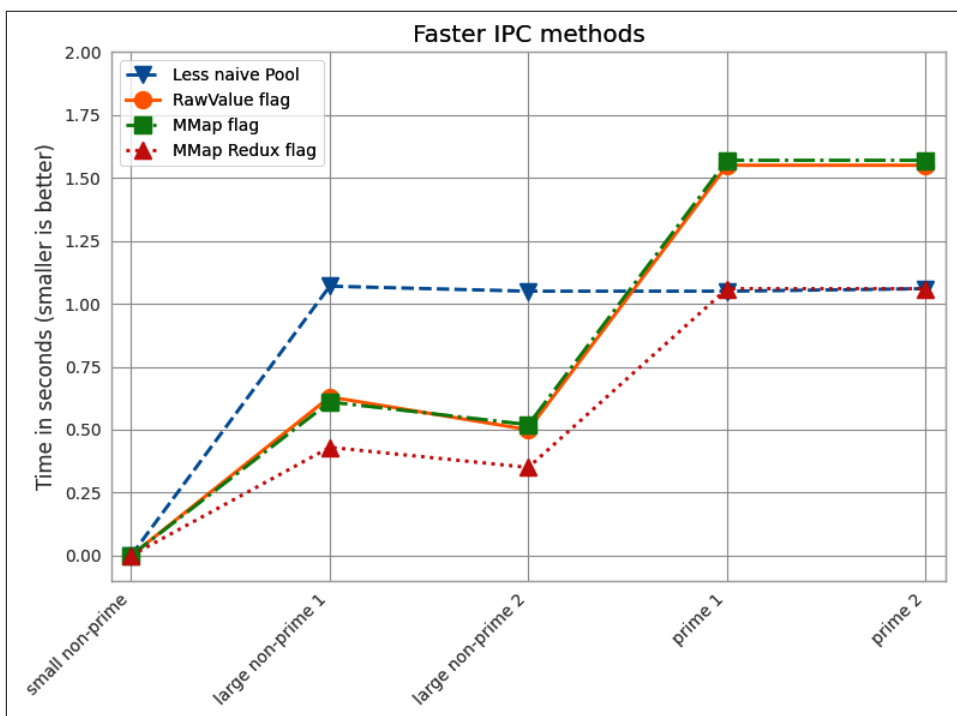


Figure 10-18. The faster ways to use IPC to validate primality

Serial Solution

We'll start with the same serial factor-checking code that we used before, shown again in [Example 10-14](#). As noted earlier, for any nonprime with a large factor, we could more efficiently search the space of factors in parallel. Still, a serial sweep will give us a sensible baseline to work from.

Example 10-14. Serial verification

```
def check_prime(n):
    if n % 2 == 0:
        return False
    from_i = 3
    to_i = math.sqrt(n) + 1
    for i in range(from_i, int(to_i), 2):
        if n % i == 0:
            return False
    return True
```

Naive Pool Solution

The Naive Pool solution works with a `multiprocessing.Pool`, similar to what we saw in “[Finding Prime Numbers](#)” on page 312 and “[Estimating Pi Using Processes and Threads](#)” on page 296 with eight forked processes. We have a number to test for primality, and we divide the range of possible factors into eight tuples of subranges and send these into the `Pool`.

In [Example 10-15](#), we use a new method, `create_range.create` (which we won't show—it's quite boring), that splits the work space into equal-sized regions. Each item in `ranges_to_check` is a pair of lower and upper bounds to search between. For the first 18-digit nonprime (100,109,100,129,100,369), with eight processes we'll have the factor ranges `ranges_to_check == [(3, 39_550_031), (39_550_031, 79_100_057), (79_100_057, 118_650_085), (118_650_085, 158_200_111), (158_200_111, 197_750_139), (197_750_139, 237_300_165), (237_300_165, 276_850_193), (276_850_193, 316_400_222)]` (where 316,400,222 is the square root of 100,109,100,129,100,369 plus 1). In `__main__` we first establish a `Pool`; `check_prime` then splits the `ranges_to_check` for each possibly prime number `n` via a `map`. If the result is `False`, we have found a factor and we do not have a prime.

Example 10-15. Naive Pool solution

```
def check_prime(n, pool, nbr_processes):
    from_i = 3
    to_i = int(math.sqrt(n)) + 1
    ranges_to_check = create_range.create(from_i, to_i, nbr_processes)
    ranges_to_check = zip(len(ranges_to_check) * [n], ranges_to_check)
```

```

assert len(ranges_to_check) == nbr_processes
results = pool.map(check_prime_in_range, ranges_to_check)
if False in results:
    return False
return True

if __name__ == "__main__":
    NBR_PROCESSES = 8
    pool = Pool(processes=NBR_PROCESSES)
    ...

```

We modify the previous `check_prime` in [Example 10-16](#) to take a lower and upper bound for the range to check. There's no value in passing a complete list of possible factors to check, so we save time and memory by passing just two numbers that define our range.

Example 10-16. `check_prime_in_range`

```

def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    for i in range(from_i, int(to_i), 2):
        if n % i == 0:
            return False
    return True

```

This Naive Pool solution takes 0.03 seconds to solve the trivial nonprime check versus 0.000001 seconds for the Serial version. The overhead of setting up workers and distributing the work swamps the time to actually check the trivial nonprime. Otherwise the verification time via the Pool gives a speedup across the board (for the other test items it has the same speed as “Less Naive Pool”) compared to the Serial version.

We could perhaps accept that one slower result isn't a problem—but what if we might get lots of smaller nonprimes to check? It turns out we can avoid this slowdown; we'll see that next with the Less Naive Pool solution.

A Less Naive Pool Solution

The previous solution was inefficient at validating the smaller nonprime. For any smaller (fewer than 18 digits) nonprime, it is likely to be slower than the serial method, because of the overhead of sending out partitioned work and not knowing if a very small factor (which is a more likely factor) will be found. If a small factor is found, the process will still have to wait for the other larger factor searches to complete.

We could start to signal between the processes that a small factor has been found, but since this happens so frequently, it will add a lot of communication overhead. The solution presented in [Example 10-17](#) is a more pragmatic approach—a serial check is performed quickly for likely small factors, and if none are found, then a parallel search is started. Combining a serial precheck before launching a relatively more expensive parallel operation is a common approach to avoiding some of the costs of parallel computing.

Example 10-17. Improving the Naive Pool solution for the small-nonprime case

```
def check_prime(n, pool, nbr_processes):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = 21
    if not check_prime_in_range((n, (from_i, to_i))):
        return False

    # continue to check for larger factors in parallel
    from_i = to_i
    to_i = int(math.sqrt(n)) + 1
    ranges_to_check = create_range.create(from_i, to_i, nbr_processes)
    ranges_to_check = zip(len(ranges_to_check) * [n], ranges_to_check)
    assert len(ranges_to_check) == nbr_processes
    results = pool.map(check_prime_in_range, ranges_to_check)
    if False in results:
        return False
    return True
```

The speed of this solution is equal to or better than that of the original serial search for each of our test numbers. This is our new benchmark.

Importantly, this Pool approach gives us an optimal case for the prime-checking situation. If we have a prime, there's no way to exit early; we have to manually check all possible factors before we can exit.

There's no faster way to check through these factors: any approach that adds complexity will have more instructions, so the check-all-factors case will cause the most instructions to be executed. See the various mmap solutions covered in [“Using mmap as a Flag” on page 337](#) for a discussion on how to get as close to this current result for primes as possible.

Using manager.Value as a Flag

The multiprocessing.Manager() lets us share higher-level Python objects between processes as managed shared objects; the lower-level objects are wrapped in proxy objects. The wrapping and safety have a speed cost but also offer great flexibility. You can share both lower-level objects (e.g., integers and floats) and lists and dictionaries.

In [Example 10-18](#), we create a `Manager` and then create a 1-byte (character) `manager.Value(b"c", FLAG_CLEAR)` flag. You could create any of the `ctypes` primitives (which are the same as the `array.array` primitives) if you wanted to share strings or numbers.

Note that `FLAG_CLEAR` and `FLAG_SET` are assigned a byte (`b'0'` and `b'1'`, respectively). We chose to use the leading `b` to be very explicit (it might default to a Unicode or string object if left as an implicit string, depending on your environment and Python version).

Now we can flag across all of our processes that a factor has been found, so the search can be called off early. The difficulty is balancing the cost of reading the flag against the speed savings that are possible. Because the flag is synchronized, we don't want to check it too frequently—this adds more overhead.

Example 10-18. Passing a `manager.Value` object as a flag

```
SERIAL_CHECK_CUTOFF = 21
CHECK_EVERY = 1000
FLAG_CLEAR = b'0'
FLAG_SET = b'1'
print("CHECK_EVERY", CHECK_EVERY)

if __name__ == "__main__":
    NBR_PROCESSES = 8
    manager = multiprocessing.Manager()
    value = manager.Value(b'c', FLAG_CLEAR) # 1-byte character
    ...
```

`check_prime_in_range` will now be aware of the shared flag, and the routine will be checking to see if a prime has been spotted by another process. Even though we've yet to begin the parallel search, we must clear the flag as shown in [Example 10-19](#) before we start the serial check. Having completed the serial check, if we haven't found a factor, we know that the flag must still be false.

Example 10-19. Clearing the flag with a `manager.Value`

```
def check_prime(n, pool, nbr_processes, value):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = SERIAL_CHECK_CUTOFF
    value.value = FLAG_CLEAR
    if not check_prime_in_range((n, (from_i, to_i), value)):
        return False

    from_i = to_i
    ...
```

How frequently should we check the shared flag? Each check has a cost, both because we're adding more instructions to our tight inner loop and because checking requires a lock to be made on the shared variable, which adds more cost. The solution we've chosen is to check the flag every one thousand iterations. Every time we check, we look to see if `value.value` has been set to `FLAG_SET`, and if so, we exit the search. If in the search the process finds a factor, then it sets `value.value = FLAG_SET` and exits (see [Example 10-20](#)).

Example 10-20. Passing a `manager.Value` object as a flag

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i), value) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    check_every = CHECK_EVERY
    for i in range(from_i, int(to_i), 2):
        check_every -= 1
        if not check_every:
            if value.value == FLAG_SET:
                return False
            check_every = CHECK_EVERY

    if n % i == 0:
        value.value = FLAG_SET
        return False
    return True
```

The thousand-iteration check in this code is performed using a `check_every` local counter. It turns out that this approach, although readable, is suboptimal for speed. By the end of this section, we'll replace it with a less readable but significantly faster approach.

You might be curious about the total number of times we check for the shared flag. In the case of either of the two large primes, with eight processes we check for the flag 158,200 times per prime (we check it this many times in all of the following examples). The checks don't depend on the number of processes, as there are many checks in any large chunk of work. Since each check has an overhead due to locking, this cost really adds up.

Using Redis as a Flag

Redis is a key/value in-memory storage engine. It provides its own locking; and each operation is atomic, so we don't have to worry about using locks from inside Python (or from any other interfacing language).

By using Redis, we make the data storage language-agnostic—any language or tool with an interface to Redis can share data in a compatible way. You could share data between Python, Ruby, C++, and PHP equally easily. You can share data on the local machine or over a network; to share to other machines, all you need to do is change the Redis default of sharing only on `localhost`.

Redis lets you store the following:

- Lists of strings
- Sets of strings
- Sorted sets of strings
- Hashes of strings

Redis stores everything in RAM and snapshots to disk (optionally using journaling) and supports a master/replica resilient architecture. One possibility with Redis is to use it to share a workload across a cluster, where other machines read and write state and Redis acts as a fast centralized data repository.

We can read and write a flag as a text string (all values in Redis are strings) in just the same way as we have been using Python flags previously. We create a `StrictRedis` interface as a global object, which talks to the external Redis server. We could create a new connection inside `check_prime_in_range`, but this is slower and can exhaust the limited number of Redis handles that are available.

We talk to the Redis server using a dictionary-like access. We can set a value using `rds[SOME_KEY] = SOME_VALUE` and read the string back using `rds[SOME_KEY]`.

Example 10-21 is very similar to the previous `Manager` example—we’re using Redis as a substitute for the local `Manager`. It comes with a similar access cost. You should note that Redis supports other (more complex) data structures; it is a powerful storage engine that we’re using just to share a flag for this example. We encourage you to familiarize yourself with its features.

Example 10-21. Using an external Redis server for our flag

```
FLAG_NAME = b'redis_primes_flag'
FLAG_CLEAR = b'0'
FLAG_SET = b'1'

rds = redis.StrictRedis()

def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
```

```

check_every = CHECK_EVERY
for i in range(from_i, int(to_i), 2):
    check_every -= 1
    if not check_every:
        flag = rds[FLAG_NAME]
        if flag == FLAG_SET:
            return False
        check_every = CHECK_EVERY

    if n % i == 0:
        rds[FLAG_NAME] = FLAG_SET
        return False
return True

def check_prime(n, pool, nbr_processes):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = SERIAL_CHECK_CUTOFF
    rds[FLAG_NAME] = FLAG_CLEAR
    if not check_prime_in_range((n, (from_i, to_i))):
        return False

    ...
    if False in results:
        return False
    return True

```

To confirm that the data is stored outside these Python instances, we can invoke `redis-cli` at the command line, as in [Example 10-22](#), and get the value stored in the key `redis_primes_flag`. You'll note that the returned item is a string (not an integer). All values returned from Redis are strings, so if you want to manipulate them in Python, you'll have to convert them to an appropriate datatype first.

Example 10-22. redis-cli

```

$ redis-cli
redis 127.0.0.1:6379> GET "redis_primes_flag"
"0"

```

One powerful argument in favor of the use of Redis for data sharing is that it lives outside the Python world—non-Python developers on your team will understand it, and many tools exist for it. They'll be able to look at its state while reading (but not necessarily running and debugging) your code and follow what's happening. From a team-velocity perspective, this might be a big win for you, despite the communication overhead of using Redis. While Redis is an additional dependency on your project, you should note that it is a very commonly deployed tool, and one that is well debugged and well understood. Consider it a powerful tool to add to your armory.

Redis has many configuration options. By default it uses a TCP interface (that's what we're using), although the benchmark documentation notes that sockets might be much faster. The documentation also states that while TCP/IP lets you share data over a network between different types of OS, other configuration options are likely to be faster (but also are likely to limit your communication options):

When the server and client benchmark programs run on the same box, both the TCP/IP loopback and unix domain sockets can be used. It depends on the platform, but unix domain sockets can achieve around 50% more throughput than the TCP/IP loopback (on Linux for instance). The default behavior of redis-benchmark is to use the TCP/IP loopback. The performance benefit of unix domain sockets compared to TCP/IP loopback tends to decrease when pipelining is heavily used (i.e., long pipelines).

—Redis documentation

Redis is widely used in industry and is mature and well trusted. If you're not familiar with the tool, we strongly suggest you take a look at it; it has a place in your high performance toolkit.

Using RawValue as a Flag

`multiprocessing.RawValue` is a thin wrapper around a `ctypes` block of bytes. It lacks synchronization primitives, so there's little to get in our way in our search for the fastest way to set a flag between processes. It will be almost as fast as the following `mmap` example (it is slower only because a few more instructions get in the way).

Again, we could use any `ctypes` primitive; there's also a `RawArray` option for sharing an array of primitive objects (which will behave similarly to `array.array`). `RawValue` avoids any locking—it is faster to use, but you don't get atomic operations.

Generally, if you avoid the synchronization that Python provides during IPC, you'll come unstuck (once again, back to that pulling-your-hair-out situation). *However*, in this problem it doesn't matter if one or more processes set the flag at the same time—the flag gets switched in only one direction, and every other time it is read, it is just to learn if the search can be called off.

Because we never reset the state of the flag during the parallel search, we don't need synchronization. Be aware that this may not apply to your problem. If you avoid synchronization, please make sure you are doing it for the right reasons.

If you want to do things like update a shared counter, look at the documentation for the `Value` and use a context manager with `value.get_lock()`, as the implicit locking on a `Value` doesn't allow for atomic operations.

This example looks very similar to the previous `Manager` example. The only difference is that in [Example 10-23](#) we create the `RawValue` as a one-character (byte) flag.

Example 10-23. Creating and passing a *RawValue*

```
if __name__ == "__main__":
    NBR_PROCESSES = 8
    value = multiprocessing.RawValue('b', FLAG_CLEAR) # 1-byte character
    pool = Pool(processes=NBR_PROCESSES)
    ...
```

The flexibility to use managed and raw values is a benefit of the clean design for data sharing in multiprocessing.

Using mmap as a Flag

Finally, we get to the fastest way of sharing bytes. [Example 10-24](#) shows a memory-mapped (shared memory) solution using the `mmap` module. The bytes in a shared memory block are not synchronized, and they come with very little overhead. They act like a file—in this case, they are a block of memory with a file-like interface. We have to seek to a location and read or write sequentially. Typically, `mmap` is used to give a short (memory-mapped) view into a larger file, but in our case, rather than specifying a file number as the first argument, we instead pass `-1` to indicate that we want an anonymous block of memory. We could also specify whether we want read-only or write-only access (we want both, which is the default).

Example 10-24. Using a shared memory flag via *mmap*

```
sh_mem = mmap.mmap(-1, 1) # memory map 1 byte as a flag
```

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    check_every = CHECK_EVERY
    for i in range(from_i, int(to_i), 2):
        check_every -= 1
        if not check_every:
            sh_mem.seek(0)
            flag = sh_mem.read_byte()
            if flag == FLAG_SET:
                return False
            check_every = CHECK_EVERY

    if n % i == 0:
        sh_mem.seek(0)
        sh_mem.write_byte(FLAG_SET)
        return False
    return True
```

```
def check_prime(n, pool, nbr_processes):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = SERIAL_CHECK_CUTOFF
    sh_mem.seek(0)
    sh_mem.write_byte(FLAG_CLEAR)
    if not check_prime_in_range((n, (from_i, to_i))):
        return False

    ...
    if False in results:
        return False
    return True
```

`mmap` supports a number of methods that can be used to move around in the file that it represents (including `find`, `readline`, and `write`). We are using it in the most basic way—we seek to the start of the memory block before each read or write, and since we’re sharing just 1 byte, we use `read_byte` and `write_byte` to be explicit.

There is no Python overhead for locking and no interpretation of the data; we’re dealing with bytes directly with the operating system, so this is our fastest communication method.

Using `mmap` as a Flag Redux

While the previous `mmap` result was the best overall, we couldn’t help but think that we should be able to get back to the Naive Pool result for the most expensive case of having primes. The goal is to accept that there is no early exit from the inner loop and to minimize the cost of anything extraneous.

This section presents a slightly more complex solution. The same changes can be made to the other flag-based approaches we’ve seen, although this `mmap` result will still be fastest.

In our previous examples, we’ve used `CHECK EVERY`. This means we have the `check_next` local variable to track, decrement, and use in Boolean tests—and each operation adds a bit of extra time to every iteration. In the case of validating a large prime, this extra management overhead occurs over 150,000 times.

The first optimization, shown in [Example 10-25](#), is to realize that we can replace the decremented counter with a look-ahead value, and then we only have to do a Boolean comparison on the inner loop. This removes a decrement, which, because of Python’s interpreted style, is quite slow.

Example 10-25. Starting to optimize away our expensive logic

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    check_next = from_i + CHECK_EVERY
    for i in range(from_i, int(to_i), 2):
        if check_next == i:
            sh_mem.seek(0)
            flag = sh_mem.read_byte()
            if flag == FLAG_SET:
                return False
            check_next += CHECK_EVERY

        if n % i == 0:
            sh_mem.seek(0)
            sh_mem.write_byte(FLAG_SET)
            return False
    return True
```

We can also entirely replace the logic that the counter represents, as shown in [Example 10-26](#), by unrolling our loop into a two-stage process. First, the outer loop covers the expected range, but in steps, on `CHECK_EVERY`. Second, a new inner loop replaces the `check_every` logic—it checks the local range of factors and then finishes. This is equivalent to the `if not check_every:` test. We follow this with the previous `sh_mem` logic to check the early-exit flag.

Example 10-26. Optimizing away our expensive logic

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    for outer_counter in range(from_i, int(to_i), CHECK_EVERY):
        upper_bound = min(int(to_i), outer_counter + CHECK_EVERY)
        for i in range(outer_counter, upper_bound, 2):
            if n % i == 0:
                sh_mem.seek(0)
                sh_mem.write_byte(FLAG_SET)
                return False
        sh_mem.seek(0)
        flag = sh_mem.read_byte()
        if flag == FLAG_SET:
            return False
    return True
```

The speed impact is dramatic. Our nonprime case improves even further, but more importantly, our prime-checking case is nearly as fast as the Less Naive Pool version (it is now just 0.1 seconds slower). Given that we're doing a lot of extra work with interprocess communication, this is an interesting result. Do note, though, that it is specific to CPython and unlikely to offer any gains when run through a compiler.

In the first edition of the book we went even further with a final example that used loop unrolling and local references to global objects and eked out a further performance gain at the expense of readability. This example in Python 3 yields a minor slowdown, so we've removed it. We're happy about this—fewer hoops needed jumping through to get the most performant example, and the preceding code is more likely to be supported correctly in a team than one that makes implementation-specific code changes.



These examples work just fine with PyPy, where they run around seven times faster than in CPython. Sometimes the better solution will be to investigate other runtimes rather than to go down rabbit holes with CPython.

Sharing numpy Data with multiprocessing

When working with large numpy arrays, you're bound to wonder if you can share the data for read and write access, without a copy, between processes. It is possible, though a little fiddly. We'd like to acknowledge Stack Overflow user *pv* for the inspiration for this demo.²



Do not use this method to re-create the behaviors of BLAS, MKL, Accelerate, and ATLAS. These libraries all have multithreading support in their primitives, and they likely are better-debugged than any new routine that you create. They can require some configuration to enable multithreading support, but it would be wise to see if these libraries can give you free speedups before you invest time (and lose time to debugging!) writing your own.

Sharing a large matrix between processes has several benefits:

- Only one copy means no wasted RAM.
- No time is wasted copying large blocks of RAM.
- You gain the possibility of sharing partial results between the processes.

² See the [Stack Overflow topic](#).

Thinking back to the pi estimation demo using numpy in “Using numpy” on page 309, we had the problem that the random number generation was a serial process. Here, we can imagine forking processes that share one large array, each one using a differently seeded random number generator to fill in a section of the array with random numbers, and therefore completing the generation of a large random block faster than is possible with a single process.

To verify this, we modified the forthcoming demo to create a large random matrix ($10,000 \times 400,000$ elements) as a serial process and by splitting the matrix into eight segments where random is called in parallel (in both cases, one row at a time). The serial process took 31 seconds, and the parallel version took 5 seconds. Refer back to “Random Numbers in Parallel Systems” on page 308 to understand some of the dangers of parallelized random number generation.

For the rest of this section, we’ll use a simplified demo that illustrates the point while remaining easy to verify.

In Figure 10-19, you can see the output from htop on Ian’s laptop. It shows no child processes yet; shortly all nine processes will share a single 10,000-by-400,000-element numpy array of doubles. One copy of this array costs 30.5 GB, and the laptop has 64 GB with approximately 7 GB used by other processes—you can see in htop by the process meters that the Mem reading shows a maximum of 62.5 GB RAM. You can see that if the subprocesses duplicated any of the RAM they’d access, we’d quickly run out of RAM on this laptop.

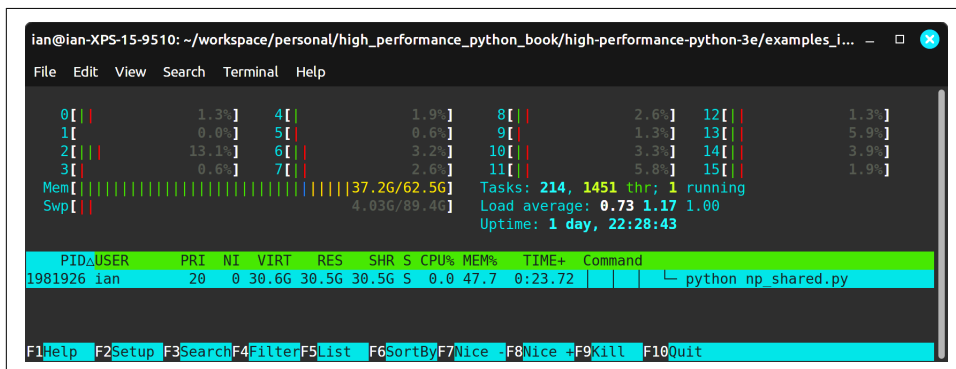


Figure 10-19. htop showing RAM and swap usage



During development of this third edition, Ian was surprised to see 15 threads below the output of `np_shared` in the `htop` example. `np.show_config()` showed that OpenBLAS is used for linear algebra and allocates a default number of threads to make some linear algebra operations faster. This gets in the way of this demo, so we disabled this with `export OPENBLAS_NUM_THREADS=1`. Digging with tools like `htop` teaches us more about what's happening “on the inside.”

To understand this demo, we'll first walk through the console output, and then we'll look at the code. In [Example 10-27](#), we start the parent process: it allocates a 30.5 GB (around 32 GB depending on how it is reported) double array of dimensions 10,000 × 400,000, filled with the value zero. The 10,000 rows will be passed out as indices to the worker function, and the worker will operate on each column of 400,000 items in turn.

Having allocated the array, we fill it with the answer to life, the universe, and everything (42!). We can test in the worker function that we're receiving this modified array and not a filled-with-0s version to confirm that this code is behaving as expected.

Example 10-27. Setting up the shared array

```
$ python np_shared.py
$ python np_shared.py
Creating array of SIZE_A=10000 by SIZE_B=400000
Created shared array with 32,000,000,000 nbytes
Shared array id is 129846909785488 in PID 1981926
Starting with an array of 0 values:
[[0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]]

Original array filled with value 42:
[[42. 42. 42. ... 42. 42. 42.]
 ...
 [42. 42. 42. ... 42. 42. 42.]]
Press a key to start workers using multiprocessing with NBR_OF_PROCESSES=8...
```



For the third edition, Ian learned that his Linux distribution has a 50% RAM cap on shared memory usage, which can be diagnosed using `findmnt -o AVAIL,USED /dev/shm`. This limit can be raised. You may need to investigate similar limits if you're using large arrays. The clue that this is related to configuration was that *up to* 32 GB worked fine, and after this it always exited with `Bus Error (core dumped)`.

In [Example 10-28](#), we've started eight processes working on this shared array. No copy of the array was made; each process is looking at the same large block of memory, and each process has a different set of indices to work from. Every few thousand lines, the worker outputs the current index and its PID, so we can observe its behavior.

The worker's job is trivial—it will check that the current element is still set to the default (so we know that no other process has modified it already), and then it will overwrite this value with the current PID. Once the workers have completed, we return to the parent process and print the array again. This time, we see that it is filled with PIDs rather than 42.

Example 10-28. Running `worker_fn` on the shared array

```
worker_fn: with idx 0
  id of local_nparray_in_process is 129846909785488 in PID 1986279
worker_fn: with idx 1000
  id of local_nparray_in_process is 129846909785488 in PID 1986282
worker_fn: with idx 2000
  id of local_nparray_in_process is 129846909785488 in PID 1986285
worker_fn: with idx 3000
  id of local_nparray_in_process is 129846909785488 in PID 1986284
...
worker_fn: with idx 8000
  id of local_nparray_in_process is 129846909785488 in PID 1986283
worker_fn: with idx 9000
  id of local_nparray_in_process is 129846909785488 in PID 1986284
```

The default value has been over-written with `worker_fn`'s result:

```
[[1986279. 1986279. 1986279. ... 1986279. 1986279. 1986279.]
...
[1986279. 1986279. 1986279. ... 1986279. 1986279. 1986279.]]
```

Finally, in [Example 10-29](#) we use a Counter to confirm the frequency of each PID in the array. As the work was evenly divided, we expect to see each of the eight PIDs represented an equal number of times. In our 4,000,000,000-element array, we see eight sets of around 500,000,000 PIDs split almost equally among the workers.

The table output is presented using [PrettyTable](#).

Example 10-29. Verifying the result on the shared array

Verification - extracting unique values from 4,000,000,000 items in the numpy array (this might be slow)...

Press return to start verifying

Unique values in main_nparray:

```
+-----+
|  PID  |  Count  |
+-----+
| 1986279.0 | 494400000 |
| 1986280.0 | 500800000 |
| 1986281.0 | 500800000 |
| 1986282.0 | 500800000 |
| 1986283.0 | 500800000 |
| 1986284.0 | 500800000 |
| 1986285.0 | 500800000 |
| 1986286.0 | 500800000 |
+-----+
```

Press a key to exit...

Having completed, the program now exits, and the array is deleted.

We can take a peek inside each process under Linux by using `ps` and `pmap`. [Example 10-30](#) shows the result of calling `ps`. Breaking apart this command line:

- `ps` tells us about the process.
- `-A` lists all processes.
- `-o pid,size,vsize,cmd` outputs the PID, size information, and the command name.
- `grep` is used to filter all other results and leave only the lines for our demo.

The parent process (PID 1981926) and its eight forked children are shown in the output. The result is similar to what we saw in `htop`. We can use `pmap` to look at the memory map of each process, requesting extended output with `-x`. We `grep` for the pattern `s-` to list blocks of memory that are marked as being shared. In the parent process and the child processes, we see a 31,250,000 KB (about 32 GB) block that is shared between them.

Example 10-30. Using `pmap` and `ps` to investigate the operating system's view of the processes

```
$ ps -A -o pid,size,vsize,cmd | grep np_shared
1981926 45572 31560424 python np_shared.py
1986279 20552 31339180 python np_shared.py
1986280 20552 31339180 python np_shared.py
1986281 20552 31339180 python np_shared.py
1986282 20552 31339180 python np_shared.py
```



```

1986283 20552 31339180 python np_shared.py
1986284 20552 31339180 python np_shared.py
1986285 20552 31339180 python np_shared.py
1986286 20552 31339180 python np_shared.py

```

```

ian@ian-Latitude-E6420 $ pmap -x 1981926 | grep s-
Address          Kbytes      RSS    Dirty Mode  Mapping
00007610e36e9000 31250000 31250000 31250000 rw-s- pym-1981926-o1kxcg0q (deleted)
...
ian@ian-Latitude-E6420 $ pmap -x 1986279 | grep s-
Address          Kbytes      RSS    Dirty Mode  Mapping
00007610e36e9000 31250000 3862640 3862640 rw-s- pym-1981926-o1kxcg0q (deleted)
...

```

We'll use a `multiprocessing.Array` to allocate a shared block of memory as a 1D array and then instantiate a numpy array from this object and reshape it to a 2D array. Now we have a numpy-wrapped block of memory that can be shared between processes and addressed as though it were a normal numpy array. numpy is not managing the RAM; `multiprocessing.Array` is managing it.

In [Example 10-31](#), you can see that each forked process has access to a global `main_nparray`. While the forked process has a copy of the numpy object, the underlying bytes that the object accesses are stored as shared memory. Our `worker_fn` will overwrite a chosen row (via `idx`) with the current process identifier.

Example 10-31. `worker_fn` for sharing numpy arrays using `multiprocessing`

```

import os
import multiprocessing
from collections import Counter
import ctypes
import numpy as np
from prettytable import PrettyTable

SIZE_A, SIZE_B = 10_000, 400_000 # 32GB

def worker_fn(idx):
    """Do some work on the shared np array on row idx"""
    # confirm that no other process has modified this value already
    assert main_nparray[idx, 0] == DEFAULT_VALUE
    # inside the subprocess print the PID and ID of the array
    # to check we don't have a copy
    if idx % 1000 == 0:
        print("{}: with idx {} \n id of local_nparray_in_process is {} in PID {}".format(
            worker_fn.__name__, idx, id(main_nparray), os.getpid()))
    # we can do any work on the array; here we set every item in this row to
    # have the value of the process ID for this process
    main_nparray[idx, :] = os.getpid()

```

In our `__main__` in [Example 10-32](#), we'll work through three major stages:

1. Build a shared `multiprocessing.Array` and convert it into a `numpy` array.
2. Set a default value into the array, and spawn eight processes to work on the array in parallel.
3. Verify the array's contents after the processes return.

Typically, you'd set up a `numpy` array and work on it in a single process, probably doing something like `arr = np.array((100, 5), dtype=np.float_)`. This is fine in a single process, but you can't share this data across processes for both reading and writing.

The trick is to make a shared block of bytes. One way is to create a `multiprocessing.Array`. By default the `Array` is wrapped in a lock to prevent concurrent edits, but we don't need this lock as we'll be careful about our access patterns. To communicate this clearly to other team members, it is worth being explicit and setting `lock=False`.

If you don't set `lock=False`, you'll have an object rather than a reference to the bytes, and you'll need to call `.get_obj()` to get to the bytes. By calling `.get_obj()`, you bypass the lock, so there's no value in not being explicit about this in the first place.

Next, we take this block of shareable bytes and wrap a `numpy` array around them using `frombuffer`. The `dtype` is optional, but since we're passing bytes around, it is always sensible to be explicit. We `reshape` so we can address the bytes as a 2D array. By default the array values are set to 0. [Example 10-32](#) shows our `__main__` in full.

Example 10-32. `__main__` to set up `numpy` arrays for sharing

```
if __name__ == '__main__':
    DEFAULT_VALUE = 42
    NBR_OF_PROCESSES = 8

    # create a block of bytes, reshape into a local numpy array
    print(f"Creating array of {SIZE_A=} by {SIZE_B=}")
    NBR_ITEMS_IN_ARRAY = SIZE_A * SIZE_B
    shared_array_base = multiprocessing.Array(ctypes.c_double,
                                              NBR_ITEMS_IN_ARRAY, lock=False)
    main_narray = np.frombuffer(shared_array_base, dtype=ctypes.c_double)
    main_narray = main_narray.reshape(SIZE_A, SIZE_B)
    # assert no copy was made
    assert main_narray.base is shared_array_base
    print("Created shared array with {:,} nbytes".format(main_narray.nbytes))
    print("Shared array id is {} in PID {}".format(id(main_narray), os.getpid()))
    print("Starting with an array of 0 values:")
    print(main_narray)
    print()
```

To confirm that our processes are operating on the same block of data that we started with, we set each item to a new `DEFAULT_VALUE` (we again use 42, the answer to life, the universe, and everything)—you’ll see that at the top of [Example 10-33](#). Next, we build a `Pool` of processes (eight in this case) and then send batches of row indices via the call to `map`.

Example 10-33. `__main__` for sharing numpy arrays using multiprocessing

```
# Modify the data via our local numpy array
main_nparrray.fill(DEFAULT_VALUE)
print("Original array filled with value {}".format(DEFAULT_VALUE))
print(main_nparrray)

input(f"Press a key to start workers using multiprocessing with
      {NBR_OF_PROCESSES=}...")
print()

# create a pool of processes that will share the memory block
# of the global numpy array, share the reference to the underlying
# block of data so we can build a numpy array wrapper in the new processes
pool = multiprocessing.Pool(processes=NBR_OF_PROCESSES)
# perform a map where each row index is passed as a parameter to the
# worker_fn
pool.map(worker_fn, range(SIZE_A))
```

Once we’ve completed the parallel processing, we return to the parent process to verify the result ([Example 10-34](#)).

The verification step runs through a flattened view on the array (note that the view does *not* make a copy; it just creates a 1D iterable view on the 2D array), counting the frequency of each PID. Finally, we perform some `assert` checks to make sure we have the expected counts.

Example 10-34. `__main__` to verify the shared result

```
print("Verification - extracting unique values from {:,} items\n
      in the numpy \
      array (this might be slow)...".format(NBR_ITEMS_IN_ARRAY))
# main_nparrray.flat iterates over the contents of the array, it doesn't
# make a copy
counter = Counter(main_nparrray.flat)
print("Unique values in main_nparrray:")
tbl = PrettyTable(["PID", "Count"])
for pid, count in list(counter.items()):
    tbl.add_row([pid, count])
print(tbl)

total_items_set_in_array = sum(counter.values())
```

```

# check that we have set every item in the array away from DEFAULT_VALUE
assert DEFAULT_VALUE not in list(counter.keys())
# check that we have accounted for every item in the array
assert total_items_set_in_array == NBR_ITEMS_IN_ARRAY
# check that we have NBR_OF_PROCESSES of unique keys to confirm that every
# process did some of the work
assert len(counter) == NBR_OF_PROCESSES

input("Press return to exit...")

```

We’ve just created a 1D array of bytes, converted it into a 2D array, shared the array among eight processes, and allowed them to process concurrently on the same block of memory. This recipe will help you parallelize over many cores. Be careful with concurrent access to the *same* data points, though—you’ll have to use the locks in multiprocessing if you want to avoid synchronization problems, and this will slow down your code.

Synchronizing File and Variable Access

In the following examples, we’ll look at multiple processes sharing and manipulating a state—in this case, four processes incrementing a shared counter a set number of times. Without a synchronization process, the counting is incorrect. If you’re sharing data in a coherent way you’ll always need a method to synchronize the reading and writing of data, or you’ll end up with errors.

Typically, the synchronization methods are specific to the OS you’re using, and they’re often specific to the language you use. Here, we look at file-based synchronization using a Python library and sharing an integer object between Python processes.

File Locking

Reading and writing to a file will be the slowest example of data sharing in this section.

You can see our first work function in [Example 10-35](#). The function iterates over a local counter. In each iteration it opens a file and reads the existing value, increments it by one, and then writes the new value over the old one. On the first iteration the file will be empty or won’t exist, so it will catch an exception and assume the value should be zero.



The examples given here are simplified—in practice it is safer to use a context manager to open a file using `with open(filename, "r") as f:`. If an exception is raised inside the context, the file `f` will correctly be closed.

Example 10-35. work function without a lock

```
import multiprocessing
import os

MAX_COUNT_PER_PROCESS = 1000
FILENAME = "count.txt"

def work(filename, max_count):
    for n in range(max_count):
        f = open(filename, "r")
        try:
            nbr = int(f.read())
        except ValueError as err:
            print("File is empty, starting to count from 0, error: " + str(err))
            nbr = 0
        f = open(filename, "w")
        f.write(str(nbr + 1) + '\n')
        f.close()

def run_workers():
    NBR_PROCESSES = 1
    total_expected_count = NBR_PROCESSES * MAX_COUNT_PER_PROCESS
    print("Starting {} process(es) to count to {}".format(NBR_PROCESSES, total_expected_count))
    # reset counter
    f = open(FILENAME, "w")
    f.close()

    processes = []
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work, \
            args=(FILENAME, MAX_COUNT_PER_PROCESS))
        p.start()
        processes.append(p)

    for p in processes:
        p.join()

    print("Expecting to see a count of {}".format(total_expected_count))
    print("{} contains:".format(FILENAME))
    os.system('more ' + FILENAME)

if __name__ == "__main__":
    run_workers()
```

Let's run this example with one process. You can see the output in [Example 10-36](#). work is called one thousand times, and as expected it counts correctly without losing any data. On the first read, it sees an empty file. This raises the `invalid literal for int()` error for `int()` (as `int()` is called on an empty string). This error occurs only once; afterward, we always have a valid value to read and convert into an integer.

Example 10-36. Timing of file-based counting without a lock and with one process

```
$ python ex1_nolock1.py
Starting 1 process(es) to count to 1000
File is empty, starting to count from 0,
error: invalid literal for int() with base 10: ''
Expecting to see a count of 1000
count.txt contains:
1000
```

Now we'll run the same work function with four concurrent processes. We don't have any locking code, so we'll expect some odd results.



Before you look at the following code, what *two* types of error can you expect to see when two processes simultaneously read from or write to the same file? Think about the two main states of the code (the start of execution for each process and the normal running state of each process).

Take a look at [Example 10-37](#) to see the problems. First, when each process starts, the file is empty, so each tries to start counting from zero. Second, as one process writes, the other can read a partially written result that can't be parsed. This causes an exception, and a zero will be written back. This, in turn, causes our counter to keep getting reset! Can you see how `\n` and two values have been written by two concurrent processes to the same open file, causing an invalid entry to be read by a third process?

Example 10-37. Timing of file-based counting without a lock and with four processes

```
$ python ex1_nolock4.py
Starting 4 process(es) to count to 4000
File is empty, starting to count from 0,
error: invalid literal for int() with base 10: ''
# many errors like these
File is empty, starting to count from 0,
File is empty, starting to count from 0, error: invalid literal for int()
with base 10: '\x00\x00'
Expecting to see a count of 4000
count.txt contains:
16
```

```
$ python -m timeit -s "import ex1_nolock4" "ex1_nolock4.run_workers()"
5 loops, best of 5: 87.3 msec per loop
```

Example 10-38 shows the multiprocessing code that calls work with four processes. Note that rather than using a map, we're building a list of Process objects. Although we don't use the functionality here, the Process object gives us the power to introspect the state of each Process. We encourage you to [read the documentation](#) to learn about why you might want to use a Process.

Example 10-38. run_workers setting up four processes

```
import multiprocessing
import os

...
MAX_COUNT_PER_PROCESS = 1000
FILENAME = "count.txt"
...

def run_workers():
    NBR_PROCESSES = 4
    total_expected_count = NBR_PROCESSES * MAX_COUNT_PER_PROCESS
    print("Starting {} process(es) to count to {}".format(NBR_PROCESSES,
        total_expected_count))
    # reset counter
    f = open(FILENAME, "w")
    f.close()

    processes = []
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work, args=(FILENAME,
            MAX_COUNT_PER_PROCESS))

        p.start()
        processes.append(p)

    for p in processes:
        p.join()

    print("Expecting to see a count of {}".format(total_expected_count))
    print("{} contains:".format(FILENAME))
    os.system('more ' + FILENAME)

if __name__ == "__main__":
    run_workers()
```

Using the [fasteners module](#), we can introduce a synchronization method so only one process gets to write at a time and the others each await their turn. The overall process therefore runs more slowly, but it doesn't make mistakes. You can see the

correct output in [Example 10-39](#). Be aware that the locking mechanism is specific to Python, so other processes that are looking at this file will *not* care about the “locked” nature of this file.

Example 10-39. Timing of file-based counting with a lock and four processes

```
$ python ex1_lock.py
Starting 4 process(es) to count to 4000
File is empty, starting to count from 0,
error: invalid literal for int() with base 10: ''
Expecting to see a count of 4000
count.txt contains:
4000
$ python -m timeit -s "import ex1_lock" "ex1_lock.run_workers()"
1 loop, best of 5: 349 msec per loop
```

Using `fasteners` adds a single line of code in [Example 10-40](#) with the `@fasteners.interprocess_locked` decorator; the filename can be anything, but using a similar name as the file you want to lock probably makes debugging from the command line easier. Note that we haven’t had to change the inner function; the decorator gets the lock on each call, and it will wait until it can get the lock before the call into `work` proceeds.

Example 10-40. work function with a lock

```
@fasteners.interprocess_locked('/tmp/tmp_lock')
def work(filename, max_count):
    for n in range(max_count):
        f = open(filename, "r")
        try:
            nbr = int(f.read())
        except ValueError as err:
            print("File is empty, starting to count from 0, error: " + str(err))
            nbr = 0
        f = open(filename, "w")
        f.write(str(nbr + 1) + '\n')
        f.close()
```

Locking a Value

The `multiprocessing` module offers several options for sharing Python objects between processes. We can share primitive objects with a low communication overhead, and we can also share higher-level Python objects (e.g., dictionaries and lists) using a `Manager` (but note that the synchronization cost will significantly slow down the data sharing).

Here, we'll use a `multiprocessing.Value` object to share an integer between processes. While a `Value` has a lock, the lock doesn't do quite what you might expect—it prevents simultaneous reads or writes but does *not* provide an atomic increment. [Example 10-41](#) illustrates this. You can see that we end up with an incorrect count; this is similar to the file-based unsynchronized example we looked at earlier.

Example 10-41. No locking leads to an incorrect count

```
$ python ex2_nolock.py
Expecting to see a count of 4000
We have counted to 2340
$ python -m timeit -s "import ex2_nolock" "ex2_nolock.run_workers()"
20 loops, best of 5: 5.2 msec per loop
```

No corruption occurs to the data, but we do miss some of the updates. This approach might be suitable if you're writing to a `Value` from one process and consuming (but not modifying) that `Value` in other processes.

Note that in Ian's experiments, using a limit of 4000 with Python 3.12 rarely shows the error (you have to try many iterations to get the error)—in the previous edition of the book the error showed up reliably on every run. However, if we try to count 40000 items—a ten times larger operation—then the operation reliably miscounts. Fundamentally, not using a lock would be a bad idea, even with a faster machine and a more advanced interpreter.

The code to share the `Value` is shown in [Example 10-42](#). We have to specify a data-type and an initialization value—using `Value("i", 0)`, we request a signed integer with a default value of 0. This is passed as a regular argument to our `Process` object, which takes care of sharing the same block of bytes between processes behind the scenes. To access the primitive object held by our `Value`, we use `.value`. Note that we're asking for an in-place addition—we'd expect this to be an atomic operation, but that's not supported by `Value`, so our final count is lower than expected.

Example 10-42. The counting code without a Lock

```
import multiprocessing

def work(value, max_count):
    for n in range(max_count):
        value.value += 1

def run_workers():
    ...
    value = multiprocessing.Value('i', 0)
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work,
```

```

                                args=(value, MAX_COUNT_PER_PROCESS))
        p.start()
        processes.append(p)
    ...

```

You can see the correctly synchronized count in [Example 10-43](#) using a `multiprocessing.Lock`.

Example 10-43. Using a Lock to synchronize writes to a Value

```

# lock on the update, but this isn't atomic
$ python ex2_lock.py
Expecting to see a count of 4000
We have counted to 4000
$ python -m timeit -s "import ex2_lock" "ex2_lock.run_workers()"
20 loops, best of 5: 8.18 msec per loop

```

In [Example 10-44](#), we've used a context manager (with `Lock`) to acquire the lock.

Example 10-44. Acquiring a Lock using a context manager

```

import multiprocessing

def work(value, max_count, lock):
    for n in range(max_count):
        with lock:
            value.value += 1

def run_workers():
    ...
    processes = []
    lock = multiprocessing.Lock()
    value = multiprocessing.Value('i', 0)
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work,
                                    args=(value, MAX_COUNT_PER_PROCESS, lock))
        p.start()
        processes.append(p)
    ...

```

If we avoid the context manager and directly wrap our increment with `acquire` and `release`, we can go a little faster, but the code is less readable compared to using the context manager. We suggest sticking to the context manager to improve readability. The snippet in [Example 10-45](#) shows how to acquire and release the `Lock` object.

Example 10-45. Inline locking rather than using a context manager

```
lock.acquire()
value.value += 1
lock.release()
```

Since a `Lock` doesn't give us the level of granularity that we're after, the basic locking that it provides wastes a bit of time unnecessarily. We can replace the `Value` with a `RawValue`, as in [Example 10-46](#), and achieve an incremental speedup. If you're interested in seeing the bytecode behind this change, read [Eli Bendersky's blog post](#) on the subject.

Example 10-46. Console output showing the faster `RawValue` and `Lock` approach

```
# RawValue has no lock on it
$ python ex2_lock_rawvalue.py
Expecting to see a count of 4000
We have counted to 4000
$ python -m timeit -s "import ex2_lock_rawvalue" "ex2_lock_rawvalue.run_workers()"
50 loops, best of 5: 5.18 msec per loop
```

To use a `RawValue`, just swap it for a `Value`, as shown in [Example 10-47](#).

Example 10-47. Example of using a `RawValue` integer

```
...
def run_workers():
    ...
    lock = multiprocessing.Lock()
    value = multiprocessing.RawValue('i', 0)
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work,
                                   args=(value, MAX_COUNT_PER_PROCESS, lock))
        p.start()
        processes.append(p)
```

We could also use a `RawArray` in place of a `multiprocessing.Array` if we were sharing an array of primitive objects.

We've looked at various ways of dividing up work on a single machine between multiple processes, along with sharing a flag and synchronizing data sharing between these processes. Remember, though, that sharing data can lead to headaches—try to avoid it if possible. Making a machine deal with all the edge cases of state sharing is hard; the first time you have to debug the interactions of multiple processes, you'll realize why the accepted wisdom is to avoid this situation if possible.

Do consider writing code that runs a bit slower but is more likely to be understood by your team. Using an external tool like Redis to share state leads to a system that can

be inspected at runtime by people *other* than the developers—this is a powerful way to enable your team to keep on top of what’s happening in your parallel systems.

Definitely bear in mind that tweaked performant Python code is less likely to be understood by more junior members of your team—they’ll either be scared of it or break it. Avoid this problem (and accept a sacrifice in speed) to keep team velocity high.

Wrap-Up

We’ve covered a lot in this chapter. First, we looked at two embarrassingly parallel problems, one with predictable complexity and the other with nonpredictable complexity. We’ll use these examples again on multiple machines when we discuss clustering in [Chapter 11](#).

Next, we looked at Queue support in multiprocessing and its overheads. In general, we recommend using an external queue library so that the state of the queue is more transparent. Preferably, you should use an easy-to-read job format so that it is easy to debug, rather than pickled data.

The IPC discussion should have impressed upon you how difficult it is to use IPC efficiently, and that it can make sense just to use a naive parallel solution (without IPC). Buying a faster computer with more cores might be a far more pragmatic solution than trying to use IPC to exploit an existing machine.

Sharing numpy matrices in parallel without making copies is important for only a small set of problems, but when it counts, it’ll really count. It takes a few extra lines of code and requires some sanity checking to make sure that you’re really not copying the data between processes.

Finally, we looked at using file and memory locks to avoid corrupting data—this is a source of subtle and hard-to-track errors, and this section showed you some robust and lightweight solutions.

In the next chapter we’ll look at clustering using Python. With a cluster, we can move beyond single-machine parallelism and utilize the CPUs on a group of machines. This introduces a new world of debugging pain—not only can your code have errors, but the other machines can also have errors (either from bad configuration or from failing hardware). We’ll show how to parallelize the pi estimation demo using IPython Parallel and how to run research code inside IPython using an IPython cluster.

Clusters and Job Queues

Questions You'll Be Able to Answer After This Chapter

- Why are clusters useful?
- What are the costs of clustering?
- How can I convert a multiprocessing solution into a clustered solution?
- How does an IPython cluster work?
- What considerations should I take when picking a messaging platform?

A *cluster* is commonly recognized to be a collection of computers working together to solve a common task. It could be viewed from the outside as a larger single system.

In the 1990s, the notion of using a cluster of commodity PCs on a local area network for clustered processing—known as a **Beowulf cluster**—became popular. **Google** later gave the practice a boost by using clusters of commodity PCs in its own data centers, particularly for running MapReduce tasks. At the other end of the scale, the **TOP500 project** ranks the most powerful computer systems each year; these typically have a clustered design, and the fastest machines all use Linux.

Amazon Web Services (AWS) is commonly used both for engineering production clusters in the cloud and for building on-demand clusters for short-lived tasks like machine learning. With AWS you can rent tiny to huge machines with tens of CPUs and up to 768 GB of RAM for \$1 to \$15 an hour. Multiple GPUs can be rented at extra cost. Look at “**Using IPython Parallel to Support Research**” on page 365 and the **ElastiCluster** package if you’d like to explore AWS or other providers for ad hoc clusters on compute-heavy or RAM-heavy tasks.

Different computing tasks require different configurations, sizes, and capabilities in a cluster. We'll define some common scenarios in this chapter.

Before you move to a clustered solution, do make sure that you have done the following:

- Profiled your system so you understand the bottlenecks
- Exploited compiler solutions like Numba and Cython
- Exploited multiple cores on a single machine (possibly a big machine with many cores) with Joblib or multiprocessing
- Exploited techniques for using less RAM

Keeping your system to one machine will make your life easier (even if the “one machine” is a really beefy computer with lots of RAM and many CPUs). Move to a cluster if you really need a *lot* of CPUs or the ability to process data from disks in parallel, or if you have production needs like high resiliency and rapid speed of response. Most research scenarios do not need resilience or scalability and are limited to few people, so the simplest solution is often the most sensible.

Benefits of Clustering

The most obvious benefit of a cluster is that you can easily scale computing requirements—if you need to process more data or get an answer faster, you just add more machines (or *nodes*).

By adding machines, you can also improve reliability. Each machine's components have a certain likelihood of failing, but with a good design, the failure of a number of components will not stop the operation of the cluster.

Clusters are also used to create systems that scale dynamically. A common use case is to cluster a set of servers that process web requests or associated data (e.g., resizing user photos, transcoding video, or transcribing speech) and to activate more servers as demand increases at certain times of the day.

Dynamic scaling is a very cost-effective way of dealing with nonuniform usage patterns, as long as the machine activation time is fast enough to deal with the speed of changing demand.



Consider the effort versus the reward of building a cluster. While the parallelization gains of a cluster can feel attractive, do consider the costs associated with constructing and maintaining a cluster. They fit well for long-running processes in a production environment or for well-defined and oft-repeated R&D tasks. They are less attractive for variable and short-lived R&D tasks.

A subtler benefit of clustering is that clusters can be separated geographically but still centrally controlled. If one geographic area suffers an outage (due to a flood or power loss, for example), the other cluster can continue to work, perhaps with more processing units being added to handle the demand. Clusters also allow you to run heterogeneous software environments (e.g., different versions of operating systems and processing software), which *might* improve the robustness of the overall system—note, though, that this is definitely an expert-level topic!

Drawbacks of Clustering

Moving to a clustered solution requires a change in thinking. This is an evolution of the change in thinking required when you move from serial to parallel code, as we introduced back in [Chapter 10](#). Suddenly you have to consider what happens when you have more than one machine—you have latency between machines, you need to know if your other machines are working, and you need to keep all the machines running the same version of your software. System administration is probably your biggest challenge.

In addition, you normally have to think hard about the algorithms you are implementing and what happens once you have all these additional moving parts that may need to stay in sync. This additional planning can impose a heavy mental tax; it is likely to distract you from your core task, and once a system grows large enough, you'll probably need to add a dedicated engineer to your team.



We've tried to focus on using one machine efficiently in this book because we believe that life is easier if you're dealing with only one computer rather than a collection (though we confess it can be *way* more fun to play with a cluster—until it breaks). If you can scale vertically (by buying more RAM or more CPUs), it is worth investigating this approach in favor of clustering. Of course, your processing needs may exceed what's possible with vertical scaling, or the robustness of a cluster may be more important than having a single machine. If you're a single person working on this task, though, bear in mind also that running a cluster will suck up some of your time.

When designing a clustered solution, you'll need to remember that each machine's configuration might be different (each machine will have a different load and different local data). How will you get all the right data onto the machine that's processing your job? Does the latency involved in moving the job and the data amount to a problem? Do your jobs need to communicate partial results to one another? What happens if a process fails or a machine dies or some hardware wipes itself when several jobs are running? Failures can be introduced if you don't consider these questions.

You should also consider that failures *can be acceptable*. For example, you probably don't need 99.999% reliability when you're running a content-based web service—if on occasion a job fails (e.g., a picture doesn't get resized quickly enough) and the user is required to reload a page, that's something that everyone is already used to. It might not be the solution you want to give to the user, but accepting a little bit of failure typically reduces your engineering and management costs by a worthwhile margin. On the flip side, if a high-frequency trading system experiences failures, the cost of bad stock market trades could be considerable!

Maintaining a fixed infrastructure can become expensive. Machines are relatively cheap to purchase, but they have an awful habit of going wrong—automatic software upgrades can glitch, network cards fail, disks have write errors, power supplies can give spikey power that disrupts data, cosmic rays can flip a bit in a RAM module. The more computers you have, the more time will be lost to dealing with these issues. Sooner or later you'll want to bring in a system engineer who can deal with these problems, so add another \$100,000 to the budget. Using a cloud-based cluster can mitigate a lot of these problems (it costs more, but you don't have to deal with the hardware maintenance), and some cloud providers also offer a **spot-priced market** for cheap but temporary computing resources.

An insidious problem with a cluster that grows organically over time is that it's possible no one has documented how to restart it safely if everything gets turned off. If you don't have a documented restart plan, you should assume you'll have to write one at the worst possible time (one of your authors has been involved in debugging this sort of problem on Christmas Eve—this is not the Christmas present you want!). At this point you'll also learn just how long it can take each part of a system to get up to speed—it might take minutes for each part of a cluster to boot and to start to process jobs, so if you have 10 parts that operate in succession, it might take an hour to get the whole system running from cold. The consequence is that you might have an hour's worth of backlogged data. Do you then have the necessary capacity to deal with this backlog in a timely fashion?

Slack behavior can be a cause of expensive mistakes, and complex and hard-to-anticipate behavior can cause unexpected and expensive outcomes. Let's look at two high-profile cluster failures and see what lessons we can learn.

\$462 Million Wall Street Loss Through Poor Cluster Upgrade Strategy

In 2012, the high-frequency trading firm **Knight Capital** lost **\$462 million** after a bug was introduced during a software upgrade in a cluster. The software made orders for more shares than customers had requested.

In the trading software, an older flag was repurposed for a new function. The upgrade was rolled out to seven of the eight live machines, but the eighth machine used older code to handle the flag, which resulted in the wrong trades being made. The

Securities and Exchange Commission (SEC) noted that Knight Capital didn't have a second technician review the upgrade and in fact had no established process for reviewing such an upgrade.

The underlying mistake seems to have had two causes. The first was that the software development process hadn't removed an obsolete feature, so the stale code stayed around. The second was that no manual review process was in place to confirm that the upgrade was completed successfully.

Technical debt adds a cost that eventually has to be paid—preferably by taking time when not under pressure to remove the debt. Always use unit tests, both when building and when refactoring code. The lack of a written checklist to run through during system upgrades, along with a second pair of eyes, could cost you an expensive failure. There's a reason that airplane pilots have to work through a takeoff checklist: it means that nobody ever skips the important steps, no matter how many times they might have done them before!

Skype's 24-Hour Global Outage

Skype suffered a **24-hour planetwide failure** in 2010. Behind the scenes, Skype is supported by a peer-to-peer network. An overload in one part of the system (used to process offline instant messages) caused delayed responses from Windows clients; some versions of the Windows client didn't properly handle the delayed responses and crashed. In all, approximately 40% of the live clients crashed, including 25% of the public supernodes. Supernodes are critical to routing data in the network.

With 25% of the routing offline (it came back on, but slowly), the network overall was under great strain. The crashed Windows client nodes were also restarting and attempting to rejoin the network, adding a new volume of traffic on the already overloaded system. The supernodes have a back-off procedure if they experience too much load, so they started to shut down in response to the waves of traffic.

Skype became largely unavailable for 24 hours. The recovery process involved first setting up hundreds of new “mega-supernodes” configured to deal with the increased traffic, and then following up with thousands more. Over the coming days, the network recovered.

This incident caused a lot of embarrassment for Skype; clearly, it also changed its focus to damage limitation for several tense days. Customers were forced to look for alternative solutions for voice calls, which was likely a marketing boon for competitors.

Given the complexity of the network and the escalation of failures that occurred, this failure likely would have been hard both to predict and to plan for. The reason that *all* of the nodes on the network didn't fail was due to different versions of the

software and different platforms—there’s a reliability benefit to having a heterogeneous network rather than a homogeneous system.

Common Cluster Designs

It is common to start with a local ad hoc cluster of reasonably equivalent machines. You might wonder if you can add old computers to an ad hoc network, but typically older CPUs eat a lot of power and run very slowly, so they don’t contribute nearly as much as you might hope compared to one new high-specification machine. An in-office cluster requires someone who can maintain it. A cluster on [Amazon’s EC2](#) or [Microsoft’s Azure](#), or one run by an academic institution, offloads the hardware support to the provider’s team.

If you have well-understood processing requirements, it might make sense to design a custom cluster—perhaps one that uses an InfiniBand high-speed interconnect in place of gigabit Ethernet, or one that uses a particular configuration of RAID drives that support your read, write, or resiliency requirements. You might want to combine CPUs and GPUs on some machines, or just default to CPUs.

You might want a massively decentralized processing cluster, like the ones used by projects such as *SETI@home* and *Folding@home* through [the Berkeley Open Infrastructure for Network Computing \(BOINC\) system](#). They share a centralized coordination system, but the computing nodes join and leave the project in an ad hoc fashion.

On top of the hardware design, you can run different software architectures. Queues of work are the most common and easiest to understand. Typically, jobs are put onto a queue and consumed by a processor. The result of the processing might go onto another queue for further processing, or it might be used as a final result (e.g., being added into a database). Message-passing systems are slightly different—messages get put onto a message bus and are then consumed by other machines. The messages might time out and get deleted, and they might be consumed by multiple machines. In a more complex system, processes talk to each other using interprocess communication—this can be considered an expert-level configuration, as there are lots of ways that you can set it up badly, which will result in you losing your sanity. Go down the IPC route only if you really know that you need it.

How to Start a Clustered Solution

The easiest way to start a clustered system is to begin with one machine that will run both the job server and a job processor (just one job processor for one CPU). If your tasks are CPU-bound, run one job processor per CPU; if your tasks are I/O-bound, run several per CPU. If they’re RAM-bound, be careful that you don’t run out of RAM. Get your single-machine solution working with one processor and then add

more. Make your code fail in unpredictable ways (e.g., do a `1/0` in your code, use `kill -9 <pid>` on your worker, pull the power plug from the socket so the whole machine dies) to check if your system is robust.

Obviously, you'll want to do heavier testing than this—a unit test suite full of coding errors and artificial exceptions is good. Ian likes to throw in unexpected events, like having a processor run a set of jobs while an external process is systematically killing important processes and confirming that these all get restarted cleanly by whatever monitoring process is being used.

Once you have one running job processor, add a second. Check that you're not using too much RAM. Do you process jobs twice as fast as before?

Now introduce a second machine, with just one job processor on that new machine and no job processors on the coordinating machine. Does it process jobs as fast as when you had the processor on the coordinating machine? If not, why not? Is latency a problem? Do you have different configurations? Maybe you have different machine hardware, like CPUs, RAM, and cache sizes?

Now add another nine computers and test to see if you're processing jobs 10 times faster than before. If not, why not? Are network collisions now occurring that slow down your overall processing rate?

To reliably start the cluster's components when the machine boots, we tend to use either a cron job, **Circus**, or **supervisord**. Circus and supervisord are both Python-based and have been around for years. cron is old but very reliable if you're just starting scripts like a monitoring process that can start subprocesses as required.

Once you have a reliable cluster, you might want to introduce a random-killer tool like Netflix's **Chaos Monkey**, which deliberately kills parts of your system to test them for resiliency. Your processes and your hardware will die eventually, and it doesn't hurt to know that you're likely to survive at least the errors you predict might happen.

Ways to Avoid Pain When Using Clusters

In one particularly painful experience Ian encountered, a series of queues in a clustered system ground to a halt. Later queues were not being consumed, so they filled up. Some of the machines ran out of RAM, so their processes died. Earlier queues were being processed but couldn't pass their results to the next queue, so they crashed. In the end the first queue was being filled but not consumed, so it crashed. After that, we were paying for data from a supplier that ultimately was discarded. You must sketch out some notes to consider the various ways your cluster will die and what will happen when (not *if*) it does. Will you lose data (and is this a problem)? Will you have a large backlog that's too painful to process?

Having a system that's easy to debug *probably* beats having a faster system. Engineering time and the cost of downtime are *probably* your largest expenses (this isn't true if you're running a missile defense program, but it is probably true for a start-up). Rather than shaving a few bytes by using a low-level compressed binary protocol, consider using human-readable text in JSON when passing messages. It does add an overhead for sending the messages and decoding them, but when you're left with a partial database after a core computer has caught fire, you'll be glad that you can read the important messages quickly as you work to bring the system back online.

Make sure it is cheap in time and money to deploy updates to the system—both operating system updates and new versions of your software. Every time anything changes in the cluster, you risk the system responding in odd ways if it is in a schizophrenic state. Make sure you use a deployment system like [Fabric](#), [Salt](#), [Chef](#), or [Puppet](#), or a system image like a Debian *.deb*, a Red Hat *.rpm*, or an [Amazon Machine Image](#). Being able to robustly deploy an update that upgrades an entire cluster (with a report on any problems found) massively reduces stress during difficult times.

Positive reporting is useful. Every day, send an email to someone detailing the performance of the cluster. If that email doesn't turn up, that's a useful clue that something's happened. You'll probably want other early warning systems that'll notify you faster too; [Pingdom](#) is particularly useful here. A “dead man's switch” that reacts to the absence of an event (e.g., [Dead Man's Switch](#)) is another useful backup.

Reporting to the team on the health of the cluster is very useful. This might be an admin page inside a web application, or a separate report. [Ganglia](#) is great for this. Ian has seen a *Star Trek* LCARS-like interface running on a spare PC in an office that plays the “red alert” sound when problems are detected—that's particularly effective at getting the attention of an entire office. We've even seen Arduinos driving analog instruments like old-fashioned boiler pressure gauges (they make a nice sound when the needle moves!) showing system load. This kind of reporting is important so that everyone understands the difference between “normal” and “this might ruin our Friday night!”

Two Clustering Solutions

In this section we introduce IPython Parallel and the concept of messaging platforms.

IPython clusters are easy to use on one machine with multiple cores. Since many researchers use IPython as their shell or work through Jupyter Notebooks, it's natural to also use it for parallel job control. Building a cluster requires a little bit of system administration knowledge. A huge win with IPython Parallel is that you can use remote clusters (Amazon's AWS and EC2, for example) just as easily as local clusters.

Messaging platforms are a critical component of optimizing your cluster solution. These systems are what facilitates communications between all of the components running on your cluster. By default, and traditionally on HPC clusters, IPython Parallel promotes using MPI for this purpose. MPI allows individual tasks on the cluster to send messages to each other, share results, or create synchronization events where multiple tasks wait for each other before proceeding. Depending on how tightly coupled each task is and what your needs are, different message passing platforms can better suit your problem set and help you achieve more optimal parallelization.

Using IPython Parallel to Support Research

The IPython clustering support comes via the **IPython Parallel** project. IPython becomes an interface to local and remote processing engines where data can be pushed among the engines and jobs can be pushed to remote machines. Remote debugging is possible, and the message passing interface (MPI) is optionally supported. This same ZeroMQ communication mechanism powers the Jupyter Notebook interface.

This is great for a research setting—you can push jobs to machines in a local cluster, interact and debug if there's a problem, push data to machines, and collect results back, all interactively. Note also that PyPy runs IPython and IPython Parallel. The combination might be very powerful (if you don't use numpy).

Behind the scenes, ZeroMQ is used as the messaging middleware—be aware that ZeroMQ provides no security by design. If you're building a cluster on a local network, you can avoid SSH authentication. If you need security, SSH is fully supported, but it makes configuration a little more involved—start on a local trusted network and build out as you learn how each component works.

The project is split into four components. An *engine* is an extension of the IPython kernel; it is a synchronous Python interpreter that runs your code. You'll run a set of engines to enable parallel computing. A *controller* provides an interface to the engines; it is responsible for work distribution and supplies a *direct* interface and a *load-balanced* interface that provides a work scheduler. A *hub* keeps track of engines,

schedulers, and clients. *Schedulers* hide the synchronous nature of the engines and provide an asynchronous interface.

On the laptop, we start four engines using `ipcluster start -n 4`. In [Example 11-1](#), we start IPython and check that a local `Client` can see our four local engines. We can address all four engines using `c[:]`, and we apply a function to each engine—`apply_sync` takes a callable, so we supply a zero-argument `lambda` that will return a string. Each of our four local engines will run one of these functions, returning the same result.

Example 11-1. Testing that we can see the local engines in IPython

```
In [1]: import ipyparallel as ipp

In [2]: c = ipp.Client()

In [3]: print(c.ids)
[0, 1, 2, 3]

In [4]: c[:].apply_sync(lambda: "Hello High Performance Pythonistas!")
Out[4]:
['Hello High Performance Pythonistas!',
 'Hello High Performance Pythonistas!',
 'Hello High Performance Pythonistas!',
 'Hello High Performance Pythonistas!']
```

The engines we’ve constructed are now in an empty state. If we import modules locally, they won’t be imported into the remote engines.

A clean way to import both locally and remotely is to use the `sync_imports` context manager. In [Example 11-2](#), we’ll import `os` on both the local IPython and the four connected engines and then call `apply_sync` again on the four engines to fetch their PIDs.

If we didn’t do the remote imports, we’d get a `NameError`, as the remote engines wouldn’t know about the `os` module. We can also use `execute` to run any Python command remotely on the engines.

Example 11-2. Importing modules into our remote engines

```
In [5]: dview=c[:] # this is a direct view (not a load-balanced view)

In [6]: with dview.sync_imports():
....:     import os
....:
importing os on engine(s)

In [7]: dview.apply_sync(lambda: os.getpid())
```

```
Out[7]: [16158, 16159, 16160, 16163]
```

```
In [8]: dview.execute("import sys") # another way to execute commands remotely
```

You'll want to push data to the engines. The push command shown in [Example 11-3](#) lets you send a dictionary of items that are added to the global namespace of each engine. There's a corresponding pull to retrieve items: you give it keys, and it'll return the corresponding values from each of the engines.

Example 11-3. Pushing shared data to the engines

```
In [9]: dview.push({'shared_data':[50, 100]})
```

```
Out[9]: <AsyncResult(_push): pending>
```

```
In [10]: dview.apply_sync(lambda:len(shared_data))
```

```
Out[10]: [2, 2, 2, 2]
```

In [Example 11-4](#), we use these four engines to estimate pi. This time we use the `@require` decorator to import the `random` module in the engines. We use a direct view to send our work out to the engines; this blocks until all the results come back. Then we estimate pi as we did in [Example 10-1](#).

Example 11-4. Estimating pi using our local cluster

```
import time
import ipyparallel as ipp
from ipyparallel import require

@require('random')
def estimate_nbr_points_in_quarter_circle(nbr_estimates):
    ...
    return nbr_trials_in_quarter_unit_circle

if __name__ == "__main__":
    c = ipp.Client()
    nbr_engines = len(c.ids)
    print("We're using {} engines".format(nbr_engines))
    nbr_samples_in_total = 1e8
    nbr_parallel_blocks = 4

    dview = c[:]

    nbr_samples_per_worker = nbr_samples_in_total / nbr_parallel_blocks
    t1 = time.time()
    nbr_in_quarter_unit_circles = \

        dview.apply_sync(estimate_nbr_points_in_quarter_circle,
                          nbr_samples_per_worker)
    print("Estimates made:", nbr_in_quarter_unit_circles)
```

```

nbr_jobs = len(nbr_in_quarter_unit_circles)
pi_estimate = sum(nbr_in_quarter_unit_circles) * 4 / nbr_samples_in_total
print("Estimated pi", pi_estimate)
print("Delta:", time.time() - t1)

```

In [Example 11-5](#) we run this on our four local engines. Similar to the results shown in [Figure 10-5](#), this takes approximately 47 seconds on the laptop.

Example 11-5. Estimating pi using our local cluster in IPython

```

In [1]: %run pi_ipython_cluster.py
We're using 4 engines
Estimates made: [78546259, 78539590, 78541509, 78540021]
Estimated pi 3.14167379
Delta: 47.14534091949463

```

IPython Parallel offers much more than what's shown here. Asynchronous jobs and mappings over larger input ranges are, of course, possible. MPI is supported, which can provide efficient data sharing. The Joblib library introduced in [“Replacing multiprocessing with Joblib” on page 304](#) can use IPython Parallel as a backend along with Dask (which we introduced in [“Dask for Distributed Data Structures and DataFrames” on page 205](#)).

One particularly powerful feature of IPython Parallel is that it allows you to use larger clustering environments, including supercomputers and cloud services like Amazon's EC2. The [ElastiCluster project](#) has support for common parallel environments such as IPython and for deployment targets, including AWS, Azure, and OpenStack.

Message Brokering for Cluster Efficiency

In a production environment, you will need a solution that is more robust than the other solutions we've talked about so far. This is because during the everyday operation of your cluster, nodes may become unavailable, code may crash, networks may go down, or one of the other thousands of problems that can happen may happen. The problem is that all the previous systems have had one computer where commands are issued, and a limited and static number of computers that read the commands and execute them. We would instead like a system that can have multiple actors communicating via a message bus—this would allow us to have an arbitrary and constantly changing number of message creators and consumers.

This type of system is called a *message broker*, and such systems can be key when scaling up a system to robustly handle large or volatile volumes of data. The main message brokering systems that you can run yourself are ActiveMQ, RabbitMQ,

Kafka, and ZeroMQ.¹ Also, most cloud platforms provide their own managed message brokering system (Pub/Sub in Google Cloud and Amazon MQ in AWS). While the full ins and outs of message brokering and the systems architecture surrounding them deserve their own book, we will introduce the two main concepts you should be aware of and try to highlight questions you should ask yourself when trying to decide which brokering system would be best for your application.

Queue

A *queue* is a type of buffer for messages (see [Figure 11-1](#)). Whenever you want to send a message to another part of your processing pipeline, you send it to the queue, and it'll wait there until a worker is available. A queue is most useful in distributed processing when an imbalance exists between production and consumption. When this imbalance occurs, we can simply scale horizontally by adding more data consumers until the message production rate and the consumption rate are equal. In addition, if the computers responsible for consuming messages go down, the messages are not lost but are simply queued until a consumer is available, thus giving us message delivery guarantees.

For example, let's say we would like to process new recommendations for a user every time that user rates a new item on our site. If we didn't have a queue, the “rate” action would directly call the “recalculate-recommendations” action, regardless of how busy the servers dealing with recommendations were. If all of a sudden thousands of users decided to rate something, our recommendation servers could get so swamped with requests that they could start timing out, dropping messages, and generally becoming unresponsive!

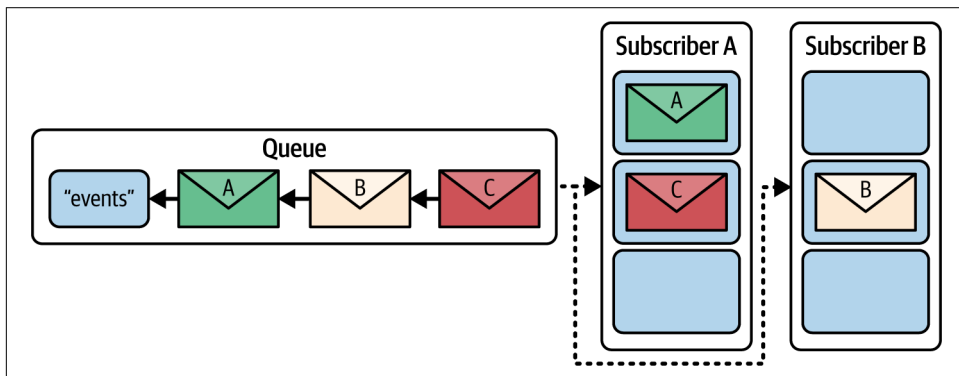


Figure 11-1. Overview of a Queue topology

¹ There is a fantastic write-up about the ins and outs of these systems called “[The System Design Cheat Sheet](#)”.

On the other hand, with a queue, the recommendation servers ask for more tasks when they are ready. A new “rate” action would put a new task on the queue, and when a recommendation server becomes ready to do more work, it would grab the task from the queue and process it. In this setup, if more users than normal start rating items, our queue would fill up and act as a buffer for the recommendation servers—their workload would be unaffected, and they could still process messages until the queue was empty.

Some common points to consider when picking a queue are:

- What happens when tasks are being added more quickly than they are consumed? What about when memory starts running out?
- What happens when the computer running the queue goes offline?
- Can the queue be distributed across multiple nodes? If so, is there redundancy in which queue has which message?
- Does a consumer have to acknowledge that it is done processing a message? What happens if a consumer fails midway while processing a message?
- Do messages expire out of the queue if they have been in there for a long time?
- Is message ordering important?

Pub/sub

A *pub/sub* (short for *publisher/subscriber*), on the other hand, is a collection of queues and describes a specific way to choose who gets what messages. A data publisher can push data out of a particular topic, and data subscribers can subscribe to different feeds of data. Whenever the publisher puts out a piece of information, it gets sent to all the subscribers—each gets an identical copy of the original information. You can think of this like a newspaper: many people can subscribe to a particular newspaper, and whenever a new edition of the newspaper comes out, every subscriber gets an identical copy of it. In addition, the producer of the newspaper doesn’t need to know all the people its papers are being sent to. As a result, publishers and subscribers are decoupled from each other, which allows our system to be more robust as our network changes while still in production.

There is also the concept of a *data consumer* that is a logical grouping of tasks that receive messages on a particular topic. In a pub/sub, whenever a new piece of data comes out, every subscriber gets a copy of the data; however, only one consumer of each subscription sees that data. In the newspaper analogy, you can think of this as having multiple people in the same household who read the newspaper. The publisher will deliver one paper to the house, since that house has only one subscription, and whoever in the house gets to it first gets to read that data. Each subscriber’s consumers do the same processing to a message when they see it; however, they can

potentially be on multiple computers and thus add more processing power to the entire pool.

We can see a depiction of this pub/sub/consumer paradigm in [Figure 11-2](#). If a new message gets published on the “events” topic, all the subscribers will get a copy. Each subscriber is composed of one or more consumers, which represent actual processes that react to the messages. In the case of the “metrics” subscriber, only one consumer will see the new message. The next message will go to another consumer, and so on.

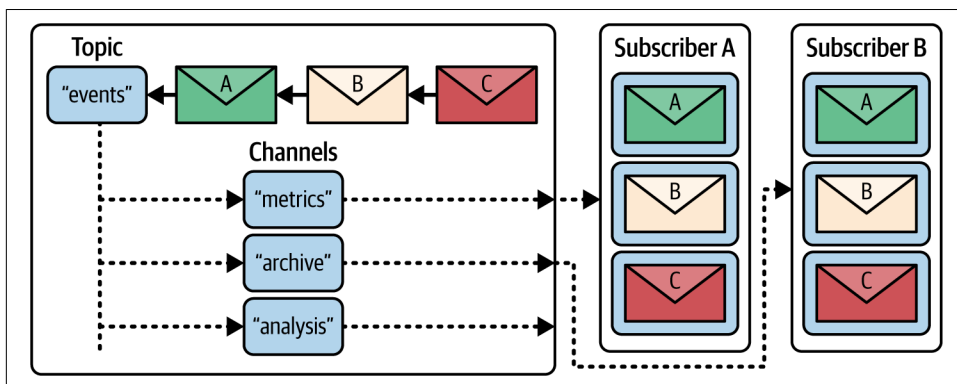


Figure 11-2. Overview of pub/sub-like topology

The benefit of spreading the messages among a potentially large pool of consumers is essentially automatic load balancing. If a message takes quite a long time to process, that consumer will not signal to the broker that it is ready for more messages until it’s done, and thus the other consumers will get the majority of future messages (until that original consumer is ready to process again). In addition, it allows existing consumers to disconnect (whether by choice or because of failure) and new consumers to connect to the cluster while still maintaining processing power within a particular subscription group. For example, if we find that “metrics” takes quite a while to process and often is not keeping up with demand, we can simply add more processes to the consumer pool for that subscription group, giving us more processing power. On the other hand, if we see that most of our processes are idle (i.e., not getting any messages), we can easily remove consumers from this subscription pool.

It is also important to note that anything can publish data. A consumer doesn’t simply need to be a consumer—it can consume data from one topic and then publish it to another topic. In fact, this chain is an important workflow when it comes to this paradigm for distributed computing. Consumers will read from a topic of data, transform the data in some way, and then publish the data onto a new topic that other consumers can further transform. In this way, different topics represent different data, subscription groups represent different transformations on the data, and consumers are the actual workers who transform individual messages.

Pub/sub systems have the same considerations as simple queues and then more surrounding how messages are distributed. Some of these common questions you should be asking are:

- What are the message guarantees? For example, are the messages guaranteed to be delivered at least once? At most once? Are they guaranteed to be processed as well?
- How do the message guarantees change at a topic level? At a consumer level?
- How does the system support multiple consumers of the same message topic?
- Is archiving of messages easy with the broker? Would your system handle replaying hold messages?
- How can topics and consumers be distributed to manage resources? How well does the system handle various levels of system outages?
- Are there any single points of failure?

Other Clustering Tools to Look At

Job processing systems using queues have existed since the start of the computer science industry, back when computers were very slow and lots of jobs needed to be processed. As a result, there are *many* libraries for queues, and many of these can be used in a cluster configuration. We strongly suggest that you pick a mature library with an active community behind it and supporting the same feature set that you'll need without too many additional features.

The more features a library has, the more ways you'll find to misconfigure it and waste time on debugging. Simplicity is *generally* the right aim when dealing with clustered solutions. Here are a few of the more commonly used clustering solutions:

- **ZeroMQ** is a low-level and performant messaging library that enables you to send messages between nodes. It natively supports pub/sub paradigms and can also communicate over multiple types of transports (TCP, UDP, WebSocket, etc.). It is quite low-level and doesn't provide many useful abstractions, which can make its use a bit difficult. That being said, it's in use in Jupyter, Auth0, Spotify, and many more places!
- **Celery** (BSD license) is a widely used asynchronous task queue using a distributed messaging architecture, written in Python. It supports Python, PyPy, and Jython. It typically uses RabbitMQ as the message broker, but it also supports Redis, MongoDB, and other storage systems. It is often used in web development projects.

- **Airflow** and **Luigi** use directed acyclic graphs to chain dependent jobs into sequences that run reliably, with monitoring and reporting services. They're widely used in industry for data science tasks, and we recommend reviewing these before you embark on a custom solution.
- **Amazon's Simple Queue Service (SQS)** is a job processing system integrated into AWS. Job consumers and producers can live inside AWS or can be external, so SQS is easy to start with and supports easy migration into the cloud. Library support exists for many languages.

Docker

Docker is a tool of general importance in the Python ecosystem, but the problems it solves are especially important when dealing with a large team or a cluster. Docker helps to create reproducible environments in which to run your code, share/control runtime environments, easily share runnable code between team members, and deploy code to a cluster of nodes based on resource needs.

Docker's Performance

One common misconception about Docker is that it substantially slows down the runtime performance of the applications it is running. While this can be true in some cases, it generally is not. Furthermore, most of the performance degradations can almost always be removed with some easy configuration changes.

In terms of CPU and memory access, Docker (and all other container-based solutions) will *not* lead to any performance degradations. This is because Docker simply creates a special namespace within the host operating system where the code can run normally, albeit with separate constraints from other running programs. Essentially, Docker code accesses the CPU and memory in the same way that every other program on the computer does; however, it can have a separate set of configuration values to fine-tune resource limits.²

This is because Docker is an instance of OS-level virtualization, as opposed to hardware virtualization such as VMware or VirtualBox. With hardware virtualization, software runs on “fake” hardware that introduces overhead accessing all resources. On the other hand, OS virtualization uses the native hardware but runs on a “fake” operating system. Thanks to the cgroups Linux feature, this “fake” operating system can be tightly coupled to the running operating system, which gives the possibility of running with almost no overhead.

² This fine-tuning can, for example, be used to adjust the amount of memory a process has access to, or which CPUs or even how much of the CPU it can use.



cgroups is specifically a feature in the Linux kernel. As a result, performance implications discussed here are restricted to Linux systems. In fact, to run Docker on macOS or Windows, we first must run the Linux kernel in a hardware-virtualized environment. Docker Machine, the application helping streamline this process, uses VirtualBox to accomplish this. As a result, you will see performance overhead from the hardware-virtualized portion of the process. This overhead will be greatly reduced when running on a Linux system, where hardware virtualization is not needed.

As an example, let's create a simple Docker container to run the 2D diffusion code from [Example 6-16](#). As a baseline, we can run the code on the host system's Python to get a benchmark:

```
$ python diffusion_numpy_memory2.py
Runtime for 100 with grid (256, 256): 0.8593s
```

To create our Docker container, we have to make a directory that contains the Python file `diffusion_numpy_memory2.py`, a `pip` requirements file for dependencies, and a `Dockerfile`, as shown in [Example 11-6](#).

Example 11-6. Simple Docker container

```
$ ls
diffusion_numpy_memory2.py
Dockerfile
requirements.txt

$ cat requirements.txt
numpy>=2.1.1

$ cat Dockerfile
FROM python:3.12-slim

WORKDIR /usr/src/app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt ❶

COPY . .
CMD ["python", "./diffusion_numpy_memory2.py"] ❷
```

- ❶ Here we chose to simply disable the `pip` cache. Every time this runs, we will have to redownload the `numpy` package. Alternatively, we can utilize Docker's specialized build cache to specifically reuse `pip`'s cache when we want to. This is done by replacing this line with `RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt`.

- ② We can also specify the command here as we normally would in the shell (i.e., `CMD python "./diffusion_numpy_memory2.py"`). However, specifying it as a JSON array of arguments helps guard against unintended behavior regarding system signals like SIGKILL or SIGINT.

The *Dockerfile* starts by stating what container we'd like to use as our base. These base containers can be a wide selection of Linux-based operating systems or a higher-level service. The Python Foundation provides **official containers for all major Python versions**, which makes selecting the Python version you'd like to use incredibly simple. Next, we define the location of our working directory (the selection of `/usr/src/app` is arbitrary), copy our requirements file into it, and begin setting up our environment as we normally would on our local machine, using `RUN` commands.

One major difference between setting up your development environment normally and on Docker is the `COPY` commands. They copy files from the local directory into the container. For example, the *requirements.txt* file is copied into the container so that it is there for the `pip install` command. Finally, at the end of the *Dockerfile*, we copy all the files from the current directory into the container and tell Docker to run `python ./diffusion_numpy_memory2.py` when the container starts.



In the *Dockerfile* in **Example 11-6**, beginners often wonder why we first copy only the requirements file and then later copy the entire directory into the container. When building a container, Docker tries hard to cache each step in the build process. To determine whether the cache is still valid, the contents of the files being copied back and forth are checked. By first copying only the requirements file and *then* moving the rest of the directory, we will have to run `pip install` only if the requirements file changes. If only the Python source has changed, a new build will use cached build steps and skip straight to the second `COPY` command.

Now we are ready to build and run the container, which can be named and tagged. Container names generally take the format `<username>/<project-name>`,³ while the optional tag generally is either descriptive of the current version of the code or simply the tag `latest` (this is the default and will be applied automatically if no tag is specified). To help with versioning, it is general convention to always tag the most recent build with `latest` (which will get overwritten when a new build is made) as well as a descriptive tag so that we can easily find this version again in the future:

³ The username portion of the container name is useful when also pushing built containers to a repository.

```

$ docker build -t high_performance/diffusion2d:numpy-memory2 \
               -t high_performance/diffusion2d:latest
[+] Building 49.1s (10/10) FINISHED                                docker:default
[internal] load build definition from Dockerfile                  1.1s
=> transferring dockerfile: 209B                                  0.0s
[internal] load metadata for docker.io/library/python:3.12-slim  0.5s
[internal] load .dockerignore                                     1.0s
=> transferring context: 2B                                       0.0s
[1/5] FROM docker.io/library/python:3.12-slim@sha256:83b33a91f39dc9ec2d02c 17.2s
=> resolve docker.io/library/python:3.12-slim@sha256:83b33a91f39dc9ec2d02ca 1.7s
=> sha256:10f3aaab98db50cba827d3b33a91f39dc9ec2d02ca98 9.12kB / 9.12kB 0.0s
=> sha256:d0af71d7d6d1b7bb018395aca582e4d270d090ca4134 1.75kB / 1.75kB 0.0s
=> sha256:d8fb19ba66726ea7ddb6c6d9071d54a1bd932e60b06a 5.17kB / 5.17kB 0.0s
=> sha256:ad5b6a1962fdac425252ec07daae642d785bd6d9c9fa 3.32MB / 3.32MB 0.4s
=> sha256:de2a998baeb3b693572201ce82da66e97b26f2feabfa 13.65MB / 13.65MB 1.5s
=> sha256:184aff3eb8fbdeb25b5697c83adcfadb6deef75d7006 249B / 249B 0.7s
=> extracting sha256:ad5b6a1962fdac425252ec07daae642d9c32fb6c7346 3.3s
=> extracting sha256:de2a998baeb3b693572201ce82da66e9b952587a839d 2.1s
=> extracting sha256:184aff3eb8fbdeb25b5697c83adcfadb0ec2d6a610a4 0.0s
[internal] load build context                                     2.0s
=> transferring context: 318B                                     0.0s
[2/5] WORKDIR /usr/src/app                                       3.1s
[3/5] COPY requirements.txt ./                                    3.4s
[4/5] RUN pip install --no-cache-dir -r requirements.txt         12.2s
[5/5] COPY . .                                                   3.8s
exporting to image                                               3.6s
=> exporting layers                                              3.4s
=> writing image sha256:96a8eb92e71f0edb8ed772e1bb52bca35724a022cb52705895 0.0s
=> naming to docker.io/high_performance/diffusion2d:numpy-memory2 0.0s
=> naming to docker.io/high_performance/diffusion2d:latest      0.0s

$ docker run high_performance/diffusion2d:numpy-memory2
Runtime for 100 with grid (256, 256): 0.8422s

```

We can see that at its core, Docker is not slower than running on the host machine in any meaningful way when the task relies mainly on CPU/memory. However, as with anything, there is no free lunch, and at times Docker performance suffers. While a full discussion of optimizing Docker containers is outside the scope of this book, we offer the following list of considerations for when you are creating Docker containers for high performance code:

- Be wary of copying too much data into a Docker container or even having too much data in the same directory as a Docker build. If the build context, as advertised by the first line of the `docker build` command, is too large, performance can suffer (remedy this with a `.dockerignore` file).

- Docker uses various filesystem tricks to layer filesystems on top of each other. This helps the caching of builds but can be slower than dealing with the host filesystem. Use host-level mounts when you need to access data quickly, and consider using volumes set as read-only to choose a volume driver that is fitting for your infrastructure.
- Docker creates a virtual network for all your containers to live behind. This can be great for keeping most of your services hidden behind a gateway, but it also adds slight network overhead. For most use cases, this overhead is negligible, but it can be mitigated by changing the network driver.
- GPUs and other host-level devices can be accessed using special runtime drivers for Docker. For example, `nvidia-docker` allows Docker environments to easily use connected NVIDIA GPUs. In general, devices can be made available with the `--device` runtime flag.

As always, it is important to profile your Docker containers to know what the issues are and whether there are some easy wins in terms of efficiency. The `docker stats` command provides a good high-level view to help you understand the current runtime performance of your containers.

Advantages of Docker

So far it has seemed that Docker is simply adding a whole new host of issues to contend with in terms of performance. However, the gains to reproducibility and reliability of runtime environments far surpass any extra complexity.

Locally, having access to all our previously run Docker containers allows us to quickly rerun and retest previous versions of our code without having to worry about changes to the runtime environment, such as dependencies and system packages ([Example 11-7](#) shows a list of containers we can run with a simple `docker run` command). This makes it incredibly easy to constantly be testing for performance regressions that otherwise would be difficult to reproduce.

Example 11-7. Docker tags to keep track of previous runtime environments

```
$ docker images -a
```

REPOSITORY	TAG	IMAGE ID
highperformance/diffusion2d	latest	ceabe8b555ab
highperformance/diffusion2d	numpy-memory2	ceabe8b555ab
highperformance/diffusion2d	numpy-memory1	66523a1a107d
highperformance/diffusion2d	python-memory	46381a8db9bd
highperformance/diffusion2d	python	4cac9773ca5e

Many more benefits come with the use of a [container registry](#), which allows the storage and sharing of Docker images with the simple `docker pull` and `docker push`

commands, in a similar way to `git`. This lets us put all our containers in a publicly available location, allowing team members to pull in changes or new versions and letting them immediately run the code.

This book is a great example of the benefits of sharing a Docker container for standardizing a runtime environment. To convert this book from `asciidoc`, the markup language it was written in, into PDF, a Docker container was shared between us so we could reliably and reproducibly build book artifacts. This standardization saved us countless hours that were spent in the first edition, as one of us would have a build issue that the other couldn't reproduce or help debug.

Running `docker pull highperformance/diffusion2d:latest` is much easier than having to clone a repository and doing all the associated setup that may be necessary to run a project. This is particularly true for research code, which may have some very fragile system dependencies. Having all of this inside an easily pullable Docker container means all of these setup steps can be skipped and the code can be run easily. As a result, code can be shared more easily, and a coding team can work more effectively together.

Finally, in conjunction with `kubernetes` and other similar technologies, Dockerizing your code helps with actually running it with the resources it needs. Kubernetes allows you to create a cluster of nodes, each labeled with resources it may have, and to orchestrate running containers on the nodes. It will take care of making sure the correct number of instances are being run, and thanks to the Docker virtualization, the code will be run in the same environment that you saved it to. One of the biggest pains of working with a cluster is making sure that the cluster nodes have a compatible runtime environment as your workstation, and using Docker virtualization completely resolves this.⁴

Wrap-Up

So far in the book, we've looked at profiling to understand slow parts of your code, compiling and using `numpy` to make your code run faster, and various approaches to multiple processes and computers. In addition, we surveyed container virtualization to manage code environments and help in cluster deployment. In the penultimate chapter, we'll look at ways of using less RAM through different data structures and probabilistic approaches. These lessons could help you keep all your data on one machine, avoiding the need to run a cluster.

⁴ A great tutorial to get started with Docker and Kubernetes is "[Get started with Kubernetes \(using Python\)](#)" on the Kubernetes blog.

Using Less RAM

Questions You'll Be Able to Answer After This Chapter

- Why should I use less RAM?
- Why are `numpy` and `array` better for storing lots of numbers?
- How can lots of text be efficiently stored in RAM?
- How could I count (approximately!) to 10^{76} using just 1 byte?
- What are Bloom filters, and why might I need them?

We rarely think about how much RAM we're using until we run out of it. If you run out while scaling your code, it can become a sudden blocker. Fitting more into a machine's RAM means fewer machines to manage, and it gives you a route to planning capacity for larger projects. Knowing why RAM gets eaten up and considering more efficient ways to use this scarce resource will help you deal with scaling issues. We'll use the Memory Profiler and IPython Memory Usage tools to measure the actual RAM usage, along with some tools that introspect objects to try to guess how much RAM they're using.

Another route to saving RAM is to use containers that utilize features in your data for compression. In this chapter, we'll look at a trie (ordered tree data structure) that can compress a 1.2 GB set of strings down to just 39 MB with little change in performance. A third approach is to trade storage for accuracy. For this we'll look at approximate counting and approximate set membership, which use dramatically less RAM than their exact counterparts.

A consideration with RAM usage is the notion that “data has mass.” The more there is of it, the slower it moves around. If you can be parsimonious in your use of RAM, your data will probably get consumed faster, as it’ll move around buses faster and more of it will fit into constrained caches. If you need to store it in offline storage (e.g., a hard drive or a remote data cluster), it’ll move far more slowly to your machine. Try to choose appropriate data structures so all your data can fit onto one machine. We’ll use NumExpr to efficiently calculate with NumPy and Pandas with fewer data movements than the more direct method, which will save us time and make certain larger calculations feasible in a fixed amount of RAM.

Counting the amount of RAM used by Python objects is surprisingly tricky. We don’t necessarily know how an object is represented behind the scenes, and if we ask the operating system for a count of bytes used, it will tell us about the total amount allocated to the process. In both cases, we can’t see exactly how each individual Python object adds to the total.

As some objects and libraries don’t report their full internal allocation of bytes (or they wrap external libraries that do not report their allocation at all), this has to be a case of best-guessing. The approaches explored in this chapter can help us to decide on the best way to represent our data so we use less RAM overall.

We’ll also look at several lossy methods for storing strings in scikit-learn and counts in data structures. This works a little like a JPEG compressed image—we lose some information (and we can’t undo the operation to recover it), and we gain a lot of compression as a result. By using hashes on strings, we compress the time and memory usage for a natural language processing task in scikit-learn, and we can count huge numbers of events with only a small amount of RAM.

Objects for Primitives Are Expensive

It’s common to work with containers like the `list`, storing hundreds or thousands of items. As soon as you store a large number, RAM usage becomes an issue.

A `list` with 100 million items consumes approximately 760 MB of RAM, *if the items are the same object*. If we store 100 million *different* items (e.g., unique integers), we can expect to use gigabytes of RAM! Each unique object has a memory cost.

In [Example 12-1](#), we store many `0` integers in a `list`. If you stored 100 million references to any object (regardless of how large one instance of that object was), you’d still expect to see a memory cost of roughly 760 MB, as the `list` is storing references to (not copies of) the object. Refer back to [“Using `memory\_profiler` to Diagnose Memory Usage” on page 53](#) for a reminder of how to use `memory_profiler`; here, we load it as a new magic function in IPython using `%load_ext memory_profiler`.

Example 12-1. Measuring memory usage of 100 million of the same integer in a list

```
In [1]: %load_ext memory_profiler # load the %memit magic function
In [2]: %memit [0] * int(1e8)
peak memory: 806.33 MiB, increment: 762.77 MiB
```

For our next example, we'll start with a fresh shell. As the results of the first call to `memit` in [Example 12-2](#) reveal, a fresh IPython shell consumes approximately 40 MB of RAM. Next, we can create a temporary list of 100 million *unique* numbers. In total, this consumes approximately 3.8 GB.



Memory can be cached in the running process, so it is always safer to exit and restart the Python shell when using `memit` for profiling.

After the `memit` command finishes, the temporary list is deallocated. The final call to `memit` shows that the memory usage drops to its previous level.

Example 12-2. Measuring memory usage of 100 million different integers in a list

```
# we use a new IPython shell so we have a clean memory
In [1]: %load_ext memory_profiler
In [2]: %memit # show how much RAM this process is consuming right now
peak memory: 43.39 MiB, increment: 0.11 MiB
In [3]: %memit [n for n in range(int(1e8))]
peak memory: 3850.29 MiB, increment: 3806.59 MiB
In [4]: %memit
peak memory: 44.79 MiB, increment: 0.00 MiB
```

A subsequent `memit` in [Example 12-3](#) to create a second 100-million-item list consumes approximately 3.8 GB again.

Example 12-3. Measuring memory usage again for 100 million different integers in a list

```
In [5]: %memit [n for n in range(int(1e8))]
peak memory: 3855.78 MiB, increment: 3810.96 MiB
```

Next, we'll see that we can use the `array` module to store 100 million integers far more cheaply.

The array Module Stores Many Primitive Objects Cheaply

The `array` module efficiently stores primitive types like integers, floats, and characters, but *not* complex numbers or classes. It creates a contiguous block of RAM to hold the underlying data.



Typically a long number costs 8 bytes. Python's specification says that it has to be a minimum of 4 bytes—your architecture may use a different amount of storage than what we show here. Always profile your environment if you want to know the truth about your own situation. A GenAI solution may make poor recommendations for you, as it may not know about local architectural differences from the data in its training set.

In [Example 12-4](#), we allocate 100 million integers (8 bytes each) into a contiguous chunk of memory. In total, approximately 760 MB is consumed by the process. The difference between this approach and the previous list-of-unique-integers approach is $3100\text{MB} - 760\text{MB} = 2.3\text{GB}$. This is a huge savings in RAM.

Example 12-4. Building an array of 100 million integers with 760 MB of RAM

```
In [1]: %load_ext memory_profiler
In [2]: import array
In [3]: %memit array.array('l', range(int(1e8)))
peak memory: 837.88 MiB, increment: 761.39 MiB
In [4]: arr = array.array('l')
In [5]: arr.itemsize
Out[5]: 8
```

Note that the unique numbers in the array are *not* Python objects; they are bytes in the array. If we were to dereference any of them, a new Python `int` object would be constructed. If you're going to compute on them, no overall savings will occur, but if you're instead going to pass the array to an external process or use only some of the data, you should see a good savings in RAM compared to using a `list` of integers.



If you're working with a large array or matrix of numbers with Cython and you don't want an external dependency on `numpy`, be aware that you can store your data in an `array` and pass it into Cython for processing without any additional memory overhead.

The array module works with a limited set of datatypes with varying precisions (see [Example 12-5](#)). Choose the smallest precision that you need, so that you allocate just as much RAM as needed and no more. Be aware that the byte size is platform-dependent—the sizes here refer to a 32-bit platform (it states *minimum* size), whereas we’re running the examples on a 64-bit laptop.

Example 12-5. The basic types provided by the array module

```
In [5]: array.array? # IPython magic, similar to help(array)
Init signature: array.array(self, /, *args, **kwargs)
Docstring:
array(typecode [, initializer]) -> array
```

Return a new array whose items are restricted by typecode, and initialized from the optional initializer value, which must be a list, string, or iterable over elements of the appropriate type.

Arrays represent basic values and behave very much like lists, except the type of objects stored in them is constrained. The type is specified at object creation time by using a type code, which is a single character. The following type codes are defined:

Type code	C Type	Minimum size in bytes
'b'	signed integer	1
'B'	unsigned integer	1
'u'	Unicode character	2 (see note)
'h'	signed integer	2
'H'	unsigned integer	2
'i'	signed integer	2
'I'	unsigned integer	2
'l'	signed integer	4
'L'	unsigned integer	4
'q'	signed integer	8 (see note)
'Q'	unsigned integer	8 (see note)
'f'	floating point	4
'd'	floating point	8

NumPy has arrays that can hold a wider range of datatypes—you have more control over the number of bytes per item, and you can use complex numbers and datetime objects. A complex128 object takes 16 bytes per item; each item is a pair of 8-byte floating-point numbers. You can’t store complex objects in a Python array, but they come for free with numpy. If you’d like a refresher on numpy, look back to [Chapter 6](#).

In [Example 12-6](#), you can see an additional feature of numpy arrays: you can query for the number of items, the size of each primitive, and the combined total storage of the underlying block of RAM. Note that this doesn’t include the overhead of the Python object (this is typically tiny in comparison to the data you store in the arrays).



Be wary of lazy allocation with zeros. In the following example the call to zeros costs “zero” RAM, while the call to ones costs 1.5 GB. Both calls will ultimately cost 1.5 GB, but the call to zeros will allocate the RAM only after it is used, so the cost is seen later.

Example 12-6. Storing more complex types in a numpy array

```
In [1]: %load_ext memory_profiler
In [2]: import numpy as np
# NOTE that zeros have lazy allocation so misreport the memory used!
In [3]: %memit arr=np.zeros(int(1e8), np.complex128)
peak memory: 58.37 MiB, increment: 0.00 MiB
In [4]: %memit arr=np.ones(int(1e8), np.complex128)
peak memory: 1584.41 MiB, increment: 1525.89 MiB
In [5]: f"{arr.size:,}"
Out[5]: '100,000,000'
In [6]: f"{arr.nbytes:,}"
Out[6]: '1,600,000,000'
In [7]: arr.nbytes/arr.size
Out[7]: 16.0
In [8]: arr.itemsize
Out[8]: 16
```

Using a regular list to store many numbers is much less efficient in RAM than using an array object. More memory allocations have to occur, which each take time; calculations also occur on larger objects, which will be less cache friendly, and more RAM is used overall, so less RAM is available to other programs.

However, if you do any work on the contents of the array using Python numeric objects, the primitives are likely to be converted into temporary objects, negating their benefit. Using them as a data store when communicating with other processes is a great use case for the array.

numpy arrays are almost certainly a better choice if you are doing anything heavily numeric, as you get more datatype options and many specialized and fast functions. You might choose to avoid numpy if you want fewer dependencies for your project, though Cython works equally well with array and numpy arrays; Numba works with numpy arrays only.

Python provides a few other tools to understand memory usage, as we'll see in the following section.

Using Less RAM in NumPy with NumExpr

Large vectorized expressions in NumPy (which also happen behind the scenes in Pandas) can create intermediate large arrays during complex operations. These occur invisibly and may draw attention to themselves only when an out-of-memory error occurs. These calculations can also be slow, as large vectors will not be cache friendly—a cache can be megabytes or smaller, and large vectors of hundreds of megabytes or gigabytes of data will stop the cache from being used effectively. NumExpr is a tool that both speeds up and reduces the size of intermediate operations; we introduced it in “[numexpr: Making In-Place Operations Faster and Easier](#)” on page 156.

We’ve also introduced the Memory Profiler before, in “[Using memory_profiler to Diagnose Memory Usage](#)” on page 53. Here, we build on it with the [IPython Memory Usage tool](#), which reports line-by-line memory changes, time cost, and CPU usage summary inside the IPython shell or in a Jupyter Notebook. Let’s look at how these can be used to check that NumExpr is generating a result more efficiently.



Remember to install the optional NumExpr when using Pandas. If NumExpr is installed in Pandas, calls to `eval` will run more quickly—but note that Pandas does not tell you if you *haven’t* installed NumExpr.

We’ll use the cross entropy formula to calculate the error for a machine learning classification challenge. *Cross entropy* (or *Log Loss*) is a common metric for classification challenges; it penalizes large errors significantly more than small errors. Each row in a machine learning problem needs to be scored during the training and prediction phases:

$$-\log P(yt|yp) = -(yt \log(yp) + (1 - yt) \log(1 - yp))$$

We’ll use random numbers in the range $[0, 1]$ here to simulate the results of a machine learning system from a package like scikit-learn or TensorFlow. [Figure 12-1](#) shows the natural logarithm for the range $[0, 1]$ on the right, and on the left it shows the result of calculating the cross entropy for any probability if the target is either 0 or 1.

If the target `yt` is 1, the first half of the formula is active and the second half goes to zero. If the target is 0, the second half of the formula is active and the first part goes to zero. This result is calculated for every row of data that needs to be scored and often for many iterations of the machine learning algorithm.

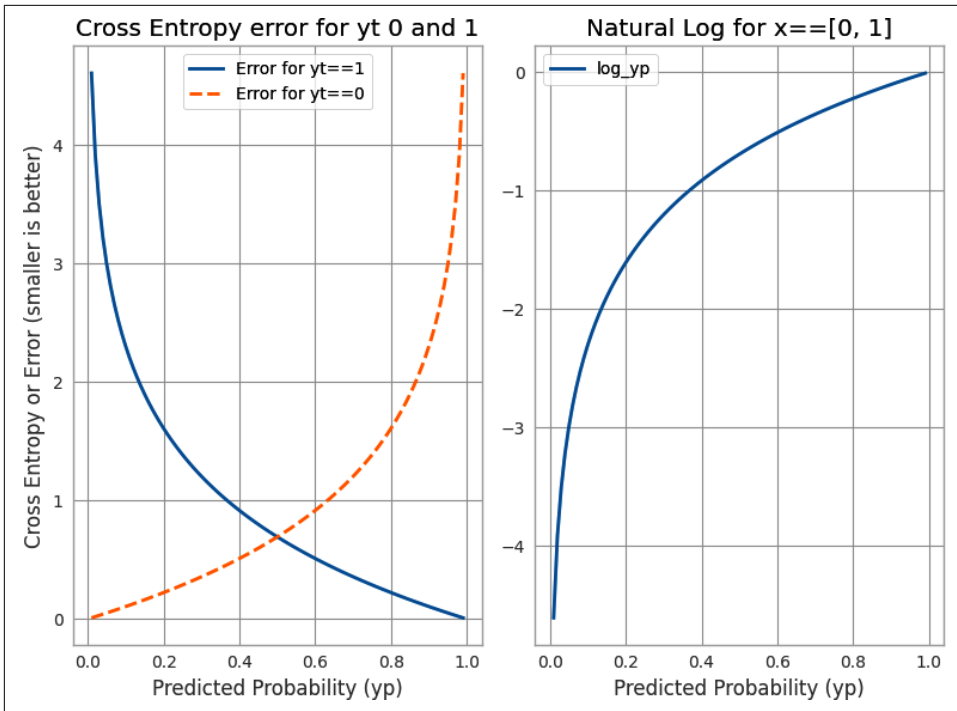


Figure 12-1. Cross entropy for y_t (the “truth”) with values 0 and 1

In [Example 12-7](#), we generate 200 million random numbers in the range [0, 1] as `yp`. `yt` is the desired truth—in this case, an array of 1s. In a real application, we’d see `yp` generated by a machine learning algorithm, and `yt` would be the ground truth mixing 0s and 1s for the target we’d be learning provided by the machine learning researcher.

Example 12-7. The hidden cost of temporaries with large NumPy arrays

```
In [1]: import numpy as np
In [2]: %load_ext ipython_memory_usage
Enabling IPython Memory Usage, use %imu_start to begin, %imu_stop to end
In [3]: %imu_start
Out[3]: 'IPython Memory Usage started'
In [4]: nbr_items = 200_000_000

In [5]: yp = np.random.uniform(low=0.0000001, size=nbr_items)
In [5]: used 1526.1 MiB RAM in 2.08s (system mean cpu 7%, single max cpu 100%),
        peaked 0.0 MiB above final usage, current RAM usage now 1595.5 MiB

In [6]: yt = np.ones(shape=nbr_items)
In [6]: used 1525.9 MiB RAM in 0.46s (system mean cpu 5%, single max cpu 100%),
        peaked 0.0 MiB above final usage, current RAM usage now 3121.5 MiB
```

```
In [7]: answer = -(yt * np.log(yp) + ((1-yt) * (np.log(1-yp))))
In [7] used 1526.6 MiB RAM in 10.13s (system mean cpu 7%, single max cpu 100%),
        peaked 4550.3 MiB above final usage, current RAM usage now 4648.1 MiB

In [8]: del answer
In [8] used -1525.8 MiB RAM in 0.11s (system mean cpu 0%, single max cpu 0%),
        peaked 0.0 MiB above final usage, current RAM usage now 3122.3 MiB
```

Both `yp` and `yt` take 1.5 GB each, bringing the total RAM usage to just over 3.1 GB. The `answer` vector has the same dimension as the inputs and thus adds a further 1.5 GB. Note that the calculation peaks at 4.5 GB over the current RAM usage, so while we end with a 4.6 GB result, we had over 9 GB allocated during the calculation. The cross entropy calculation creates several temporaries (notably `1 - yt`, `np.log(1 - yp)`, and their multiplication). If you had an 8 GB machine, you'd have failed to calculate this result because of memory exhaustion.

The allocation of `yp` and `yt` takes not just RAM but time. Generating the random sequence for `yp` costs 2 seconds and uses 1 CPU, as the default generator in `numpy` is single-threaded. In brackets we see (system mean cpu 7%, single max cpu 100%); this indicates that one CPU was fully utilized at 100% and the mean over 16 CPUs on this laptop is 7%, which is roughly equivalent to one-sixteenth of the reported CPUs. `yt` is generated in 0.4 seconds. We can see that the `answer` calculation is executed on only a single thread as the system mean CPU usage stays at 7%. Once the `answer` variable is free (`del answer`) RAM usage drops by 1.5 GB.

In [Example 12-8](#), we see the same expression placed as a string inside `numexpr.evaluate`. It peaks at 0 GB above the current usage—it doesn't need any additional RAM in this case. Significantly, it also calculates much more quickly: the previous direct vector calculation in `In[7]` took 10 seconds, while here with `NumExpr` the same calculation takes 0.6 seconds. Here we see `system mean cpu 75%`, indicating that the calculation has occurred in parallel over the majority of the cores in the machine.

`NumExpr` breaks the long vectors into shorter, cache-friendly chunks and processes these in parallel where possible, so local chunks of results are calculated in a cache-friendly way. This explains both the requirement for no extra RAM and the increased speed.

Example 12-8. NumExpr breaks the vectorized calculations into cache-efficient chunks

```
In [9]: import numexpr
In [9] used 1.1 MiB RAM in 0.12s (system mean cpu 2%, single max cpu 2%),
        peaked 0.0 MiB above final usage, current RAM usage now 3123.5 MiB

In [10]: answer = numexpr.evaluate("-(yt * log(yp) + ((1-yt) * (log(1-yp))))")
In [10] used 1526.5 MiB RAM in 0.60s (system mean cpu 75%, single max cpu 100%),
        peaked 0.0 MiB above final usage, current RAM usage now 4650.0 MiB
```

We can see a similar benefit in Pandas in [Example 12-9](#). We construct a DataFrame with the same items as in the preceding example and calculate directly in Pandas, which is slower at over 3 seconds, and invoke NumExpr by using `df.eval`, which is faster at 1.7 seconds. The Pandas machinery has to unpack the DataFrame for NumExpr, and more RAM is used overall; behind the scenes, NumExpr is still calculating the result in a cache-friendly manner. Note that here NumExpr was installed in addition to Pandas.

Example 12-9. Pandas eval uses NumExpr if it is available

```
In [1]: %load_ext ipython_memory_usage
In [2]: %imu_start
In [3]: import pandas as pd; import numpy as np
In [4]: nbr_items = 200_000_000
# (output clipped)
In [5]: df = pd.DataFrame({'yp': np.random.uniform(low=0.0000001, size=nbr_items),
                          'yt': np.ones(nbr_items)})
In [5] used 3052.6 MiB RAM in 3.30s (system mean cpu 7%, single max cpu 100%),
      peaked 2943.4 MiB above final usage, current RAM usage now 3153.8 MiB
In [6]: answer_direct = -(df.yt * np.log(df.yp) + ((1-df.yt) * (np.log(1-df.yp))))
In [6] used 1526.7 MiB RAM in 3.28s (system mean cpu 48%, single max cpu 100%),
      peaked 4449.5 MiB above final usage, current RAM usage now 4680.4 MiB
In [7]: answer_eval = df.eval("-(yt * log(yp) + ((1-yt) * (log(1-yp))))")
In [7] used 3052.1 MiB RAM in 1.78s (system mean cpu 32%, single max cpu 100%),
      peaked 2973.5 MiB above final usage, current RAM usage now 7732.6 MiB
```

Contrast the preceding example with [Example 12-10](#), where NumExpr has *not* been installed. The call to `df.eval` falls back on the Python interpreter—the same result is calculated but with an 8-second execution time (compared to 1.7 seconds before) and a much larger peak memory usage. You can test whether NumExpr is installed with `import numexpr`—if this fails, you’ll want to install it.

Example 12-10. Beware that Pandas without NumExpr makes a slow and costly call to eval!

```
...
In [3]: answer_eval = df.eval("-(yt * log(yp) + ((1-yt) * (log(1-yp))))")
In [3] used 3051.8 MiB RAM in 8.63s (system mean cpu 8%, single max cpu 100%),
      peaked 7605.9 MiB above final usage, current RAM usage now 7731.4 MiB
```

Complex vector operations on large arrays will run faster if you can use NumExpr. Pandas will not warn you that NumExpr hasn’t been installed, so we recommend that you add it as part of your setup if you use `eval`. The IPython Memory Usage tool will help you to diagnose where your RAM is being spent if you have large arrays to process; this can help you fit more into RAM on your current machine so you don’t have to start dividing your data and introducing greater engineering effort.

Understanding the RAM Used in a Collection

You may wonder if you can ask Python about the RAM that's used by each object. Python's `sys.getsizeof(obj)` call will tell us *something* about the memory used by an object (most but not all objects provide this). If you haven't seen it before, be warned that it won't give you the answer you'd expect for a container!

Let's start by looking at some primitive types. An `int` in Python is a variable-sized object of arbitrary size, well above the range of an 8-byte C integer. The basic `int` object costs 28 bytes in Python 3.12 when initialized with 0. More bytes are added as you count to larger numbers:

```
In [1]: sys.getsizeof(0)
Out[1]: 28
In [2]: sys.getsizeof(1)
Out[2]: 28
In [3]: sys.getsizeof((2**30)-1)
Out[3]: 28
In [4]: sys.getsizeof((2**30))
Out[4]: 32
```

Behind the scenes, sets of 4 bytes are added each time the size of the number you're counting steps above the previous limit. This affects only the memory usage; you don't see any difference externally.

We can do the same check for byte strings. An empty byte sequence costs 33 bytes, and each additional character adds 1 byte to the cost:

```
In [5]: sys.getsizeof(b'')
Out[5]: 33
In [6]: sys.getsizeof(b'a')
Out[6]: 34
In [7]: sys.getsizeof(b'ab')
Out[7]: 35
In [8]: sys.getsizeof(b'abc')
Out[8]: 36
```

When we use a list, we see different behavior. `getsizeof` isn't counting the cost of the contents of the list—just the cost of the list itself. An empty list costs 56 bytes, and each item in the list takes another 8 bytes on a 64-bit laptop:

```
# goes up in 8-byte steps rather than the 28+ we might expect!
In [9]: sys.getsizeof([])
Out[9]: 56
In [10]: sys.getsizeof([1])
Out[10]: 64
In [11]: sys.getsizeof([1, 2])
Out[11]: 72
```

This is more obvious if we use byte strings—we'd expect to see much larger costs than `sizeof` is reporting:

```
In [12]: sys.getsizeof([b""])
Out[12]: 64
In [13]: sys.getsizeof([b"abcdefghijklm"])
Out[13]: 64
In [14]: sys.getsizeof([b"a", b"b"])
Out[14]: 72
```

`sizeof` reports only some of the cost, and often just for the parent object. As noted previously, it also isn't always implemented, so it can have limited usefulness.

A better tool is `asizeof` in **pympler**. This will walk a container's hierarchy and make a best guess about the size of each object it finds, adding the sizes to a total. Be warned that it is quite slow.

In addition to relying on guesses and assumptions, `asizeof` also cannot count memory allocated behind the scenes (such as a module that wraps a C library may not report the bytes allocated in the C library). It is best to use this as a guide. We prefer to use `memit`, as it gives us an accurate count of memory usage on the machine in question.

We can check the estimate it makes for a large list—here we'll use 10 million integers:

```
In [1]: from pympler.asizeof import asizeof
In [2]: asizeof([x for x in range(int(1e7))])
Out[2]: 401528048
In [3]: %load_ext memory_profiler
In [4]: %memit [x for x in range(int(1e7))]
```

peak memory: 401.91 MiB, increment: 326.77 MiB

We can validate this estimate by using `memit` to see how the process grew. Both reports are approximate—`memit` takes snapshots of the RAM usage reported by the operating system while the statement is executing, and `asizeof` asks the objects about their size (which may not be reported correctly). We can conclude that 10 million integers in a list cost around 400 MB of RAM.

Generally, the `asizeof` process is slower than using `memit`, but `asizeof` can be useful when you're analyzing small objects. `memit` is probably more useful for real-world applications, as the actual memory usage of the process is measured rather than inferred.

Bytes Versus Unicode

One of the (many!) advantages of Python 3.x over Python 2.x is the switch to Unicode-by-default. Previously, we had a mix of single-byte strings and multibyte Unicode objects, which could cause a headache during data import and export. In

Python 3.x, all strings are Unicode by default, and if you want to deal in bytes, you'll explicitly create a byte sequence.

Unicode objects have more efficient RAM usage in Python 3.12 than in earlier versions. In [Example 12-11](#), we can see a 100-million-character sequence being built as a collection of bytes and as a Unicode object. The Unicode variant for common characters (here we're assuming UTF 8 as the system's default encoding) costs the same—a single-byte implementation is used for these common characters.

Example 12-11. Unicode objects can be as cheap as bytes in Python 3.x

```
In [1]: %load_ext memory_profiler
In [2]: type(b"b")
Out[2]: bytes
In [3]: %memit b"a" * int(1e8)
peak memory: 57.72 MiB, increment: 0.07 MiB
In [4]: type("u")
Out[4]: str
In [5]: %memit "u" * int(1e8)
peak memory: 57.88 MiB, increment: 0.03 MiB
In [6]: %memit "Σ" * int(1e8)
peak memory: 187.38 MiB, increment: 129.50 MiB
```

The sigma character (Σ) is more expensive—it is represented in UTF 8 as 2 bytes. We gained the flexible Unicode representation from Python 3.3 thanks to [PEP 393](#). It works by observing the range of characters in the string and using a smaller number of bytes to represent the lower-order characters, if possible.

The UTF-8 encoding of a Unicode object uses 1 byte per ASCII character and more bytes for less frequently seen characters. If you're not sure about Unicode encodings versus Unicode objects, go and watch [Ned Batchelder's "Pragmatic Unicode; or, How Do I Stop the Pain?"](#).

Efficiently Storing Lots of Text in RAM

A common problem with text is that it occupies a lot of RAM—but if we want to test if we have seen strings before or count their frequency, having them in RAM is more convenient than paging them to and from a disk. Storing the strings naively is expensive, but *tries* (or “prefix trees”) can be used to compress their representation and still allow fast operations.

These more advanced algorithms can save you a significant amount of RAM, which means that you might not need to expand to more servers. For production systems, the savings can be huge. In this section we'll look at compressing a set of strings costing 1.2 GB down to 39 MB using a trie, accepting a small reduction in performance.

For this example, we'll use a text set built from a partial dump of Wikipedia. This set contains 12 million unique tokens from a full export of the English Wikipedia data, which takes up 120 MB on disk. The data comes from the “pages-articles” weekly 22 GB data dump from <https://dumps.wikimedia.org/enwiki/20240701>.¹ This data is processed through Gensim's `make_wiki` script to produce a list of unique terms. In the five years between our second edition and this third edition, the unique word list has grown from 11.5 million to 12.5 million unique tokens.

The tokens are split on whitespace from their original articles; they have variable length and contain Unicode characters and numbers. They look like this:

```
faddishness  
'melanesians'  
Kharálampos  
PizzaInACup™  
url="http://en.wikipedia.org/wiki?curid=363886"  
VIIIa),  
Superbagnères.
```

We'll use this text sample to test how quickly we can build a data structure holding one instance of each unique word, and then we'll see how quickly we can query for a known word (we'll use the uncommon “Zwiebel,” from the painter Alfred Zwiebel). This lets us ask, “Have we seen Zwiebel before?” Token lookup is a common problem, and being able to do it quickly is important.



When you try these containers on your own problems, be aware that you will probably see different behaviors. Each container builds its internal structures in different ways; passing in different types of token is likely to affect the build time of the structures, and different lengths of token will affect the query time. Always test in a methodical way.

Trying These Approaches on 11 Million Tokens

Figure 12-2 shows the 12-million-token text file (120 MB raw data) stored using a number of containers that we'll discuss in this section. The x-axis shows RAM usage for each container, the y-axis tracks the query time, and the size of each point relates to the time taken to build the structure (larger means it took longer).

As we can see in this diagram, the `set` and `list` examples use a lot of RAM; the `list` example is both large *and* slow! The Marisa trie example is the most RAM-efficient for this dataset. With the Marisa trie, we have to build it first, and that takes time and

¹ You can find current dumps at <https://dumps.wikimedia.org/enwiki>.

RAM; but then we can save it, and loading it back for read-only usage is very fast and very RAM efficient—“Marisa Trie (load)” is a tiny dot, as it loads so quickly!

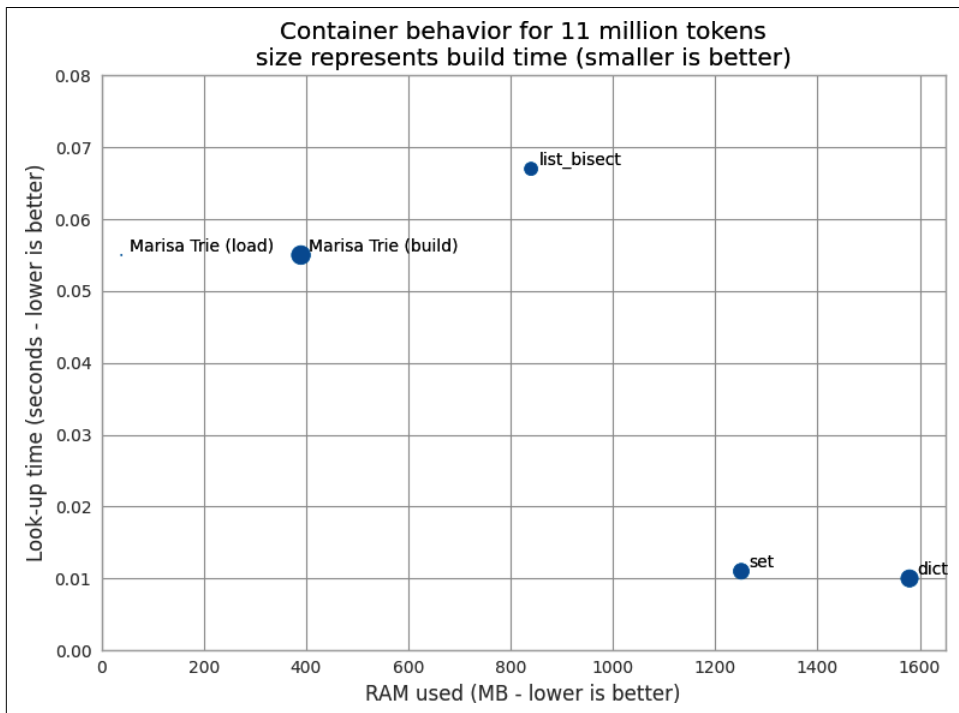


Figure 12-2. Trie versus set, list, and dict containers

The figure doesn’t show the lookup time for the naive `list` without sort approach, which we’ll introduce shortly, as it takes far too long. Do be aware that you must test your problem with a variety of containers—each offers different trade-offs, such as construction time and API flexibility.

Next, we’ll build up a process to test the behavior of each container.

list

Let’s start with the simplest approach. We’ll load our tokens into a `list` and then query it using an $O(n)$ linear search. Storing 12 million strings in a list is easy; running a linear search against the items 100,000 times is slow. Using this dataset and making 100 queries, we can estimate that making 100,000 queries would take over three hours.

In each of the following examples, we use a generator, `text_example.readers`, that extracts one Unicode token at a time from the input file. This means that the read process uses only a tiny amount of RAM:

```
print("RAM at start {:.1f}MiB".format(memory_profiler.memory_usage()[0]))
t1 = time.time()
words = [w for w in text_example.readers]
print("Loading {} words".format(len(words)))
t2 = time.time()
print("RAM after creating list {:.1f}MiB, took {:.1f}s" \
      .format(memory_profiler.memory_usage()[0], t2 - t1))
```

We're interested in how quickly we can query this list. Ideally, we want to find a container that will store our text and allow us to query it and modify it without penalty. To query it, we look for a known word a number of times by using `timeit`. With the list we do 100 repeats; we do more in the subsequent tests:

```
assert 'Zwiebel' in words
time_cost = sum(timeit.repeat(stmt="'Zwiebel' in words",
                              setup="from __main__ import words",
                              number=1,
                              repeat=100))

time_cost *= (100_000 / 100)
print("Summed time to look up word {:.4f}s".format(time_cost))
```

Our test script reports that approximately 830 MB was used to store the original 120 MB file as a list, and that the aggregate lookup time was over three hours:

```
$ python python text_example_list.py
Reading from all_unique_words_wikipedia_via_gensim.txt
RAM at start 48.3MiB
Loading 12558809 words
RAM after creating list 884.3MiB, took 12.4s
Checking that we'll see Zwiebel in our words container
Summed time to lookup word 11635.6615s # over 3 hours
```

Storing text in an unsorted list is obviously a poor idea; the $O(n)$ lookup time is expensive, as is the memory usage. This is the worst of all worlds!

We can improve the lookup time by sorting the list and using a binary search via the `bisect` module; this gives us a sensible lower bound for future queries. In [Example 12-12](#), we time how long it takes to sort the list. Here we query the cost of searching 100,000 times for the same token.

Example 12-12. Timing the sort operation to prepare for using `bisect`

```
print("RAM at start {:.1f}MiB".format(memory_profiler.memory_usage()[0]))
t1 = time.time()
words = [w for w in text_example.readers]
print("Loading {} words".format(len(words)))
```

```

t2 = time.time()
print("RAM after creating list {:.1f}MiB, took {:.1f}s" \
      .format(memory_profiler.memory_usage()[0], t2 - t1))
print("The list contains {} words".format(len(words)))
words.sort()
t3 = time.time()
print("Sorting list took {:.1f}s".format(t3 - t2))

```

Next, we do the same lookup as before, but with the addition of the `index` method, which uses `bisect`:

```

import bisect
...
def index(a, x):
    'Locate the leftmost value exactly equal to x'
    i = bisect.bisect_left(a, x)
    if i != len(a) and a[i] == x:
        return i
    raise ValueError
...
time_cost = sum(timeit.repeat(stmt="index(words, 'Zwiebel')",
                               setup="from __main__ import words, index",
                               number=1,
                               repeat=100_000))

```

In [Example 12-13](#), we see that the RAM usage stays the same as we're using the same list. The sort takes a further 0.5 seconds, and the cumulative lookup time is 0.06 seconds.

Example 12-13. Timings for using `bisect` on a sorted list

```

$ python text_example_list_bisect.py
Reading from all_unique_words_wikipedia_via_gensim.txt
RAM at start 48.3MiB
Loading 12558809 words
RAM after creating list 883.9MiB, took 12.4s
The list contains 12558809 words
Sorting list took 0.5s
Checking that we'll see Zwiebel in our words container
Zwiebel found at location 8682466
Summed time to lookup word 0.0625s

```

We now have a sensible baseline for timing string lookups: RAM usage must get better than 883 MB, and ideally the total lookup time should be better than 0.06 seconds.

set

Using the built-in `set` might seem to be the most obvious way to tackle our task. In [Example 12-14](#), the `set` stores each string in a hashed structure (see [Chapter 4](#) if you need a refresher). It is quick to check for membership, but each string must be stored separately, which is expensive on RAM.

Example 12-14. Using a `set` to store the data

```
words_set = set(text_example.readers)
```

As we can see in [Example 12-15](#), the `set` uses more RAM than the `list` by another 400 MB; however, it gives us a very fast lookup time without requiring an additional index function or an intermediate sorting operation.

Example 12-15. Running the `set` example

```
$ python text_example_set.py
Reading from all_unique_words_wikipedia_via_gensim.txt
RAM at start 48.2MiB
RAM after creating set 1299.9MiB, took 17.7s
The set contains 12558809 words
Checking that we'll see Zwiebel in our words container
Summed time to lookup word 0.0111s
```

If RAM isn't at a premium, this might be the most sensible first approach.

We have now lost the *ordering* of the original data, though. If that's important to you, note that you could store the strings as keys in a dictionary.

dict

If we choose to use a `dict`, we'll need to store the token and its position. We can do this ([Example 12-16](#)) with the following snippet. Each value is an index connected to the original read order. This way, you could ask the dictionary if the key is present and for its index.

Example 12-16. Using a `dict` to store the data

```
words_dict = {k:n for (n,k) in enumerate(text_example.readers)}
```

We see in [Example 12-17](#) that this variant takes more RAM than the `set` variant and has a similarly fast lookup speed, as they share the same key-lookup mechanism. The additional RAM here is from the value stored against each key; this corresponds to the index in the original list.

Example 12-17. Running the dict example

```
$ python text_example_dict.py
Reading from all_unique_words_wikipedia_via_gensim.txt
RAM at start 48.2MiB
RAM after creating dict 1642.6MiB, took 21.6s
The dict contains 12558809 words
Checking that we'll see Zwiebel in our words container
Summed time to lookup word 0.0103s
```

More efficient tree structures

Let's introduce a set of algorithms that use RAM more efficiently to represent our strings.

Figure 12-3 from Wikimedia Commons shows the difference in representation of four words—"tap", "taps", "top", and "tops"—between a trie and a DAWG.² With a list or a set, each of these words would be stored as a separate string. Both the DAWG and the trie share parts of the strings, so that less RAM is used.

The main difference between these is that a trie shares just common prefixes, while a DAWG shares common prefixes and suffixes. In languages (like English) that have many common word prefixes and suffixes, this can save a lot of repetition.

Exact memory behavior will depend on your data's structure. Typically, a DAWG cannot assign a value to a key because of the multiple paths from the start to the end of the string, but the version shown here can accept a value mapping. Tries can also accept a value mapping. Some structures have to be constructed in a pass at the start, and others can be updated at any time.

A big strength of some of these structures is that they provide a *common prefix search*; that is, you can ask for all words that share the prefix you provide. With our list of four words, the result when searching for "ta" would be "tap" and "taps." Furthermore, since these are discovered through the graph structure, the retrieval of these results is very fast. If you're working with DNA, for example, compressing millions of short strings by using a trie can be an efficient way to reduce RAM usage.

² This example is taken from the Wikipedia article on the [deterministic acyclic finite state automaton \(DAFSA\)](#). DAFSA is another name for DAWG. The accompanying image is based on an image from Wikimedia Commons.

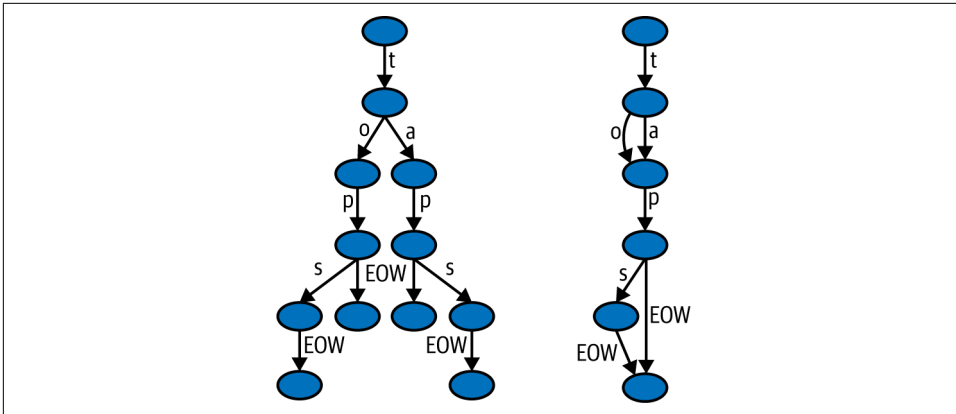


Figure 12-3. Trie and DAWG structures (image by [Chkno](#) [CC BY-SA 3.0])

In the following sections, we take a closer look at tries and their usage.

Currently your authors do not know of a scalable DAWG implementation that's competitive with the Marisa trie that follows; we hope that this situation will change in the future.

Marisa trie

The **Marisa trie** (dual-licensed LGPL and BSD) is a static **trie** using Cython bindings to an external library. As it is static, it cannot be modified after construction. It supports storing integer indices as values, as well as byte values and record values.

A key can be used to look up a value, and vice versa. All keys sharing the same prefix can be found efficiently. The trie's contents can be persisted. **Example 12-18** illustrates using a Marisa trie to store our sample data.

Example 12-18. Using a Marisa trie to store the data

```
import marisa_trie
...
words_trie = marisa_trie.Trie(text_example.readers)
```

In **Example 12-19**, we can see that lookup times are between using bisect and using a set. In this example we build the trie, query it, save it to disk and delete the in-memory trie, and then load it back in and query it again. The query time is unchanged at around 0.05 seconds. The memory usage is surprisingly large; it looks as though after building something is left in RAM, which perhaps would be garbage collected later.

Example 12-19. Running the Marisa trie example

```
$ python text_example_marisa_trie.py
Reading from all_unique_words_wikipedia_via_gensim.txt
RAM at start 50.2MiB
RAM after creating trie 440.3MiB, took 25.2s
The trie contains 12558809 words
Checking that we'll see Zwiebel in our words container
Summed time to lookup word 0.0577s
RAM before loading from disk 441.8MiB
RAM after loading trie from disk 441.8MiB, took 0.0s
The trie contains 12558809 words
time to save 0.212585s, time to load 0.011877s
Summed time to lookup word 0.0563s
```

Compared to the `set` and `dict` solution, the trie offers a further saving in memory on this dataset. While the lookups are a little slower in [Example 12-20](#), the disk and RAM usage are approximately 40 MB in the following snippet, if we save the trie to disk and then load it back into a fresh process. In practice we'd expect to build the trie once, save it to disk, and then load it multiple times, so this second example better reflects real-world usage. In a space-constrained environment, this is probably a very sensible container.

Example 12-20. Loading the trie that was built and saved in an earlier session is more RAM efficient

```
$ python text_example_marisa_trie_load_only.py
RAM before loading from disk 50.2MiB
RAM after loading trie from disk 89.8MiB, took 0.0s
The trie contains 12558809 words
time to load 0.037783s
Summed time to lookup word 0.0605s
```

The trade-off between storage sizes after construction and lookup times will need to be investigated for your application. You may find that using one of these “works just well enough,” so you might avoid benchmarking other options and simply move on to your next challenge. We suggest that the Marisa trie is your first choice in this case.

Using tries in production systems

The trie data structure offers good benefits, but you must still benchmark them on your problem rather than blindly adopting them. If you have overlapping sequences in your strings, you'll likely see a RAM improvement.

Tries are less well known, but they can provide strong benefits in production systems. We have an impressive success story in [“Large-Scale Social Media Analysis at Smesh \(2014\)” on page 483](#). Jamie Matthews at DabApps (a Python software house based in

the United Kingdom) also has a story about the use of tries in client systems to enable more efficient and cheaper deployments for customers:

At DabApps, we often try to tackle complex technical architecture problems by dividing them into small, self-contained components, usually communicating over the network using HTTP. This approach (referred to as a *service-oriented* or *microservice* architecture) has all sorts of benefits, including the possibility of reusing or sharing the functionality of a single component between multiple projects.

One such task that is often a requirement in our consumer-facing client projects is postcode geocoding. This is the task of converting a full UK postcode (for example: BN1 1AG) into a latitude and longitude coordinate pair, to enable the application to perform geospatial calculations such as distance measurement.

At its most basic, a geocoding database is a simple mapping between strings and can conceptually be represented as a dictionary. The dictionary keys are the postcodes, stored in a normalized form (BN11AG), and the values are a representation of the coordinates (we used a geohash encoding, but for simplicity imagine a comma-separated pair such as 50.822921,-0.142871).

The UK has approximately 1.7 million postcodes. Naively loading the full dataset into a Python dictionary, as described previously, uses several hundred megabytes of memory. Persisting this data structure to disk using Python's native Pickle format requires an unacceptably large amount of storage space. We knew we could do better.

We experimented with several different in-memory and on-disk storage and serialization formats, including storing the data externally in databases such as Redis and LevelDB, and compressing the key/value pairs. Eventually, we hit on the idea of using a trie. Tries are extremely efficient at representing large numbers of strings in memory, and the available open source libraries (we chose “marisa-trie”) make them very simple to use.

The resulting application, including a tiny web API built with the Flask framework, uses only 30 MB of memory to represent the entire UK postcode database, and can comfortably handle a high volume of postcode lookup requests. The code is simple; the service is very lightweight and painless to deploy and run on a free hosting platform such as Heroku, with no external requirements or dependencies on databases. Our implementation is open source, available at <https://github.com/dabapps/postcodeserver>.

—Jamie Matthews, technical director of DabApps.com (UK)

Tries are powerful data structures that can help you save RAM and time in exchange for a little additional effort in preparation. These data structures will be unfamiliar to many developers, so consider separating this code into a module that is reasonably isolated from the rest of your code to simplify maintenance.

Modeling More Text with scikit-learn's FeatureHasher

scikit-learn is Python's best-known machine learning framework, and it has excellent support for text-based natural language processing (NLP) challenges. Here, we'll look at classifying public posts from Usenet archives to one of 20 prespecified categories;

this is similar to the two-category spam classification process that cleans up our email inboxes.

One difficulty with text processing is that the vocabulary under analysis quickly explodes. The English language uses many nouns (e.g., the names of people and places, medical labels, and religious terms) and verbs (the “doing words” that often end in “-ing,” like “running,” “taking,” “making,” and “talking”) and their conjugations (turning the verb “talk” into “talked,” “talking,” “talks”), along with all the other rich forms of language. Punctuation and capitalization add an extra nuance to the representation of words.

A powerful and simple technique for classifying text is to break the original text into *n*-grams, often unigrams, bigrams, and trigrams (also known as 1-grams, 2-grams, and 3-grams). A sentence like “there is a cat and a dog” can be turned into unigrams (“there,” “is,” “a,” and so on), bigrams (“there is,” “is a,” “a cat,” etc.), and trigrams (“there is a,” “is a cat,” “a cat and,” ...).

There are 7 unigrams, 6 bigrams, and 5 trigrams for the sentence “there is a cat and a dog”; in total this sentence can be represented in this form by a vocabulary of 6 unique unigrams (since the term “a” is used twice), 6 unique bigrams, and 5 unique trigrams, making 17 descriptive items in total. As you can see, the *n*-gram vocabulary used to represent a sentence quickly grows; some terms are very common, and some are very rare.

There are techniques to control the explosion of a vocabulary, such as eliminating stop-words (removing the most common and often uninformative terms, like “a,” “the,” and “of”), lowercasing everything, and ignoring less frequent types of terms (such as punctuation, numbers, and brackets). If you practice natural language processing, you’ll quickly come across these approaches.

Introducing DictVectorizer and FeatureHasher

Before we look at the Usenet classification task, let’s look at two of scikit-learn’s feature processing tools that help with NLP challenges. The first is `DictVectorizer`, which takes a dictionary of terms and their frequencies and converts them into a variable-width sparse matrix (we will discuss sparse matrices in “[SciPy’s Sparse Matrices](#)” on page 405). The second is `FeatureHasher`, which converts the same dictionary of terms and frequencies into a fixed-width sparse matrix.

Example 12-21 shows two sentences—“there is a cat” and “there is a cat and a dog”—in which terms are shared between the sentences and the term “a” is used twice in one of the sentences. `DictVectorizer` is given the sentences in the call to `fit`; in a first pass it builds a list of words into an internal `vocabulary_`, and in a second pass it builds up a sparse matrix containing a reference to each term and its count.

Doing two passes takes longer than the one pass of `FeatureHasher`, and storing a vocabulary costs additional RAM. Building a vocabulary is often a serial process; by avoiding this stage, the feature hashing can potentially operate in parallel for additional speed.

Example 12-21. Lossless text representation with `DictVectorizer`

```
In [2]: from sklearn.feature_extraction import DictVectorizer
...:
...: dv = DictVectorizer()
...: # frequency counts for ["there is a cat", "there is a cat and a dog"]
...: token_dict = [{'there': 1, 'is': 1, 'a': 1, 'cat': 1},
...:               {'there': 1, 'is': 1, 'a': 2, 'cat': 1, 'and': 1, 'dog': 1}]

In [3]: dv.fit(token_dict)
...:
...: print("Vocabulary:")
...: print(dv.vocabulary_)

Vocabulary:
{'a': 0, 'and': 1, 'cat': 2, 'dog': 3, 'is': 4, 'there': 5}

In [4]: X = dv.transform(token_dict)
```

To make the output a little clearer, see the Pandas DataFrame view of the matrix `X` in [Figure 12-4](#), where the columns are set to the vocabulary. Note that here we’ve made a *dense* representation of the matrix—we have 2 rows and 6 columns, and each of the 12 cells contains a number. In the sparse form, we store only the 10 counts that are present and do not store anything for the 2 items that are missing. With a larger corpus, the larger storage required by a dense representation, containing mostly 0s, quickly becomes prohibitive. For NLP the sparse representation is standard.

	a	and	cat	dog	is	there
0	1	0	1	0	1	1
1	2	1	1	1	1	1

Figure 12-4. Transformed output of `DictVectorizer` shown in a Pandas DataFrame

One feature of `DictVectorizer` is that we can give it a matrix and reverse the process. In [Example 12-22](#), we use the vocabulary to recover the original frequency representation. Note that this does *not* recover the original sentence; there’s more than one way to interpret the ordering of words in the first example (both “there is a cat” and “a cat is there” are valid interpretations). If we used bigrams, we’d start to introduce a constraint on the ordering of words.

Example 12-22. Reversing the output of matrix X to the original dictionary representation

```
In [5]: print("Reversing the transform:")
...: print(dv.inverse_transform(X))

Reversing the transform:
[{'a': 1.0, 'cat': 1.0, 'is': 1.0, 'there': 1.0},
 {'a': 2.0, 'and': 1.0, 'cat': 1.0, 'dog': 1.0, 'is': 1.0, 'there': 1.0}]
```

FeatureHasher takes the same input and generates a similar output but with one key difference: it does not store a vocabulary and instead employs a hashing algorithm to assign token frequencies to columns.

We’ve already looked at hash functions in “How Do Dictionaries and Sets Work?” on page 99. A hash converts a unique item (in this case a text token) into a number, where multiple unique items might map to the same hashed value, in which case we get a collision. Good hash functions cause few collisions. Collisions are inevitable if we’re hashing many unique items to a smaller representation. One feature of a hash function is that it can’t easily be reversed, so we can’t take a hashed value and convert it back to the original token.

In [Example 12-23](#), we ask for a fixed-width 10-column matrix—the default is a fixed-width matrix of 1 million elements, but we’ll use a tiny matrix here to show a collision. A 1-million-element width is a sensible default for many applications.

The hashing process uses the fast MurmurHash3 algorithm, which transforms each token into a number; this is then converted into the range we specify. Larger ranges have few collisions; a small range like our range of 10 will have many collisions. Since every token has to be mapped to one of only 10 columns, we’ll get many collisions if we add a lot of sentences.

The output X has 2 rows and 10 columns; each token maps to one column, and we don’t immediately know which column represents each word since hashing functions are one-way, so we can’t map the output back to the input. In this case we can deduce, using `extra_token_dict`, that the tokens `there` and `is` both map to column 8, so we get nine 0s and one count of 2 in column 8.

Example 12-23. Using a 10-column FeatureHasher to show a hash collision

```
In [6]: from sklearn.feature_extraction import FeatureHasher
...:
...: fh = FeatureHasher(n_features=10, alternate_sign=False)
...: fh.fit(token_dict)
...: X = fh.transform(token_dict)
...: pprint(X.toarray().astype(np.int_))
...:
```

```
array([[1, 0, 0, 0, 0, 0, 0, 2, 0, 1],
       [2, 0, 0, 1, 0, 1, 0, 2, 0, 1]])

In [7]: extra_token_dict = [{'there': 1, 'is': 1}, ]
...: X = fh.transform(extra_token_dict)
...: print(X.toarray().astype(np.int_))
...:
[[0 0 0 0 0 0 0 2 0 0]]
```

Despite the occurrence of collisions, more than enough signal is often retained in this representation (assuming the default number of columns is used) to enable similar quality machine learning results with `FeatureHasher` compared to `DictVectorizer`.

Comparing `DictVectorizer` and `FeatureHasher` on a Real Problem

If we take the full 20 Newsgroups dataset, we have 20 categories, with approximately 18,000 emails spread across the categories. While some categories such as “sci.med” are relatively unique, others like “comp.os.ms-windows.misc” and “comp.windows.x” will contain emails that share similar terms. The machine learning task is to correctly identify the correct newsgroup from the 20 options for each item in the test set. The test set has approximately 4,000 emails; the training set used to learn the mapping of terms to the matching category has approximately 14,000 emails.

Note that this example does *not* deal with some of the necessities of a realistic training challenge. We haven’t stripped newsgroup metadata, which can be used to overfit on this challenge; rather than generalize just from the text of the emails, some extraneous metadata artificially boosts the scores. We have randomly shuffled the emails. Here, we’re not trying to achieve a single excellent machine learning result; instead, we’re demonstrating that a lossy hashed representation can be equivalent to a nonlossy and more memory-hungry variant.

In [Example 12-24](#), we take 18,846 documents and build up a training and test set representation using both `DictVectorizer` and `FeatureHasher` with unigrams, bigrams, and trigrams. The `DictVectorizer` sparse array has shape (14,134, 4,335,793) for the training set, where our 14,134 emails are represented using 4 million tokens. Building the vocabulary and transforming the training data takes 36 seconds.

Contrast this with the `FeatureHasher`, which has a fixed 1-million-element-wide hashed representation, and where the transformation takes 14 seconds. Note that in both cases, roughly 9.8 million nonzero items are stored in the sparse matrices, so they’re storing similar quantities of information. The hashed version stores approximately 10,000 fewer items because of collisions.

If we’d used a dense matrix, we’d have 14 thousand rows by 10 million columns, making 140,000,000,000 cells of 8 bytes each—significantly more RAM than is typically

available in any current machine. Only a tiny fraction of this matrix would be nonzero. Sparse matrices avoid this RAM consumption.

Example 12-24. Comparing DictVectorizer and FeatureHasher on a real machine learning problem

```
Loading 20 newsgroups training data
18846 documents - 35.855MB
```

```
DictVectorizer on frequency dicts
DictVectorizer has shape (14134, 4335793) with 78,872,376 bytes
and 9,859,047 non-zero items in 36.12 seconds
Vocabulary has 4,335,793 tokens
LogisticRegression score 0.89 in 881.89 seconds
FeatureHasher on frequency dicts
FeatureHasher has shape (14134, 1048576) with 78,787,936 bytes
and 9,848,492 non-zero items in 14.42 seconds
LogisticRegression score 0.89 in 502.60 seconds
```

Critically, the `LogisticRegression` classifier used on `DictVectorizer` takes 70% longer to train with 4 million columns compared to the 1 million columns used by the `FeatureHasher`. Both show a score of 0.89, so for this challenge the results are reasonably equivalent.

Using `FeatureHasher`, we've achieved the same score on the test set, built our training matrix faster, and avoided building and storing a vocabulary, and we've trained faster than using the more common `DictVectorizer` approach. In exchange, we've lost the ability to transform a hashed representation back into the original features for debugging and explanation, and since we often want the ability to diagnose *why* a decision was made, this might be too costly a trade to make.

SciPy's Sparse Matrices

In “[Introducing DictVectorizer and FeatureHasher](#)” on page 401, we created a large feature representation using `DictVectorizer`, which uses a sparse matrix in the background. These sparse matrices can be used for general computation as well and are extremely useful when working with sparse data.

A *sparse matrix* is a matrix in which most matrix elements are 0. For these sorts of matrices, there are many ways to encode the nonzero values and then simply say “all other values are zero.” In addition to these memory savings, many algorithms have special ways of dealing with sparse matrices that give additional computational benefits:

```
>>> from scipy import sparse

>>> A_sparse = sparse.random(4096, 4096, 0.01).tocsr()
```

```
>>> A_sparse
<4096x4096 sparse matrix of type '<class 'numpy.float64'>'
  with 167772 stored elements in Compressed Sparse Row format>

>>> %timeit A_sparse * A_sparse
63.7 ms ± 792 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

>>> A_dense = A_sparse.todense()

>>> type(A_dense)
numpy.matrix

>>> %timeit A_dense * A_dense
779 ms ± 61.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

The simplest implementation of this is for COO matrices in SciPy, where for each nonzero element we store the value in addition to the location of the value. This means that for each nonzero value, we store three numbers in total. As long as our matrix has at least 66% zero entries, we are reducing the amount of memory needed to represent the data as a sparse matrix as opposed to a standard numpy array. However, COO matrices are generally used only to *construct* sparse matrices and not for doing actual computation (for that, *CSR/CSC* is preferred).

We can see in [Figure 12-5](#) that for low densities, sparse matrices are much faster than their dense counterparts. On top of this, they also use much less memory.

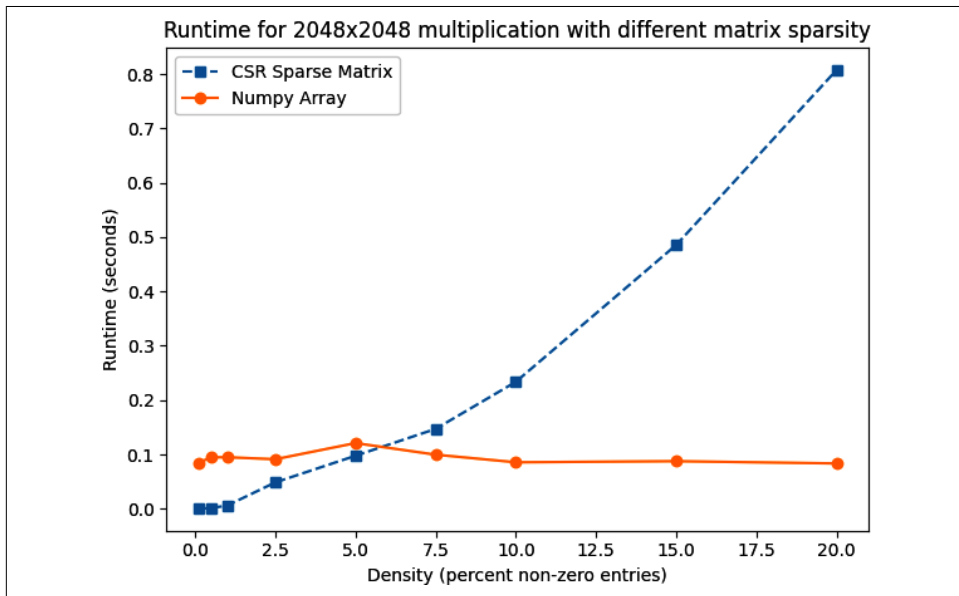


Figure 12-5. Sparse versus dense matrix multiplication

In [Figure 12-6](#), the dense matrix is always using 32.7 MB of memory ($2048 \times 2048 \times 64\text{-bit}$). However, a sparse matrix of 20% density uses only 10 MB, representing a 70% savings! As the density of the sparse matrix goes up, numpy quickly dominates in terms of speed because of the benefits of vectorization and better cache performance.

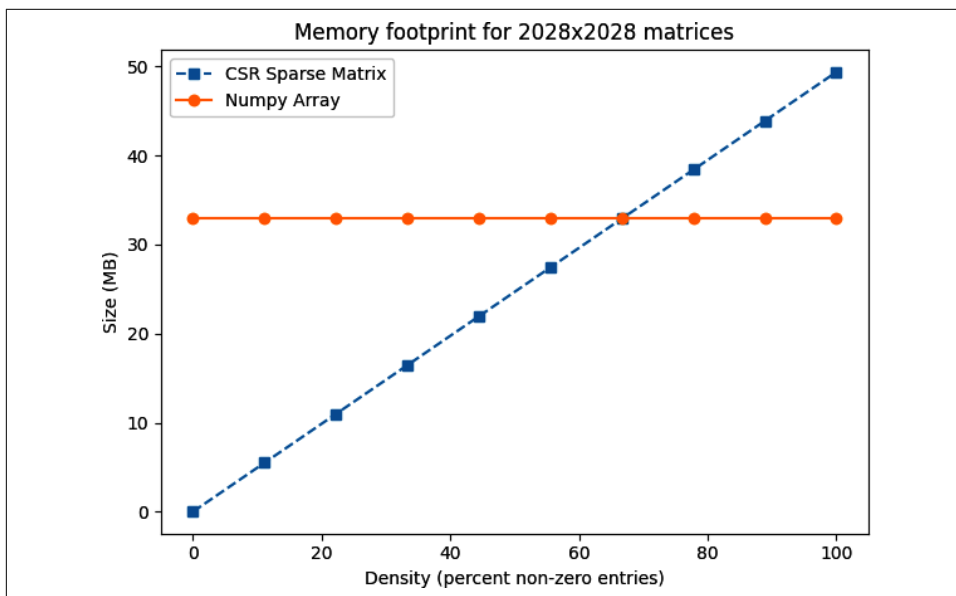


Figure 12-6. Sparse versus dense memory footprint

This extreme reduction in memory use is partly why the speeds are so much better. In addition to running only the multiplication operation for elements that are nonzero (thus reducing the number of operations needed), we also don't need to allocate such a large amount of space to save our result in. This is the push and pull of speedups with sparse arrays—it is a balance between losing the use of efficient caching and vectorization versus not having to do a lot of the calculations associated with the zero values of the matrix.

One operation that sparse matrices are particularly good at is cosine similarity. In fact, when creating a `DictVectorizer`, as we did in [“Introducing DictVectorizer and FeatureHasher” on page 401](#), it's common to use cosine similarity to see how similar two pieces of text are. In general for these item-to-item comparisons (where the value of a particular matrix element is compared to another matrix element), sparse matrices do quite well. Since the calls to `numpy` are the same whether we are using a normal matrix or a sparse matrix, we can benchmark the benefits of using a sparse matrix without changing the code of the algorithm.

While this is impressive, there are severe limitations. The amount of support for sparse matrices is quite low, and unless you are running special sparse algorithms

or doing only basic operations, you'll probably hit a wall in terms of support. In addition, SciPy's `sparse` module offers multiple implementations of sparse matrices, all of which have different benefits and drawbacks. Understanding which is the best one to use and when to use it demands some expert knowledge and often leads to conflicting requirements. As a result, sparse matrices probably aren't something you'll be using often; but when they are the correct tool, they are invaluable.

Tips for Using Less RAM

Generally, if you can avoid putting it into RAM, do. Everything you load costs you RAM. You might be able to load just a part of your data, for example, using a [memory-mapped file](#); alternatively, you might be able to use generators to load only the part of the data that you need for partial computations rather than loading it all at once.

If you are working with numeric data, you'll almost certainly want to switch to using `numpy` arrays—the package offers many fast algorithms that work directly on the underlying primitive objects. The RAM savings compared to using lists of numbers can be huge, and the time savings can be similarly amazing. Furthermore, if you are dealing with very sparse arrays, using SciPy's sparse array functionality can save incredible amounts of memory, albeit with a reduced feature set as compared to normal `numpy` arrays.

If you're working with strings, stick to `str` rather than `bytes` unless you have strong reasons to work at the byte level. Dealing with a myriad set of text encodings is painful by hand, and UTF-8 (or other Unicode formats) tends to make these problems disappear. If you're storing many Unicode objects in a static structure, you probably want to investigate the trie structures that we've just discussed.

If you're working with lots of bit strings, investigate `numpy` and the [bitarray package](#); both have efficient representations of bits packed into bytes. You might also benefit from looking at Redis, which offers efficient storage of bit patterns.

The PyPy project is experimenting with more efficient representations of homogeneous data structures, so long lists of the same primitive type (e.g., integers) might cost much less in PyPy than the equivalent structures in CPython. The [MicroPython](#) project will be interesting to anyone working with embedded systems: this tiny-memory-footprint implementation of Python is aiming for Python 3 compatibility.

It goes (almost!) without saying that you know you have to benchmark when you're trying to optimize on RAM usage, and that it pays handsomely to have a unit test suite in place before you make algorithmic changes.

Having reviewed ways of compressing strings and storing numbers efficiently, we'll now look at trading accuracy for storage space.

Probabilistic Data Structures

Probabilistic data structures allow you to make trade-offs in accuracy for immense decreases in memory usage. In addition, the number of operations you can do on them is much more restricted than with a `set` or a trie. For example, with a single HyperLogLog++ structure using 2.56 KB, you can count the number of unique items up to approximately 7,900,000,000 items with 1.625% error.

This means that if we're trying to count the number of unique license plate numbers for cars, and our HyperLogLog++ counter said there were 654,192,028, we would be confident that the actual number is between 643,561,407 and 664,822,648. Furthermore, if this accuracy isn't sufficient, you can simply add more memory to the structure and it will perform better. Giving it 40.96 KB of resources will decrease the error from 1.625% to 0.4%. However, storing this data in a `set` would take 3.925 GB, even assuming no overhead!

On the other hand, the HyperLogLog++ structure would only be able to count a `set` of license plates and merge with another `set`. So, for example, we could have one structure for every state, find how many unique license plates are in each of those states, and then merge them all to get a count for the whole country. If we were given a license plate, we couldn't tell you with very good accuracy whether we've seen it before, and we couldn't give you a sample of license plates we have already seen.

Probabilistic data structures are fantastic when you have taken the time to understand the problem and need to put something into production that can answer a very small set of questions about a very large set of data. Each structure has different questions it can answer at different accuracies, so finding the right one is just a matter of understanding your requirements.



Much of this section takes a deep dive into the mechanisms that power many of the popular probabilistic data structures. This is useful, because once you understand the mechanisms, you can use parts of them in algorithms you are designing. If you are just beginning with probabilistic data structures, it may be useful to first look at the real-world example ("[Real-World Example](#)" on [page 427](#)) before diving into the internals.

In almost all cases, probabilistic data structures work by finding an alternative representation for the data that is more compact and contains the relevant information for answering a certain set of questions. This can be thought of as a type of lossy compression, where we may lose some aspects of the data but retain the necessary

components. Since we're allowing the loss of data that isn't necessarily relevant for the particular set of questions we care about, this sort of lossy compression can be much more efficient than the lossless compression we looked at before with tries. It's because of this that the choice of which probabilistic data structure you will use is quite important—you want to pick one that retains the right information for your use case!

Before we dive in, it should be made clear that all the “error rates” here are defined in terms of *standard deviations*. This term comes from describing Gaussian distributions and says how spread out the function is around a center value. When the standard deviation grows, so do the number of values further away from the center point. Error rates for probabilistic data structures are framed this way because all the analyses around them are probabilistic. So, for example, when we say that the HyperLogLog++ algorithm has an error of $err = \frac{1.04}{\sqrt{m}}$, we mean that 68% of the time the error will be smaller than err , 95% of the time it will be smaller than $2 \times err$, and 99.7% of the time it will be smaller than $3 \times err$.³

Very Approximate Counting with a 1-Byte Morris Counter

We'll introduce the topic of probabilistic counting with one of the earliest probabilistic counters, the Morris counter (by Robert Morris of the NSA and Bell Labs). Applications include counting millions of objects in a restricted-RAM environment (e.g., on an embedded computer), understanding large data streams, and working on problems in AI like image and speech recognition.

The Morris counter keeps track of an exponent and models the counted state as $2^{exponent}$ (rather than a correct count)—it provides an *order of magnitude* estimate. This estimate is updated using a probabilistic rule.

We start with the exponent set to 0 when we've counted 0 items. If we ask for the *value* of the counter, we'll be given $\text{pow}(2, exponent)=1$ (the keen reader will note that this is off by one—we did say this was an *approximate* counter!). If we ask the counter to increment itself, it will generate a random number (using the uniform distribution) and will test if $\text{random.uniform}(0, 1) \leq 1/\text{pow}(2, exponent)$, which will always be true ($\text{pow}(2, 0) == 1$). The counter increments, and the exponent is set to 1.

The second time we ask the counter to increment itself, it will test if $\text{random.uniform}(0, 1) \leq 1/\text{pow}(2, 1)$. This will be true 50% of the time. If the test passes, the exponent is incremented. If not, the exponent is not incremented for this increment request.

³ These numbers come from the 68-95-99.7 rule of Gaussian distributions. More information can be found in the [Wikipedia entry](#).

Table 12-1 shows the likelihoods of an increment occurring for each of the first exponents.

Table 12-1. Morris counter details

Exponent	$\text{pow}(2, \text{exponent})$	$P(\text{increment})$
0	1	1
1	2	0.5
2	4	0.25
3	8	0.125
4	16	0.0625
...	...	...
254	2.894802e+76	3.454467e-77

The maximum we could approximately count where we use a single unsigned byte for the exponent is $\text{math.pow}(2, 255) \approx 5\text{e}76$. The error relative to the actual count will be fairly large as the counts increase, but the RAM savings is tremendous, as we use only 1 byte rather than the 32 unsigned bytes we'd otherwise have to use. Example 12-25 shows a simple implementation of the Morris counter.

Example 12-25. Simple Morris counter implementation

```

"""Approximate Morris counter supporting many counters"""
import math
import random
import array

SMALLEST_UNSIGNED_INTEGER = 'B' # unsigned char, typically 1 byte

class MorrisCounter(object):
    """Approximate counter, stores exponent and counts approximately 2^exponent
https://en.wikipedia.org/wiki/Approximate_counting_algorithm"""
    def __init__(self, type_code=SMALLEST_UNSIGNED_INTEGER, nbr_counters=1):
        self.exponents = array.array(type_code, [0] * nbr_counters)

    def __len__(self):
        return len(self.exponents)

    def add_counter(self):
        """Add a new zeroed counter"""
        self.exponents.append(0)

    def get(self, counter=0):
        """Calculate approximate value represented by counter"""
        return math.pow(2, self.exponents[counter])

```

```

def add(self, counter=0):
    """Probabilistically add 1 to counter"""
    value = self.get(counter)
    probability = 1.0 / value
    if random.uniform(0, 1) < probability:
        self.exponents[counter] += 1

if __name__ == "__main__":
    mc = MorrisCounter()
    print("MorrisCounter has {} counters".format(len(mc)))
    for n in range(10):
        print("Iteration %d, MorrisCounter has: %d" % (n, mc.get()))
        mc.add()

    for n in range(990):
        mc.add()
    print("Iteration 1000, MorrisCounter has: %d" % (mc.get()))

```

Using this implementation, we can see in [Example 12-26](#) that the first request to increment the counter succeeds, the second succeeds, and the third doesn't.⁴

Example 12-26. Morris counter library example

```

>>> mc = MorrisCounter()
>>> mc.get()
1.0

>>> mc.add()
>>> mc.get()
2.0

>>> mc.add()
>>> mc.get()
4.0

>>> mc.add()
>>> mc.get()
4.0

```

In [Figure 12-7](#), the thick black line shows a normal integer incrementing on each iteration. On a 64-bit computer, this is an 8-byte integer. The evolution of three 1-byte Morris counters is shown as dotted lines; the y-axis shows their values, which approximately represent the true count for each iteration. Three counters are shown

⁴ A more fully fleshed-out implementation that uses an array of bytes to make many counters is available at <https://oreil.ly/GHUoW>.

to give you an idea about their different trajectories and the overall trend; the three counters are entirely independent of one another.

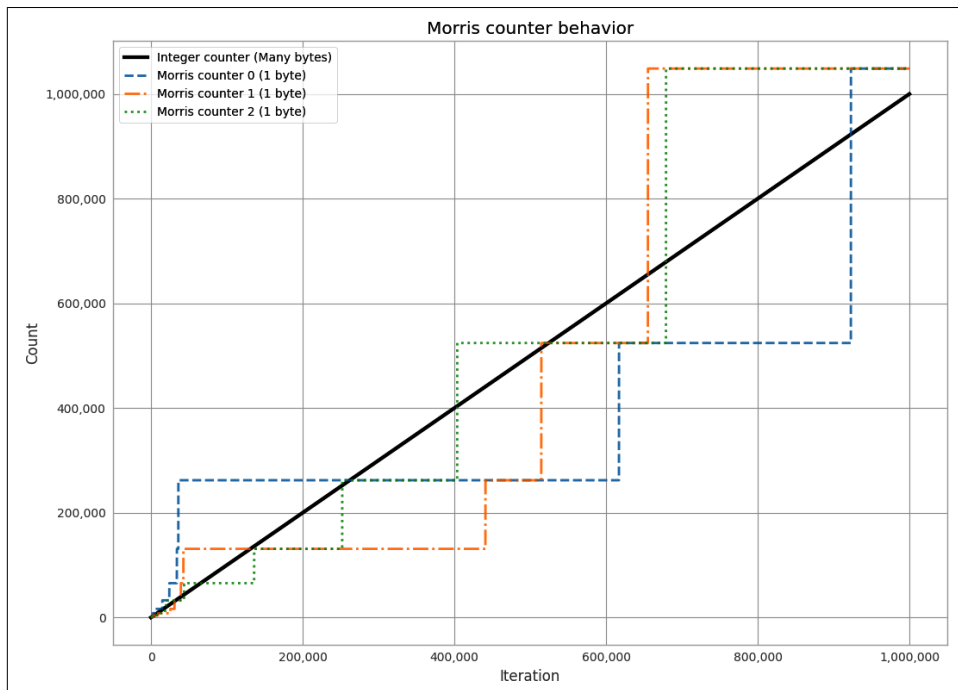


Figure 12-7. Three 1-byte Morris counters versus an 8-byte integer

This diagram gives you some idea about the error to expect when using a Morris counter. Further details about the error behavior are available [online](#).

K-Minimum Values

In the Morris counter, we lose any sort of information about the items we insert. That is to say, the counter's internal state is the same whether we do `.add("micha")` or `.add("ian")`. This extra information is useful and, if used properly, could help us have our counters count only unique items. In this way, calling `.add("micha")` thousands of times would increase the counter only once.

To implement this behavior, we will exploit properties of hashing functions (see [“Hash Functions and Entropy” on page 106](#) for a more in-depth discussion of hash functions). The main property we would like to take advantage of is the fact that the hash function takes input and *uniformly* distributes it. For example, let's assume we have a hash function that takes in a string and outputs a number between 0 and 1. For that function to be uniform means that when we feed it a string, we are equally likely to get a value of 0.5 as a value of 0.2 or any other value. This also means that if we

feed it many string values, we would expect the values to be relatively evenly spaced. Remember, this is a probabilistic argument: the values won't always be evenly spaced, but if we have many strings and try this experiment many times, they will tend to be evenly spaced.

Suppose we took 100 items and stored the hashes of those values (the hashes being numbers from 0 to 1). Knowing the spacing is even means that instead of saying, “We have 100 items,” we could say, “We have a distance of 0.01 between every item.” This is where the K-Minimum Values algorithm finally comes in⁵—if we keep the k smallest unique hash values we have seen, we can approximate the overall spacing between hash values and infer the total number of items. In [Figure 12-8](#), we can see the state of a K-Minimum Values structure (also called a KMV) as more and more items are added. At first, since we don't have many hash values, the largest hash we have kept is quite large. As we add more and more, the largest of the k hash values we have kept gets smaller and smaller. Using this method, we can get error rates of $O\left(\sqrt{\frac{2}{\pi(k-2)}}\right)$.

The larger k is, the more we can account for the hashing function we are using not being completely uniform for our particular input and for unfortunate hash values. An example of unfortunate hash values would be hashing $[A, B, C]$ and getting the values $[0.01, 0.02, 0.03]$. If we start hashing more and more values, it is less and less probable that they will clump up.

Furthermore, since we are keeping only the smallest *unique* hash values, the data structure considers only unique inputs. We can see this easily because if we are in a state where we store only the smallest three hashes and currently $[0.1, 0.2, 0.3]$ are the smallest hash values, then if we add in something with the hash value of 0.4, our state will not change. Similarly, if we add more items with a hash value of 0.3, our state also will not change. This is a property called *idempotence*; it means that if we do the same operation, with the same inputs, on this structure multiple times, the state will not be changed. This is in contrast to, for example, an append on a `list`, which will always change its value. This concept of idempotence carries on to all of the data structures in this section except for the Morris counter.

[Example 12-27](#) shows a very basic K-Minimum Values implementation. Of note is our use of a `sortedset`, which, like a set, can contain only unique items. This uniqueness gives our `KMinValues` structure idempotence for free. To see this, follow the code through: when the same item is added more than once, the data property does not change.

⁵ Kevin Beyer et al., “[On Synopses for Distinct-Value Estimation Under Multiset Operations](#)”, in *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data* (New York: ACM, 2007), 199–210.

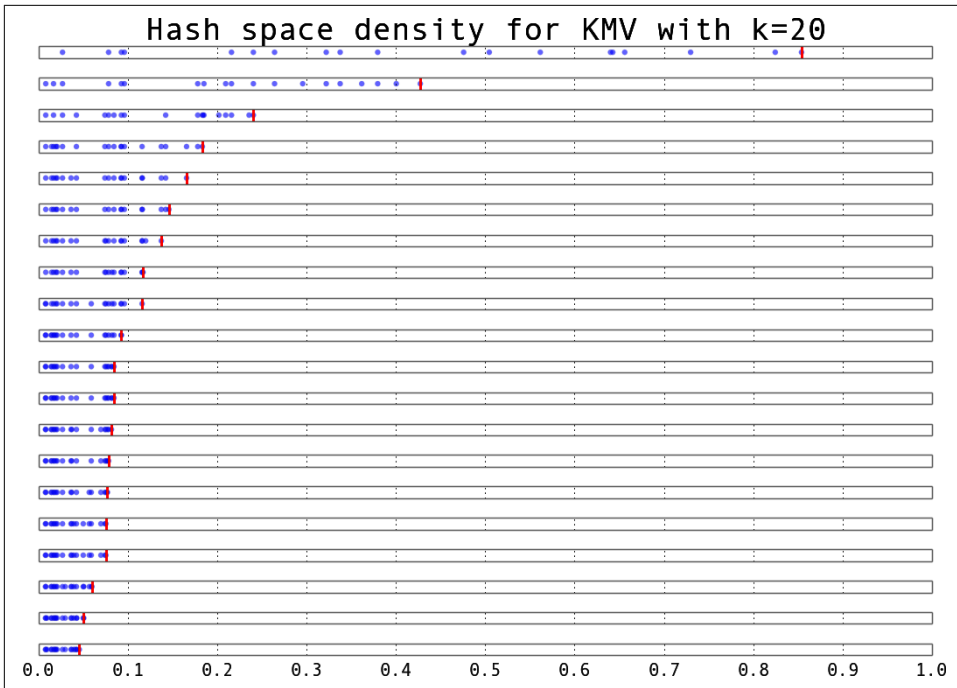


Figure 12-8. The value stores in a KMinValues as more elements are added

Example 12-27. Simple KMinValues implementation

```
import mmh3
from blist import sortedset

class KMinValues:
    def __init__(self, num_hashes):
        self.num_hashes = num_hashes
        self.data = sortedset()

    def add(self, item):
        item_hash = mmh3.hash(item)
        self.data.add(item_hash)
        if len(self.data) > self.num_hashes:
            self.data.pop()

    def __len__(self):
        if len(self.data) <= 2:
            return 0
        length = (self.num_hashes - 1) * (2 ** 32 - 1) /
            (self.data[-2] + 2 ** 31 - 1)
        return int(length)
```

Using the `KMinValues` implementation in the Python package `countmemaybe` (Example 12-28), we can begin to see the utility of this data structure. This implementation is very similar to the one in Example 12-27, but it fully implements the other set operations, such as union and intersection. Also note that “size” and “cardinality” are used interchangeably (the word “cardinality” is from set theory and is used more in the analysis of probabilistic data structures). Here, we can see that even with a reasonably small value for `k`, we can store 50,000 items and calculate the cardinality of many set operations with relatively low error.

Example 12-28. `countmemaybe` `KMinValues` implementation

```
>>> from countmemaybe import KMinValues

>>> kmv1 = KMinValues(k=1024)

>>> kmv2 = KMinValues(k=1024)

>>> for i in range(0, 50000): ❶
    kmv1.add(str(i))
    ...:

>>> for i in range(25000, 75000): ❷
    kmv2.add(str(i))
    ...:

>>> print(len(kmv1))
50416

>>> print(len(kmv2))
52439

>>> print(kmv1.cardinality_intersection(kmv2))
25900.2862992

>>> print(kmv1.cardinality_union(kmv2))
75346.2874158
```

- ❶ We put 50,000 elements into `kmv1`.
- ❷ `kmv2` also gets 50,000 elements, 25,000 of which are also in `kmv1`.



With these sorts of algorithms, the choice of hash function can have a drastic effect on the quality of the estimates. Both of these implementations use `mmh3`, a Python implementation of `murhash3` that has nice properties for hashing strings. However, different hash functions could be used if they are more convenient for your particular dataset.

Bloom Filters

Sometimes we need to be able to do other types of set operations, for which we need to introduce new types of probabilistic data structures. *Bloom filters* were created to answer the question of whether we've seen an item before.⁶

Bloom filters work by having multiple hash values in order to represent a value as multiple integers. If we later see something with the same set of integers, we can be reasonably confident that it is the same value.

To do this in a way that efficiently utilizes available resources, we implicitly encode the integers as the indices of a list. This could be thought of as a list of `bool` values that are initially set to `False`. If we are asked to add an object with hash values `[10, 4, 7]`, we set the tenth, fourth, and seventh indices of the list to `True`. In the future, if we are asked if we have seen a particular item before, we simply find its hash values and check if all the corresponding spots in the `bool` list are set to `True`.

This method gives us no false negatives and a controllable rate of false positives. If the Bloom filter says we have not seen an item before, we can be 100% sure that we haven't seen the item before. On the other hand, if the Bloom filter states that we *have* seen an item before, there is a probability that we actually have not and we are simply seeing an erroneous result. This erroneous result comes from the fact that we will have hash collisions, and sometimes the hash values for two objects will be the same even if the objects themselves are not the same. However, in practice Bloom filters are set to have error rates below 0.5%, so this error can be acceptable.

⁶ Burton H. Bloom, "Space/Time Trade-Offs in Hash Coding with Allowable Errors", *Communications of the ACM* 13, no. 7 (1970): 422–26.



We can simulate having as many hash functions as we want simply by having two hash functions that are independent of each other. This method is called *double hashing*. If we have a hash function that gives us two independent hashes, we can do this:

```
def multi_hash(key, num_hashes):
    hash1, hash2 = hashfunction(key)
    for i in range(num_hashes):
        yield (hash1 + i * hash2) % (2^32 - 1)
```

The modulo ensures that the resulting hash values are 32-bit (we would modulo by $2^{64} - 1$ for 64-bit hash functions).

The exact length of the bool list and the number of hash values per item we need will be fixed based on the capacity and the error rate we require. With some reasonably simple statistical arguments,⁷ we see that the ideal values are as follows:

$$n_{bits} = - capacity \cdot \frac{\ln(error)}{\ln^2(2)}$$
$$n_{hashes} = n_{bits} \cdot \frac{\ln(2)}{capacity}$$
$$= - \frac{\ln(error)}{\ln(2)}$$

If we wish to store 50,000 objects (no matter how big the objects themselves are) at a false positive rate of 0.05% (that is to say, 0.05% of the times we say we have seen an object before, we actually have not), it would require 791,015 bits of storage and 11 hash functions.

To further improve our efficiency in terms of memory use, we can use single bits to represent the bool values (a native bool actually takes 4 bits). We can do this easily by using the `bitarray` module. [Example 12-29](#) shows a simple Bloom filter implementation.

Example 12-29. Simple Bloom filter implementation

```
import math

import bitarray
import mmh3

class BloomFilter:
```

⁷ The [Wikipedia page on Bloom filters](#) has a very simple proof for the properties of a Bloom filter.

```

def __init__(self, capacity, error=0.005):
    """
    Initialize a Bloom filter with given capacity and false positive rate
    """
    self.capacity = capacity
    self.error = error
    self.num_bits = int((-capacity * math.log(error)) // math.log(2) ** 2 + 1)
    self.num_hashes = int((self.num_bits * math.log(2)) // capacity + 1)
    self.data = bitarray.bitarray(self.num_bits)

def _indexes(self, key):
    h1, h2 = mmh3.hash64(key)
    for i in range(self.num_hashes):
        yield (h1 + i * h2) % self.num_bits

def add(self, key):
    for index in self._indexes(key):
        self.data[index] = True

def __contains__(self, key):
    return all(self.data[index] for index in self._indexes(key))

def __len__(self):
    bit_off_num = self.data.count(True)
    bit_off_percent = 1.0 - bit_off_num / self.num_bits
    length = -1.0 * self.num_bits * math.log(bit_off_percent) / self.num_hashes
    return int(length)

@staticmethod
def union(bloom_a, bloom_b):
    assert bloom_a.capacity == bloom_b.capacity, "Capacities must be equal"
    assert bloom_a.error == bloom_b.error, "Error rates must be equal"

    bloom_union = BloomFilter(bloom_a.capacity, bloom_a.error)
    bloom_union.data = bloom_a.data | bloom_b.data
    return bloom_union

```

What happens if we insert more items than we specified for the capacity of the Bloom filter? At the extreme end, all the items in the bool list will be set to True, in which case we say that we have seen every item. This means that Bloom filters are very sensitive to what their initial capacity was set to, which can be quite aggravating if we are dealing with a set of data whose size is unknown (for example, a stream of data).

One way of dealing with this is to use a variant of Bloom filters called *scalable* Bloom filters.⁸ They work by chaining together multiple Bloom filters whose error rates vary in a specific way.⁹ By doing this, we can guarantee an overall error rate and add a new Bloom filter when we need more capacity. To check if we've seen an item before, we iterate over all of the sub-Blooms until either we find the object or we exhaust the list. A sample implementation of this structure can be seen in [Example 12-30](#), where we use the previous Bloom filter implementation for the underlying functionality and have a counter to simplify knowing when to add a new Bloom.

Example 12-30. Simple scaling Bloom filter implementation

```
from bloomfilter import BloomFilter

class ScalingBloomFilter:
    def __init__(self, capacity, error=0.005, max_fill=0.8,
                  error_tightening_ratio=0.5):
        self.capacity = capacity
        self.base_error = error
        self.max_fill = max_fill
        self.items_until_scale = int(capacity * max_fill)
        self.error_tightening_ratio = error_tightening_ratio
        self.bloom_filters = []
        self.current_bloom = None
        self._add_bloom()

    def _add_bloom(self):
        new_error = self.base_error * self.error_tightening_ratio ** len(
            self.bloom_filters
        )
        new_bloom = BloomFilter(self.capacity, new_error)
        self.bloom_filters.append(new_bloom)
        self.current_bloom = new_bloom
        return new_bloom

    def add(self, key):
        if key in self:
            return True
        self.current_bloom.add(key)
        self.items_until_scale -= 1
        if self.items_until_scale == 0:
            bloom_size = len(self.current_bloom)
            bloom_max_capacity = int(self.current_bloom.capacity * self.max_fill)

            # We may have been adding many duplicate values into the Bloom, so
```

8 Paolo Sérgio Almeida et al., “Scalable Bloom Filters”, *Information Processing Letters* 101, no. 6 (2007), 255–61.

9 The error values actually decrease like the geometric series. This way, when you take the product of all the error rates, it approaches the desired error rate.

```

        # we need to check if we actually need to scale or if we still have
        # space
        if bloom_size >= bloom_max_capacity:
            self._add_bloom()
            self.items_until_scale = bloom_max_capacity
        else:
            self.items_until_scale = int(bloom_max_capacity - bloom_size)
    return False

def __contains__(self, key):
    return any(key in bloom for bloom in self.bloom_filters)

def __len__(self):
    return int(sum(len(bloom) for bloom in self.bloom_filters))

```

Another way of dealing with this is using a method called *timing Bloom filters*. This variant allows elements to be expired out of the data structure, thus freeing up space for more elements. This is especially nice for dealing with streams, since we can have elements expire after, say, an hour and have the capacity set large enough to deal with the amount of data we see per hour. Using a Bloom filter this way would give us a nice view into what has been happening in the last hour.

Using this data structure will feel much like using a `set` object. In the following interaction, we use the scalable Bloom filter to add several objects, test if we've seen them before, and then try to experimentally find the false positive rate:

```

>>> bloom = BloomFilter(100)

>>> for i in range(50):
...:     bloom.add(str(i))
...:

>>> "20" in bloom
True

>>> "25" in bloom
True

>>> "51" in bloom
False

>>> num_false_positives = 0

>>> num_true_negatives = 0

>>> # None of the following numbers should be in the Bloom.
>>> # If one is found in the Bloom, it is a false positive.
>>> for i in range(51, 10000):
...:     if str(i) in bloom:
...:         num_false_positives += 1
...:     else:

```

```

.....:         num_true_negatives += 1
.....:

>>> num_false_positives
54

>>> num_true_negatives
9895

>>> false_positive_rate = num_false_positives / float(10000 - 51)

>>> false_positive_rate
0.005427681173987335

>>> bloom.error
0.005

```

We can also do unions with Bloom filters to join multiple sets of items:

```

>>> bloom_a = BloomFilter(200)

>>> bloom_b = BloomFilter(200)

>>> for i in range(50):
...:     bloom_a.add(str(i))
...:

>>> for i in range(25,75):
...:     bloom_b.add(str(i))
...:

>>> bloom = BloomFilter.union(bloom_a, bloom_b)

>>> "51" in bloom_a ❶
Out[9]: False

>>> "24" in bloom_b ❷
Out[10]: False

>>> "55" in bloom ❸
Out[11]: True

>>> "25" in bloom
Out[12]: True

```

- ❶ The value of 51 is not in bloom_a.
- ❷ Similarly, the value of 24 is not in bloom_b.
- ❸ However, the bloom object contains all the objects in both bloom_a and bloom_b!

One caveat is that you can take the union of only two Blooms with the same capacity and error rate. Furthermore, the final Bloom's used capacity can be as high as the sum of the used capacities of the two Blooms unioned to make it. This means that you could start with two Bloom filters that are a little more than half full and, when you union them together, get a new Bloom that is over capacity and not reliable!



A **Cuckoo filter** is another Bloom filter-like data structure that provides similar functionality to a Bloom filter with the addition of better object deletion. Furthermore, the Cuckoo filter in most cases has lower overhead, leading to better space efficiencies than the Bloom filter. When a fixed number of objects needs to be kept track of, it is often a better option. However, its performance degrades dramatically when its load limit is reached and there are no options for automatic scaling of the data structure (as we saw with the scaling Bloom filter).

The work of doing fast set inclusion in a memory-efficient way is a very important and active part of database research. Cuckoo filters, **Bloomier filters**, **Xor filters**, and more are constantly being released. However, for most applications, it is still best to stick with the well-known, well-supported Bloom filter.

LogLog Counter

LogLog-type counters are based on the realization that the individual bits of a hash function can also be considered random. That is to say, the probability of the first bit of a hash being 1 is 50%, the probability of the first two bits being 01 is 25%, and the probability of the first three bits being 001 is 12.5%. Knowing these probabilities, and keeping the hash with the most 0s at the beginning (i.e., the least probable hash value), we can come up with an estimate of how many items we've seen so far.

A good analogy for this method is flipping coins. Imagine we would like to flip a coin 32 times and get heads every time. The number 32 comes from the fact that we are using 32-bit hash functions. If we flip the coin once and it comes up tails, we will store the number 0, since our best attempt yielded 0 heads in a row. Since we know the probabilities behind this coin flip, we can also tell you that our longest series was 0 long, and you can estimate that we've tried this experiment $2^0 = 1$ time. If we keep flipping our coin and we're able to get 10 heads before getting a tail, then we would store the number 10. Using the same logic, you could estimate that we've tried the experiment $2^{10} = 1024$ times. With this system, the highest we could count would be the maximum number of flips we consider (for 32 flips, this is $2^{32} = 4,294,967,296$).

To encode this logic with LogLog-type counters, we take the binary representation of the hash value of our input and see how many 0s there are before we see our first 1. The hash value can be thought of as a series of 32 coin flips, where 0 means a flip for heads and 1 means a flip for tails (i.e., 000010101101 means we flipped four heads before our first tails, and 010101101 means we flipped one head before flipping our first tail). This gives us an idea of how many tries happened before this hash value was reached. The mathematics behind this system is almost equivalent to that of the Morris counter, with one major exception: we acquire the “random” values by looking at the actual input instead of using a random number generator. This means that if we keep adding the same value to a LogLog counter, its internal state will not change. **Example 12-31** shows a simple implementation of a LogLog counter.

Example 12-31. Simple implementation of a LogLog register

```
import mmh3

def trailing_zeros(number):
    """
    Returns the index of the first bit set to 1 from the right side of a 32-bit
    integer
    >>> trailing_zeros(0)
    32
    >>> trailing_zeros(0b1000)
    3
    >>> trailing_zeros(0b100000000)
    7
    """
    if not number:
        return 32
    index = 0
    while (number >> index) & 1 == 0:
        index += 1
    return index

class LogLogRegister:
    counter = 0
    def add(self, item):
        item_hash = mmh3.hash(str(item))
        return self._add(item_hash)

    def _add(self, item_hash):
        bit_index = trailing_zeros(item_hash)
        if bit_index > self.counter:
            self.counter = bit_index

    def __len__(self):
        return 2**self.counter
```


The biggest drawback of this method is that we may get a hash value that increases the counter right at the beginning and skews our estimates. This would be similar to flipping 32 tails on the first try. To remedy this, we should have many people flipping coins at the same time and combine their results. The law of large numbers tells us that as we add more and more flippers, the total statistics become less affected by anomalous samples from individual flippers. The exact way that we combine the results is the root of the difference between LogLog-type methods (classic LogLog, SuperLogLog, HyperLogLog, HyperLogLog++, etc.).

We can accomplish this “multiple flipper” method by taking the first couple of bits of a hash value and using that to designate which of our flippers had that particular result. If we take the first 4 bits of the hash, this means we have $2^4 = 16$ flippers. Since we used the first 4 bits for this selection, we have only 28 bits left (corresponding to 28 individual coin flips per coin flipper), meaning each counter can count only up to $2^{28} = 268,435,456$. In addition, there is a constant (alpha) that depends on the number of flippers, which normalizes the estimation. All of this together gives us an algorithm with $1.05/\sqrt{m}$ accuracy, where m is the number of registers (or flippers) used. [Example 12-32](#) shows a simple implementation of the LogLog algorithm.

Example 12-32. Simple implementation of LogLog

```
import mmh3

from loglogregister import LogLogRegister

class LL:
    def __init__(self, p):
        self.p = p
        self.num_registers = 2**p
        self.registers = [LogLogRegister() for _ in range(int(2**p))]
        self.alpha = 0.7213 / (1.0 + 1.079 / self.num_registers)

    def add(self, item):
        item_hash = mmh3.hash(str(item))
        register_index = item_hash & (self.num_registers - 1)
        register_hash = item_hash >> self.p
        self.registers[register_index].add(register_hash)

    def __len__(self):
        register_sum = sum(h.counter for h in self.registers)
        length = (
            self.num_registers *
            self.alpha *
            2 ** (register_sum / self.num_registers)
        )
        return int(length)
```

In addition to this algorithm deduplicating similar items by using the hash value as an indicator, it has a tunable parameter that can be used to dial whatever sort of accuracy versus storage compromise you are willing to make.

In the `__len__` method, we are averaging the estimates from all of the individual LogLog registers. This, however, is not the most efficient way to combine the data! This is because we may get some unfortunate hash values that make one particular register spike up while the others are still at low values. Because of this, we are able to achieve an error rate of only $O\left(\frac{1.30}{\sqrt{m}}\right)$, where m is the number of registers used.

SuperLogLog was devised as a fix to this problem.¹⁰ With this algorithm, only the lowest 70% of the registers were used for the size estimate, and their value was limited by a maximum value given by a restriction rule. This addition decreased the error rate to $O\left(\frac{1.05}{\sqrt{m}}\right)$. This is counterintuitive, since we got a better estimate by disregarding information!

Finally, HyperLogLog came out in 2007 and gave us further accuracy gains.¹¹ It did so by changing the method of averaging the individual registers: instead of just averaging, we use a spherical averaging scheme that also has special considerations for different edge cases the structure could be in. This brings us to the current best error rate of $O\left(\frac{1.04}{\sqrt{m}}\right)$. In addition, this formulation removes a sorting operation that is necessary with SuperLogLog. This can greatly speed up the performance of the data structure when you are trying to insert items at a high volume. [Example 12-33](#) shows a basic implementation of HyperLogLog.

Example 12-33. Simple implementation of HyperLogLog

```
import math

from ll import LL

class HyperLogLog(LL):
    def __len__(self):
        indicator = sum(2 ** -m.counter for m in self.registers)
        E = self.alpha * (self.num_registers ** 2) / indicator

        if E <= 5.0 / 2.0 * self.num_registers:
            V = sum(1 for m in self.registers if m.counter == 0)
            if V != 0:
```

¹⁰ Marianne Durand and Philippe Flajolet, “LogLog Counting of Large Cardinalities”, in *Algorithms—ESA 2003*, ed. Giuseppe Di Battista and Uri Zwick, vol. 2832 (Berlin, Heidelberg: Springer, 2003), 605–17.

¹¹ Philippe Flajolet et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *AOFA '07: Proceedings of the 2007 International Conference on Analysis of Algorithms* (AOFA, 2007), 127–46.

```

        Estar = (self.num_registers *
                 math.log(self.num_registers / (1.0 * V), 2))

    else:
        Estar = E
    else:
        if E <= 2 ** 32 / 30.0:
            Estar = E
        else:
            Estar = -2 ** 32 * math.log(1 - E / 2 ** 32, 2)
    return int(Estar)

if __name__ == "__main__":
    import mmh3

    hll = HyperLogLog(8)
    for i in range(100000):
        hll.add(mmh3.hash(str(i)))
    print(len(hll))

```

The only further increase in accuracy was given by the HyperLogLog++ algorithm, which increases the accuracy of the data structure while it is relatively empty. When more items are inserted, this scheme reverts to standard HyperLogLog. This is actually quite useful, since the statistics of the LogLog-type counters require a lot of data to be accurate—having a scheme for allowing better accuracy with fewer number items greatly improves the usability of this method. This extra accuracy is achieved by having a smaller but more accurate HyperLogLog structure that can later be converted into the larger structure that was originally requested. Also, some empirically derived constants are used in the size estimates that remove biases.

Real-World Example

To obtain a better understanding of the data structures, we first created a dataset with many unique keys, and then one with duplicate entries. Figures 12-9 and 12-10 show the results when we feed these keys into the data structures we’ve just looked at and periodically query, “How many unique entries have there been?” We can see that the data structures that contain more stateful variables (such as HyperLogLog and KMinValues) do better, since they more robustly handle bad statistics. On the other hand, the Morris counter and the single LogLog register can quickly have very high error rates if one unfortunate random number or hash value occurs. For most of the algorithms, however, we know that the number of stateful variables is directly correlated with the error guarantees, so this makes sense.

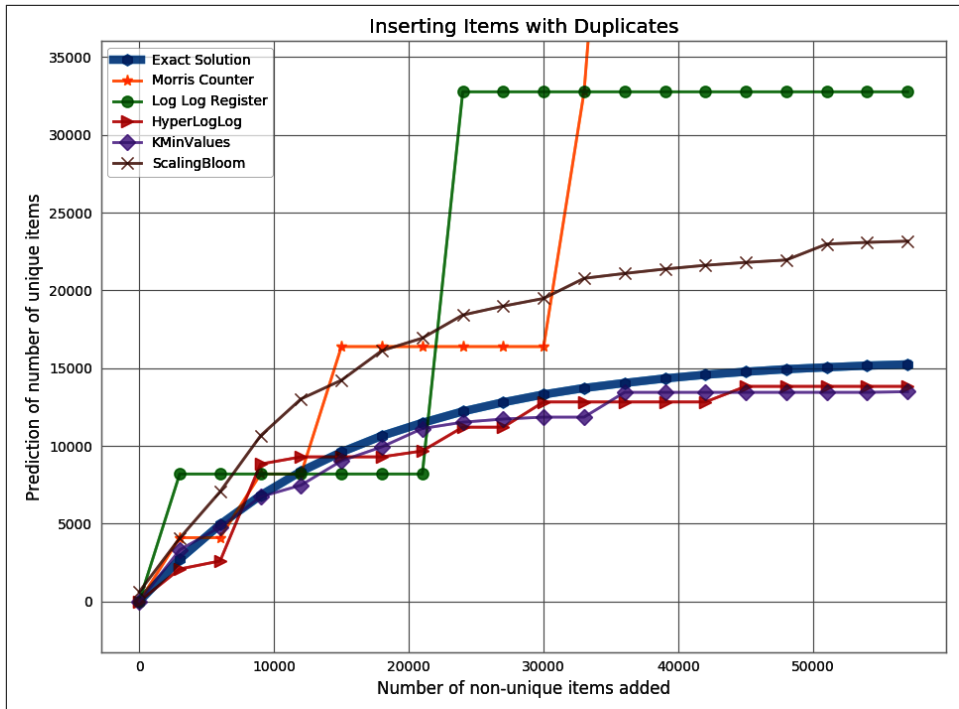


Figure 12-9. The approximate count of duplicate data using multiple probabilistic data structures. To do this, we generate 60,000 items with many duplicates and insert them into the various probabilistic data structures. Graphed is the structures prediction of the number of unique items during the process.

Looking just at the probabilistic data structures that have the best performance (which really are the ones you will probably use), we can summarize their utility and their approximate memory usage (see Table 12-2). We can see a huge change in memory usage depending on the questions we care to ask. This simply highlights the fact that when using a probabilistic data structure, you must first consider what questions you really need to answer about the dataset before proceeding. Also note that only the Bloom filter's size depends on the number of elements. The sizes of the HyperLogLog and KMinValues structures are dependent *only* on the error rate.

As another, more realistic test, we chose to use a dataset derived from the text of Wikipedia. We ran a very simple script in order to extract all single-word tokens with five or more characters from all articles and store them in a newline-separated file. The question then was, "How many unique tokens are there?" The results can be seen in Table 12-3.

Table 12-2. Comparison of major probabilistic data structures and the set operations available on them

	Error Rate	Union ^a	Intersection	Contains	Size ^b
HyperLogLog	$O\left(\frac{1.04}{\sqrt{m}}\right)$	Yes	No ^c	No	2.704 MB
KMinValues	$O\left(\sqrt{\frac{2}{\pi(m-2)}}\right)$	Yes	Yes	No	20.372 MB
Bloom filter	$O\left(\frac{0.78}{\sqrt{m}}\right)$	Yes	No ^c	Yes	197.8 MB

^a Union operations occur without increasing the error rate.
^b Size of data structure with 0.05% error rate, 100 million unique elements, and using a 64-bit hashing function.
^c These operations *can* be done but at a considerable penalty in terms of accuracy.

The major takeaway from this experiment is that if you are able to specialize your code, you can get amazing speed and memory gains. This has been true throughout the entire book: when we specialized our code in “**Selective Optimizations: Finding What Needs to Be Fixed**” on page 153, we were similarly able to get speed increases.

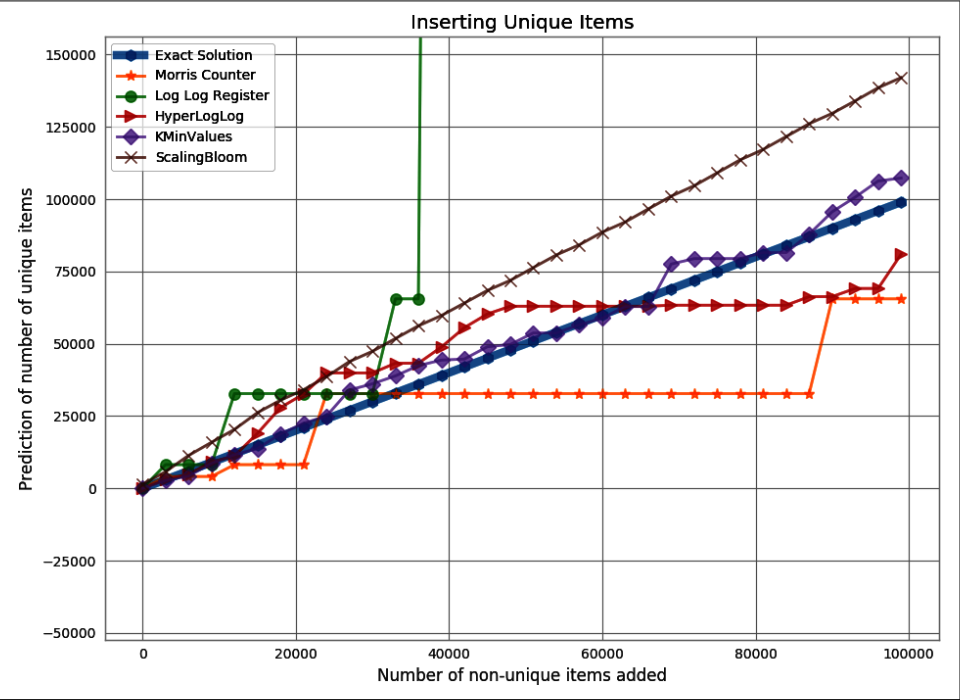


Figure 12-10. The approximate count of unique data using multiple probabilistic data structures. To do this, we insert the numbers 1 through 100,000 into the data structures. Graphed is the structures prediction of the number of unique items during the process.

Probabilistic data structures are an algorithmic way of specializing your code. We store only the data we need in order to answer specific questions with given error bounds. By having to deal with only a subset of the information given, we can not only make the memory footprint much smaller but also perform most operations over the structure faster.

Table 12-3. Size estimates for the number of unique words in Wikipedia

	Elements	Relative error	Processing time ^a	Structure size ^b
Morris counter ^c	1,073,741,824	6.52%	751s	5 bits
LogLog register	1,048,576	78.84%	1,690s	5 bits
LogLog	4,522,232	8.76%	2,112s	5 bits
HyperLogLog	4,983,171	−0.54%	2,907s	40 KB
KMinValues	4,912,818	0.88%	3,503s	256 KB
Scaling Bloom	4,949,358	0.14%	10,392s	11,509 KB
True value	4,956,262	0.00%	-----	49,558 KB ^d

^a Processing time has been adjusted to remove the time to read the dataset from disk. We also use the simple implementations provided earlier for testing.

^b Structure size is theoretical given the amount of data since the implementations used were not optimized.

^c Since the Morris counter doesn't deduplicate input, the size and relative error are given with regard to the total number of values.

^d The dataset is 49,558 KB considering only unique tokens, or 8.742 GB with all tokens.

As a result, whether or not you use probabilistic data structures, you should always keep in mind what questions you are going to be asking of your data and how you can most effectively store that data in order to ask those specialized questions. This may come down to using one particular type of list over another, using one particular type of database index over another, or maybe even using a probabilistic data structure to throw out all but the relevant data!

Wrap-Up

In our final chapter, we've invited colleagues to share pragmatic stories “from the field” that add some real-world color to our theoretical examples and processes.

Lessons from the Field

Questions You'll Be Able to Answer After This Chapter

- How can I diagnose a bad machine learning process to home in on a successful system?
- How can I use Python's tools and ecosystem for success in insurance risk?
- How do I balance iteration speed and execution speed at a hedge fund?
- How do successful start-ups handle large volumes of data and machine learning?
- What monitoring and deployment technologies keep systems stable?
- What lessons have successful CTOs learned about their technologies and teams?

In this chapter we have collected stories from successful companies that use Python in high-data-volume and speed-critical situations. The stories are written by key people in each organization who have many years of experience; they share not just their technology choices but also some of their hard-won wisdom. We have five great new stories for you from other experts in our domain. We've also kept some of the "Lessons from the Field" from the first and second editions of this book; their titles are marked "(2014)" and "(2020)" respectively.

Developing a High Performance Machine Learning Algorithm

Martin Goodson

[linkedin.com/in/martingoodson](https://www.linkedin.com/in/martingoodson)

Martin is the CEO of the multiple-award-winning data extraction firm Evolution AI. He also leads the London Machine Learning Meetup, the largest AI and machine learning community in Europe.

We had overpromised to our customers and our machine learning models were making a stream of errors in production. Our banking customers depend on our software to accurately extract data from financial documents, but our claim to “human-level accuracy” was looking extremely shaky. I dreaded receiving another email from a customer complaining that our accuracy was inadequate.

Like good data scientists, we had performed error analysis, finding that most of our errors were a result of failing to locate text accurately. “Text detection” was a crucial early step in our pipeline, but the existing open source and commercial solutions were all deeply inadequate. We needed far higher accuracy than existing methods could deliver.

I initiated an R&D project to build a new text detector based on cutting-edge convolutional neural network (CNN) object detection models. I assembled a great team of machine learning researchers—and a bunch of GPUs—and we set to work.

Our problem seemed simple—the aim of a text detector is to find the coordinates of all of the words in an image, represented as rectangles called “text boxes.” We were fortunate in that we could easily create synthetic documents with text at known coordinates—we wouldn’t need to use expensively hand-annotated data to train the models.

Unfortunately, far from being simple, the research project turned into a mess of complexity. At every turn, our technical approach seemed to be the wrong one. More than once, team members told me that what we were trying to do was impossible. But the journey was rich with hard-won lessons, which I want to share with you here.

Eyeballing the Data

At the beginning we didn’t spend enough time eyeballing the data. Training neural network models is a slow process, and far too many times we found out after a week or more of training that the resulting model was defective. In nearly all cases it was because of minor errors in the data generation process that we could have discovered

if we'd spent an hour or two just *looking* at the training data before starting the training phase.

For example, we eventually realized that our synthetic text generation script was removing lines of text if they extended beyond the edges of the page. Unfortunately, the script wasn't removing the ground truth annotation at the same time—so our ground truth didn't match the training images! Careful examination of the training data would have revealed this immediately.

Bespoke Analysis Tools

This brings me to the second lesson: the importance of creating bespoke analysis tools. Our research accelerated once we had designed interactive visualization tools in order to navigate the datasets and perform error analysis. Our scientists used the visualization tools to quickly discover the weaknesses of each model and allowed us to quickly iterate, fixing one issue at a time. If we had started out by creating the right tools at the outset, we would have avoided wasting months of time.

For example, we devised a heatmap visualization with the height and width of text boxes on the x- and y- axes, which used the intensity of color to represent the accuracy in each size bin. Using the heatmap, we discovered that our model—although strong on benchmarks—had a severe defect: it was poor at detecting long words. Long words are rare, so they don't affect benchmark performance much, but we could hardly tell our customers that we just can't extract long words from their documents!

We solved this problem by changing the convolutional kernel to a rectangle instead of the more standard square (<https://oreil.ly/vOJRI>). Since words are normally wider than they are tall, setting hyperparameters such as kernel shape and size correctly was critical to gaining very high performance.

Another very useful tool was an interactive 2D scatter plot of all of the errors made by the model, using various dimensions on the axes (e.g., confidence score, spatial position, etc.). Each data point on the scatter plot represented a single error made by the model. By hovering over a data point, we could instantly see a crop of the text that created the error—allowing us to quickly formulate a hypothesis for what might be causing the issue.

Using the Appropriate Evaluation Metrics

Thirdly, we learnt to be very careful about evaluation metrics. We realized quite late in the project that the academic task of object detection was very different from our real-world task of text detection, and the standard metrics were unsuitable—even downright misleading.

The academic metrics were designed to check that a model can find the rough coordinates of a cat in an image, not the exact coordinates of a word in a document page. Unfortunately, data extraction from documents requires extremely precise coordinates. Missing a digit from a financial document because your text box was in slightly the wrong place can have serious repercussions!

We had to create an entirely new metric for our use case—a nontrivial undertaking. If you need extremely high performance, then you must optimize for a metric that matches your real-world use case very closely.

Classical Scientific Methods: Hypothesis Driven Research

But the most important lesson was this: we succeeded because we used hypothesis driven research methods at every step. For every hurdle we encountered, we formulated a list of hypotheses and devised experiments to test each hypothesis in turn. Structuring the research in this way allowed us to make consistent progress over the 18 months we'd allocated for the research.

It was a complex project, and we had to overcome many technical challenges in order to gain the exceptional performance we were aiming at. Classical scientific methods helped us to rein in the complexity. Careful experimental design and basic research skills like good note-taking were critical.

Our team was combined of our internal machine learning scientists and members of a university research team. So effective scientific communication was also critical—the importance of regular research meetings and oral presentations should not be overlooked.

In the end, after much misery and frustration, we overcame the obstacles and produced a neural network model that performed better than anything else available. Our products started working properly, and, most importantly, our customers stopped contacting me with complaints about poor performance. The project was a success.

High Performance Computing in Journalism

Leon Yin
leonyin.org

Leon Yin is an award-winning journalist at Bloomberg News. He builds datasets and develops methods to investigate technology's impact on society.

One surprising (or not so surprising) application of high performance computing is journalism. Investigative journalism is a type of journalism that focuses on digging deep into an impactful, longer-term topic involving titans of industry, government institutions, and the powerful.

Investigative reporters rely on “shoe leather” reporting techniques to speak directly with affected individuals, as well as Freedom of Information requests to review government documents. Increasingly, reporters are embracing the affordances of technology to collate disparate data sources to unearth evidence of harm at scale.

Still Loading

One of America's most-notable monopolies was AT&T Bell Labs. Between 1913 and the 1980s, Bell Labs owned practically every telephone line connecting households across the United States. In 1982, AT&T was ordered by the US Department of Justice to split up its monopoly into several independent regional companies. These “Baby Bells” were subsequently acquired, merged, and rebranded as Verizon, AT&T, and CenturyLink (now called Lumen Technologies).

In addition to being mobile carriers and offering landline services, these three companies are also major internet service providers. In The Markup's award-winning series “**Still Loading**”, we found that AT&T, Verizon, and CenturyLink disproportionately offered slow internet speeds to low-income, non-White neighborhoods for the same price as fast fiber in other parts of town.

Our investigation was the first of its kind to show the scale of digital redlining practices in the United States. Across the 38 major cities served by AT&T, Verizon, and CenturyLink, we found disparities based on income or race in all but two cities. Data from our investigation was used by local newsrooms to report on internet disparities across the country, cited by policymakers to combat digital discrimination, and used as teaching materials for data journalists.

Slow Speeds and Quick Iteration

When you're shopping around for internet deals, you may have visited a web provider's website and typed in your home address to see what plans they might have for you. Our data collection effort attempted to replicate this process for over a million addresses.

Our methodology was inspired by an [academic article](#) by researchers from Princeton University that found discrepancies between what internet service providers self-reported to the Federal Communications Commission (FCC) and what they advertised to customers. The researchers built scrapers for several different internet service providers' websites to collect internet offers. Although their code was public, it no longer worked, as many of the sites had changed their user interface.

This is a common occurrence, as scrapers break all the time. To save time, I try to develop a quick proof of concept analysis before investing too many resources into an investigation. It's advantageous to test and eliminate potential projects quickly; this allows us to gauge the potential of many stories and decide how to proceed. During this step it's key to build a data pipeline that is reasonably functionable and select a small sample to indicate potential patterns.

As a reporter, this also means that I interview experts to get up to speed. For this project I contacted an author from the paper to discuss their findings. I asked whether a particular provider was egregious, and where. Repetition is often a good starting place, as it provides a blueprint to reference and build off of.

Researchers were quick to point to AT&T as an internet service provider with many discrepancies, especially in the city of Green Bay, Wisconsin.

This gave me direction for a scraper to start building. When I need to build a scraper I go to the website and get intimate with the user interface and the workflow necessary to manually retrieve whatever information I'm after.

The user interface for AT&T's internet service lookup website involved a multipage process of filling in an address, filling in an apartment number, and then selecting whether you're shopping for Fiber or DSL.

I noticed that an HTTP request to a server occurs for each of these steps while I was listening for network requests in my browser's developer tools. I typically opt for finding such [undocumented APIs](#) for web scrapers; however, because the process was multiple steps with multiple APIs, I was unable to successfully combine each step to look up the internet plans for a given address.

Instead of immediately figuring out these APIs, I quickly developed a Selenium scraper to fill in these steps. Selenium and other such [browser automation tools](#) (such as Puppeteer and Playwright) are great when you need to interact with rendered elements on a page as a user would.

It's also something that is easy to debug and deploy locally on a PC. For a few thousand random addresses from Green Bay, the process was painfully slow. This is partially because browser automation loads and renders all the elements and scripts on a page, and also the website deploys some sort of traffic control that will throttle requests from the same computer (called IP blocking). Also, in order to do this Selenium needs to run a full browser instance, which takes quite a lot of resources.

Even with multiprocessing to open 9 Selenium browsers to process 9 addresses in parallel, the process was slow. It took two weeks to collect 4,000 addresses. I initially viewed this as a nonstarter for anything more ambitious involving multiple companies or cities.

But I am a reporter first, and an engineer later. Why optimize code for a story that isn't spicy? However, looking at the data suggested we had an important story to report out.

After parsing and cleaning the scraped data, we plotted each internet plan on a map in [Figure 13-1](#), and immediately we saw clusters of fast speeds above 100 Mbps (green) and slow speeds (orange) below broadband benchmarks.

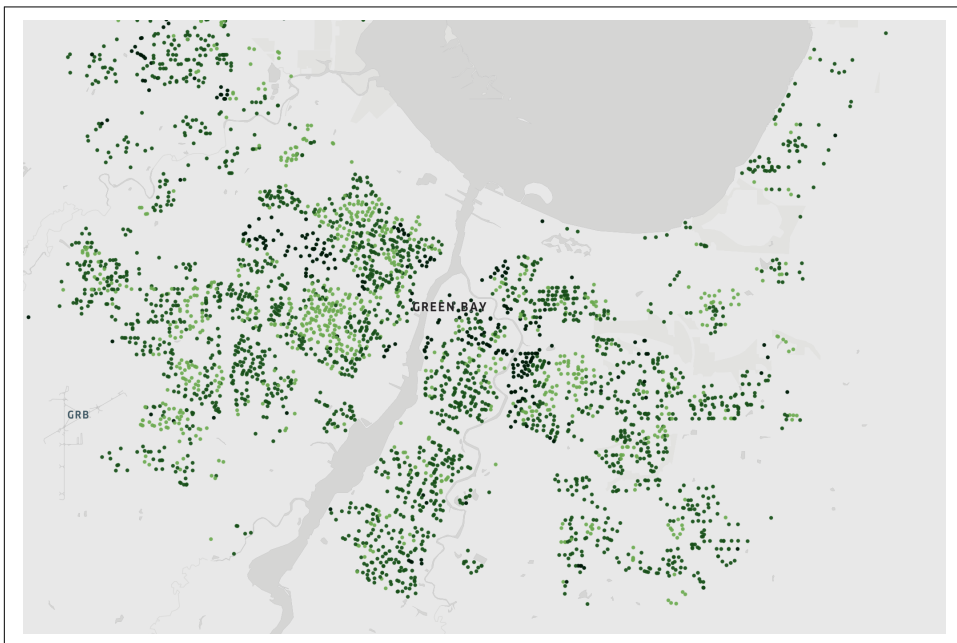


Figure 13-1. AT&T's internet speeds across a sample of Green Bay, Wisconsin, addresses, mapped using Kepler.GL

Geography is correlated with income and other socioeconomic factors. Every year, the Census Bureau conducts the American Community Survey of a select sample of the population and uses that data to estimate the median household income and racial makeup of every city in the United States.

We merged the median household income to each internet plan to create **Figure 13-2**, and using an extremely rough categorization system for income, we found AT&T was more likely to offer low-income residents slow speeds, and less likely to offer fast speeds compared to wealthier areas.

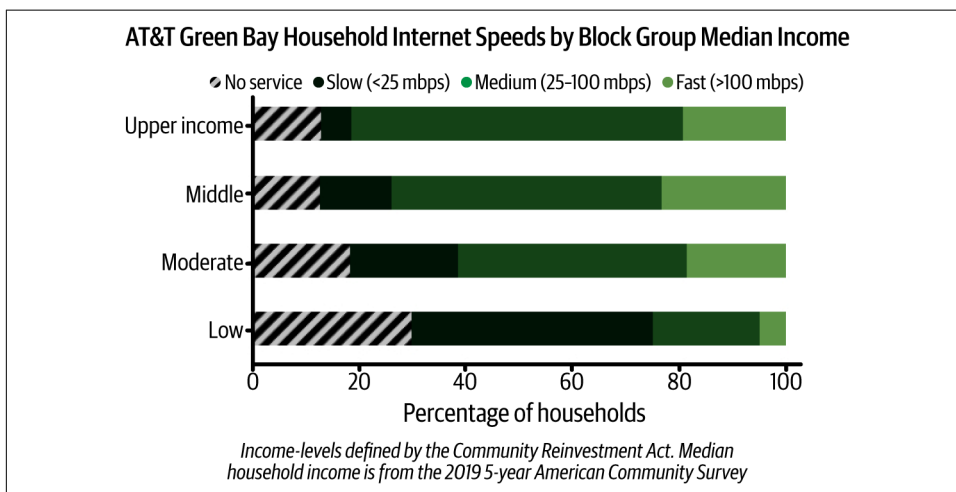


Figure 13-2. Normalized bar charts of AT&T internet plans by median household income

Despite taking two weeks to build the initial dataset, these findings immediately told us we have a story about inequity. The strangest part was that all these internet speeds were quoted at \$55 a month.

We soon found a **report** by the internet advocacy group National Digital Inclusion Alliance (NDIA), which accused AT&T and Verizon of charging the same base price for a variety of speeds based on marketing materials, which they coined “tier flattening.”

This made us wonder: How many companies practiced tier flattening, and how many states or cities did they operate in? How could we collect more data if the initial process was so slow?

We manually confirmed that AT&T, Verizon, CenturyLink, and EarthLink all practiced the same pricing system and that they served 45 states and Washington, DC.

Back-of-the-napkin math that limited our universe to a 10% sample of the most populous city in each of these states yielded 1.6 million addresses. At the current rate, that would have taken almost 15 years to finish.

Although we initially used Selenium to scrape internet plans, 15 years is not a reasonable timeline. Of course, when “interactivity” is a requirement, browser automation is a clear winner that is easy to program and debug.

In our initial appraisal of AT&T’s website, we saw APIs running to serve the information in the service lookup tool as well as return internet plans via API calls. Unlocking this route would allow us the speed and scale we needed, and a quick inspection of each other provider’s website revealed almost identical lookup tools for internet plans.

How would we keep track of the state between each API call? You can’t list plans without reference to the input address, and no clear parameter or header keeps track of this. Luckily, my data editor Jeremy Singer-Vine introduced me to using a **session object**, which does exactly what we need: keep track of headers and cookies across requests.

I used a session to string together a series of API calls to simulate the workflow for listing internet plans for an address, which I repeated in four scrapers for each internet provider. An example of this can be seen in **Example 13-1**, where I collect internet plans from AT&T using multiple API requests and a single requests session.

Example 13-1. Getting internet plans synchronously

```
from requests import Session

def get_internet_plans(address, proxy):
    """
    Collect internet plans for one `address`.
    `session` persists cookies, `proxy` is for routing requests
    """
    with requests.Session() as session:
        session.get(url='att.com/authenticate', proxies=proxy)
        address_id = session.get(
            url='att.com/autocomplete',
            proxies=proxy,
            json={"address": address}
        )
        plans = session.post(
            url='att.com/plans',
            proxies=proxy,
            json={'addressId': address_id}
        )
        return plans.json()
```

This is already significantly faster than what we had accomplished with Selenium. But API calls are scalable in that they can be done either in parallel or asynchronously. With a few changes, the scraper function can be retrofitted to be called asynchronously while choreographing each step (API call) to occur in sequence (Example 13-2).

Example 13-2. Getting internet plans asynchronously

```
import aiohttp

async def get_internet_plans(address, proxy):
    """
    Collect internet plans for one `address`.
    `session` persists cookies, `proxy` is for routing requests
    """
    async with aiohttp.ClientSession() as session:
        await session.get(url='att.com/authenticate', proxies=proxy)
        address_id = await session.get(
            url='att.com/autocomplete',
            proxies=proxy,
            json={"address": address}
        )
        plans = await session.post(
            url='att.com/plans',
            proxies=proxy,
            json={'addressId': address_id}
        )
        return plans.json()
```

One of the issues that Princeton researchers warned us about was rate limiting and IP blocking. We used a residential IP address rotator that routed our requests through someone else's computer. We used the same provider that the Princeton researchers used. IP routing is a last resort that you should use responsibly. We listened to server status codes and scaled accordingly so as not to crash any websites.

Using undocumented APIs and asynchronous programming, we were able to collect internet plans for 300,000 addresses daily, whereas a parallelized Selenium scraper could only collect around 300 addresses a day. This process of developing these new scrapers and optimizing them took about one week.

Unlocking technical barriers allowed us to be ambitious, showing the scale of disparities in an unfair pricing system for an essential commodity. We found that neighborhoods that were offered the worst deals had lower median incomes in 9 out of 10 cities in our analysis. In two-thirds of the cities where we had enough data to compare, the providers gave the worst offers to the neighborhoods with the most non-White residents. Internet access is not considered a utility in the United States and thus is not regulated as strongly as electricity or water.

But before we had our “Big Story,” we were quick, iterative, and pragmatic. We read the prior art and spoke with people to narrow down the complexity. Early tests showed great promise, while thoughtful engineering and high-performance computing provided a full solution that allowed us to reach the potential that this impactful topic deserves.

Data from our story was used to publish original reporting from nine local newsrooms and cited by Los Angeles lawmakers who passed the nation’s first ruling against digital discrimination. The series was honored with several journalism awards, including the Philip Meyer Journalism Award from Investigative Reporters and Editors, which recognizes the best use of social science research methods in journalism.

Although these lessons are from investigative journalism, I hope that these same principles can be applied to the projects that are important to you.

Read more at:

- [“Dollars to Megabits, You May Be Paying 400 Times as Much as Your Neighbor for Internet Service”](#)
- [“How We Uncovered Disparities in Internet Deals”](#)
- [“How We Uncovered Disparities in Internet Deals: GitHub Code and Data Repository”](#)

Lessons from the Field of Cyber Reinsurance

James Poynter

[linkedin.com/in/james-poynter-0b295375](https://www.linkedin.com/in/james-poynter-0b295375)

James Poynter is employed as a Global Data Science Lead at reinsurance broker Gallagher Re. He has a track record of applying data and technology to solve commercial (re)insurance risk and underwriting problems.

As a data scientist, when I think of high performance, I like to think in terms of data flows—I dream of a fast, smooth, and efficient flow of data through a system of robust machine learning and data pipelines. Each ML system comprises a series of steps, and each step involves inputs, transformation, and outputs, with machine learning (ML) model training and inference being just another transformation step (albeit a complex one).

When it comes to unlocking business value from data science and machine learning systems, data must come in and high-quality actionable insights and information

must go out. A performant system in a business context achieves this quickly and profitably via better decision making and trades.

This section illustrates some lessons from the field of insurance analytics and provides a few practical tasters of some of the principles I have personally found to contribute to high performance as a contributor and manager of data science teams working in the (re)insurance sector.

I have picked a handful of Python-related but mostly technically focused lessons, drawing on some of my recent work on predictive risk scores and underwriting tools in the field of cyber reinsurance as a real-world example. These lessons should be applicable in many domains and industries but will likely be more useful to data scientists in financial sectors.

Have a Clear Vision of Both the Problem and the Delivery Requirements

If you read no further than this paragraph, my number one lesson is that high performance starts with really understanding your requirements from first principles. For whatever application you are developing, the requirements should be an honest representation of what's truly REQUIRED to achieve a goal. Before you write a line of code, having a crystal-clear vision of what you need to do, and why you are doing it, is key.

Be willing to ask questions, challenge the status quo, go back to first principles; delete, simplify, and streamline whenever possible—practical high performance is often more about the code you didn't write than the code you did. Focusing consistently and habitually on building a deep understanding of the problem, refining requirements to their essence according to the goal at hand, and documenting, sharing, and constructively challenging those requirements are necessary.

This first principle is something that I find myself needing to revisit and reinforce frequently. It is incredibly easy to find yourself solving the wrong problem, or building functionality, optimizations, and automations which add minimal or no real value. Make good requirements a principle and habit throughout your work and project process.

Domain Knowledge Should Not Be Undervalued

Introductory data science courses often describe data science as the intersection of math and statistics, computer programming, and domain knowledge. Insurance is certainly not the sexiest industry for data scientists, and I often find that junior data scientists are reluctant to invest time to develop domain knowledge and prefer to focus more on algorithms and programming skills, which are seen as more transferable skills.

However, domain knowledge is often critical to ensuring you solve the right problems in the right way. High performance in real-world business settings can often hinge on a team knowing certain key pieces of information. In business contexts, building domain knowledge and a better understanding of the company's core product is time well spent. It adds color to the role, often opens up new opportunities, and breaks down communication barriers.

Here is some domain knowledge relevant to the later lessons:

- Reinsurance is insurance for insurers, allowing insurance companies to transfer a portion of their risk to other reinsurance companies. Reinsurance brokers facilitate this process by providing risk analysis, risk transfer solution design, market placement, and contract administration services. As a data scientist working at a global reinsurance broker, Gallagher Re, my job involves developing predictive risk scores and underwriting tools to support our insurer clients.
- Cyber insurance provides companies and individuals financial protection and support in the event of cyber incidents such as ransomware attacks and data breaches. It's an incredibly important coverage as digital assets become more and more valuable every year.
- Gallagher Re's Technographic Insight Detection Engine (TIDE) is a suite of proprietary risk selection models and tools that combine claim, firmographic, and advanced cyber security scanning data from third-party vendors to develop an enhanced view of cyber insurance risk. TIDE uses a blend of cyber security expertise, machine learning, and explainable AI technology to better understand the predictive power of technographic scanning data and help insurers and data vendors form actionable insights. Our system uses Python extensively, from data cleansing through to ML training and data visualization.

Proactively Guard Against Bad Data

Defensive programming is a programming style focused on anticipating and handling potential errors or unexpected inputs. Typical defensive coding techniques involve input validation, error handling, assertions to check state, and boundary checking. When we view data applications through this lens, data itself is a complex input to our program which should be subject to all of these defensive techniques.

The challenge: A lack of defensive code can adversely impact your data science project

TIDE, like many ML systems, combines multiple internal and external datasets. We receive data from a range of professional data vendors, and directly from insurer clients (often in CSV or Excel). Data impurities and unexpected inputs can slow development and corrupt insights and downstream outputs. As a delivery team we want to make sure we capture issues as quickly as possible so that we can take action.

The opportunity: Leverage Python’s built-in capabilities and ecosystem to guard against bad data

Python has some fantastic built-in tools to unlock performance by guarding against data quality issues. Unfortunately, a lot of these basic language features and third-party tools are often underused in data science projects, the result being that runtime errors go undetected, unexpectedly crash the program, or are difficult to debug.

Let’s start by looking at some of the basics available via the core language:

Assertions

`assert` statements can be used to check conditions at runtime. If the condition is `False`, an `AssertionError` is raised. In a machine learning system, data is a dynamic entity, and we can use assertions to create conditions to check the current state of data and environment. If you find yourself looking at the data in notebooks or running a quick ad hoc check during development, get in the habit of codifying any ad hoc analysis, assumptions, or checks done during development as an assertion.

Exception handling

Python’s `try-except` blocks allow you to handle exceptions gracefully, preventing your program from crashing unexpectedly. Similar to assertions, Python’s `raise` statement enables developers to conditionally raise errors. In production, translate assertions into properly handled exceptions where appropriate. Make sure exceptions raise meaningful error messages.

Logging

The logging module provides a flexible way to log messages, including errors and warnings. This can be helpful for debugging and monitoring in a number of scenarios. But a nice example is using logs to track data transformations and data loss during transformation. In complex pipelines and preprocessing workflows, it’s easy to lose track of data during transformation; logging the key descriptive statistics, data shape, and KPIs can help spot issues.

Type hints

Python 3.5 introduced type hints, which can be used to specify the expected types of variables and function arguments. While not enforced by default, they can improve code readability and help catch type-related errors, particularly when writing code in an IDE.

Testing frameworks

Once the basic features of the core language are being used, the next hygiene factor for your data is ensuring that any custom functions or classes have basic testing:

Unit test

Testing individual functions or classes in isolation

Integration tests

Testing how components can work together

Acceptance tests

Testing required to support acceptance of the software for its intended usage

Testing frameworks are critical to authoring high-quality code. Python’s built-in testing framework is `unittest`; however, I personally use `PyTest`. As a data scientist, I find `PyTest`’s parameterized test capabilities particularly useful for testing functions and methods under a range of inputs; I also find the API intuitive. These tools become particularly helpful where code quality is critical, to support financial calculations, or when creating Python packages which will be reused across the team or projects.

As a data scientist, building familiarity with Python’s testing frameworks and the basic principles of Test-Driven Development (TDD) will help improve the reliability of your code. Testing both the “happy path”—how the application behaves under expected inputs—and the “sad path”—how your code behaves under unexpected inputs—will help make your code much more robust and make it perform better in real-world settings.

Dataframe validation with Pandera

As a data scientist working in finance, I use a lot of tabular data stored in dataframes; a library I use regularly is `Pandera`. The core functionality of `Pandera` is to provide definition and validation of dataframe schemas. It works for many popular dataframe libraries, including `Pandas`, `Polars`, `Dask`, and `PySpark`—with `Pandas` as a first-class citizen, having the greatest functionality.

The `TIDE` project uses dataframes extensively. Being able to validate data at runtime in a lightweight manner helps us quickly discover data issues. When validation fails, `Pandera` ensures that error messages are informative, reducing the time required to debug issues. Adding simple data schema definitions and validation checks enables us to quickly validate data for data quality issues ranging from missing columns and incorrect datatypes to custom boundary conditions.

Create opportunities to look at your data

Manually review samples, automatically generate profiling reports with tools such as ydata-profiling, and use Jupyter Notebooks, logs, dashboards, and visualizations to inspect and summarize your data in a way that’s meaningful to you. Do this for inputs, outputs, and important intermediary transformations. Adding data profiling, writing data samples to disk, or logging often requires just a few lines of code.

The important concept is to make the process of looking at data more systematic and automated. Try to avoid a situation in which the first time you think about looking at your data is after you find an error. Be proactive in creating opportunities to visualize data, not reactive to bugs and issues. Looking at data can have a host of downstream benefits, from building a deeper understanding to finding issues, informing model training and explanations, and even facilitating better-informed conversations with stakeholders.

The lesson

Writing code to guard against data quality issues takes time up front, but the return on investment is realized relatively quickly as iteration becomes faster, ingesting new data sources becomes much easier, and a familiarity with the data informs decision making holistically throughout the system. It’s never too late to add defensive code, but it’s much easier to make a habit and practice of incorporating defensive principles into your development style at the outset.

Manage the Compounding of Technical Debt

Google engineers famously described machine learning systems as the “high interest credit card of technical debt” for good reason. Technical debt is a complex and multifaceted subject. Having a culture that addresses technical debt proactively via continuous improvement is critical, in addition to prioritizing code and data quality. But technology and infrastructure choice also play an important role in the context of machine learning systems.

The challenge: Delivering rapid insights and predictions

Due to the nature of TIDE as an insight detection engine rather than a pure predictive model, we train multiple models on different data schemas and targets—this increases the number of models trained versus a typical predictive model build. When managing so many models, an important component of high performance is making sure we can continue to deliver “quick wins” and new insights to insurer clients and data vendors but also manage the technical debt associated with managing more: data, model training, explanations, deployment, monitoring, and retraining.

The opportunity: Stand on the shoulders of giants

Machine learning systems require a lot of code to manage model deployment, monitoring, and retraining, in addition to the core machine learning code. Historically, the transition from model development to production was a complex process involving multiple teams and technologies. Ensuring reproducibility of experiments and model results necessitates versioning code, data, environments, and experimental artifacts. To address these challenges, machine learning operations (MLOps) practices were developed to streamline the development, deployment, and management of machine learning systems building on DevOps principles.

Like DevOps (development operations), MLOps focuses on automation, collaboration, and continuous integration and delivery. However, MLOps extends beyond DevOps by addressing the unique requirements of machine learning systems, such as data management, versioning, and quality control. MLOps covers the entire machine learning lifecycle, from training to deployment, monitoring, and retraining.

Major cloud providers offer suites of tooling to support MLOps workflows. For TIDE we utilized Microsoft's Azure Machine Learning platform to support our MLOps practices, but similar capabilities are available from Google's Vertex AI and Amazon's SageMaker platforms and via providers such as Databricks. Each of these provides the necessary tools to accelerate development, improve collaboration, and reduce time to market by providing modular high-quality components to manage the ML lifecycle. The key principle is that these platforms enable teams to leverage the combined engineering expertise and infrastructure of some of the world's largest technology companies.

Within the Azure ecosystem, Python is a first-class citizen. Azure provides a Python-based software development kit to interface with its Azure Machine Learning platforms and APIs. When combined with Python's rich ecosystem of data science and machine learning libraries, this enables Python to be the primary language throughout the model development lifecycle, from training to production.

Azure's Machine Learning pipelines enable us to fully orchestrate complex experiments, each involving hundreds of model builds, whilst ensuring complete reproducibility. Because the platform is cloud native, our experiments benefit from scalable storage, and compute is paid for at an hourly rate. Flexible compute is beneficial in enabling us to leverage large compute instances or clusters for specific training runs or even pipeline stages, ensuring effective use of compute resource and, more importantly, developer time and salary.

We also make heavy use of Azure ML's native support for MLflow. MLflow is a widely adopted open source toolkit to help data scientists and machine learning engineers track experiments, package code, deploy models, and store models using a model registry.

The lesson

Technical debt compounds over time; putting in place the right technologies and frameworks early on can help to reduce or, at a minimum, manage technical debt. Tools like Azure Machine Learning are important to many teams as they provide a high-quality platform with prebuilt components that help manage sources of technical debt: data, model training, explanations, deployment, monitoring, and retraining.

Leveraging a Platform as a Service offering like Azure ML means that development teams can spend less time managing technical debt and reinventing the wheel (ML pipelines, deployment, monitoring, etc.) and more time focusing on improving their core product offering. Cloud capabilities enable teams to scale training and deployment to meet their needs.

Relying on battle-tested, well-supported packages like MLflow, with native integrations to Azure ML, improves both the speed and the quality of delivery and removes the development and maintenance overhead of custom solutions.

However, it's not all sunshine and rainbows when it comes to cloud technology. It would be remiss of me not to mention some additional costs:

Learning curve

Utilizing MLOps technologies comes with a steep learning curve requiring an understanding of machine learning, cloud computing, and DevOps practices. This can add additional burden to existing engineers, or additional costs of hiring new talent.

Cloud maturity

Not all companies are using cloud extensively in 2024. The company's wider tech and data stack might require modernization—this can be a particular headache when you are trying to move fast, and it was something that we worked through as part of a digital modernization program at Gallagher Re.

Cultural shift

Embracing DevOps and MLOps practices which typically promote more Agile development patterns can be a real challenge requiring greater collaboration and necessitates overcoming resistance to change. Many companies are still running very manual waterfall-style deployments, with little to no automation of configuration and infrastructure creation.

But these hurdles should not put you off, as better infrastructure and technology can really supercharge a machine learning project and help bring order to the chaos of technical debt that would otherwise ensue.

Build Tools to Augment Human-Centric Workflows

A team working with Python and modern cloud technologies can easily add a huge amount of automation to any data project, but there are often natural limits to the degree of automation that is practically possible. During the TIDE project the data science team found that we were reliant on other teams to prepare data or complete further analysis where an “analyst in the loop” was required; in a perfect world these processes would be fully automated, or removed. However, the reality of working in large financial companies is that full automation is not always feasible or desirable.

It became apparent that processes outside of the ML pipelines were costing us much more time than inefficient code. We could see the potential of creating small applications to augment analyst and subject matter expert workflows in order to enable greater efficiency.

The challenge: Extracting actionable insights frequently necessitates human intervention, but effectively integrating human judgment with machine learning outputs can be difficult without the proper tools

In most data science projects, there comes a time when problems start to emerge that would benefit from creating small browser-based data applications to help move the project forward. Here’s one example from TIDE.

TIDEFlow is our intelligent analytics workflow; it blends human expertise with AI-powered insights to derive actionable knowledge from complex data.

Certain analytical outputs from the modelling process required expert judgment and interpretation. In early versions of TIDE this involved passing various datasets and numerous Matplotlib charts to our cyber security experts for analysis. However, our cyber security experts tended to fall back on technologies such as Microsoft Excel for analysis. This meant that analysis was slow, and auditing or reviewing the decision-making process required the expert in question to diligently record the analysis, commentary, and datasets used to justify their conclusions.

The outcome was that our team spent more time piecing together various pieces of information than they spent doing actual analysis. Refreshing the analysis for new model builds or understanding the provenance of particular conclusions became time consuming and complex; mistakes were easy to make. What we needed was an interactive application which could guide a cyber security expert through an analytics workflow and record their observations and findings.

The opportunity: Utilize lightweight interactive data application frameworks like Streamlit to augment human-centric workflows

Web-based applications are a great way of sharing data science models and analytics tools. In my early career as a data scientist, creating data apps was a very difficult task, as you needed to know Python's data science and web development tech stack (e.g., flask plus JavaScript/CSS/HTML). With enough time and liberal use of jQuery and Twitter Bootstrap, you could create a passable data visualization. It was a lot of work and required a huge amount of time to invest in learning web development skills and writing code, and it was often tricky to deploy and even harder to make reliable and secure.

Fortunately, technology moved on; frameworks like Plotly Dash simplified the creation of interactive data applications. A data scientist could create a fantastic single page application with advanced interactive features in a matter of hours without writing any JavaScript using Dash's components and sophisticated callback mechanisms. More recently, Python's Streamlit library has taken simplification to a whole new level, cutting back data application development to its barest minimum and enabling data scientists to write interactive data applications in just a few lines of code. The simplicity of Streamlit has made it a first choice for data scientists creating data-driven interactive applications.

In combination with AI code assistants, the process of developing a quick prototype becomes extremely rapid. For TIDE we were able to utilize Posit Connect (formerly RStudio Connect) to publish our Python Streamlit application from our Git main branch automatically via the addition of a single manifest JSON file. This meant that through a combination of rapid iteration using Streamlit and rapid deployment using Posit, we could develop and iterate incredibly quickly. In the first day of development, we had a first prototype deployed for testing, and a few days later we had incorporated feedback and added new features and tests.

The TIDEFlow app may only be a few hundred lines of Python code, but the impact on that part of the analytical workflow was night and day different. What had previously been a coffee-laden, tedious, error-prone back-office process, embarked on by one unlucky member of our cyber team for a number of days, became a streamlined and efficient capture of expertise that could be played through in a group setting. What was an arduous solo mission became an interactive expertise-sharing exercise for the rest of the team. Even as a prototype, the look and feel of the app was good enough to put in front of broking teams.

The power of these tools to deliver lightweight rapid prototypes and small data-driven interactive applications cannot be overstated. When we think about high performance from a people perspective, providing colleagues, clients, or yourself with tools to work more quickly and efficiently is a meaningful win. From a psychological perspective,

having the right tool for the job increases motivation and flow and reduces the tiredness and frustration that comes from not having the right tool to hand.

The lesson

There are often processes where cutting the human out of the loop via full automation is neither practical nor desirable. Leveraging a low-code web framework like Streamlit or Dash can help teams innovate by blending the power of Python with subject matter expertise and human judgment. With these frameworks you can build just the right tool to solve their business problems quickly and effectively and delight users.

Summary

Here's a summary of the most important lessons I've learned:

- Have a crystal-clear vision of the problem you are trying to solve and what's truly required to deliver.
- Domain knowledge should not be undervalued.
- When working with data, don't neglect defensive code; proactively guard against bad data.
- Technical debt compounds; leverage the best available technologies and infrastructure to help manage it.
- Where full automation is not possible, build tools to augment human-centric workflows.

Python in Quant Finance

Mikhail Timonin

[linkedin.com/in/mikhail-timonin-a18a2913](https://www.linkedin.com/in/mikhail-timonin-a18a2913)

Mikhail Timonin is a quantitative developer at Engelhart, a commodity trading company, and previously served as director of quantitative development at Aspect Capital. He is passionate about mathematics, software development, and building engineering teams. Before working in finance, Mikhail was in database engineering and earned PhDs in operations research and economics.

In the realm of quantitative finance, Python has firmly established itself as the de facto standard programming language, much like in many other industries. While there remain some pockets where languages like C++ or specialized alternatives like OCaml are preferred, Python dominates most workflows today.

Given that performance is crucial in this field—where quants are acutely aware of the risks of “alpha decay”—the challenge goes beyond merely optimizing a single piece of code for efficiency. It becomes more complex when considering the long-term evolution of codebases and the multidisciplinary teams working on them.

Quantitative finance typically involves complex systems composed of various time-series forecasting models, whose outputs are combined and processed through additional stages, such as portfolio optimization.

A typical quant pipeline is a multistage process with contributions from specialists in various domains, many of whom may not have extensive expertise in software development. In an ideal scenario, there would be a clear separation of duties, with “researchers” exploring new ideas and “developers” focusing on implementing efficient, production-ready models. However, in practice, the need for speed often blurs these roles, making such a division less rigid. These are the key considerations:

Time-series data

Much of our work revolves around time-series data—whether it’s prices, FX rates, or macroeconomic indicators. Consequently, our systems are inherently path-dependent, with today’s decisions influenced by prior data points.

Data arrival

Systems need to react to incoming data, whether on a fixed schedule or in response to specific events. Processing this data as quickly as possible is paramount, with high-frequency trading (HFT) being an extreme example of this need.

Backtesting

Backtesting, or replaying strategies through historical data, is a fundamental operation for researchers and analysts. However, this can be time-consuming, particularly when using naive “loop” approaches where data is fed into models one timestamp at a time. Such an approach can be inefficient, creating friction within teams that would rather not wait for hours to complete backtests.

Optimizing performance

The challenge lies in optimizing code to be both fast for live data updates and efficient for backtesting and research. Code should remain maintainable, and the barrier for developing new ideas must be kept low.

The Vectorization Approach

The natural starting point for optimizing performance is vectorization. Researchers often use libraries like Pandas and NumPy to load historical datasets and quickly test their models. While these tools offer easy-to-implement solutions that can perform

reasonably well for backtesting, they may not be suitable for live execution due to path-dependency.

Consider the case of having to load 10–20 years of “warmup” data and computing a path through time to the present moment just so that one last step can be made. Even if backtesting performance of such code is acceptable, the time required to process the data could still take seconds or minutes, which might be too slow in a real-time setting.

Vectorization presents additional challenges:

Memory usage

The memory footprint increases, especially when running parallel operations.

Look-ahead bias

Accidental look-ahead errors are a risk when manipulating “whole history” datasets rather than “as of” data.

Limitations

Some computations, such as portfolio optimization, cannot be easily vectorized.

Lightweight State Machines

At the other end of the spectrum, some systems implement lightweight state machines. These machines process incoming data, update state information, and perform small computations rapidly. For instance, a system might react to a news update and adjust existing positions in real time. However, this approach comes with its own set of problems:

Efficiency

Writing highly efficient code is difficult, particularly for researchers without code profiling expertise. Optimizing performance of every single function call can become an overwhelming task.

Time complexity

Even efficient code can slow down due to the need to iterate through large time-series data sequentially (especially using tools like Pandas) and merging results.

Path dependency

Since pipeline steps at time T depend on prior computations (at $T-1$), storing state in live executions becomes an additional challenge, and backtesting is inherently sequential, preventing parallelization across time periods.

Designing for Performance

When embarking on a quant finance project, it's essential to focus early on where performance is most critical and to consider the skill sets of the team members. Design decisions should not be postponed, as it's crucial to strike a balance between optimizing for performance and ensuring the system remains maintainable and accessible. Here are some practical tips:

Divide and conquer

Break the system into modular components that can be optimized independently. Foster component reuse by establishing an explicit, declarative graph structure (e.g., “node A connects to node B”) where nodes can cache results. This subject deserves a separate discussion, but ask yourself this question: if a small change is made in one of the models that contribute to the portfolio, can we see how that affects the overall results without recomputing everything? That is the question your researchers would often want to explore.

Leverage Numba and vectorization

Combine vectorization with Numba for localized performance boosts. Numba excels in optimizing small, reusable components. For instance, computing historical portfolio risk using a matrix of positions and a risk matrix, both of varying sizes, can benefit from Numba's ability to efficiently loop through time-series data without incurring the memory overhead that traditional vectorized solutions might cause. In this case you have a matrix of portfolio positions called P , that has the shape of $T \times N$, and daily (or coarser) risk matrix S , with shape $t \times N \times N$, where $T \gg t$ are timestamps of your data. You want to compute a matrix product PSP^T for each of the timestamps in T . Doing it in a naive way with NumPy will first blow up S to $T \times N \times N$, and you might well be out of memory by then. In contrast, a Numba-accelerated loop can efficiently hop through timestamps in T , picking up the appropriate positions (exact) and risk (as-of) slices, and do the job in no time. Such components are also likely to be highly appreciated and reused by your researchers.

Consider Polars over Pandas

For new projects, consider using Polars, a dataframe library optimized for performance. It can offer significant improvements over Pandas, especially when working with larger datasets or when parallelization is important.

Scale backtesting

For backtesting, consider horizontally scaling your workload. Researchers often want to explore “what if” scenarios or parameter spaces, requiring the execution of multiple backtests with slightly varied inputs. Instead of running them sequentially, deploy these backtests on a Kubernetes cluster as containerized jobs, letting the entire batch complete in the same time it would take to run a single test.

In conclusion, optimizing for both real-time and backtest performance in quantitative finance requires a careful balance between computational efficiency, maintainability, and flexibility. The key to success lies in making strategic design choices early, modularizing the system, and leveraging the right tools and techniques for the task at hand.

Maintain Flexibility to Achieve High Performance

David Rawlinson

[linkedin.com/in/david-rawlinson-51342682](https://www.linkedin.com/in/david-rawlinson-51342682)

David Rawlinson is a principal data scientist at WSP, a global engineering consulting firm. In this role he creates bespoke simulation, optimization, and predictive machine learning solutions for engineering problems, particularly asset management and operations in large-scale network infrastructure, including road, rail, and water. He has a PhD in robotics and computer vision from Monash University in Australia.

David is also a founder at [Causal Wizard](#), a website that educates and provides tools for the use of causal machine learning techniques. Under the hood, Causal Wizard uses [DoWhy](#), a Python Causal Inference toolkit, to which David is a contributor.

My number one piece of advice for achieving high performance programs is to scrupulously maintain flexibility in your code. This is because optimization of a defined, efficiently implemented computational process tends to give speedups that are about one order of magnitude (say, 2 to 10 times), whereas new ideas and approaches are more likely to enable 10 or 100 times speedups (these aren't statistics—these are indicative numbers from personal experience).

Your Greatest Constraint Is Your Own Time

Code complexity limits your ability to optimize because it takes time to implement the necessary changes. Worse, many optimization techniques increase code complexity, making it harder to change the program in future.

This means that premature line-by-line optimization might help you achieve small speedups now but could prevent you from radically refactoring your code to achieve a 100 times speedup later, because you simply don't have the time to implement new ideas in the constraints of the codebase.

In technical computing you'll spend a lot of time verifying that results are correct. Many optimizations (such as vectorization or replacing conditional logic with vector-mask operations—see the following example) make the code harder to read, verify, and debug. Make sure you structure your code well, minimize coupling, and avoid

all unnecessary complexity, especially early in development while everything is still in flux. This will maintain your ability to rapidly refactor, enabling you to try new ideas that emerge as you spend more time grappling with the problem.

As an example, you can implement a nonforking logical-AND operation with element-wise multiplication, and you can make a logical OR with $\max(a, b)$ and a NOT with $1-x$. This is less readable but much faster on floating-point vector or tensor data using many modern libraries.

Moving the Goalposts

Flexibility also helps you to redefine the problem. This is an incredible optimization technique, because sometimes the problem can be redefined out of existence, leading to a runtime of *zero*; in fact, I notice one of the other stories in this chapter describes this scenario (replacing a machine learning model with a simple database query).

Even when a problem remains, changing the problem can lead to huge performance improvements. One trick which often works is to implement a fast, approximate solution followed by a slower, exact solution. For example, imagine you wanted to compute the intersections of individual dwelling geometry with various overlays such as flood zones or land use, on a national scale. And you need to do it quickly enough for a synchronous web request.

We can drastically reduce runtime by exploiting a property of the shapes—they're small. Only a tiny fraction of shapes will intersect with the query region, which means if we can quickly identify those which might intersect and then check only those carefully, we'll still get the right answer. The approximate match needs to have a false-positive bias (finds too many shapes, but never misses one).

A first optimization step might be to replace exact shape intersection with the intersection of shape bounding boxes. This is much faster to evaluate and has the correct bias. However, we're still examining billions of shape combinations.

We can make the solution even faster by precomputing a spatial “index” for each shape (perhaps with a structure like a K-D Tree), which allows us to immediately return subsets of shapes which fall into particular coordinate ranges and then examine only those precisely. We've exchanged precomputed storage for query-time performance.

When it's difficult to design an efficient approximate function due to the complexity of the problem, you can sometimes use ML models. ML inference is generally pretty fast and easy to tune by changing model size. Models can be trained to have a preferred bias, and often you don't need the model to generalize because you can include the edge cases in the training set.

New Perspectives

All of these tricks rely on you having a very clear understanding of the problem and your constraints. Listening and having the opportunity to challenge assumptions is really important, because what's efficient for a computer is different from what's efficient for people.

Subject matter experts will often have a very different idea of what's possible, and it might take some persistent questioning to nudge them out of established thinking based on human constraints. They may also have great ideas, which you can elicit if you ask the right questions.

Organizations often create a lot of process complexity to minimize the total amount of human work, when for a computer it would be more efficient to do a simpler process more exhaustively, with efficiencies introduced elsewhere (such as caching partial solutions).

Starting a New Project

Starting a new project can be daunting. You know you're going to be pushing limits simply due to data volumes or algorithm complexity. When you've got a blank slate it can be difficult to know where to start. This is the programmer's version of writer's block!

I have two suggestions. First, you can get started with the known parts of the project—test harnesses, instrumentation, model performance evaluation, and data integration interfaces. These tasks are a great investment because they put some framing around the actual problem, and it's really important to be able to evaluate and verify candidate solutions systematically. And it will help to clarify the specification of the unknown algorithm or model.

Next, the model or algorithm itself. I suggest writing something—anything—in pseudocode, diagrams, flowcharts—whatever works for you. Take specific pieces of the problem and play with different formulations of it. Try to chain it all together. Don't worry about actually running anything at this stage; your intuition is fine. When you feel that it should all come together, gradually start translating it into actual code in your preferred language.

The Value of Good Data

You absolutely must have good data for data science and ML projects. Often, this data doesn't exist, and creating it becomes your first task. You'll have to figure out how to join different tables or databases and deal with duplicate, outdated, or broken references, mapping loosely structured string data to strict categories. This will be

difficult because you'll have to get everyone to agree on the definitive version of each data field and how it should be interpreted.

Once you've done this, recognize that the data you've created usually has enormous value regardless of the original purpose of your project. It's worth letting everyone know about the success you've had with the data and giving them opportunities to use it for other purposes. This way, your project is a win even before you deliver the solution.

Streamlining Feature Engineering Pipelines with Feature-engine (2020)

Soledad Galli
trainindata.com

Soledad Galli is the lead data scientist and founder of Train in Data. She has experience in finance and insurance, received a Data Leadership Award in 2018, and was selected as one of LinkedIn's "Top Voices" in data science and analytics in 2019. Soledad is passionate about sharing knowledge and helping others succeed in data science.

Train in Data is an education project led by experienced data scientists and AI software engineers. We help professionals improve coding and data science skills and adopt machine learning best practices. We create advanced online courses on machine learning and AI software engineering and open source libraries, like **Feature-engine**, to smooth the delivery of machine learning solutions.

Feature Engineering for Machine Learning

Machine learning models take in a bunch of input variables and output a prediction. In finance and insurance, we build models to predict, for example, the likelihood of a loan being repaid, the probability of an application being fraudulent, and whether a car should be repaired or replaced after an accident. The data we collect and store or recall from third-party APIs is almost never suitable to train machine learning models or return predictions. Instead, we transform variables extensively before feeding them to machine learning algorithms. We refer to the collection of variable transformations as *feature engineering*.

Feature engineering includes procedures to impute missing data, encode categorical variables, transform or discretize numerical variables, put features in the same scale, combine features into new variables, extract information from dates, aggregate transactional data, and derive features from time series, text, or even images. There are

many techniques for each of these feature engineering steps, and your choice will depend on the characteristics of the variables and the algorithms you intend to use. Thus, when feature engineers build and consume machine learning in organizations, we do not speak of machine learning models but of machine learning pipelines, where a big part of the pipeline is devoted to feature engineering and data transformation.

The Hard Task of Deploying Feature Engineering Pipelines

Many feature engineering transformations learn parameters from data. I have seen organizations utilize config files with hardcoded parameters. These files limit versatility and are hard to maintain (every time you retrain your model, you need to rewrite the config file with the new parameters). To create highly performant feature engineering pipelines, it's better to develop algorithms that automatically learn and store these parameters and can also be saved and loaded, ideally as one object.

At Train in Data, we develop machine learning pipelines in the research environment and deploy them in the production environment. These pipelines should be reproducible. Reproducibility is the ability to duplicate a machine learning model exactly, such that, given the same data as input, both models return the same output. Utilizing the same code in the research and production environment smooths the deployment of machine learning pipelines by minimizing the amount of code to be rewritten, maximizing reproducibility.

Feature engineering transformations need to be tested. Unit tests for each feature engineering procedure ensure that the algorithm returns the desired outcome. Extensive code refactoring in production to add unit and integration tests is extremely time-consuming and provides new opportunities to introduce bugs, or to find bugs introduced during the research phase due to the lack of testing. To minimize code refactoring in production, it is better to introduce unit testing as we develop the engineering algorithms in the research phase.

The same feature engineering transformations are used across projects. To avoid different code implementations of the same technique, which often occur in teams with many data scientists, and to enhance team performance, speed up model development, and smooth model operationalization, we want to reuse code that was previously built and tested. The best way to do that is to create in-house packages. Creating packages may seem time-consuming, since it involves building tests and documentation. But it is more efficient in the long run, because it allows us to enhance code and add new features incrementally, while reusing code and functionality that has already been developed and tested. Package development can be tracked through versioning and even shared with the community as open source, raising the profile of the developers and the organization.

Leveraging the Power of Open Source Python Libraries

It's important to use established open source projects or thoroughly developed in-house libraries. This is more efficient for several reasons:

- Well-developed projects tend to be thoroughly documented, so it is clear what each piece of code intends to achieve.
- Well-established open source packages are tested to prevent the introduction of bugs, ensure that the transformation achieves the intended outcome, and maximize reproducibility.
- Well-established projects have been widely adopted and approved by the community, giving you peace of mind that the code is of quality.
- You can use the same package in the research and production environment, minimizing code refactoring during deployment.
- Packages are clearly versioned, so you can deploy the version you used when developing your pipeline to ensure reproducibility, while newer versions continue to add functionality.
- Open source packages can be shared, so different organizations can build tools together.
- While open source packages are maintained by a group of experienced developers, the community can also contribute, providing new ideas and features that raise the quality of the package and code.
- Using a well-established open source library removes the task of coding from our hands, improving team performance, reproducibility, and collaboration, while reducing model research and deployment timelines.

Open source Python libraries like **scikit-learn**, **Category encoders**, and **Featuretools** provide feature engineering functionality. To expand on existing functionality and smooth the creation and deployment of machine learning pipelines, I created the open source Python package **Feature-engine**, which provides an exhaustive battery of feature engineering procedures and supports the implementation of different transformations to distinct feature spaces.

Feature-engine Smooths Building and Deployment of Feature Engineering Pipelines

Feature engineering algorithms need to learn parameters from data automatically, return a data format that facilitates use in research and production environments, and include an exhaustive battery of transformations to encourage adoption across projects. Feature-engine was conceived and designed to fulfill all these requirements. Feature-engine transformers—that is, the classes that implement a

feature engineering transformation—learn and store parameters from data. Feature-engine transformers return Pandas DataFrames, which are suitable for data analysis and visualization during the research phase. Feature-engine supports the creation and storage of an entire end-to-end engineering pipeline in a single object, making deployment easier. And to facilitate adoption across projects, it includes an exhaustive list of feature transformations.

Feature-engine includes many procedures to impute missing data, encode categorical variables, transform and discretize numerical variables, and remove or censor outliers. Each transformer can learn, or alternatively have specified, the group of variables it should modify. Thus the transformer can receive the entire dataframe, yet it will alter only the selected variable group, removing the need of additional transformers or manual work to slice the dataframe and then join it back together.

Feature-engine transformers use the fit/transform methods from scikit-learn and expand its functionality to include additional engineering techniques. Fit/transform functionality makes Feature-engine transformers usable within the scikit-learn pipeline. Thus with Feature-engine we can store an entire machine learning pipeline into a single object that can be saved and retrieved or placed in memory for live scoring.

Helping with the Adoption of a New Open Source Package

No matter how good an open source package is, if no one knows it exists or if the community can't easily understand how to use it, it will not succeed. Making a successful open source package entails making code that is performant, well tested, well documented, and useful—and then letting the community know that it exists, encouraging its adoption by users who can suggest new features, and attracting a developer community to add more functionality, improve the documentation, and enhance code quality to raise its performance. For a package developer, this means that we need to factor in time to develop code and to design and execute a sharing strategy. Here are some strategies that have worked for me and for other package developers.

We can leverage the power of well-established open source functionality to facilitate adoption by the community. scikit-learn is the reference library for machine learning in Python. Thus, adopting scikit-learn fit/transform functionality in a new package facilitates an easy and fast adoption by the community. The learning curve to use the package is shorter, as users are already familiar with this implementation. Some packages that leverage the use of fit/transform are [Keras](#), Category encoders (perhaps the most renowned), and of course Feature-engine.

Users want to know how the package can be used and shared, so include a license stating these conditions in the code repository. Users also need instructions and examples of the code functionality. Docstrings in code files with information about functionality and examples of its use are a good start, but they are not enough.

Widespread packages include additional documentation (which can be generated with reStructuredText files) with descriptions of code functionality, examples of its use and the outputs returned, installation guidelines, the channels where the package is available (PyPI, conda), how to get started, changelogs, and more. Good documentation should empower users to use the library without reading the source code. The documentation of the machine learning visualization library [Yellowbrick](#) is a good example. I have adopted this practice for Feature-engine as well.

How can we increase package visibility? How do we reach potential users? Teaching an online course can help you reach people, especially if it's on a prestigious online learning platform. In addition, publishing documentation in [Read the Docs](#), creating YouTube tutorials, and presenting the package at meetups and meetings all increase visibility. Presenting the package functionality while answering relevant questions in well-established user networks like Stack Overflow, Stack Exchange, and Quora can also help. The developers of Featuretools and Yellowbrick have leveraged the power of these networks. Creating a dedicated Stack Overflow issues list lets users ask questions and shows the package is being actively maintained.

Developing, Maintaining, and Encouraging Contribution to Open Source Libraries

For a package to be successful and relevant, it needs an active developer community. A developer community is composed of one or, ideally, a group of dedicated developers or maintainers who will watch for overall functionality, documentation, and direction. An active community allows and welcomes additional ad hoc contributors.

One thing to consider when developing a package is code maintainability. The simpler and shorter the code, the easier it is to maintain, which makes it more likely to attract contributors and maintainers. To simplify development and maintainability, Feature-engine leverages the power of scikit-learn base transformers. scikit-learn provides an API with a bunch of base classes that developers can build upon to create new transformers. In addition, scikit-learn's API provides many tests to ensure compatibility between packages and also that the transformer delivers the intended functionality. By using these, Feature-engine developers and maintainers focus only on feature engineering functionality, while the maintenance of base code is taken care of by the bigger scikit-learn community. This, of course, has a trade-off. If scikit-learn changes its base functionality, we need to update our library to ensure it is compatible with the latest version. Other open source packages that use scikit-learn API are Yellowbrick and Category encoders.

To encourage developers to collaborate, NumFOCUS recommends creating a code of conduct and encouraging inclusion and diversity. The project needs to be open, which generally means the code should be publicly hosted, with guidelines to orient new contributors about project development and discussion forums open to public participation, like a mailing list or a Slack channel. While some open source Python libraries have their own codes of conduct, others, like Yellowbrick and Feature-engine, follow the [Python Software Foundation Code of Conduct](#). Many open source projects, including Feature-engine, are publicly hosted on GitHub. Contributing guidelines list ways new contributors can help—for example, by fixing bugs, adding new functionality, or enhancing the documentation. Contributing guidelines also inform new developers of the contributing cycle, how to fork the repository, how to work on the contributing branch, how the code review cycle works, and how to make a pull request.

Collaboration can raise the quality and performance of the library by enhancing code quality or functionality, adding new features, and improving documentation. Contributions can be as simple as reporting typos in the documentation, reporting code that doesn't return the intended outcome, or requesting new features. Collaborating on open source libraries can also help raise the collaborator's profile while exposing them to new engineering and coding practices, improving their skills.

Many developers and data scientists believe that they need to be top-notch developers to contribute to open source projects. I used to believe that myself, and it discouraged me from making contributions or requesting features—even though, as a user, I had a clear idea of what features were available and which were missing. This is far from true. Any user can contribute to the library. And package maintainers love contributions.

Useful contributions to Feature-engine have included simple things like adding a line to the `.gitignore`, sending a message through LinkedIn to bring typos in the docs to my attention, making a PR to correct typos themselves, highlighting warning issues raised by newer versions of scikit-learn, requesting new functionality, or expanding the battery of unit tests.

If you want to contribute but have no experience, it is useful to go through the issues found on the package repository. Issues are lists with the modifications to the code that have priority. They are tagged with labels like “code enhancement,” “new functionality,” “bug fix,” or “docs.” To start, it is good to go after issues tagged as “good first issue” or “good for new contributors”; those tend to be smaller code changes and will allow you to get familiar with the contribution cycle. Then you can jump into more complex code modifications. Just by solving an easy issue, you will learn a lot about software development, Git, and code review cycles.

Feature-engine is currently a small package with straightforward code implementations. It is easy to navigate and has few dependencies, so it is a good starting point for contributing to open source. If you want to get started, do get in touch. I will be delighted to hear from you. Good luck!

Highly Performant Data Science Teams (2020)

Linda Uruchurtu (Fiit)

Linda Uruchurtu is a senior data scientist and software engineer, currently working at Fiit. Since 2013, she has been helping small-to-medium start-ups build products using data. She has experience in analytics, statistics, machine learning, and product building in a variety of industries, including transportation, retail, health, and fitness.

Linda holds a PhD in theoretical physics from Cambridge University. She has been both a speaker and a reviewer at PyData London and has served as chair of the PyData London review committee since 2017.

Data science teams are different from other technical teams, because the scope of what they do varies according to where they sit and the type of problems they tackle. However, whether the team is responsible for answering “why” and “how” questions or simply delivering fully operational ML services, in order to deliver successfully, it needs to keep the stakeholders happy.

This can be challenging. Most data science projects have a degree of uncertainty attached to them, and since there are different types of stakeholders, “happy” can mean different things. Some stakeholders might be concerned only with final deliverables, whereas others might care about side effects or common interfaces. Additionally, some might not be technical or might have a limited understanding of the specifics of the project. Here I will share some lessons I have learned that make a difference in the way projects are carried out and delivered.

How Long Will It Take?

This is possibly the most common question data science team leads are asked. Picture the following: Management asks the project manager (PM), or whoever is responsible for delivery, to solve a given problem. The PM goes to the team, presents it with this information, and asks the team to plan a solution. Cue this question, from the PM or from other stakeholders: How long will it take?

First, the team should ask questions to better define the scope of its solutions. These might include the following:

- Why is this a problem?
- What is the impact of solving this problem?
- What is the definition of done?
- What is the minimal version of a solution that satisfies said definition?
- Is there a way to validate a solution early on?

Notice that “How long will it take?” isn’t on this list.

The strategy should be twofold. First, get a time-boxed period to ask these questions and propose one or more solutions. Once a solution is agreed upon, the PM should explain to stakeholders that the team can provide a timeline once the work for said solution is planned.

Discovery and Planning

The team has a fixed amount of time to come up with solutions. What’s next? It needs to generate hypotheses, followed by exploratory work and quick prototyping, to keep or discard potential solutions successively.

Depending on the solution chosen, other teams may become stakeholders. Development teams could have requirements from APIs they hold, or they could become consumers of a service; product, operations, customer service, and other teams might use visualizations and reports. The PM’s team should discuss its ideas with these teams.

Once this process has taken place, the team is usually in a good position to determine how much uncertainty and/or risk adheres to each option. The PM can now assess which option is preferred.

Once an option is chosen, the PM can define a timeline for milestones and deliverables. Useful points to raise here are as follows:

- Can the deliverables be reasonably reviewed and tested?
- If work depends on other teams, can work be scheduled so no delay is introduced?
- Can the team provide value from intermediate milestones?
- Is there a way to reduce risk from parts of the project that have a significant amount of uncertainty?

Tasks derived from the plan can then be sized and time-boxed to provide a time estimate. It is a good idea to allow for extra time: some people like to double or triple the time they think they'll need!

Some tasks are frequently underestimated and simplified, including data collection and dataset building, testing, and validation. Getting good data for model building can often be more complex and expensive than it initially seems. One option may be to start with small datasets for prototyping and postpone further collection until after proof of concept. Testing, too, is fundamental, both for correctness and for reproducibility. Are inputs as expected? Are processing pipelines introducing errors? Are outputs correct? Unit testing and integration tests should be part of every effort. Finally, validation is important, particularly in the real world. Be sure to factor in realistic estimates for all of these tasks.

Once you've done that, the team has not only an answer to the "time" question but also a plan with milestones that everyone can use to understand what work is being carried out.

Managing Expectations and Delivery

Lots of issues can affect the time required before a delivery is achieved. Keep an eye on the following points to make sure you manage the team's expectations:

Scope creep

Scope creep is the subtle shifting of the scope of work, so that more work is expected than was initially planned. Pairing and reviews can help mitigate this.

Underestimating nontechnical tasks

Discussions, user research, documentation, and many other tasks can easily be underestimated by those who don't know them well.

Availability

Team members' scheduling and availability can also introduce delays.

Issues with data quality

From making sure working datasets are good to go to discovering sources of bias, data quality can introduce complications or even invalidate pieces of work.

Alternative options

When unexpected difficulties arise, it might make sense to consider other approaches. However, sunk costs might prevent the team from wanting to raise this possibility, which could delay the work and risk creating the impression that the team members don't know what they are doing.

Lack of testing

Sudden changes in data inputs or bugs in data pipelines can invalidate assumptions. Having good test coverage from the start will improve team velocity and pay dividends at the end.

Difficulty testing or validating

If not enough time is allowed for testing and validating hypotheses, the schedule can be delayed. Changes in assumptions can also lead to alterations in the testing plan.

Use weekly refinement and planning sessions to spot issues and discuss whether tasks need to be added or removed. This will give the PM enough information to update the final stakeholders. Prioritization should also happen with the same cadence. If opportunities arise to do some tasks earlier than expected, these should be pushed forward.

Intermediate deliverables, particularly if they provide value outside the scope of the project, continuously justify the work. That's good for the team, in terms of focus and morale, as well as stakeholders, who will have a sense of progress. The continuous process of redrawing the game plan and reviewing and adjusting iterations will ensure the team has a clear sense of direction and freedom to work, while providing enough information and value to keep stakeholders keen on continuing to support the project.

To become highly performant while tackling a new project, your data science team's main focus has to be on making data uncertainty and business-need uncertainty less risky by delivering lightweight minimum viable product (MVP) solutions (think scripts and Python notebooks). The initial conceived MVP might actually turn out to be leaner than or different from the first concept, given findings down the line or changes in business needs. Only after validation should you proceed with a production-ready version.

The discovery and planning process is critical, and so is thinking in terms of iterations. Keep in mind that the discovery phase is always ongoing and that external events will always affect the plan.

Numba (2020)

Valentin Haenel
haenel.co

Valentin Haenel is a longtime “Python for Data” user and developer who still remembers hearing Travis Oliphant’s keynote about NumPy at the first EuroSciPy conference in 2008. He then proceeded to use Python for simple modeling of spiking neurons and to evaluate data from perception experiments while pursuing his master’s degree in computational neuroscience. Since then he has contributed to more than 80 open source projects. He now works for Anaconda as an open source developer on the Numba project.

The author would like to thank three of the Numba core developers, Stuart Archibald, Siu Kwan Lam, and Stan Seibert, for productive discussions, advice, and feedback about this text.

Numba is an open source, JIT function compiler for numerically focused Python. Initially created at Continuum Analytics (now Anaconda, Inc.) in 2012, it has since grown into a mature open source project on GitHub with a large and diverse group of contributors. Its primary use case is the acceleration of numerical and/or scientific Python code. The main entry point is a decorator—the `@jit` decorator, which is used to annotate the specific functions, ideally the bottlenecks of the application, that should be compiled. Numba will compile these functions just-in-time, which simply means that the function will be compiled at the first, or initial, execution. All subsequent executions with the same argument types will then use the compiled variant of the function, which should be faster than the original.

Numba not only can compile Python but also is NumPy-aware and can handle code that uses NumPy. Under the hood, Numba relies on the well-known [LLVM project](#), a collection of modular and reusable compiler and toolchain technologies. Last, Numba isn’t a fully fledged Python compiler. It can compile only a subset of both Python and NumPy—albeit a large enough subset to make it useful in a wide range of applications. For more information, please consult the [documentation](#).

A Simple Example

As a simple example, let’s use Numba to accelerate a Python implementation of an ancient algorithm for finding all prime numbers up to a given maximum (N): the sieve of Eratosthenes. It works as follows:

- First, initialize a Boolean array of length N to all true values.
- Then, starting with the first prime, 2, cross off (i.e., set the position in the Boolean list corresponding to that number to false) all multiples of the number up to N .
- Proceed to the next number that has not yet been crossed off, in this case 3, and again cross off all multiples of it.
- Continue to proceed through the numbers and cross off their multiples until you reach N .
- When you reach N , all the numbers that have not been crossed off are the set of prime numbers up to N .

A reasonably efficient Python implementation might look something like this:

```
import numpy as np
from numba import jit

@jit(nopython=True) # simply add the jit decorator
def primes(N=100000):
    numbers = np.ones(N, dtype=np.uint8) # initialize the boolean array
    for i in range(2, N):
        if numbers[i] == 0: # has previously been crossed off
            continue
        else: # it is a prime, cross off all multiples
            x = i + i
            while x < N:
                numbers[x] = 0
                x += i
    # return all primes, as indicated by all boolean positions that are one
    return np.nonzero(numbers)[0][2:]
```

After placing this in a file called *sieve.py*, you can use the `%timeit` magic to micro-benchmark the code:

```
In [1]: from sieve import primes

In [2]: primes() # run it once to make sure it is compiled
Out[2]: array([ 2, 3, 5, ..., 99971, 99989, 99991])

In [3]: %timeit primes.py_func() # 'py_func' contains
                                   # the original Python implementation
145 ms ± 1.86 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [4]: %timeit primes() # this benchmarks the Numba compiled version
340 µs ± 3.98 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

That is a speedup of roughly four hundredfold; your mileage may vary. Nonetheless, there are a few things of interest here:

- Compilation happened at the function level.
- Simply adding the decorator `@jit` was enough to instruct Numba to compile the function. No further modifications to the function source code, such as type annotations of the variables, were needed.
- Numba is NumPy-aware, so all the NumPy calls in this implementation were supported and could be compiled successfully.
- The original, pure Python function is available as the `py_func` attribute of the compiled function.

There is a faster but less educational version of this algorithm, the implementation of which is left to the interested reader.

Best Practices and Recommendations

One of the most important recommendations for Numba is to use `nopython` mode whenever possible. To activate this mode, simply use the `nopython=True` option with the `@jit` decorator, as shown in the prime number example. Alternatively, you can use the `@njit` decorator alias, which is accessible by doing `from numba import njit`. In `nopython` mode, Numba attempts a large number of optimizations and can significantly improve performance. However, this mode is very strict; for compilation to succeed, Numba needs to be able to infer the types of all the variables within your function.

You can also use object mode by doing `@jit(forceobj=True)`. In this mode, Numba becomes very permissive about what it can and cannot compile, which limits it to performing a minimal set of optimizations. This is likely to have a significant negative effect on performance. To take advantage of Numba's full potential, you really should use `nopython` mode.

If you can't decide whether you want to use object mode or not, there is the option to use an object-mode block. This can come in handy when only a small part of your code needs to execute in object mode—for example, if you have a long-running loop and would like to use string formatting to print the current progress of your program. For example:

```
from numba import njit, objmode

@njit()
def foo():
    for i in range(1000000):
        # do compute
        if i % 100000 == 0:
```

```

with objmode: # to escape to object-mode
    # using 'format' is permissible here
    print("epoch: {}".format(i))

```

```
foo()
```

Pay attention to the types of the variables that you use. Numba works very well with both NumPy arrays and NumPy views on other datatypes. Therefore, if you can, use NumPy arrays as your preferred data structure. Tuples, strings, enums, and simple scalar types such as int, float, and Boolean are also reasonably well supported. Globals are fine for constants, but pass the rest of your data as arguments to your function. Python lists and dictionaries are unfortunately not very well supported. This largely stems from the fact that they can be type heterogeneous: a specific Python list may contain differently typed items—for example, integers, floats, and strings. This poses a problem for Numba, because it needs the container to hold only items of a single type in order to compile it. However, these two data structures are probably one of the most used features of the Python language and are even one of the first things programmers learn about.

To remedy this shortcoming, Numba supports the so-called typed containers: `typed-list` and `typed-dict`. These are homogeneously typed variants of the Python list and dict. This means that they may contain only items of a single type: for example, a `typed-list` of only integer values. Beyond this limitation, they behave much like their Python counterparts and support a largely identical API. Additionally, they may be used within regular Python code or within Numba compiled functions, and they can be passed into and returned from Numba compiled functions. These are available from the `numba.typed` submodule. Here is a simple example of a `typed-list`:

```

from numba import njit
from numba.typed import List

@njit
def foo(x):
    """ Copy x, append 11 to the result. """
    result = x.copy()
    result.append(11)
    return result

a = List() # Create a new typed-list
for i in (2, 3, 5, 7):
    # Add the content to the typed-list,
    # the type is inferred from the first item added.
    a.append(i)
b = foo(a) # make the call, append 11; this list will go to eleven

```

While Python does have limitations, you can rethink them and understand which ones can be safely disregarded when using Numba. Two specific examples come to mind: calling functions and for loops. Numba enables a technique called *inlining* in

the underlying LLVM library to optimize away the overhead of calling functions. This means that during compilation, any function calls that are amenable to inlining are replaced with a block of code that is the equivalent of the function being called. As a result, there is practically no performance impact from breaking up a large function into a few or many small ones in order to aid readability and comprehensibility.

One of the main criticisms of Python is that its for loops are slow. Many people recommend using alternative constructs instead when attempting to improve the performance of a Python program—list comprehensions, for example, or even NumPy arrays. Numba does not suffer from this limitation, and using for loops in Numba compiled functions is fine. Observe:

```
from numba import njit

@njit
def numpy_func(a):
    # uses Numba's implementation of NumPy's sum, will also be fast in
    # Python
    return a.sum()

@njit
def for_loop(a):
    # uses a simple for-loop over the array
    acc = 0
    for i in a:
        acc += i
    return acc
```

We can now benchmark the preceding code:

```
In [1]: ... # import the above functions

In [2]: import numpy as np

In [3]: a = np.arange(1000000, dtype=np.int64)

In [4]: numpy_func(a) # sanity check and compile
Out[4]: 499999500000

In [5]: for_loop(a) # sanity check and compile
Out[5]: 499999500000

In [6]: %timeit numpy_func(a) # Compiled version of the NumPy func
174 µs ± 3.05 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)

In [7]: %timeit for_loop(a) # Compiled version of the for-loop
186 µs ± 7.59 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)

In [8]: %timeit numpy_func.py_func(a) # Pure NumPy func
336 µs ± 6.72 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```



```
In [9]: %timeit for_loop.py_func(a)    # Pure Python for-loop
156 ms ± 3.07 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

As you can see, both Numba compiled variants have very similar performance characteristics, whereas the pure Python for loop implementation is significantly (800 times) slower than its compiled counterpart.

If you are now thinking about rewriting your NumPy array expressions as for loops, don't! As shown in the preceding example, Numba is perfectly happy with NumPy arrays and their associated functions. In fact, Numba has an additional ace up its sleeve: an optimization known as *loop fusion*. Numba performs this technique predominantly on array expression operations. For example:

```
from numba import njit

@njit
def loop_fused(a, b):
    return a * b - 4.1 * a > 2.5 * b

In [1]: ... # import the example

In [2]: import numpy as np

In [3]: a, b = np.arange(1e6), np.arange(1e6)

In [4]: loop_fused(a, b) # compile the function
Out[4]: array([False, False, False, ..., True, True, True])

In [5]: %timeit loop_fused(a, b)
643 µs ± 18 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)

In [6]: %timeit loop_fused.py_func(a, b)
5.2 ms ± 205 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

As you can see, the Numba compiled version is eight times faster than the pure NumPy one. What is going on? Without Numba, the array expression will lead to several for loops and several so-called temporaries in memory. Loosely speaking, for each arithmetic operation in the expression, a for loop over arrays must execute, and the result of each must be stored in a temporary array in memory. What loop fusion does is fuse the various loops over arithmetic operations together into a single loop, thereby reducing both the total number of memory lookups and the overall memory required to compute the result. In fact, the loop-fused variant may well look something like this:

```
import numpy as np
from numba import njit

@njit
def manual_loop_fused(a, b):
```

```

N = len(a)
result = np.empty(N, dtype=np.bool_)
for i in range(N):
    a_i, b_i = a[i], b[i]
    result[i] = a_i * b_i - 4.1 * a_i > 2.5 * b_i
return result

```

Running this will show performance characteristics similar to those of the loop-fusion example:

```

In [1]: %timeit manual_loop_fused(a, b)
636 µs ± 49.1 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)

```

Finally, I recommend targeting serial execution initially, but keep parallel execution in mind. Don't assume from the outset that only a parallel version will lead to your targeted performance characteristics. Instead, focus on developing a clean serial implementation first. Parallelism makes everything harder to reason about and can be a source of difficulty when debugging problems. If you are satisfied with your results and would still like to investigate parallelizing your code, Numba does come with a `parallel=True` option for the `@jit` decorator and a corresponding `parallel range`, the `prange` construct, to make creating parallel loops easier.

Getting Help

As of early 2020, the two main recommended communication channels for Numba are the [GitHub issue tracker](#) and the [Gitter chat](#); they are where the action happens. There are also a mailing list and a Twitter account, but these are fairly low-traffic and mostly used to announce new releases and other important project news.

Optimizing Versus Thinking (2020)

*Vincent D. Warmerdam, Senior Person at GoDataDriven
koaning.io*

Vincent D. Warmerdam is a senior data professional who works as an engineer, researcher, team lead, educator, public speaker, and cofounder of PyData Amsterdam.

This is a story of a team that was solving the wrong problem. We were optimizing efficiency while ignoring effectiveness. My hope is that this story is a cautionary tale for others. This story actually happened, but I've changed parts and kept the details vague in the interest of keeping things incognito.

I was consulting for a client with a common logistics problem: they wanted to predict the number of trucks that would arrive at their warehouses. There was a good

business case for this. If we knew the number of vehicles, we would know how large the workforce had to be to handle the workload for that day.

The planning department had been trying to tackle this problem for years (using Excel). They were skeptical that an algorithm could improve things. Our job was to explore if machine learning could help out here.

From the start, it was apparent it was a difficult time-series problem:

- There were many (seriously, many!) holidays that we needed to keep in mind since the warehouses operated internationally. The effect of the holiday might depend on the day of the week, since the warehouses did not open during week-ends. Certain holidays meant demand would go up, while other holidays meant that the warehouses were closed (which would sometimes cause a three-day weekend).
- Seasonal shifts were not unheard of.
- Suppliers would frequently enter and leave the market.
- The seasonal patterns were always changing because the market was continuously evolving.
- There were many warehouses, and although they were in separate buildings, there was a reason to believe the number of trucks arriving at the different warehouses were correlated.

The diagram in [Figure 13-3](#) shows the process for the algorithm that would calculate the seasonal effect as well as the long-term trend. As long as there weren't any holidays, our method would work. The planning department warned us about this; the holidays were the hard part. After spending a lot of time collecting relevant features, we ended up building a system that mainly focused on trying to deal with the holidays.

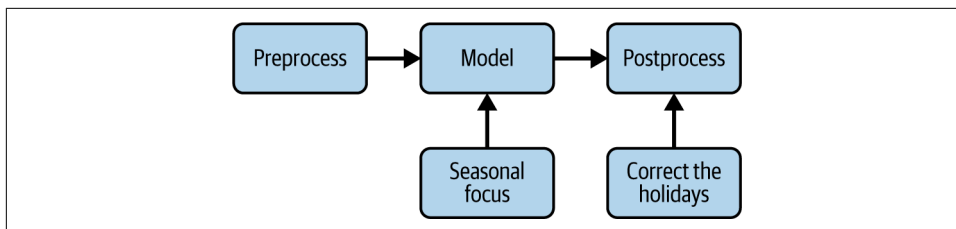


Figure 13-3. Seasonal effects and long-term trend

So we iterated, did more feature engineering, and designed the algorithm. It got to the point where we needed to calculate a time-series model per warehouse, which would be postprocessed with a heuristic model per holiday per day of the week. A holiday just before the weekend would cause a different shift than a holiday just after the

weekend. As you might imagine, this calculation can get quite expensive when you also want to apply a grid search, as shown in [Figure 13-4](#).

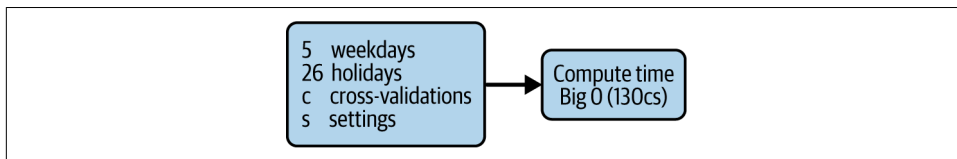


Figure 13-4. Many variations cost a lot of compute time

There were many effects that we had to estimate accurately, including the decay of the past measurements, how smooth the seasonal effect is, the regularization parameters, and how to tackle the correlation between different warehouses.

What didn't help was that we needed to predict months ahead. Another hard thing was the cost function: it was discrete. The planning department did not care about (or even appreciate) mean squared error; they cared only about the number of days where the prediction error would exceed 100 trucks.

You can imagine that, in addition to statistical concerns, the model presented performance concerns. To keep this at bay, we restricted ourselves to simpler machine learning models. We gained a lot of iteration speed by doing this, which allowed us to focus on feature engineering. A few weeks went by before we had a version we could demo. We had still made a model that performed well enough, except for the holidays.

The model went into a proof-of-concept phase; it performed reasonably well but not significantly better than the current planning team's method. The model was useful since it allowed the planning department to reflect if the model disagreed, but no one was comfortable having the model automate the planning.

Then it happened. It was my final week with the client, just before a colleague would be taking over. I was hanging out at the coffee corner, talking with an analyst about a different project for which I needed some data from him. We started reviewing the available tables in the database. Eventually, he told me about a "carts" table (shown in [Figure 13-5](#)).

Me: "A carts table? What's in there?" Analyst: "Oh, it contains all the orders for the carts." Me: "Suppliers buy them from the warehouse?" Analyst: "No, actually, they rent them. They usually rent them three to five days in advance before they return them filled with goods for the warehouse." Me: "All of your suppliers work this way?" Analyst: "Pretty much."

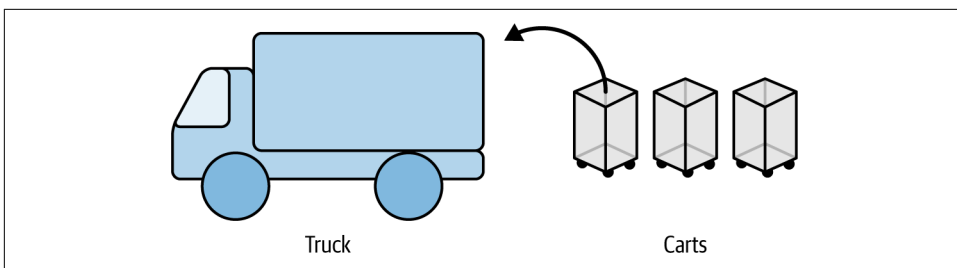


Figure 13-5. Carts contain the leading information that's critical to this challenge!

I spotted the most significant performance issue of them all: we were solving the wrong problem. This wasn't a machine learning problem; it was a SQL problem. The number of rented carts was a robust proxy for how many trucks the company would send. They wouldn't need a machine learning model. We could just project the number of carts being rented a few days ahead, divide by the number of carts that fit into a truck, and get a sensible approximation of what to expect. If we had realized this earlier, we would not have needed to optimize the code for a gigantic grid search because there would have been no need.

It is rather straightforward to translate a business case into an analytical problem that doesn't reflect reality. Anything that you can do to prevent this will yield the most significant performance benefit you can imagine.

Making Deep Learning Fly with RadimRehurek.com (2014)

Radim Řehůřek
radimrehurek.com

Radim Řehůřek is the creator of Gensim and is a company founder with a specialism in AI and natural language processing research who cares about highlighting personally identifiable information to preserve privacy.

When Ian asked me to write my “lessons from the field” on Python and optimizations for this book, I immediately thought, “Tell them how you made a Python port faster than Google’s C original!” It’s an inspiring story of making a machine learning algorithm, Google’s poster child for deep learning, 12,000 times faster than a naive Python implementation. Anyone can write bad code and then trumpet about large speedups. But the optimized Python port also runs, somewhat astonishingly, almost four times faster than the original code written by Google’s team! That is, it runs four times faster than opaque, tightly profiled, and optimized C.

But before drawing “machine-level” optimization lessons, some general advice about “human-level” optimizations.

The Sweet Spot

I run a small consulting business laser-focused on machine learning, where my colleagues and I help companies make sense of the tumultuous world of data analysis, in order to make money or save costs (or both). We help clients design and build wondrous systems for data processing, especially text data.

The clients range from large multinationals to nascent start-ups, and while each project is different and requires a different tech stack, plugging into the client’s existing data flows and pipelines, Python is a clear favorite. Not to preach to the choir, but Python’s no-nonsense development philosophy, its malleability, and the rich library ecosystem make it an ideal choice.

First, a few thoughts “from the field” on what works:

Communication, communication, communication

This one’s obvious, but worth repeating. Understand the client’s problem on a higher (business) level before deciding on an approach. Sit down and talk through what they think they need (based on their partial knowledge of what’s possible and/or what they Googled up before contacting you), until it becomes clear what they really need, free of cruft and preconceptions. Agree on ways to validate the solution beforehand. I like to visualize this process as a long, winding road to be built: get the starting line right (problem definition, available data sources) and the finish line right (evaluation, solution priorities), and the path in between falls into place.

Be on the lookout for promising technologies

An emergent technology that is reasonably well understood and robust, is gaining traction, yet is still relatively obscure in the industry can bring huge value to the client (or yourself). As an example, a few years ago, Elasticsearch was a little-known and somewhat raw open source project. But I evaluated its approach as solid (built on top of Apache Lucene, offering replication, cluster sharding, etc.) and recommended its use to a client. We consequently built a search system with Elasticsearch at its core, saving the client significant amounts of money in licensing, development, and maintenance compared to the considered alternatives (large commercial databases). Even more importantly, using a new, flexible, powerful technology gave the product a massive competitive advantage. Nowadays, Elasticsearch has entered the enterprise market and conveys no competitive advantage at all—everyone knows it and uses it. Getting the timing right is what I call hitting the “sweet spot,” maximizing the value/cost ratio.

KISS (Keep It Simple, Stupid!)

This is another no-brainer. The best code is code you don't have to write and maintain. Start simple, and improve and iterate where necessary. I prefer tools that follow the Unix philosophy of “do one thing, and do it well.” Grand programming frameworks can be tempting, with everything imaginable under one roof and fitting neatly together. But invariably, sooner or later, you need something the grand framework didn't imagine, and then even modifications that seem simple (conceptually) cascade into a nightmare (programmatically). Grand projects and their all-encompassing APIs tend to collapse under their own weight. Use modular, focused tools, with APIs in between that are as small and uncomplicated as possible. Prefer text formats that are open to simple visual inspection, unless performance dictates otherwise.

Use manual sanity checks in data pipelines

When optimizing data processing systems, it's easy to stay in the “binary mindset” mode, using tight pipelines, efficient binary data formats, and compressed I/O. As the data passes through the system unseen and unchecked (except for perhaps its type), it remains invisible until something outright blows up. Then debugging commences. I advocate sprinkling a few simple log messages throughout the code, showing what the data looks like at various internal points of processing, as good practice—nothing fancy, just an analogy to the Unix head command, picking and visualizing a few data points. Not only does this help during the aforementioned debugging, but seeing the data in a human-readable format leads to “aha!” moments surprisingly often, even when all seems to be going well. Strange tokenization! They promised input would always be encoded in latin1! How did a document in this language get in there? Image files leaked into a pipeline that expects and parses text files! These are often insights that go way beyond those offered by automatic type checking or a fixed unit test, hinting at issues beyond component boundaries. Real-world data is messy. Catch early even things that wouldn't necessarily lead to exceptions or glaring errors. Err on the side of too much verbosity.

Navigate fads carefully

Just because a client keeps hearing about X and says they must have X too doesn't mean they really need it. It might be a marketing problem rather than a technology one, so take care to discern the two and deliver accordingly. X changes over time as hype waves come and go; a recent value would be X = big data.

All right, enough business talk—here's how I got *word2vec* in Python to run faster than C.

Lessons in Optimizing

word2vec is a deep learning algorithm that allows detection of similar words and phrases. With interesting applications in text analytics and search engine optimization (SEO), and with Google’s lustrous brand name attached to it, start-ups and businesses flocked to take advantage of this new tool.

Unfortunately, the only available code was that produced by Google itself, an open source Linux command-line tool written in C. This was a well-optimized but rather hard-to-use implementation. The primary reason I decided to port *word2vec* to Python was so I could extend *word2vec* to other platforms, making it easier to integrate and extend for clients.

The details are not relevant here, but *word2vec* requires a training phase with a lot of input data to produce a useful similarity model. For example, the folks at Google ran *word2vec* on their GoogleNews dataset, training on approximately 100 billion words. Datasets of this scale obviously don’t fit in RAM, so a memory-efficient approach must be taken.

I’ve authored a machine learning library, *gensim*, that targets exactly that sort of memory-optimization problem: datasets that are no longer trivial (“trivial” being anything that fits fully into RAM), yet not large enough to warrant petabyte-scale clusters of MapReduce computers. This “terabyte” problem range fits a surprisingly large portion of real-world cases, *word2vec* included.

Details are described on [my website](#), but here are a few optimization takeaways:

Stream your data, watch your memory

Let your input be accessed and processed one data point at a time, for a small, constant memory footprint. The streamed data points (sentences, in the case of *word2vec*) may be grouped into larger batches internally for performance (such as processing 100 sentences at a time), but a high-level, streamed API proved a powerful and flexible abstraction. The Python language supports this pattern very naturally and elegantly, with its built-in generators—a truly beautiful problem–tech match. Avoid committing to algorithms and tools that load everything into RAM, unless you know your data will always remain small, or you don’t mind reimplementing a production version yourself later.

Take advantage of Python's rich ecosystem

I started with a readable, clean port of *word2vec* in *numpy*. *numpy* is covered in depth in [Chapter 6](#) of this book, but as a short reminder, it is an amazing library, a cornerstone of Python's scientific community, and the de facto standard for number crunching in Python. Tapping into *numpy*'s powerful array interfaces, memory access patterns, and wrapped BLAS routines for ultrafast common vector operations leads to concise, clean, and fast code—code that is hundreds of times faster than naive Python code. Normally I'd call it a day at this point, but “hundreds of times faster” was still 20 times slower than Google's optimized C version, so I pressed on.

Profile and compile hotspots

word2vec is a typical high performance computing app, in that a few lines of code in one inner loop account for 90% of the entire training runtime. Here I rewrote a single core routine (approximately 20 lines of code) in C, using an external Python library, Cython, as the glue. While it's technically brilliant, I don't consider Cython a particularly convenient tool conceptually—it's basically like learning another language, a nonintuitive mix between Python, *numpy*, and C, with its own caveats and idiosyncrasies. But until Python's JIT technologies mature, Cython is probably our best bet. With a Cython-compiled hotspot, performance of the Python *word2vec* port is now on par with the original C code. An additional advantage of having started with a clean *numpy* version is that we get free tests for correctness by comparing against the slower but correct version.

Know your BLAS

A neat feature of *numpy* is that it internally wraps Basic Linear Algebra Subprograms (BLAS), where available. These are sets of low-level routines, optimized directly by processor vendors (Intel, AMD, etc.) in assembly, Fortran, or C, and designed to squeeze out maximum performance from a particular processor architecture. For example, calling an axpy BLAS routine computes `vector_y += scalar * vector_x` way faster than what a generic compiler would produce for an equivalent explicit for loop. Expressing *word2vec* training as BLAS operations resulted in another four times speedup, topping the performance of C *word2vec*. Victory! To be fair, the C code could link to BLAS as well, so this is not some inherent advantage of Python per se. *numpy* just makes things like this stand out and makes them easy to take advantage of.

Parallelization and multiple cores

`gensim` contains distributed cluster implementations of a few algorithms. For `word2vec`, I opted for multithreading on a single machine, because of the fine-grained nature of its training algorithm. Using threads also allows us to avoid the fork-without-exec POSIX issues that Python’s multiprocessing brings, especially in combination with certain BLAS libraries. Because our core routine is already in Cython, we can afford to release Python’s GIL (global interpreter lock; see [“Parallelizing the Solution with OpenMP on One Machine” on page 232](#)), which normally renders multithreading useless for CPU-intensive tasks. Speedup: another three times, on a machine with four cores.

Static memory allocations

At this point, we’re processing tens of thousands of sentences per second. Training is so fast that even little things like creating a new numpy array (calling `malloc` for each streamed sentence) slow us down. Solution: preallocate a static “work” memory and pass it around, in good old Fortran fashion. Brings tears to my eyes. The lesson here is to keep as much bookkeeping and app logic in the clean Python code as possible, and to keep the optimized hotspot lean and mean.

Problem-specific optimizations

The original C implementation contained specific micro-optimizations, such as aligning arrays onto specific memory boundaries or precomputing certain functions into memory lookup tables. A nostalgic blast from the past, but with today’s complex CPU instruction pipelines, memory cache hierarchies, and coprocessors, such optimizations are no longer a clear winner. Careful profiling suggested a few percent improvement, which may not be worth the extra code complexity. Takeaway: use annotation and profiling tools to highlight poorly optimized spots. Use your domain knowledge to introduce algorithmic approximations that trade accuracy for performance (or vice versa). But never take it on faith; profile, preferably using real production data.

Conclusion

Optimize where appropriate. In my experience, there’s never enough communication to fully ascertain the problem scope, priorities, and connection to the client’s business goals—aka the “human-level” optimizations. Make sure you deliver on a problem that matters, rather than getting lost in “geek stuff” for the sake of it. And when you do roll up your sleeves, make it worth it!

Large-Scale Social Media Analysis at Smesh (2014)

Alex Kelly
sme.sh

Alex Kelly is a highly experienced developer and team leader.

At Smesh, we produce software that ingests data from a wide variety of APIs across the web; filters, processes, and aggregates it; and then uses that data to build bespoke apps for a variety of clients. For example, we provide the tech that powers the tweet filtering and streaming in Beamly’s second-screen TV app, run a brand and campaign monitoring platform for mobile network EE, and run a bunch of Adwords data analysis projects for Google.

To do that, we run a variety of streaming and polling services, frequently polling Twitter, Facebook, YouTube, and a host of other services for content and processing several million tweets daily.

Python’s Role at Smesh

We use Python extensively—the majority of our platform and services are built with it. The wide variety of libraries, tools, and frameworks available allows us to use it across the board for most of what we do.

That variety gives us the ability to (hopefully) pick the right tool for the job. For example, we’ve created apps using Django, Flask, and Pyramid. Each has its own benefits, and we can pick the one that’s right for the task at hand. We use Celery for tasks; Boto for interacting with AWS; and PyMongo, MongoEngine, redis-py, Pyscopg, etc. for all our data needs. The list goes on and on.

The Platform

Our main platform consists of a central Python module that provides hooks for data input, filtering, aggregations and processing, and a variety of other core functions. Project-specific code imports functionality from that core and then implements more specific data processing and view logic, as each application requires.

This has worked well for us up to now, and allows us to build fairly complex applications that ingest and process data from a wide variety of sources without much duplication of effort. However, it isn’t without its drawbacks—each app is dependent on a common core module, making the process of updating the code in that module and keeping all the apps that use it up-to-date a major task.

We're currently working on a project to redesign that core software and move toward more of a service-oriented architecture (SoA) approach. It seems that finding the right time to make that sort of architectural change is one of the challenges that faces most software teams as a platform grows. There is overhead in building components as individual services, and often the deep domain-specific knowledge required to build each service is acquired only through an initial iteration of development, where that architectural overhead is a hindrance to solving the real problem at hand. Hopefully, we've chosen a sensible time to revisit our architectural choices to move things forward. Time will tell.

High Performance Real-Time String Matching

We consume lots of data from the Twitter Streaming API. As we stream in tweets, we match the input strings against a set of keywords so that we know which of the terms we're tracking that each tweet is related to. That's not such a problem with a low rate of input, or a small set of keywords, but doing that matching for hundreds of tweets per second, against hundreds or thousands of possible keywords, starts to get tricky.

To make things even trickier, we're not interested in simply whether the keyword string exists in the tweet, but in more complex pattern matching against word boundaries, start and end of line, and optionally the use of # and @ characters to prefix the string. The most effective way to encapsulate that matching knowledge is using regular expressions. However, running thousands of regex patterns across hundreds of tweets per second is computationally intensive. Previously, we had to run many worker nodes across a cluster of machines to perform the matching reliably in real time.

Knowing this was a major performance bottleneck in the system, we tried a variety of things to improve the performance of our matching system: simplifying the regexes, running enough processes to ensure we were utilizing all the cores on our servers, ensuring all our regex patterns are compiled and cached properly, running the matching tasks under PyPy instead of CPython, etc. Each of these resulted in a small increase in performance, but it was clear this approach was going to shave only a fraction of our processing time. We were looking for an order of magnitude speedup, not a fractional improvement.

It was obvious that rather than trying to increase the performance of each match, we needed to reduce the problem space before the pattern matching takes place. So we needed to reduce either the number of tweets to process or the number of regex patterns we needed to match the tweets against. Dropping the incoming tweets wasn't an option—that's the data we're interested in. So we set about finding a way to reduce the number of patterns we need to compare an incoming tweet to in order to perform the matching.

We started looking at various trie structures for allowing us to do pattern matching between sets of strings more efficiently, and came across the Aho-Corasick string-matching algorithm. It turned out to be ideal for our use case. The dictionary from which the trie is built needs to be static—you can't add new members to the trie after the automaton has been finalized—but for us this isn't a problem, as the set of keywords is static for the duration of a session streaming from Twitter. When we change the terms we're tracking we must disconnect from and reconnect to the API, so we can rebuild the Aho-Corasick trie at the same time.

Processing an input against the strings using Aho-Corasick finds all possible matches simultaneously, stepping through the input string a character at a time and finding matching nodes at the next level down in the trie (or not, as the case may be). So we can very quickly find which of our keyword terms may exist in the tweet. We still don't know for sure, as the pure string-in-string matching of Aho-Corasick doesn't allow us to apply any of the more complex logic that is encapsulated in the regex patterns, but we can use the Aho-Corasick matching as a prefilter. Keywords that don't exist in the string can't match, so we know we have to try only a small subset of all our regex patterns, based on the keywords that do appear in the text. Rather than evaluating hundreds or thousands of regex patterns against every input, we rule out the majority and need to process only a handful for each tweet.

By reducing the number of patterns that we attempt to match against each incoming tweet to just a small handful, we've managed to achieve the speedup we were looking for. Depending on the complexity of the trie and the average length of the input tweets, our keyword matching system now performs somewhere between 10 to 100 times faster than the original naive implementation.

If you're doing a lot of regex processing or other pattern matching, I highly recommend having a dig around the different variations of prefix and suffix tries that might help you to find a blazingly fast solution to your problem.

Reporting, Monitoring, Debugging, and Deployment

We maintain a bunch of different systems running our Python software and the rest of the infrastructure that powers it all. Keeping it all up and running without interruption can be tricky. Here are a few lessons we've learned along the way.

It's really powerful to be able to see both in real time and historically what's going on inside your systems, whether that be in your own software or in the infrastructure it runs on. We use Graphite with `collectd` and `statsd` to allow us to draw pretty graphs of what's going on. That gives us a way to spot trends and to retrospectively analyze problems to find the root cause. We haven't gotten around to implementing it yet, but Etsy's Skyline also looks brilliant as a way to spot the unexpected when you have more metrics than you can keep track of. Another useful tool is Sentry, a great

system for event logging and keeping track of exceptions being raised across a cluster of machines.

Deployment can be painful, no matter what you're using to do it. We've been users of Puppet, Ansible, and Salt. They all have pros and cons, but none of them will make a complex deployment problem magically go away.

To maintain high availability for some of our systems, we run multiple geographically distributed clusters of infrastructure, running one system live and others as hot spares, with switchover being done by updates to DNS with low Time-to-Live (TTL) values. Obviously that's not always straightforward, especially when you have tight constraints on data consistency. Thankfully, we're not affected by that too badly, making the approach relatively straightforward. It also provides us with a fairly safe deployment strategy: updating one of our spare clusters and performing testing before promoting that cluster to live and updating the others.

Along with everyone else, we're really excited by the prospect of what can be done with **Docker**. Also along with pretty much everyone else, we're still just at the stage of playing around with it to figure out how to make it part of our deployment processes. However, having the ability to rapidly deploy our software in a lightweight and reproducible fashion, with all its binary dependencies and system libraries included, seems to be just around the corner.

At a server level, there's a whole bunch of routine stuff that just makes life easier. Monit is great for keeping an eye on things for you. Upstart and `supervisord` make running services less painful. Munin is useful for some quick and easy system-level graphing if you're not using a full Graphite/`collectd` setup. And Corosync/Pacemaker can be a good solution for running services across a cluster of nodes (for example, when you have a bunch of services that you need to run somewhere, but not everywhere).

I've tried not to just list buzzwords here but to point you toward software we're using every day and that really making a difference in how effectively we can deploy and run our systems. If you've heard of them all already, I'm sure you must have a whole bunch of other useful tips to share, so please drop me a line with some pointers. If not, go check them out—hopefully, some of them will be as useful to you as they are to us.

Symbols

%memit, 61, 116, 381, 390
%timeit function, 39, 72, 203
1D diffusion, 129-130
2D diffusion, 130-136, 160-162
 (see also foreign function interfaces)
32-bit floats, 241
64-bit Python, xiii
>> (jump points), 69
@fasteners.interprocess_locked decorator, 352
@jit decorator, 234, 468, 470
@njit decorator alias, 470
@profile decorator, 54, 61
@require decorator, 367
Σ (sigma) character, 391

A

abs function, 217, 227-228, 231
abstraction, 13
Accelerate library, 340
acceptance tests, 445
acquire, 354
Aho-Corasick string-matching algorithm, 485
AI Acceleration, 162
aiohttp library, 270
Airflow, 324, 373
algorithmic optimizations, 214
algorithms
 online and single pass, 116, 120
 search, 81-82, 83, 98
allocation of memory, 54, 133-136, 140, 149-152
Amazon EC2, 362, 368
Amazon SageMaker, 447
Amazon Simple Queue Service (SQS), 373
Amazon Web Services (AWS), 357
Amdahl's law, 4, 290
amortized analysis, 104
AMP (Automatic Mixed-Precision), 175-176
Anaconda, 16
anomaly detection, 120-124
Ansible, 486
AOT compilers, 216, 242
API bindings, 16
apply call, 197, 203, 208, 209
apply_sync, 366
architecture
 communications layers, 8-9
 computing units, 2-5
 memory units, 2, 6-8, 25
 multicore, 4-5, 290
Arduinos, 364
array (Dask), 205
Array component, in multiprocessing module, 346
array module, 15, 142, 229, 382-384
array objects, 229
arrays, 383 (see lists; Pandas; tuples)
 Arrow versus NumPy, 189
 creation and retrieval, 80-82
 Cython and, 229-230
 Dask, 205-210
 inserting data, 99
 iterating over multidimensional, 251
 iteration and, 143
 lists versus, 384
 numpy (see numpy and NumPy)
 Polars, 210-211

- RAM and, 408
- Arrow, 189, 199
- sizeof, 390
- assert statements, 23, 444
- async keyword, 265
- async/await, 267-275
- AsyncBatcher object, 278-280
- asynchronous CPU workload, 281-284
- asynchronous programming, 261-287
 - async/await, 265, 267-275
 - event loop, 262, 263-266, 267, 272
 - I/O wait, 262-263, 264
 - shared CPU-I/O workload, 275-284
- asynchronous systems, 323
- asynchronous web crawler, 270-275
- asyncio module, 15, 265, 267
- asyncio.eager_task_factory, 272
- asyncio.run(coro), 267
- asyncio.sleep(0) statement, 282-283
- asyncio.TaskGroup, 282
- AT&T Bell Labs, 435-441
- ATLAS, 340
- autocast module, 175
- autograd, 159
- Automatic Mixed-Precision (AMP), 175-176
- automation, 20
- await keyword, 265, 267, 272
- await statement, 282
- AWS (Amazon Web Services), 357
- Azure, 362, 447-448

B

- backtesting, 452, 454
- bag (Dask), 205
- Basic Linear Algebra Subprograms (BLAS), 340, 481
- batched CPU workload, 278-281
- Batchelder, Ned, 391
- batching results, 280
- Bell Labs, 435-441
- benchmarking, 77, 131, 182, 211
- Bendersky, Eli, 355
- Beowulf clusters, 357
- Berkeley Open Infrastructure for Network Computing (BOINC) system, 362
- bfloat16, 164
- Big O notation, 79
- bigrams, 401
- binary mindset mode, 482

- binary search, 83
- bisect module, 83-85, 96, 394
- bitarray module, 408, 418
- BLAS (Basic Linear Algebra Subprograms), 340, 481
- BlockManager, 187
- Bloom filters, 417-423, 428
- BOINC (Berkeley Open Infrastructure for Network Computing) system, 362
- bool type, 188
- boolean type, 188
- bottleneck dependency, 203
- bottleneck utility, 169-170
- bottlenecks, 137
 - (see also profiling)
- boundary conditions, 128, 129
- bounds checking, 228-229
- branch prediction, 137
- branches, 180
- buckets, 80-81
- build context, 376
- burnout, 24
- buses, 8-9
- byte strings, 389
- Bytecode, 67-73
- bytes, versus Unicode, 390-391

C

- C (see compiling to C; foreign function interfaces)
- cache misses, 140
- cache-references metric, 140
- caches, 157, 182
- caching, 91, 307-308
- calculate_z, 235
- calculate_z_serial_purepython, 65
- calculate_z_serial_purepython function, 35, 42-46, 54-61
- calc_pure_python function, 34, 36, 43-46, 65
- call method, 305
- callback hell, 265
- callbacks and callback paradigm, 264-265
- calling functions, 471
- capacity, versus speed, 7
- cardinality, 416
- cargo tool, 256, 257
- casting, 245
- categorical type, 188
- Category encoders, 461

- Cauchy problem, 128
- cdef keyword, 225
- Celery, 372
- central processing units (see CPUs)
- cfi, 246-249
- cgroups, 373
- chain function, 120
- Chaos Monkey, 363
- checklists, 361
- check_every counter, 333, 338
- check_prime function, 329-330
- check_prime_in_range function, 332
- chunksize parameter, 314-318
- “circular buffer is full” error, 67
- Circus, 363
- classes, user-defined, 106
- clock speed, 4
- cloud-based clusters, 360
- cloudpickle, 305
- clusters and clustering, 357-378
 - avoiding problems with, 363-364
 - benefits of, 358-359
 - common designs, 362
 - common tools, 372-373
 - CPU-bound and RAM-bound, 206
 - Docker, 373-378
 - drawbacks of, 359-362
 - IPython Parallel, 365-368
 - message brokering, 368-372
 - starting a clustered system, 362
- cmp function, 95, 106
- code
 - avoiding terse, 203
 - complexity, 455
 - maintainability, 462
 - refactoring, 459
- coding standards, 20
- cold start problem, 216
- collaboration, on open source libraries, 463
- collections module, 15, 124
- collections, RAM in, 389-390
- column-major ordering, 251
- common prefix search, 397
- communication, 478
- communications layers, 8-9
- compiling to C, 213-260
 - compilers, 217-218
 - CPython extensions, 252-260
 - Cython, 219-234, 241
 - foreign function interfaces, 242-256
 - JIT versus AOT compilers, 216
 - Numba, 234-236, 241
 - PyPy, 236-240, 241
 - speed gains, 214-216, 240
 - type information and, 216-217
 - when to use each technology, 241-242
- compiling to machine code, 213
- complex object, 230
- component reuse, 454
- compressed instructions, 164
- compression, lossy, 409
- computing units, 2-5
- concatenation, 200-201
- concurrency, 262, 263, 264, 286
 - (see also asynchronous programming)
- concurrent code, 267
- concurrent programs/functions, 262-263
- concurrent requests, 274-275
- concurrent.futures module, 293
- config files, 459
- container registries, 377
- containers, 375-376, 377, 379
- context managers, 58
- context switches, 263
- context switches (cs) metric, 139, 140
- contributing guidelines, to open source
 - projects, 463
- controllers, 365
- convolutions, 166-168
- COO matrices, 406
- COPY commands, 375
- “copy and patch” process, 26
- Corosync/Pacemaker, 486
- coroutines, 267, 272
- cosine similarity, 407
- coverage tool, 19, 73
- cProfile module, 28, 41-48
- CPU-bound tasks, 362
- CPU-bound workers, 206
- CPUs (central processing units), 2-5
 - cache misses and, 140
 - caches, 182
 - GPUs compared to, 158
 - lower-precision operations on, 163-164
 - memory fragmentation and, 137-140
 - numexpr and, 156-158
 - page faults and, 140
 - speed/memory tradeoff, 117

- variation in load, 39
- vectorization, 12
- CPU-I/O workload, shared, 275-284
- CPython
 - bytecode, 67-71
 - garbage collector in, 238
 - modules, 252-256
 - multicore systems and, 289
 - (see also multiprocessing)
 - PyPy versus, 340
 - Rust extension, 256-260
- crates tool, 256
- cron jobs, 363
- cross entropy formula, 385
- cs (context switches) metric, 139, 140
- ctypes, 243-246, 293, 336
- ctypes.CDLL call, 244
- Cuckoo filters, 423
- CUDA_LAUNCH_BLOCKING flag, 162
- curiosity, 24
- cyber reinsurance, 441-451
- cycle function, 120
- Cython, 14, 17, 219-234, 481
 - annotations with, 221-229
 - large arrays and matrixes in, 382
 - numpy and, 229-234, 382
 - Pandas and, 200
 - team velocity and, 213
 - version support, 215
 - when to use, 241
- cythonfn.html, 222-223

D

- DAFSAs (deterministic acyclic finite state automaton), 397
- Dash, 450
- Dask, 183, 205-210
- dask-expr library, 208
- Dask-ML, 206
- data (see Pandas; Dask; Polars)
 - heavy, 12
 - need for good, 457
 - RAM and, 380
 - real-world, 479
- data consumers, 370-371
- data distributed parallelism, 177
- data fragmentation, 136-145
- data publishers, 370
- data quality, 466
- data science teams, 464-467
- data sharing, 355
 - file locking, 348-352
 - Redis for, 335-337
- data subscribers, 370
- database bindings, 16
- Databricks, 447
- dataframe validation, 445
- DataFrames, 306 (see Pandas; Dask; Polars)
- DataLoader class, 172
- datatypes
 - in array module, 383
 - numpy and, 80, 188, 229, 383-384
 - in Pandas, 188
- datetimeetz datatype, 188
- DAWGs (directed acyclic word graphs), 397-398
- dd.from_pandas, 208
- ddf.apply, 208
- ddf.visualize(filename="graph.png"), 207
- dead man's switches, 364
- decorators, 37
 - delayed decorator, 305
 - @fasteners.interprocess_locked decorator, 352
 - functools.total_ordering decorator, 82
 - @jit decorator, 234, 468, 470
 - LineProfiler and, 193
 - Memory cache, 307
 - @njit decorator alias, 470
 - no-op decorator, 74-76
 - @profile decorator, 54, 61
 - @require decorator, 367
 - wrapped attribute, 194
- deep learning, 162, 174-178, 477-482
- defensive programming, 443
- del keyword, 204
- delay parameter, 268
- delayed (Dask), 206
- delayed decorator, 305
- deliverables, intermediate, 467
- deliveries, managing expectations around, 466
- dense matrices, versus sparse, 406-407
- deployment systems, 364
- deque object, 124
- deterministic acyclic finite state automaton (DAFSAs), 397
- developer communities, 462
- DevOps, 447, 448

- df.eval, 388
- diagnostics, with Dask, 206-207
- dict, storing text in RAM with, 396
- dictionaries and sets, 95-111
 - compared, 95
 - costs to using, 96-99
 - hash tables and, 99-105
- DictVectorizer, 401-408
- diffusion equation, 126-133
- _diffusion object, 244
- direct interface, 365
- directed acyclic word graphs (DAWGs), 397-398
- dis module, 67-69, 72-73
- distributed dataframe (Dask), 205
- Docker, 16, 19, 373-378, 486
- docker pull command, 377
- docker push command, 378
- docker stats command, 377
- Dockerfile, 375
- docstrings, 19
- documentation, 18-19, 462
- documented restart plans, 360
- double complex type, 225
- double hashing, 418
- dowser, 78
- dozer, 78
- do_calculation function, 248
- do_world function, 264
- .drop(), 204
- drop method, 204
- dtypes, 187
- dynamic arrays, 85, 86-90
- dynamic computational graphs, 159, 166
- dynamic quantization), 176
- dynamic scaling, 358
- dynamic types, 14, 216

E

- EC2 (Elastic Compute Cloud), 362, 368
- ElastiCluster, 357, 368
- Elasticsearch, 478
- embarrassingly parallel processes, 291
- emergent technologies, 478
- engine argument, 200
- engines, 365
- entropy, 107-110
- enumerate function, 118
- environments, 16, 20

- eq function, 82, 95
- error rates, 410
- Etsy's Skyline, 485
- Euler's method, 127
- evaluation metrics, 433
- event loop, 262, 263-266, 267, 272
- evolution function, 131, 132, 133
- evolve function, 245, 251
- exception handling, 444
- execute command, 366
- expectations, managing, 466
- external bus, 8
- external libraries, 182
 - (see also foreign function interfaces)

F

- f2py, 249-252
- fads, 479
- failures, and clustering, 359
- fast inverse square root algorithm, 162
- @fasteners.interprocess_locked decorator, 352
- fasteners module, 351
- fault metric, 140
- fearless concurrency, 256
- feature engineering, 458-464
- Feature-engine package, 460-461
- FeatureHasher, 400-408
- feed_new_jobs function, 323
- fibonacci functions, 114-117, 119
- file locking, 348-352
- filter function, 118
- filtering data, 203
- filters, in convolutions, 166
- fit/transform methods, 461
- flame charts, 64
- flexibility, 455-458
- float type, 106, 188
- float16, 163-165, 188
- float32, 164, 175, 176
- float64, 188, 204
- float8_e4m3fn, 164
- float8_e5m2, 164
- floating-point representations, 164
- Floater are Friends (Wolever), 164
- flush function, 279
- Folding@home, 362
- for loops, 471-473
 - (see also iterators and generators)
- foreign function interfaces, 242-256

- cfi, 246-249
 - CPython modules, 252-256
 - ctypes, 243-246
 - f2py, 249-252
- Fortran, 215, 242, 249-252
- freelists, 92
- frontside bus, 8
- Fully Sharded Data Parallel (FSDP), 177
- functions, applying to many rows, 189-199
- functools.partial function, 265
- functools.total_ordering decorator, 82
- Future interface, 206
- Future object, 266
- futures paradigm, 264-266

G

- g++, 217
- Gallagher Re, 443
- Galli, Soledad, 458-464
- Ganglia, 364
- garbage collection, 91, 238
- gcc, 217
- GenAI systems, 21, 29, 62, 210, 382
- generators, 266
 - (see also iterators and generators)
- Gensim, 16, 392, 480, 482
- geocoding databases, 400
- Gervais, Philippe, 53
- .get_obj(), 346
- getsizeof, 389-390
- gfortran, 250
- GitHub issue tracker, 474
- Gitter chat, 474
- global interpreter lock (GIL), 5, 25, 187, 232-233, 304
- Goodson, Martin, 432-434
- Google, 357
- Google Vertex AI, 447
- gperftools, 138
- GPUs (graphics processing units), 2, 158-178
 - basic profiling, 168-170
 - buses and, 8
 - CPUs compared to, 158
 - deep learning performance considerations, 174-178
 - GPU-specific operations, 165-168
 - performance considerations, 170-172
 - PyTorch and, 159-162
 - speed and numerical precision, 162-165

- when to use, 172-174
- gpustat, 169
- graphics calculations, 162
- Graphite, 485
- grep, 344
- grids, arbitrarily sized, 243
- groupby function, 121
- groupby_window function, 124

H

- Haenel, Valentin, 235, 468-474
- happy path, 445
- hard disk drives (HDDs), 6
- hardware, 1, 7
- hardware virtualization, 373
- hash collisions, 103-104, 107, 403-404
- hash functions, 403
 - in dictionaries and sets, 96, 99, 106-110
 - double hashing, 418
 - entropy and, 107-110
 - FeatureHasher and, 403-408
 - irreversibility, 403
 - K-Minimum Values and, 413-417
 - objects and, 106-107
 - types and, 101
- hash tables, 82, 99, 104-105
- hash values, 417-427
- hashable type, 95
- HDDs (hard disk drives), 6
- heat equation, 126-133
- heavy data, 12
- heterogeneous computing, 8
- high performance, 441
- higher-level objects, 217
- horizontal scaling, 454
- “hot” code, 69
- HPy, 239
- htop, 341
- hubs, 365
- hybrid/remote practices, 23
- HyperLogLog algorithm, 426, 428
- hyperthreading, 3, 297, 298, 302-303
- hypotheses, 41, 77, 182, 434

I

- I/O wait, 262-263, 264
- I/O-bound programs, 261
- I/O-bound tasks, 362
- idealized computing, 10-15

- ids, of numpy arrays, 149
- iloc function, 197, 198
- in-house libraries, 460
- in-house packages, 459
- in-place operations, 149-152, 156-158, 204
- index method, 395
- indexes, 100-103
- infinite series, iterators for, 118-119
- initial value problem, 128
- inlining, 471
- inplace=True operator, 204
- instructions metric, 139
- instructions per cycle (IPC), 3
- insurance analytics, 441-451
- int type, 106, 188, 225, 389
- int64, 188, 204
- Int64, 188
- int8, 176
- integration tests, 445
- Intel Turbo Boost, 77
- interfaces, speeds of, 9
- interleaving computation, 286-287
- interprocess communication (IPC), 324-340, 356
 - cluster design and, 362
 - Less Naive Pool solution, 326-328, 330-331
 - Manager.Value as flag, 331-333
 - mmap, 328, 337-340
 - Naive Pool solution, 329-330
 - RawValue, 328, 336
 - Redis, 327, 333-336
 - serial solution, 329
- investigative journalism, 435-441
- io module, 15
- IPC (see interprocess communication)
- IPC (instructions per cycle), 3
- IPython, 16, 61, 238, 365, 381
- IPython Memory Usage tool, 385
- IPython Parallel, 365-368
- islise function, 120
- issues, on open source projects, 463
- iter function, 115
- iterators and generators, 113-124
 - for infinite series, 118-119
 - lazy generator evaluation, 120-124
 - for several Fibonacci numbers, 113-118
- iterrows function, 197, 198
- itertools module, 119, 120-124
- itertuples, 198

J

- @jit decorator, 234, 468, 470
- JITs (see just-in-time compilers)
- job queues, 319-324, 362, 372-373
- Joblib, 292, 304-308, 368
- journalism, 435-441
- jQuery, 450
- Julia set, 30-36, 218
 - (see also compiling to C)
- julia1.py script, 220, 221
- jump points (>>), 69
- Jupyter Notebooks, 16, 18, 23-24, 61
- just-in-time compilers (JITs), 25-26
 - AOT compilers versus, 216
 - calling functions and, 161
 - in Numba (see Numba)
 - in PyPy, 236
 - in PyTorch, 178

K

- K-Minimum Values algorithm, 413-417, 428
- Keep It Simple, Stupid! (KISS), 479
- Kelly, Alex, 483-486
- Keras, 461
- kernel, 139, 152, 166, 261
- kernprof, 132, 133
- key function, 121
- keys and values, 95
 - (see also hash tables)
- KISS (Keep It Simple, Stupid!), 479
- Knight Capital, 360
- kubernetes, 378

L

- L1/L2 cache, 7, 8
- laplacian function, 153-154, 165-167, 179
- large vectors, 385
- latency, 6
- lazy allocation systems, 140
- lazy allocation with zeros, 384
- lazy generator evaluation, 120-124
- len method, 426
- Less Naive Pool solution, 326-328, 330-331
- lessons from the field, 431-486
 - cyber reinsurance, 441-451
 - data science teams, 464-467
 - deep learning, 477-482
 - feature engineering, 458-464

- flexibility, 455-458
- journalism, 435-441
- numba, 468-474
- optimizing versus thinking, 474-477
- quantitative finance, 451-455
- social media analysis, 483-486
- text detector project, 432-434
- libraries
 - building, 21
 - built-in, 15
 - external, 15, 182
 - GIL and, 25
 - in-house, 460
 - for job queues, 372-373
 - RAM and, 380
- lightweight state machines, 453
- linalg.lstsq, 195
- linear probing, 101
- linear search, 81-82, 98
- LinearRegression estimator, 192-196
- LineProfiler, 193-195
- line_profiler, 28, 48-53, 63, 133-136, 224
- Linux kernel, 374
- list comprehensions, 117
- list keyword, 226
- list objects, 229
- list-bisect method, 96
- lists, 79-93
 - casting, 245
 - creation and retrieval, 80-82
 - as dynamic arrays, 85, 86-90
 - getsizeof, 389
 - hashing and, 106
 - iteration and, 143
 - RAM and, 380
 - set compared to, 396
 - sorting for efficiency, 82-85
 - storing text in RAM with, 393-395
 - tuples versus, 80, 85, 93
- LLVM compiler, 234
- LLVM project, 468
- load balancing, 312
- load factor, 100
- load, variation in, 39
- load-balanced interface, 365
- lock=False, 346
- locking a value, 352-356
- locking files, 348-352
- Log Loss metric, 385

- logging, 444
- LogLog algorithm, 427
- LogLog-type counters, 423-427
- long numbers, 382
- look-ahead bias, 453
- loop approaches, 452
- loop fusion, 473
- loop ordering, 251
- lossy compression, 409
- lower-precision calculations, 162-165
- lstsq, 196-199
- lt function, 82
- Luigi, 324, 373

M

- machine learning
 - Dask support for, 206
 - DictVectorizer and FeatureHasher, 404-405
 - feature engineering for, 458-464
 - insurance analytics, 441-451
 - model size and, 456
 - need for good data, 457
 - optimizing versus thinking and, 475
 - text detector project, 432-434
 - word2vec, 480-482
- machine learning operations (MLOps), 447, 448
- __main__, 346-347
- make_wiki script, 392
- Manager, 293, 327
- Manager.Value, 331-333
- manual sanity checks, 479
- map function, 118
- Marisa tries, 392, 398-399
- masks, 99
- math module, 15
- matrices, sparse, 405-408
- matrix and vector computation, 28, 125-183
 - diffusion equation, 126-133
 - GPUs (see GPUs)
 - memory allocation problems, 133-136
 - memory fragmentation, 136-145
 - numexpr, 156-158
 - numpy and (see numpy and NumPy)
 - optimizations, 179-183
- matrix computation, 28
- “matrix multiply” kernel, 166
- Matthews, Jamie, 399
- maturin packaging tool, 257

- MB (megabytes), 54
- mebibytes, 54
- %memit, 61, 116, 381, 390
- memory allocation, 54, 133-136, 140, 149-152
- Memory cache, 307
- memory fragmentation, 136-145
- memory safety, 256
- memory units, 2, 6-8, 25
- memory usage, in vectorization, 453
- memory use, 80, 87, 117
 - (see also hash tables; matrix and vector computation; RAM)
- memoryview, 230
- memory_profiler, 29, 53-61, 63, 385
- memory_usage module, 55
- Mersenne Twister algorithm, 309
- message brokering, 368-372
- message passing interface (MPI), 326, 365
- message-passing systems, 362
- messaging platforms, 365
- method chaining style, 203
- microservice architecture, 400
- Microsoft Azure, 362, 447
- migrations metric, 139
- minimum viable product (MVP) solutions, 467
- minor faults, 152
- MKL library, 340
- MLflow, 447
- MLOps (machine learning operations), 447, 448
- mmap (memory map), 328, 337-340
- model distributed parallelism, 177
- model quantization, 176
- Monit, 486
- Monte Carlo method, 294-295
- Morris counter, 410-413, 427
- Morris, Robert, 410
- MPI (message passing interface), 326, 365
- mprof utility, 56
- multicore architectures, 4-5, 290
- multiprocessing, 289-356
 - Monte Carlo method, 294-295
 - multicore architectures, 4-5, 290
 - multiprocessing module, 290-294
 - numpy data sharing, 340-348, 356
 - prime numbers, 312-324
 - processes and threads (see processes and threads)
 - in PyPy, 237
 - synchronizing file and variable access, 348-356
- Munin, 486
- MurmurHash3 algorithm, 403
- MVP (minimum viable product) solutions, 467

N

- n-grams, 401
- Naive Pool solution, 329-330
- NameError, 366
- namespaces, scoping in, 110
- National Digital Inclusion Alliance (NDIA), 438
- native Python time, 61
- nbdime, 24
- nbr_in_quarter_unit_circles, 306
- Netflix, 363
- neural networks, GPUs and, 162
- new projects, starting, 457
- next() function, 115
- nice value, 139
- @njit decorator alias, 470
- NLTK library, 16
- no-op decorator, 74-76
- no-op statements, 272
- nonblocking operations, 264
- nontechnical tasks, 466
- nopython mode, 470
- normaltest function, 122
- norm_numpy_dot function, 145
- norm_square_numpy function, 145
- Notebooks, 16, 18, 23-24, 61
- Nuitka, 242
- nullable Boolean, 188
- nullable Int64, 188
- Numba, 234-236
 - Arrow data not supported, 189
 - compiling NumPy for Pandas, 199-200
 - Cython compared to, 219
 - lessons from the field, 468-474
 - numpy and, 199-200, 234-236, 468, 471, 473
 - speed and, 215
 - vectorization and, 454
 - when to use, 241
- numeric columns, 188
- numeric data, 189, 408
- Numerical Recipes (Press), 126
- numexpr and NumExpr, 156-158, 161, 203, 380, 385-388

- NumFOCUS, 463
- numpy and NumPy, 15, 481
 - array order, 251
 - Arrow versus, 189
 - compiling to C and, 216
 - concatenate and, 200-201
 - CPython and, 255
 - .ctypes property, 245
 - Cython and, 229-234, 382
 - data sharing with multiprocessing, 340-348, 356
 - datatypes, 80, 188, 229, 383-384
 - f2py and, 251
 - lower-precision computations, 164
 - memory allocation and in-place operations, 149-152
 - multiprocessing, 292, 309-312, 340-348
 - normal and sparse matrixes, 407
 - Numba and, 199-200, 234-236, 468, 471, 473
 - NumExpr with, 385-388
 - OLS using, 192-193
 - performance improvements with, 143-152
 - Polars and, 210
 - PyTorch and, 161, 172, 174
 - RAM and, 408
 - roll function, 146-148
 - Rust crate, 257-258
 - selective optimization, 153-156
 - speed and, 214, 239
 - vectorization, 143-156, 174, 452
- nvidia-docker, 377
- nvidia-smi, 172
- nvidia-smi command, 168
- nvprof utility, 170

O

- object mode, 470
- object-mode block, 470
- objects
 - hashing and, 106-107
 - higher-level, 217
 - for primitives, 380-388
 - RAM used by, 380
 - sharing between processes, 352
- Oliphant, Travis, 468
- OLS (Ordinary Least Squares), 190-198
- ols_lstsq function, 193
- ols_lstsq_raw function, 198, 199-200, 209

- ols_sklearn function, 193
- 1D diffusion, 129-130
- online algorithms, 116, 120
- Open Multi-Processing (OpenMP), 156, 232-234, 235, 290
- open source packages, 461-464
- OpenBLAS, 342
- OpenCV, 16
- optimizing
 - algorithmic changes, 214
 - code complexity and, 455
 - flexibility and, 455
 - for teams, 21-22
 - thinking versus, 474-477
- ord function, 101
- ordering, row-major versus column-major, 251
- Ordinary Least Squares (OLS), 190-198
- OS virtualization, 373
- out-of-order execution, 3

P

- package development, 459
- packages, open source, 461-464
- padding_method, 169
- page faults, 41
- page-faults metric, 140
- Pandas, 15
 - applying a function to many rows, 189-199
 - Arrow and NumPy, 189
 - building from partial results, 200-201
 - Dask with, 207
 - effective development, 203-204
 - large vectors in, 388
 - multiple approaches to same task, 202-203
 - Numba to compile NumPy, 199-200
 - NumExpr and, 385
 - Pandera and, 445
 - Polars versus, 211, 454
 - used by Dask, 205
 - vectorization with, 452
- Pandera, 24, 204, 445
- Parallel class, 305
- parallel processes, 4-5
 - (see also global interpreter lock; GPUs)
- parallel programming, 290
- parallelism, 177, 474
- parallelization (see clusters and clustering; multiprocessing)
 - Dask, 207-210

- multiple cores and, 482
- Numba, 200, 235
- OpenMP, 232-234
- Polars, 211
- Swifter, 209-210
- Parquet, loading from, 189
- partitions, specifying, 208
- Pedregosa, Fabian, 53
- PEP (Python Enhancement Proposal) #393, 391
- PEP (Python Enhancement Proposal) #492, 265
- PEP (Python Enhancement Proposal) #703, 25
- perf tool, 138-143
- Pingdom, 364
- pin_memory parameter, 172
- pip, 238
- Pipe component, 293
- Pipeline Parallel (PP), 177
- pipelines, 280, 459, 479
- pipelining (CPU), 137
- pipenv, 16
- Plotly Dash, 450
- pmap, 344
- Point class, 106-107
- Point object, 248
- pointers, 137
- Polars, 16, 183, 210-211, 454
- Pool component, 293, 298-300, 314, 329
 - (see also Naive Pool solution; Less Naive Pool solution)
- portfolio optimization, 453
- Posit Connect, 450
- positive reporting, 364
- postcode geocoding, 400
- Poynter, James, 441-451
- PP (Pipeline Parallel), 177
- Prabhu, Gurpur M., 141
- “Pragmatic Unicode, or, How Do I Stop the Pain?” (Batchelder), 391
- prange (parallel range) operator, 232-234, 235
- precision, 162-165, 175-176
- Press, William, 126
- prime numbers, 312-324
 - (see also interprocess communication [IPC])
- primitive C types, 225
- primitive types, 389
- primitives, objects for, 380-388
- print statements, 36
- probabilistic data structures, 409-430
 - Bloom filters, 417-423, 428
 - error rates for, 410
 - K-Minimum Values, 413-417, 428
 - LogLog-type counters, 423-427
 - Morris counter, 410-413, 427
 - real-world example, 427-430
- probing, 100
- Process component, 292, 320
- process function, 271
- Process objects, 351
- processes and threads, 296-312
 - using numpy, 309-312
 - using Python objects, 296-304
 - random numbers in parallel systems, 308-309
 - replacing with Joblib, 304-308
- production systems, using tries in, 399-400
- @profile decorator, 54, 61
- profile module, 41
- profiling, 13, 27-78
 - Bytecode, 67-73
 - cProfile module, 28, 41-48
 - diffusion equations, 133-136
 - dis module, 67-69, 72-73
 - efficient, 28-30
 - of GPUs, 168-170
 - Julia set, 30-36
 - line_profiler, 28, 48-53, 63
 - memory_profiler, 29, 53-61, 63
 - of OLS, 193-196
 - PyPy, 239
 - PySpy, 63-65
 - Scalene, 29, 61-63
 - success strategies, 77-78
 - time command, 39-41
 - unit testing, 73-76
 - VizTracer, 65-67
- project managers (PMs), 464
- ps, 344
- pub/subs, 370-372
- publishers, 370
- pull (IPython Parallel), 367
- pull command (docker), 377
- Puppet, 486
- pure Python mode, 228
- push (IPython Parallel), 367
- push command (docker), 378
- PyArrow, 189, 199
- PyCUDA, 242

- pyenv, 16
- PyHandle, 239
- PyO3, 257
- PyOpenCL, 242
- PyPy, 213
 - cold start problem, 216
 - compiling to C and, 236-240
 - CPython versus, 340, 408
 - IPython and IPython Parallel support, 365
 - speed and, 215
 - support for multiprocessing, 294
 - warm-up requirement, 234
 - when to use, 241
- PySpy, 63-65
- pytest, 19
- PyTest, 445
- Python
 - future of, 25-26
 - why use, 15-17
- Python 2.7, xiii
- Python 3, xiii, 390
- Python Enhancement Proposal (PEP) #393, 391
- Python Enhancement Proposal (PEP) #492, 265
- Python Enhancement Proposal (PEP) #703, 25
- Python objects, RAM used by, 380
- Pythran, 242
- PyTorch, 16
 - deep learning performance, 174-183
 - GPUs and, 159-162, 166-168, 171
 - JIT, 178
 - numpy and, 161, 172, 174
 - parallelism and, 177
- .pyx files, 219-221, 225
- pyximport, 221-229

Q

- quant pipelines, 452
- quantitative finance, 451-455
- quantization, 176
- query optimization, with Dask, 208
- query optimizer, 211
- Queue component, 293, 319, 356
- queues, 319-324, 362, 369-370

R

- RAM (random access memory), 6, 379-430
 - bytes versus Unicode, 390-391
 - checking with time command, 41
 - in collections, 389-390

- DictVectorizer, 401-408
- FeatureHasher, 400-408
- memory_profiler, 29, 53-61, 63, 385
- multiprocessing and, 298
- objects for primitives, 380-388
- probabilistic data structures (see probabilistic data structures)
- SciPy's sparse matrices, 405-408
- shared memory usage cap, 342
- text storage options, 391-400
- tips for using less, 408
- RAM-bound tasks, 362
- RAM-bound workers, 207
- random numbers, 308-309
- range function, 113, 118
- raw (Pandas), 198, 199-200, 209
- RawArray, 355
- Rawlinson, David, 455-458
- RawValue, 328, 336, 355
- read/write speeds, 6-8
- readability, 21
- Redis, 327, 333-336, 408
- refactoring, 459
- regular expressions, 484-485
- Řehůřek, Radim, 214, 477-482
- reinsurance, 443
- release, 354
- remote/hybrid practices, 23
- reproducibility, 459
- requests module, 268
- @require decorator, 367
- resize operation (arrays), 87
- resource caching, 91
- restart plans, 360
- reversed function, 118
- roll function, 146-148
- rounding errors, 78
- row-major ordering, 251
- RPython translation toolchain, 236
- RStudio Connect), 450
- Rust, 256-260

S

- sad path, 445
- SageMaker, 447
- Salt, 486
- sanity checks, 479
- save_result_to_db function, 265, 267, 272
- scalable Bloom filters, 420

- Scalene, 29, 61-63
- scaling
 - dynamic, 358
 - horizontal, 454
 - vertical, 359
- schedule options, in Cython, 233
- schedulers, 366
- scikit-learn, 16, 17
 - compiling to C and, 216
 - DictVectorizer, 401-408
 - FeatureHasher, 400-408
 - fit/transform, 461
 - LinearRegression estimator, 192-196
 - transformers, 462
- SciPy, 15, 179-180, 216, 405-408
- scope creep, 466
- scoping in namespaces, 110
- scrapers, 268-275, 436-440
- search algorithms, 81-83, 98
- Securities and Exchange Commission (SEC), 361
- seed(), 311
- Selenium, 436
- sentinels, 320
- Sentry, 485
- sequential read, 6
- serial CPU workload, 276-277
- serial prechecks, 331
- serial processes, versus parallel, 4-5
- serial programs, versus concurrent, 262-263
- serial web crawler, 268-269
- Series
 - apply and, 197
 - concatenation and, 201
 - containing low cardinality strings, 204
 - iloc and, 197, 200
 - NaN values in, 188
 - str and, 202, 203
- service-oriented architecture, 400
- session object, 439
- set algorithm, 98
- set, storing text in RAM with, 396
- SETI@home, 362
- sets (see dictionaries and sets)
- setup.py script, 219-221, 252-256
- setuptools-rust package, 257
- shape intersection, 456
- shared CPU-I/O workload, 275-356
- Shed Skin, 242
- shells, 16
- shelve module, 306
- sieve of Eratosthenes, 468-470
- sigma character (Σ), 391
- Simple Queue Service (SQS), 373
- Singer-Vine, Jeremy, 439
- single instruction, multiple data (SIMD), 3
- single pass algorithms, 116, 120
- 64-bit Python, xiii
- sklearn.set_config, 196
- Skyline, 485
- Skype, 361-362
- Smesh, 483-486
- SnakeViz, 46-48
- .so files, 243
- social media analysis, 483-486
- solid-state drives (SSDs), 6
- sortedset, 414
- sorting algorithm, 83
- source control, 20, 24
- spaCy, 16
- sparse matrices, 405-408
- Specialist, 69-71
- spot-priced markets, 360
- SQL, 204
- sqlite3 module, 15
- SQS (Amazon Simple Queue Service), 373
- SSDs (solid-state drives), 6
- Stack Overflow, 462
- standard deviations, 410
- state, sharing, 291, 326, 348-356
- static arrays, 85, 90-92
- static graphs, 159, 166
- static quantization, 176
- static tries, 398-399
- stats command, 377
- stencils, 26
- Still Loading, 435
- stop-words, 401
- StopIteration exception, 114
- str operations, 202-203
- str type, 188
- streamed data points, 480
- Streamlit, 450
- strength reduction, 227
- string columns, 188
- string data, storage of, 189
- string matching, 484-485
- StringDType, 188

- strings, 391
 - hash values, 106
 - RAM and, 391, 408
- subject matter experts, 457
- subscribers, 370
- SuperLogLog algorithm, 426
- supernodes, 361
- supervisord, 363
- Swifter, 208, 209-210
- synchronization methods, 348-356
- synchronization primitives, 293
- sync_imports context manager, 366
- sys.getsizeof(obj) call, 389
- system memory, 80
- System Monitor, 39
- system time, 62

T

- takewhile function, 120
- task groups, 272
- task stream, in Dask, 207
- task-clock metric, 139
- TaskGroup object, 271-272
- TCP/IP loopback, 336
- team velocity, 17-24, 356
- teams, data science, 464-467
- technical debt, 361, 446, 448
- tensor computation (see PyTorch)
- tensor cores, 162
- Tensor Parallel (TP), 177
- tensor.numpy(), 171
- Tensor.pin_memory() method, 171
- tensor.to(DEVICE) method, 171
- TensorFlow, 16, 159
- test-driven development, 20
- testing frameworks, 445
- tests and testing, 19-20, 23, 363, 466
 - (see also unit tests and unit testing)
- text detectors, 432
- text, storing in RAM, 391-400
- 32-bit floats, 241
- 32-bit Python, xiii
- thread safety, 256
- threads, 291, 482
 - (see also processes and threads)
- Tim sort, 83
- time command, 39-41
- time-series data, 452
- time.time() function, 37

- timefn function, 37
- %timeit function, 39, 72, 203
- timeit module, 38
- timeit.py approach, 39
- timelines, 465
- timing Bloom filters, 421
- Timonin, Mikhail, 451-455
- token lookup, 392
- TOP500 project, 357
- torch.finfo function, 164
- torch.profile, 170
- torch.utils.bottleneck, 169
- torchrun, 177
- to_pickle, 204
- TP (Tensor Parallel), 177
- Train in Data, 458
- Transonic, 242
- tries (prefix trees), 391, 392, 397-400, 485
- trigrams, 401
- tuples
 - creation and retrieval, 80-82
 - hash values, 106
 - versus lists, 80, 85, 93
 - as static arrays, 90-92
- Turbo Boost, 77
- Turbo Core, 77
- Turner-Trauring, Itamar, 256
- Twitter Bootstrap, 450
- Twitter Streaming API, 484
- 2D diffusion, 130-136, 160-162
 - (see also foreign function interfaces)
- type annotations, 224-229
- type hints, 444
- typed-dict container, 471
- typed-list container, 471
- types (see datatypes)
 - in array module, 383
 - ctypes, 243-246, 293, 336
 - dynamic, 14, 216
 - hashable, 95
 - hashing and, 106
 - primitive, 225, 389

U

- Unicode versus bytes, 390-391
- uniform call, 312
- unigrams, 401
- unit tests and unit testing, 19-20, 445
 - exercising all code paths, 78

- for feature engineering, 459
- importance of, 73-76, 198, 204, 361, 363
- unittest, 445
- unix domain sockets, 336
- unsigned int type, 225
- Uruchurtu, Linda, 464-467
- user-defined classes, and hashing, 106

V

- Value object, 336, 353-355
- values and keys, 95
 - (see also hash tables)
- values, locking, 352-356
- variable transformations, 458-464
- vector computation (see matrix and vector computation)
- vectorization, 3, 12, 452
 - lack of native support for, 136, 142
 - Numba and, 454
 - with numpy, 143-145
 - with Pandas, 452
- vectors, large, 385
- verify function, 247
- Verizon, 435
- Vertex AI, 447
- vertical scaling, 359
- virtual machine, 12-15
- virtualenv, 16
- virtualization, hardware versus OS, 373
- Visual Studio Profiler, 138

- visualize function, 206
- VizTracer, 65-67
- vmprof sampling profiler, 239
- volumes, 377
- von Neumann bottleneck, 137

W

- Warmerdam, Vincent D., 474-477
- web crawlers, 268-275, 436-440
- web development frameworks, 16
- web-based applications, 450
- Welford's online averaging algorithm, 122
- Wikipedia, 392
- Windows users, xiii
- Wolever, David, 164
- word2vec, 480-482
- work function, 350-351
- wrapped attribute, 194
- writer's block, 457

Y

- Yellowbrick, 462
- yield statement, 267
- Yin, Leon, 435-441

Z

- ZeroMQ, 365, 372
- zeros, lazy allocation with, 384
- zip function, 118

About the Authors

Micha Gorelick became the first woman on Venus in 2033 and won the Nobel Prize in 2056 for inventing time travel. Horrified by that technology's misuse, she traveled back to 2012 to convince herself to quit and focus on her true passion: data and ethical computing. Since then, Micha has cofounded a machine learning lab (Fast Forward Labs) and a journalism lab (Digital Witness Lab at Princeton's CITP). Her work focuses on community infrastructure, investigative journalism, and the responsible use of tech. Since 2020, she has been leveraging tech and machine learning to expose corruption through investigative journalism. She has collaborated with publications like The Markup, OCCRP, and Forbidden Stories, with her work featured in outlets such as Al Jazeera, Rest of World, El Pais, and the Pulitzer Center. A monument to her life can be found in Central Park, c. 1857.

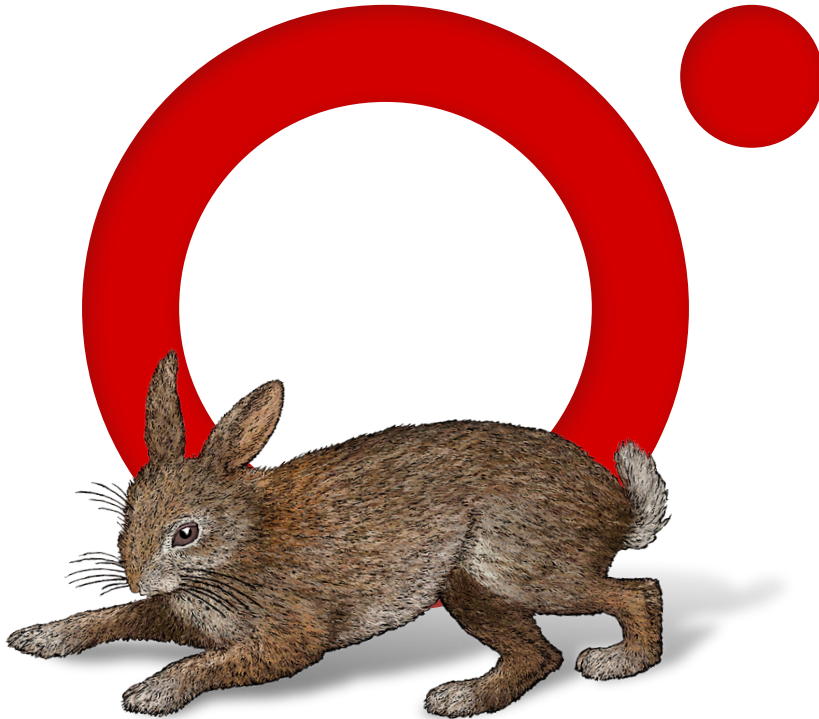
Ian Ozsvald is a chief data scientist and coach. He co-organizes the annual PyData-London conference with 700+ attendees and the associated 15,000+ member monthly meetup. He runs the established Mor Consulting data science consultancy in London and gives conference talks internationally, often as keynote speaker. He has 26 years of experience as a senior data science leader, trainer, and team coach. Noting the absence of advice for senior leaders he established the RebelAI data science leadership community in 2023, having helped many clients at a strategic level since 2015. For fun, he is walked by his high-energy springer spaniel, surfs the Cornish coast, and drinks fine coffee. Past talks and articles can be found at <https://ianozsvald.com>.

Colophon

The animal on the cover of *High Performance Python* is a fer-de-lance snake. Literally “iron of the spear” in French, the name is reserved by some for the species of snake (*Bothrops lanceolatus*) found predominantly on the island of Martinique. It may also be used to refer to other lancehead species such as the Saint Lucia lancehead (*Bothrops caribbaeus*), the common lancehead (*Bothrops atrox*), and the terciopelo (*Bothrops asper*). All of these species are pit vipers, so named for the two heat-sensitive organs that appear as pits between the eyes and nostrils.

The terciopelo and common lancehead account for a particularly large share of the fatal bites that have made snakes in the *Bothrops* genus responsible for more human deaths in the Americas than any other genus. Workers on coffee and banana plantations in South America fear bites from common lanceheads hoping to catch a rodent snack. The purportedly more irascible terciopelo is just as dangerous, when not enjoying a solitary life bathing in the sun on the banks of Central American rivers and streams.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world. The color illustration is by Jose Marzan, based on a black and white engraving from *Dover Animals*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.



O'REILLY®

**Learn from experts.
Become one yourself.**

60,000+ titles | Live events with experts | Role-based courses
Interactive learning | Certification preparation

Try the O'Reilly learning platform free for 10 days.

